



STANDARD ATLAS LANGUAGE FOR MODULAR TEST

ARINC SPECIFICATION 626-3

PUBLISHED: JANUARY 30, 1995

AN **ARINC** DOCUMENT

Prepared by
AIRLINES ELECTRONIC ENGINEERING COMMITTEE
Published by
AERONAUTICAL RADIO, INC.
2551 RIVA ROAD, ANNAPOLIS, MARYLAND 21401

This document is based on material submitted by various participants during the drafting process. Neither AEEC nor ARINC has made any determination whether these materials could be subject to claims of patent or other proprietary rights by third parties, and no representation or warranty, express or implied, is made in this regard. Any use of or reliance on this document shall constitute an acceptance hereof “as is” and be subject to this disclaimer.

© 1995 by
AERONAUTICAL RADIO, INC.
2551 Riva Road
Annapolis, Maryland 21401-7465

ARINC SPECIFICATION 626-3
STANDARD ATLAS LANGUAGE FOR MODULAR TEST

Published: January 30, 1995

Prepared by the Airlines Electronic Engineering Committee

Characteristic 626	Adopted by the Airlines Electronic Engineering Committee:	May 30, 1987
Characteristic 626	Industry Review Completed	November 30, 1987

Summary of Document Supplements

<u>Supplement</u>	<u>Adoption Date</u>	<u>Published</u>
Characteristic 626-1	October 19, 1988	April 4, 1989
Characteristic 626-2	October 10, 1991	March 23, 1992
Characteristic 626-3	October 21, 1994	January 30, 1995

A description of the changes introduced by each supplement is included on goldenrod paper at the end of this document.

NOTICE

In recognition of the fact that substantial portions of this document are based upon IEEE Standard ATLAS© 1989 by the Institute of Electrical and Electronics Engineers, Inc., (IEEE) and are reproduced with the expressed permission of IEEE, ARINC has authorized IEEE to incorporate any part of this document in future editions of IEEE Standard ATLAS. All other rights are reserved.

No part of this publication may be reproduced in any form, in an electronic retrieval system or otherwise, without the prior written permission of the publisher.

INTRODUCTION

The ATLAS Test Language was originally developed under the sponsorship of Aeronautical Radio, Inc. (ARINC) in response to the need for an application and documentation language for manual and automatic testing in airline avionics. Initial development activities began in 1967 by an Automatic Test Equipment Subcommittee of the Airlines Electronic Engineering Committee (AEEC). The original version of **ARINC Specification 416**, *Abbreviated Test Language for Avionics Systems (ATLAS)*, was adopted by AEEC in October 1968 and published by ARINC in June 1969.

ARINC Specification 416 underwent a number of revision cycles over the next few years as a result of continuing efforts to expand the avionics testing capabilities of ATLAS. By 1976, ARINC Specification 416 reached revision level 13. Also during the 1969-1976 period, interest in ATLAS was expressed for potential applications in industrial and military testing.

AEEC recommended that in view of the rapidly expanding applications other than airline avionics testing, the future development of ATLAS should be conducted by a standards organization representing a broader cross-section of the electronics industry. As a result, AEEC authorized the transfer of ATLAS to IEEE jurisdiction and management in September 1976. IEEE Std.416-1976, *IEEE/ARINC Standard ATLAS Test Language*, was published in November 1976. Further revision and subsequent adoption by the American National Standards Institute (ANSI) led to the publication of ANSI IEEE Std.416-1978 (Revision Level 14) in September 1978.

The ensuing development and expansion of ATLAS for industrial and military applications reflected in IEEE Std.416-1978 created the need for defining a subset of ATLAS containing only those language features needed for commercial avionics testing. An outline for an 'avionics subset' was prepared by the Boeing Commercial Airplane Company and presented to the IEEE ATLAS Committee in April 1979. In response to encouragement from the Committee, Boeing prepared and submitted a fully detailed proposal at the next meeting in September 1979. However, the IEEE ATLAS Committee concluded that a better approach would be to define a procedure for creating subsets rather than "approve" any one subset.

The airline had an urgent need for such a subset to support the new generation of "digital" aircraft (Boeing 757/767 and Airbus A-310) and asked the AEEC to complete coordination and development through the AEEC Test Equipment Guidance (TEG) Subcommittee. The resulting document was adopted by AEEC in June 1980 as **ARINC Specification 616**, *Avionics Subset of ATLAS Language*.

The further development of the avionics system for new aircraft demanded an update to the previously agreed subset of the ATLAS Language as defined in ARINC Specification 616. Accordingly, a new subset of the ATLAS Language was prepared, aligned to IEEE Std.416 (Revision Level 34). It was adopted by AEEC in March 1987 and was published as **ARINC Specification 626**, *Standard ATLAS Language for Modular Test*.

This document was developed by the inclusion of new and revised capabilities that were developed by the AEEC Test Equipment Guidance (TEG) Subcommittee.

Further development by AEEC has produced this revision. In the meantime, the IEEE ATLAS Committee has decided to terminate the development of IEEE Std.416 and to withdraw it as a current standard. Consequently, ARINC Specification 626-2 is a language definition in its own right.

ARINC Specification 626 was section numbered to be consistent with the original numbering of IEEE Std.416. The same section numbering was retained in ARINC Specifications 626-1 and 626-2. Since IEEE Std.416 is no longer available from the IEEE, and this document is a significant update to the original ARINC Specification 616, it has been decided that to retain the original section numbering is of little benefit to users.

Therefore, in ARINC Specification 626-3 the unused sections which maintained the IEEE Std.416 ordering have been eliminated and the sections renumbered to be consecutive. Furthermore, to facilitate cross-referring between the text and the Formal Syntax Definition (FSD), the latter now precisely tracks the textual presentation of the language definitions.

FOREWORD

Activities of AERONAUTICAL RADIO, INC. (ARINC)

and the

Purpose of ARINC Characteristics

Aeronautical Radio, Inc. is a corporation in which the United States scheduled airlines are the principal stockholders. Other stockholders include a variety of other air transport companies, aircraft manufacturers and non-U.S. airlines.

Activities of ARINC include the operation of an extensive system of domestic and overseas aeronautical land radio stations, the fulfillment of systems requirements to accomplish ground and airborne compatibility, the allocation and assignment of frequencies to meet those needs, the coordination incident to standard airborne compatibility, the allocation and assignment of frequencies to meet those needs, the coordination incident to standard airborne communications and electronics systems and the exchange of technical information. ARINC sponsors the Airlines Electronic Engineering Committee (AEEC), composed of airline technical personnel. The AEEC formulates standards for electronic equipment and systems for the airlines. The establishment of Equipment Characteristics is a principal function of this Committee.

An ARINC Equipment Characteristic is finalized after investigation and coordination with the airlines who have a requirement or anticipate a requirement, with other aircraft operators, with the Military services having similar requirements, and with the equipment manufacturers. It is released as an ARINC Equipment Characteristic only when the interested airline companies are in general agreement. Such a release does not commit any airline or ARINC to purchase equipment so described nor does it establish or indicate recognition of the existence of an operational requirement for such equipment, nor does it constitute endorsement of any manufacturer's product designed or built to meet the Characteristic. An ARINC Characteristic has a twofold purpose, which is:

- (1) To indicate to the prospective manufacturers of airline electronic equipment the considered opinion of the airline technical people, coordinated on an industry basis, concerning requisites of new equipment, and
- (2) To channel new equipment designs in a direction which can result in the maximum possible standardization of those physical and electrical characteristics which influence interchangeability of equipment without seriously hampering engineering initiative.

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
1.0	PURPOSE	1
1.1	ATLAS Characteristics	1
1.1.1	Unit Under Test Orientation	1
1.1.2	Unambiguous Communication	1
1.1.3	Test Equipment Independence	1
1.2	Management Procedures for the use of Extensibility Constructs	1
1.2.1	Introduction	2
1.2.2	Extensibility Management Philosophy	2
1.2.3	Extensibility Techniques	3
2.0	REFERENCE MATERIAL	5
2.1	Applicable Documents	5
2.2	Document Precedence	5
2.3	How to Use This Specification	5
2.3.1	Structure	5
2.3.2	Terminology	7
2.3.3	How to Read the Formal Syntax	7
2.3.4	How to Read Syntax Diagrams	7
2.3.5	Relationship of Formal Syntax to Syntax Diagrams	8
3.0	ATLAS TEST SPECIFICATION	10
3.1	<atlas test specification structure>	10
3.1.1	<atlas program structure>	11
3.1.2	<atlas module structure>	12
3.1.3	<non-atlas module structure>	12
3.2	Basic Statement Elements	12
3.2.1	Flag Field	13
3.2.2	Statement Number Field	13
3.2.3	Verb Field	13
3.2.4	Field Separator	13
3.2.5	Remainder of Statement	13
3.2.6	Statement Terminator (\$)	13
3.3	Manual Actions During Testing	13
4.0	STRUCTURE DELIMITER STATEMENTS	15
4.1	BEGIN/TERMINATE Statements	15
4.1.1	<begin atlas program statement>	15
4.1.2	<terminate atlas program statement>	15
4.1.3	<begin atlas module statement>	16
4.1.4	<terminate atlas module statement>	16
4.2	Block Definition	17
4.2.1	<block structure>	17
4.2.2	<begin block statement>	18
4.2.3	<block body>	18
4.2.4	<leave block statement>	19
4.2.5	<end block statement>	19
4.3	<commence main procedure statement>	20
5.0	(Not Used)	21
6.0	PREAMBLE STATEMENTS	22
6.1	Main Preamble Structure	23
6.1.1	<program preamble structure>	23
6.1.2	<module preamble structure>	25
6.2	<local preamble structure>	26
6.3	<declare statement>	27
6.4	<define statement structure>	37
6.5	<define drawing statement>	38
6.6	<define signal statement>	46
6.7	PROCEDURE Definition	49
6.7.1	<define procedure structure>	49
6.7.2	<define procedure statement>	50
6.7.3	<procedure body>	52
6.7.4	<leave procedure statement>	53

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
6.7.5	<end procedure statement>	53
6.8	FUNCTION Definition	54
6.8.1	<define function structure>	54
6.8.2	<define function statement>	55
6.8.3	<function body>	56
6.8.4	<leave function statement>	57
6.8.5	<end function statement>	57
6.9	<require statement>	58
6.10	<include statement>	63
6.11	<establish protocol statement>	64
6.12	<identify statements>	76
6.13	<identify timer statement>	77
6.14	<identify signal based event statement>	78
6.15	<identify event based event statement>	81
6.16	<identify event interval statement>	84
6.17	<identify event indicator statement>	86
6.18	<identify time based event statement>	89
6.19	DIGITAL CONFIGURATION Definition	90
6.19.1	<define digital configuration structure>	90
6.19.2	<define digital configuration statement>	92
6.19.3	<define digital source statement>	92
6.19.4	<define digital sensor statement>	102
6.19.5	<specify delay min statement>	106
6.19.6	<end digital configuration statement>	106
6.20	<define exchange statement>	107
6.21	<extend atlas statement>	112
6.22	<define exchange configuration statement>	121
7.0	PROCEDURAL STATEMENTS	123
7.1	<main procedural structure>	123
7.2	<main procedural statements>	123
8.0	DATA PROCESSING STATEMENTS	126
8.1	<procedural statements, data processing>	126
8.2	<calculate statement>	127
8.3	<compare statement>	139
9.0	INPUT/OUTPUT STATEMENTS	140
9.1	<procedural statements input output>	140
9.2	<input statement>	140
9.3	<output statement>	144
9.4	<enable file access statement>	147
9.5	<disable file access statement>	148
9.6	<create file statement>	149
9.7	<delete file statement>	149
10.0	CONTROL STATEMENTS	150
10.1	<procedural statements, control>	150
10.2	IF THEN ELSE/ELSE IF Capability	151
10.2.1	<if then else/else if structure>	151
10.2.2	<if then statement>	154
10.2.3	<procedural segment if then else else if>	154
10.2.4	<leave if statement>	156
10.2.5	<else statement>	157
10.2.6	<end if statement>	157
10.2.7	<else if then statement>	158
10.3	WHILE THEN Capability	159
10.3.1	<while then structure>	159
10.3.2	<while then statement>	160
10.3.3	<procedural segment while then>	161
10.3.4	<leave while statement>	162
10.3.5	<resume while statement>	163
10.3.6	<end while statement>	164
10.4	FOR THEN Capability	165
10.4.1	<for then structure>	165

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
10.4.2	<for then statement>	166
10.4.3	<procedural segment for then>	167
10.4.4	<leave for statement>	168
10.4.5	<resume for statement>	169
10.4.6	<end for statement>	170
10.5	<go to statement>	171
10.6	<perform statement>	172
10.7	<finish statement>	173
10.8	ENABLE/DISABLE MANUAL INTERVENTION Capability	174
10.8.1	<enable disable manual intervention structure>	174
10.8.2	<enable manual intervention statement>	175
10.8.3	<disable manual intervention statement>	175
10.9	<wait for manual intervention statement>	176
10.10	<enable digital configuration statement>	177
10.11	ESCAPE Mechanism	179
10.11.1	<enable escape statement>	179
10.11.2	<disable escape statement>	180
11.0	SIGNAL ORIENTED STATEMENTS	181
11.1	<procedural statements signal>	181
11.1.3	Syntax Diagram	189
11.1.4	Rules	189
11.2	Single Action Verb Statements	191
11.2.1	Single Action Verb Statements: General Description	191
11.2.2	<setup statement>	196
11.2.3	<connect statement>	198
11.2.4	<disconnect statement>	199
11.2.5	<arm statement>	200
11.2.6	<fetch statement>	201
11.2.7	<change statement>	202
11.2.8	<enable event statement>	203
11.2.9	<disable event statement>	204
11.2.10	<reset statement>	205
11.3	<apply statement>	206
11.4	<remove statement>	207
11.5	<measure statement>	209
11.6	<monitor statement>	210
11.7	<verify statement>	211
11.8	<read statement>	212
11.9	<initiate statement>	213
11.10	Digital Verbs	214
11.10.1	<stimulate statement>	214
11.10.2	<sense statement>	216
11.10.3	<prove statement>	218
12.0	TIMING STATEMENTS	220
12.1	<procedural statements timing>	220
12.2	<wait for statement>	222
12.3	TIME INTERVAL Measurement Capability	223
12.3.1	<event> based measurements	223
12.4	DO/END DO capability	224
12.4.1	<do simultaneous structure>	224
12.4.2	<do simultaneous statement>	225
12.4.3	<do simultaneous body>	226
12.4.4	<end do statement>	226
12.4.5	<do timed digital structure>	227
12.4.6	<do timed digital statement>	227
12.4.7	<do timed digital body>	229
12.4.8	<do sequential structure>	232
12.4.9	<do sequential statement>	232
12.4.10	<do sequential body>	233
12.5	<reset timer statement>	233
12.6	<read timer statement>	234

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
13.0	DATABUS STATEMENTS	235
13.1	<do exchange statement>	237
13.2	<update exchange statement>	249
13.3	<enable exchange configuration statement>.	252
13.4	<connect exchange configuration statement>	254
13.5	<disconnect exchange configuration statement>	255
13.6	<disable exchange configuration statement>	256
13.7	<fetch exchange configuration statement>	257
14.0	FIELD AND SUBFIELD DEFINITION	260
14.1	<statement characteristics>	260
14.2	<real characteristic subfield>	263
14.3	<sync subfield>	264
14.4	<real errlim> (Real Error Limit)	247
14.5	<dim lim> (Dimension Limit)	269
14.6	<pc lim> (Percent Limit)	269
14.7	<measured characteristic>	270
14.8	<evaluation field>	272
14.9	<eval statement characteristics> (evaluation statement characteristics)	275
14.10	<max time>	275
14.11	<time quantity>	276
14.12	<condition>	276
14.13	<conn> (Connection Field)	277
14.14	<conn set> (Connection Set)	278
14.15	<signal value>	284
14.16	<real quantity>	285
14.17	<index>	285
14.18	<noun field>	286
14.19	<dimensioned number>	286
14.20	<require control>	287
14.21	<require capability>	288
14.22	<require limit>	291
14.23	<require cnx>	292
14.24	<exchange expression>	293
14.25	<exchange frame>	294
14.26	<gate field>	298
14.27	<stim rate>	299
14.28	<sense rate>	299
14.29	<sense delay>	300
14.30	<stim event> 1	300
14.31	<sense event>	301
14.32	<when field>	301
14.33	<digital source characteristics>	302
14.34	<digital sensor characteristics>	303
14.35	<inst val>	304
14.36	<time subfield>	304
14.37	<on field>	305
14.38	<formal parameter>	306
14.39	<data store>	307
14.40	<array range>	308
14.41	<bus parameter>	309
14.42	<set protocol parameter>	312
14.43	<fetch protocol parameter>	314
14.44	<role field> 1	316
14.45	<digital keyword>	317
15.0	LANGUAGE COMPONENT SPECIFICATION	318
15.1	Authorized Characters	318
15.2	Representation of Numbers and Other Data Types	320
15.2.1	<decimal number>	320
15.2.2	<unsigned decimal number>	321
15.2.3	<unsigned integer number>	321
15.2.4	<binary number>	322
15.2.5	<octal number>	323

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
15.2.6	<hexadecimal number>	324
15.2.7	<digital number>	325
15.2.8	<step>	325
15.2.9	<character string>	325
15.2.10	<connection>	325
15.3	Flag and Statement Numbers	326
15.3.1	<fstatno>	326
15.3.2	<flag statement> and <comment statement>	327
15.3.3	<statement number>	328
15.4	Blank Space Requirements	329
15.5	<label>	330
15.6	Retention of Labelled Information	331
15.6.1	Global/Local Considerations	331
15.6.2	MEASUREMENT	332
15.6.3	Retention of Measured Sensor Values	332
15.6.4	<function>, <procedure>, <protocol> and <exchange>	332
15.6.5	<parameter>	333
15.6.6	<data store>	333
15.6.7	Other Variables ()	334
15.6.8	<measured characteristic mnemonic>	334
15.6.9	<monitored characteristic>	334
15.6.10	<terminal identifier>	334
15.7	Range Subfield	334
15.7.1	Range Requirement for <data store> or <parameter>	334
15.8	Dimensions	334
15.8.1	Units of Rate of Change	337
15.8.2	Units which are a Product	337
15.8.3	Units which are not pure quotients	337
15.8.4	Square and Cubic Units	337
15.8.5	<dim>	338
15.8.5.1	<dim name>	338
15.8.6	<freq dimension>	338
15.8.7	<time dimension>	338
15.8.8	Representation of Reciprocals	338
15.9	Character Code Definitions	338
15.9.1	ASCII(American National Standard Code for Information Interchange)	339
15.10	Special Character Sets	341
15.10.1	<character string>	341
15.10.2	<message text>	341
15.10.3	<msgchar> (message character)	341
15.10.4	<drawing identifier>	341
15.10.5	<modifier descriptor>	341
15.10.5.1	<modifier descriptor name>	341
15.10.6	<module name>	341
15.10.7	<noun>	341
15.10.7.1	<noun name>	341
15.10.8	<pin descriptor>	341
15.10.8.1	<pin descriptor name>	342
15.10.9	<mnemonic>	342
15.10.9.1	<modifier name>	342
15.10.10	<mod type>	342
15.10.11	<digit>	342
15.10.12	<binary digit>	342
15.10.13	<octal digit>	342
15.10.14	<hexadecimal digit>	342
15.10.15	<test equip role name>	342
15.10.16	<bus mode name>	342
15.10.17	<bus parameter name>	342
15.10.18	<protocol parameter name>	343
15.10.19	<bus specification>	343
15.11	Miscellaneous Language Elements	343
15.11.1	<reference phase>	343
15.11.2	<constant identifier>	343
15.11.3	<type identifier>	343
15.11.4	<variable identifier>	343

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
15.11.5	<enumeration element>	343
15.11.6	<record field>	343
15.11.7	<number>	343
15.11.8	<modifier mnemonic>	343
15.11.9	<procedure>	344
15.11.10	<function>	344
15.11.11	<signal>	344
15.11.12	<event>	344
15.11.13	<event indicator>	344
15.11.14	<event interval>	344
15.11.15	<timer>	344
15.11.16	<protocol>	344
15.11.17	<exchange>	344
15.11.18	<configuration>	344
15.11.19	<digital source>	344
15.11.20	<digital sensor>	345
15.11.21	<dim name>	345
15.11.22	<external semantics specification>	345
15.11.23	<external syntax specification>	345
15.11.24	<drawing>	345
15.11.25	<device identifier name>	345
15.11.26	<requirement>	345
15.11.27	<exchange configuration>	345
15.11.28	<boolean variable>	345
15.11.29	<external digital specification>	345
15.11.30	<program name>	346
15.12	Miscellaneous Keywords	346
15.12.1	MANUAL-INTERVENTION	346
15.12.2	MAX-TIME	346
16.0	NOUNS AND THEIR MODIFIERS	347
16.1	AC SIGNAL (Alternating Current Signal)	349
16.2	AMBIENT CONDITIONS	351
16.3	AM SIGNAL (Amplitude Modulation Signal)	352
16.4	COMMON	353
16.5	DC SIGNAL (Direct Current Signal)	355
16.6	DOPPLER	356
16.7	EARTH	356
16.8	EVENTS	357
16.9	FM SIGNAL (Frequency Modulation Signal)	358
16.10	IMPEDANCE	359
16.11	LIGHT	360
16.12	LOGIC SIGNAL	361
16.13	PAM (Pulse Amplitude Modulation)	363
16.14	PM SIGNAL (Phase Modulated Signal)	364
16.15	PULSED AC (Pulsed Alternating Current Signal)	365
16.16	PULSED AC TRAIN	366
16.17	PULSED DC (Pulsed Direct Current Signal)	369
16.18	PULSED DC TRAIN	370
16.19	PULSED DOPPLER	374
16.20	RADAR SIGNAL	375
16.21	RAMP SIGNAL	376
16.22	RANDOM NOISE	377
16.23	RESOLVER	378
16.24	ROTATION	379
16.25	SHORT	383
16.26	SQUARE WAVE	384
16.27	STEP SIGNAL	385
16.28	SUP CAR SIGNAL (Suppressed Carrier Signal)	386
16.29	SYNCHRO	387
16.30	TIME INTERVAL	387
16.31	TRIANGULAR WAVE SIGNAL	388
16.32	VIBRATION	389
16.33	VOR (VHF Omnidirectional Radio Range)	390
16.34	WAVEFORM	392

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
17.0	NOUN--MODIFIER DEFINITION	395
17.1	Mnemonics for Pulse Type Signals	395
17.2	Modifier Prefixes and Suffixes	396
17.2.1	Impedance Modifier Prefixes	396
17.2.2	Amplitude Modifier Suffixes	396
17.2.3	Phase Identifier Suffixes	399
17.2.4	Ratio Modifier Suffixes	399
17.2.5	Pulse Identifier Suffixes	400
17.2.6	Quiescent Signal Suffix	400
17.2.7	Signal Transition Suffix	400
17.2.8	Reference Phase Identifier Suffix	401
17.3	NOUN MODIFIERS	402
18.0	ATLAS FORMAL SYNTAX DEFINITION	423
18.1	Introduction	423
18.1.1	Formal Definition of the Syntax	424
18.1.2	References Between the Formal Syntax Definition and the ATLAS Language Specification	424
18.2	Syntax Notation	425
18.2.1	Tokens	425
18.2.2	Selection	427
18.2.3	Grouping "{ }"	428
18.2.4	Optionality Brackets "[]"	428
18.2.5	Repetition	428
18.2.6	Execution of Operator Tokens	429
18.2.7	Comment "!"	429
18.2.8	Layout Blanks	430
18.2.9	Note Identification "<<*.,>>"	430
18.2.10	Coherence between FSD Listing and Syntax Diagrams	430
18.3	Program Structure Syntax	431
18.3.1	Range and Order of the Program Structure Syntax	431
18.3.2	Name Conventions	431
18.4	Notes	432
18.5	Lexical Symbols Syntax	433
18.5.1	Range and Order of the Lexical Symbols Syntax	433
18.5.2	Name Conventions	433
18.6	SYNTAX DEFINITION	434
18.6.1	ATLAS TEST SPECIFICATION STRUCTURE	434
18.6.2	STRUCTURE DELIMITER STATEMENTS	434
18.6.3	PREAMBLE STATEMENTS	435
18.6.4	MAIN PROCEDURAL STRUCTURE	452
18.6.5	PROCEDURAL STATEMENTS - DATE PROCESSING	453
18.6.6	PROCEDURAL STATEMENTS - INPUT-OUTPUT	455
18.6.7	PROCEDURAL STATEMENTS - CONTROL	457
18.6.8	PROCEDURAL STATEMENTS - SIGNAL	461
18.6.9	PROCEDURAL STATEMENTS - TIMING	466
18.6.10	PROCEDURAL STATEMENTS - DATABUS	469
18.6.11	FIELDS / SUBFIELDS	472
18.6.12	BASIC SYMBOLS / CHARACTER SETS	484
18.6.13	IDENTIFIERS / LITERALS / LABELS	487
18.6.14	SEPARATORS / STATEMENT-NUMBERS	490
18.6.15	DESCRIPTORS	491
18.6.16	NOUNS	492
18.6.17	NOUN-MODIFIERS	492
18.6.18	NOUN-MODIFIERS - PREFIXES / SUFFIXES	495
18.6.19	DIMENSIONS	496
18.7	CROSS-REFERENCE LISTING OF DEFINITIONS AND USAGE OF SYNTAX PRODUCTION RULES, SYMBOLS AND LABELS	499
18.7.1	CROSS-REFERENCE LISTING OF SYNTAX PRODUCTION RULE NAMES	499
18.7.2	CROSS-REFERENCE LISTING OF SYMBOLS	516
18.7.3	CROSS-REFERENCE LISTING OF LABELS	543

APPENDIX A1

COORDINATED AND RECOMMENDED USE OF EXTEND	545
---	-----

ARINC SPECIFICATION 626
TABLE OF CONTENTS

<u>ITEM</u>	<u>SUBJECT</u>	<u>PAGE</u>
 <u>FIGURES</u>		
Figure 3-1	ATLAS Test Specification Family Tree	11
Figure 3-2	ATLAS Program Structure	11
Figure 3-3	ATLAS Module Structure	12
Figure 3-4	General ATLAS Statement Structure	13
Figure 6-1	DEFINE DRAWING Test Problem 1	40
Figure 6-2	DEFINE DRAWING Test Problem 2	41
Figure 6-3	DEFINE DRAWING Test Problem 3	42
Figure 6-4	DEFINE DRAWING Test Problem 4	43
Figure 6-5	DEFINE DRAWING Test Problem 5	44
Figure 6-6	DEFINE DRAWING Test Problem 6	45
Figure 6-7	DEFINE DRAWING Test Problem 6 - Associated Drawings	45
Figure 6-8	EVENT Detection Example 1	87
Figure 6-9	EVENT Detection Example 2	87
Figure 6-10	RZ, NRZ, and BIP Pulse Types for Digital Word B'10110010'	101
Figure 10-1	Functional Flow Diagram of <if then> sub-structure	151
Figure 10-2	Functional Flow Diagram of <if then else> sub-structure	152
Figure 10-3	Functional Flow Diagram of <if then else/else if structure>	152
Figure 10-4	Functional Flow Diagram of <while then structure>	159
Figure 10-5	Functional Flow Diagram of <for then structure>	165
Figure 11-1	State Diagram for a Source or Load Virtual Test Resource without Event Monitor using Single Action and Multi Action Verbs	185
Figure 11-2	State Diagram for a Source or Load Virtual Test Resource with Event Monitor using Single Action and Multi Action Verbs	186
Figure 11-3	State Diagram for a Sensor Virtual Test Resource without Event Monitor for making a Measurement using Single and Multi Action Verbs	187
Figure 11-4	State Diagram for a Sensor Virtual Test Resource with Event Monitor for making a Measurement using Single and Multi Action Verbs	188
Figure 11-5	Single Action Analog Verbs Illustration	192
Figure 13-1	Diagram for EXCHANGE, Example 1.	242
Figure 13-2	Definition of Bus Traffic for EXCHANGE, Example 1.	244
Figure 14-1	TRUE and COMPL Components for a Serial Binary Signal.	282
Figure 14-2	Example for <gate field>.	298
Figure 16-1	Illustration for COMMON, Example 1	353
Figure 16-2	Illustration for COMMON, Example 2	354
Figure 16-3	Illustration of COMMON, Example 3	354
Figure 16-4	Illustration of PULSED DC TRAIN, Example 1	367
Figure 16-5	Illustration for PULSED AC TRAIN, Example 2	368
Figure 16-6	Illustration for PULSED DC TRAIN, Example 1.	371
Figure 16-7	Illustration for PULSED DC TRAIN, Example 2	372
Figure 16-8	Illustration for PULSED DC TRAIN, Example 3	373
Figure 16-9	Illustration for Rule 3 of ROTATION	380
Figure 16-10	Illustration of ROTATION, Example 1	381
Figure 16-11	Definition of PERIOD, SAMPLE-TIME, SAMPLE-WIDTH and SAMPLE-SPACING	393
Figure 17-1	Pulse Definition	395
Figure 17-2	Peak and Peak-to-Peak Illustrations	397
Figure 17-3	Illustration of Positive and Negative Peaks of an Asymmetrical Waveform	398
 <u>TABLES</u>		
Table 8-1	Permitted Operations	132
Table 8-2	<predefined function>	134
Table 13-1	Definition of Bus Traffic for EXCHANGE, Example 1	243
Table 14-1	Evaluation Field Relationships	274
Table 15-1	Table of Authorized Quantities and their Dimensions	335
Table 15-2	Seven Bit ASCII Code	340

1.0 PURPOSE

This language specification defines an ARINC controlled subset of the Abbreviated Test Language for All Systems (ATLAS). The term “all” was substituted for the original “avionics” in recognition of the wider application of the language.

ATLAS is a standard abbreviated English language used in the preparation and documentation of test procedures which can be implemented either manually or with automatic or semi-automatic test equipment.

This document amplifies the ATLAS Formal Syntax Definition in Chapter 18 by providing functional description and rules applicable to ATLAS constructs over and above the syntactic definition of the language. To provide for a complete, but concise, description of each language construct a reference to the appropriate section(s) of the Formal Syntax Definition of the ATLAS Language in Chapter 18 together with a diagrammatic representation of the language syntax are included, where deemed appropriate, within this document. The diagrammatic representations are referred to as “syntax diagrams” and are intended to provide a guide in constructing valid ATLAS statements.

1.1 ATLAS Characteristics

The following are the general characteristics of ATLAS.

1.1.1 Unit Under Test Orientation

The language is dedicated to defining the test requirements of the Unit Under Test (UUT) with no reference to nor dependence upon the test equipment which may be used. The latter may be automatic, manual or of hybrid design.

1.1.2 Unambiguous Communication

The selective use of English language terms which are compatible with the description of test requirements and a formal structure for their use (as defined in this document) constitute an environment to ensure an unambiguous description of the requirements of a test procedure for the UUT designers, developers, users, and maintenance personnel.

1.1.3 Test Equipment Independence

The specification of test requirements in terms of the UUT by ATLAS facilitates the transportability of those test specifications from implementation on one set of test equipment to another, providing that all of the test requirements can be satisfied by the test equipments.

1.2 Management Procedures for the Use of Extensibility

The capabilities provided by the extensibility facilities in ATLAS permit users to establish local definitions of additional capabilities beyond those defined by the ATLAS Standard. It is highly desirable that these should be used in ways which are consistent within the underlying principles of standardization. The following operating procedures will support this without inordinately inhibiting the expeditious use of ATLAS.

To avoid confusion it is recommended that any word used in an ATLAS extension should be consistent with the IEEE Standard Dictionary of Electrical and Electronic Terms (IEEE Std 100).

The whole user community will benefit if uses of extensibility are reported back to the ATLAS Committee. This will permit the Committee to determine which uses of extensibility address test situations which are of sufficiently general applicability to justify the addition of new material to ATLAS. In practice the service which the ATLAS Committee provides to the user community is totally dependent upon the feedback of problems and solutions employed. All users of this specification are strongly urged to ensure that their staff and contractors provide this feedback and make maximum use of the information included in Appendix A of this specification. Compliance with this requirement should be a significant factor in assessing the degree of compliance with quality assurance requirements.

The ARINC 626 standard Subcommittee will fulfil the IEEE requirement for the IEEE Standards Co-ordination Committee 20 (SCC 20) to be kept informed of usage of EXTEND by supplying details of the contents of Appendix A and of periodic updates to it to the IEEE.

1.0 PURPOSE

It is intended that the extensibility facilities should have the capability to add any required extension. If it is found that this objective has not been achieved and an extension can only be defined by adapting ATLAS then the problems should be reported to the ARINC 626 Subcommittee to enable it to consider ways of improving the extensibility facilities.

1.2.1 Introduction

1.2.1.1

ATLAS is a standard language whose purpose is to specify test requirements for a wide range of equipments. Test requirements contain a procedural segment of statements containing test values related to the UUT. To this end it is able to describe tests involving many different signal types. ATLAS can be used as an effective tool by customer and supplier organizations to enforce standardization.

1.2.1.2

In recognition of future requirements to define tests in particular specialist areas such as mechanical, automotive, power generation and so on, extensibility constructs have been included in ATLAS to enable new language elements to be established to specify these test requirements as they arise.

1.2.1.3

Where a new user community has the need to specify test requirements it is essential that all the capabilities inherent in ATLAS are easily accessible and that the user community is given the maximum incentive to use ATLAS and not some other means of specifying the test requirements.

The object of the extensibility facilities is to allow specialists in a particular test discipline to develop ATLAS to meet their immediate needs without the delays necessarily involved in the formal approval procedures needed to review new ATLAS language.

1.2.1.4

While the extensibility facilities help to avoid inordinate growth in the standard which would make it difficult to use, there is clearly a danger of weakening of the Standard if extensibility is used in an excessive and ill-considered way. In order to prevent this potential weakening of the Standard, management procedures for extensibility are essential.

1.2.2 Extensibility Management Philosophy

1.2.2.1

The ATLAS Language constructs are defined in a way that enables writing a single preferred group of test statements for a specific test. If the necessary ATLAS Language constructs exist in this Specification, they should be used. It is improper to use the Extensibility Constructs to define an alternative way of writing material which can be written using the ATLAS Language Constructs as defined by this Specification without recourse to the extensibility mechanism that is defined in this Specification.

1.2.2.2

If no vocabulary exists in this Specification that will adequately describe the testing required, the Extensibility Facility as described below should be used to generate new language vocabulary which will provide the required testing capability.

If no vocabulary exists in this Specification but does exist in IEEE Std.716 titled “C/ATLAS Test Language,” the Extensibility Facility should be used to introduce the required vocabulary from the IEEE Std.716 document into the test requirement written in the ATLAS language.

1.0 PURPOSE

New vocabulary hitherto not defined in any other ATLAS Language definition should only be used when all other ATLAS Language resources have been exhausted.

1.2.2.3

It is unlikely but possible that the extensibility facility may be unable to introduce the vocabulary that is required for particular testing. Only then should non-ATLAS modules be used.

1.2.2.4

Whenever EXTEND is used beyond the prior uses that are documented in Appendix A and when other facilities such as non-ATLAS are used they should be supplied in detail to ARINC for consideration by the ATLAS Language Working Group which is responsible for generating material for the ARINC Specification 626 subcommittee. ARINC will acknowledge the receipt of the information and confirm that it has been supplied in sufficient detail. It will also advise if it appears to be in conflict with the earlier content of Appendix A and will draw attention to alternative material in Appendix A which should be considered to determine whether it satisfies the users need. It is strongly recommended to users that QA review of the acknowledgement should form part of the formal acceptance of any test program using EXTEND or non-ATLAS. Appendix A is in five parts.

The first part contains an alphabetic list of EXTEND NOUN uses that have been registered with ARINC. In each case the actual EXTEND statement employed is included in the Appendix. If two users have used the same noun for the same signal in different but compatible ways the entry will be an amalgam of the two statements which is the superset of their statements. Each EXTEND NOUN statement entry in Appendix A is immediately followed by all its associated EXTEND CONNECTION statements and then its associated EXTEND MODIFIER statements.

The second section of Appendix A is a list of all nouns in alphabetical order. It includes nouns that are part of ARINC Specification 626 and nouns added by EXTEND. A second column indicates the source of the noun. The entries may be 626, 416 for those nouns that are added by EXTEND from IEEE Std.416, ARINC (if ARINC is authorized by the user which submitted the particular use of EXTEND to distribute copies of the associated detailed definitions upon request) or the name and address of the original source of the definition if that user requires subsequent users of the EXTEND to obtain details of the meaning of the added vocabulary from him.

In this latter case, if a second user subsequently offers definitions for ARINC to distribute, they will become the formal definitions accepted by ARINC even if this leaves the original user incompatible with the rest of the industry.

The third section is an alphabetical list of all modifiers with the same second column indication of the source of the modifier definition and a third column indicating the noun(s) under which an EXTEND modifier is defined.

The fourth section is a similar alphabetical list of dimensions.

The fifth section is a similar alphabetical list of connections.

1.2.3 Extensibility Techniques

1.2.3.1

ATLAS has always had a number of features which have permitted shorthand ways of defining complex tests to be employed. Examples include DEFINE PROCEDURE and DEFINE SIGNAL. However it is not these facilities which are meant in Section 1.2.2.2 when it refers to Extensibility Facilities. These are the included facilities which permit the definition of language to handle tests which are outside the scope of ATLAS. This may require the definition of new types of signals, possibly in entirely new technologies, or of new test actions. The following sections describe these techniques.

1.0 PURPOSE

1.2.3.2 Extend Nouns

This defines how additional types of signal can be handled by associating noun names with them.

1.2.3.3 Extend Modifiers

This defines how the set of modifiers associated with a new signal (noun) can be established. It also permits an additional modifier to be added to an existing (standardized) noun to permit a parameter of signals of that type to be addressed. Where necessary original syntax can be defined and be associated with the modifier.

1.2.3.4 Extend Connections

This defines how the connection description associated with a new noun can be established and how the permitted connection descriptors associated with a standardized noun can be supplemented.

2.0 REFERENCE MATERIAL

2.1 Applicable Documents

IEEE Std 771	IEEE Guide to the Use of the ATLAS Specification
IEEE Std. 716	IEEE Standard C/ATLAS Test Language
ARINC Char. 573	Aircraft Integrated Data System - Mark 2(AIDS) for the URZ Code
ARINC Char. 575	Mark 3 Subsonic Air Data System (Digital) DADS for the ARINC Grey Code (Attachment 5)
ARINC Char. 579	Airborne VOR Receiver - Supplement 2
ARINC Report 627	Programmers' Guide for SMART™ Systems Using ARINC 626 ATLAS
ANSI X3.12-1966	American National Standards Institute document “Vocabulary for Information Processing”
ANSI X3.4-1977	USA Standard Code for Information Interchange, for the Seven-Bit ASCII Code
ISO 1000	International Standard Dimensions
BS 2G 199:1984	Tables relating to altitudes, airspeed and Mach numbers for use in aeronautical instrument design and calibration
106-71	Telemetry Standards, Revised January 1971, Telemetry Working Group of the Inter-Range Commanders Council, White Sands Missile Range, New Mexico 88002 (See Figure 4 on page 42)

2.2 Document Precedence

In case of any conflict in the definition of ATLAS, the following precedence is determined in the order listed:

(1) ARINC Specification 626 Standard ATLAS for Modular Test (this document)

- a. Formal Syntax
- b. Rules and Descriptions
- c. Syntax Diagrams
- d. Notes and Examples

(2) ARINC Report 627 Programmers' Guide for SMART™ Systems Using ARINC 626 ATLAS

(3) IEEE Std 716 IEEE Standard C/ATLAS Test Language

- a. Formal Syntax
- b. Rules and Descriptions
- c. Syntax Diagrams
- d. Notes and Examples

(4) IEEE Std 771 IEEE Guide to the Use of the ATLAS Specification

2.3 How to Use this Specification

2.3.1 Structure

This document is organized in paragraphs which represent the major elements of ATLAS. The paragraph organization is shown in Table 2-1. The sectional numbering within chapters has been retained to align with previous issues of ARINC Specification 626 thereby facilitating cross-reference between issues. This means that some section numbers are not used and are marked so.

2.0 REFERENCE MATERIAL**Table 2-1. ATLAS Specification Organization**

Chapter	Title	Topic
1.0	Purpose	Describes ATLAS characteristics, levels of ATLAS, rules for creating ATLAS subsets and Extensibility Management Procedures.
2.0	Reference Material	Lists applicable documents, document precedence and the use of this Specification.
3.0	ATLAS Test Specification	The structure of an ATLAS test specification, including the main sections, their allowed order and statement structure.
4.0	Structure Delimiter Statement	ATLAS statements which serve to delimit the test specification sections.
5.0		Reserved for future use.
6.0	Preamble Statements	ATLAS statements which are only allowed in a Preamble Section.
7.0	Procedural Statements	ATLAS procedural statement structure.
8.0	Data Processing Statements	ATLAS statements which are concerned with data processing functions.
9.0	Input/Output Statements	ATLAS statements which are concerned with input/output functions.
10.0	Control Statements	ATLAS statements which are concerned with control of the execution of the test specification.
11.0	Signal Oriented signals. Statements	ATLAS statements which are concerned with the application and measurement of signals.
12.0	Timing Statements	ATLAS statements which are primarily concerned with timing.
13.0	Databus Statements	ATLAS statements which are MACRO functions.
14.0	Field and Subfield Definition	Definition of the major fields and subfields of ATLAS statements.
15.0	Language Component	Definition of the fundamental language components of the ATLAS Specification statements, fields and sub-fields.
16.0	Nouns and their Modifiers	Definition of the nouns (signals) and identification of the associated modifiers in ATLAS.
17.0	Noun-Modifier Definition	Definition of noun-modifiers (signal characteristics) for ATLAS nouns.
18.0	ATLAS Formal Syntax Definition	Definition of ATLAS syntax, semantics.

2.0 REFERENCE MATERIAL

2.3.2 Terminology

Formal ATLAS words and characters are written throughout this Specification in uppercase letters. Formal ATLAS words and characters are all that an automatic compiler is expected to understand.

In describing the syntax of the language, words or phrases were developed which are peculiar to the ATLAS definition, but which should not be written in ATLAS test specifications. These words are written in lower case, enclosed by an “<“and”>” (e.g., <data store>) when used in this manner.

This Specification contains many non-ATLAS words which are in common use in the automatic data processing community. Extensive use has been made of the ANSI document X3.12-1966 entitled “Vocabulary for Information Processing.”

2.3.3 How to Read the Formal Syntax

See the Introduction to Chapter 18 titled “ATLAS Formal Syntax Definition.”

2.3.4 How to Read Syntax Diagrams

2.3.4.1 Capital Letters

All capital letters represent themselves. (e.g., CNX represents CNX in an ATLAS statement.)

2.3.4.2 Lower Case Letters

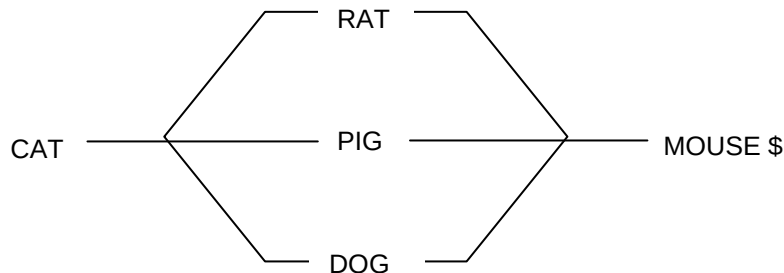
Words in lower-case letters will always be enclosed by “<“and”>” and represent various statement elements. These statement elements are all defined in terms of formal ATLAS words or characters in this Specification.

2.3.4.3 Other Characters

All other characters, such as the currency symbol (\$), slash(/), etc. represent themselves.

2.3.4.4 Syntax Diagram Notation

The syntax diagram notation employs lines similar to flowcharting lines to show how elements may be put together to form complete statements. Flow lines must be followed from left to right except where an arrowhead on the line indicates an exception, such as a feedback loop. The following syntax drawing is an example:



2.0 REFERENCE MATERIAL

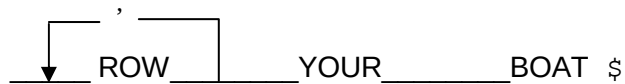
This tells us that CAT RAT MOUSE \$ and CAT DOG MOUSE \$ are legitimate statements. Requirements for spaces as delimiters are not shown in syntax drawings since spaces are generally not significant in ATLAS. Any precise space requirements are specified in the Formal Syntax.

The next syntax diagram example uses a feedback loop to indicate the repetition of an element and defines statements such as:

ROW YOUR BOAT \$

ROW, ROW YOUR BOAT \$

ROW, ROW, ROW YOUR BOAT \$

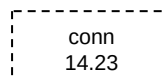
**2.3.4.5 Procedural Statement Diagrams**

The diagrams of procedural statements themselves are illustrated as they would appear after all substitutions for pre-defined segments of the statement are made. For example, all or part of a MEASURE statement could be represented by a <signal> label in an actual procedural statement. The syntax diagram defines the format of the statement when the <signal> <label> has been replaced by the fields it represents.

2.3.4.6 Sub-Diagrams

To avoid repetition of certain statement elements which are common to several statement diagrams, these elements are presented as sub-diagrams and referenced in other diagrams by the associated lower-case letter word enclosed in a broken border.

Also included in the broken border is a set of characters which identify the Section in this Specification where that element is defined. For example:



This means that the sub-element <conn> is defined in Section 14.13.

When a sub-diagram is defined in syntax diagram format (in terms of ATLAS reserved words or maybe other sub-diagrams), the sub-diagram is on the left hand side of the top major path of the syntax diagram and the border around it is solid.

2.3.5 Relationship of Formal Syntax to Syntax Diagrams

The following examples illustrate the relationships between the Formal Syntax and Syntax Diagrams.

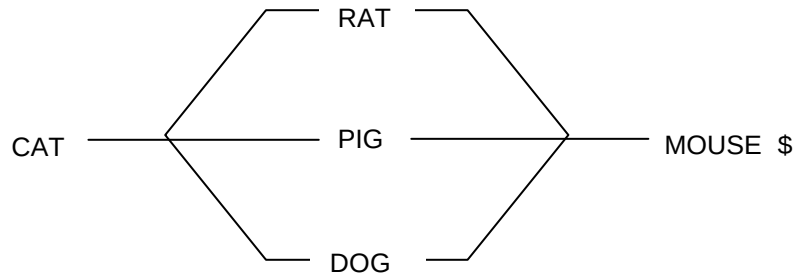
2.0 REFERENCE MATERIAL**2.3.5.1 Example 1 - Alternative Syntax Elements**

Formal Syntax

cat-statement::

“CAT” { “RAT” | “PIG” | “DOG” } “MOUSE” statement-terminator ;

Syntax Diagram



The Syntax Diagram shows at a glance that CAT may be followed by RAT or PIG or DOG, any one of which must be followed by MOUSE. The variable name <cat statement> will be referred to in a higher level construct.

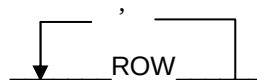
2.3.5.2 Example 2 - Replication

Formal Syntax

row-field::

{ “ROW” # fd } ;

Syntax Diagram



This example illustrates that ROW may be repeated one or more times preceded by the field delimiter (,).

2.3.5.3 References to the Formal Syntax Definition (FSD)

The section titled “Formal Syntax” for each ATLAS Structure or Statement has a cross-reference to the appropriate line in the Formal Syntax Definition that is provided in Section 18.6, SYNTAX DEFINITION, in Chapter 18. A cross-reference is of the form:-

FSD / Line Number

where the Line Number is that which begins the Production Rule for the particular ATLAS Structure or Statement being defined in the Formal Syntax Definition (FSD) in Section 18.6 of Chapter 18.

3.0 ATLAS TEST SPECIFICATION

An <atlas test specification> is wholly dedicated to defining the test requirements for the Unit Under Test in a manner which is independent of the test equipment used for its implementation. The actual implementation may use manual test equipment, automatic test equipment, or a combination of both.

The <atlas test specification> may be viewed as specifying a set of resources required to test the Unit Under Test. These resources are termed “virtual resources,” since they cite only test requirements of the Unit Under Test and not the actual resources used for implementation. ATLAS provides an optional capability for either implicitly or explicitly specifying these virtual resources.

Virtual resources are explicitly specified when the test procedure writer establishes and labels a summary for the desired signal oriented test capacity through use of a REQUIRE statement in the <program preamble structure>. The <requirement> label thus established is written in the <noun field> of all ATLAS statements which reference the virtual resource associated with this test capacity. The <requirement> label also provides a linking mechanism for equating the virtual resources to the actual test resources used for implementation of the <atlas test specification>. A virtual resource is implicitly specified when the test procedure writer chooses not to label and summarize the desired test capacity in a <require statement>. In this case, the resource requirements must be established by a detailed analysis of the entire <atlas test specification>.

The implementation of an <atlas test specification> requires checking and equating the virtual resources defining the test requirements for the Unit Under Test to the actual test resources comprising a particular configuration of manual or automatic test equipment. The process often requires additional information related to the detailed test characteristics of a particular test equipment which are not known to the test procedure writer of the <atlas test specification>.

3.1 <atlas test specification structure>

FSD / 13

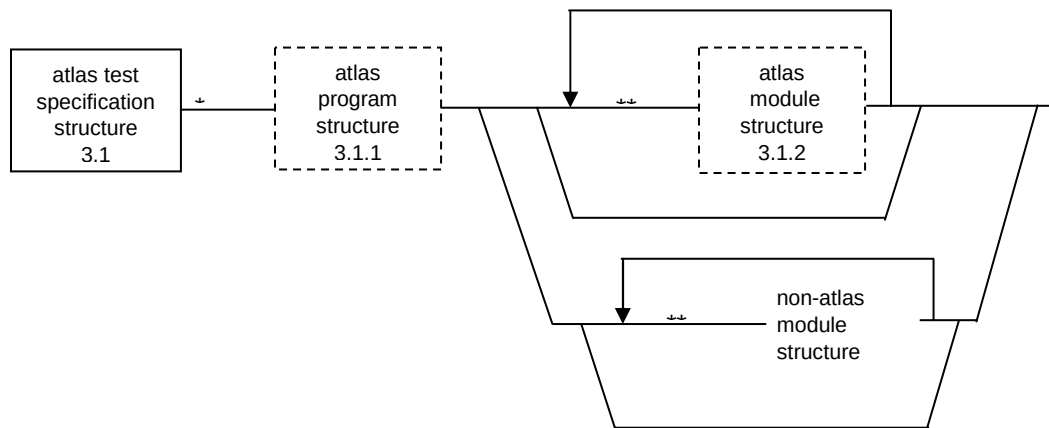
The document or set of documents which records the task to be applied to a UUT, which may fulfill the requirement for a test specification, a test procedure or a test program, is called an <atlas test specification structure>. An <atlas test specification structure> consists of one <atlas program structure> and optionally any number of <atlas module structures> and <non atlas module structures>.

ATLAS and NON-ATLAS Modules exist as entities separate from the <atlas program structure> and form part of the <atlas test specification structure> preamble. They are linked together as a result of an <include statement> in the <atlas program structure>.

ATLAS and NON-ATLAS Modules provide a facility for preparing a large <atlas test specification structure> in a number of smaller manageable portions with a controlled interface. Some portions may be shared between two <atlas test specification structures> by being referenced in an <include statement> in both programs.

3.0 ATLAS TEST SPECIFICATION

Figure 3-1 illustrates the structure of an ATLAS test specification and identifies the section where each structure is defined in this specification.



Note: * Mandatory
 ** Optional; none or any number

Figure 3-1 ATLAS Specification Family Tree

3.1.1 <atlas program structure>

FSD / 21

An <atlas program structure> consists of five distinct elements; the <begin atlas program statement> serving to delimit the start of the structure, a <program preamble structure>, a <commence main procedure statement> serving to delimit the start of the main procedure, the <main procedural structure> and the <terminate atlas program statement> serving to delimit the end of the structure.

The <program preamble structure> precedes the <main procedural structure>.

Program preamble statements do not cause any tests to be executed, but the information contained in the <program preamble structure> may be referenced in the <main procedural structure>, any <atlas module structure> and any <non atlas module structure>.

The <program preamble structure> also contains references to information contained in any <atlas module structure> and <non atlas module structure>.

The <main procedural structure> is where the execution of the test procedure starts.

Figure 3-2 illustrates the structure of an ATLAS program and identifies the section where each structure or statement is located in this specification.

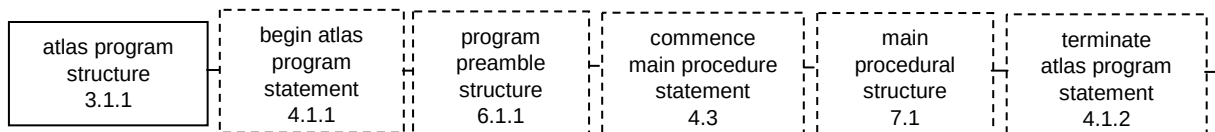


Figure 3-2 ATLAS Program Structure

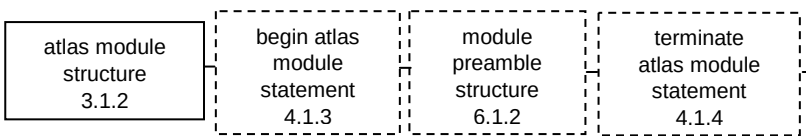
3.0 ATLAS TEST SPECIFICATION**3.1.2 <atlas module structure>**

FSD / 29

An <atlas module structure> consists of three distinct elements; the <begin atlas module statement> serving to delimit the start of the structure, the <module preamble structure>, and the <terminate atlas module statement> serving to delimit the end of the structure.

Module preamble statements do not cause any tests to be executed, but the information described in the <module preamble structure> may be referenced in a <program preamble structure>, any <atlas module structure> and any <non-atlas module structure>. The <module preamble structure> also contains references to information contained in the <program preamble structure>, in other <atlas module structures>, and <non-atlas module structures>.

Figure 3-3 illustrates the structure of an ATLAS module and identifies the section where each structure or statement is located in this specification.

**Figure 3-3 ATLAS Module Structure****3.1.3 <non-atlas module structure>**

FSD / 35

A <non-atlas module structure> is similar to an <atlas module structure> except that it is written in a language other than ATLAS. The <non-atlas module structure> is an independent entity and its implementation is dependent upon the run-time system. Information contained in a <non-atlas module structure> may be referenced in the <atlas program structure>, in an <atlas module structure> or another <non-atlas module structure> when they have been associated by <include statement>s. Refer to Section 1.2.2.3 for permissible use of non-ATLAS modules within an <atlas test specification Structure>.

3.2 Basic Statement Elements

The structure of a general ATLAS statement is shown in Figure 3-4. Exceptions to this structure are <flag statement>s and <comment statement>s which are described in Section 15.3.2.

The detailed format for each verb type is specified in the appropriate section of this document. There is no limit to the length of an ATLAS statement. It is not necessary to complete the statement in one line. By inserting spaces between statement elements, as allowed by Section 15.4, the writer can begin a new line at any point in the statement with subsequent lines indented, if desired. This flexibility allows the writer to highlight some portion of the statement, such as the label, by its location on the page. The first three fields, namely flag, statement number and verb, must be written on the first row as required by the format. Blank lines can be included between statements to improve human readability.

3.0 ATLAS TEST SPECIFICATION

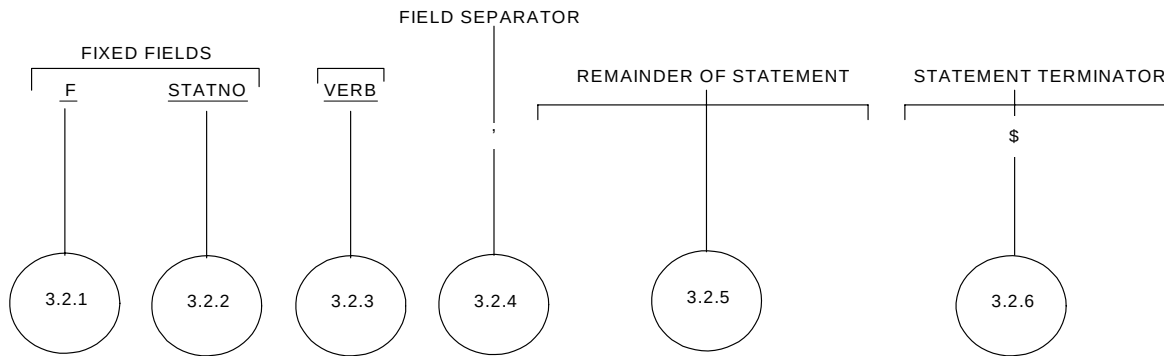


Figure 3-4 General ATLAS Statement Structure

Note: Numbers in circles refer to sections in this specification.

3.2.1 Flag Field

A flag in a flag field can be used to associate a special meaning to an ATLAS statement or to designate the beginning of a line of text which is commentary (see Sections 15.3.1 and 15.3.2 for the use of flags).

3.2.2 Statement Number Field

The statement number field is used to uniquely designate ATLAS statements (see Section 15.3.3).

3.2.3 Verb Field

The verb field identifies the test or data processing action for each statement. The ATLAS verbs are reserved words having unique and special meanings in the ATLAS language and therefore must not be used as variable names.

3.2.4 Field Separator

Since the fields following the verb are variable, a separator is required to identify the start of each new major field. A comma followed by zero or more spaces is designated for this purpose. Within some major fields, blank spaces are used as subfield separators (see Section 15.4).

3.2.5 Remainder of Statement

This part of each particular type of ATLAS statement varies considerably. The detailed format following each verb type is specified in the appropriate section of this document.

3.2.6 Statement Terminator

The last character of every ATLAS statement will be the currency symbol (\$).

3.3 Manual Actions During Testing

A test requirement written in ATLAS is independent of the method of test, namely manual or automatic. The interpretation and means of execution of the ATLAS statements are dependent on the test method.

When using automatic test methods the Automatic Test Equipment (ATE) will implement all the statements except those which include MANUAL INTERVENTION or MANUAL DATA. For statements containing manual instructions the operator will do one of the following:

3.0 ATLAS TEST SPECIFICATION

- a. operate a control on the UUT,
- b. act directly as a sensor,
- c. input data to the test equipment,
- d. receive data from the test equipment,
- e. record data for test documentation.

When using manually operated test methods the operator will implement all statements in the test requirement, not merely those containing either MANUAL INTERVENTION or MANUAL DATA.

Whenever a manual action is required during an automatic test procedure the test operator must be instructed. The ATE provides this instruction by implementing an <output statement>.

For ATE implementations, the test operator performs the manual action for statements which contain the verbs:

WAIT FOR	used in conjunction with the noun MANUAL INTERVENTION.
MONITOR	which includes an implicit WAIT FOR MANUAL INTERVENTION, the nature of which must have been conveyed to the test operator by a previous <output statement>.

4.0 STRUCTURE DELIMITER STATEMENTS**4.1 BEGIN/TERMINATE Statements**

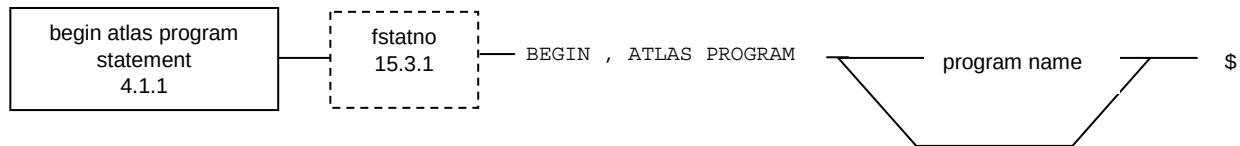
The BEGIN and TERMINATE statements indicate the first and last statement of an <atlas program structure> or <atlas module structure>.

4.1.1 <begin atlas program statement>**4.1.1.1 Function**

To indicate the start of the text of an <atlas program structure>.

4.1.1.2 Formal Syntax

FSD / 45

4.1.1.3 Syntax Diagram**4.1.1.4 Rules**

1. The first statement of every <atlas program structure> is a <begin atlas program statement>. This statement can be used only once within single <atlas program structure>.
2. The optional <program name> is used to identify the <atlas program structure>.

4.1.1.5 Notes and Examples

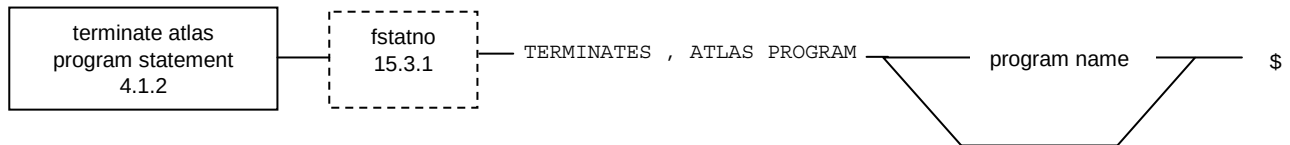
```
110000 BEGIN, ATLAS PROGRAM 'SERVO AMPLIFIER' $
```

4.1.2 <terminate atlas program statement>**4.1.2.1 Function**

To indicate the end of the text of an <atlas program structure>.

4.1.2.2 Formal Syntax

FSD / 51

4.1.2.3 Syntax Diagram**4.1.2.4 Rules**

1. The last statement in every <atlas program structure> is a <terminate atlas program statement>. This statement can be used only once within a single <atlas program structure>.
2. The optional <program name> must be identical to the <program name> in the <begin atlas program structure>.

4.0 STRUCTURE DELIMITER STATEMENTS**4.1.2.5 Notes and Examples**

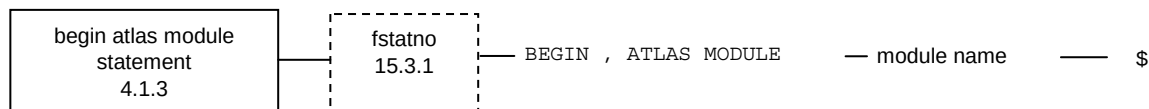
```
999999 TERMINATE, ATLAS PROGRAM 'SERVO AMPLIFIER' $
```

4.1.3 <begin atlas module statement>**4.1.3.1 Function**

To indicate the start of the text of an <atlas module structure>.

4.1.3.2 Formal Syntax

FSD / 57

4.1.3.3 Syntax Diagram**4.1.3.4 Rules**

1. The first statement of every <atlas module structure> is a <begin atlas module statement>. This statement can be used only once within a single <atlas module structure>.
2. The <module name> is to identify the <atlas module structure>, and is referenced by an <include statement> within the <program preamble structure>.
3. The <module name> has to be unique within one <atlas test specification structure>.

4.1.3.5 Notes and Examples

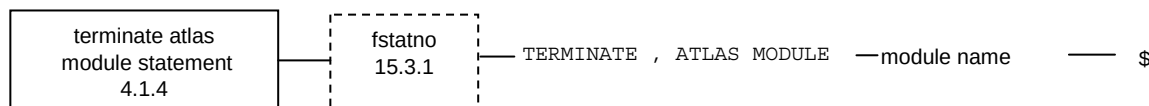
```
120000 BEGIN, ATLAS MODULE 'CALIBRATION PROCS' $
```

4.1.4 <terminate atlas module statement>**4.1.4.1 Function**

To indicate the end of the text of an <atlas module structure>.

4.1.4.2 Formal Syntax

FSD / 62

4.1.4.3 Syntax Diagram**4.1.4.4 Rules**

1. The last statement in every <atlas module structure> is a <terminate atlas module statement>. This statement can be used only once within a single <atlas module structure>.
2. The <module name> is to identify the <atlas module structure>, and must be identical with the <module name> in the <begin atlas module statement> of the same <atlas module structure>.

4.0 STRUCTURE DELIMITER STATEMENTS

4.2 Block Definition

This paragraph and paragraphs 10.1 through 10.4 define the structures which permit the ATLAS User to write well structured test specifications. Their use is preferable to other ATLAS control statements (e.g., GO TO). It is incorrect to write structures which are not wholly contained within outer structures. For example, if an IF THEN statement appears between a WHILE THEN and its associated END, WHILE, then the associated END, IF must appear prior to the END, WHILE.

4.2.1 <block structure>

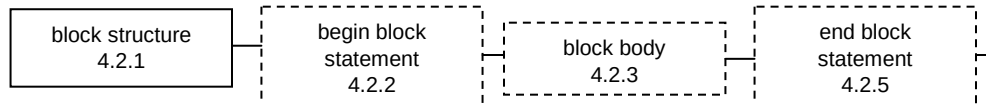
4.2.1.1 Function

To establish non-interfering program structures of the desired hierarchical levels. From the structural aspect a block may be considered as a single statement, in fact the block structure is referred to as a "compound statement" in other programming languages.

4.2.1.2 Formal Syntax

FSD / 67

4.2.1.3 Syntax Diagram



4.2.1.4 Rules

1. A <block structure> may appear only within the procedural section of an <atlas program structure> or <procedure body> or <function body>. It may not be used within another structure except in the special case that it appears within another <block structure> which has an E flag on its <begin block statement>.
2. <block structure>s are non-interfering stand-alone structures. Statement number references within the <block body> may only reference statement numbers within that <block body>. Statement numbers within a <block body> may not be referenced from outside that <block body>.

4.2.1.5 Notes and Examples

```

E200000 BEGIN, BLOCK, 'TURN-ON-SEQUENCE' $
      10 DECLARE, VARIABLE, 'TIME-IS-UP' IS BOOLEAN $
      60 CALCULATE, 'TIME-IS-UP' = FALSE $
      70 RESET, 'WARM-UP' $
      80 CALCULATE, 'T' = 0 $
C
C          APPLY UUT POWER $
C
      90 APPLY, AC SIGNAL, .... $
      .
      .
      .
200200 END, BLOCK 'TURN-ON-SEQUENCE' $
  
```

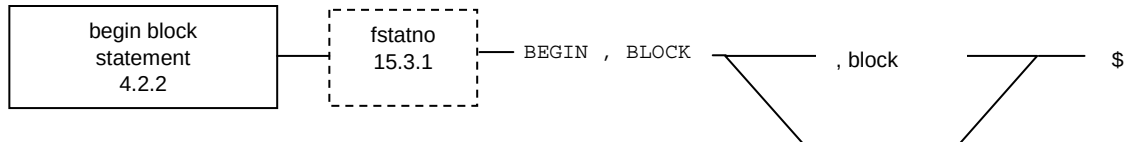
It is recommended that statements within a <block body> be indented, starting with the verb, in order to provide a clear visual indication of the <test specification structure>. Structures within other structures should be further indented so that all structural levels are clearly represented.

4.0 STRUCTURE DELIMITER STATEMENTS**4.2.2 <begin block statement>****4.2.2.1 Function**

To establish a block and identify its start.

4.2.2.2 Formal Syntax

FSD / 73

4.2.2.3 Syntax Diagram**4.2.2.4 Rules**

1. Each <begin block statement> must have a corresponding <end block statement>.
2. If a <begin block statement> is to be an entry point it must have an "E" flag.

4.2.2.5 Notes and Examples

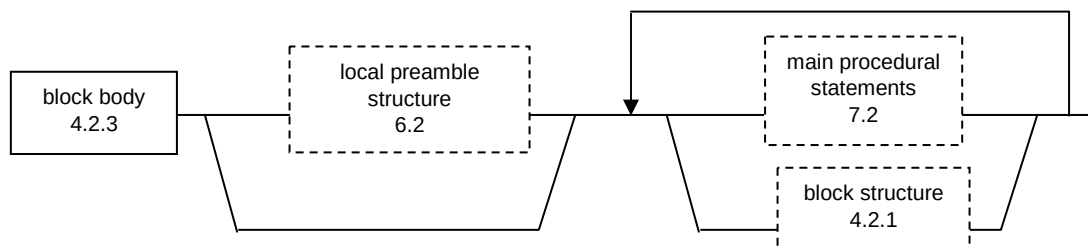
See Section 4.2.1.5

4.2.3 <block body>**4.2.3.1 Function**

To define the required test actions.

4.2.3.2 Formal Syntax

FSD / 79

4.2.3.3 Syntax Diagram**4.2.3.4 Rules**

1. Labels declared within the <local preamble structure> are local to that block.
2. Labels referenced within a block which are neither declared within that block nor in the main <program preamble structure> are invalid references.
3. Labels referenced within a block which are not declared in that block but are declared in the main <program preamble structure> relate to main program meaning.
4. The block body may not contain any entry points.

4.0 STRUCTURE DELIMITER STATEMENTS**4.2.3.5 Notes and Examples**

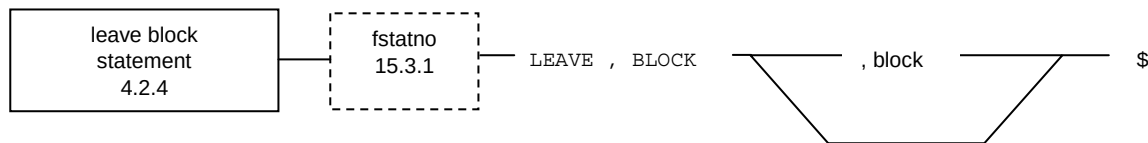
See Section 4.2.1.5

4.2.4 <leave block statement>**4.2.4.1 Function**

To cause the execution of a block to terminate prior to normal sequential execution of the <end block statement>.

4.2.4.2 Formal Syntax

FSD / 87

4.2.4.3 Syntax Diagram**4.2.4.4 Rules**

1. When a <leave block statement> is executed, control is transferred to the <end block statement>.
2. Any structure containing a <leave block statement> which is itself contained within the referenced block, is also exited upon execution of the <leave block statement>.

4.2.4.5 Notes and Examples

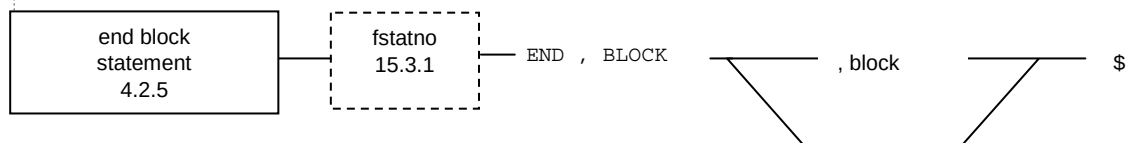
See Section 4.2.1.5

4.2.5 <end block statement>**4.2.5.1 Function**

To establish and identify the end of a block.

4.2.5.2 Formal Syntax

FSD / 93

4.2.5.3 Syntax Diagram

4.0 STRUCTURE DELIMITER STATEMENTS

4.2.5.4 Rules

1. Each <end block statement> must have a corresponding <begin block statement>.
2. The <block> name must be identical to the <block> name of the corresponding <begin block statement>.

4.2.5.5 Notes and Examples

See Section 4.2.1.5

4.3 <commence main procedure statement>

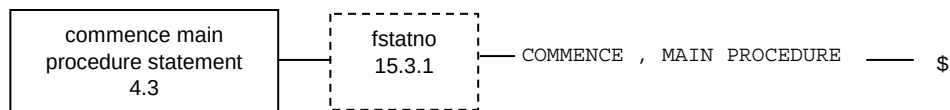
4.3.1 Function

To establish and identify the start of the <main procedural structure>.

4.3.2 Formal Syntax

FSD / 99

4.3.3 Syntax Diagram



4.3.4 Rules

1. There must be one and only one <commence main procedure statement> in an <atlas program structure>.
2. The <commence main procedure statement> is a directive and is not an executable statement.
3. The flag field portion of the <fstatno> field in a <commence main procedure statement> must contain either a blank or an E flag.

5.0 NOT USED

6.0 PREAMBLE STATEMENTS

6.0 Preamble Statements

An <atlas test specification structure> may contain a single <main preamble structure>, and each <define procedure structure>, <define function structure> or <block structure> may contain a single <local preamble structure>. In each case the preamble serves to identify the names and attributes of each <label> which is subsequently referenced within the appropriate structure.

Within the <atlas preamble structure> labels may be established for use in the <atlas program structure>, an <atlas module structure> or a <non-atlas module structure>. These labels are generally classified as follows:

a. Global

A label is global in scope if it has been established with the GLOBAL attribute in one of the structures mentioned above and with the EXTERNAL attribute in the others in which it is to be referenced.

b. Local

A label is local in scope if it is established with neither the GLOBAL nor the EXTERNAL attribute. It may then only be accessed within the structure in which it was established.

A full description of global and local attributes is located in Section 15.6.1.

6.0 PREAMBLE STATEMENTS

6.1 Main Preamble Structure

6.1.1 <program preamble structure>

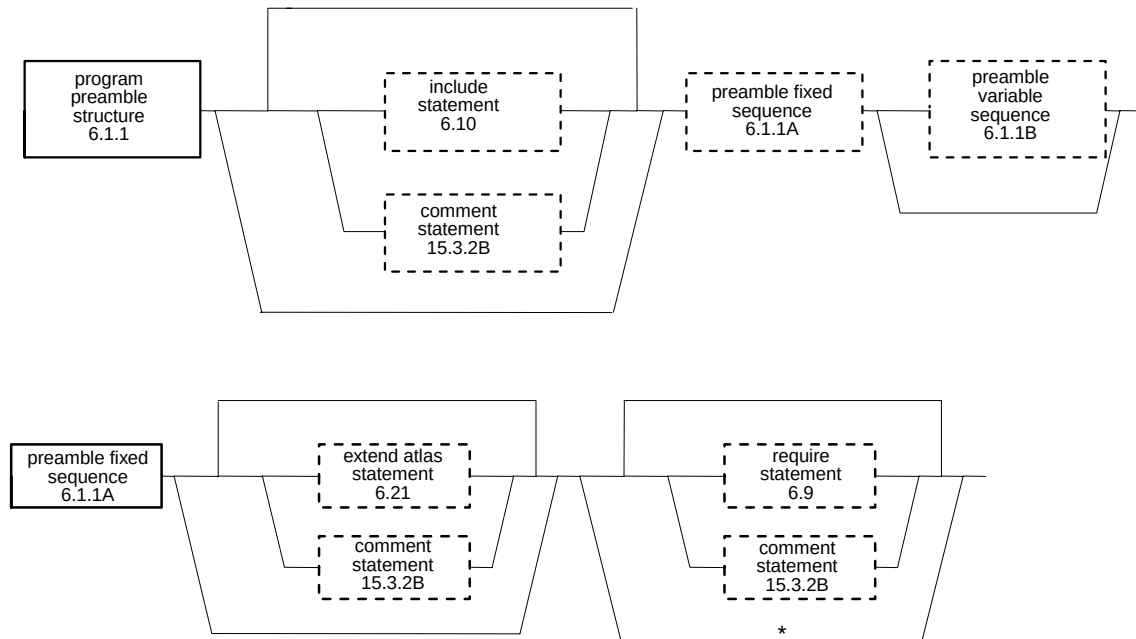
6.1.1.1 Function

To establish the name and attributes for global labels whose scope includes the whole of the <atlas program structure>.

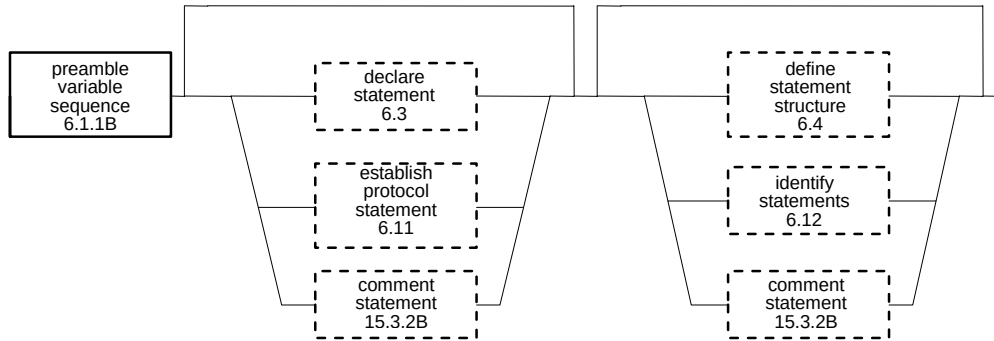
6.1.1.2 Formal Syntax

FSD / 107

6.1.1.3 Syntax Diagram



Note: * This path is used only in those unusual applications when no signal resources are demanded by the program or module. When the program or module demands real signal resources for its implementation, the ATLAS test requirement must be written in terms of virtual resources as defined by a <require statement>.

6.0 PREAMBLE STATEMENTS**6.1.1.4 Rules**

1. There may be only one <program preamble structure> within an entire <atlas program structure>.
2. The <program preamble structure> must appear immediately following the <begin atlas program statement>.
3. The <program preamble structure> is delimited by a <commence main procedure statement> which indicates the beginning of the <main procedural structure>.
4. The sequence of statements within the <program preamble structure> starts with <include statement>s, followed by <extend atlas statement>s, followed by <require statement>s then any sequence of <declare statement>s and <establish protocol statement>, followed by any sequence of <define statement>s and <identify statement>s. Each of these statement types is optional, thus, the <program preamble structure> may be empty.
5. Any <label> identified within the <program preamble structure> can be referenced subsequently in the entire <atlas program structure> unless an identical <label> is established at a lower level within a <local preamble structure>.

6.0 PREAMBLE STATEMENTS

6.1.2 <module preamble structure>

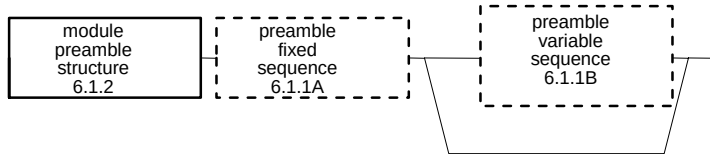
6.1.2.1 Function

To establish the name and attributes for global labels whose scope includes the whole of the <atlas module structure>.

6.1.2.2 Formal Syntax

FSD / 146

6.1.2.3 Syntax Diagram



6.1.2.4 Rules

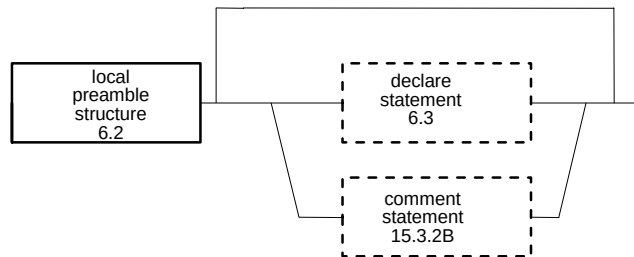
1. There may be only one <module preamble structure> within an entire <atlas module structure>.
2. The <module preamble structure> must appear immediately following the <begin atlas module statement>.
3. The sequence of statements within a <module preamble structure> starts with <extend statement>s followed by <require statement>s, followed by any sequence of <declare statement>s and <establish protocol statement>s followed by any sequence of <define statement>s and <identify statement>s.
4. Any <label> identified within the <module preamble structure> can be referenced subsequently in the entire <atlas module structure> unless an identical <label> is established at a lower level within a <local preamble structure>.
5. Those labels identified as being GLOBAL can be referenced from other <atlas module structure>s or the <atlas program structure> if identified therein as being EXTERNAL.

6.0 PREAMBLE STATEMENTS**6.2 <local preamble structure>****6.2.1 Function**

To introduce the body of a <block structure>, a <define procedure structure> or a <define function structure> and serves to identify the name and attributes of each <label> which is local to the appropriate structure.

6.2.2 Formal Syntax

FSD / 151

6.2.3 Syntax Diagram**6.2.4 Rules**

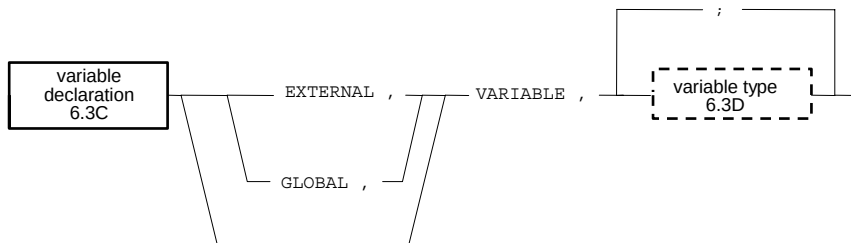
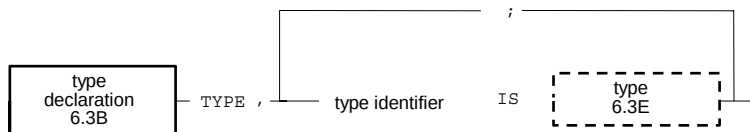
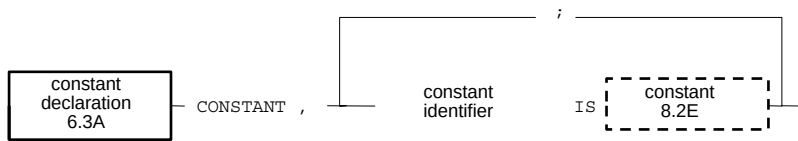
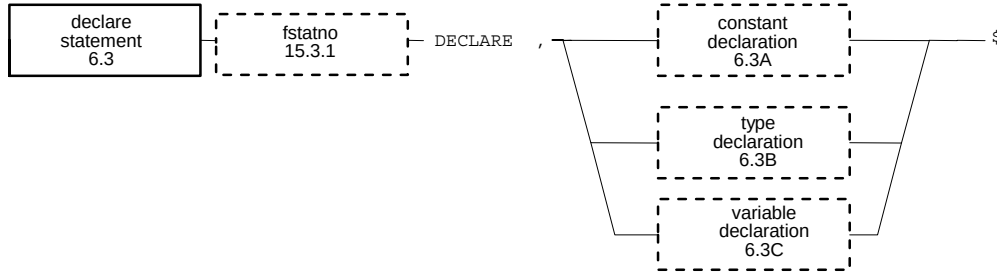
1. There may only be one <local preamble structure> within each <block structure>, <define procedure structure> or <define function structure>.
2. The <local preamble structure> must appear immediately following a <define procedure statement>, a <begin block statement> or a <define function statement> within the appropriate structure.
3. A <local preamble structure> is delimited by the first statement which is not a <declare statement>.
4. Any <label> identified within a <local preamble structure> is local to the <define procedure structure>, <block structure> or <define function structure>.
5. A <declare statement> within a <local preamble structure> is always local and consequently may not specify either a GLOBAL or EXTERNAL attribute.

6.0 PREAMBLE STATEMENTS**6.3 <declare statement>****6.3.1 Function**

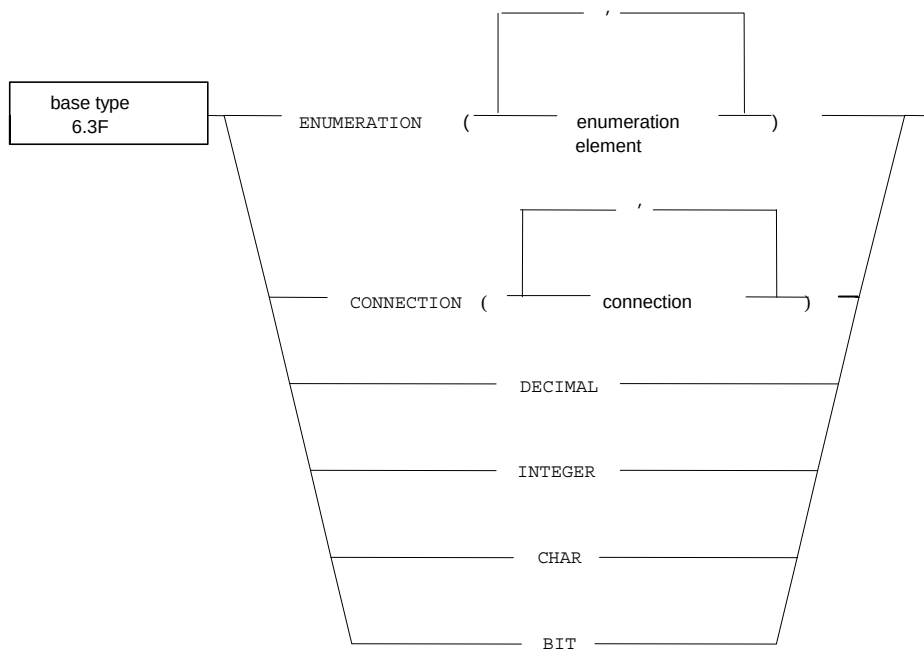
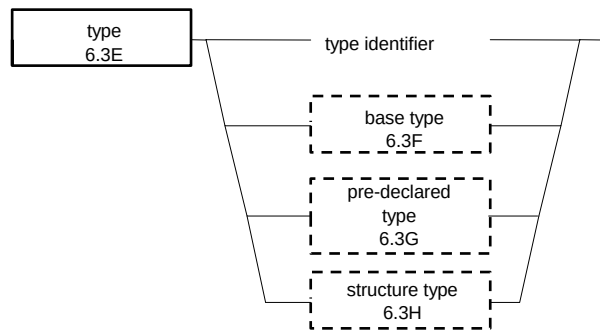
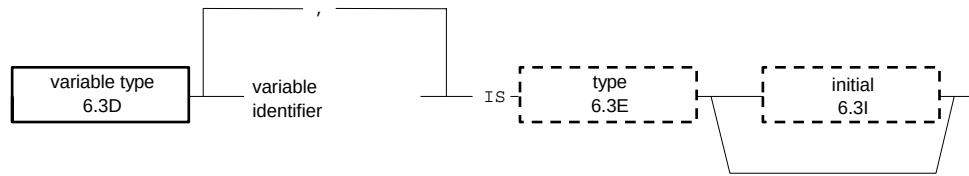
To establish, label and identify data types, constants and variables.

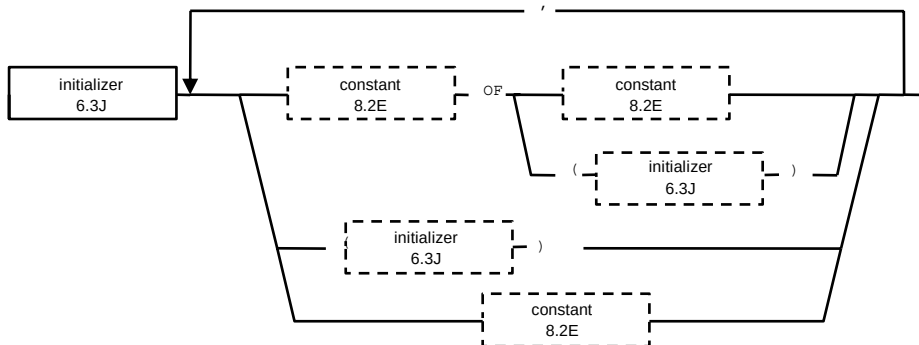
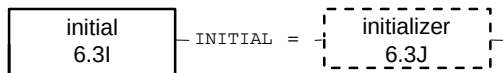
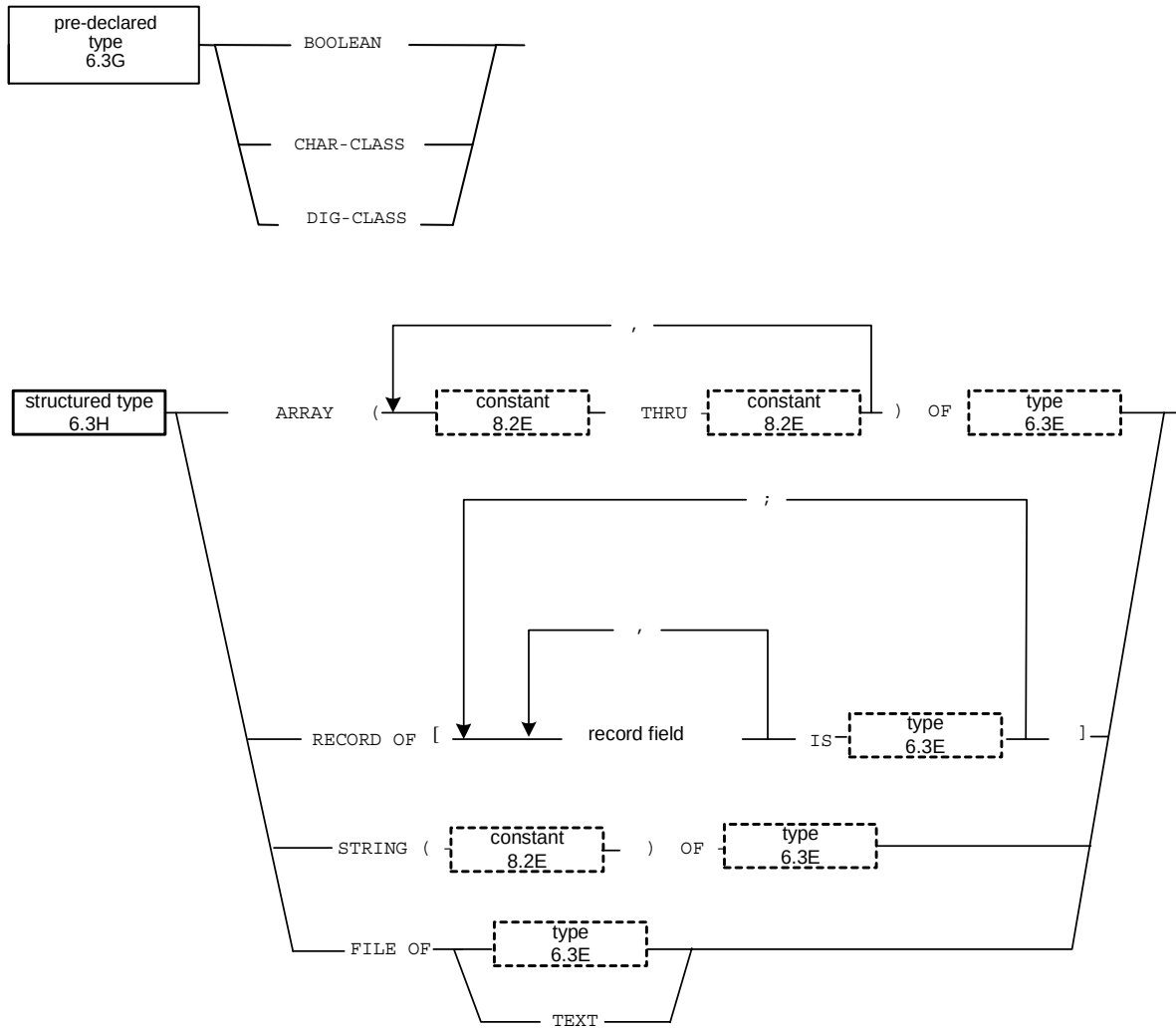
6.3.2 Formal Syntax

FSD / 158

6.3.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS



6.0 PREAMBLE STATEMENTS

6.0 PREAMBLE STATEMENTS**6.3.4 Rules for the <declare statement>****6.3.4.1 Rules for CONSTANT, TYPE, and VARIABLE forms of <declare statement>**

(Syntax Diagrams 6.3A, 6.3B and 6.3C)

1. The CONSTANT, TYPE, or VARIABLE <label> created is accessible at the structural level where it is declared and at any inner nested structural levels providing that same <label> has not been re-declared at a more local level.
2. Repetition via the “;” allows for the creation of multiple CONSTANT, TYPE, or VARIABLE <label>s in one <declare statement> without keyword repetition.
3. References to CONSTANT, TYPE, and VARIABLE <label>s can only be made after textually preceding declarations of these <label>s.
4. A <label> that is declared as GLOBAL must be unique within the entire <atlas test specification structure>. A global <label> declared in the <atlas program structure> may be accessed in an <atlas module structure> if that <atlas module structure> contains a <declare statement> for that <label> as EXTERNAL. A global <label> declared in an <atlas module structure> may be accessed in another <atlas module structure> or in the <atlas program structure> if that <label> is declared as EXTERNAL in those structures.
5. The attributes GLOBAL and EXTERNAL are not allowed in <local preamble structure>s.
6. Within any <main preamble structure> or any <local preamble structure> all declared labels must be unique. However, <record field> labels in different record types may be the same. Where nested preambles use the same labels, the innermost sense shall prevail following the redefinition until the structure in which it is redefined is exited.
7. If a VARIABLE is declared as GLOBAL in one structure and as EXTERNAL in another then the two must be of the same <type> and
 - a) if they are ARRAYs, the corresponding dimensions must have the same number of components with the same index number
 - b) if they are STRINGs, they must have the same number of components.
 - c) if they are RECORDs, they must have the same number of fields of the same <type>s and sizes in the same order.
8. INITIAL cannot be used to initialize an EXTERNAL VARIABLE.

6.3.4.2 Rules for CONSTANT form of <declare statement> (Syntax Diagram 6.3A)

1. The type of the constant label created is determined by inspection of the form of <constant> in accordance with the definitions of number and other data type representations in Section 15.2”.

6.3.4.3 Rules for TYPE form of <declare statement> (Syntax Diagram 6.3B)

1. TYPE declarations are used to avoid repetition of identical descriptions of data structures in VARIABLE declarations.

6.0 PREAMBLE STATEMENTS

6.3.4.4 Rules for VARIABLE form of <declare statement> (Syntax Diagrams 6.3C and 6.3D)

1. A VARIABLE declaration creates storage space for that variable according to its <type>. The <type> is expressed either explicitly in the VARIABLE declaration or by reference to a previously declared <type identifier>. The VARIABLE will be initialized if <initial> is invoked.
2. Any number of VARIABLES of the same <type> can be created by use of the “,” path.
3. VARIABLES declared in a <local preamble structure> cannot be relied upon to retain their values between invocations of the structure containing that <local preamble structure>.

6.3.4.5 Rules for <type> (Syntax Diagram 6.3E)

1. A TYPE form of <declare statement> establishes a <type identifier> which is associated with the described type.
2. When used in a VARIABLE form of <declare> statement or following <parameter>, the description of <type> can be stated explicitly without reference to a <type identifier> label.
3. Two expressions are type compatible if they have one of the following relationships:
 - a. of the same <base type>,
 - b. records with identical structure and type compatible corresponding components,
 - c. arrays with the same number of dimensions and type compatible components,
 - d. strings with type compatible components,
 - e. files that are one of the following:
 - i. of the same type,
 - ii. both TEXT.
4. An <expression> is assignment compatible with a <data store> when it fulfills either of the following conditions:
 - a. the <expression> and <data store> are type compatible.
 - i. when they are ARRAYS; corresponding dimensions must have the same number of components.
 - ii. when they are STRINGS; the current dynamic length of the <expression> is less than or equal to the declared length of the <data store>.
 - b. the <expression> is of type INTEGER and the <data store> is of type DECIMAL.

6.3.4.6 Rules for <base type> (Syntax Diagram 6.3F)

1. A variable of <base type> must assume a value whose type conforms to the declared <base type>. Thus for example; a <variable identifier> of type DECIMAL may only assume a <decimal number>.
2. The <enumeration element>s of a particular declared ENUMERATION type are constants ordered by the sequence in which they are written into the declaration.
3. The <label>s in all ENUMERATION type declarations must be unique. Thus <enumeration element>s are global constants and may not be redefined in inner scopes.
4. A CONNECTION variable identifies a UUT <connection> as defined in Section 14.13 <conn> (connection field).

6.0 PREAMBLE STATEMENTS**6.3.4.7 Rules for <pre declared type>**

(Syntax Diagram 6.3G)

1. The pre declared types are commonly used enumerations with enumeration elements defined by Syntax Diagram 8.2G. <pre declared type> exists as though each ATLAS program contained the following declaration:

```
DECLARE, TYPE, 'BOOLEAN' IS ENUMERATION ('FALSE', 'TRUE');
          'DIG-CLASS' IS ENUMERATION ('BNR', 'B1C', 'B2C',
          'BSM', 'BCD', 'SBCD');
          'CHAR-CLASS' IS ENUMERATION ('ASCII7') $
```

2. The <type identifier>s and <enumeration element>s which are enclosed by apostrophes in the above <declare statement> defining the <pre declared type>s are written without apostrophes in an <atlas test specification structure> in accordance with rule 2 of Section 15.5.
3. DIG-CLASS variables have values that are associated with digital data formats as follows:

BNR for unsigned positive binary numbers (no sign bit)

B1C for ones-complement binary numbers

B2C for twos-complement binary numbers

BSM for sign-plus-magnitude binary numbers

BCD for binary-coded-decimal numbers (4 bits per digit)

SBCD for sign-plus-BCD numbers (4 bits per digit plus a sign bit)

- BCD and SBCD numbers have an 8-4-2-1 coding.

- Signed numbers have a sign bit in the most-significant-bit position with a binary zero representing a positive sign.

- Other types of coding and sign conventions can be represented by BNR with the value of each bit of a number being translated into and out of ATLAS number types by conversion routines defined by either a <define procedure structure> or a <define function structure>.

4. CHAR-CLASS variables have values that are associated with character codes as follows:

ASCII7 for ASCII characters transmitted across the UUT/test equipment interface (7 bits per character respectively) (see Section 15.9.1)

6.3.4.7.1 Notes:

1. Values of DIG-CLASS are arguments to the pre-defined functions DIG and DEC.
2. Values of CHAR-CLASS are arguments to the pre-defined functions CHAR and BITS.

6.3.4.8 Rules for <structured type>

(Syntax Diagram 6.3H)

1. An ARRAY is a collection of data items of the same type. An ARRAY may have any number of dimensions. Each dimension has an associated index that must be bounded by upper and lower limits. A multi-dimensional ARRAY declaration is effectively a declaration of an ARRAY of an ARRAY.

For example: ARRAY(1 THRU 7, 1 THRU 9) is effectively

ARRAY(1 THRU 7) OF ARRAY(1 THRU 9).

6.0 PREAMBLE STATEMENTS

The constants that specify the boundary limits of a dimension of an ARRAY must be of the same <type>. The <type> must be either INTEGER, CHAR, or <enumeration element> of the same ENUMERATION type. The lower boundary limit is written first. Any element in an ARRAY can be addressed by its indices.

2. Array elements are assigned in row major order, such that the last written index varies most rapidly and the first written index varies more slowly in the conventional manner.
3. Operations between entire arrays require that the arrays involved are assignment compatible and have an equal number of elements. The only operations permitted between entire arrays are assignment and test for equality and inequality of all corresponding elements in both arrays. These operations shall not be performed on arrays with elements of type STRING.
4. A RECORD is a collection of data items that need not be of the same type or structure. Individual fields within a RECORD are given field names. Items of a RECORD are addressed by appending a period followed immediately by the <record field> label to the RECORD label. The declaration of fields of records is the same as for variables and the rules of Section 6.3.4.4 apply.
5. The only operations permitted between entire records are assignment and test for equality and inequality of all corresponding fields in both records. Records shall be assignment compatible and have equal number of elements. These operations shall not be performed on arrays with elements of type STRING.
6. The <constant> specifying the length of a STRING must be greater than zero and of base type INTEGER.
7. The base type of a STRING must be either BIT or CHAR.
8.
 - a. A STRING has an associated dynamic length attribute which holds the current length of the STRING. Initialization, input, assignment, and parameter passing operations set the dynamic length.
 - b. References to an individual BIT or CHAR of a STRING by the functions INSERT, COPY and DELETE are allowed within the current dynamic length subject to Rule 2 of Section 8.2.4.4.
 - c. Operations between STRINGs of the same base type and different declared sizes are permitted.
 - d. It is an error if the operation on a STRING would cause the current dynamic length to exceed the declared length.
9. Conversion of a STRING OF BIT to and from a numerical value is accomplished by using the DEC and DIG functions respectively in accordance with the DIG-CLASS parameter.
10. Conversion of a STRING OF BIT to and from a character is accomplished by using the CHAR and BITS functions respectively in accordance with the CHAR-CLASS parameter.
11. Within RECORDs, any number of fields of the same <type> can be created by use of the “,” path.
12. Within RECORDs, use of the “;” repetition operator allows for the creation of any number of fields which may be of different types.
13. A FILE is a collection of data items which may not contain another FILE. Each data item has a position in the file beginning with zero and advancing by one for each data item. Two classes of FILE are specified:
 - a. A known type file consists of a sequence of data items all of which have the same type. Input and Output must be to and from a variable of identical type.

6.0 PREAMBLE STATEMENTS

- b. A TEXT file consists of a stream of characters which equate to DECLARED data types after possible automatic Input/Output conversion.
14. A STRING OF BIT <expression> contains numbers of the <digital number> form.

6.3.4.9 Rules for <initial>

(Syntax Diagram 6.3I and 6.3J)

1. The action of an <initial> field is to set the initial value of VARIABLES. Initialization of variables always takes place in the order in which the variables are declared as follows:
 - a. For declarations in a <program preamble structure> or <module preamble structure>s and not in a <local preamble structure>, initialization takes place prior to the execution of the statement following the <commence main procedure statement> in the <atlas program structure>.
 - b. For declarations in a <local preamble structure>, initialization or re-initialization takes place prior to the first executable statement following the <local preamble structure> that contains the declaration.
2. VARIABLES cannot be initialized in either of the following circumstances:
 - a. They are of type FILE
 - b. They are EXTERNAL
3. INITIAL can be used to initialize a single VARIABLE or a series of VARIABLES. The <initializer> following INITIAL is a comma delimited list of values having a one-to-one correspondence with the VARIABLES. Each value may be one of the following:
 - a. A <constant> which corresponds to a simple variable.
 - b. A <constant> number OF <constant>s where the first <constant> must evaluate to a positive integer which specifies the number of instances of the second <constant>. Each second <constant> corresponds to a simple variable.
 - c. A parenthesized list of values having a one-to-one correspondence with the elements of an ARRAY or <record field> of a RECORD.
 - d. A shorthand notation for a repeated occurrence of a parenthesized list of values defined by c above. This is in the form of “<constant> OF (list of values)” where <constant> must be a positive integer which specifies the number of times the parenthesized list is to be repeated.

For example: “2 OF (0, 1)” means “(0, 1), (0, 1)” .
4. The <constant> value supplied by the <initializer> must be type compatible with the corresponding variable with the exception that DECIMAL variables may be initialized with INTEGERS.
5. For ARRAYS, initial values are assigned starting with the lowest value of all indices and varying the last written index most rapidly.

6.3.5 Notes and Examples for a <declare statement>

6.0 PREAMBLE STATEMENTS**6.3.5.1 Notes**

1. DIG-CLASS has values that can be used to denote the numerical weighting of each bit within a STRING OF BIT. The ATLAS writer may construct a RECORD with a STRING OF BIT <record field> and a related DIG-CLASS <record field>.
2. CHAR-CLASS has values that can be used to denote the coding of a STRING OF BIT that represents one or more characters. The ATLAS writer may construct a RECORD with a STRING OF BIT <record field> and a related CHAR-CLASS <record field>.

6.3.5.2 Examples

1. DECLARE, VARIABLE, 'PIN-1', 'PIN-2', 'PIN-3' IS CONNECTION (J1, J2, P10) \$

This example shows the multiple variable use of a <declare statement>. In this example, three locations are set aside for pin designations.

2. DECLARE, VARIABLE, 'MESSAGE' IS STRING(36) OF CHAR \$

This example shows how storage is set aside for a variable whose type is STRING OF CHAR. The length of the character string which can be stored in MESSAGE is equal to or less than 36 characters.

3. DECLARE, VARIABLE, 'STIM', 'MASK', 'RESP' IS STRING(10) OF BIT \$

This example illustrates how three storage locations, each of width 10 bits, are specified. The format of the data stored in these locations can be controlled by using the <predefined function> DIG for digital numbers in which a bit weight is to be associated with each bit position.

4. DECLARE, VARIABLE, 'PATTERN' IS RECORD OF
 ['PART1' IS STRING(4) OF BIT;
 'PART2' IS STRING(8) OF BIT;
 'PART3' IS STRING(10) OF BIT] \$

This example is a use of <record field> to specify storage for 'PATTERN' consisting of 22 bits composed of three individual parts 'PART1', 'PART2', and 'PART3' consisting of 4, 8 and 10 bits respectively.

5. DECLARE, VARIABLE, 'INDEX', 'WEIGHTFACTOR', 'COUNTER' IS
 ARRAY (1 THRU 5) OF INTEGER \$

This example shows how three one dimensional arrays, each containing five elements, are declared.

6. DECLARE, VARIABLE, 'PINSET1', 'PINSET2', IS ARRAY (1 THRU 5) OF CONNECTION
 (G10, P100, T200, J12, J13) \$

This example shows how storage may be established for an array of connection variables.

7. DECLARE, VARIABLE, 'COMMAND' IS ARRAY (1 THRU 5) OF STRING (20) OF BIT \$

This example establishes storage for five bit strings each of which may be up to 20 bits in length.

6.0 PREAMBLE STATEMENTS

8. DECLARE, VARIABLE, 'SINGLE' IS DECIMAL INITIAL = 1.234;
 'LEFT','RIGHT','CENTER' IS BOOLEAN INITIAL = 3 OF TRUE;
 'PATTERN' IS ARRAY (1 THRU 1200) OF STRING (16) OF BIT
 INITIAL = (1200 of X'0000');
 'WEEK-DAY' IS RECORD OF
 ['SUN', 'MON', 'TUE', 'WED', 'THURS', 'FRI', 'SAT'
 IS BOOLEAN] INITIAL = (FALSE, 5 OF TRUE, FALSE) \$

This example illustrates how several structures may be declared and initialized within a single <declare statement>.

9. DECLARE, VARIABLE,
 'CHESS-BOARD' IS ARRAY (1 THRU 8, C'A' THRU C'H') OF
 ENUMERATION ('EMPTY','BLACK-PIECE','WHITE PIECE')
 INITIAL = ((2 OF (8 OF 'BLACK PIECE')),
 (4 OF (8 OF 'EMPTY')),
 (2 OF (8 OF 'WHITE PIECE')));
 'FAILURES' IS RECORD OF ['COUNT' IS INTEGER;
 'PRESS-MESS','VOLT-MESS' IS STRING (20) OF CHAR
 INITIAL = (0, (C'OVER PRESSURE', C'OVER VOLTAGE')) \$

This example illustrates the declaration and initialization of enumerated data elements and the record structure.

10. DECLARE, VARIABLE, 'DATES' IS ARRAY (1 THRU 100) OF
 RECORD OF ['DAY','MONTH','YEAR' IS INTEGER]
 INITIAL = (100 of (0,0,0)) \$

This example illustrates the initialization of an array composed of records.

6.0 PREAMBLE STATEMENTS

6.4 <define statement structure>

6.4.1 Function

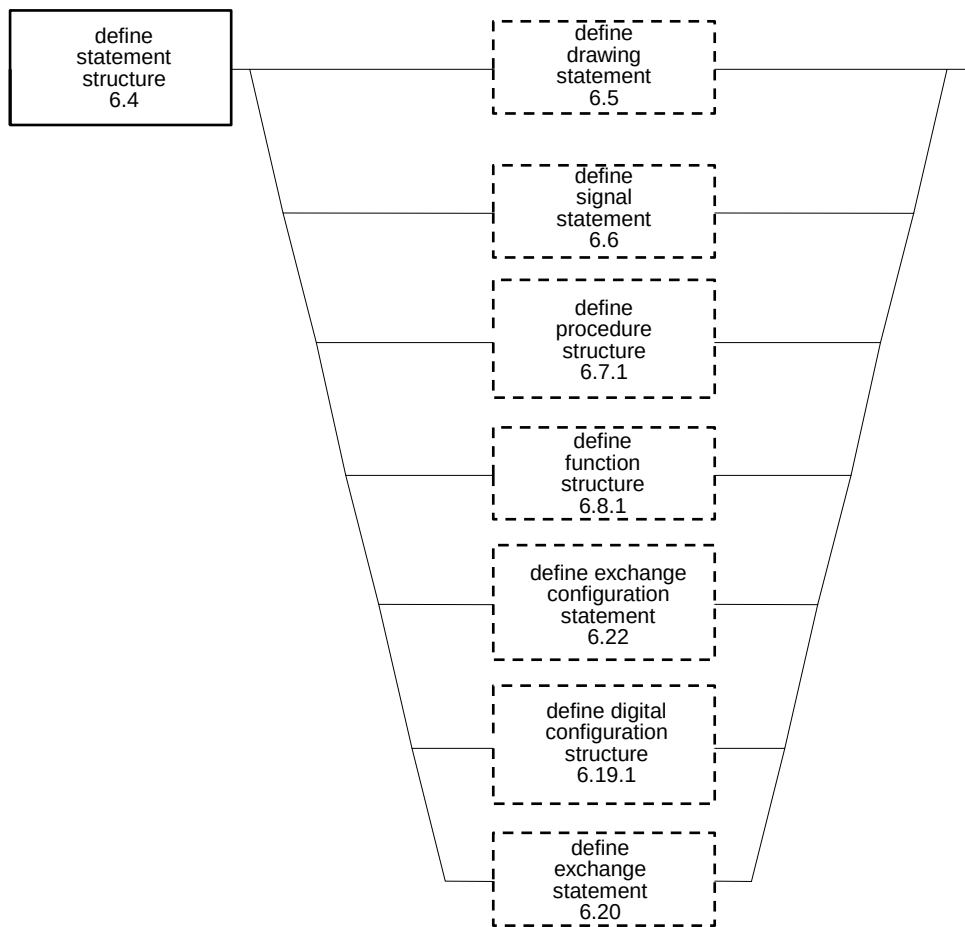
To establish and label a portion of the <atlas test specification structure> which is not executed until the label is invoked by direct or indirect reference in a subsequent <main procedural statement>.

A <define statement> establishes and identifies all or part of one statement or a series of statements. This capability can be employed to avoid rewriting often-used ATLAS segments by simply writing the <label> in the appropriate place in a subsequent <define statement> or <procedural statement>.

6.4.2 Formal Syntax

FSD /265

6.4.3 Syntax Diagram



6.4.4 Rules Applicable to all DEFINE Statements

1. A <define statement> is not allowed in a <local preamble structure>.
2. A <define statement> may only refer to a previous <define statement> which it does by reference to the appropriate <label>.

6.0 PREAMBLE STATEMENTS**6.5 <define drawing statement>****6.5.1 Function**

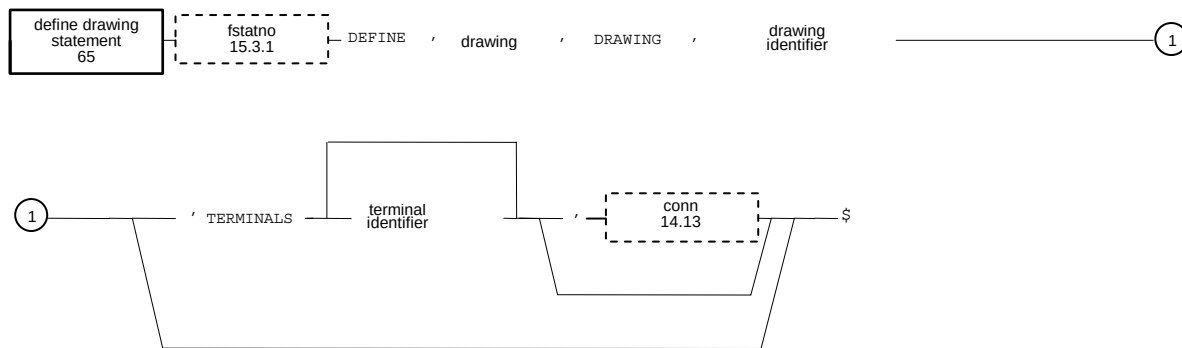
To provide test information which cannot be expressed in ATLAS language statements. This information should be provided in a drawing which is part of the <atlas test specification structure>. Such information may only be included for complex waveforms or for special circuit networks which are neither a part of the test equipment nor a part of the UUT.

The drawing form of a <define statement> should associate the drawing reference identifier with a given <label>. The reference identifier could simply be the reference number and issue of the drawing.

Where the drawing covers a special circuit network, then provision is made in the <define statement> to correlate the connections between the terminals of the circuit network and the connector pins of the UUT to which the circuit network must be connected during testing.

6.5.2 Formal Syntax

FSD / 277

6.5.3 Syntax Diagram**6.5.4 Rules**

1. The <define drawing statement> establishes and labels information for either special circuit network connections or waveforms neither of which can be expressed in other ATLAS statements.
2. Drawing definitions may only be referenced as follows:
 - a. special circuit networks, by the <apply statement> and the <remove statement>.
 - b. complex waveforms, by the <apply statement>, the <monitor statement> and the <remove statement>.
3. The identifier of the drawing can be any legal identifier and is often used to reflect the drawing reference number and issue.
4. Where the drawing covers a special circuit network, which is not part of either the test equipment or the UUT, the connection points on the circuit are referred to as terminals. The identifiers of these terminals are those quoted on the drawing.

6.0 PREAMBLE STATEMENTS

5. Pin descriptors in the connection field are, or are a subset of, the terminal identifiers which follow the word TERMINALS in the statement. Each terminal identifier in the connection field is preceded by an asterisk. Terminal identifiers as introduced by a <define drawing statement> must not be either ATLAS reserved words or previously defined pin descriptor names.
6. The bypass around TERMINALS <terminal identifier> is prohibited when terminal identifiers on a special test circuit are introduced by a drawing or when a complex waveform drawing is to be referenced in a subsequent <apply statement> or <remove statement>.
7. When the <define drawing statement> is used to describe a complex waveform which is to be measured in a subsequent <monitor statement>, the bypass of the TERMINALS and <conn> branches must be used. Connection information must be provided by one or both of the following:
 - a. within the <monitor statement> where the complex waveform is measured.
 - b. through another <define drawing statement> describing a special circuit network if one is necessary for the measurement of the waveform. An associated <apply statement> may also be necessary.
8. The use of the <monitor statement> with a complex waveform which is labelled in a <define drawing statement> is limited to operator manual intervention for waveform measurement.

6.5.5 Notes and Examples**6.5.5.1 Sample Statements**

```

006000 DEFINE, 'CIRCUIT', DRAWING, 67A3000 ISSUE D, TERMINALS P3 P5,
          CNX *P3 J1-1 *P5 J1-2 $
006010 DEFINE, 'WAVEFORM', DRAWING, 5962 $
006011 DEFINE, 'RESPONSE', DRAWING, 5977 $
006120 APPLY, 'CIRCUIT' $
006131 APPLY, 'WAVEFORM', CNX HI J1-11 LO J1-21 $
006201 MONITOR, 'RESPONSE', CNX HI J1-19 LO J1-20 $

```

6.5.5.2 Test Problems

The following test problems are best solved in ATLAS by using the <define drawing statement>.

If we make it mandatory for a writer to quote all the terminals that are defined on all the drawings which are part of a test procedure, a list of all TERMINALS in relation to each drawing label could be drawn. This would be useful to the test analysis engineer who is assessing the test procedure for implementation. By this means the test implementation engineer would have a print-out of the connection matrix necessary to connect all special test circuit networks needed to execute the test procedure.

6.0 PREAMBLE STATEMENTS**Test Problem 1**

To specify a special waveform (shown in Figure 6-1) in an <apply statement> which cannot be expressed in other ATLAS statements.

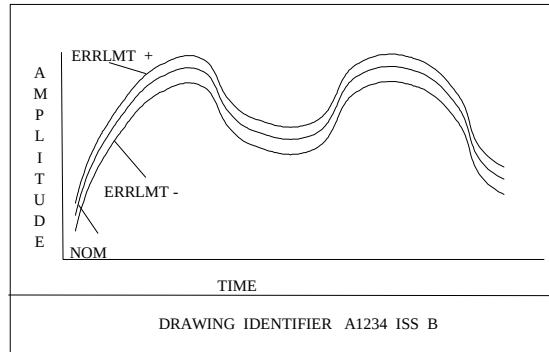


Figure 6-1 DEFINE DRAWING Test Problem 1

There are three possible options for the implementation of this requirement:-

- (i) the waveform is permanently applied to the UUT (e.g. by adapter installation).
- (ii) the waveform is applied by operator intervention during the testing.

In both of these cases, the identification of the terminals and UUT pins where the waveform is applied are optional but should be included for completeness. For example:-

```
DEFINE, 'WAVEFORM', DRAWING, A1234 ISS B, TERMINALS T-HI T-LO,
                                     CNX *T-HI J1-1 * T-LO J1-2 $
```

- (iii) the waveform is available in the adapter but must be subsequently connected to the UUT using <apply statement>s which identify the UUT pins to be connected.

In this case the drawing terminals to be connected must be identified in the <define drawing statement> and the connections to the UUT pins are established by an associated <apply statement>. For example:-

```
either: DEFINE, 'WAVEFORM', DRAWING, A1234 ISSB, TERMINALS T-HI T-LO $
        .
```

```
        APPLY, 'WAVEFORM', CNX *T-HI J1-1 *T-LO J1-2 $
```

```
or:      DEFINE, 'WAVEFORM', DRAWING, A1234 ISSB, TERMINALS T-HI T-LO,
        .
        .
        .
        CNX *T-HI ( ) *T-LO ( ) $
```

```
        APPLY, 'WAVEFORM', J1-1, J1-2 $
```

6.0 PREAMBLE STATEMENTS**Test Problem 2**

To specify a special waveform (shown in Figure 6-2) in a <monitor statement> which cannot be expressed in other ATLAS statements.

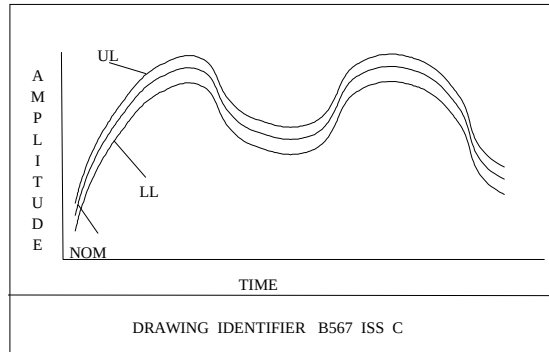


Figure 6-2 DEFINE DRAWING Test Problem 2

This illustrates the use of a <define drawing statement> where the same waveform is present at more than one pair of UUT connections. For example:-

```
either: DEFINE, 'RESPONSE', DRAWING, B567 ISS C, TERMINALS T-HI T-LO $
        .
        .
        MONITOR, 'RESPONSE', CNX *T-HI J2-3 *T-LO J2-4 $

or:     DEFINE, 'RESPONSE', DRAWING, B567 ISS C, TERMINALS T-HI T-LO,
        .
        .
        .
        CNX *T-HI ( ) *T-LO ( ) $
        MONITOR, 'RESPONSE', J2-3, J2-4 $
```

6.0 PREAMBLE STATEMENTS**Test Problem 3**

For multiple use of a test network (shown in Figure 6-3), the drawing can show all the connections.

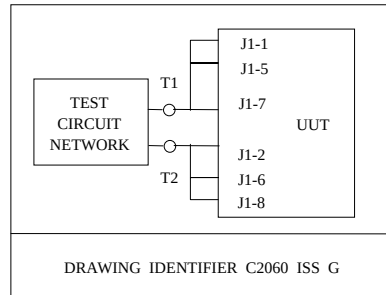


Figure 6-3 DEFINE DRAWING Test Problem 3

The <define drawing statement> would be for example:-

```

DEFINE, 'NETWORK', DRAWING, C2060 ISS G, TERMINALS T1 T2 $
.
.
APPLY, 'NETWORK', *T1 J1-1 *T2 J1-2 $
.
APPLY, 'NETWORK', *T1 J1-5 *T2 J1-6 $
.
APPLY, 'NETWORK', *T1 J1-7 *T2 J1-8 $

```


6.0 PREAMBLE STATEMENTS**Test Problem 4**

To specify a special waveform (shown in Figure 6-4) in an <apply statement> through a special test circuit network neither of which can be expressed in other ATLAS statements.

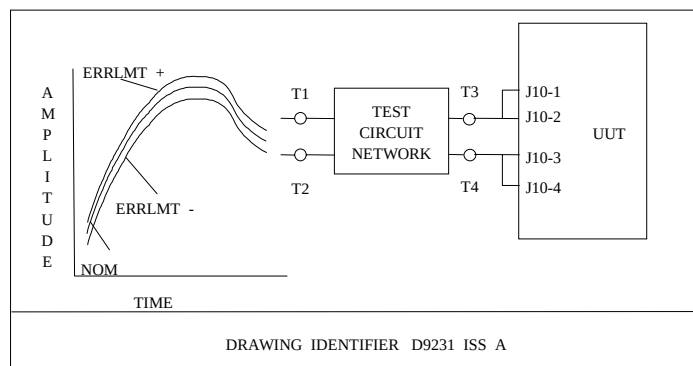


Figure 6-4 DEFINE DRAWING Test Problem 4

This problem can be specified for example as:

```

DEFINE, 'UUT TEST NO-1', DRAWING, D-9231 ISS A,
      TERMINALS T1 T2 T3 T4, CNX *T3 ( ) *T4 ( ) $
.
.
APPLY, 'UUT TEST NO-1', J10-1, J10-2 $
.
APPLY, 'UUT TEST NO-1', J10-7, J10-8 $

```

To specify a special waveform (shown in Figure 6-5) in a <monitor statement> through a special test circuit network, which is connected to the UUT.

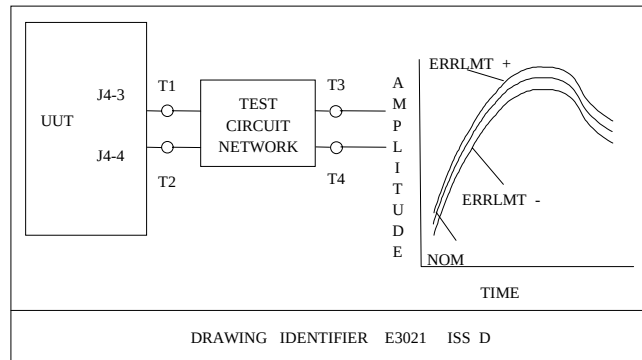


Figure 6-5 DEFINE DRAWING Test Problem 5

This example shows a single drawing containing both a special circuit network and a complex waveform to be monitored. There are alternative possible implementations of this problem depending on how the connections are to be established.

1. TERMINALS T1 and T2 are to be connected under Test Program Control. TERMINALS T3 and T4 are to be connected manually prior to the execution of the <monitor statement>. For example:-

```

REQUIRE, 'MONITOR-SCOPE', SENSOR (VOLTAGE), WAVEFORM ... $

```

Note:- This <require statement> has no CNX field.

```
DEFINE, 'UUT-TEST-NO-2', DRAWING, E3021 ISS D, TERMINALS T1 T2 T3 T4,  
CNX *T1 J4-3 *T2 J4-4 $
```

```
APPLY, 'UUT TEST NO-2', CNX *T1 J4-3 *T2 J4-4 $
OUTPUT, TEXT, ... C'Connect T3 and T4 to oscilloscope.' $
MONITOR, (VOLTAGE), WAVEFORM, USING 'MONITOR-SCOPE', UUT-TEST-NO-2' ... $
```

Note:- This <monitor statement> has no CNX field.

2. TERMINALS T1 and T2 are manually connected by adapter installation. TERMINALS T3 and T4 are connected under Test Program Control. For example:-

```

REQUIRE, 'MONITOR-SCOPE', SENSOR (VOLTAGE), WAVEFORM ... CNX HI LO $
DEFINE, 'UUT-TEST-NO-2', DRAWING, E3021 ISS D, TERMINALS T1 T2 T3 T4,
                                CNX *T1 J4-3 *T2 J4-4 $

```

```
MONITOR, (VOLTAGE), WAVEFORM, USING 'MONITOR-SCOPE', UUT-TEST-NO-2' ... ,
                                CNX HI *T3 LO *T4 $
```

6.0 PREAMBLE STATEMENTS**Test Problem 6**

To specify a special waveform (shown in Figures 6-6 and 6-7) in a <monitor statement> through a special test circuit using an oscilloscope for waveform monitoring.

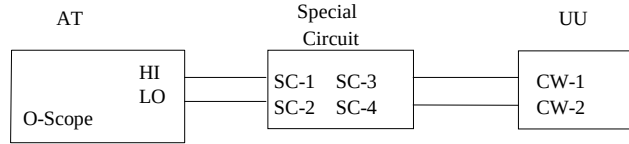


Figure 6-6 DEFINE DRAWING Test Problem 6

This problem can be specified for example as:

```

REQUIRE, 'O-SCOPE', SENSOR (VOLTAGE), WAVEFORM ... CNX HI LO $

DEFINE, 'SPEC-CIR-1', DRAWING, RH446-CIR,
    TERMINALS SC-1 SC-2 SC-3 SC-4 $

DEFINE, 'WAVEFORM-1', DRAWING, RH446-WF $
.
.

APPLY, 'SPEC-CIR-1', CNX *SC-3 CW-1 *SC-4 CW-2 $
OUTPUT, TEXT, FROM C'Set up the oscilloscope to ... .' $
MONITOR, (VOLTAGE), WAVEFORM USING 'O-SCOPE', 'WAVEFORM-1' ... ,
    CNX HI *SC-1 LO *SC-2 $

```

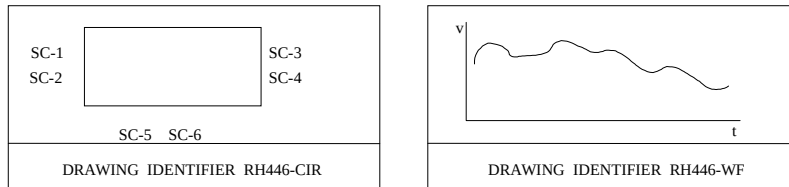


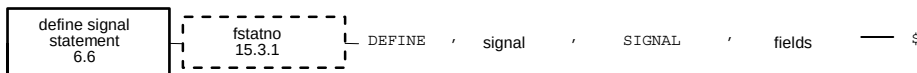
Figure 6-7 DEFINE DRAWING Test Problem 6 - Associated Drawings

6.0 PREAMBLE STATEMENTS**6.6 <define signal statement>****6.6.1 Function**

To assign a <label> to one or more adjacent fields of an ATLAS source, sensor or load type statement. When the <signal> label is written in a subsequent statement, a strict character substitution takes place to form a complete statement. Quantities in the <define signal statement> may be represented in parentheses () to indicate that the quantity is variable and a specific value will be provided when the label is cited in the subsequent <define statement> or <main procedural statement>. In a <define signal statement> the number of spaces between the parentheses is not important.

6.6.2 Formal Syntax

FSD / 289

6.6.3 Syntax Diagram

Note: Any character sequence that may appear between two commas in a procedural statement or between a comma and a \$ may be written here, for example, it may be a single modifier field, a single <conn> (connection) field, a <noun field> and two modifier fields, a modifier field and a <conn> field, a measured characteristic, a measured characteristic followed by a <noun field> which may be followed by modifier fields, a <noun field> followed by modifier fields followed by a <conn> field, and so on. A <real quantity>, number-dimension pairs, <event>, <requirement> and <connection> etc. may be designated as variable by replacing the item by parentheses (). A <label> that is defined in a <define signal statement> as above may be used subsequently in either a <define statement> or a <procedural statement>. The value for parentheses in the definition must be stated explicitly whenever the <label> is used. The values are listed in sequence immediately following the <label>. They will replace the parentheses in the definition in strict sequence. Then the complete character sequence from the definition will replace the <label> with all variables, if any, following it.

A valid ATLAS statement must result when the substituted information is combined with the other information in the statement. Following the parentheses designating a variable quantity, a range field containing non-variable quantities must be given as an option. Source and load functions may only be inserted in source-load type statements. Sensor functions may be inserted only in sensor type statements.

6.6.4 Rules

1. This type of statement is employed to establish and identify one or more fields which follow the verb of a source, sensor or load function for direct substitution in place of the <label> when the <label> is written in a subsequent statement.
2. Only complete comma-to-comma fields which are adjacent in the completed procedural statement may be included in the same <define signal statement>.
3. Connections and the numerical values associated with any <real quantity> in a <define signal statement> may be designated variable by inserting a parentheses in place of the number or connection. The dimensions of the variables shall immediately follow the parenthesis pairs. Only blank spaces, if desired, may be written between the left and right parentheses. When a characteristic is designated variable in a <define signal statement> a range minor field must be included to state the lowest and highest values which will be assigned to that characteristic in subsequent statements.

6.0 PREAMBLE STATEMENTS

4. In the statement which references the definition, the <label> is inserted in place of the defined fields and is followed by the value(s) of any parenthetical variables from the <define signal statement>. These values may be either constant or DECLARE variables. Each value is written in the same sequence as defined, and preceded by a comma regardless of the location of commas in the <define signal statement>.
5. SIGNAL definitions may be referenced in any signal oriented statement. The SOURCE and LOAD definitions may only be referenced by source type verbs and the SENSOR definitions may only be referenced by measurement type verbs.
6. If a <define signal statement> references the <label> of a previous <define signal statement>, then any parentheses in the referenced <define signal statement> must be filled explicitly. They may not be filled by “daisy-chaining” through other parentheses in the calling <define signal statement>.

6.6.5 Notes and Examples

Each example is written with a corresponding procedural calling statement.

Example 1.

```
000011 DEFINE, 'RZ STIM', SIGNAL, AC SIGNAL USING 'AC-GEN',
          VOLTAGE ( ) RANGE 100 V TO 200 V,
          FREQ ( ) RANGE 1HZ TO 1000HZ,
          CNX HI J2-1 LO J2-2
.
.
111133 APPLY, 'RZ STIM', 110, 400 $
```

Note that the label is inserted in its appropriate place in the procedural statement. Definition 000011 included the noun so in step 111133, 'RZ STIM' is written following the verb.

Example 2.

```
000022 DEFINE, 'GRID', SIGNAL,
          VOLTAGE ( ) RANGE 0 V TO 10 V,
          CNX HI J2-3 LO J2-4 $
,
.
111144 APPLY, DC SIGNAL USING 'DC-GEN', AC-COMP MAX 5 MV, 'GRID', 5 $
.
.
111155 APPLY, AC SIGNAL USING 'AC-GEN', DC-OFFSET 20 V, 'GRID', 1.10 $
```

Step 000022 did not include the noun so in steps 111144 and 111155, it is possible to use any noun and provide statement characteristics in addition to VOLTAGE. The only restriction in this regard is that the completed statement, after substitutions, conforms to ATLAS format requirements.

6.0 PREAMBLE STATEMENTS

Example 3.

```

000033 DEFINE, 'PWR PINS', SIGNAL, CNX HI ( ) LO ( ) $
.
.

000099 APPLY, DC SIGNAL USING 'DC-GEN', VOLTAGE 500 V, AC-COMP 5 V,
      'PWR PINS', J1-1, J1-4 $

```

This example illustrates how a connector field can be defined by itself. Note that the definition can only be called in source-type statements. Also note that the pins are separated by commas in the APPLY statement because they are separate “values for variables,” even though a comma does not appear in the normal pin format.

Example 4.

```

112244 DEFINE, 'LIMITS', SIGNAL,
      FREQ MAX 410 HZ, VOLTAGE MAX 15 V,
      CURRENT MAX 1 MA, UUT-IMP 40+J50 OHM,
      CNX HI J1-2 LO J1-3 $
.
.

88 MEASURE, (FREQ), AC SIGNAL USING 'COUNTER', 'LIMITS' $
.
.

98 MEASURE, (VOLTAGE), AC SIGNAL USING 'MULTIMETER', 'LIMITS' $
.
.

99 MEASURE, (CURRENT), AC SIGNAL USING 'MULTIMETER', 'LIMITS' $

```

This example illustrates how a sensor function may be defined; and then in the procedural section, various <measured characteristic> fields can be supplied to make the function serve for several measuring operations.

6.0 PREAMBLE STATEMENTS

6.7 PROCEDURE Definition

6.7.1 <define procedure structure>

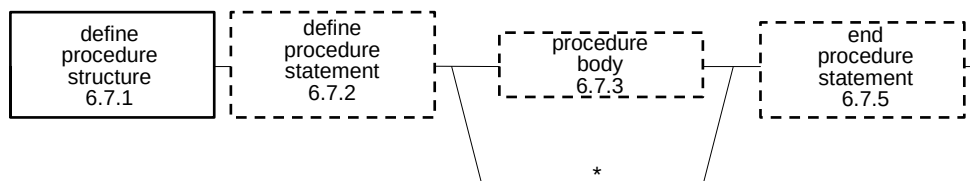
6.7.1.1 Function

To identify one or more complete ATLAS statements which may be subsequently invoked. The definition consists of three parts; the <define procedure statement>, the <procedure body> and finally the <end procedure statement>.

6.7.1.2 Formal Syntax

FSD / 299

6.7.1.3 Syntax Diagram



Note: * This bypass must be used if the <define procedure statement> has the EXTERNAL attribute. It must not be used otherwise.

6.7.1.4 Rules

1. The <define procedure structure> is employed to establish and label one or more complete ATLAS statements which are executed in the sequence defined whenever the procedure label is called by either a subsequent <perform statement> or invoked as the result of an escape following an <enable escape statement>.
2. The first statement of the structure is a <define procedure statement>. A sequence of ATLAS statements is written next as the <procedure body>. The last statement of the structure is an <end procedure statement>.
3. There is no limit to the number of statements which may be included in the <procedure body> of a <define procedure structure>.

6.7.1.5 Notes and Examples

```

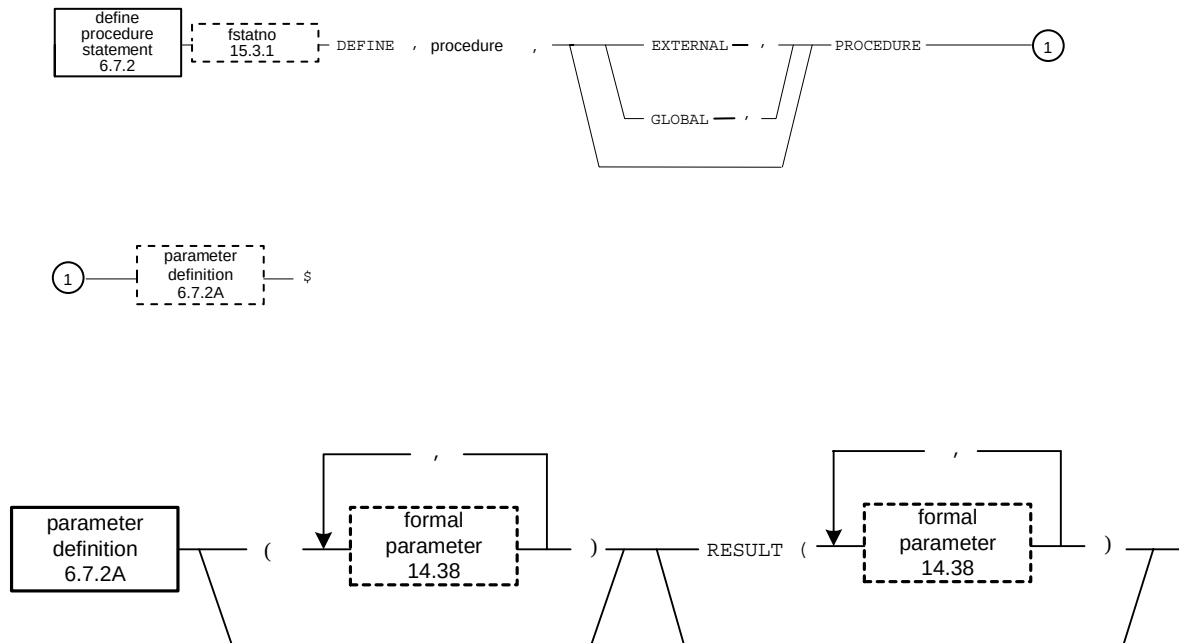
000421 DEFINE, 'NBD', PROCEDURE ....$
.
.   DECLARE, ... $
.
.   C   The following statement is part of the procedure body $
.       APPLY, ... $
.
.   END, 'NBD' $
  
```

6.0 PREAMBLE STATEMENTS**6.7.2 <define procedure statement>****6.7.2.1**

To identify the following <procedure body> and if necessary to assign <label>s known as formal parameters to data items, which are referenced within the following <procedure body>. The formal parameters are equated to actual parameters at the time of invocation. This vehicle for transferring data to and from the procedure provides the facility to change specified data items at each procedure invocation.

6.7.2.2 Formal Syntax

FSD / 308

6.7.2.3 Syntax Diagram**6.7.2.4 Rules**

1. Formal <parameter> names for a specific <define procedure statement> must be unique. However, the names may be the same as declared <label>s, formal <parameter>s of other procedures. But it is recommended that such naming conflicts be avoided.
2. The <parameter> names included in the formal <parameter> list are written in place of the corresponding data in the ATLAS statements within the <procedure body>.

A <label> specified in a <declare statement> in the <main preamble structure> and the specific <label> called MEASUREMENT may be referenced from within a <procedure body> without the need for it to be passed as a formal <parameter> name.
3. When the procedure is referenced by a <perform statement>, a value is passed to the RESULT <parameter>, thus making that value available for use in subsequent statements such as COMPARE or CALCULATE within the <procedure body>.

6.0 PREAMBLE STATEMENTS

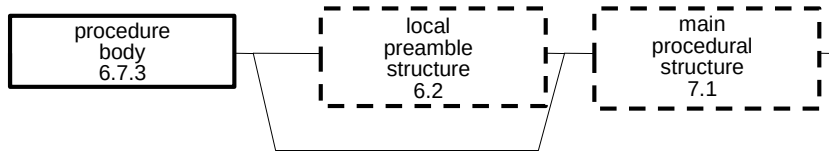
4. Any formal <parameter> appearing in the first parentheses should only be referenced in the <procedure body> as an independent variable since they are called by value. Any assignment to this class of parameters will not cause assignment of the equivalent actual parameter during procedure invocation.
5. Any formal <parameter> appearing in the second parentheses following RESULT may be referenced in the <procedure body> as either dependent or independent variables since they are called by name.
6. All formal <parameter>s are local. They are followed by their type.
7. A <procedure> can be made global using the GLOBAL attribute. In this case, the <procedure> has to be unique within the entire <atlas test specification structure>. A global <procedure> can be referenced in modules if it is established in the <atlas program structure> or in another module of the <atlas test specification structure>; it can be referenced in an <atlas program structure> if it is established in a module of the <atlas test specification structure>.
8. To reference a global <procedure>, the <define procedure statement> must be used with the attribute EXTERNAL. The sequence of parameters has to be the same as in the corresponding global definition. For example; the first <parameter> of the external definition corresponds to the first<parameter> of the global definition.

6.0 PREAMBLE STATEMENTS6.7.3 <procedure body>6.7.3.1 Function

To provide information which defines the test sequence to be performed when the structure containing the <procedure body> is executed.

6.7.3.2 Formal Syntax

FSD / 332

6.7.3.3 Syntax Diagram6.7.3.4 Rules

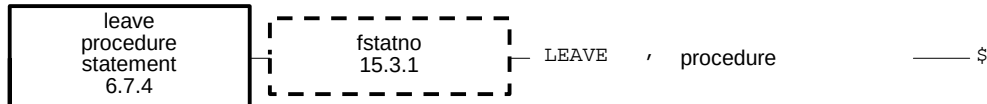
1. Statement numbers appearing within a <procedure body> may only be referenced from within the bounds of that procedure.
2. Within a single <atlas program structure> or <atlas module structure>, a previously defined procedure may be invoked in a subsequent procedure by including a <perform statement> within the <procedure body>. Recursion by the use of procedures in different <atlas module structure>s or the <atlas program structure> is not permitted. That is, an ATLAS procedure can never be nested within itself.
3. The order in which names are listed in the formal <parameter> list need not correspond to the order of appearance of those names in the series of statements being defined.
4. A <procedure body> may not contain entry points (E-Flags).
5. When the <define procedure statement> is used with the attribute EXTERNAL, the <procedure body> must be empty.

6.0 PREAMBLE STATEMENTS**6.7.4 <leave procedure statement>****6.7.4.1 Function**

To Cause the execution of a <procedure> to terminate prior to normal sequential execution of the <end procedure statement>.

6.7.4.2 Formal Syntax

FSD / 337

6.7.4.3 Syntax Diagram**6.7.4.4 Rules**

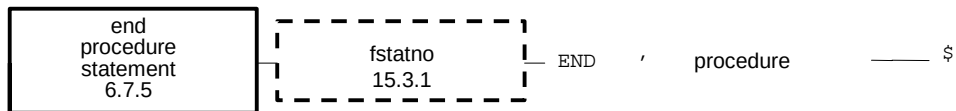
1. When a <leave procedure statement> is executed, control is transferred to the <end procedure statement>.
2. A <leave procedure statement> must be located within the applicable <define procedure structure>.
3. Any embedded structures within the <procedure body> and containing the <leave procedure statement> are also exited upon execution of the <leave procedure statement>.

6.7.5 <end procedure statement>**6.7.5.1 Function**

To delimit a procedure definition.

6.7.5.2 Formal Syntax

FSD / 342

6.7.5.3 Syntax Diagram**6.7.5.4 Rules**

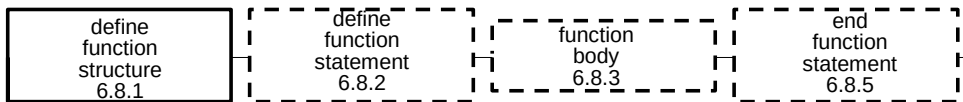
1. The procedure <label> following the END verb must be identical to that appearing within the <define procedure statement>.
2. Following an <end procedure statement>, program execution continues with the statement after the invocation of the procedure. If the procedure was invoked by a <perform statement>, execution continues at the statement following the <perform statement>. If the procedure was invoked as an escape that is enabled by an <enable escape statement>, execution continues at the statement following the point of activation of the escape.

6.0 PREAMBLE STATEMENTS**6.8 FUNCTION Definition****6.8.1 <define function structure>****6.8.1.1 Function**

To establish a series of ATLAS statements which are executed whenever the pre-defined mathematical function name is invoked.

6.8.1.2 Formal Syntax

FSD / 347

6.8.1.3 Syntax Diagram**6.8.1.4 Rules**

1. A defined <function> can, following its definition, be used anywhere mathematical or digital functions are permitted.
2. A defined <function> type STRING OF BIT can also, following its definition, be used in the <measured characteristic> field of the digital PROVE and SENSE verbs or compounds thereof.
3. The order of evaluation of function follows the order of evaluation of expressions of the same type.

6.0 PREAMBLE STATEMENTS

6.8.2 <define function statement>

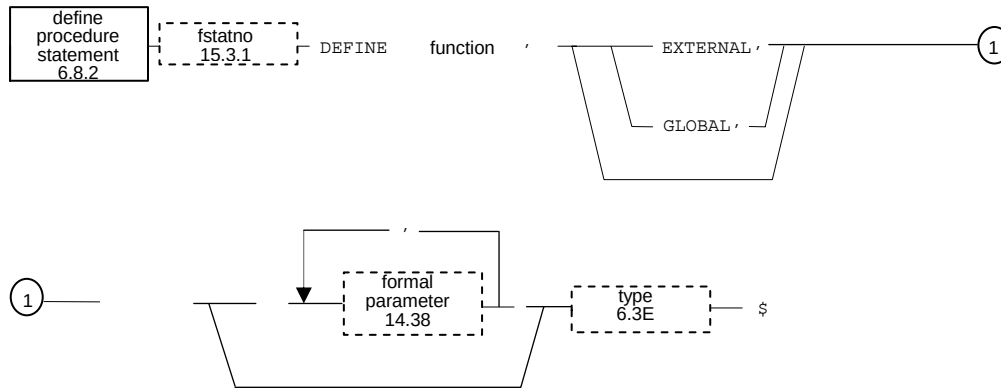
6.8.2.1 Function

To identify the following <function body> and to assign <label>s known as formal parameters to data items which are referenced within the <function body>. The result of the <function> is of the data type specified and is returned via the function name.

6.8.2.2 Formal Syntax

FSD / 353

6.8.2.3 Syntax Diagram



6.8.2.4 Rules

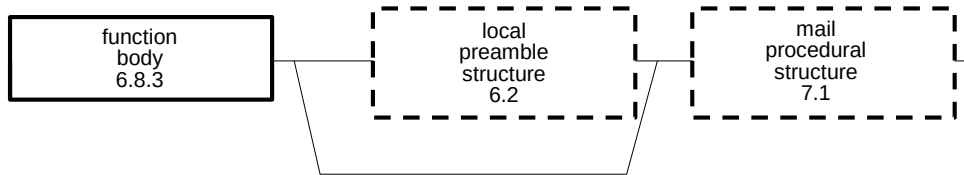
1. Formal <parameter> names for a specific <define function statement> must be unique. However, the names may be the same as explicitly declared <label>s or formal <parameter>s of other functions. But it is recommended that such naming conflicts be avoided.
2. The <parameter> names included in the formal <parameter> list are written in place of the corresponding data in the ATLAS statements within the <function body>.
3. All inputs to the <function> are of a call-by-value nature. Thus the only external result of invoking a <function> is the value it returns.
4. A <function> can be made global using the GLOBAL attribute. In this case, the <function> has to be unique within the entire <atlas test specification structure>. A global <function> can be referenced in modules if it is established in the <atlas program structure> or in another module of the <atlas test specification structure>; it can be referenced in an <atlas program structure> if it is established in a module of the <atlas test specification structure>.
5. To reference a global <function>, the <define function statement> must be used with the attribute EXTERNAL. The sequence of parameters has to be the same as in the corresponding global definition.

6.0 PREAMBLE STATEMENTS**6.8.3 <function body>****6.8.3.1 Function**

To provide information which defines the processing sequence that is to be executed to determine the value of the <function> when the <function> is invoked in an expression or digital statement.

6.8.3.2 Formal Syntax

FSD / 361

6.8.3.3 Syntax Diagram**6.8.3.4 Rules**

1. Statement numbers appearing within a <function body> may only be referenced from within the bounds of that function.
2. Only one exit from a <function> is possible.
3. At least one assignment (CALCULATE) to the <function> name is required in the <function body>. If no assignment is executed, the result of the function call is undefined.
4. Within a <function body> its name may not appear except as the destination of an assignment (CALCULATE).
5. When a defined <function> is used within the <measured characteristics> field of digital <sense statement>s or <prove statement>s the VALUE of the UUT response is passed to the first <parameter> of the <function>.
6. The types of the input <parameter>s are stated following the <parameter> names.
7. <function body> must not contain signal statements, timing statements, or digital statements.

6.8.3.5 Notes and Examples

C USER DEFINED FUNCTION EXAMPLE 1 \$

```

100000 BEGIN, ATLAS PROGRAM, 'EXAMPLE 1' $
110000 DEFINE, 'SIDE-A',
        FUNCTION('SIDE-B', 'SIDE-C', 'ANGLE-A' IS DECIMAL)
        DECIMAL $
    50 CALCULATE, 'SIDE-A' = SQRT('SIDE-B'**2+'SIDE-C'**2
        -2*'SIDE-B'*'SIDE-C'*COS('ANGLE-A')) $
110100 END, 'SIDE-A' $

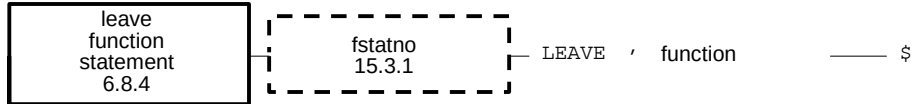
111111 COMMENCE, MAIN PROCEDURE $
222222 IF, 'SIDE-A' (5, 12, 90) EQ 13, THEN $
        OUTPUT, C'INCREDIBLY ACCURATE MATH' $
        ELSE $
        OUTPUT, C'NORMAL ACCURACY MATH' $
        END, IF $
333333 TERMINATE, ATLAS PROGRAM $
  
```

6.0 PREAMBLE STATEMENTS**6.8.4 <leave function statement>****6.8.4.1 Function**

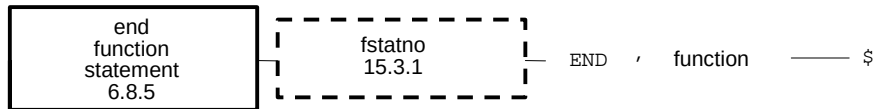
To cause the execution of a <function> to terminate prior to normal sequential execution of the <end function statement>.

6.8.4.2 Formal Syntax

FSD / 366

6.8.4.3 Syntax Diagram**6.8.4.4 Rules**

1. When a <leave function statement> is executed, control is transferred to the <end function statement>.
2. A <leave function statement> must be located within the applicable <define function structure>.
3. Any embedded structures within the <function body> and containing the <leave function statement> are also exited upon execution of a <leave function statement>.

**6.8.5 <end function statement>****6.8.5.1 Function**

To delimit a function definition.

6.8.5.2 Formal Syntax

FSD / 371

6.8.5.3 Syntax Diagram**6.8.5.4. Rules**

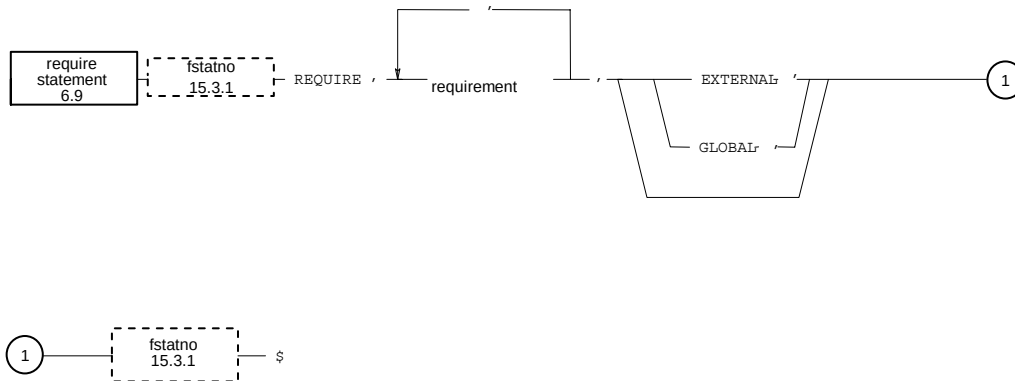
1. The function <label> following the END verb must be identical to that appearing within the <define function statement>.
2. Following the execution of the <end function statement> control is transferred back to the function invocation with the result value returned.

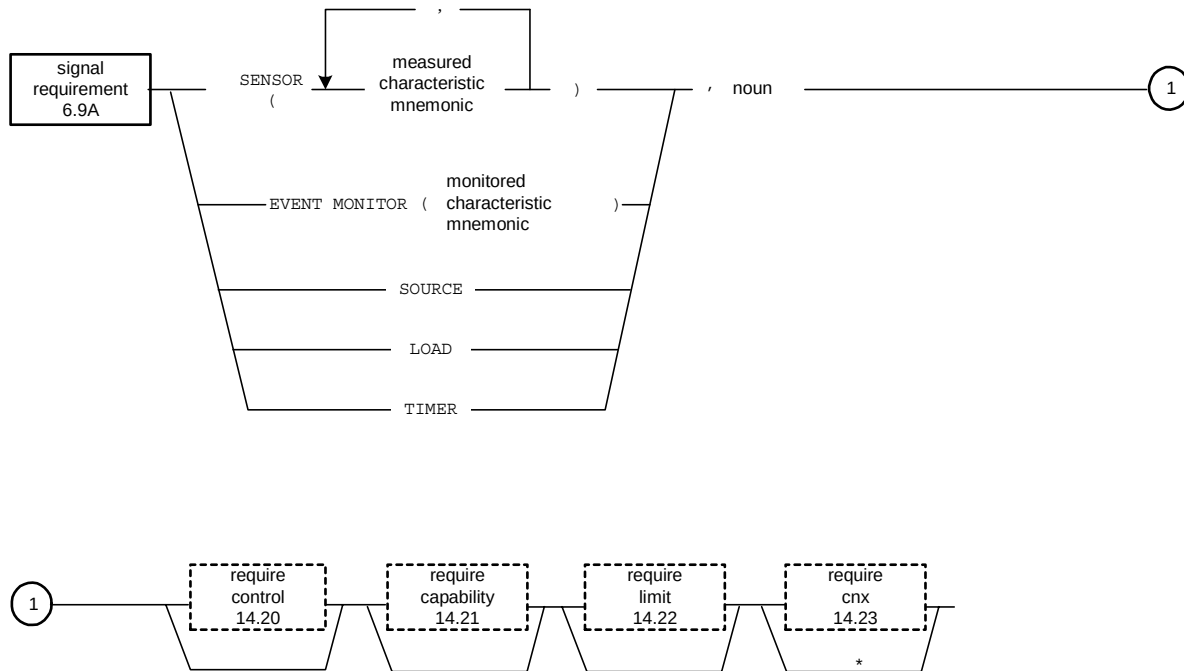
6.0 PREAMBLE STATEMENTS**6.9 <require statement>****6.9.1 Function**

To allow a test procedure writer to establish and label a Unit-Under Test oriented summary of the characteristics needed by a test resource to implement the <atlas test specification>. The test resources thus established are termed “virtual resources” since they cite Unit Under Test requirements and not the “real resources” in the hardware used for implementation. The implementation of the <atlas test specification> requires a checking and equating of the “virtual resources” to the “real resources” available for implementation. The <requirement> label serves as a linking mechanism which may be utilized in this process. The checking and equating of “virtual resources” to “real resources” is a complex implementation problem which is outside the scope of the <atlas test specification structure>.

6.9.2 Formal Syntax

FSD / 376

6.9.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS

Note: * This bypass must be used with nouns that do not require a CNX field, e.g., TIME INTERVAL.

6.9.4 Rules**6.9.4.1 Rules Applicable to All REQUIRE Statements**

1. The <require statement> for the test resources must contain only information that is condensed from those ATLAS signal-oriented statements which reference the <requirement> label following the word USING.
2. The contents of each <require statement> must be a complete summary of the union of all of the virtual resource specifications contained in each of the individual statements that reference the <requirement> label established in the <require statement>.
3. Each <require statement> can only establish the test requirements for the ATLAS noun.
4. Test requirements which occur simultaneously, but which cannot be fulfilled from a single virtual resource, must reference a separate <requirement> label.
5. A specific modifier <mnemonic>, with appropriate prefixes and suffixes, may appear only once in any given <require control>, <require capability>, or <require limit> field. The modifier cannot be used in a combination of the <require control> and <require capability> fields. It may be used in combinations of the <require control> and <require limit> or <require capability> and <require limit> fields.
6. The <require control> field specifies the value or range of values for those test characteristics which must be controlled by the test resource to execute the <atlas test specification structure>.
7. The <require limit> field specifies the range of values for those test characteristics which must be constrained by some discrete action of the resource to execute the <atlas test specification structure>.

6.0 PREAMBLE STATEMENTS

8. The <require capability> field specifies the required parameter range for those test characteristics which relate to the capacity of the test resource to meet a test requirement, but which are not required to be either controlled or limited to execute the <atlas test specification structure>.
9. An appropriate <mnemonic> must be included in the <require capability> field of SENSOR type resources to specify ranging information of the resources.
10. The accuracy of a SENSOR type resource, where applicable, is written in the ERRLMT subfield of the <require capability> field.
11. The <require cnx> field lists the pin descriptors associated with the connection points of the resource associated for the <requirement>.
12. A <requirement> label can be made global for the entire <atlas test specification structure> by using the GLOBAL attribute. The statement REQUIRE GLOBAL may only appear in the preamble of an <atlas program structure>. The statement REQUIRE EXTERNAL may only appear in the preamble of an <atlas module structure>. When a <require statement> is declared as EXTERNAL, it must have a corresponding GLOBAL <require statement>. The GLOBAL <requirement> must encompass the characteristics of all the EXTERNAL <requirement>s with the same label.
13. Modifier Descriptor type modifiers, when used in the <require capability> field, may have only one <modifier descriptor> associated with it.
14. Multiple <requirement> labels can be used in a <require statement> only when all the labeled virtual resources have identical characteristics.
15. MAX is the greatest positive numerical value.
MIN is the greatest negative numerical value.

6.9.4.2 Rules Applicable to SOURCE and LOAD Resource Types

1. The <require control> field specifies those test characteristics which must be established or directly controlled by the test resource.
2. The <require capability> field specifies those intrinsic test characteristics which are not required to be either directly controlled or limited by the test resource.
3. The optional BY branch is not used in the <require capability> field for SOURCE or LOAD resource types.
4. The <require limit> field specifies those test characteristics which must be constrained (limited) by a discrete action of the test resource to provide a valid test or to safeguard the UUT from malfunction of either itself or the test resource.
5. The <require cnx> field specifies the pin descriptors for the connection points to the test resource.

6.9.4.3 Rules Applicable to SENSOR and EVENT MONITOR Resource Types

1. The <require control> field specifies those test characteristics which must be directly controlled by the test resource.
2. The <require capability> field specifies characteristics to be evaluated by the sensor function and any other intrinsic characteristics which are not required to be either controlled or limited by the test resource.

6.0 PREAMBLE STATEMENTS

3. The <require limit> field specifies those test characteristics which must be constrained (limited) by a discrete action of the test resource to provide a valid test or to safeguard the UUT from malfunction of either itself or the test resource.
4. The <require cnx> field specifies the pin descriptors for the connection points to the test resource.
5. A <measured characteristic mnemonic> subfield is included for SENSOR type resources to specify the characteristic to be evaluated by the sensor function.
6. An appropriate <mnemonic> must be included in the <require capability> field of SENSOR type resources to specify ranging information of the resource.
7. The accuracy of a SENSOR type resource, where applicable, is written in the ERRLMT subfield of the <require capability> field.
8. A <monitored characteristic> subfield is included for EVENT MONITOR type resources to specify the characteristics of the signal to be monitored by that resource.
9. The use of multiple measured characteristics specifies the requirement for a virtual sensor which simultaneously obtains the value of all of these characteristics from a data set captured during a single measurement. The subsequent <require capability> field must specify the ranges associated with each measured characteristic.

6.9.5 Notes and Examples

See Sections 14.20 thru 14.23 for further definitions of the subfields of a <require statement>.

6.9.5.1 SOURCE Examples

Given that the unit being tested non-concurrently requires a DC source having values of 1 volt at 10 amps max, 5 volts at 2 amps max, and 9 volts at 0.1 amps max. It is also required that the current not be allowed to exceed 12 amps at any time.

```
000100 REQUIRE, 'DC SUPPLY', SOURCE, DC SIGNAL,
        CONTROL,
            VOLTAGE RANGE 1 V TO 9 V BY 4 V,
        CAPABILITY,
            CURRENT MAX 10 A,
        LIMIT,
            CURRENT-LMT MAX 12 A,
        CNX HI LO $
010100 APPLY, DC SIGNAL USING 'DC SUPPLY',
        VOLTAGE 5V,
        CURRENT MAX 10 A,
        CURRENT-LMT MAX 12 A,
        CNX HI P-100 LO P-101 $
```

Given that the unit being tested requires a ramp signal in the frequency range of 75 hertz to 150 hertz with a resolution of 5 hertz and an accuracy of 2 hertz. The amplitude of the signal must be continuously adjustable over the range of 7.3 to 7.7 volts peak to peak.

```
000200 REQUIRE, 'RAMP', SOURCE, RAMP SIGNAL,
        CONTROL,
            FREQ RANGE 75 HZ TO 150 HZ BY 5 HZ ERRLMT +-2 HZ,
            VOLTAGE-PP RANGE 7.3 V TO 7.7 V CONTINUOUS,
        CNX HI LO $
```

6.0 PREAMBLE STATEMENTS

This requirement's usage in the procedure of the program could appear as follows:

```
050000 APPLY, RAMP SIGNAL USING 'RAMP',
      FREQ 105 HZ ERRLMT +-2 HZ, VOLTAGE-PP 7.5V,
      CNX HI J1-7 LO J3-5 $
```

Given that the unit being tested requires a pulsed DC input with a PRF of either 2000 or 6000 PPS (never both) depending on where it is used. The rise time will be specified in the range of 50 to 55 usec in 1 usec steps. The maximum peak voltage must not exceed 7.5 volts peak.

```
000300 REQUIRE, 'DC PULSE', SOURCE, PULSED DC,
      CONTROL,
      PRF 2000 HZ 6000 HZ
      RISE-TIME RANGE 50 USEC TO 55 USEC BY 1 USEC,
      LIMIT,
      VOLTAGE-P-LMT MAX 7.5V,
      CNX HI LO $
```

6.9.5.2 SENSOR Examples

Given that the procedure for the unit being tested requires a sensor that can measure a DC voltage in the range of 15 to 25 volts with an accuracy of 100 millivolts.

```
000400 REQUIRE, 'DMM', SENSOR (VOLTAGE), DC SIGNAL,
      CAPABILITY,
      VOLTAGE RANGE 15 V to 25 V ERRLMT +-0.1 V,
      CNX HI LO $
```

This requirement's usage in the procedure of the program could appear as follows:

```
055000 MEASURE, (VOLTAGE ERRLMT +-0.1 V), DC SIGNAL USING 'DMM',
      VOLTAGE RANGE 15 V to 20 V, CNX HI J5-9 LO J1-E $
```

Given that the procedure for the unit being tested requires a sensor that can measure the frequency of an AC signal in the voltage range of 4 to 7 volts AC riding on a maximum DC level of 75 volts. The frequency will be in the range of 0 HZ to 1000 HZ and needs to be measured with an accuracy of 2 HZ.

```
000500 REQUIRE, 'COUNTER', SENSOR (FREQ), AC SIGNAL,
      CAPABILITY,
      FREQ RANGE 0 HZ TO 1000 HZ ERRLMT +-2 HZ,
      VOLTAGE RANGE 4 V TO 7 V,
      DC-OFFSET RANGE 0 V TO 75 V,
      CNX HI LO $
```

6.9.5.3 Notes

Note the different meaning of MAX and MIN in CAPABILITY and LIMIT fields. Thus, the field CAPABILITY, CURRENT MAX 100 MA means the resource must be capable of supplying 100 MA, whereas the field LIMIT, CURRENT-LMT MAX 100 MA means that the resource must ensure that the current does not exceed 100 MA in any circumstance.

6.0 PREAMBLE STATEMENTS

6.10 <include statement>

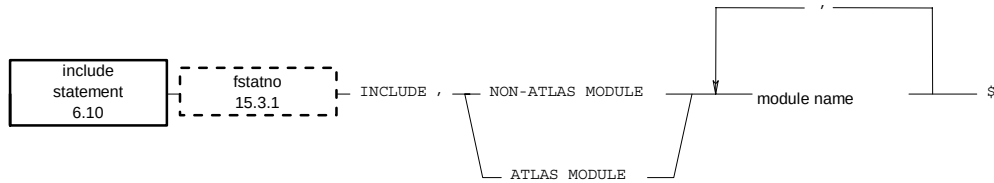
6.10.1 Function

To reference the <module name> of the modules that are part of the <atlas test specification structure>. Thus, a complete summary of modules used in an <atlas test specification structure> is established in the <program preamble structure>.

6.10.2 Formal Syntax

FSD / 412

6.10.3 Syntax Diagram



6.10.4 Rules

1. An <include statement> may only be written in the <program preamble structure>.
2. Any module that is part of an <atlas test specification structure> must be referenced by an INCLUDE statement in the associated <program preamble structure>.

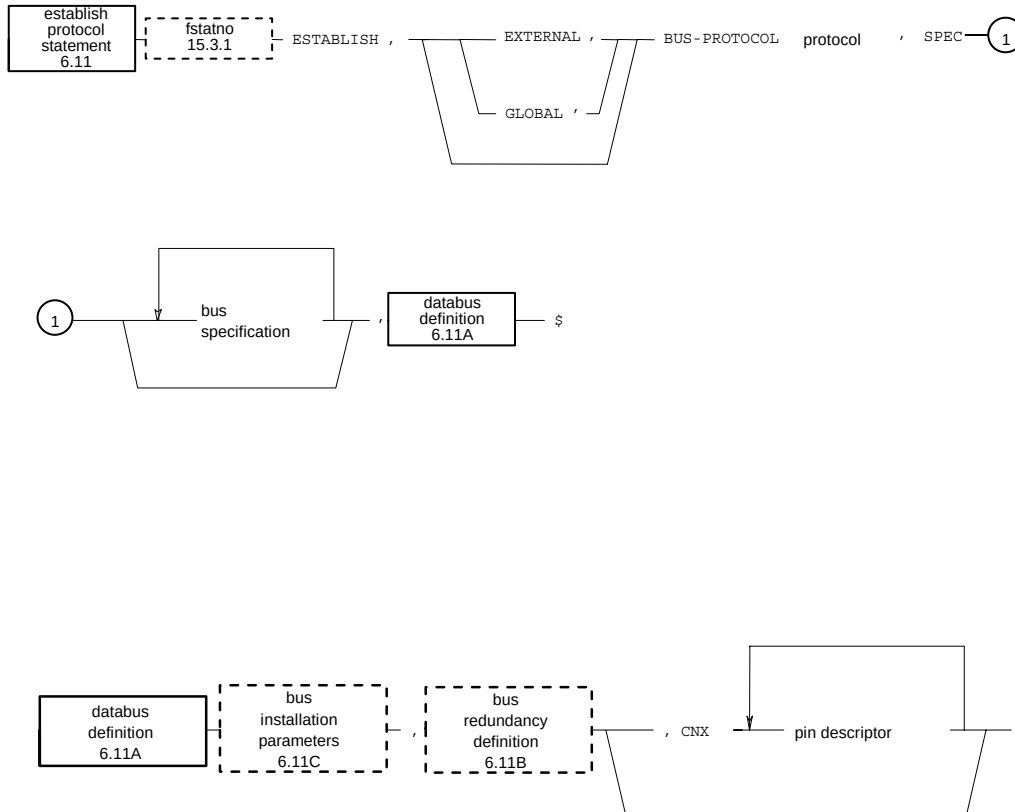
6.0 PREAMBLE STATEMENTS**6.11 <establish protocol statement>****6.11.1 Function**

To specify a bus protocol, a format for all the fields which can be encountered later in a program, new vocabulary used in statements which reference the <establish protocol statement> and lists the pin descriptors associated with the connection points of the resource “for the <exchange configuration>.”

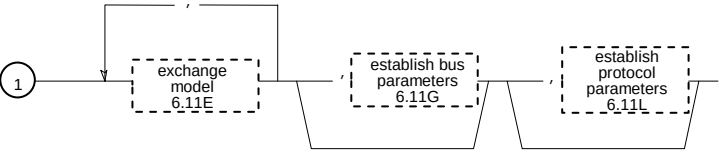
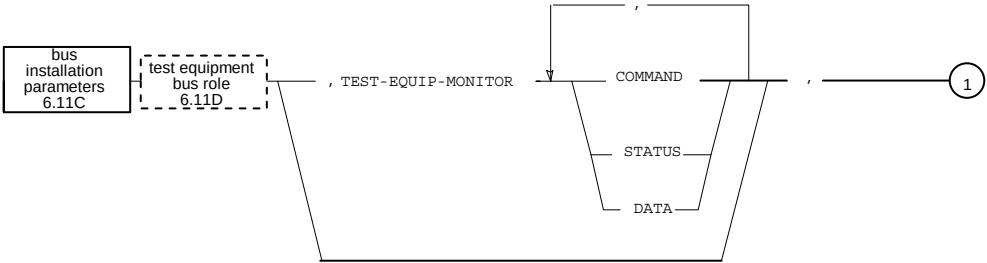
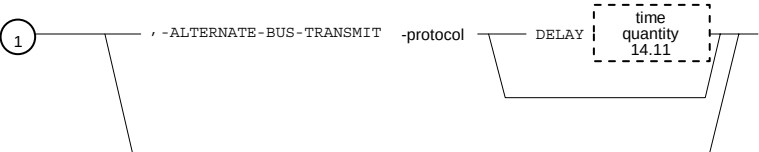
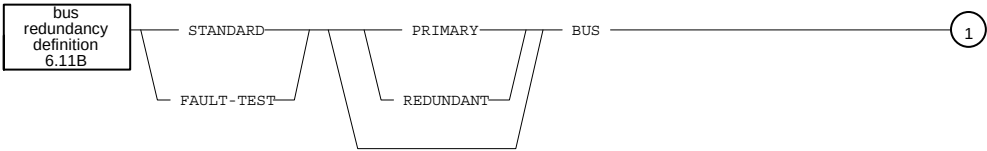
Whenever an <exchange> in a <do exchange statement> references the <protocol> label defined in the <establish protocol statement>, the execution of that <exchange> initiates the specified protocol.

6.11.2 Formal Syntax

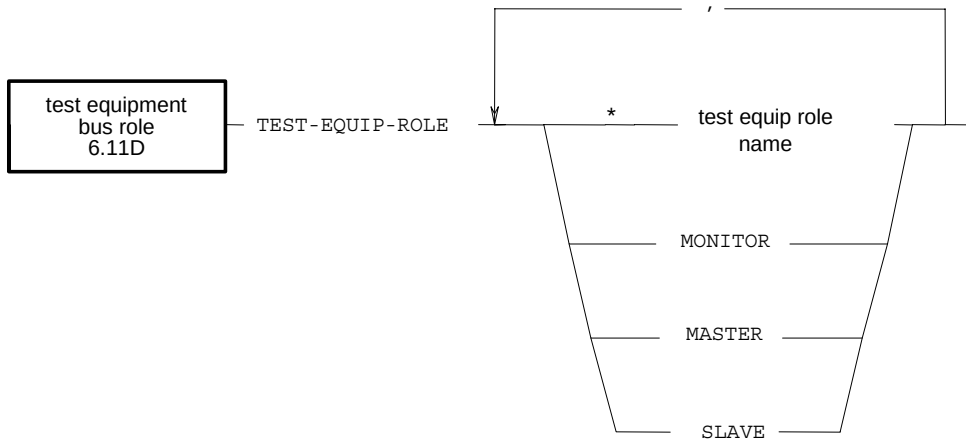
FSD / 426

6.11.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS

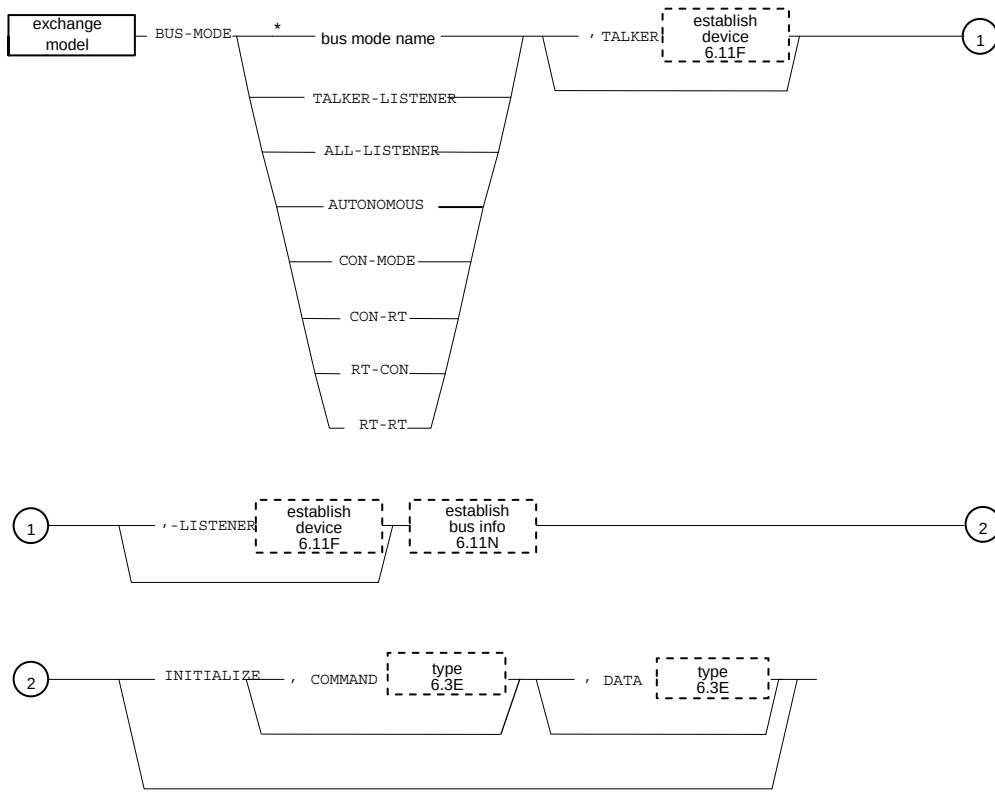


6.0 PREAMBLE STATEMENTS



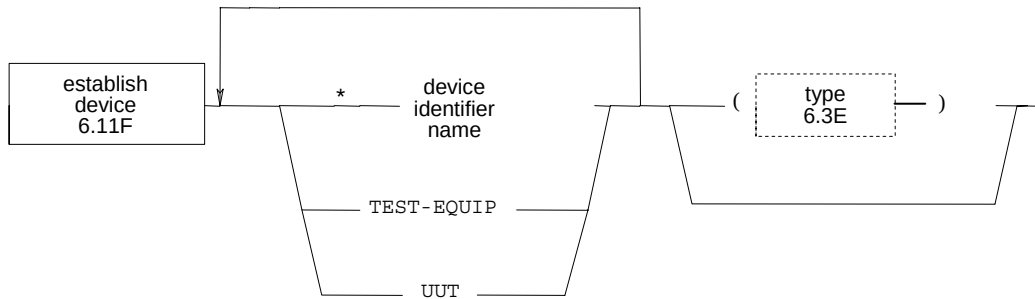
Note : * The name must be one that has been introduced by an EXTEND statement of the associated type.

6.0 PREAMBLE STATEMENTS

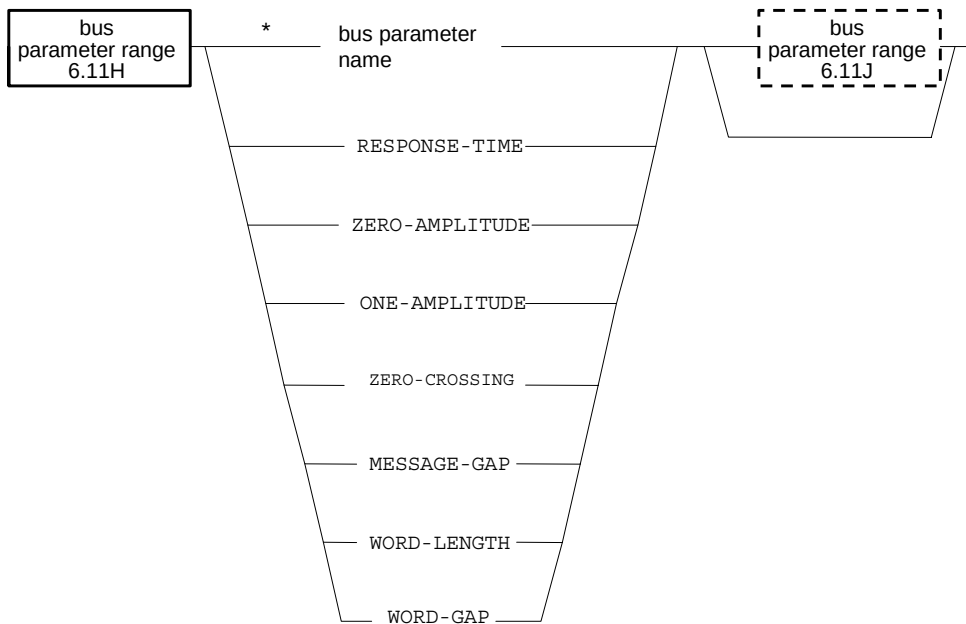
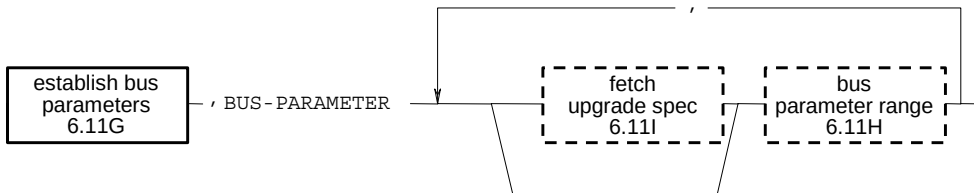


Note : * The name must be one that has been introduced by an EXTEND statement of the associated type.

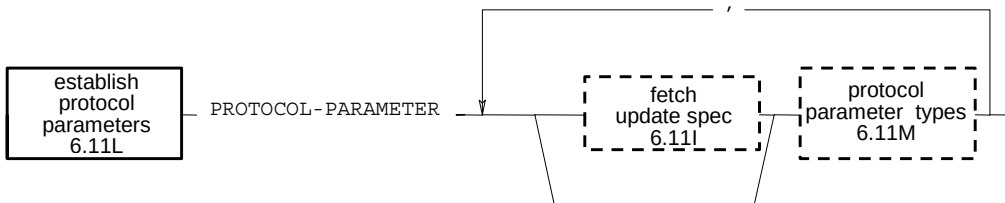
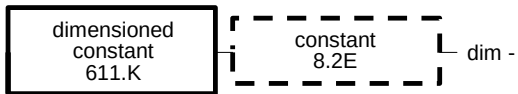
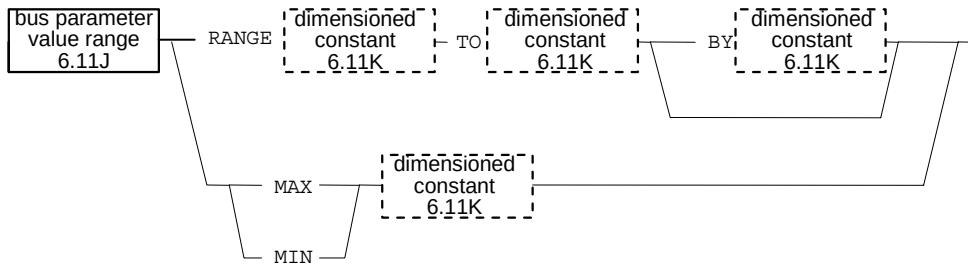
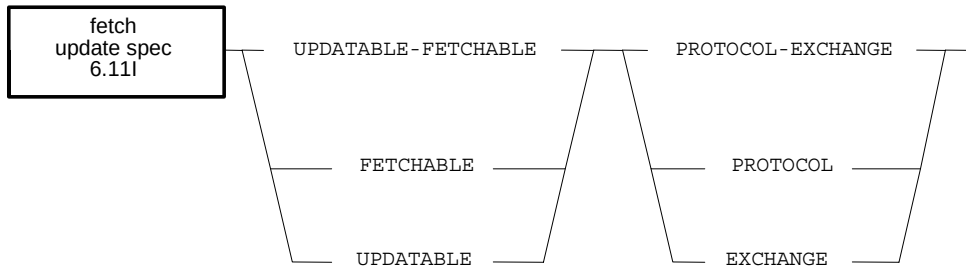
6.0 PREAMBLE STATEMENTS

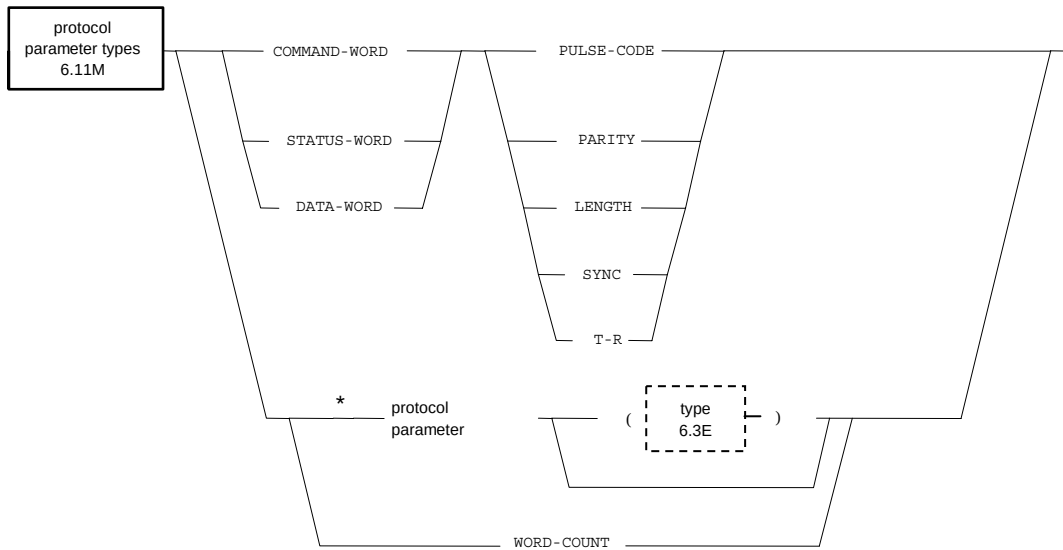


Note: * The name must be one that has been introduced by an enumeration declaration.

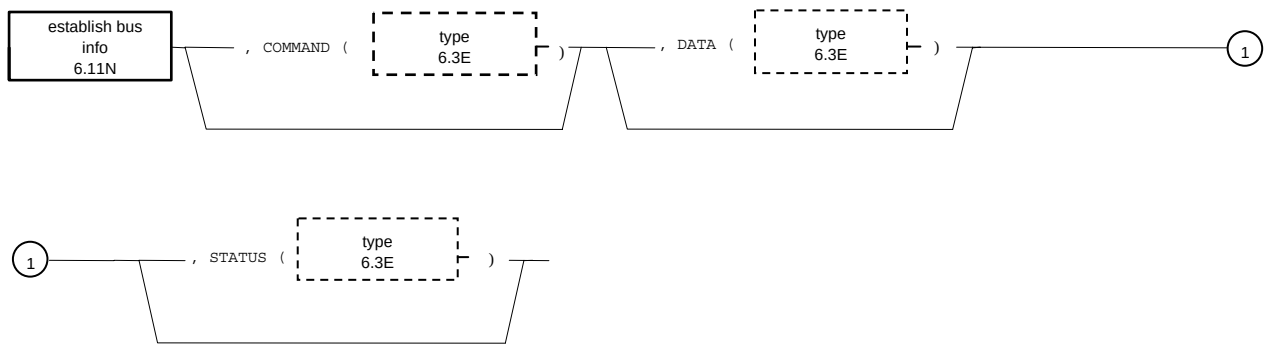


NOTE : * The name must be one that has been introduced by an EXTEND statement of the associated type.

6.0 PREAMBLE STATEMENTS

6.0 PREAMBLE STATEMENTS

Note: * The name must be one that has been introduced by an EXTEND statement of the associated type.

**6.11.4 Rules****6.11.4.1 Rules Applicable to All <establish protocol statement>s**

1. A databus is a line or group of lines used to transfer <exchange>s. A databus may include both data and control lines.
2. Use of the GLOBAL attribute makes the <protocol> global. The GLOBAL attribute may only appear in the preamble of an <atlas program structure>. The statement ESTABLISH, EXTERNAL may only appear in the preamble of an <atlas module structure>. An ESTABLISH, GLOBAL statement must define each label defined by an ESTABLISH, EXTERNAL statement. The global <establish statement> must encompass the requirements of the ESTABLISH, EXTERNAL statement.

6.0 PREAMBLE STATEMENTS

3. <external bus specification> is a text string that fully identifies a specification in which the databus operations in the context are fully , or partially defined. Where one exists such a referenced specification must be a standard produced by an organization that issues standards for that discipline. If no such comprehensive specification exists, the set of referenced specifications may include one or more produced locally. Any referenced external specification is a part of the test documentation and must be available to all users of the ATLAS test program.

The <external bus specification> text string may include any characters except the dollar sign and apostrophe. Apostrophes must always enclose the text string.

4. <bus installation parameters> establish the only keywords which are permissible entries for the associated fields of subsequent statements required for the databus tests. The entries may be standardized keywords associated with the field, or they may be <enumeration element>s from a <define statement> which defines the set of <device identifier name>s or they may be new keywords introduced by an <extend statement>. Every keyword included in a field in the <bus installation parameters> must be fully defined in the context of the particular databus by a specification referenced in the SPEC field.
5. When the keyword STANDARD is used for a bus protocol, the options specified in any <define exchange statement> that references the <protocol> must be within the parameter limits for that bus as specified in the referenced <external bus specification>.
6. When the keyword FAULT-TEST is used for a bus protocol, the options specified in any <define exchange statement> that references the <protocol> are not subjected to the same restriction and may lie outside the specified standard parameter limits for the databus.

7. <exchange model>s together with BUS-PARAMETERs and PROTOCOL-PARAMETERs establish the vocabulary associated with the particular bus installation. Each occurrence of <exchange model> defines one consistent set of parameters on the databus. When the protocol> is referenced, all the associated parameters must match one of the <exchange model>s. The set of <exchange model>s must define all the vocabulary associated with the particular installation.

It declares the number and type of parameter for each field used in the statements which reference the <protocol>.

8. The REDUNDANT bus mode may only be used when there are two separate <establish protocol statement>s using different protocol labels and establishing two inter-related but independent buses connected to the UUT.
9. The second of the <establish protocol statement>s must include the keyword PRIMARY and must contain an ALTERNATE-BUS-TRANSMIT field which references the previous <establish protocol statement> containing REDUNDANT.

The referenced <establish protocol statement> must include the keyword REDUNDANT and no ALTERNATE-BUS-TRANSMIT field.

The optional DELAY field determines a fixed time interval between the PRIMARY and REDUNDANT transmissions.

10. If <test-equip-role name> is used, it must have been defined in an earlier <extend statement>.
11. The bypass of the CNX field may only occur if the specifications referenced in the statement establish permanent, unswitchable connections for the databus.

6.0 PREAMBLE STATEMENTS**6.11.4.2 Rules Applicable to the <exchange model> Fields**

1. The various fields define the permitted entries in the corresponding fields of subsequent <define exchange statement>s, <enable exchange configuration statement>s, <do exchange statement>s, <update exchange statement>s and <fetch exchange statement>s.
2. In the TALKER and LISTENER fields, TEST-EQUIP indicates that the TEST-EQUIP is required to be an operational device on the databus during some of the testing of the UUT.

UUT indicates that the UUT is required to be an operational device on the databus and is not referenced by a <device identifier name>.

3. In the TALKER and LISTENER fields, the optional <type> in parentheses indicates the type of any index that is required to fully specify the TALKER or LISTENER. The valid types are restricted to: STRING-OF-BIT, STRING-OF-CHAR, DECIMAL, INTEGER and enumeration types. In addition to the above simple types, the type can be specified as a RECORD. This RECORD can contain any combination of the above simple types except enumeration types but must not contain any RECORD or ARRAY types. This field is used where a UUT has several channels or other sub-addresses. If this subfield is omitted the calling statement must not attempt to use any index for the device. One use of a RECORD is to achieve sub-addressing by successive record fields.
4. All <device identifier name>s used in a single ESTABLISH statement must be enumeration elements from a single declared enumeration type.
5. If <bus mode name> is used, it must have been defined in an earlier <extend statement>.
6. The <type> definitions specified in the COMMAND, DATA and STATUS fields provide a template for the specification of constants and variables to represent databus words in subsequent <define exchange statement>s, <update exchange statement>s and <fetch exchange statement>s.

The valid types are restricted to: BIT, STRING-OF-BIT, CHAR, STRING-OF-CHAR, DECIMAL, INTEGER, and enumeration types. In addition to the above simple types. the type can be specified as a RECORD. This RECORD can contain any combination of the above simple types except enumeration types but must not contain any RECORD or multi-dimensional ARRAY types. A RECORD permits both field structured information and multiple words to be transmitted in a single <exchange>.

7. BUS-MODE field - the BUS-MODE field identifies the mode of bus operation for the exchange. It is used to indicate which of the possible operating modes of the bus is to be used. It may be one of the default entries listed below or a <bus mode name> label. Whichever is employed must be fully defined as it applies to that bus in the set of <external bus specification>s referenced in the <establish protocol statement> and must be listed in the <exchange model> section of this <establish protocol statement>. The standard bus modes are:

CON-RT	Bus controller to remote terminal transfers. The controller is talking and one device is listening. The TALKER field is not used
RT-CON	Remote terminal to bus controller transfers. One device is talking and the controller is listening. The LISTENER field is not used.
RT-RT	Remote terminal to remote terminal transfers. One device is talking and another device is listening.
CON-MODE	Transfer from bus controller to remote terminal(s). The controller is talking to all devices. The TALKER field is not used.
TALKER-LISTENER	The talker and listener(s) are explicitly designated.

6.0 PREAMBLE STATEMENTS

ALL-LISTENER One talker is explicitly designated and all other devices are listening. The LISTENER field is not used.

AUTONOMOUS No central bus controller.

8. The COMMAND and DATA fields specified in the INITIALIZE subfield provide a template for the specification of constants and variables that represent individual test resource characteristics.

The valid types are restricted to: BIT, STRING-OF-BIT, CHAR, STRING-OF-CHAR, DECIMAL, INTEGER, and enumeration types. In addition to the above simple types, the type can be specified as a RECORD. This RECORD can contain any combination of the above simple types except enumeration types but must not contain any RECORD or multi-dimensional ARRAY types. A RECORD permits both field structured information and multiple words to be transmitted in a single <exchange>.

6.11.4.3 General Rules Applicable to <bus parameters> and <protocol parameters>

1. The presence of FETCHABLE indicates that the following bus or protocol parameter may be read and stored for subsequent analysis.

The presence of UPDATABLE indicates that the following bus parameter may be updated during testing without inhibiting continuing tests. The use of FETCHABLE-UPDATABLE means that the following bus or protocol parameter can be either updated or read at will. The associated bus or protocol parameter must be typed.

2. The PROTOCOL option is used when the following bus or protocol parameter may only be updated or fetched globally for all exchanges using the protocol.
3. The EXCHANGE option is used when the following bus or protocol parameter may be updated or fetched for a specific exchange.
4. The PROTOCOL-EXCHANGE option is used when the following bus or protocol parameter may be updated or fetched either globally for a protocol or specifically for an exchange.

6.11.4.3.1 Rules Applicable to <bus parameter>

1. If the <bus parameter name> is used, it must be defined and typed in an earlier <extend atlas statement>.
2. The UPDATABLE option means that the following bus parameter may be updated using the <bus parameter> field in an <enable exchange configuration statement>, a <do exchange statement>, or an <update exchange statement>. If a value is required to fully specify a bus parameter it must be followed by a <bus parameter range> field which defines the type for subsequent <bus parameter> usage unless it is of STRING-OF-BIT or STRING-OF-CHAR type.
3. The <bus parameter range> defines the extreme values for the associated parameter which are required to be set in the <atlas program structure> or <atlas module structure>. In the case of an <atlas program structure> the <bus parameter range> must encompass the complete <bus parameter range> for every <atlas module structure> associated with the <atlas test specification structure>. The significance of the standardized BUS-PARAMETER keywords is defined in Section 14.41. The bypass of the <bus parameter range> can only be used where the type of the bus parameter does not facilitate ranging e.g. STRING-OF-BIT and STRING-OF-CHAR.
4. The optional <dim> following the bus parameter must be compatible with the bus parameter as established by the standard parameter or by the defining <extend statement>.
5. If <dim> is used anywhere in the <bus parameter range> then all the optional <dim> fields must contain the same dimension.

6.0 PREAMBLE STATEMENTS

6. The FETCHABLE option means that the value of the following bus parameter may be read using the TEST-EQUIP-MONITOR field in a <do exchange statement>, or a <fetch exchange statement>. The type of any variable specified in subsequent statements to receive the value must be of the type defined in the associated <extend atlas statement>.

6.11.4.3.2 Rules Applicable to <protocol parameters>

1. If the <protocol-parameter name> is used, it must have been defined in an earlier <extend statement>.
2. The UPDATABLE option means that the following protocol parameter may be updated using the <update protocol parameter> field in an <enable exchange configuration statement>, <do exchange statement> or <update exchange statement>. If a value is required to be specified for a parameter with a <protocol parameter name> in subsequent statements it must be followed by a <type> definition. If the protocol parameter is not followed by a <type> its value will be a standard value defined in the specification referenced in the SPEC field. The value can still be changed to an invalid one by using the INSERT-INVALID option in the <update protocol parameter> field. In this case the <update protocol parameter> field must not contain any constant or data store specification and the exact meaning of the Invalid parameter must be defined in the referenced external SPECS.
3. For the predefined PROTOCOL-PARAMETER keywords the FETCHABLE option means that a count of occurrences of errors identified by the protocol parameter may be read using the TEST-EQUIP-MONITOR field in a <do exchange statement>, or a <fetch exchange statement>. All FAULT-COUNTs are of integer type.

If a <protocol parameter name> is specified it may be followed by a <type> in which case it is possible to read the value of the specified parameter in subsequent <do exchange statement>s and <fetch exchange statement>s. If it is not followed by a <type> its value cannot be read but a count of the faults on this parameter can still be read using the FAULT-COUNT branch after the <protocol parameter name> in the appropriate <fetch protocol parameter> field.

4. The significance of the standardized PROTOCOL-PARAMETER keywords is defined in Section 14.42.
5. The COMMAND-WORD, DATA-WORD and STATUS-WORD keywords indicate the ability to modify a characteristic of one or all of the relevant words in a transmission package in a manner defined by the subsequent keyword.
6. The PULSE-CODE keyword indicates that the pulse code associated with a particular BIT of the word is altered. The calling statement includes the keyword BIT and an <unsigned integer number> to indicate which BIT is to be altered in a particular test. The exact specification of the PULSE-CODE and any permitted alteration must be fully defined in the context of the particular databus by a specification referenced in the SPEC field.

6.0 PREAMBLE STATEMENTS

6.11.5 Notes and Examples

Tests employing databuses may require the use of twelve types of statement. ESTABLISH, DEFINE EXCHANGE CONFIGURATION, DEFINE EXCHANGE, ENABLE and DO statements are compulsory and EXTEND, DECLARE, CONNECT, DISCONNECT, UPDATE, FETCH and DISABLE are optional. It is essential to understand the relationship between these statements to appreciate fully the role of any individual statement. This note describes the theoretical role of each:

<extend statement> - This preamble statement establishes the names of non-standardized characteristics of the set of databuses associated with the <atlas test specification structure>.

<declare statement> - In the context of databus testing, this preamble statement declares an enumeration data type, the enumeration elements of which include all the <device identifier name>s associated with the databus installation.

<establish statement> - This preamble statement labels a databus and references the specifications which define the databus. It defines the vocabulary and parameters used with this protocol.

<define exchange configuration statement> - This preamble statement associates one or several databuses which have to work together.

<define exchange statement> - This preamble statement defines and labels a single operation on the databus. It defines the bus mode, the devices which are operational on the bus and the command, data or status information associated with the exchange.

<enable exchange configuration statement> - This procedural statement initiates parameters which are common to all the exchanges. It connects a bus to a particular UUT pin set.

<do exchange statement> - This procedural statement defines a test situation on a databus. It defines which sequences of exchanges are executed on the databus and for each exchange the role of the ATE.

<connect exchange configuration statement> - This procedural statement connects a bus or a particular UUT pin set.

<disconnect exchange configuration statement> - This procedural statement disconnects a bus from a particular pin set.

<update exchange statement> - This procedural statement changes data, status or command information of one of the defined exchanges initiated by the preceding <do exchange statement> without interrupting the databus operation. A variant of the statement changes the default bus or protocol parameters for all exchanges of a protocol without interrupting the databus operation.

<fetch exchange statement> - This procedural statement reads command, data and status information or the value of any bus or protocol parameters for one of the defined exchanges initiated by the preceding <do exchange statement> without interrupting the databus operation.

A variant of the statement reads globally (for a protocol) applicable bus or protocol parameters without interrupting the databus operation.

<disable exchange configuration statement> - This procedural statement renders inactive the set of protocols and disconnects buses.

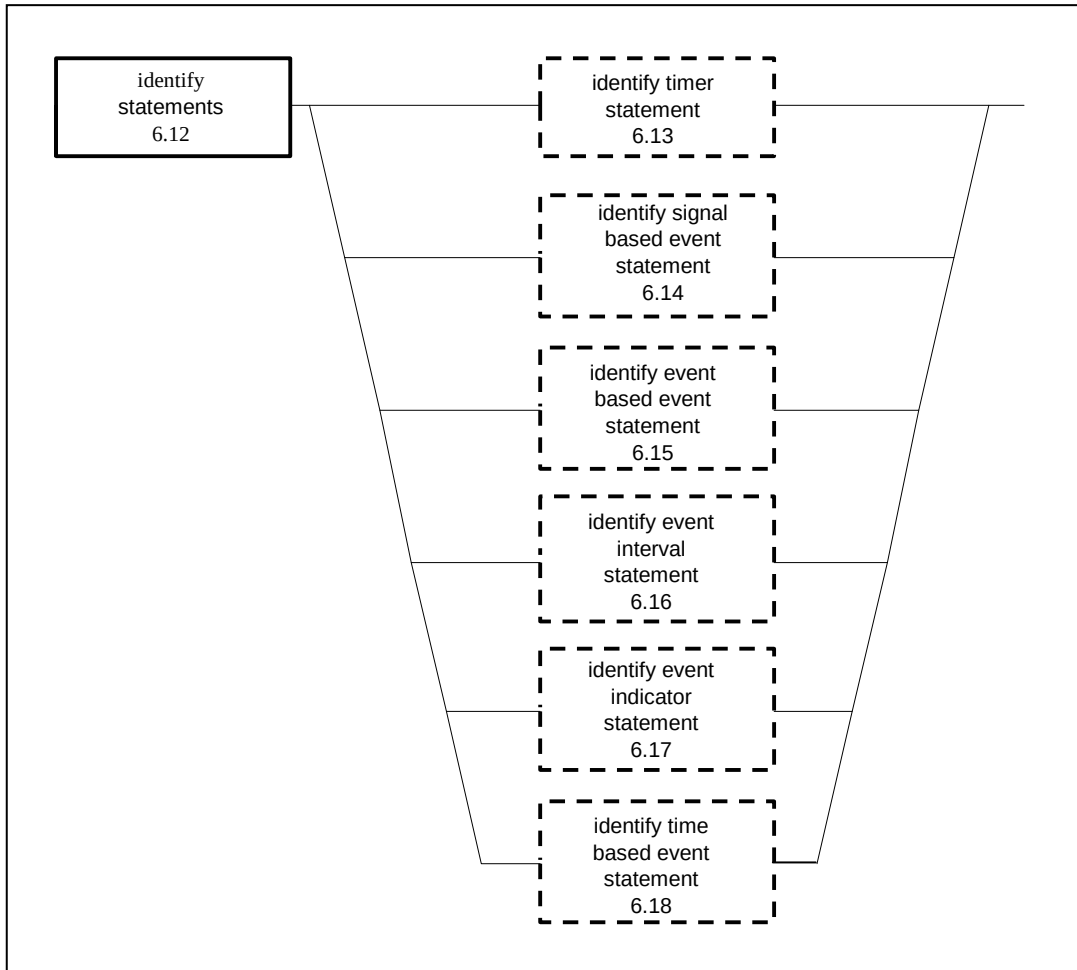
Section 13.1.5 includes examples of databus tests.

6.0 PREAMBLE STATEMENTS6.12 <identify statements>6.12.1 Function

To label and describe an <event>, <event interval>, <event indicator> or <timer> that is used for the purpose of allowing the establishment and implementation of signal and test action timing relationships.

6.12.2 Formal Syntax

FSD / 609

6.12.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS

6.13 <identify timer statement>

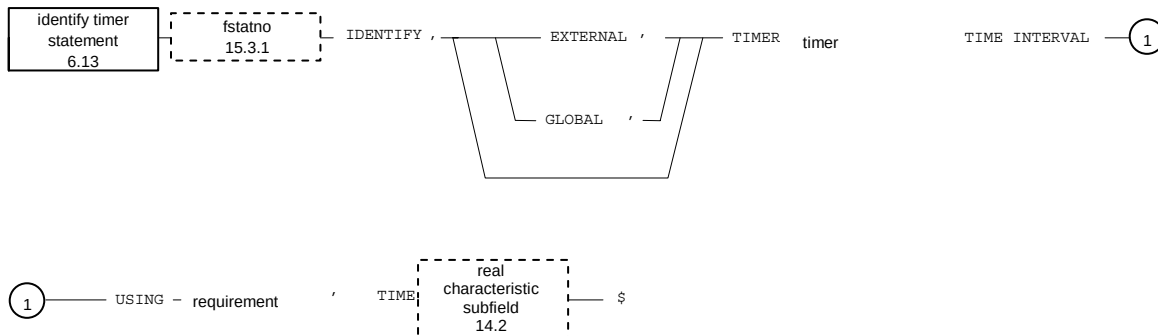
6.13.1 Function

An <identify timer statement> identifies a <timer> which is used for the timing of critical actions. A <timer> is a constantly running device which allows the definition of a time scale on which critical actions may be precisely and unambiguously positioned.

6.13.2 Formal Syntax

FSD / 620

6.13.3 Syntax Diagram



6.13.4 Rules

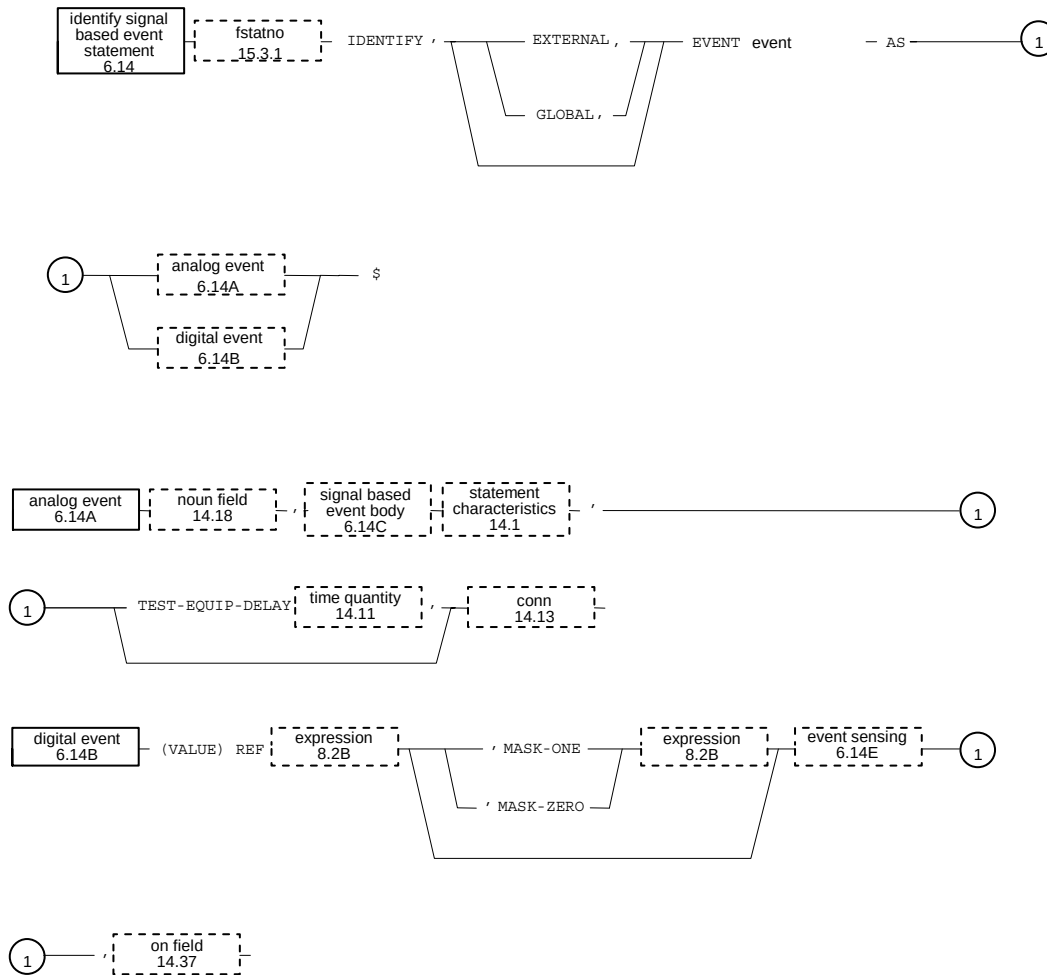
1. A <timer> has no idle state. It starts counting time from zero as soon as it is reset.
2. The attribute <global> may only appear in the preamble of an <atlas program structure>. The attribute <external> may only appear in the preamble of an <atlas module structure>. Each <label> defined as <external> must also be defined as <global> in a statement in which the requirements encompass those of the <external> definition.

6.0 PREAMBLE STATEMENTS**6.14 <identify signal based event statement>****6.14.1 Function**

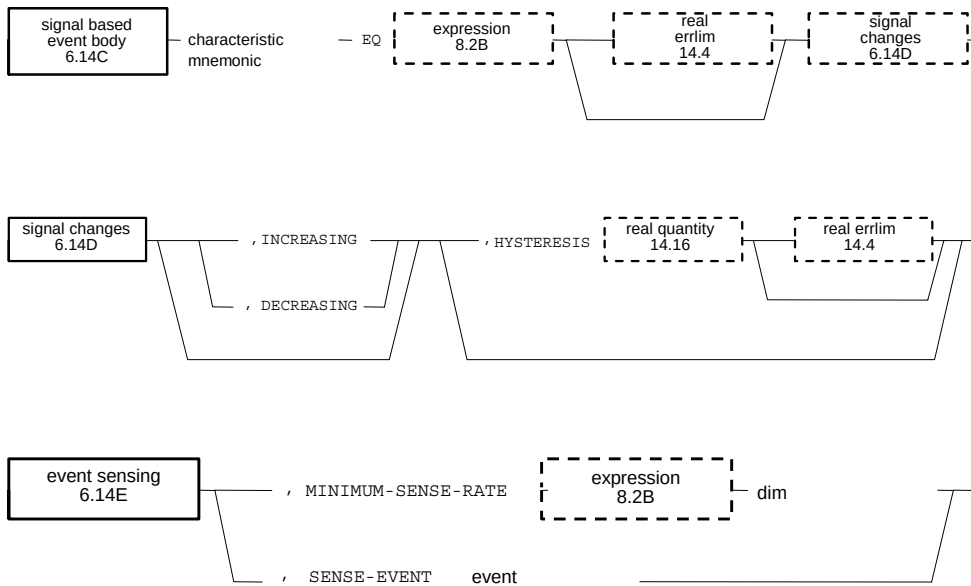
To describe instances in time that are based on signal conditions occurring at the UUT interface. An event will be generated each time the stated signal conditions are satisfied.

6.14.2 Formal Syntax

FSD / 629

6.14.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS



6.14.4 Rules

1. <events> are described in terms of certain basic UUT signal characteristics. An event will be generated each time a signal achieves the specified condition. If the signal satisfies the specified signal condition for a period of time after first achieving that characteristic, additional <events> will not be generated during that time. In order for a new <event> to be generated, the signal must deviate from the specified condition. The optional <real errlim> field indicates the accuracy required of the event monitoring resource.
2. If the INCREASING branch is selected, an <event> will be generated only if the value or the <monitored characteristic> is increasing when it passes through the specified trigger point. If the DECREASING branch is selected, an <event> will be generated only if the value of the <monitored characteristic> is decreasing when it passes through the trigger point. If the by-pass around DECREASING and INCREASING is selected, an event will be generated each time the <monitored characteristic> passes through the trigger point regardless of the direction. For the purpose of providing variable control of slope, the DECREASING/INCREASING option may be bypassed and a RANGE field used in the <statement characteristic> to identify the desired slope as well as total signal range. When a RANGE field is used in this manner, the rule above for INCREASING applies when the algebraic magnitude of the final quantity is greater than the initial quantity and the rule for DECREASING applies for the opposite case.
3. The option of specifying hysteresis allows the user to specify the amount that the <monitored characteristic>, after passing through the trigger point and generating an <event>, must vary from the trigger point before an <event> is able to be regenerated.
4. The optional TEST-EQUIP-DELAY field indicates the maximum delay that is acceptable from the recognition by the monitoring resource that the signal being sensed has satisfied the specified condition to the generation of the <event>. Any specified delay must be non-negative. If this field is omitted the delay is assumed to be non-critical.
5. The only allowable <monitored characteristic>s are those which are identified as measured characteristics in Chapter 16.

6.0 PREAMBLE STATEMENTS

6. An IDENTIFY, EVENT based on sensing the <measured characteristic mnemonic> of a signal at the UUT interface requires a resource called an EVENT MONITOR. EVENT MONITOR characteristics are defined by an IDENTIFY EVENT statement and connected and started in an appropriate statement.
7. The configuration that contains the event digital sensor definition must be enabled prior to the explicit or implicit enabling of the appropriate EVENT MONITOR.
8. The optional MINIMUM-SENSE-RATE field, which may occur in digital <event> identification, specifies the minimum rate at which the <event> monitor must interrogate the <digital sensor> that is explicitly referenced in the <on field>. This is to determine whether it has yet achieved the required pattern as specified in the REF field after any necessary processing arising from the optional MASK field. The operation of REF and MASK subfields is described in the rules for the <prove statement>.
9. Similarly the optional SENSE-EVENT field, which may occur in digital <event> identifications, specifies an <event> which identifies the intervals at which the <event> monitor must interrogate the digital sensor that is explicitly referenced in the <on field>.
10. An <identify signal based event statement> that senses the <monitored characteristic> of a signal at the UUT interface requires a resource called an EVENT MONITOR. EVENT MONITOR characteristics are defined by an <identify event statement> and executed by an appropriate statement, for example; a <measure statement>.
11. The <sync subfield> path through <statement characteristic> subfield is not allowed for the <identify signal based event statement> unless the <monitored characteristic> is the VALUE of either a data or control word which is related to a sync reference as defined in rules 6(a) and 6(c).
12. If variables are used for any of the quantities in an <identify statement>, values must be assigned to these variables prior to the execution of any <setup statement> or <change statement> which references the identified <event>. When the <setup statement> or <change statement> is executed, the current values of these variables are used to complete the <event> specification.

If variables are used for any of the connections in an <identify statement>, values must be assigned to these variables prior to the execution of any <connect statement> which references the identified <event>. When the <connect statement> is executed the current values of these variables are used to complete the <event> specification.

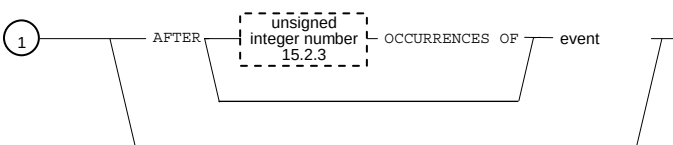
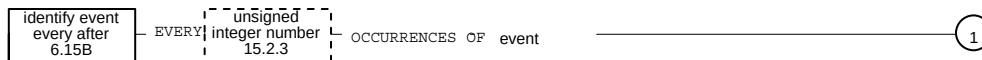
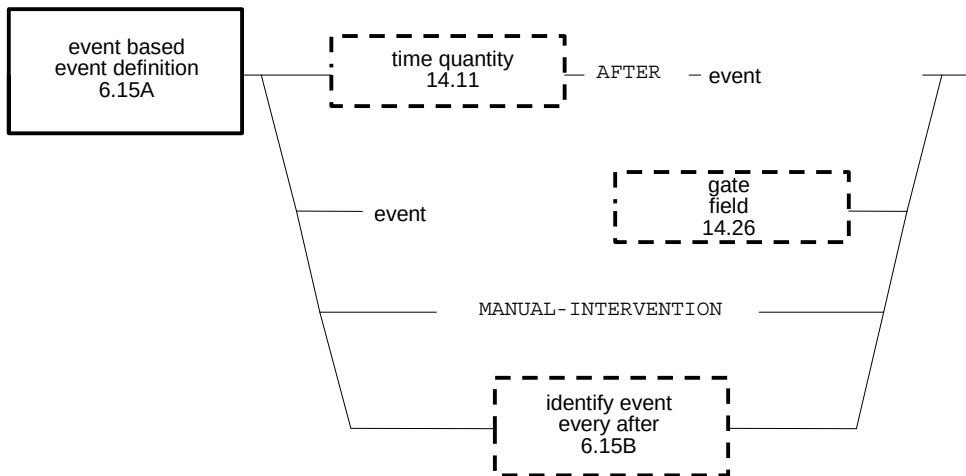
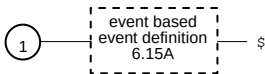
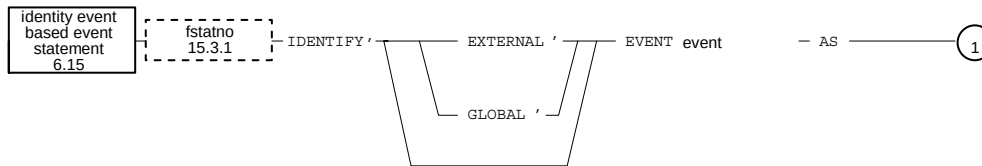
Succeeding changes of these variable values do not affect the configured <event>.
13. All <expression>s must evaluate to a type which is equivalent to the type of the associated modifier or field. An optional <real errlim> field following an <expression> may only be used if the <expression> evaluates to a <decimal number> or whole <decimal number>.
14. The attribute <global> may only appear in the preamble of an <atlas program structure>. The attribute <external> may only appear in the preamble of an <atlas module structure>. Each <label> defined as <external> must also be defined as <global> in a statement in which the requirements encompass those of the <external> definition.

6.0 PREAMBLE STATEMENTS**6.15 <identify event based event statement>****6.15.1 Function**

To identify instances in time that are determined by other instances in time.

6.15.2 Formal Syntax

FSD / 679

6.15.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS

6.15.4 Rules

1. <event>s may be described in terms of other <event>s, possibly in conjunction with <event interval>s. The possibilities are as follows:

- a. EVERY n OCCURRENCES.

An <event> will be generated every time another referenced <event> occurs n times.

- b. EVERY n OCCURRENCES AFTER m OCCURRENCES.

After m occurrences of one specified <event>, an <event> will be generated every n occurrence of another specified <event>. It is allowable to specify the same <event> label for the EVERY n OCCURRENCES OF <event> as for the AFTER m OCCURRENCES OF <event> but they are not required to be the same.

- c. n OCCURRENCES OF.

An <event> will be generated on the nth occurrence of a specified <event>. After this generation of the <event> no further <event>s will be generated unless the <event> is disabled, re-enabled and then n more referenced <event>s occur.

- d. n OCCURRENCES OF AFTER m OCCURRENCES.

An <event> will be generated on the nth occurrence of a specified <event> after m occurrence of another specified <event>. After this <event> generation, no further <event>s will be generated unless first disabled and re-enabled.

- e. In items b and d above the “m OCCURRENCES OF” phrase may be omitted when the integer m or n to be specified is equal to one (1).

- f. GATED BY.

An <event> will be generated each time a referenced <event> occurs during a referenced <event interval>.

- g. AFTER.

An <event> will be generated after the specified <event>.

- h. MANUAL INTERVENTION.

An <event> will be specified each time that a MANUAL INTERVENTION is performed while MANUAL INTERVENTION is ENABLED.

2. Items a - d of Rule 1 above will require the counting of other referenced <event>s and are affected by the enabling of the identified <event>. Each time that the identified <event> is enabled the counting mechanism(s) for that <event> will be set to a zero count.

3. <event> being identified shall not reference themselves in their definition.

6.0 PREAMBLE STATEMENTS

4. If variables are used for any of the quantities in an <identify statement>, values must be assigned to these variables prior to the execution of any <setup statement> or <change statement> which references the identified <event>. When the <setup statement> or <change statement> is executed the current values of these variables are used to complete the <event> specification.

If variables are used for any of the connections in an <identify statement>, values must be assigned to these variables prior to the execution of any <connect statement> which references the identified <event>. When the <connect statement> is executed the current values of these variables are used to complete the <event> specification.

Succeeding changes of these variable values do not affect the configured <event>.

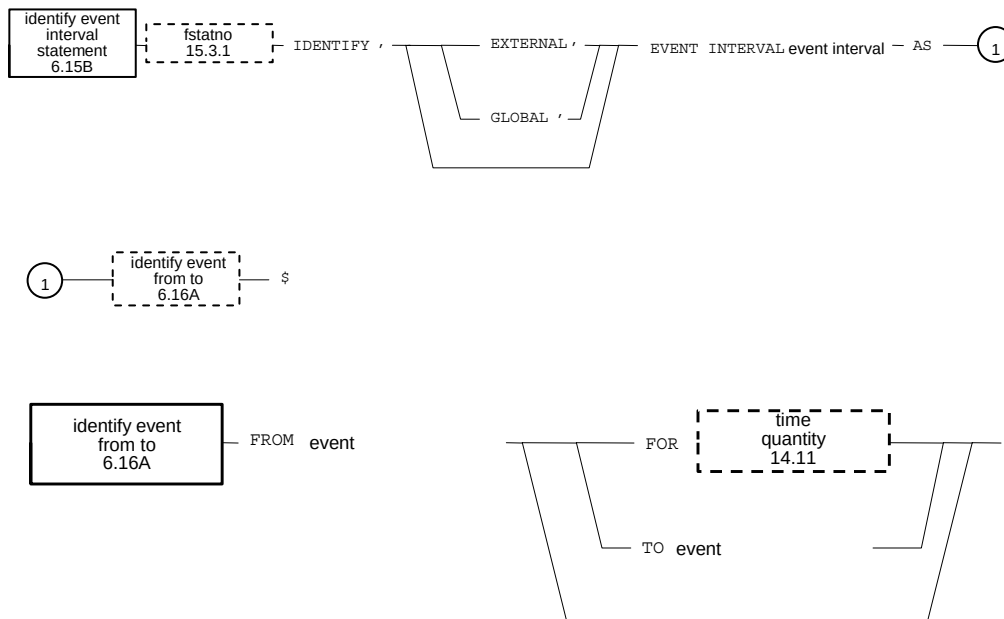
5. The attribute <global> may only appear in the preamble of an <atlas program structure>. The attribute <external> may only appear in the preamble of an <atlas module structure>. Each <label> defined as <external> must also be defined as <global> in a statement in which the requirements encompass those of the <external> definition.

6.0 PREAMBLE STATEMENTS6.16 <identify event interval statement>6.16.1 Function

To identify a period of time.

6.16.2 Formal Syntax

FSD / 706

6.16.3 Syntax Diagram6.16.4 Rules for EVENT INTERVAL

1. <event interval>s identify particular regions of time. These regions may be specified in the following manners:

a. FROM <event> FOR (a specified time)

An <event interval> may be defined to begin on the occurrence of a specified <event> and to end a specified time period after the occurrence of that <event>. If the referenced <event> reoccurs during this time period, the period of the <event interval> will be extended by the specified time period from the point of the reoccurrence.

b. FROM <event> TO <event>

An <event interval> may be defined as the time period between the occurrence of one referenced <event> and the next occurrence of another referenced <event>. Different <event>s must be specified to start and end the <event interval>.

c. FROM <event>

An <event interval> may be defined as the time period starting at the first occurrence of a specified <event> and lasting for the duration of the <atlas test specification structure>.

6.0 PREAMBLE STATEMENTS

2. If variables are used for any of the quantities in an <identify statement>, values must be assigned to these variables prior to the execution of any <setup statement> or <change statement> which references the identified <event>. When the <setup statement> or <change statement> is executed the current values of these variables are used to complete the <event> specification.

If variables are used for any of the connections in an <identify statement>, values must be assigned to these variables prior to the execution of any <connect statement> which references the identified <event>. When the <connect statement> is executed the current values of these variables are used to complete the <event> specification.

Succeeding changes of these variable values do not affect the configured <event>.

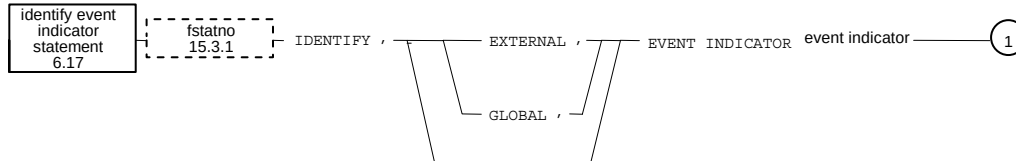
3. The attribute <global> may only appear in the preamble of an <atlas program structure>. The attribute <external> may only appear in the preamble of an <atlas module structure>. Each <label> defined as <external> must also be defined as <global> in a statement in which the requirements encompass those of the <external> definition.

6.0 PREAMBLE STATEMENTS**6.17 <identify event indicator statement>****6.17.1 Function**

To identify that a particular Boolean variable that had previously been declared is an <event indicator>.

6.17.2 Formal Syntax

FSD / 720

6.17.3 Syntax Diagram

① — AS SET BY event — \$

6.17.4 Rules for EVENT INDICATOR

1. <event indicator>s are Boolean variables that are used to indicate the occurrence of an <event>. They have the properties of normal Boolean variables with an additional feature that they are automatically set when the <event> with which they are associated occurs while the <event> is enabled.
2. Once an <event indicator> is set to the value TRUE by the occurrence of a referenced <event>, it will remain TRUE until specifically reset in a <calculate statement>.
3. Before an <event indicator> is identified, it must have been declared as a boolean variable.
4. The attribute <global> may only appear in the preamble of an <atlas program structure>. The attribute <external> may only appear in the preamble of an <atlas module structure>. Each <label> defined as <external> must also be defined as <global>.

6.17.5 Notes and Examples

In these examples a <monitored characteristic> trigger point is set equal to 5.0 V - INST and the nominal measure time should be 13.6 NSEC. Case 1 shows that the measure time has shortened to 13.2 NSEC due to a 1 V-INST error in the actual trigger point. Case 2 illustrates the possibility of generating two events on a noise perturbation that occurs near the trigger point. Case 3 shows, if hysteresis were specified of about a volt or so, this double generation of the event could be avoided.

6.0 PREAMBLE STATEMENTS

Example 1

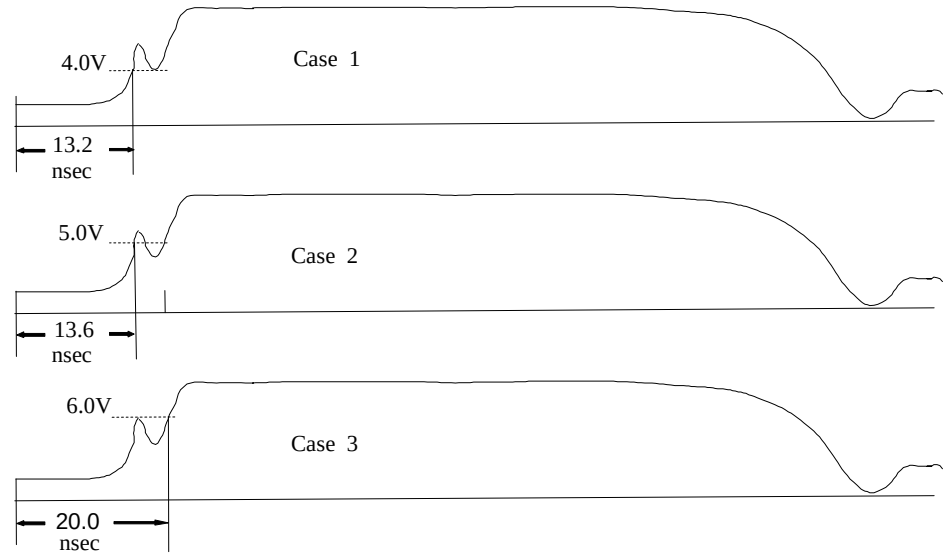


Figure 6-8 EVENT Detection

Example 2

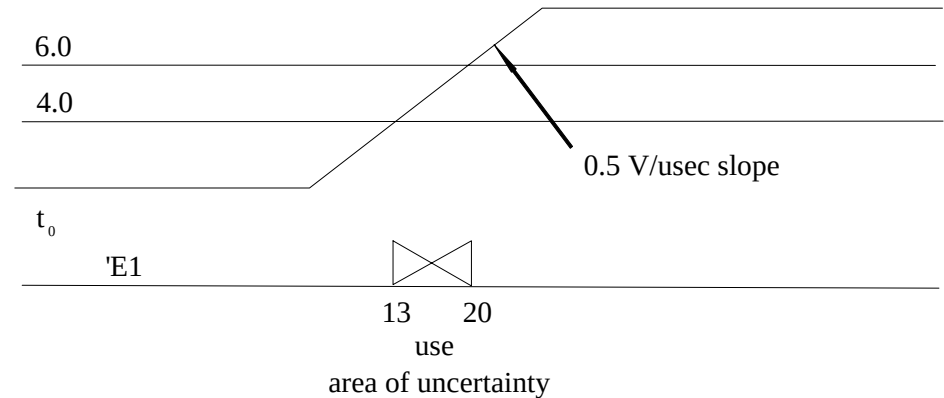


Figure 6-9 EVEN Detection

6.0 PREAMBLE STATEMENTS

```

000100 IDENTIFY, EVENT 'E1' AS PULSED DC,
      VOLTAGE-INST EQ 5.0 V ERRLMT +-1.0 V, INCREASING,
      VOLTAGE-INST MAX 12.0 V, PRF RANGE 1.0 KHZ TO 2.0 KHZ,
      TEST-EQUIP-DELAY MAX 3.0 USEC, CNX HI J1-1 LO J1-2 $

```

With a nominal 5.0 V threshold and 0.0 USEC delay, the event 'E1' would occur 15.0 USEC after t_0 . Due to the uncertainty in both the trigger point and the delay however, 'E1' may actually occur anywhere from 13.0 USEC after t_0 , with a 4.0 V trigger and 0.0 USEC delay, to 20.0 USEC after t_0 , with a 6.0 V trigger and 3.0 USEC delay.

```

000100 IDENTIFY, EVENT 'POS-5V' AS AC SIGNAL USING 'THRESHOLD A',
      VOLTAGE-INST EQ 5.0 V, VOLTAGE MAX 10.0 V,
      FREQ RANGE 50.0 HZ TO 70.0 HZ, CNX HI J1-A LO J1-B $
000200 IDENTIFY, EVENT 'POS-6V' AS AC SIGNAL USING 'THRESHOLD B',
      VOLTAGE-INST EQ 6.0 V, VOLTAGE MAX 10.0 V,
      FREQ RANGE 50.0 HZ TO 70.0 HZ, CNX HI J1-A LO J1-B $
000300 DECLARE, VARIABLE, 'MARKER-5V', 'MARKER-6V' IS BOOLEAN
      INITIAL = FALSE, FALSE $
001100 IDENTIFY, EVENT INDICATOR 'MARKER-5V' AS SET BY 'POS-5V' $
001200 IDENTIFY, EVENT INDICATOR 'MARKER-6V' AS SET BY 'POS-6V' $
100200 MEASURE, (TIME), TIME INTERVAL USING 'TIMER',
      TIME MAX 5.0 MSEC, FROM 'POS-5V' TO 'POS-6V',
      MAX-TIME 1.0 SEC $
100300 IF, MAX-TIME THEN $
100350     IF, 'MARKER-5V' EQ FALSE, THEN $
100400         OUTPUT, TEXT, FROM C'FAILURE OF AC SIGNAL AT J1-A
              TO REACH 5.0 V LEVEL' $
100500     ELSE $
100600         OUTPUT, TEXT, FROM C'FAILURE OF AC SIGNAL AT J1-A
              TO REACH 6.0 V LEVEL' $
100700     END, IF $
100800     FINISH $
100900 END, IF $
101000 APPLY, PULSED DC USING 'PULSE-GEN', VOLTAGE-P 5.0 V,
      PULSE-WIDTH 1.0 USEC,
      SYNC VOLTAGE-INST EQ 5.0V POS-EDGE TO EVENT 'POS-5V',
      CNX HI J1-W LO J1-X $

```

In the example above, the time between an ac signal reaching the 5V level and reaching the 6V level is measured on Pin J1-A in statement 100200. If the measurement did not complete within the 1.0 second time limit, then an error message is printed and the program execution completes in statement 100800. If the measurement does not exceed the maximum time limit, then a PULSED DC signal is applied in statement 101000 on pin J1-W. This signal has a pulse width of 1.0 USEC at J1-W passing through the 5.0 V threshold.

6.0 PREAMBLE STATEMENTS

6.18 <identify time based event statement>

6.18.1 Function

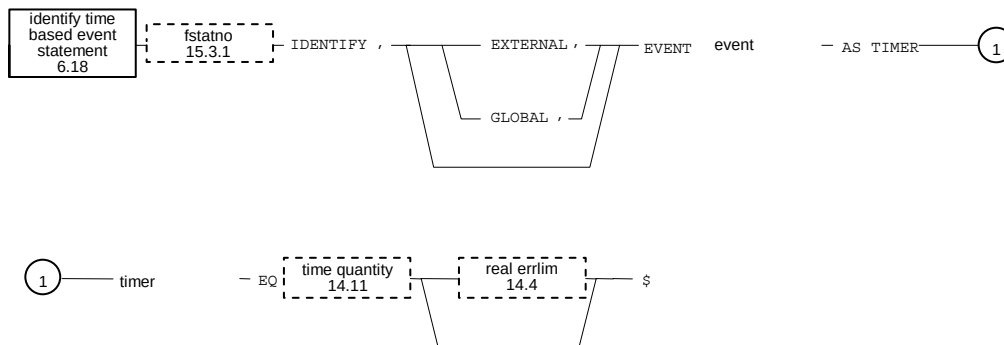
To identify one or more points in time that are directly derived from a <timer>. An <event> will be generated each time the <timer> matches the specified time value and the <event> is enabled.

The accuracy and resolution of the timer are defined in the <identify time statement>.

6.18.2 Formal Syntax

FSD / 728

6.18.3 Syntax Diagram



6.18.4 Rules

1. The attribute <global> may only appear in the preamble of an <atlas program structure>. The attribute <external> may only appear in the preamble of an <atlas module structure>. Each <label> defined as <external> must also be defined as <global> in a statement in which the requirements encompass those of the <external> definition.
2. If variables are used for any of the quantities in an <identify statement>, values must be assigned to these variables prior to the execution of any <setup statement> or <change statement> which references the identified <event>. When the <setup statement> or <change statement> is executed the current values of these variables are used to complete the <event> specification.

If variables are used for any of the connections in an <identify statement>, values must be assigned to these variables prior to the execution of any <connect statement> which references the identified <event>. When the <connect statement> is executed the current values of these variables are used to complete the <event> specification.

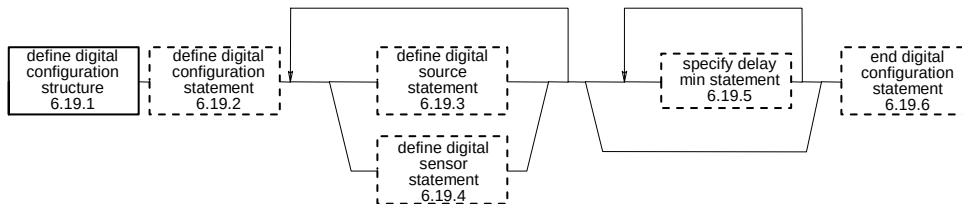
Succeeding changes of these variable values do not affect the configured <event>.

6.0 PREAMBLE STATEMENTS**6.19 DIGITAL CONFIGURATION Definition****6.19.1 <define digital configuration structure>****6.19.1.1 Function**

To define a structure combining digital data and source and sensor signals and the relationships between them.

6.19.1.2 Formal Syntax

FSD / 735

6.19.1.3 Syntax Diagram**6.19.1.4 Rules**

1. More than one <define digital configuration structure> may be defined but only one configuration will be active at any one time.
2. When a <specify delay min statement> is used, the DELAY-MIN field in this statement must be compatible with the <max rate> field in the <define digital source statement> and the <define digital sensor statement>.

6.19.1.5 Notes and Examples

1. A define configuration structure for a bidirectional bus.

```

100000 DEFINE, 'TTL-BUS', DIGITAL CONFIGURATION $
100100 DEFINE, 'TTL-IN', DIGITAL SOURCE, DC SIGNAL,
    VOLTAGE RANGE 0 V TO 5 V,
    CURRENT RANGE -10 UA TO 10 UA,
    LEVEL-LOGIC-ZERO VOLTAGE 0.4 V,
    LEVEL-LOGIC-HIZ CURRENT MIN -5 UA,
    LEVEL-LOGIC-ONE VOLTAGE 3.2 V,
    LEVEL-LOGIC-HIZ CURRENT MAX +5 UA,
    ILLEGAL STATE INDICATOR 'INPUT-FAULT',
    SKEW-TIME MAX 1 USEC,
    CNX HI D7 D6 D5 D4 D3 D2 D1 D0 LO J1-1 $
100200 DEFINE, 'TTL-OUT', DIGITAL SENSOR, (VOLTAGE), DC SIGNAL,
    VOLTAGE RANGE 0 V TO 5 V,
    LEVEL-LOGIC-ZERO UL 0.8 V LL 0 V,
    LEVEL-LOGIC-ONE UL 5 V LL 2.4 V,
    LEVEL-LOGIC-HIZ CURRENT MIN -10 UA,
    ILLEGAL STATE INDICATOR 'OUTPUT-FAULT',
    SKEW-TIME MAX 1 USEC,
    CNX HI D7 D6 D5 D4 D3 D2 D1 D0 LO J1-1 $
100300 END, 'TTL-BUS' $
  
```


6.0 PREAMBLE STATEMENTS

Statement 100100 illustrates a parallel digital source with a tri-state capability. As two LEVEL-LOGIC-HIZ specifications are included, the test system leakage current is defined for both impressed logic states. 'INPUT-FAULT' would be set TRUE if, when a procedural STIMULATE statement was executed, the test system could not drive the UUT.

Statement 100200 illustrates a parallel digital sensor again with a tri-state capability. As only one LEVEL-LOGIC-HIZ specification is included, the UUT leakage current applies to both impressed logic states. 'OUTPUT-FAULT' would be set TRUE if, when a procedural SENSE or PROVE statement was executed, the UUT signals did not fall within the defined logic states.

2. A define digital source statement for a BIP serial bus.

```
102000 DEFINE, 'BI-PHASE-MARK', DIGITAL SOURCE,
        LOGIC SIGNAL USING 'BIP-STIM',
        WORD-RATE MAX 2 KWORDS/SEC,
        LEVEL-LOGIC-ONE VOLTAGE 1 V,
        LEVEL-LOGIC-ZERO VOLTAGE 0 V,
        TRANS-LOGIC-ONE 00 OR 11,
        TRANS-LOGIC-ZERO 01 OR 10,
        TRANS-SYNC 01,
        SERIAL-MSB-FIRST,
        WORD-LENGTH 16 BITS,
        PULSE-CLASS BIP,
        BIT-RATE 100 KBITS/SEC,
        CNX HI J1-1 LO J1-0 $
```

Statement 102000 illustrates a serial digital source providing a BIP form of pulse coding. For this type of serial signal a transition occurs at the start of each bit period, with binary '0' having a second transition, and binary '1' having no such second transition.

3. A define digital source statement for a serial bus using frequency coding.

```
103000 DEFINE, 'CONFIG1', DIGITAL CONFIGURATION $
104000 DEFINE, 'FREQ-LOGIC', DIGITAL SOURCE,
        LOGIC SIGNAL USING 'MODEM',
        VOLTAGE 10 V,
        FREQ RANGE 1 MHZ TO 2.5 MHZ,
        WORD-RATE MAX 10 KWORDS/SEC,
        LEVEL-LOGIC-ZERO FREQ 2 MHZ,
        LEVEL-LOGIC-ONE FREQ 2.5 MHZ,
        LEVEL-LOGIC-QUIES FREQ 1 MHZ,
        SERIAL-LSB-FIRST,
        WORD-LENGTH 8 BITS,
        BIT-RATE 5 KBITS/SEC,
        CNX HI J1-2 LO J1-0 $
```

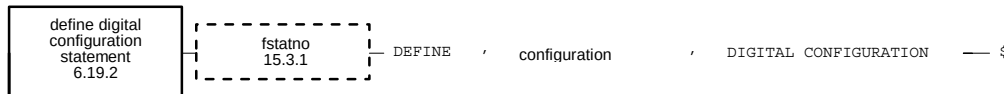
Statement 104000 illustrates a serial digital source where steady states are defined in term of a frequency.

6.0 PREAMBLE STATEMENTS**6.19.2 <define digital configuration statement>****6.19.2.1 Function**

To name and introduce a <define digital configuration structure>.

6.19.2.2 Formal Syntax

FSD / 749

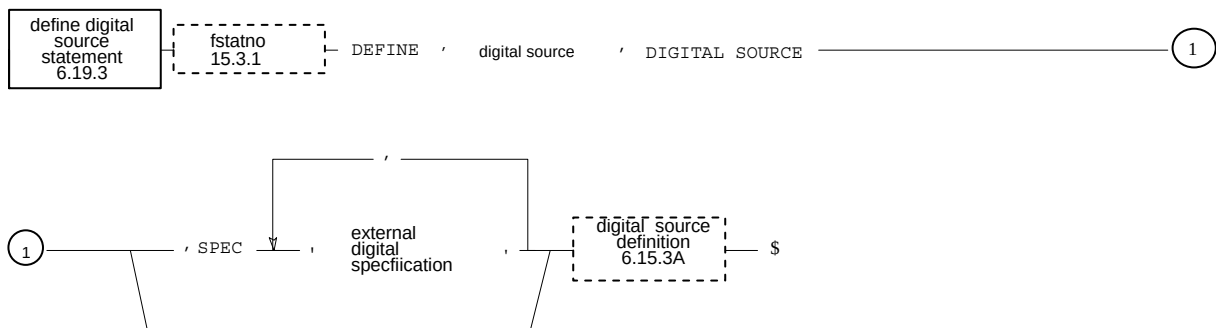
6.19.2.3 Syntax Diagram**6.19.3 <define digital source statement>****6.19.3.1 Function**

This statement establishes a label <digital source> for a noun, a set of digital source characteristics and a set of connections to be used as a digital source. For each signal connection it allows for the definition of the physical signal corresponding to each logic state. For LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO the physical characteristics of signals may be defined. This allows a precise definition of a range of “pulse-classes” such as BIP, AMI etc.

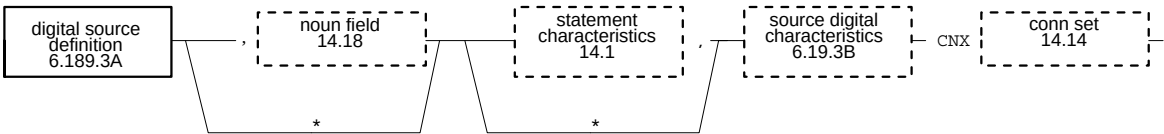
The statement allows secondary as well as primary characteristics of signals to be specified. The primary characteristics, in which the transformation operates, can have temporal qualifications which can be used to synchronize them to other UUT signals or to a test resource time-scale. Serial as well as parallel relationships can be defined.

6.19.3.2 Formal Syntax

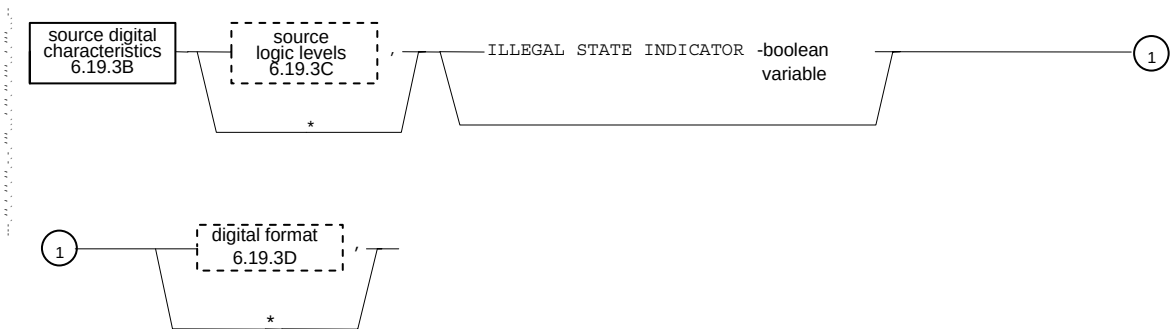
FSD / 755

6.19.3.3 Syntax Diagram

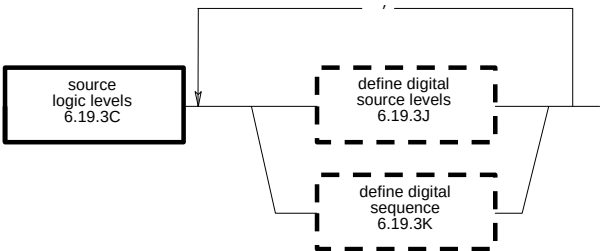
6.0 PREAMBLE STATEMENTS



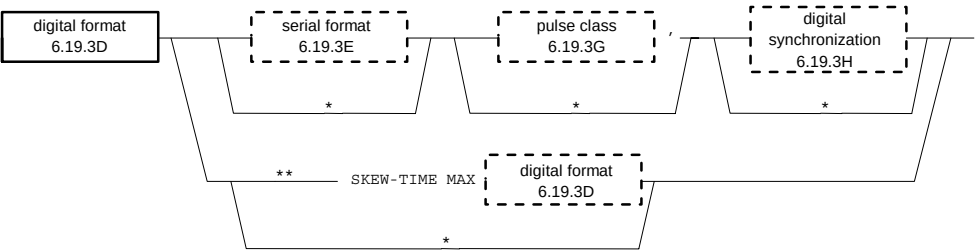
NOTE : * This bypass is used when the relevant information is completely defined in the specification identified by the <external digital specification>.



NOTE : * This bypass is used when the relevant information is completely defined in the specification identified by the <external digital specification>.

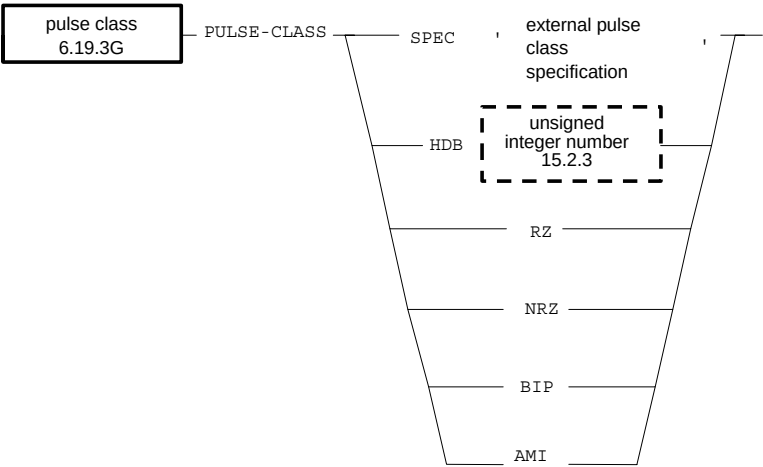
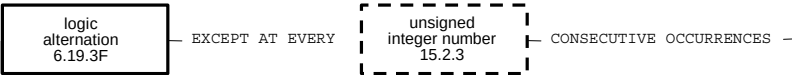
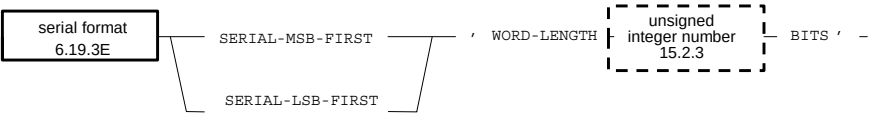


6.0 PREAMBLE STATEMENTS

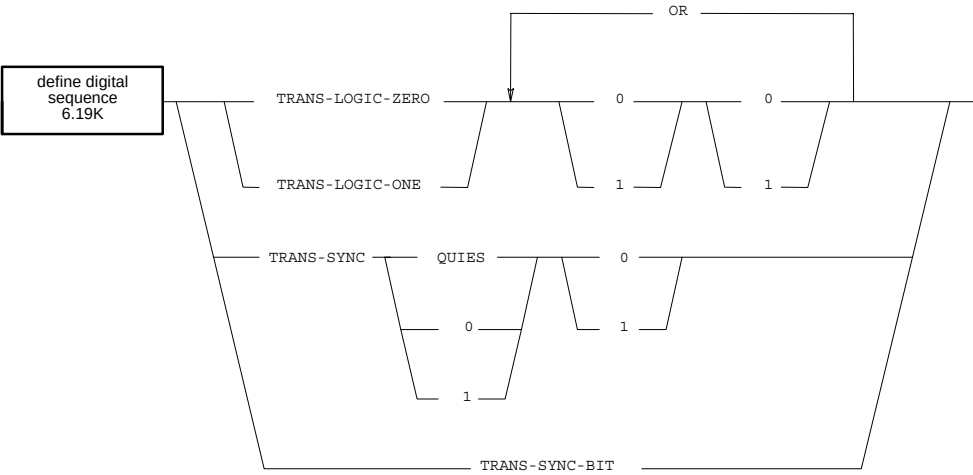
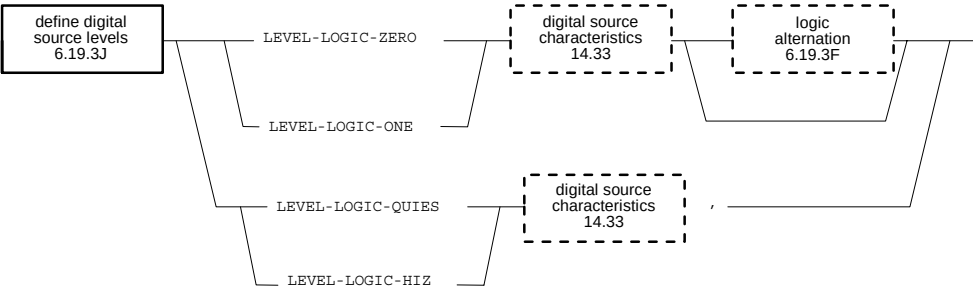
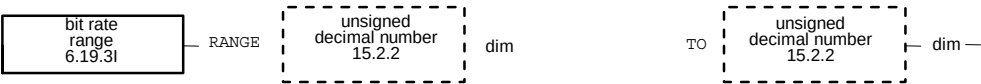
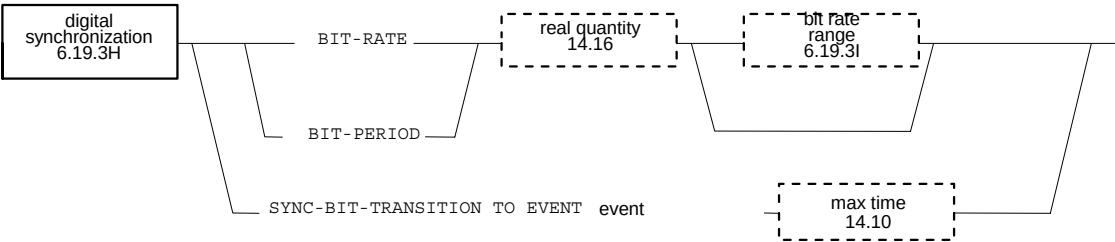


Note: * This bypass is used only if all the information has been defined in the <external digital specification> and the signals are parallel.

** This path is used only with parallel digital signals.



6.0 PREAMBLE STATEMENTS



6.0 PREAMBLE STATEMENTS**6.19.3.4 Rules for All <define digital source statements>**

1. For parallel sources the order in which signal connections are written after CNX is significant. The first connection corresponds to the most significant bit.
2. The <external digital specification> option is used when the signal characteristics of a digital word are defined in an existing specification. The label, <external digital specification>, must identify the specification for the signal characteristics of the digital source. If the information identified by <external digital specification> is either incomplete or inconsistent with the requirement, then information must be supplied explicitly by the appropriate fields in the <define digital source statement> which then take precedence over the information in the <external digital specification>.
3. Explicit signal characteristics of LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO may be defined in terms of LEVELS, which are steady states that hold true throughout the bit period and for all instances of a given state. If the <pin descriptor>s TRUE and COMPL are employed the value stated is that of the TRUE connection with respect to the COMPL connection.

Provision is made for the exceptional insertion of a different level. This technique is used to break up long chains of a given state in some digital transmission systems such as HDB.
4. When the PULSE-CLASS field is used, it must be consistent with the information included in the <digital logic levels> and/or the <external digital specification>. The SPEC subfield of the PULSE-CLASS field is used to identify additional pulse classes.
5. LEVEL-LOGIC-QUIES, when specified, refers to the state which exists between bits or between words. For bus systems, this is the neutral state when no data is present.
6. A named <digital source> can be individually referenced in a <stimulate statement>.
7. The term LEVEL-LOGIC-HIZ applies only to electrical systems where LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO are defined in terms of voltages. LEVEL-LOGIC-HIZ is defined in terms of the maximum current flow when the defined LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO levels are impressed on test system outputs. If only one LEVEL-LOGIC-HIZ is specified the allowed leakage current applies to both of the impressed logic states. If it is required to specify different leakages for the two impressed logic states, then the LEVEL-LOGIC-HIZ specification must be repeated and the position, which must be immediately following the LEVEL-LOGIC-ONE or LEVEL-LOGIC-ZERO field, defines to which state the leakage relates.
8. The initial state that will exist prior to encountering the first <stimulate statement> will be LEVEL-LOGIC-HIZ, if defined, else LEVEL-LOGIC-QUIES, if defined, else LEVEL-LOGIC-ZERO.
9. When an ILLEGAL STATE INDICATOR is used, the associated <boolean variable> will be set TRUE if the <digital source> which generates the signal is out of specification.
10. The BIT-RATE/BIT-PERIOD sub-field, when present, specifies the speed of bit transmissions for SERIAL logic signals.
11. For signals which utilize an independent clock source, the synchronizing event must be identified as an <event>. The beginning of every bit period of the serial data stream generated by the <digital source> is then synchronized to this <event> via the SYNC BIT-TRANSITION TO EVENT branch.
12. If an <event> is specified for serial data clocking, the <max time> field indicates the maximum time that the digital configuration is to wait, following a STIMULATE start, for the first occurrence of the <event>. If the <event> does not occur within this time limit, a MAX-TIME condition is set and the test resumes execution without completion of the <stimulate statement>.

6.0 PREAMBLE STATEMENTS

13. The TRANS-LOGIC-ONE and TRANS-LOGIC-ZERO fields are used to define transitions between the defined -ONE and -ZERO signal levels which correspond to a digital one and a digital zero respectively. The OR is available because some transmission systems permit a logic one or zero to be represented in two different ways depending on the start condition. These transitions occur at the mid-point of a data bit duration in Bi-Phase Signals.
14. The TRANS-SYNC field is used to define that there must be a transition at the start of each word. A number pair following the key-word indicates the transition required prior to the first bit. If the keyword QUIES occurs after TRANS-SYNC it indicates that the signal is required to be at the Quiescent level prior to the initial transition of the word. The second entry indicates the level it acquires at the start of the word transmission. The TRANS-SYNC-BIT field is used to define that there must be a transition at the start of each bit period.
15. In statements employing the TRANS-LOGIC-ONE and TRANS-LOGIC-ZERO fields indicating that the logical information is carried by signal transitions, the -ONE and -ZERO suffixes can be used after the mnemonic in the <statement characteristics> field to indicate that the associated signal characteristic carries the digital information and assumes the stated value when the signal is a ONE or a ZERO respectively. If one of these suffixes is used the same mnemonic must occur in another field with the other suffix.
16. The SKEW-TIME MAX field defines the maximum acceptable skew between the various output ports employed by a parallel digital pattern.
17. SERIAL-MSB-FIRST and SERIAL-LSB-FIRST identify the digital source as being serial and identify respectively that the left-most bit or the right-most bit is to be transmitted first.
18. When using the noun LOGIC SIGNAL, noun modifiers having the same name as keywords cannot exist in the same statement.

6.19.3.5 Rules Applicable to the <pulse class> Field

(Syntax Diagram 6.19.3G)

1. The PULSE CLASS is required to be explicitly or implicitly defined for all SERIAL DATA statements, for both SOURCE and SENSOR. It cannot be used as a measured characteristic in SENSOR statements.
2. The subfield after PULSE-CLASS identifies the class of pulse coding used to represent the digital data. The following classes are defined:

RZ	Return to zero
NRZ	Non return to zero
BIP	Bi-phase
AMI	Alternate Mark Inversion
HDBn	High Density Bipolar where n is an <unsigned integer number>.
3. The keyword SPEC followed by a reference to an <external pulse class specification> may be used when a pulse class that is not one of the standard classes is required. The referenced specification must fully describe the pulse coding of the transmission.

6.19.3.6 Rules Applicable to the <def dig source levels> and the <def dig sequence> Fields

1. The <mnemonic> chosen must be applicable to the <noun> specified in the associated <digital source field>.
2. No source can employ both LEVEL-LOGIC-... and TRANS-LOGIC-... keywords.

6.0 PREAMBLE STATEMENTS

3. When used following LEVEL-LOGIC-HIZ to specify the high impedance condition of the test system, the direction of leakage current flow is specified as follows:

- current flowing from the test system as a positive current;
- current flowing into the test system as a negative current.

6.19.3.7 Notes and Examples

- A. Notes regarding the mnemonic PULSE-CLASS.

1. For the RZ type, the bit period is subdivided into two sub-periods. The binary data is carried in the first sub-period. If the signal is carried by voltage levels the noun LOGIC SIGNAL is used as follows:

- a. A binary 1, by a pulse of LEVEL-LOGIC-ONE VOLTAGE amplitude
- b. A binary 0, by a pulse of LEVEL-LOGIC-ZERO VOLTAGE amplitude

The second sub-period contains a “no pulse” condition as a LEVEL-LOGIC-ZERO level.

Generally, the two sub-periods are of equal duration; this is the default case in ATLAS. For non-equal sub-periods either a PULSE-WIDTH or DUTY-CYCLE modifier is used to define the duration of the first sub-period.

2. The NRZ type corresponds to ARINC 573 and is also known as the TWG/RCC NRZ-Level (or NRZ Change). If the signal is carried by voltage levels it employs the noun LOGIC SIGNAL and is characterized by the following:
 - a. A binary 1 and binary 0, respectively, are represented by voltages of amplitude LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO.
 - b. The width of each pulse is the full bit period.
3. The BIP type represents a family of waveforms that are further defined by the TRANSition modifier. These waveforms are characterized by at least one change in voltage level (and for some BIP types, by two changes) for each data bit.

Refer to: “Telemetry Standards” (Revised January 1971), Document 106-71, issued by the Telemetry Working Group (TWG) of the Inter-Range Instrumentation Group of the Range Commanders Council (RCC), White Sands Missile Range, New Mexico, 88002. See PULSE-CLASS Figure 1.

4. The AMI type of waveform is similar to the NRZ waveform. Code elements may take the values +1, -1 and 0 where each successive binary '1' value is encoded using alternate inversions; i.e. +1, -1, +1, -1 etc. All binary '0's are encoded as '0'. Three signal levels need to be defined in the LEVEL-LOGIC-... fields. Two -ONE levels and one -ZERO level are defined. The -ONE level defined by the first LEVEL-LOGIC-ONE field is assumed by the initial binary '1' value in the data string and then by the third and subsequent alternate logic ones. The level defined by the second LEVEL-LOGIC-ONE field is assumed by the second, fourth and subsequent alternate logic ones.

Refer to: CCITT Document, Vol. 3, Rec G702, Entry No. 8003

6.0 PREAMBLE STATEMENTS

5. The HDB type of waveform is similar to the AMI waveform, where each successive binary '1' value is encoded using alternate inversions. 'n' is an integer which specifies the maximum number of consecutive binary '0's, encoded as '0', allowed in the data string. Thus all binary '0's apart from those in sequences of greater than 'n' are encoded as '0'. In any sequence of 'n' consecutive binary '0's the ultimate '0' shall be replaced by a +1 or -1 level, the polarity being the same as that of the preceding '1' value in the data string. The only difference from defining an AMI waveform is that the EXCEPT AT EVERY sequence is employed to define the maximum permitted number of successive logic zeros to be represented by the LEVEL-LOGIC-ZERO signal.

Refer to: CCITT Document, Vol. 3, Rec G702, Entry No. 8005

B. Examples

1. Figure 6-12 illustrates several typical SERIAL DATA waveforms, using voltages to carry digital information, including several BIP types. The illustrated waveforms are defined as follows:
- a. Basic RZ.

 LEVEL-LOGIC-ONE VOLTAGE 1 V,
 LEVEL-LOGIC-ZERO VOLTAGE 0 V, . . . ,
 PULSE-CLASS RZ,
 - b. Basic NRZ. Referred to as the NRZ code (ARINC 573) or the TWG/RCC NRZ-Level (or NRZ Change code). Described as in (a) but with the "NRZ" designation.
 - c. Bi-Phase-Level (or Split Phase, Manchester 11 + 180). Each bit period is divided into two sub-periods. The binary data is represented in the TRANS modifier fields as follows:
 - (1) A binary 1 by a sub 10
 - (2) A binary 0 by a sub 01

The waveform is described in ATLAS as:

 LEVEL-LOGIC-ONE VOLTAGE 1 V,
 LEVEL-LOGIC-ZERO VOLTAGE 0 V, . . . ,
 TRANS-LOGIC-ONE 10, TRANS-LOGIC-ZERO 01, . . . ,
 PULSE-CLASS BIP,
 - d. Harvard Bi-Phase (ARINC 573). A transition occurs at the start of each bit period. Each bit is represented as follows:
 - (1) A binary 1 by a second transition at the mid point of the bit period.
 - (2) A binary 0 by no second transition.

The waveform is described in ATLAS as:

 LEVEL-LOGIC-ONE VOLTAGE 1 V,
 LEVEL-LOGIC-ZERO VOLTAGE 0 V, . . . ,
 TRANS-LOGIC-ONE 01 OR 10,
 TRANS-LOGIC-ZERO 00 OR 11,
 TRANS-SYNC 01, ,
 PULSE-CLASS BIP,

6.0 PREAMBLE STATEMENTS

- e. Bi-Phase-Mark. A transition occurs at the start of each bit period. Each bit is represented as follows:

- (1) A binary 0 by a second transition at the mid point of the bit period.
- (2) A binary 1 by no second transition.

The waveform is described in ATLAS as:

```
LEVEL-LOGIC-ONE VOLTAGE 1 V,
LEVEL-LOGIC-ZERO VOLTAGE 0 V, ... ,
TRANS-LOGIC-ONE 00 OR 11,
TRANS-LOGIC-ZERO 01 OR 10,
TRANS-SYNC 01, .... ,
PULSE-CLASS BIP,
```

- f. Bi-Polar RZ (ARINC 575). Three voltage levels are defined and each bit period is subdivided into two sub-periods. Each bit is represented during the first sub-period as follows:

- (1) A binary 1 by the LEVEL-LOGIC-ONE level
- (2) A binary 0 by the LEVEL-LOGIC-ZERO level

The second sub-period contains the DC-OFFSET level.

```
DC-OFFSET 0 V, ... ,
LEVEL-LOGIC-ONE VOLTAGE 1 V,
LEVEL-LOGIC-ZERO VOLTAGE -1 V,
PULSE-CLASS RZ,
```

- g. Non-symmetric RZ. Same as (a), the Basic RZ, but with a duty cycle of 25 percent.

The waveform is described in ATLAS as:

```
DUTY-CYCLE 25 PC,
LEVEL-LOGIC-ONE VOLTAGE 1 V,
LEVEL-LOGIC-ZERO VOLTAGE 0 V, ... ,
PULSE-CLASS RZ,
```

- h. AMI - Three voltage levels need to be defined.

The waveform is described in ATLAS as:

```
LEVEL-LOGIC-ONE VOLTAGE 1 V,
LEVEL-LOGIC-ONE VOLTAGE -1 V,
LEVEL-LOGIC-ZERO VOLTAGE 0 V,
PULSE-CLASS AMI,
```

- i. HDBn - where 'n' is an unsigned integer number. As in AMI, three voltage levels need to be defined. Since n=1 the second consecutive binary '0' value in the data string is replaced by a +1 level because the polarity is the same as the preceding binary '1' level; marked by 'X' in Figure 6-12.

The waveform is described in ATLAS as:

```
LEVEL-LOGIC-ONE VOLTAGE 1 V,
LEVEL-LOGIC-ONE VOLTAGE -1 V,
LEVEL-LOGIC-ZERO VOLTAGE 0 V EXCEPT AT EVERY 2
CONSECUTIVE OCCURRENCES, ... ,
PULSE-CLASS HDB2,
```

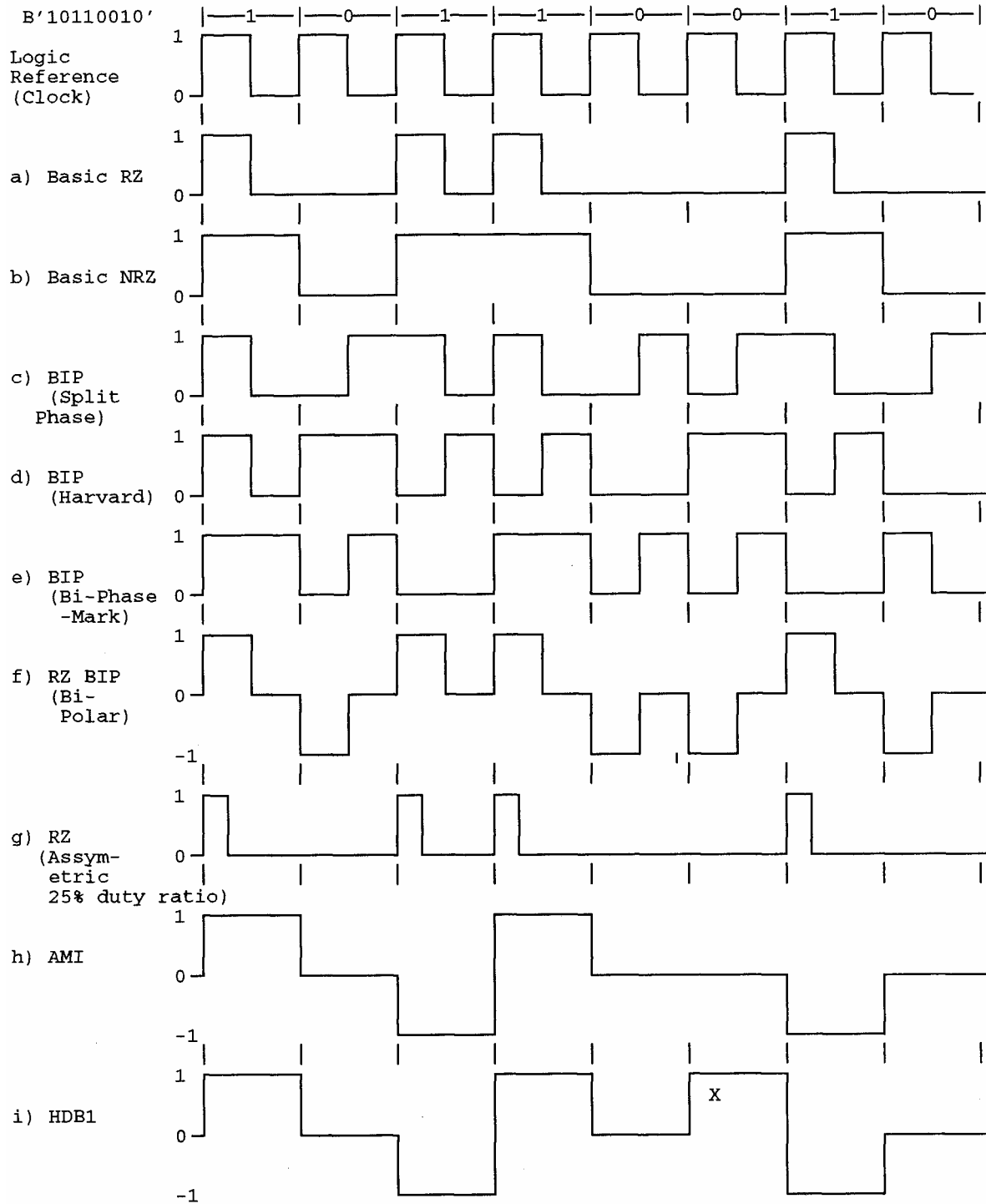
6.0 PREAMBLE STATEMENTS

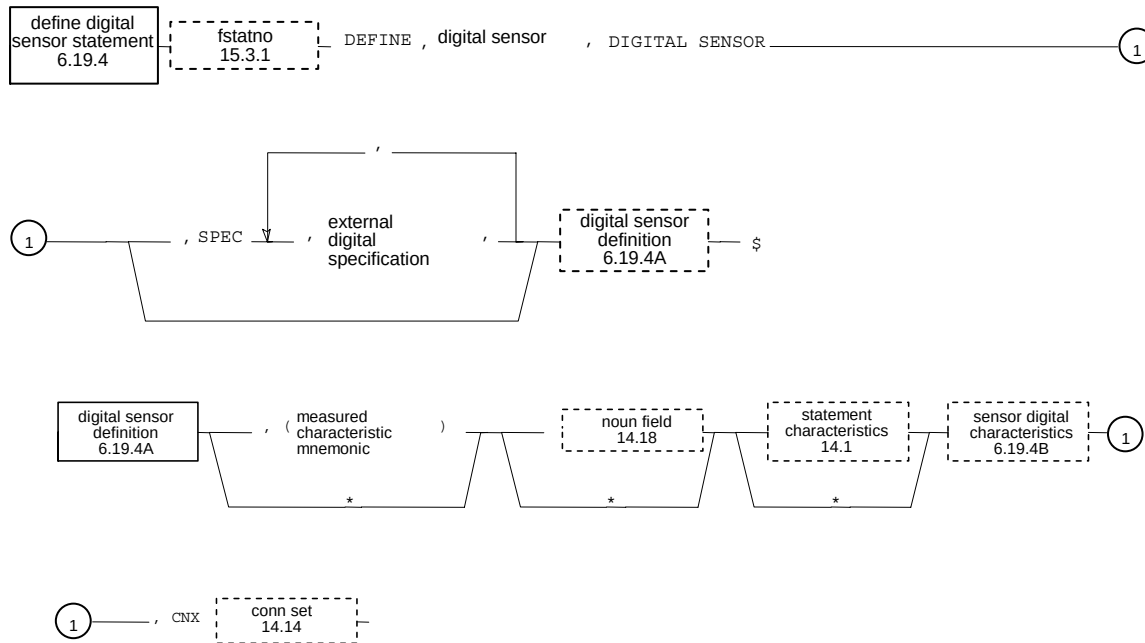
Figure 6-10 RZ, NRZ, and BIP Pulse Types for Digital Word B'10110010'

6.0 PREAMBLE STATEMENTS**6.19.4 <define digital sensor statement>****6.19.4.1 Function**

To establish a label, <digital sensor>, for a noun associated with a measured characteristic, a set of digital sensor characteristics and a set of connections to be used as a digital sensor. For each signal connection it allows for the definition of the signal corresponding to each logic state.

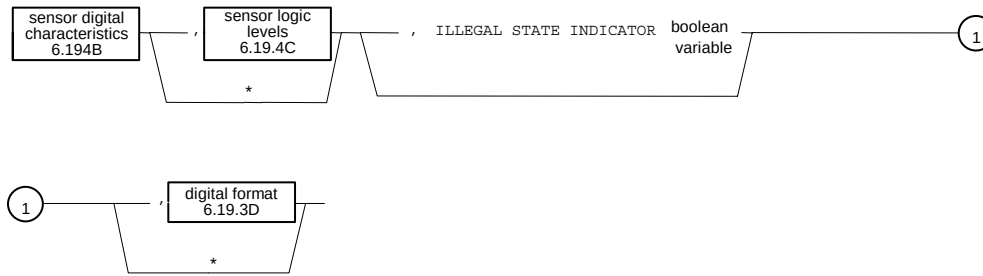
6.19.4.2 Formal Syntax

FSD / 882

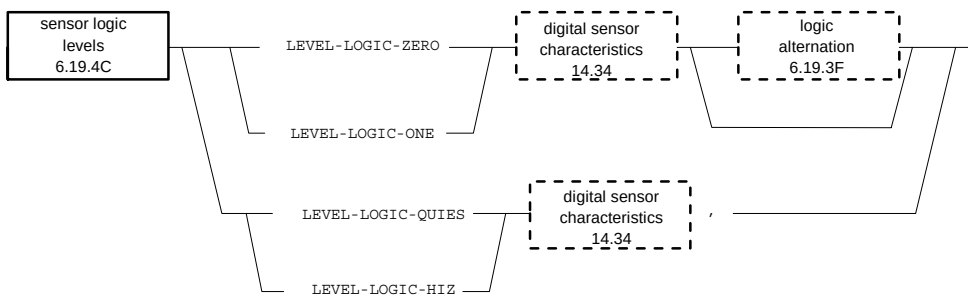
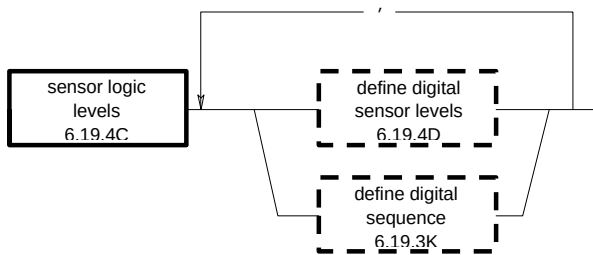
6.19.4.3 Syntax Diagram

Note: * This bypass is used when the relevant information is completely defined in the specification identified by the <external digital specification>.

6.0 PREAMBLE STATEMENTS



Note: * This by-pass is used when the relevant information is completely defined in the specification identified by the <external digital specification>.



6.19.4.4 Rules

1. For parallel sensors the order in which signal connections are written after CNX is significant. The first connection corresponds to the most significant bit.
2. The <external digital specification> option is used when the signal characteristics of a digital word are defined in an existing specification. The label, <external digital specification>, must identify the specification for the signal characteristics of the digital sensor.

If the information identified by <external digital specification> is either incomplete or inconsistent with the requirement, then information must be supplied explicitly by the appropriate fields in the <define digital sensor statement> which then take precedence over the information in the <external digital specification>.

6.0 PREAMBLE STATEMENTS

3. Explicit signal characteristics of LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO may be defined in terms of LEVELS, which are steady states that hold true throughout the bit period and for all instances of a given state. If the <pin descriptor>'s TRUE and COMPL are employed the value stated is that of the TRUE connection with respect to the COMPL connection.

Provision is made for the exceptional insertion of a different level. This technique is used to break up long chains of a given state in some digital transmission systems such as HDB.

4. When the PULSE-CLASS field is used, it must be consistent with the information included in the <digital logic levels> and/or the <external digital specification>. The SPEC subfield of the PULSE-CLASS field is used to identify additional pulse classes.
5. LEVEL-LOGIC-QUIES, when specified, refers to the state which exists between bits or between words. For bus systems, this is the neutral state when no data is present.
6. A named <digital sensor> can be individually referenced in a <sense statement> or a <prove statement>.
7. The term LEVEL-LOGIC-HIZ applies only to electrical systems, where LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO are defined in terms of voltages. Measurement of LEVEL-LOGIC-HIZ is a difficult test system engineering problem, but although few systems may have this capability, ATLAS allows this measurement to be specified. To achieve this the requirement for the definition of the characteristics of high impedance to be in terms of the <measured characteristic mnemonic> are relaxed. LEVEL-LOGIC-HIZ is defined in terms of the maximum current flows when the defined LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO levels are impressed on the test subject outputs. If only one LEVEL-LOGIC-HIZ is specified the allowed leakage current applies to both of the impressed logic states. If it is required to specify different leakages for the two impressed logic states, then the LEVEL-LOGIC-HIZ specification must be repeated and the position, which must be immediately following the LEVEL-LOGIC-ONE or LEVEL-LOGIC-ZERO field, defines to which state the leakage relates.
8. The initial state that will exist prior to encountering the first <sense statement> or <prove statement> will be LEVEL-LOGIC-HIZ, if defined, else LEVEL-LOGIC-QUIES, if defined, else LEVEL-LOGIC-ZERO.
9. When an ILLEGAL STATE INDICATOR is used the associated <boolean variable> will be set TRUE if the signals sensed by the <digital sensor> do not fall within the defined logic states expressed in the <evaluation field> of the <digital sensor characteristic>.

If only LEVEL-LOGIC-ONE and LEVEL-LOGIC-ZERO are defined, the LEVEL-LOGIC-HIZ state is assumed to be illegal and will cause the ILLEGAL STATE INDICATOR <boolean variable> to be set TRUE.

10. The BIT-RATE/BIT-PERIOD sub-field, when present, specifies the speed of bit transmissions for SERIAL logic signals.
11. For signals which utilize an independent clock source, the synchronizing event must be identified as an <event>. The beginning of every bit period of the serial data stream sensed by the <digital sensor> is then synchronized to this <event> via the SYNC BIT-TRANSITION TO EVENT branch.
12. If an <event> is specified for serial data clocking, the <max-time> field indicates the maximum time that the digital configuration is to wait, following a <sense statement> start or <prove statement> start, for the first occurrence of the <event>. If the <event> does not occur within this time limit, a MAX-TIME condition is set and the test resumes execution without completion of the <sense statement> or <prove statement>.
13. The TRANS-LOGIC-ONE and TRANS-LOGIC-ZERO fields are used to define transitions between the defined -ONE and -ZERO signal levels which correspond to a digital one and a digital zero respectively.

6.0 PREAMBLE STATEMENTS

The OR is available because some transmission systems permit a logic one or zero to be represented in two different ways depending on the start condition. These transitions occur at the mid-point of a data bit duration in Bi-Phase Signals.

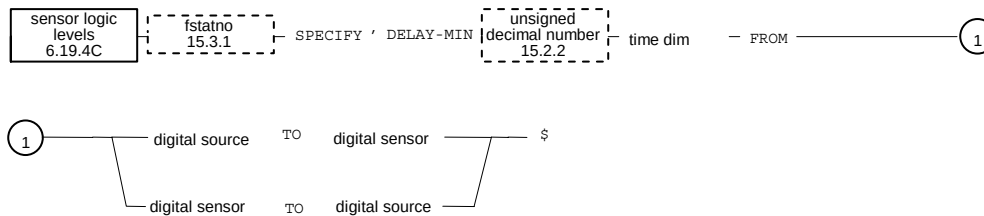
14. The TRANS-SYNC field is used to define that there must be a transition at the start of each word. A number pair following the key-word indicates the transition required prior to the first bit. If the keyword QUIES occurs after TRANS-SYNC it indicates that the signal is required to be at the Quiescent level prior to the initial transition of the word. The second entry indicates the level it acquires at the start of the word transmission. The TRANS-SYNC-BIT field is used to define that there must be a transition at the start of each bit period.
15. In statements employing the TRANS-LOGIC-ONE and TRANS-LOGIC-ZERO fields indicating that the logical information is carried by signal transitions, the -ONE and -ZERO suffixes can be used after the mnemonic in the <statement characteristics> field to indicate that the associated signal characteristic carries the digital information and assumes the stated value when the signal is a ONE or a ZERO respectively. If one of these suffixes is used the same mnemonic must occur in another field with the other suffix.
16. The SKEW-TIME MAX field defines the maximum skew between the various output ports employed by a parallel digital pattern that must be accepted by the sensor and be treated as being acceptably simultaneous.
17. SERIAL-MSB-FIRST and SERIAL-LSB-FIRST identify the digital sensor as being serial and identify respectively that the left-most bit or the right-most bit is to be received first.
18. The <def dig sensor levels> fields are used only when the TRANS-LOGIC-... options are used. They specify the acceptable values of the signal characteristics which determine whether the signal is at a one or a zero.
19. When using the noun LOGIC SIGNAL, noun modifiers having the same name as keywords cannot exist in the same statement.

6.0 PREAMBLE STATEMENTS6.19.5 <specify delay min statement>6.19.5.1 Function

To specify the minimum delay times between the actioning of a <digital source> and a <digital sensor> which must have been defined previously.

6.19.5.2 Formal Syntax

FSD / 943

6.19.5.3 Syntax Diagram6.19.5.4 Rule

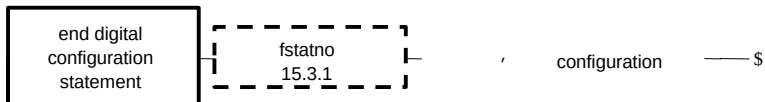
1. The <specify delay-min statement> can only be used as part of a <define configuration structure>.

6.19.6 <end digital configuration statement>6.19.6.1 Function

To terminate a <define configuration structure>.

6.19.6.2 Formal Syntax

FSD / 953

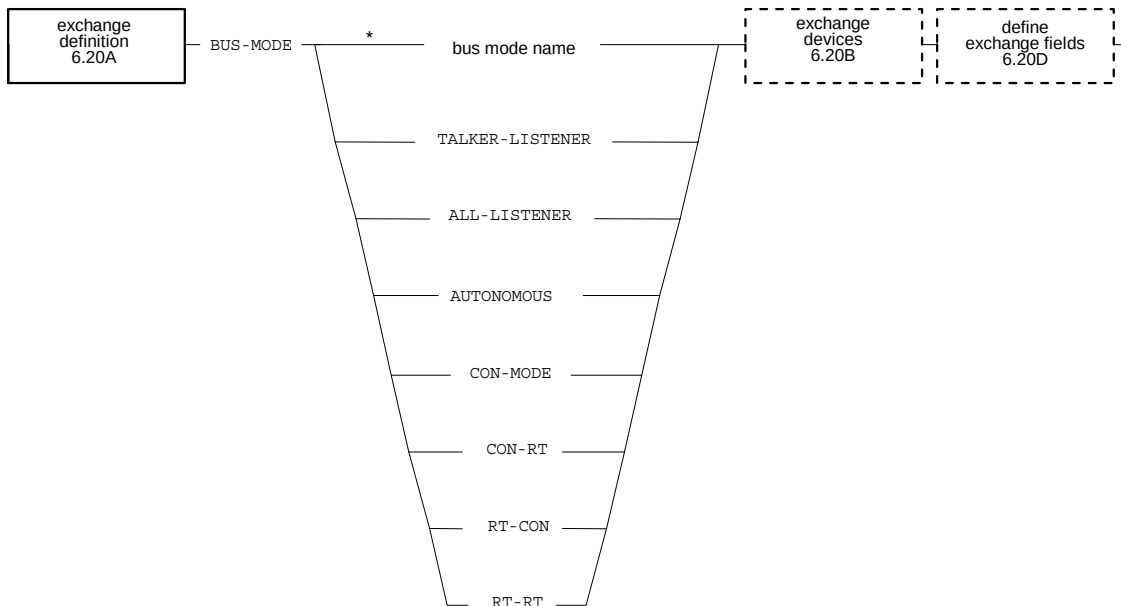
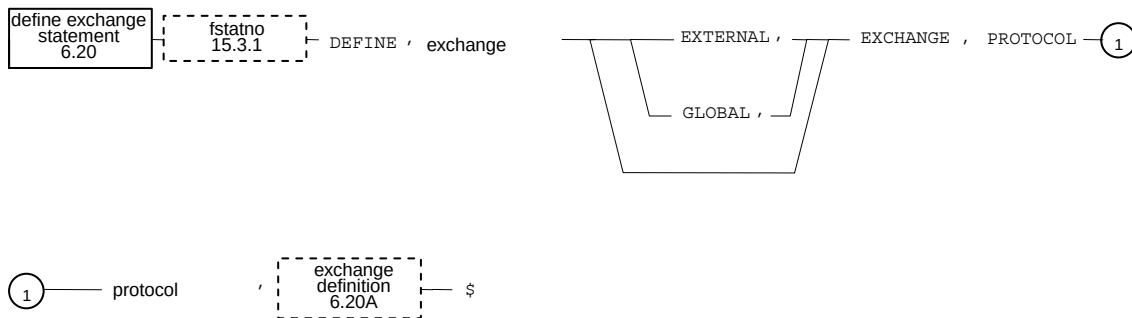
6.19.6.3 Syntax Diagram

6.0 PREAMBLE STATEMENTS**6.20 <define exchange statement>****6.20.1 Function**

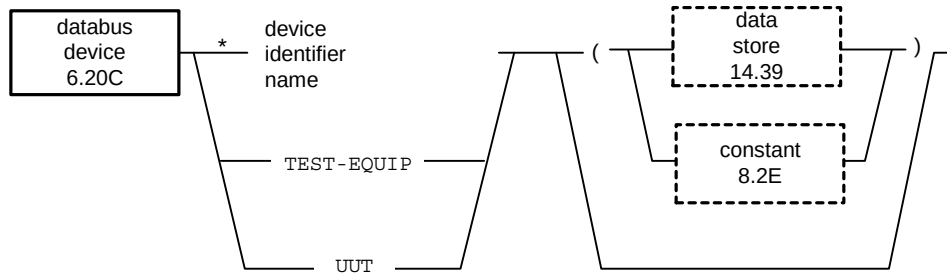
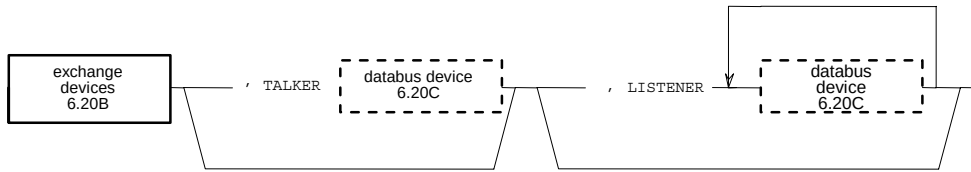
To establish an <exchange> label corresponding to a particular exchange (or transaction) on a databus.

6.20.2 Formal Syntax

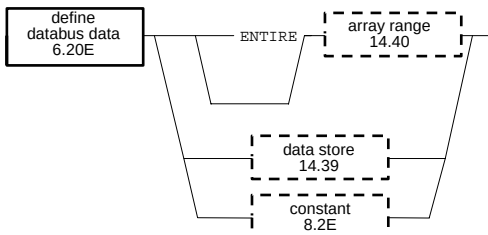
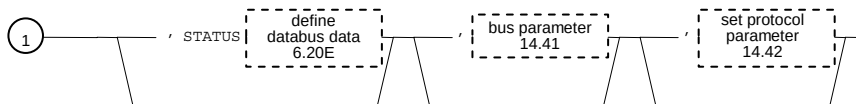
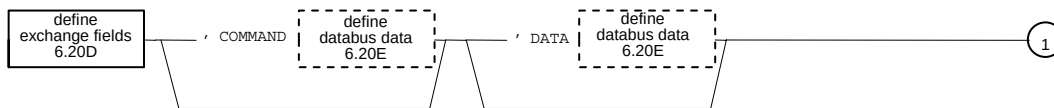
FSD / 958

6.20.3 Syntax Diagram

Note: * The <bus-mode-name> must be one associated with the particular <protocol> by its ESTABLISH statement.

6.0 PREAMBLE STATEMENTS

Note: * The device identifier name must have been established as a valid TALKER or LISTENER in the <establish statement> as appropriate for the <protocol>.



6.0 PREAMBLE STATEMENTS

6.20.4 Rules

1. The <define exchange statement> establishes and labels one <exchange> on a databus.
2. An <exchange> involves one block of information traveling on the databus. An <exchange> can involve control information, data, status information or a combination of two or all three.
3. The global <define exchange statement> uses the GLOBAL attribute. This may only appear in the preamble of an <atlas program structure>. The statement DEFINE, <exchange>, EXTERNAL may only appear in the preamble of an <atlas module structure>. Each label defined by an EXTERNAL <exchange> definition statement must have a corresponding definition in a GLOBAL <exchange> definition statement which encompasses the requirements of the EXTERNAL <exchange> definition statement.
4. **PROTOCOL field** - The <protocol> references the previously established <protocol> which controls the databus. The specifications referenced by the <establish protocol statement> must include all the information required to select the databus with the correct connections. They must also include the information necessary to control operations, to transfer data, to handle synchronization and to handle timing. The <define exchange statement> supplies the required mode of bus operation, the bus devices involved and any data, command or status information. If the GLOBAL or EXTERNAL attribute is used in the <define exchange statement> then the <establish protocol statement> for the <protocol> must also have the same GLOBAL or EXTERNAL attribute.
5. The BUS-MODE, TALKER, LISTENER, COMMAND, DATA and STATUS fields must be consistent with a single <exchange model> within the <establish protocol statement> of the referenced <protocol>.
6. **BUS-MODE field** - the BUS-MODE field identifies the mode of bus operation for the exchange. It is used to indicate which of the possible operating modes of the bus is to be used. It may be one of the default entries listed below or a <bus mode name> label. Whichever is employed must be fully defined as it applies to that bus in the set of <external bus specification>s referenced in the <establish protocol statement> and must be listed in the <exchange model> section of this <establish protocol statement>.

The standard bus modes are:

CON-RT	Bus controller to remote terminal transfers. The controller is talking and one device is listening. The TALKER field is not used
RT-CON	Remote terminal to bus controller transfers. One device is talking and the controller is listening. The LISTENER field is not used.
RT-RT	Remote terminal to remote terminal transfers. One device is talking and another device is listening.
CON-MODE	Transfer from bus controller to remote terminal(s). The controller is talking to all devices. The TALKER field is not used.
TALKER-LISTENER	The talker and listener(s) are explicitly designated.
ALL-LISTENER	One talker is explicitly designated and all other devices are listening. The LISTENER field is not used.
AUTONOMOUS	No central bus controller.

6.0 PREAMBLE STATEMENTS

7. TALKER field - The TALKER field identifies the device which will talk (emit data) when the <exchange> transfers data (as opposed to control signals). For this purpose command and status information is a special form of data.

If the transaction transfers data, command or status information, the TALKER field is normally required. The only exception is when a particular bus mode defines the TALKER by default.

The optional <constant> or <data store> permits addressing and sub-addressing within the device. The type of the <constant> or <data store> must be of the <type> appearing in the corresponding <exchange model>.

The <data store> must reference a previously declared variable. The value of the <data store> will be evaluated when any statement referencing the defined <exchange> is executed.

The TALKER field must contain one of the keywords or enumeration elements listed in the corresponding TALKER field in the <establish protocol statement>. The keyword TEST-EQUIP indicates that the TALKER is the Test Equipment. The keyword UUT indicates that the TALKER is the Unit Under Test.

8. LISTENER field - The LISTENER field establishes the devices which will listen. They will receive data (as opposed to control signals) transferred during the <exchange>.

The number and selection of listeners must be consistent with the bus mode selected for the <exchange>.

If the transaction transfers data, command or status information, the LISTENER field is normally required. The only exception is when a particular bus mode defines the LISTENERS by default.

The optional <constant> or <data store> permits addressing and sub-addressing within the device. The <type> of the <constant> or <data store> must be consistent with the corresponding <exchange model>.

The <data store> must reference a previously declared variable. The value of the <data store> will be evaluated when any statement referencing the defined <exchange> is executed.

The LISTENER field must contain at least one of the keywords or enumeration elements listed in the corresponding LISTENER field in the <establish protocol statement>. The keyword TEST-EQUIP indicates that the TALKER is the Test Equipment. The keyword UUT indicates that the TALKER is the Unit Under Test.

9. COMMAND field - The COMMAND field occurs if the selected bus mode requires a command word to be transmitted. The use of the <constant> branch is restricted to the specification of a simple type in the associated <exchange model> of the <establish protocol statement>. The simple types are restricted to: BIT, STRING-OF-BIT, CHAR, STRING-OF-CHAR, DECIMAL, INTEGER and enumeration types. The <data store> branch, in addition to the above simple types, permits the specification of a variable of type RECORD. This RECORD can contain any combination of the above simple types but must not contain any RECORD or multi-dimensional ARRAY types. The type of the variable must be identical to the type defined in the relevant <exchange model> from the <establish protocol statement>.
10. DATA field - The DATA field occurs if the selected bus mode requires data words to be transmitted. The use of the <constant> branch is restricted to the specification of a simple type in the associated <exchange model> of the <establish protocol statement>. The simple types are restricted to: BIT, STRING-OF-BIT, CHAR, STRING-OF-CHAR, DECIMAL, INTEGER and enumeration types.

The <data store> branch, in addition to the above simple types, permits the specification of a variable of type RECORD. This RECORD can contain any combination of the simple types: BIT, STRING-OF-BIT, CHAR, STRING-OF-CHAR, DECIMAL and INTEGER and must not contain any RECORD or multi-dimensional ARRAY types.

6.0 PREAMBLE STATEMENTS

An <array range> of RECORD as defined above or of the simple types is allowed. If <array range> is used with the keyword ENTIRE, all the specified words in the <array range> are sent every time the exchange is invoked. If used without the keyword ENTIRE, consecutive transmissions of the specific <exchange> will take their values from successive elements of the array on an auto-incrementing basis.

The type of the variable in all cases must be identical to the type defined in the relevant <exchange model> from the <establish protocol statement>.

11. STATUS field - The STATUS field occurs if the selected bus mode requires a status word to be transmitted. The use of the <constant> branch is restricted to the specification of a simple type in the associated <exchange model> of the <establish protocol statement>. The simple types are restricted to: BIT, STRING-OF-BIT, CHAR, STRING-OF-CHAR, DECIMAL, INTEGER and enumeration types.

The <data store> branch, in addition to the above simple types, permits the specification of a variable of type RECORD. This RECORD can contain any combination of the above simple types but must not contain any RECORD or multi-dimensional ARRAY types. The type of the variable must be identical to the type defined in the relevant <exchange model> from the <establish protocol statement>.

12. Talker and listener are terms used with the exchange of information. They are unrelated to the concepts of master and slave associated with the control of many databuses. Appropriate bus modes permit control-only <exchange>s. This case might require the DATA field although the TALKER and LISTENER fields are unused.
13. Each bus mode requires a predetermined set of fields. The BUS-MODE determines how these fields are converted into command, status and data transmission information on the databus. All fields which are required by the particular bus mode must be included in the <define exchange statement>. Fields which a bus mode cannot use may occur. If they do occur the data included in the field is not transmitted on the databus.
14. Command, Status and Data information have to be included in a <define exchange statement> either for transmission by the TEST-EQUIP or by another device. If a subsequent <do exchange statement> requires the associated data to be transmitted by the TEST-EQUIP the value to be transmitted will be extracted from the fields of the <define exchange statement>. These fields must be completely defined in the <define exchange statement>.
15. Each <bus parameter> in the corresponding <establish protocol statement> must be repeated. The optional <dim>, if used, must be the same as the <dim> of the <establish protocol statement>. The value of the <constant> or <data store> must be within the limits defined by the corresponding field in the <establish protocol statement>. The <type> of the <constant> or <data store> must be compatible with the <bus parameter> in the <establish protocol statement>.
16. Each <protocol parameter> must be repeated from the <establish protocol statement>. If <constant> or <data store> is used, its type must be the same as the type defined in the corresponding <establish protocol statement>.

6.20.5 Notes and Examples

Section 6.11.5 describes the fundamental philosophy of databus testing.

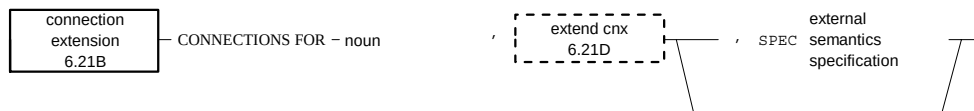
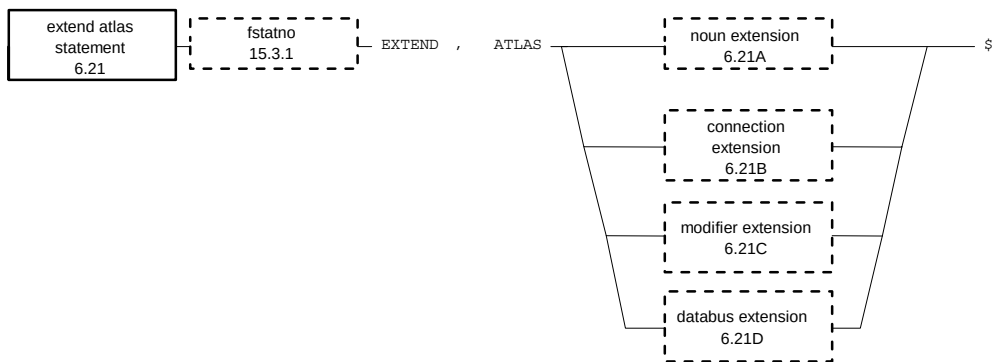
Section 13.1.5 includes examples of databus testing.

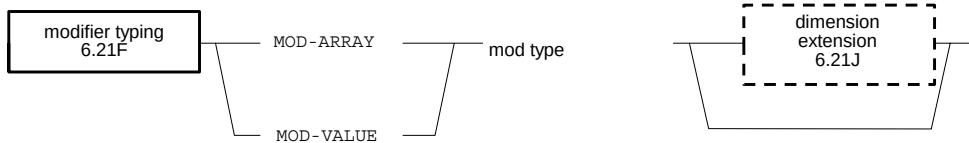
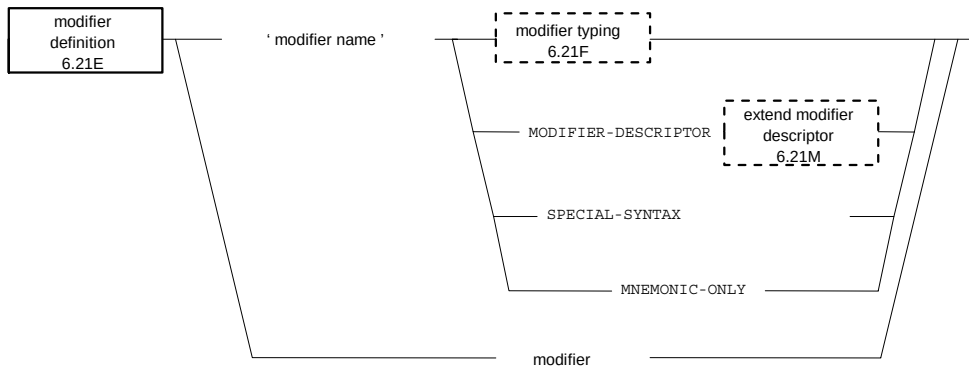
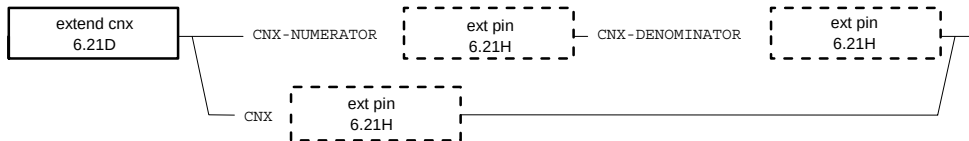
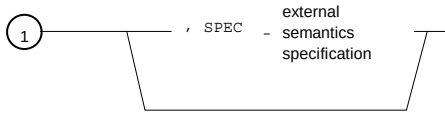
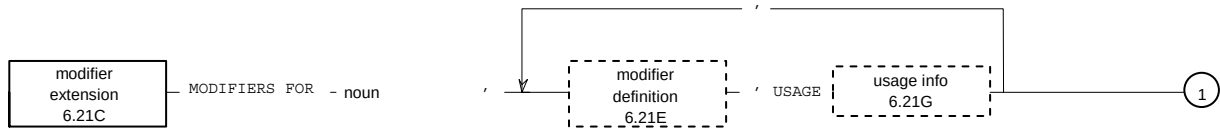
6.0 PREAMBLE STATEMENTS**6.21 <extend atlas statement>****6.21.1 Function**

To provide facilities for adding nouns, modifiers and connections for signals and for adding bus parameters, protocol parameters, bus modes and test equipment roles to the ATLAS language for use in a particular <atlas program structure> or <atlas module structure> in accordance with the Extensibility Management Procedures in Section 1.5. and to reference the external semantics of such extensions.

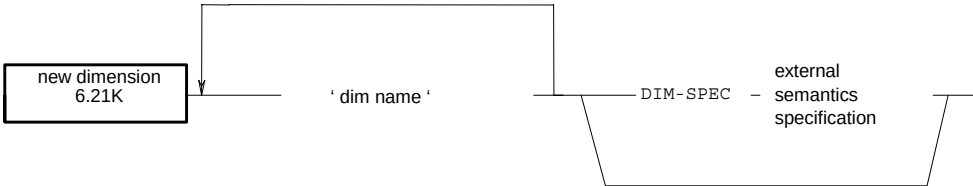
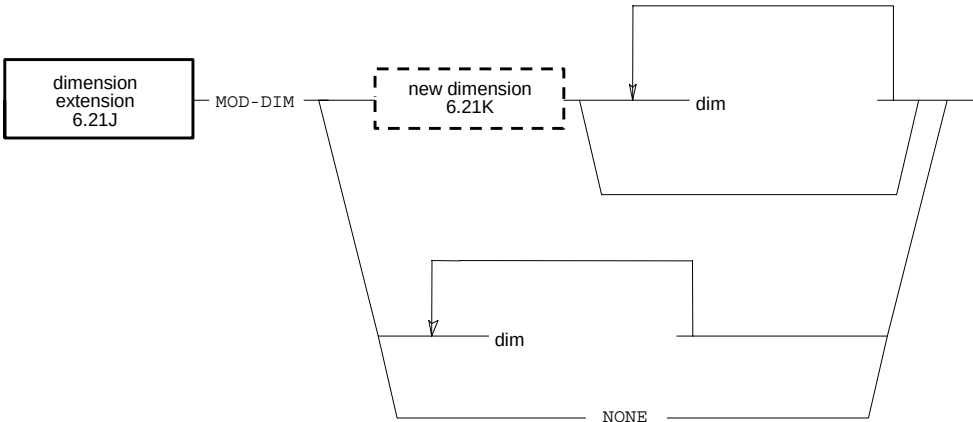
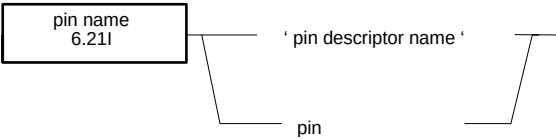
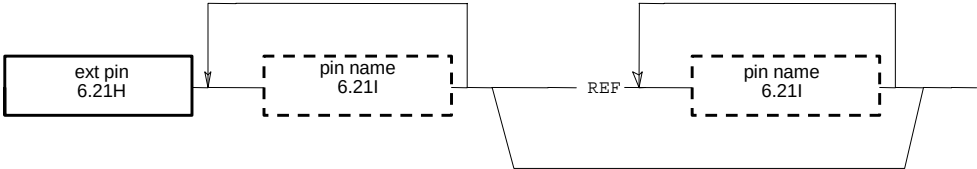
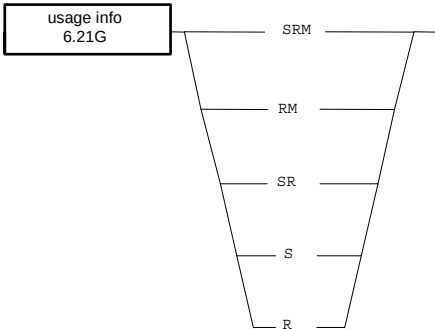
6.21.2 Formal Syntax

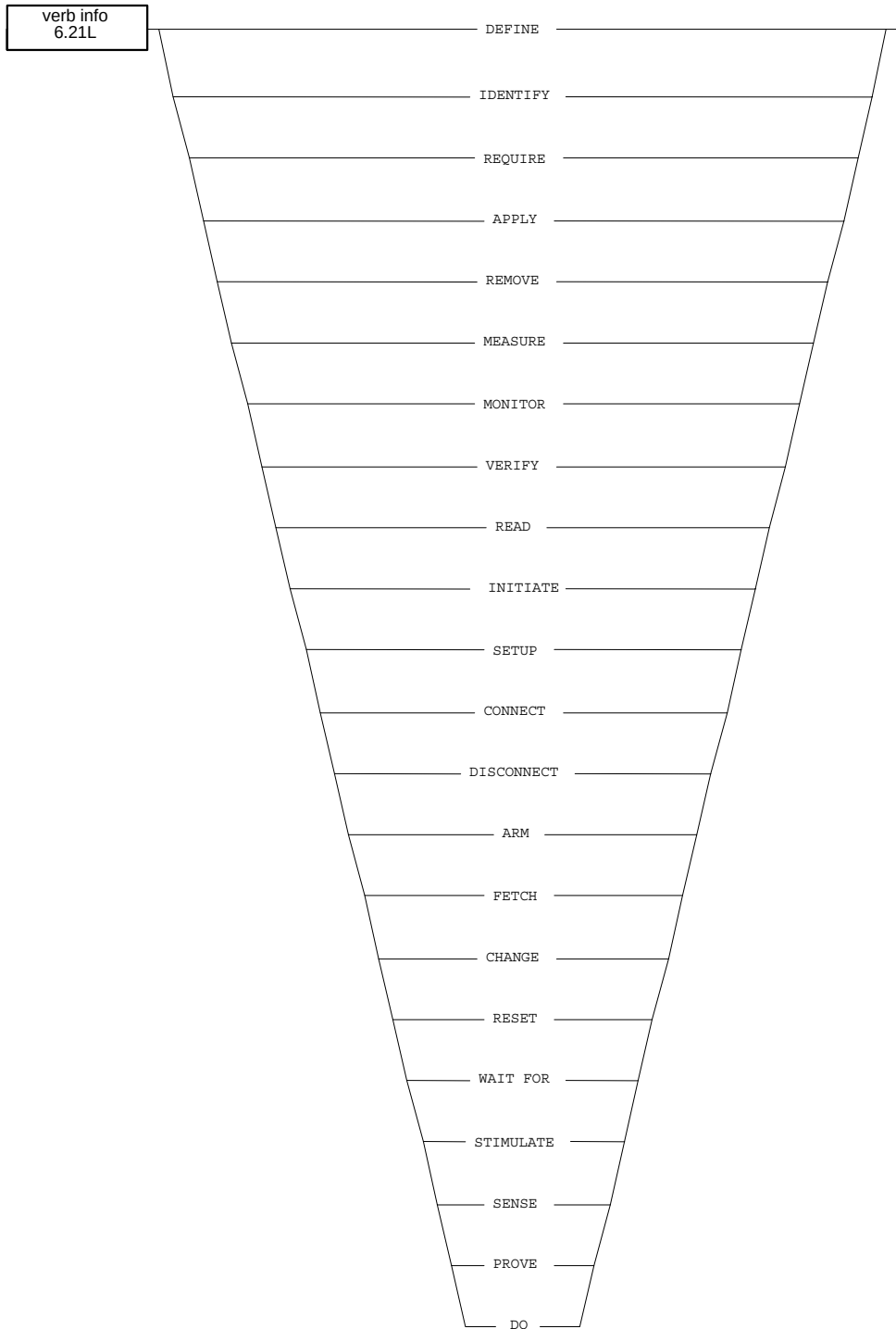
FSD / 1020

6.21.3 Syntax Diagram

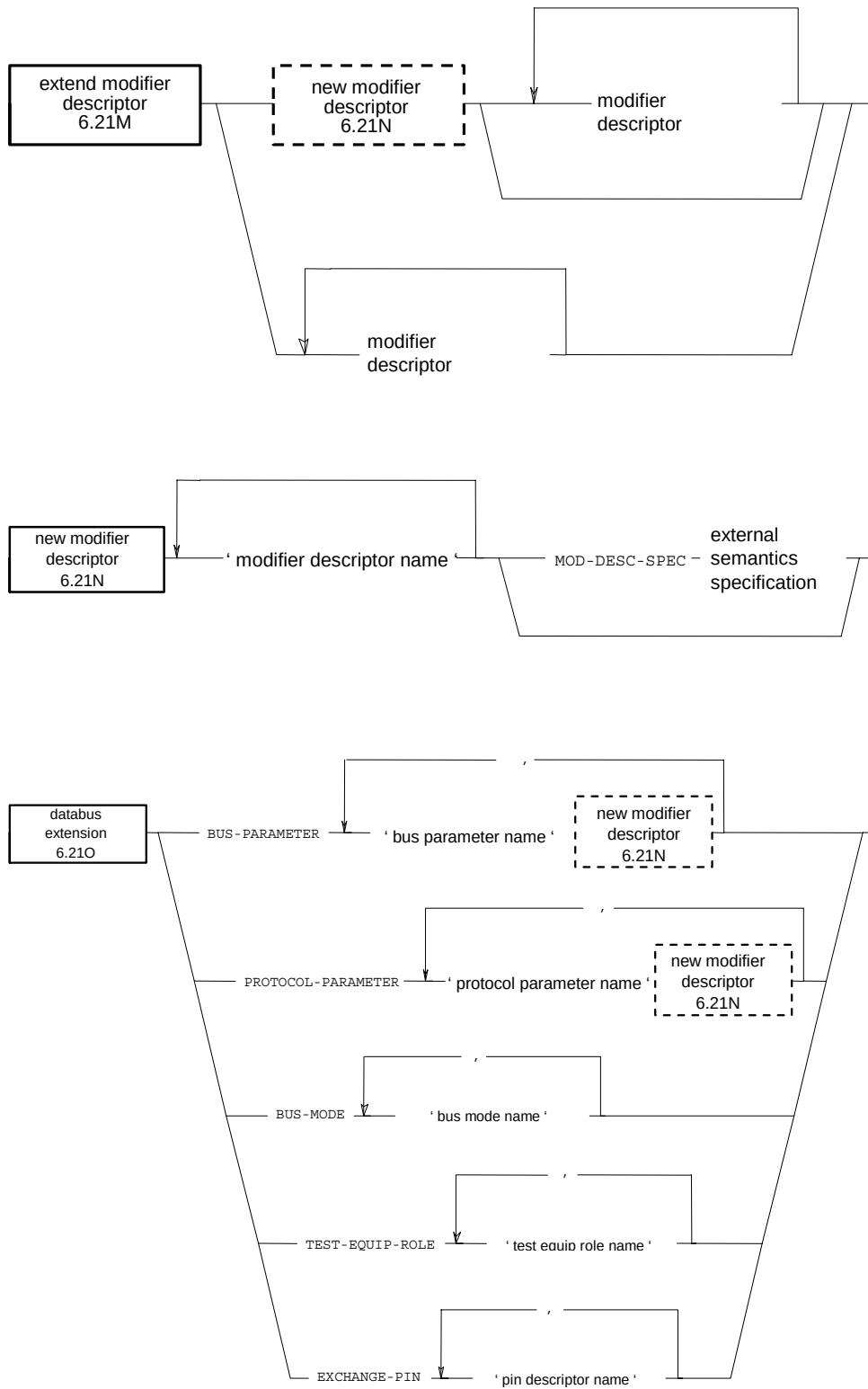
6.0 PREAMBLE STATEMENTS

6.0 PREAMBLE STATEMENTS

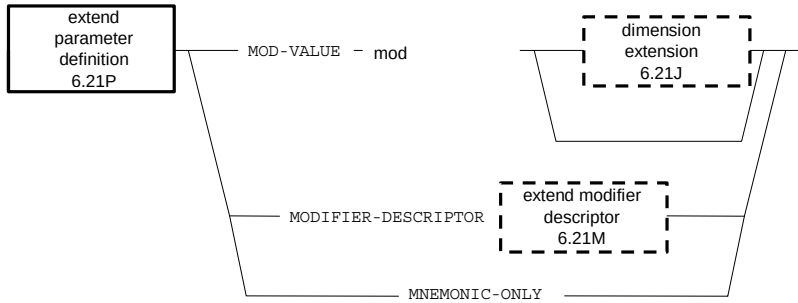


6.0 PREAMBLE STATEMENTS

6.0 PREAMBLE STATEMENTS



6.0 PREAMBLE STATEMENTS



6.21.4 Rules

6.21.4.1 Rules for all <extend atlas statement>

1. An <extend atlas statement> must not be used to define an alternative method of writing test requirements which can be written in this ATLAS subset without recourse to extensibility.
2. The names of new nouns, modifiers, pin descriptors and databus related vocabulary must not be ATLAS reserved words or previously introduced noun, modifier, pin descriptor or databus related names.
3. The names of new nouns, modifiers, pin descriptors and databus related vocabulary are written enclosed in apostrophes when they are first introduced by an <extend atlas statement>. Thereafter they are written without apostrophes in the normal ATLAS form for nouns, modifiers and databus related keywords.
4. New nouns, modifiers and pin descriptors can be used in exactly the same way as any of the permanent nouns and modifiers of Chapters 16 and 17 respectively throughout the <atlas program structure> or <atlas module structure> in which the language element was introduced.
5. <external semantics> is a text string that fully identifies a specific document in which the meaning of the extension in the context of the related test discipline is defined.
6. The <extend atlas statement> can only be used to extend the semantics for signal oriented syntax.
7. All verbs that reference an extended noun must be explicitly listed.

6.21.4.2 Rules for EXTEND ATLAS NOUNS Statements

1. The <noun name> must be unique to the entire <atlas test specification structure> and cannot be the same as any ATLAS reserved word.
2. <verb info> identifies those verbs with which the extended noun can be used.
3. Every modifier and connection to be used with a newly defined ATLAS noun must be defined and referenced to the new noun in an EXTEND ATLAS MODIFIERS or EXTEND ATLAS CONNECTIONS statement.

6.21.4.3 Rules for EXTEND ATLAS MODIFIERS Statements

1. The <noun> must be either the name of a permanent noun from Chapter 16 or the name of a new noun in a previous <extend atlas statement>.
2. A <modifier name> which is enclosed in single quotation marks must be the name of a new modifier defined in the current <extend atlas statement>. A new modifier introduced by a previous <extend atlas statement> can be introduced through the <modifier mnemonic> path by including it without single quotation marks.

6.0 PREAMBLE STATEMENTS

3. Each <modifier name> must be composed as described in Section 15.10.9.1, must have a unique <modifier definition> and must have a unique specification. If new modifiers require the use of either prefixes or suffixes, then each of these must be treated as a separate modifier.
4. USAGE is entered in accordance with the usage codes of Chapter 16.0 (Modifier Usage).
5. MOD-VALUE is used to designate a single-valued modifier as specified in Chapter 16.0 Table 16-0, items 1 through 1.5. MOD-ARRAY is used to designate a multi-valued or list-type modifier as specified in Chapter 16.0, Table 16-0 items 2 through 2.4. MNEMONIC-ONLY designates non-valued modifiers as specified in Chapter 16.0, Table 16-0 items 3 and 4.

SPECIAL-SYNTAX is used to designate those modifiers that require special unique diagrams to define their syntax.

6. <external syntax> is a text string that fully identifies a specific document in which the meaning of the extension in the context of the related test discipline, is defined. Such a referenced document must be a standard produced by an organization that issues standards for that discipline. If no such standard exists, it may be a locally produced document that is made available and is agreed for use on a particular project. Any characters except for the dollar sign and apostrophe may be used in this text string which is always enclosed in apostrophes.
7. The SPEC field may be omitted only if the associated noun is a new noun and its associated SPEC field contains a reference to semantic descriptions relevant to the new modifier being introduced.
8. The <dim name> may not be a reserved ATLAS word.
9. If <dim> is specified the <dim> must be one of those listed in Section 15.8, Table 15-1 or must be a new dimension defined in a previous <extend atlas statement>.
10. The DIM-SPEC field may be omitted only if the associated noun is a new noun and its associated SPEC field contains a reference to semantic descriptions relevant to the <dim name>.
11. When the <extend atlas statement> is used to associate an ATLAS modifier or a previously introduced <modifier name> with a new noun, the modifier may not be redefined.

An ATLAS modifier is associated with the new noun by use of the <modifier mnemonic> path.

A previously introduced <modifier name> is associated with the new noun by use of the <modifier name> path which has no single quotation marks.

12. The <modifier descriptor name> must be a modifier descriptor which can be associated with the particular modifier. The <modifier descriptor name> may not be an ATLAS reserved word.
13. The MOD-DESC-SPEC field may be omitted only if the associated noun is a new noun and its associated SPEC field contains reference to semantic descriptions relevant to the <modifier descriptor name>.

6.0 PREAMBLE STATEMENTS

6.21.4.4 Rules for EXTEND ATLAS CONNECTIONS Statements

1. The <noun> must be either the name of a permanent noun from Chapter 16 or the name of a new noun in a previous <extend atlas statement>.
2. Each <pin descriptor name> must be composed as described in Chapter 15, but it may not be a reserved ATLAS word.
3. The SPEC field may be omitted only if the associated noun is a new noun and its associated SPEC field contains a reference to semantic descriptions relevant to the new pin descriptor(s).

6.21.4.5 Rules for EXTEND BUS-PARAMETERS Statements

1. Every <bus parameter name> used in the <atlas test specification structure> must appear in an <extend atlas statement>. A <bus parameter name> is used where none of the standardized keywords associated with BUS-PARAMETER in the <define exchange statement> is applicable to a parameter of the databus which needs to be defined to produce a complete test program. The mod-type and <dimension extension> establish the number type and dimensions associated with the associated bus characteristic. MNEMONIC-ONLY indicates that the parameter has no associated numeric or signal value.

6.21.4.6 Rules for EXTEND PROTOCOL PARAMETERS Statements

1. Every <protocol parameter name> used in the <atlas test specification structure> must appear in an <extend atlas statement>. A <protocol parameter name> is used where none of the standardized keywords associated with PROTOCOL-PARAMETER in the <define exchange statement> is applicable to a parameter of the databus protocol which needs to be defined to produce a complete test program. The mod-type and <dimension extension> establish the number type and dimensions associated with the associated bus characteristic. MNEMONIC-ONLY indicates that the parameter has no associated numeric or signal value.

6.21.4.7 Rules for EXTEND BUS MODES Statements

1. Every <bus mode name> used in the <atlas test specification structure> must appear in an <extend atlas statement>. A <bus mode name> is used where none of the standardized keywords associated with BUS-MODE in the <define exchange statement> is applicable to an operational mode of the databus which needs to be used to produce a complete test program.

6.21.4.8 Rules for EXTEND TEST EQUIP ROLES Statements

1. Every <test equipment role name> used in the <atlas test specification structure> must appear in an <extend atlas statement>.

A <test equip role name> is used where none of the standardized keywords associated with TEST-EQUIP-ROLE in the <do exchange statement> is applicable to a role the test equipment has to fulfill during databus testing to produce a complete test program.

6.21.4.9 Rules Applicable to EXTEND EXCHANGE-PIN Statements

1. Every <pin descriptor name> used in the <atlas test specification structure> must appear in an <extend atlas statement>. A <pin descriptor name> is used where the standard <pin descriptors> described in Section 14.14 (<conn set>) are not applicable to the databus.
2. <pin descriptor name>s introduced by this option can only be referenced in an <establish protocol statement> and its associated <enable exchange configuration statement>s, <connect exchange configuration statement>s and <disconnect exchange configuration statement>s.

6.0 PREAMBLE STATEMENTS**6.21.5 Notes and Examples**

No Specification is referenced in the case where EXTEND is used to add vocabulary which is used with a databus because there is a requirement that every item of vocabulary used with a databus, whether or not it is introduced by using EXTEND, must be fully defined in a specification which is referenced from the ESTABLISH statement associated with a particular databus installation. Consequently, reference to a specification in the EXTEND statement would introduce an unnecessary redundancy.

The textual references in the following statements are imaginary and are supplied as 'fill' characters in the appropriate fields. The new noun RATTLE is nearly synonymous with the existing noun VIBRATION.

```

000100 EXTEND, ATLAS NOUNS 'RATTLE',
        FOR VERBS APPLY/MEASURE/VERIFY,
        SPEC 'IMAGINARY SPEC 626-5 PARA 16.54' $

01 EXTEND, ATLAS MODIFIERS FOR RATTLE, 'RATTLE-AMPLITUDE',
        MOD-VALUE R, MOD-DIM MILS, USAGE SRM,
        SPEC 'IMAGINARY SPEC 626-5 SECT 17' $

02 EXTEND, ATLAS MODIFIERS FOR RATTLE, 'RATTLE-RATE',
        MOD-VALUE R, MOD-DIM MILS/SEC, USAGE SRM,
        SPEC 'IMAGINARY SPEC 626-5 SECT 17' $

03 EXTEND, ATLAS MODIFIERS FOR RATTLE, 'FREQ', USAGE S,
        SPEC 'ARINC SPEC 626-2 SECT 17' $

04 EXTEND, ATLAS MODIFIERS FOR RATTLE, 'SYNCH',
        SPECIAL-SYNTAX 'IMAGINARY ARINC SPEC 626-5',
        USAGE SR,
        SPEC 'IMAGINARY SPEC 626-5 SECT 17' $

05 EXTEND, ATLAS CONNECTIONS FOR RATTLE,
        CNX 'MOUNT-1' 'MOUNT-2' 'STEADY',
        SPEC 'CODE OF PRACTICE-SHAKER' $

.
.

000150 REQUIRE, 'VIBRATOR', SOURCE, RATTLE,
        CONTROL
        RATTLE-AMPLITUDE RANGE 1 MILS TO 10 MILS BY 1 MILS,
        FREQ RANGE 1 HZ TO 5 HZ BY 1 HZ,
        CAPABILITY,
        SYNCH . . . . .,
        CNX MOUNT-1 MOUNT-2 STEADY $

.
.

000200 IDENTIFY, EVENT 'BEARING- NOISE' AS .....$

.
.

E100000 COMMENCE, MAIN PROCEDURE $

.
.

123456 APPLY, RATTLE USING 'VIBRATOR', RATTLE-AMPLITUDE 5 MILS,
        FREQ 3 HZ, SYNCH TO EVENT 'BEARING-NOISE',
        CNX MOUNT-1 WIDGET-TOP MOUNT-2 WIDGET-BOTTOM
        STEADY WIDGET-FLEXURE $

```

6.0 PREAMBLE STATEMENTS

6.22 <define exchange configuration statement>

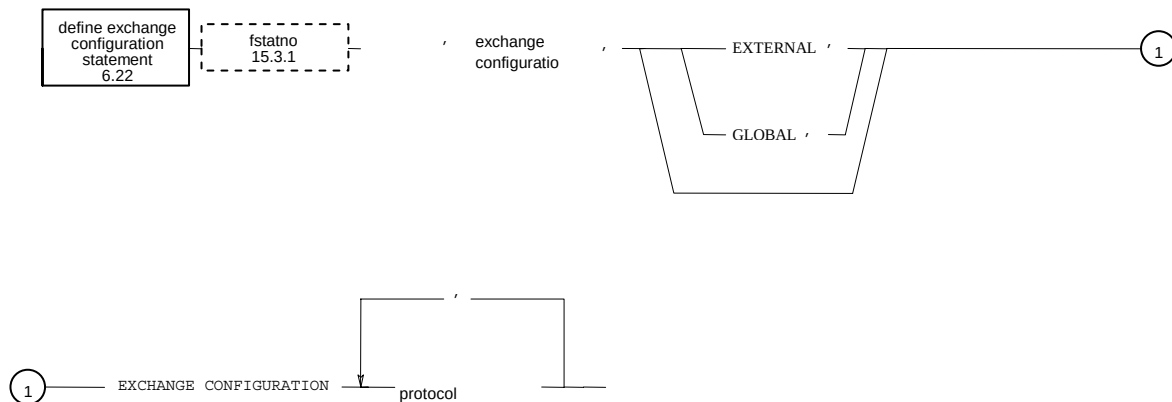
6.22.1 Function

To define a set of databuses which are employed simultaneously in a <do exchange statement>.

6.22.2 Formal Syntax

FSD / 1189

6.22.3 Syntax Diagram



6.22.4 Rules

1. The <define exchange configuration statement> defines a set of databus <protocol>s which are employed simultaneously in a <do exchange statement>. There is a <define exchange configuration statement> for each combination of protocols.
2. Every <do exchange statement>, <enable exchange configuration statement>, <connect exchange configuration statement>, <disconnect exchange configuration statement>, <disable exchange configuration statement>, <update exchange statement>, and <fetch exchange configuration statement> in an <atlas test specification structure> must reference a defined <exchange configuration>.
3. A second <define exchange configuration statement> is necessary if two independent <do exchange statement>s have to be active simultaneously. Simultaneous <do exchange statement>s must not reference the same <define exchange configuration statement>.
4. If there are two or more <define exchange configuration statement>s some <protocol>s may appear in more than one of them. If they do so, it is an error for a <protocol> to be active in two <exchange configuration>s concurrently.
5. A <protocol> becomes active when it is referenced in an <enable exchange configuration statement> and remains active until the associated <exchange configuration> is referenced in a <disable exchange configuration statement>.
6. The GLOBAL attribute may only appear in the main <program preamble structure> and the <exchange configuration> must then be unique to the entire <atlas test specification structure>.

6.0 PREAMBLE STATEMENTS

7. The EXTERNAL attribute may only appear in a <module preamble structure>. There must be a corresponding GLOBAL <exchange configuration> in the main <program preamble structure> and the list of <protocol>s must be identical in the two definitions.
8. Each <protocol> must be unique and refer to an <establish protocol statement> within the same <atlas program structure> or < atlas module structure>. The use of the GLOBAL and EXTERNAL attribute must be the same for the <define exchange configuration statement> and each of its <establish protocol statement>s.

7.0 PROCEDURAL STATEMENTS

7.1 <main procedural structure>

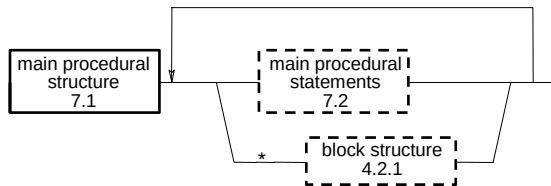
7.1.1 Function

To define the sequence of test statements and structures that define either the testing required by a UUT or the testing relevant to a structure within the UUT test requirement.

7.1.2 Formal Syntax

FSD / 1200

7.1.3 Syntax Diagram



Note: * This path is for <atlas program structure>, <procedure body> and <function body> only.

7.2 <main procedural statements>

7.2.1 Function

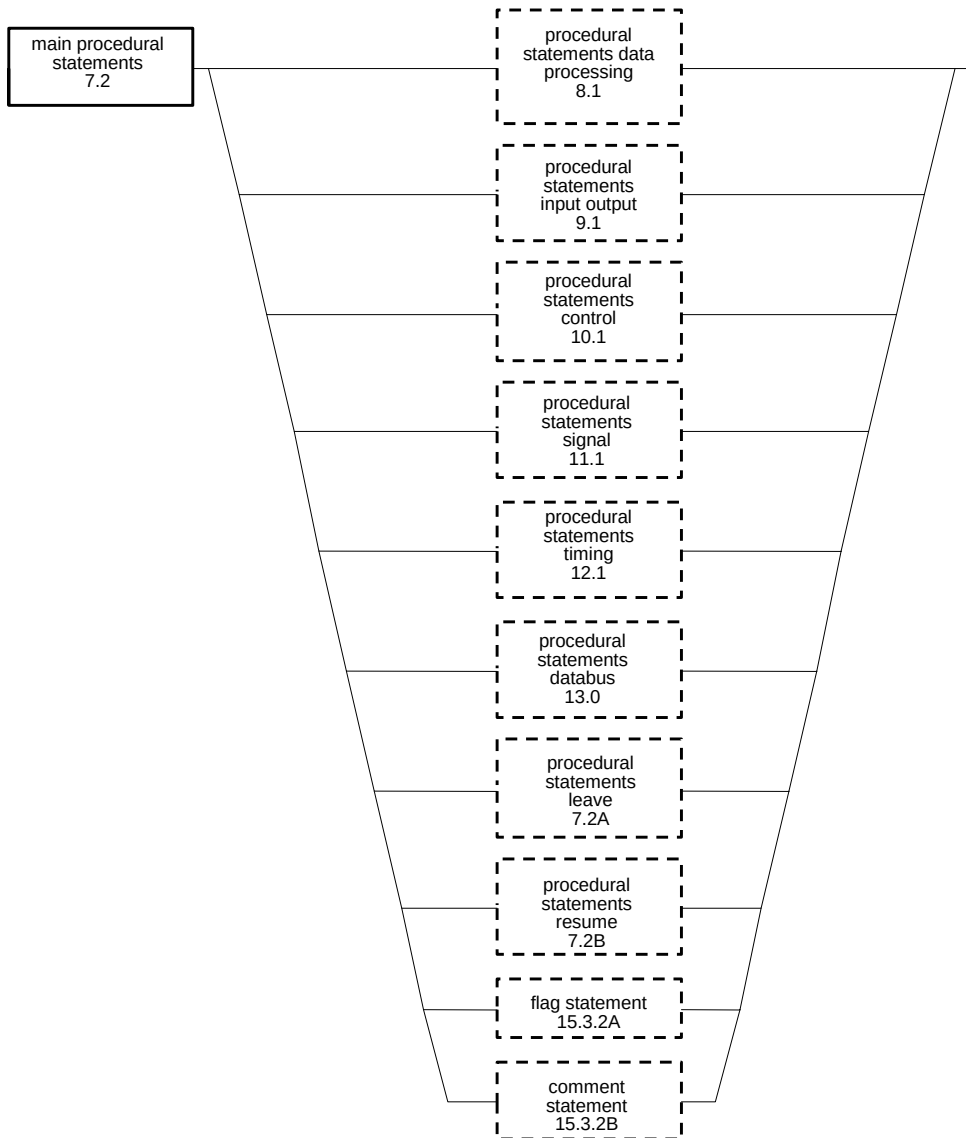
The <main procedural statements> are a series of statements or structures, each of which describes a portion of the required test which must be completed prior to the next statement or structure. Each statement or structure implies an instruction to proceed to the following statement or structure immediately after completion of the present one, unless directed otherwise in a branching statement. The referenced paragraph numbers in the syntax diagram contain the detailed definition of each of the classes of ATLAS procedural statements.

7.2.2 Formal Syntax

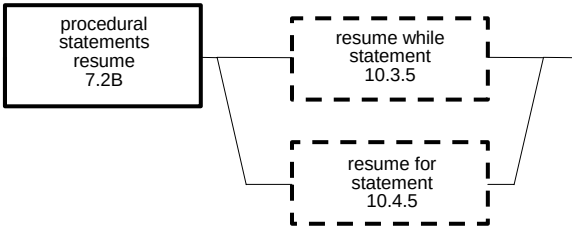
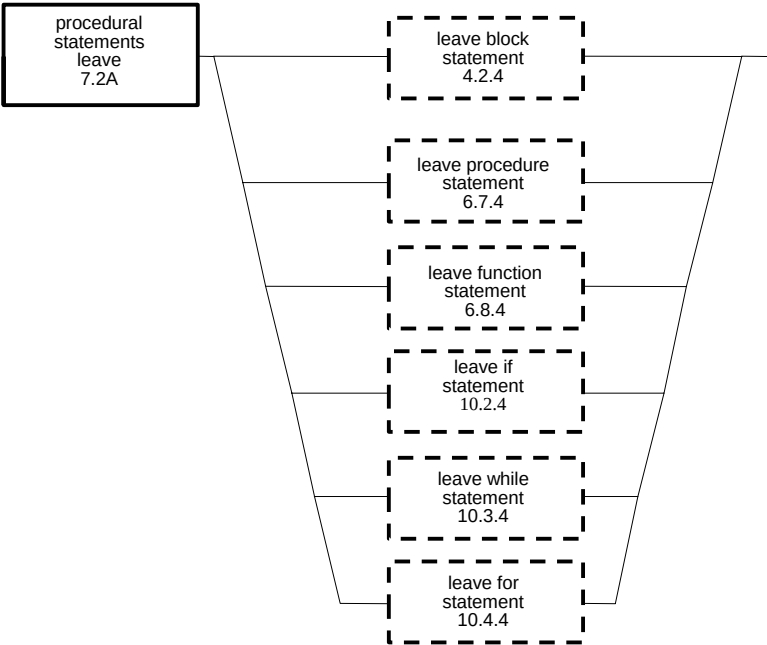
FSD / 1209

7.0 PROCEDURAL STATEMENTS

7.2.3 Syntax Diagram



7.0 PROCEDURAL STATEMENTS



8.0 DATA PROCESSING STATEMENTS

8.1 <procedural statements, data processing>

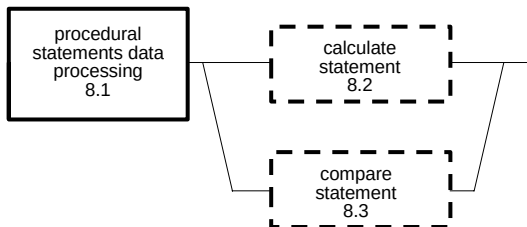
8.1.1 Function

To provide the capability to perform calculations, compare test values with specified limits and retain test values for subsequent use in the test procedure.

8.1.2 Formal Syntax

FSD / 1249

8.1.3 Syntax Diagram

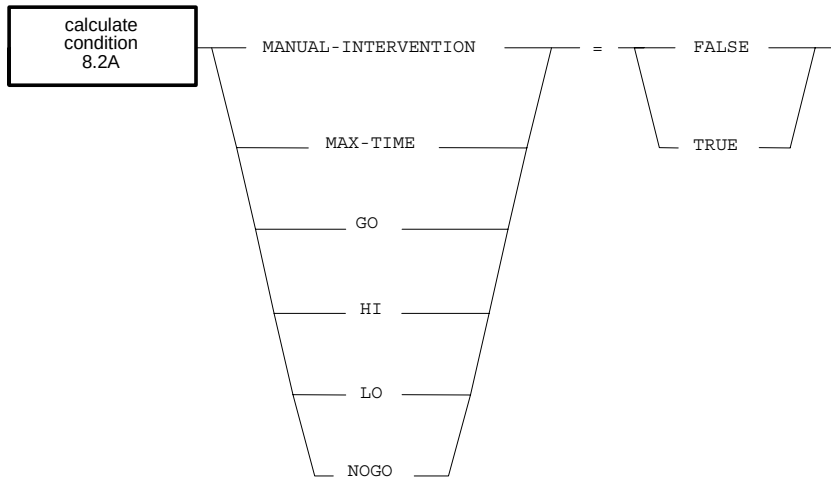
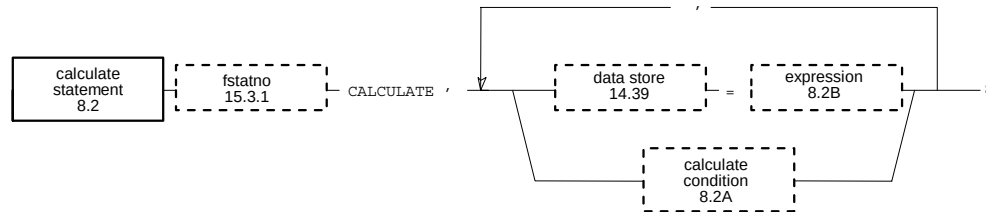


8.0 DATA PROCESSING STATEMENTS**8.2 <calculate statement>****8.2.1 Function**

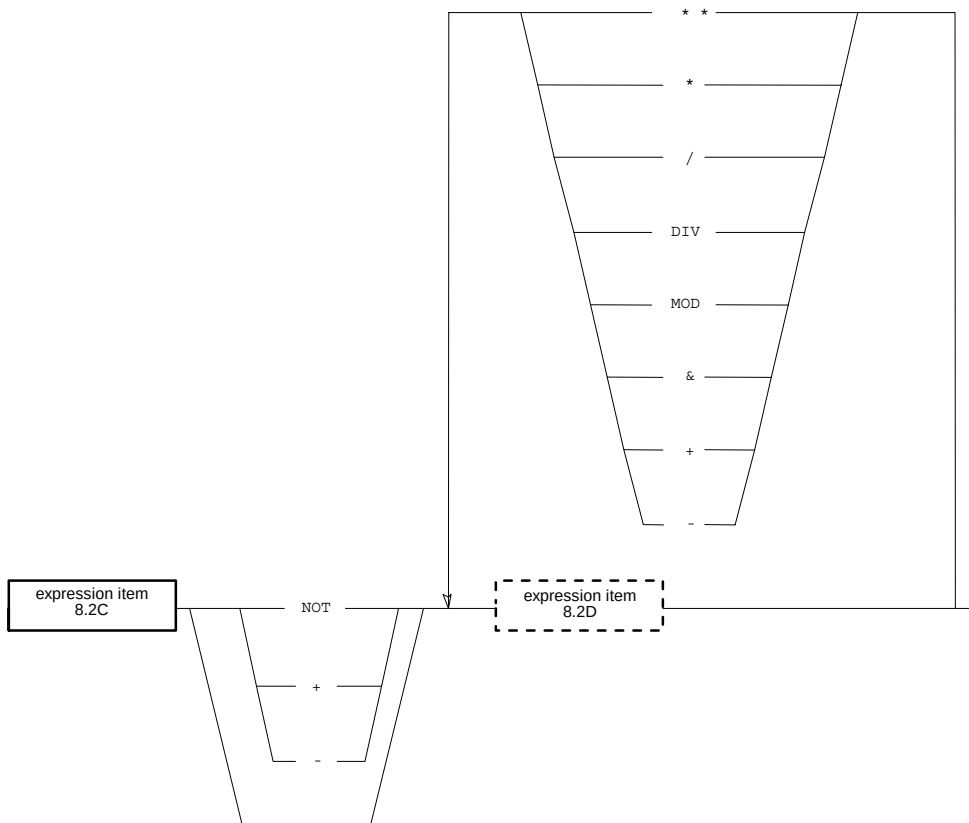
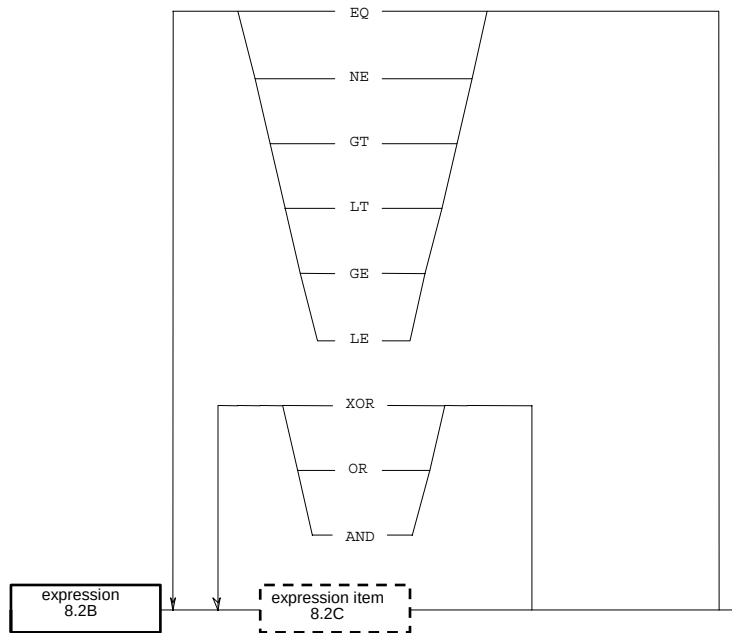
To evaluate an <expression> on the right side of an equals sign, "=", and assign the value of that <expression> to the <data store> on the left of the equals sign. The <expression> may evaluate a test condition to a boolean condition TRUE or FALSE. One or more such evaluations and assignments may be made in one <calculate statement>.

8.2.2 Formal Syntax

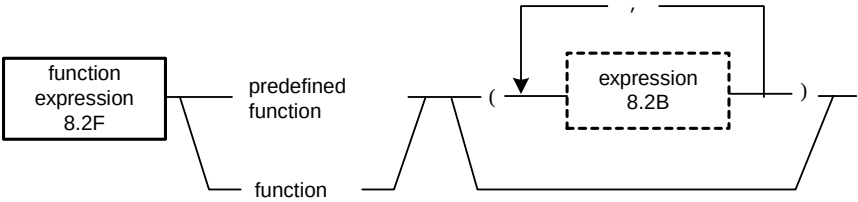
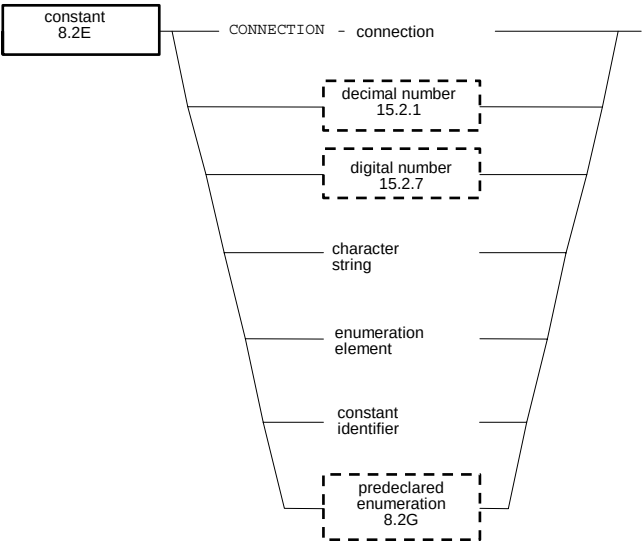
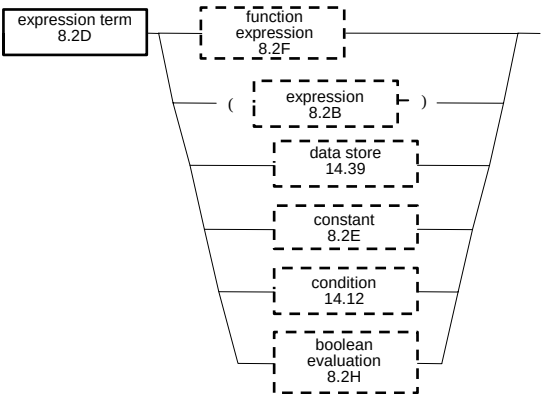
FSD / 1256

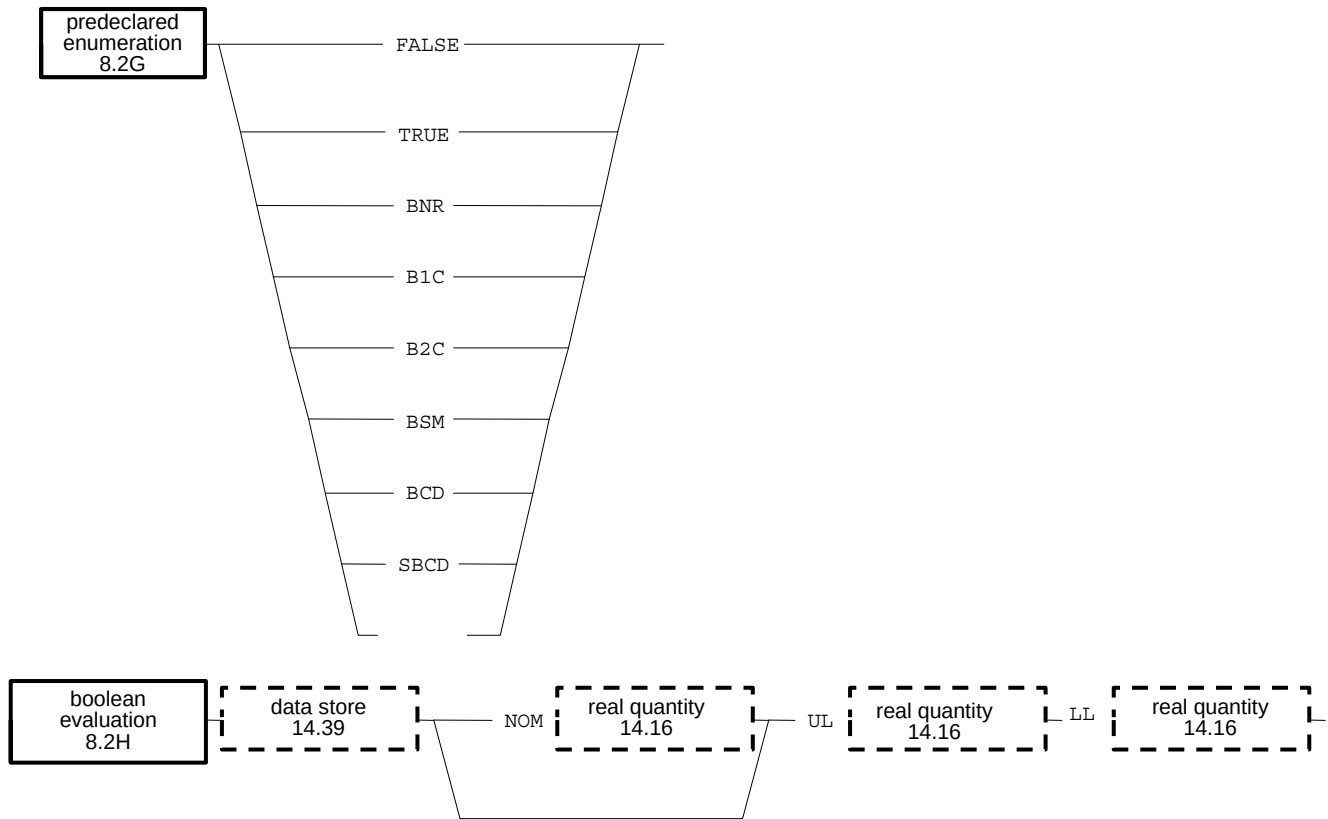
8.2.3 Syntax Diagram

8.0 DATA PROCESSING STATEMENTS



8.0 DATA PROCESSING STATEMENTS



8.0 DATA PROCESSING STATEMENTS**8.2.4 Rules****8.2.4.1 General**

1. The <expression> can be of any type except FILE.
2. The <data store> to which assignment is made must be assignment compatible with the <expression>.

8.2.4.2 General Rules for <expression>

1. An <expression> may contain literal constants, labeled constants, variables, functions, and operators.
2. The operators which are permitted within an <expression> are tabulated in Table 8-1 of Section 8.2.4.3. This table also specifies the precedence level, the number and types of operands, and the type of the resultant <expression> for each operator.
3. Each <function> and <predefined function> operates upon a set of parameters of predetermined types and yields a result of predetermined type. Table 8-2 of Section 8.2.4.4 tabulates the set of <predefined function>s.
4. For nested parenthesized expressions, the innermost expression is evaluated first.
5. At any one depth of parentheses the precedence of evaluation is given by the level of the operators tabulated in Section 8.2.4.3. Operators are evaluated in the order of their increasing level numbers.
6. For operators of equal precedence level, evaluation is from left to right.

8.0 DATA PROCESSING STATEMENTS

7. Signed DECIMAL <constant>s are not permitted.
8. A <condition> is treated as a special BOOLEAN operand which is established and maintained by the system.
9. When an <expression> is used, evaluation of the <expression> must be assignment compatible with its usage.

8.2.4.3 Rules Associated with Permitted Operations

1. Table 8-1 of Section 8.2.4.3 lists the permitted operations. Each operation is located in a precedence level. Each operation has an operator and permitted types of operand each with an associated result type. Operations between type compatible parts of dissimilar structures are allowed.

Note: The subsequent rules apply only where referenced from Table 8-1 of Section 8.2.4.3.

2. If an operation on a STRING OF BIT is applied to STRINGs of unequal current dynamic length, the shorter is treated as though it had extra B'O's at the left-hand (MSB) end. The dynamic length of the resultant is equal to that of the longer operand.
3. The resultant is the concatenation of the operands in the order written. The dynamic length of the resultant is the sum of the dynamic lengths of the operands.
4. In comparison operations the operands must be of identical types except that INTEGER types may be compared with DECIMAL types. When INTEGER operands are compared to DECIMAL operands, the INTEGER operand is coerced to DECIMAL before the comparison and the resultant has BOOLEAN type.
5. Comparison of strings proceeds as follows:
 - a. If the strings are of unequal current dynamic length, the shorter STRING OF BIT is treated as though it were padded on the left with B'O's and the shorter STRING of CHAR is treated as though it were padded on the right with ASCII NUL characters.
 - b. Comparison proceeds from left to right until either corresponding unequal values are found or until the longer dynamic length is reached. If an unequal position value is found, the relationship between the STRINGs is determined by the relationship of the unequal BITs or CHARs. If no inequality is found, the relationship between the STRINGs is determined by the relationship of their current dynamic lengths.
 - c. The relationship of characters is determined by the ordinal values of the characters in the ISO or ASCII alphabet to which they belong.
6. The NOT operator does not alter the dynamic length of a STRING OF BIT.

8.0 DATA PROCESSING STATEMENTS**Table 8-1 Permitted Operations**

Prec Level	Operation	Operand Type Structure or Base Types	Action	Result Type	Resultant
1	NOT x Reference Rule 6 of Section 8.2.4.3	BOOLEAN BIT STRING OF BIT	Logical NOT Logical NOT Logical NOT	BOOLEAN BIT STRING OF BIT	TRUE if x is FALSE FALSE if x is TRUE complement of x bit-by-bit complement of x
1	-x	INTEGER DECIMAL	unary – unary –	INTEGER DECIMAL	negation of x
1	+x	INTEGER DECIMAL	unary + unary +	INTEGER DECIMAL	x
2	x XOR y	BOOLEAN BIT STRING OF BIT	exclusive OR exclusive OR exclusive OR	BOOLEAN BIT STRING OF BIT	((x AND NOT y) or (y AND NOT x))
2	x ** y	INTEGER DECIMAL x is DECIMAL y is INTEGER	exponentiate exponentiate exponentiate	INTEGER DECIMAL DECIMAL	x to the power y
3	x * y	INTEGER DECIMAL INTEGER and DECIMAL	multiply multiply multiply	INTEGER DECIMAL DECIMAL	product of x and y
3	x / y	INTEGER DECIMAL X is INTEGER Y is DECIMAL	division division division	DECIMAL DECIMAL DECIMAL	quotient of x and y
3	x DIV y	INTEGER	division	INTEGER	INT(x/y)
3	x MOD y	INTEGER	modulo	INTEGER	REMAINDER(x/y)
3	x AND y Reference Rule 2 of Section 8.2.4.3	BOOLEAN BIT STRING OF BIT	logical AND logical AND logical AND	BOOLEAN BIT STRING OF BIT	TRUE if both x and y are TRUE else FALSE B'1' if both x and y are B'1' else B'0'. Each bit as BIT.
4	x & y Reference Rule 3 of Section 8.2.4.3	STRING OF CHAR or CHAR STRING OF BIT or BIT	concatenate	STRING OF CHAR STRING OF BIT	STRING comprising x followed by y
4	x + y	INTEGER DECIMAL INTEGER and DECIMAL	addition addition addition	INTEGER DECIMAL DECIMAL	sum of x and y
4	x – y	INTEGER DECIMAL INTEGER and DECIMAL	subtraction subtraction subtraction	INTEGER DECIMAL DECIMAL	difference between x and y

8.0 DATA PROCESSING STATEMENTS

Prec Level	Operation	Operand Type Structure or Base Types	Action	Result Type	Resultant
4	x OR y Reference Rule 2 of Section 8.2.4.3	BOOLEAN BIT STRING OF BIT	logical OR logical OR logical OR	BOOLEAN BIT STRING OF BIT	TRUE if x or y are TRUE else FALSE B'1' if x or y are B'1' else B'0'
5	x EQ y Reference Rule 4 of Section 8.2.4.3	all except FILE	equality	BOOLEAN	TRUE for identical x and y else FALSE
5	x NE y Reference Rule 4 of Section 8.2.4.3	all except FILE	inequality	BOOLEAN	FALSE for identical x and y else TRUE
5	x GT y Reference Rules 4 & 5 of Section 8.2.4.3	INTEGER, CHAR, DECIMAL, STEP, STRING, BIT, and ENUMERATION	greater than	BOOLEAN	TRUE for x greater than y else FALSE
5	x LT y Reference Rules 4 & 5 of	as for GT above	less than	BOOLEAN	TRUE for x less than y else TRUE
5	x GE y Reference Rules 4 & 5 of Section 8.2.4.3	as for GT above	GT or EQ	BOOLEAN	TRUE for (x GT y) OR (x EQ y) else FALSE
5	x LE y Reference Rules 4 & 5 of Section 8.2.4.3	as for GT above	LT or EQ	BOOLEAN	TRUE for (x LT y) OR (x EQ y) else FALSE

8.0 DATA PROCESSING STATEMENTS**Table 8-2 <predefined function>**

Function	Parameter <type>	Action	Result <type>	Resultant
EOF (x)	FILE	test for end of file	BOOLEAN	TRUE if at end of file else FALSE
ODD (x)	INTEGER or BIT	odd	BOOLEAN	For INTEGER: TRUE if x odd else FALSE For BIT: TRUE if BIT is a 1 else FALSE
PE (x)	STRING OF BIT or BIT	parity even	BIT	B'1' if x has even parity else B'0'
PO (x)	STRING OF BIT or BIT	parity odd	BIT	B'1' if x has odd parity else B'0'
ORD (x)	ENUMERATION or CHAR	ordinal	INTEGER	Ordinal position where 0 is the first in permitted range of values of x
LEN (x)	STRING OF BIT or STRING OF CHAR	length	INTEGER	Current dynamic length of string x, 0 if empty
LOCN (x, y) Reference Rule 2 of Section 8.2.4.4	STRING OF BIT or SRING OF CHAR	location	INTEGER	Position of the first occurrence of string x within string y, 0 if not found
COUNT (x, y)	x is a BIT or CHAR and y is a compatible string	count occurrences of x in y	INTEGER	The number of occurrences of the BIT or the CHAR in the string y.
SIZE(f)	f is a FILE	size	INTEGER	The number of CHAR in a TEXT file referenced by a <file> <variable identifier f, or the number of data items if f is a typed file, 0 for an empty file
SUCC(x) Reference Rule 5 of Section 8.2.4.4.	ENUMERATION or CHAR	successor	same as x	Next sequential value of x
PRED(x) Reference Rule 6 of Section 8.2.4.4	ENUMERATION or CHAR	predecessor	same as x	Previous sequential value of x
DATE Reference Rule 7 of Section 8.2.4.4	none	date-time	STRING OF CHAR	YYMMDDhhmmssss
CHAR (x, y)	x if char-class y is STRING OF BIT	conversion to character	CHAR	Character represented in CHAR- CLASS x by the STRING OF BIT y
BITS (x, y)	x if CHAR-CLASS y is CHAR	conversion of a character to bits	STRING OF BIT	STRING OF BIT Representing character y in CHAR-CLASS x
COPY (x, y, z) Reference Rule 2 of Section 8.2.4.4	y and z are INTEGER and x is STRING OF BIT or STRING OF CHAR	copy substring	same as x	Copy of string x from location y for z elements

8.0 DATA PROCESSING STATEMENTS

Function	Parameter <type>	Action	Result <type>	Resultant
ROTATE (x, y) Reference Rule 8 of Section 8.2.4.4	x is a STRING OF BIT and y is an INTEGER	rotate	BIT STRING	x rotated y places. Rotation is to the right (towards the LSB) for positive values of y and left for negative values of y
SHIFT (x, y) Reference Rule 8 of Section 8.2.4.4	as for ROTATE above	shift	BIT STRING	x shifted y places. Shifting is to the right (towards the LSB) for positive values of y and left for negative values of y. Bits shifted off the end are lost. Vacated positions set to B'0'.
A-SHIFT (x, y) Reference Rule 8 of Section 8.2.4.4	as for ROTATE above	arithmetic shift	BIT STRING	As SHIFT except that an assumed sign bit (at MSB) is not shifted and for a right shift it is replicated in the vacated bit positions.
DIG (x, y, w, f) Reference Rule 3 of Section 8.2.4.4	x is DIG-CLASS, y is INTEGER or DECIMAL, w and f are INTEGER	string conversion	STRING OF BIT	A STRING OF BIT representation of y in a BNR, B1C, etc., form allowing w bits for the whole part and f bits for the fractional part. For signed number types, w includes the sign bit.
DEC (x, y, w) Reference Rule 4 of Section 8.2.4.4	x is DIG-CLASS, y is a STRING OF BIT, w is an INTEGER	float	DECIMAL	A DECIMAL representation of y in a BNR, B1C, etc., form, where w bits are the whole part including a sign bit for signed number types.
INT (x)	DECIMAL	integer part	INTEGER	x truncated. The fractional part is ignored.
ROUND (x)	DECIMAL	rounding	INTEGER	x rounded. INT (x+0.5) for x GT 0 else INT (x-0.5)
ABS (x)	DECIMAL or INTEGER	absolute	same as x	x if x was GE 0 else -x
SIN (x)	DECIMAL or INTEGER x in degrees	trig function	DECIMAL	sine of x
COS (x)	as for SIN	trig func	DECIMAL	cosine of x
TAN (x)	as for SIN	trig func	DECIMAL	tangent of x
LN (x)	DECIMAL or INTEGER x GT 0	natural log	DECIMAL	log to the base e
LOG (x)	as for LN	common log	DECIMAL	log to the base 10
ALOG (x)	DECIMAL or INTEGER	antilog	DECIMAL	number whose log to the base 10 equals x
ATAN(x)	DECIMAL or INTEGER	angle of TAN	DECIMAL	angle (in degrees) whose TAN is x
EXP (x)	DECIMAL or INTEGER	e to power x	DECIMAL	e to the power x
SQRT (x)	DECIMAL or INTEGER	square root	DECIMAL	square root of x x GE 0

8.0 DATA PROCESSING STATEMENTS

Function	Parameter <type>	Action	Result <type>	Resultant
DELETE (s, p, w) Reference Rule 2 of Section 8.2.4.4	s is STRING OF BIT or STRING OF CHAR p and w are of type INTEGER	delete a substring	same as type of s	A copy of s with w elements deleted starting at position p
INSERT (s, t, p) Reference Rule 2 of Section 8.2.4.4	s and t are both of type STRING OF BIT or both of type STRING OF CHAR p is INTEGER	insert a string into a copy of another string	same as s and t	A copy of t with s inserted at position p+1

8.2.4.4 Rules Associated with <pre-defined function>s

1. Table 8-2 of Section 8.2.4.4 lists the <predefined function>s. The number and permitted types of parameter for each function are listed with the associated result type.

Note: The subsequent rules apply only where referenced from Table 8-2 of Section 8.2.4.4.

2. For STRING OF CHAR each CHAR is numbered starting from the left with number 1. For STRING OF BIT, each BIT is numbered starting from the right with number 1, which is the least significant bit.
 - a. The value of LOCN(x,y) is the first position within y for which the STRINGS match for the LEN(x).
 - b. For COPY, if $z > (\text{LEN}(x) - y + 1)$, then only $(\text{LEN}(x) - y + 1)$ CHARs or BITs are copied. If $(\text{LEN}(x) - y + 1) < 1$, then no copying results.
 - c. For DELETE, $1 \leq p \leq \text{LEN}(s)$ and if $w > (\text{LEN}(s) - p + 1)$, then the function returns a copy of s with only $(\text{LEN}(s) - p + 1)$ CHARs or BITs deleted.
 - d. For INSERT, $1 \leq p \leq \text{LEN}(t)$. The STRING s is inserted starting at position p+1.

8.0 DATA PROCESSING STATEMENTS

3. For the function DIG, w must be greater than or equal to the value k shown below.

x	Meaning	k
BNR:	Binary Unsigned Positive	k is the smallest number such that $2^{**k} > \text{ABS}(y)$.
B1C:	Binary Ones Complement	k is the smallest number such that $2^{**k} > \text{ABS}(y)$.
B2C:	Binary Twos Complement	
BSM:	Binary Sign and Magnitude	
BCD:	Binary Coded Decimal	k is four times the number of digits in INT(y) and w and f must be multiples of four.
SBCD:	Sign plus Binary Coded Decimal	k is one plus four times the number of digits in INT(y), w must be one plus a multiple of four, and f must be a multiple of four.

If w exceeds the above value, then extra bits are inserted as follows:

BNR & BCD: Leading B'0's are inserted.

B1C & B2C: The first <sign> bit is replicated as required.

BSM & SBCD: B'0's are inserted after the sign bit.

If f is too small to represent the precision of the fractional part of the y then y is truncated.

The resultant STRING OF BIT is of length w+f.

Examples of the DIG function are:

<function>	y = 6	y = 3	y = 1.5	y = -1.5	y = -3
DIG(BNR,y,3,3)	110000	011000	001100	ILLEGAL	ILLEGAL
DIG(B2C,y,3,3)	ILLEGAL	011000	001100	110100	101000
DIG(B1C,y,3,3)	ILLEGAL	011000	001100	110011	100111
DIG(BSM,y,3,3)	ILLEGAL	011000	001100	101100	111000
DIG(BCD,y,4,4)	01100000	00110000	00010101	ILLEGAL	ILLEGAL
DIG(SBCD,y,5,4)	001100000	000110000	000010101	100010101	100110000

4. For the DEC function, w must be less than or equal to the dynamic length of y. Bits in excess of w are considered to be fractional.
5. The action of SUCC(x), where ORD(x) is the last of a range of values, is undefined.
6. The action of PRED(x), where ORD(x) is the first of a range of values, is undefined.

8.0 DATA PROCESSING STATEMENTS

7. For the DATE function, the returned STRING OF CHAR will always contain 15 characters in the format YYMMDDhhmmsssss where:

YY represents two digits (00-99) indicating the year,

MM represents two digits (01-12) indicating the month in the year,

DD represents two digits (01-31) indicating the day in the month,

hh represents two digits (00-23) indicating the hour in the day,

mm represents two digits (00-59) indicating the minute in the hour,

sssss represents five digits (00000-59999) indicating the millisecond in the minute.

8. The functions ROTATE SHIFT and A-SHIFT operate over the dynamic length of a STRING OF BIT.

8.2.4.5 Rules for <constant>

(Syntax Diagram 8.2-E)

1. For literal constants, the type of the constant is determined by inspection of its textual form with keywords to resolve the ambiguities that CONNECTION constants would otherwise introduce.
2. When a <constant> is an enumeration element, this element must be used as part of a variable or type declaration within the same ATLAS PROGRAM or MODULE structure and occur within a textually preceeding statement.

8.2.4.6 Rules for <boolean evaluation>

(Syntax Diagram 8.2-H)

1. A <boolean evaluation> produces a BOOLEAN result.
2. The conditions of Section 14.17 are not affected by processing a <boolean evaluation>.
3. The types of the <data store> and the quantities following NOM, UL, and LL in a <boolean evaluation> must be compatible.
4. The dimensions following NOM, UL, and LL in a <boolean evaluation> must be identical.

8.2.4.7 Rules for <predetermined evaluation>

(Syntax Diagram 8.2-G)

1. The keywords established by the <predetermined evaluation> syntax are the <enumeration element>s of the <predeclared type>s established by Syntax Diagram 6.4-G of Section 6.4.3 and comply with the rules in Section 6.4.4.7.

8.2.5 Notes and Examples for <expression>s

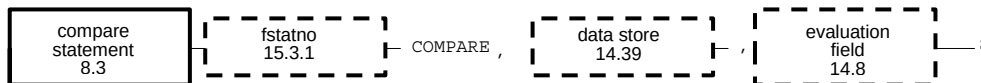
The meanings of NOM, UL, and LL are as defined in <evaluation field> Section 14.11.

8.0 DATA PROCESSING STATEMENTS**8.3 <compare statement>****8.3.1 Function**

To establish the state of the HI, LO, GO, and NOGO conditions, in accordance with Figure 14-1 of Section 14.8, based on the relationship between a labeled value and the specified limit(s).

8.3.2 Formal Syntax

FSD / 1391

8.3.3 Syntax Diagram**8.3.4 Rules**

1. Following the verb in the <compare statement> the label of a <data store> is written to indicate the object of the evaluation. An evaluation field completes the statement.
2. Wherever appropriate a null dimension may be used in the <evaluation field>.

8.3.5 Notes and Examples

```

E543210 COMPARE, 'MEASUREMENT', GT 5.1 V $
888888 COMPARE, 'MEASUREMENT', LT 'GAIN 1' $
999999 COMPARE, 'OUT 23', NOM 23V UL 23.75 V LL 22.25 V $
711711 COMPARE, 'INDEX', UL 'UPPER' LL 'LOWER' $
  
```

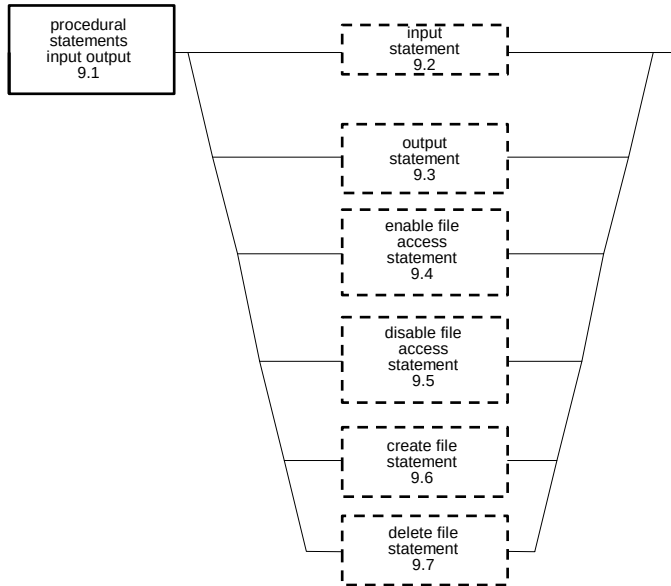
Note: The use of NOM (nominal) in the evaluation field is not required to accurately specify limits. This capability is an option for better understanding by the human reader.

9.0 INPUT/OUTPUT STATEMENTS**9.1 <procedural statements input output>****9.1.1 Function**

To provide the means whereby data or information can be manually or automatically inserted or extracted during the testing process.

9.1.2 Formal Syntax

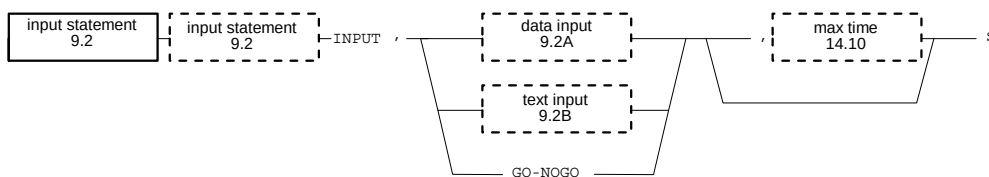
FSD / 1400

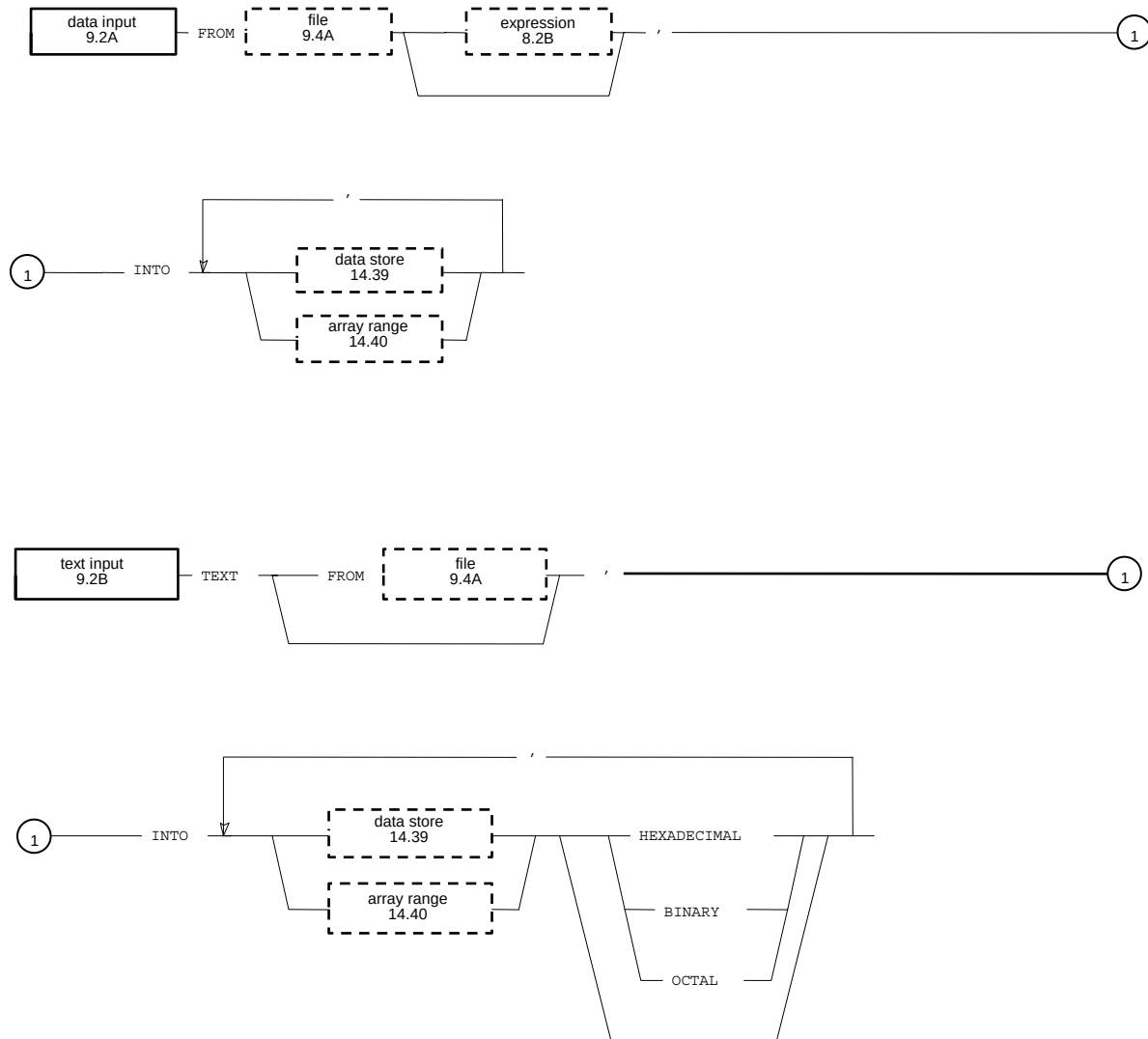
9.1.3 Syntax Diagram**9.2 <input statement>****9.2.1 Function**

To transfer external non-signal data into an ATLAS test specification, procedure or program.

9.2.2 Formal Syntax

FSD / 1411

9.2.3 Syntax Diagram

9.0 INPUT/OUTPUT STATEMENTS**9.2.4 Rules****9.2.4.1 Rules for all INPUT Statements**

1. If FROM <file> is not specified in the statement, input is from the system dependent standard input device.
2. The FROM <file>, which identifies the source of the data to be input, must have been ENABLED prior to the execution of the INPUT statement referencing it.

9.0 INPUT/OUTPUT STATEMENTS

3. Reading of data items to INTO variables is sequential in a forward direction from the current source file position. After execution of the <input statement>, the current position will have advanced by the number of INTO <variable identifier>s and/or array elements filled. The initial file position is established as follows:
 - a. If the INPUT statement contains the optional file position <expression> following the file identifier, the file position is set to the value of this <expression>. This <expression> must evaluate to an INTEGER which is greater than or equal to zero and equal to or less than SIZE (<file>).
 - b. If the INPUT statement does not contain a file position <expression>, the file position is as set by the execution of the previous statement referencing this file. If this previous statement was an ENABLE of the file, then the current position of the file is zero.
4. For sequential reads, the pre-defined function EOF (<file>) will return TRUE if invoked after the last data item of the file has been read. If the end of file is reached before the <data store>(s) have been completely filled, any <data store> left unfilled by the INPUT operation will have undefined contents. The point at which the end of file was reached during the INPUT operation is not directly available.
5. Input to an <array> may be an input to a whole <array> from a <file> with identical size and array element type; in which case the <variable identifier> of the <array> is included without a range. Alternatively the input may be to individual elements of an <array> from a <file> with the same <type> as the array elements; in which case the <array> must have an associated range.
6. The optional <max time field> provides the capability to limit the elapsed time for completion of the INPUT operation. If the time allocated by the <max time field> is exceeded before the input is complete:
 - a. The INPUT action is aborted
 - b. The MAX TIME flag is set.
 - c. The <data store>s identified in the INTO field of the statement will all have undefined contents.

9.2.4.2 Rules for GO-NOGO Type Input

1. The GO-NOGO path is used to obtain input from the operator to establish a new GO or NOGO condition.

9.2.4.3 Rules for TEXT Type Input

1. When input is specified using the TEXT branch, the <input statement> takes characters from the input source and converts the input to match the destination <data store> type.
2. If specified, the <file> label immediately following FROM must have been declared as FILE OF TEXT.
3. Table 9-1 in Rule 4 specifies the permitted <type>s of INTO <variable identifier>s. Each permitted type has a set of legal characters. For each type, INPUT characters are ignored until the first legal character is seen. Type conversion of the input proceeds until an illegal character, as defined by Table 9-1, is found. This allows meaningful data to be separated by blank, new-line, comma, etc.

9.0 INPUT/OUTPUT STATEMENTS

4. HEXADECIMAL, OCTAL and BINARY can only be used when the INTO variable is a STRING OF BIT.

Table 9-1 Permitted Data Types For Automatic Conversion and OUTPUT

TYPE OF VARIABLE	ALLOWABLE CHARACTERS
DECIMAL	'+', '-', '0' thru '9', '.', 'E'
INTEGER	'+', '-', '0' thru '9'
*CHAR	Any in Section 15.1
*BIT	'0', '1'
STRING OF CHAR	Any in Section 15.1 except new line
STRING OF BIT	'0' thru '9'
STRING OF BIT with HEXADECIMAL	'0' thru '9', 'A' thru 'F'
STRING OF BIT with OCTAL	'0' thru '7'
STRING OF BIT with BINARY	'0', '1'
CONNECTION	Any in Section 14.1 except those excluded by Section 14.14.4

Note: * INPUT to these types inputs only one CHAR or BIT.
 OUTPUT from these types outputs only one CHAR or BIT.

5. INPUT to a STRING OF BIT, unqualified by HEXADECIMAL, OCTAL or BINARY, is assumed to be an unsigned INTEGER format. For the INTEGER case, the unsigned INTEGER is converted into its equivalent binary representation for storage in the string.
6. When inputting to a string the dynamic length is set as follows:

<u>String</u>	<u>Dynamic Length</u>
a. STRING OF CHAR	Number of characters input
b. STRING OF BIT with HEXADECIMAL	Four times the number of characters input
c. STRING OF BIT with OCTAL	Three times the number of characters input
d. STRING OF BIT with BINARY	Number of characters input
e. STRING OF BIT with no qualifier	Minimum required to represent the INTEGER input

9.2.4.4 Rules for all Other Data Types

1. Using the alternative branch to TEXT transfers data items from a <file> of known type data to INTO variables.
2. The following apply:
 - a. The INTO variable must be of the same <type> as the input <file>.
 - b. One data item is read from the input <file> for each <data store> specified in the INTO field.
 - c. After execution of the statement, the current file position will have advanced by the number of inputted elements.

9.2.5 Notes and Examples

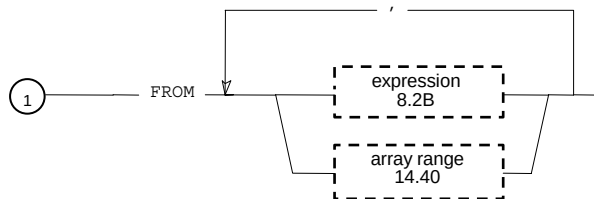
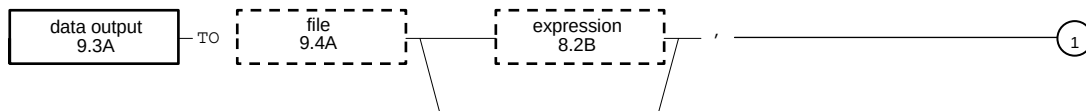
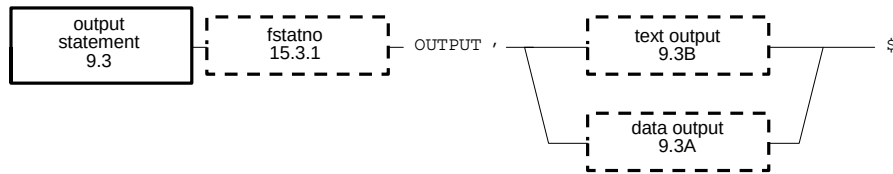
1. Since output of a STRING OF BIT using the default INTEGER format is done by suppressing leading zeros, the output may not reflect the actual dynamic length of the stored string. Subsequent usage of that output as an input where the input format is defaulted to INTEGER may result in a different dynamic length.

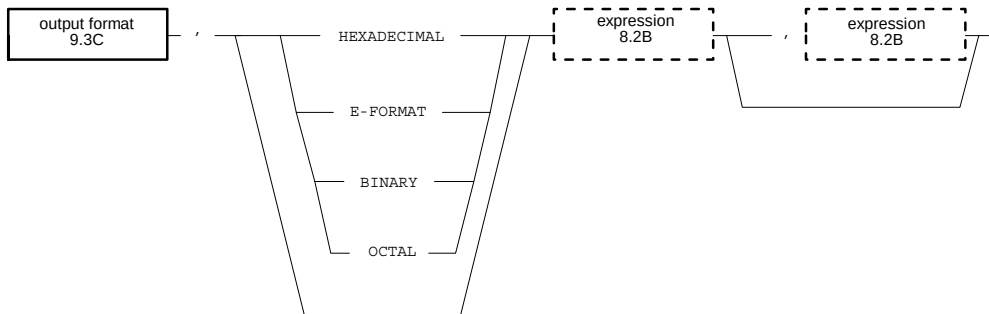
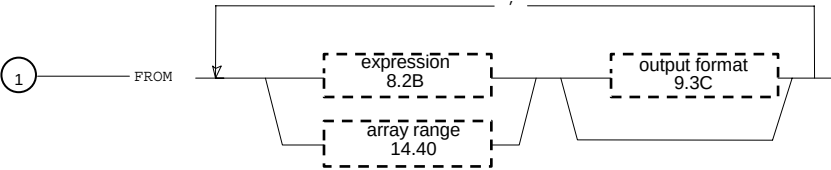
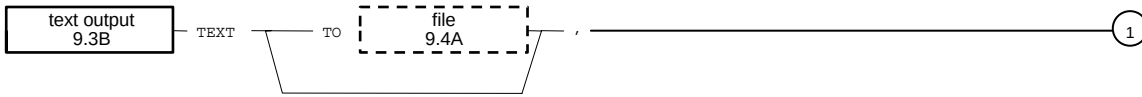
9.0 INPUT/OUTPUT STATEMENTS**9.3 <output statement>****9.3.1 Function**

To transfer non-signal data from an ATLAS test specification to an external resource. ATLAS OUTPUT statements operate on virtual I/O resources.

9.3.2 Formal Syntax

FSD / 1446

9.3.3 Syntax Diagram

9.0 INPUT/OUTPUT STATEMENTS**9.3.4 Rules****9.3.4.1 Rules for all OUTPUT Statements**

1. If TO <file> is not specified in the statement, output is to the system dependent standard output device.
2. The TO <file>, which identifies the destination of the data to be output, must have been ENABLEd prior to the execution of the OUTPUT statement referencing it.
3. Writing of the data items into the TO <file> is sequential in a forward direction from the current file position. After execution of the <output statement>, the current position will have advanced by the number of FROM <variable identifier>s and/or array elements transferred. The initial file position is established as follows:
 - a. If the OUTPUT statement contains the optional file position <expression> following the file identifier, the current position is set to the value of this <expression>. This <expression> must evaluate to an INTEGER which is greater than or equal to zero and equal to or less than SIZE (<file>).
 - b. If the OUTPUT statement does not contain a file position <expression>, the current position remains as set by the execution of the previous statement referencing this file. If this previous statement was an ENABLE of the file, then the current position of the file is zero.

9.0 INPUT/OUTPUT STATEMENTS

4. For sequential writes, the pre-defined function EOF (<file>) will return TRUE if invoked after data has been written to the last data item of the file. If the end of the file is reached before all data items are transferred, execution of the <output statement> will be terminated.
5. Output from an <array> may be an output of a whole <array> to a <file> with identical size and array element type; in which case the <variable identifier> of the <array> is included without a range. Alternatively the output may be from individual elements of an <array> to a <file> with the same <type> as the array elements; in which case the <array> must have an associated range.

9.3.4.2 Rules for TEXT Type Output

1. When OUTPUT is specified using the TEXT branch, characters are written to the output destination simultaneously converting the value of a FROM <expression> to a text string.
2. If specified, the <file> label immediately following TO must have been declared as FILE OF TEXT.
3. Table 9-1 in Rule 4 of Section 9.2.3.3 specifies the permitted data <type>s in a FROM <expression> or <array range>. Each permitted type has a set of legal characters. Type conversion of the value of the source <expression> or <array range> into its character representation occurs automatically on output.
4. HEXADECIMAL, OCTAL and BINARY character output can only be used when the FROM <expression> is a STRING OF BIT.
5. OUTPUT from a STRING OF BIT, unqualified by HEXADECIMAL, OCTAL or BINARY, consists of a logical conversion from binary representation to an unsigned INTEGER format with leading zeros suppressed in conformity with Rule 6e of Section 9.2.3.3.
6. E-FORMAT can only be used when the FROM <expression> or <array range> is of type DECIMAL. E-FORMAT outputs a <floating point decimal number> as defined in Section 15.2.1 <decimal number>.
7. Two optional formatting <expressions>s may follow the FROM <expression>. Both must evaluate to INTEGERS which are greater than or equal to zero. The first <expression>, field-width, indicates the total number of characters to be written. Leading blank spaces will be added as required to match this format. An output longer than this <expression> will not be truncated, but will be written using the smallest number of characters which will represent the data items. The second <expression>, precision, is only used with a FROM <expression> of type DECIMAL. It indicates the number of characters to be written to represent the fractional part of the number. Trailing zeroes will be added as necessary. truncation at the specified number of digits will occur.
8. If DECIMAL numbers have no format, the output is implementation dependent.

9.3.4.3 Rules for All Other Types of Data Output

1. Using the alternative branch to TEXT transfers the value of a FROM <expression> or <array range> to a <file> of identical type.
2. The following apply:
 - a. The data items resulting from the evaluation of the FROM <expression> or <array range> must be of the same <type> as the TO <file>.
 - b. A binary image of the resultant source data items will be transferred to the file at the current file position.

9.0 INPUT/OUTPUT STATEMENTS

9.3.5 Notes and Examples

1. A pre-determined <character string> is output by using a <character string> as a <constant> form of <expression>
2. Since output of a STRING OF BIT using the default INTEGER format is done by suppressing leading zeros, the output may not reflect the actual dynamic length of the stored string. Subsequent usage of that output as an input where the input format is defaulted to INTEGER may result in a different dynamic length.

9.4 <enable file access statement>

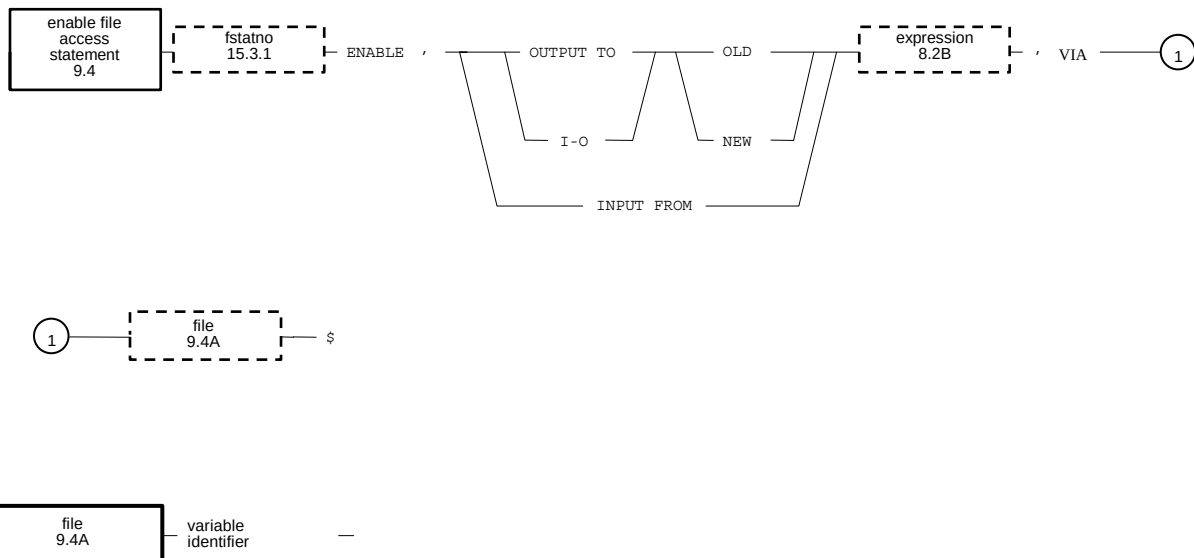
9.4.1 Function

To associate a declared <file> <variable identifier> with an actual file to allow subsequent access to the file.

9.4.2 Formal Syntax

FSD / 1487

9.4.3 Syntax Diagram



9.4.4 Rules

1. The actual filename <expression> must evaluate to a STRING OF CHAR. This string identifies, to the operating system, the actual file. It may include identification of volume and/or physical device as well as the file name. The format of this string is implementation dependent.
2. The actual file must be currently in a disabled state.
3. Only one <file> <variable identifier> can be associated with an actual file.

9.0 INPUT/OUTPUT STATEMENTS

4. Subsequent references to the actual file are by means of the <file> <variable identifier>.
5. The current position is set to provide access to the first file record or byte. Immediately following this statement, calls to the pre-defined function EOF (<file>) will return FALSE for non-empty actual files.
6. Specifying NEW creates an actual file designated by the filename <expression>. The actual file is created with no data items in it and the current position is set to zero. Specifying OLD allows access to an existing actual file and sets the current position to zero.
7. The <file> <variable identifier> must have been declared as being of a FILE <type>.

9.4.5 Notes and Examples

Since the name of an actual file is constrained by the operating system under which the software is run, it is impossible to ensure transportability of statements containing the name of a file. For instance, a file might be called:

C:\PROGRAM\SUBDIRECTORY\TESTA.TXT

in an IBM PC running DOS

PROCESSORA::DISCA:[PROGRAM.SUBDIRECTORY]TESTA.TXT

in a DEC VAX cluster using VMS

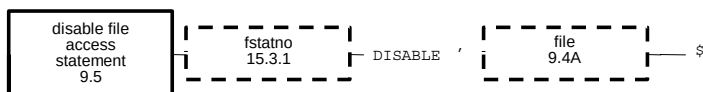
These are as near as possible to being the same, and are clearly not interchangeable. ATLAS minimizes this problem by only referring to the actual file name in the <enable file access statement>, the <create file statement> and the <delete file statement>. An ATLAS <variable identifier> with a FILE <type> is associated with the actual file by the <enable file access statement> and is used as the means of referencing the file in all other statements until the file is disabled by a <disable file access statement>. This <variable identifier> is identified as <file> in all statements that use it to cross reference the actual file.

9.5 <disable file access statement>**9.5.1 Function**

To dissociate a declared <file> <variable identifier> from an actual file and thereby to release the <file> <variable identifier> for future use.

9.5.2 Formal Syntax

FSD / 1502

9.5.3 Syntax Diagram**9.5.4 Rules**

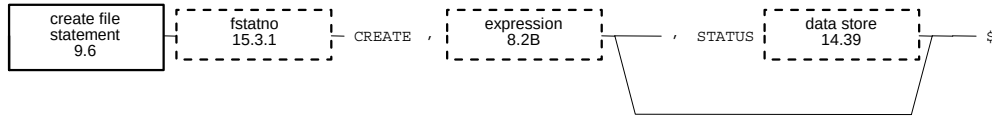
1. The statement has no effect unless the file is currently enabled.
2. Following execution of this statement a call to EOF(<file>) will return an undefined result.

9.0 INPUT/OUTPUT STATEMENTS**9.6 <create file statement>****9.6.1 Function**

To add a file to the file structure of the operating system.

9.6.2 Formal syntax

FSD / 1507

9.6.3 Syntax Diagram**9.6.4 Rules**

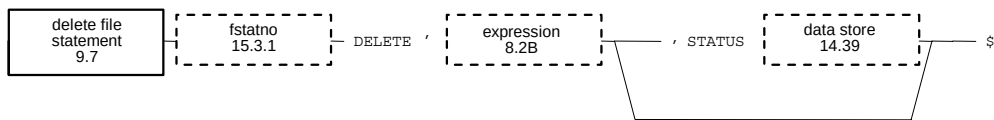
1. The actual file name <expression> must evaluate to a STRING OF CHAR. This string identifies, to the operating system, the actual file. It may include identification of volume as well the file name. The format of this string is implementation dependent.
2. The <data store> must be of the type BOOLEAN. This variable is set to TRUE when the file is created. It is set to FALSE if the file already exists or the operating system cannot create the file for some other implementation reason.

9.7 <delete file statement>**9.7.1 Function**

To remove a file from the file structure of the operating system.

9.7.2 Formal Syntax

FSD / 1513

9.7.3 Syntax Diagram**9.7.4 Rules**

1. The actual file name <expression> must evaluate to a STRING OF CHAR. This string identifies, to the operating system, the actual file. It may include identification of volume as well as the file name. The format of this string is implementation dependent.
2. The file must be currently in a disabled state.
3. The <data store> must be of the type BOOLEAN. This variable is set to TRUE when the file is deleted. It is set to FALSE if the file does not exist, the file is currently in an enabled state or the operating system cannot delete the file for some other implementation reason.

10.0 CONTROL STATEMENTS

10.0 Control Statements

10.1 <procedural statements, control>

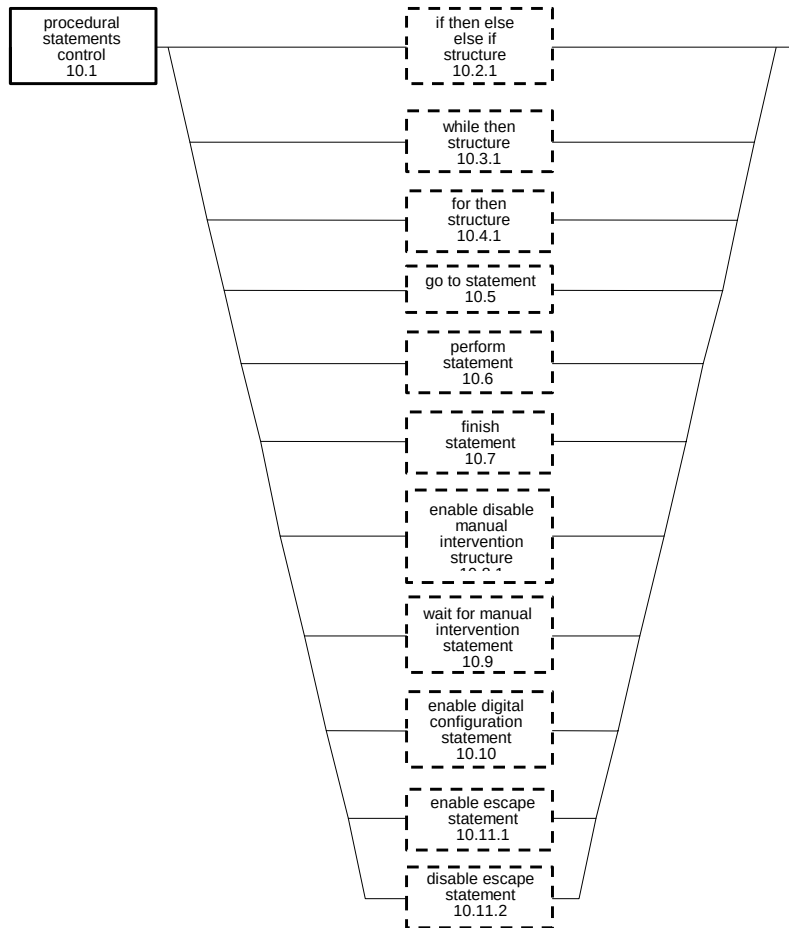
10.1.1 Function

To provide the capability to control the sequence of statement execution.

10.1.2 Formal Syntax

FSD / 1522

10.1.3 Syntax Diagram



10.0 CONTROL STATEMENTS**10.2 IF THEN ELSE/ELSE IF Capability****10.2.1 <if then else/else if structure>****10.2.1.1 Function**

To introduce the <if then> sub-structure and to specify the conditional execution of one or more statements based upon the logical value (TRUE or FALSE) of a conditional expression. If the optional <else> sub-structure is included then the execution of one of two sets of statements, or single statements, is dependent upon the logical value of the conditional expression. If the optional <else if> sub-structure is included, then the execution of subsequent statement(s) is (are) dependent upon the logical value of the conditional expression(s). The functional flow of these structures is shown in Figures 10-1, 10-2 and 10-3.

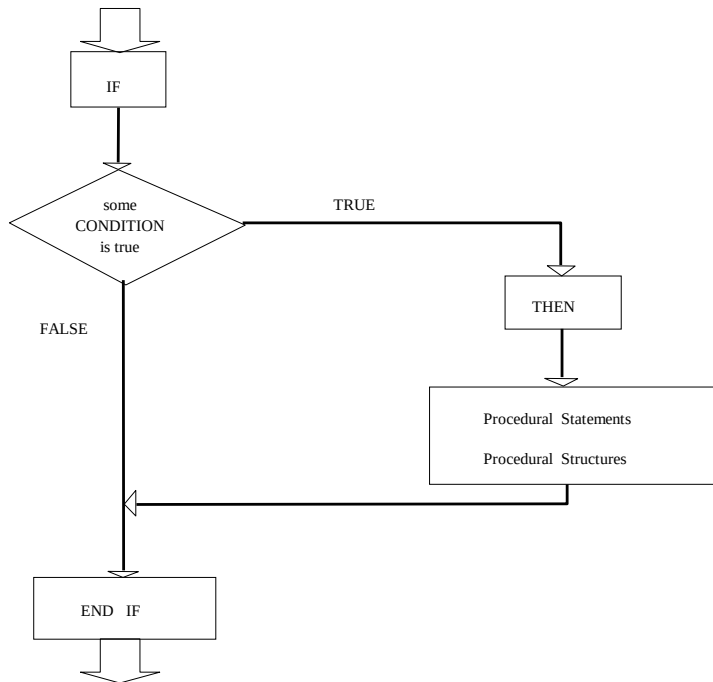
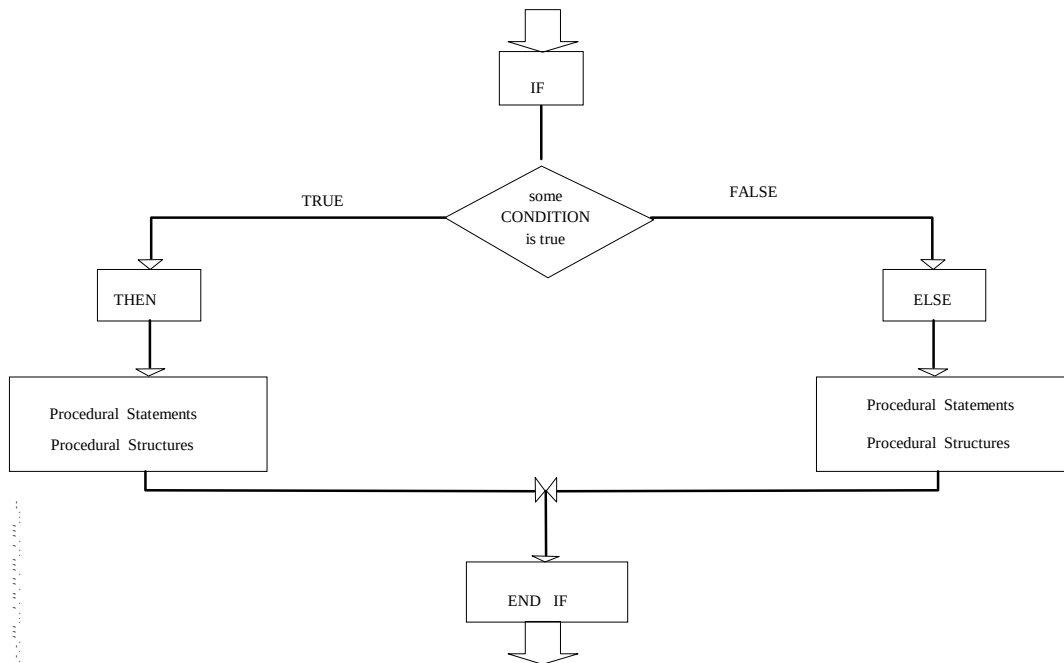
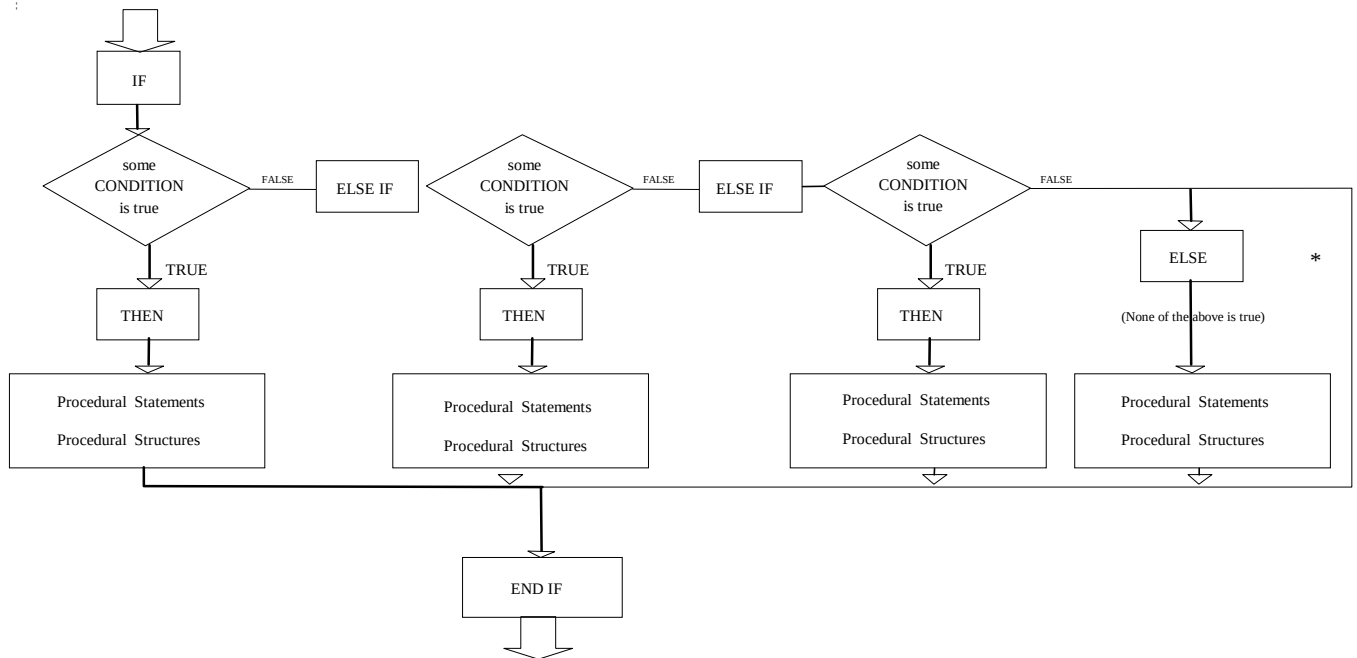


Figure 10-1 Functional Flow Diagram of <if then> Sub-structure

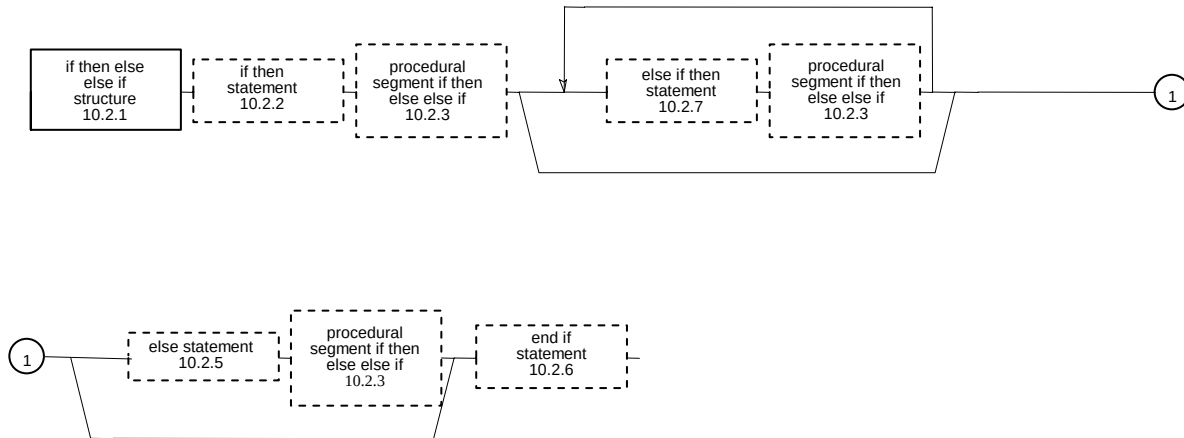
10.0 CONTROL STATEMENTS**Figure 10-2 Functional Flow Diagram of <if then else> Sub-structure**

Note: * When no ELSE statement is included

Figure 10-3 Functional Flow Diagram of <if then else/else if structure>

10.0 CONTROL STATEMENTS10.2.1.2 Formal Syntax

FSD / 1538

10.2.1.3 Syntax Diagram10.2.1.4 Rules

1. The functional flow diagrams shown in Figures 10-1, 10-2 and 10-3 unambiguously illustrate the logical control of the <if then else/else if structure> and provide the rules pertaining to these structures.

10.2.1.5 Notes and Examples~

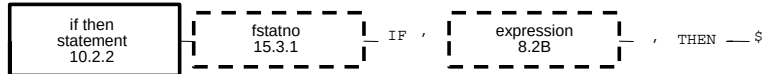
See Section 10.2.3.5 of <procedural segment> (IF THEN ELSE/ELSE IF)

10.0 CONTROL STATEMENTS**10.2.2 <if then statement>****10.2.2.1 Function**

To introduce an <if then> sub-structure and an <if then else/else if structure> for specifying the logical conditions which will cause the appropriate <procedural segment> to be executed.

10.2.2.2 Formal Syntax

FSD / 1552

10.2.2.3 Syntax Diagram**10.2.2.4 Rules**

1. <expression> must evaluate to a boolean result. The <procedure segment> for IF will be executed for those cases when the result evaluates to TRUE.
2. Processing of the <if then statement> does not affect the GO, NOGO, HI or LO flags.

10.2.2.5 Notes and Examples

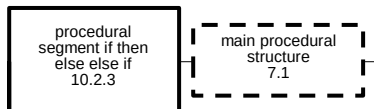
See Section 10.2.3.5 of <procedural segment> (IF THEN ELSE/ELSE IF)

10.2.3 <procedural segment if then else else if>**10.2.3.1 Function**

To define those statements to be performed if the stated condition is fulfilled (if the <procedural segment > is before ELSE) or if the stated condition is not fulfilled (if the <procedural segment> is after ELSE). A <procedural segment> consists of one or more complete ATLAS procedural statements or structures.

10.2.3.2 Formal Syntax

FSD / 1558

10.2.3.3 Syntax Diagram**10.2.3.4 Rules**

1. Statement numbers within a <procedural segment> may only be referenced from within the bounds of that segment.
2. Entry points must not appear within a <procedural segment>.

10.0 CONTROL STATEMENTS**10.2.3.5 Notes and Examples****EXAMPLE A: Using IF THEN only**

```

000100      IF, 'TEST' EQ TRUE, THEN $
01          OUTPUT, TEXT, FROM C'UUT IS GOOD' $
02      ELSE $
03          OUTPUT, TEXT, FROM C'UUT IS BAD' $
04      END, IF $

000205      IF, ('OUTPUT' + 'BIAS')**2 GT 12.5, THEN $
07          SETUP, DC SIGNAL USING 'DC-GEN', VOLTAGE 5V,
              CNX HI J1-1 LO J1-2 $
09      END, IF $

```

EXAMPLE B-1: Using IF THEN ELSE only

```

001000      IF, (('TEST1' OR 'TEST2') AND NOT 'TEST3') EQ TRUE, THEN $
01          IF, 'OUTPUT' EQ B '10110', THEN $
02              OUTPUT, TEXT, FROM C'UUT IS GOOD' $
03          ELSE $
04              OUTPUT, TEXT, FROM C'UUT IS BAD' $
05          END, IF $
06      ELSE $
07          APPLY, AC SIGNAL USING 'AC-GEN', ... $
08          DELAY, 10 SEC $
09          MEASURE, (VOLTAGE-P), ... $
10          IF, ('MEASUREMENT' GE 15.0) AND ('MEASUREMENT' LE 25.0),
              THEN $
11              CALCULATE, 'GO' = TRUE $
12          END, IF $
13      END, IF $

```

EXAMPLE B-2: using IF THEN ELSE only

```

010110      IF , 'MEASUREMENT' LT 10, THEN $
12          LEAVE, IF $
14      ELSE $
16          IF, 'MEASUREMENT' GE 20, THEN $
18              OUTPUT, TEXT, FROM C'REPLACE 1A1A13' $
20          ELSE $
22              IF, 'MEASUREMENT' GE 15, THEN $
24                  OUTPUT, TEXT, FROM C'REPLACE 1A1A15' $
26              ELSE $
28                  IF, 'MEASUREMENT' GE 12, THEN $
30                      OUTPUT, TEXT, FROM C'REPLACE 1A1A10' $
32                      ELSE $
34                          OUTPUT, TEXT, FROM C'REPLACE 1A1A12' $
36                          END, IF $
38                  END, IF $
40              END, IF $
42          END, IF $
44      FINISH $

```

EXAMPLE C: using entire IF THEN ELSE/ELSE IF structure

```

010110      IF, 'MEASUREMENT' LT 10, THEN $
12          LEAVE, IF $
14      ELSE IF, 'MEASUREMENT' GE 20, THEN $
16          OUTPUT, TEXT, FROM C'REPLACE 1A1A13' $
18      ELSE IF, 'MEASUREMENT' GE 15, THEN $
20          OUTPUT, TEXT, FROM C'REPLACE 1A1A15' $
22      ELSE IF, 'MEASUREMENT' GE 12, THEN $
24          OUTPUT, TEXT, FROM C'REPLACE 1A1A10' $
26      ELSE $
28          OUTPUT, TEXT, FROM C'REPLACE 1A1A12' $
30      END, IF $
32      FINISH $

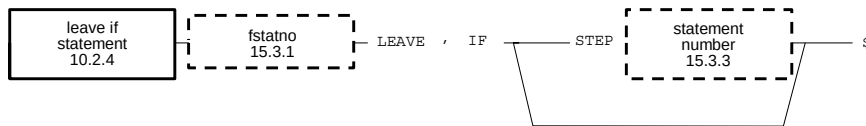
```

10.0 CONTROL STATEMENTS**10.2.4 <leave if statement>****10.2.4.1 Function**

To cause execution of a <procedural segment> of an <if then else/else if structure> to terminate prior to normal sequential execution of the last statement of the <procedural segment>. Control is transferred to the first executable statement following the <if then else/else if structure>.

10.2.4.2 Formal Syntax

FSD / 1562

10.2.4.3 Syntax Diagram**10.2.4.4 Rules**

1. When a statement number reference is used, it must be the statement number of either an <if then statement> or an <else if then statement> which introduces an <if then else/else if structure> containing the <leave if statement>.
2. When a statement number reference is omitted, the <leave if statement> is taken to refer to the innermost <if then else/else if structure> containing the <leave if statement>.
3. Any structures that contain a <leave if statement> which are contained within the <if then else/else if structure> to which it applies, are also exited upon execution of the <leave if statement>.

10.2.4.5 Notes and Examples

In the following example, execution of statement number 423003 will transfer control to statement number 423011.

```

423000      IF, 'X' EQ TRUE, THEN $
      01      APPLY, DC SIGNAL, ... $
      02      IF, 'Y' EQ TRUE, THEN $
      03          LEAVE, IF STEP 423000 $
      04      END, IF $
      05          MEASURE, ... $
      . . .
      . . .

      10      END, IF $
      11      OUTPUT, ... $

```

10.0 CONTROL STATEMENTS

10.2.5 <else statement>

10.2.5.1 Function

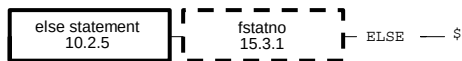
To introduce an <else> sub-structure to allow the execution of an alternative <procedural segment> to that which is associated with the <if then> substructure condition being true, if the condition is false.

The <else statement> is an optional statement following one or more optional <else if then statement>s of an <if then else/else if structure>. The <procedural segment> of an <if then else/else if structure> which follows the <else if then statement> is executed only when none of the previously defined <procedural segment>s have been executed.

10.2.5.2 Formal Syntax

FSD / 1568

10.2.5.3 Syntax Diagram



10.2.5.4 Rules

1. The <else statement> is a component of the <if then else/else if structure> so it can only be used in that context and has no other usage within the ATLAS language.

10.2.5.5 Notes and Examples

See Section 10.2.3.5 of <procedural segment> (IF THEN ELSE/ELSE IF)

10.2.6 <end if statement>

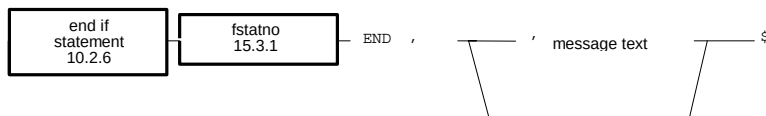
10.2.6.1 Function

To delimit either an <if then> sub-structure or an <if then else/else if structure>. Optional commentary may be included following the IF field to identify the IF structure which it delimits.

10.2.6.2 Formal Syntax

FSD / 1573

10.2.6.3 Syntax Diagram



10.0 CONTROL STATEMENTS10.2.6.4 Rules

1. The <end if statement> is a component of the <if then else/else if structure> so it can only be used in that context and has no other usage within the ATLAS language.

10.2.6.5 Notes and Examples

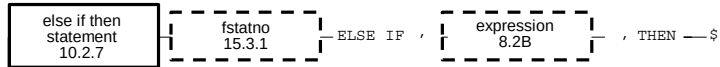
See Section 10.2.3.5 of <procedural segment> (IF THEN ELSE/ELSE IF)

10.2.7 <else if then statement>10.2.7.1 Function

To continue an <if then else/ else if structure> so that further evaluation of conditional expressions can be added.

10.2.7.2 Formal Syntax

FSD / 1579

10.2.7.3 Syntax Diagram10.2.7.4 Rules

1. The <else if then statement> is a component of the <if then else/else if structure> so it can only be used in that context.
2. <expression> must evaluate to a boolean result. The <procedure segment> for ELSE IF will be executed if the result evaluates to TRUE.

10.0 CONTROL STATEMENTS**10.3 WHILE THEN Capability****10.3.1 <while then structure>****10.3.1.1 Function**

To identify the logical condition under which a <procedural segment> of an ATLAS test specification is to be iteratively performed and establishes the boundaries of that <procedural segment>. The functional flow of this structure is shown in Figure 10-4.

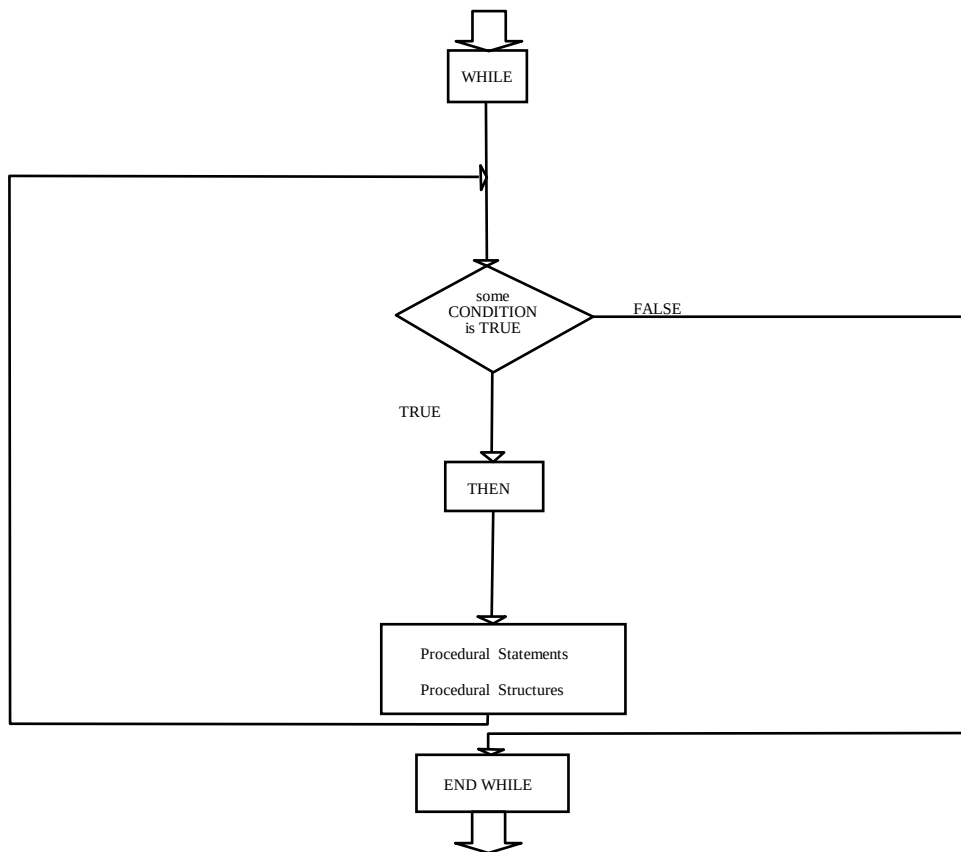


Figure 10-4 Functional Flow Diagram of <while then structure>

10.3.1.2 Formal Syntax

FSD / 1584

10.3.1.3 Syntax Diagram

10.0 CONTROL STATEMENTS10.3.1.4 Rules

1. 1.The <procedural segment> will be iteratively executed while the stated condition is TRUE.
2. 2.If the stated condition is not fulfilled, execution continues with the next statement following the <end while statement>.

10.3.1.5 Notes and Examples

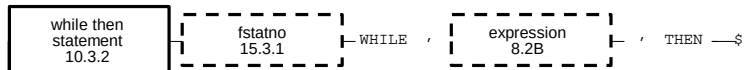
See Section 10.3.3.5 of <procedural segment> (WHILE THEN)

10.3.2 <while then statement>10.3.2.1 Function

To introduce the <while then structure> and to specify the logical conditions which will cause the <procedural segment> to be executed at each iteration.

10.3.2.2 Formal Syntax

FSD / 1590

10.3.2.3 Syntax Diagram10.3.2.4 Rules

1. <expression> must evaluate to a boolean result. The <procedural segment> for WHILE will be executed if the result evaluates to TRUE.

10.3.2.5 Notes and Examples

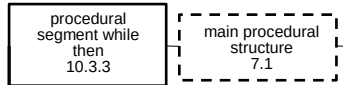
See Section 10.3.3.5 of <procedural segment> (WHILE THEN)

10.0 CONTROL STATEMENTS10.3.3 <procedural segment while then>10.3.3.1 Function

To define the statements to be performed during each iteration of the <while then structure>.

10.3.3.2 Formal Syntax

FSD / 1596

10.3.3.3 Syntax Diagram10.3.3.4 Rules

1. Statement numbers within a <procedural segment> may only be referenced from within the bounds of that segment.
2. Entry points must not appear within a <procedural segment>.

10.3.3.5 Notes and Examples

```

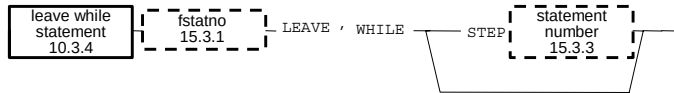
102000      SETUP, DC SIGNAL USING 'DMM', ... $
           10      CONNECT, DC SIGNAL USING 'DMM', ... $
           20      READ, (VOLTAGE), ... $
           40      WHILE 'MEASUREMENT' LT 15.3, THEN $
           50              READ, (VOLTAGE), ... $
           60      END, WHILE $
  
```

10.0 CONTROL STATEMENTS**10.3.4 <leave while statement>****10.3.4.1 Function**

To cause termination of a <while then structure>.

10.3.4.2 Formal Syntax

FSD / 1600

10.3.4.3 Syntax Diagram**10.3.4.4 Rules**

1. When a statement number reference is used, it must be the statement number of a <while then statement> which introduces a <while then structure> containing the <leave while statement>.
2. When a statement number reference is omitted, the <leave while statement> is taken to refer to the innermost <while then structure> containing the <leave while statement>.
3. Any structures that contain a <leave while statement> which are contained within the <while then structure> to which it applies, are also exited upon execution of the <leave while statement>.

10.3.4.5 Notes and Examples

```

200010    WHILE, 'MEASUREMENT' LT 10, THEN $
          20      CALCULATE, 'BIAS' = 'BIAS' - .5 $
          30      IF, 'BIAS' LT 0, THEN $
          40          LEAVE, WHILE STEP 200010 $
          50      END, IF $
          60      SETUP, ... $
          70      READ, (VOLTAGE), DC SIGNAL USING 'DMM', ... $
          80      END, WHILE $
          90      OUTPUT, ... $

```

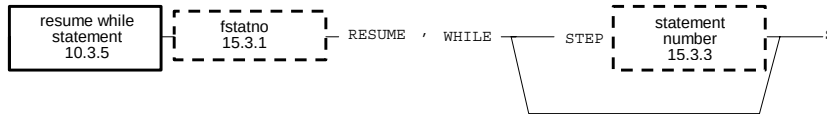
In the preceding example, the <while then structure> loop will be terminated when <data store> 'BIAS' becomes negative, regardless of the value of 'MEASUREMENT'.

10.0 CONTROL STATEMENTS**10.3.5 <resume while statement>****10.3.5.1 Function**

To cause the next iteration of a <while then structure> to be initiated.

10.3.5.2 Formal Syntax

FSD / 1606

10.3.5.3 Syntax Diagram**10.3.5.4 Rules**

1. When a statement number reference is used, it must be the statement number of a <while then statement> which introduces a <while then structure> containing the <resume while statement>.
2. When a statement number reference is omitted, the <resume while statement> is taken to refer to the innermost <while then structure> containing the <resume while statement>.
3. Any structures that contain a <resume while statement> which are contained within the <while then structure> to which it applies, are exited upon execution of the <resume while statement>.

10.3.5.5 Notes and Examples

In the following example, the <calculate statement> and the <setup statement> will not be executed once the <data store> 'BIAS' has reached a value less than zero.

```

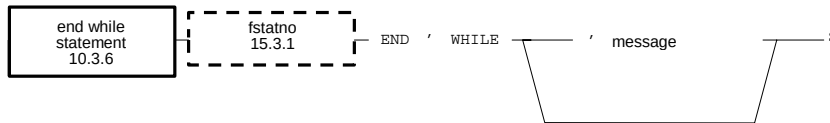
250010      WHILE, 'MEASUREMENT' LT 10, THEN $
20          READ, (VOLTAGE), DC SIGNAL USING 'DMM', ... $
30          IF, 'BIAS' LT 0, THEN $
40              RESUME, WHILE STEP 250010 $
50          END, IF $
60          CALCULATE, 'BIAS'='BIAS' -0.5 $
70          SETUP, ... $
80      END, WHILE $
  
```

10.0 CONTROL STATEMENTS10.3.6 <end while statement>10.3.6.1 Function

To delimit the <while then structure>. Optional commentary may be included following the WHILE field to identify the <while then structure> which it delimits.

10.3.6.2 Formal Syntax

FSD / 1612

10.3.6.3 Syntax Diagram10.3.6.4 Rules

1. The <end while statement> is a component of the <while then structure> so it can only be used in that context and has no other usage within the ATLAS language.

10.3.6.5 Notes and Examples

See Section 10.3.3.5 of <procedural segment> (WHILE THEN)

10.0 CONTROL STATEMENTS**10.4 FOR THEN Capability****10.4.1 <for then structure>****10.4.1.1 Function**

To identify a requirement for the repetitive execution of a <procedural segment>, to establish the bounds of that <procedural segment> and to identify a control variable which will be assigned a value prior to each iteration of that <procedural segment>. The functional flow of this structure is shown in Figure 10-5.

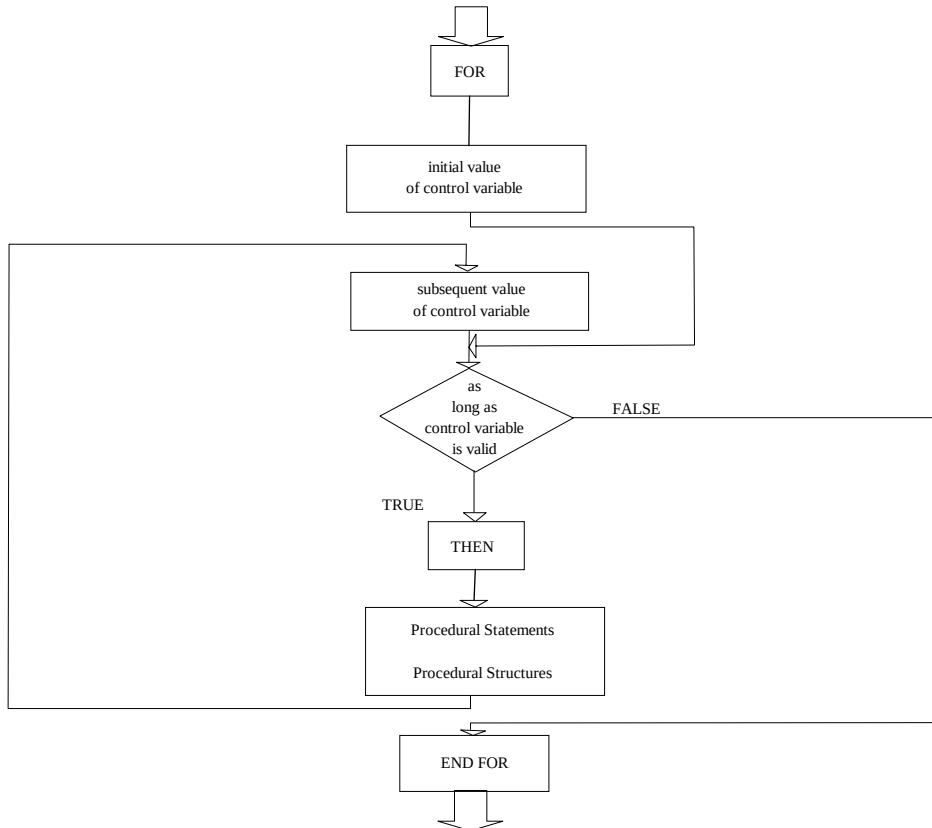


Figure 10-5 Functional Flow Diagram of <for then structure>

10.4.1.2 Formal Syntax

FSD / 1618

10.4.1.3 Syntax Diagram

10.0 CONTROL STATEMENTS**10.4.1.4 Rules**

1. The number of iterations of the <procedural segment> is specified by the form of the <for then statement>.
2. Upon completion of the final iterative execution of the <procedural segment> execution continues with the next statement following the <end for statement>.

10.4.1.5 Notes and Examples

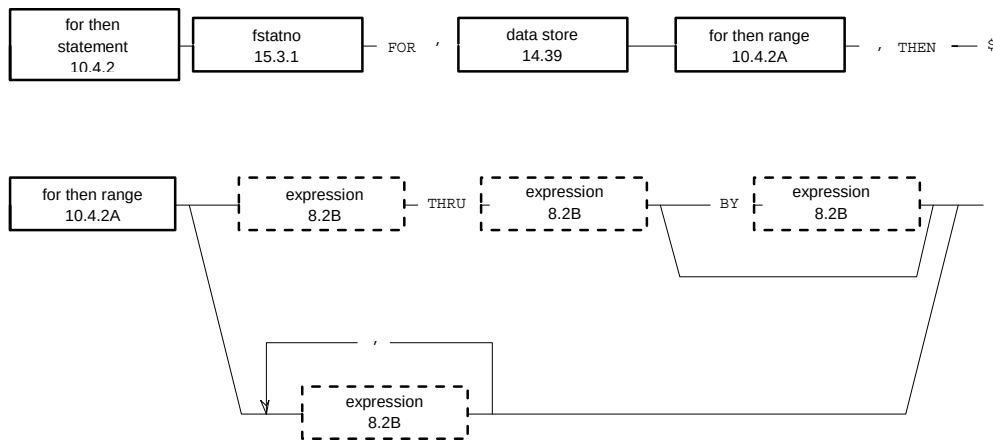
See Section 10.4.3.5 of <procedural segment> (FOR THEN)

10.4.2 <for then statement>**10.4.2.1 Function**

To introduce the <for then structure>, to identify the control variable and to specify the values that the control variable will take for each iteration of the <procedural segment>. The <for then statement> may be broken down to two forms. One form specifies a list of independent values for the control variable. The other forms a computed sequence of values for the control variable.

10.4.2.2 Formal Syntax

FSD / 1624

10.4.2.3 Syntax Diagram**10.4.2.4 Rules****10.4.2.4.1 Rules Applicable to Both Forms of the <for-then statement>**

1. The control variable must be a single element of base type INTEGER, DECIMAL, ENUMERATION or CHAR.
2. <expression> must evaluate to an INTEGER, DECIMAL or CHAR base type or to an <enumeration element>.
3. Upon completion of the final iterative execution of the procedural segment, the value of the control variable is undefined.

10.0 CONTROL STATEMENTS

10.4.2.4.2 Rules Applicable to the Value List Form of the <for then statement>

1. The list of values for the control variable has the form $e_1, e_2, e_3 \dots e_n$ where e_1 thru e_n represent any valid arithmetic expression that is consistent with the control variable type. On the first iteration, the control variable is assigned the value of e_1 . On successive iterations the control variable is assigned the values of $e_2, e_3 \dots e_n$, thus the number of iterations will equal the number of items in the value list. Each expression is evaluated once upon initial entry into the <for then structure>.

10.4.2.4.3 Rules Applicable to the Value Sequence Form of the <for then statement> for INTEGER and DECIMAL control variables

1. The range of values for the control variable has the form e_1 THRU e_2 or e_1 THRU e_2 BY e_3 . On the first iteration the control variable is assigned the value of e_1 . On successive iterations, e_3 (or 1, if BY e_3 is not specified) is added to the control variable. If the value of e_3 is negative the value of e_1 is decremented. The iterative sequence is continued whilst $e_1 \#$ control variable $\# e_2$ (if $e_1 < e_2$) or $e_2 \#$ control variable $\# e_1$ (if $e_2 < e_1$). When the value of the control variable goes outside of these boundaries, control is transferred to the statement following the <end for statement>. The values of e_1, e_2 and e_3 are each evaluated once upon entry into the <for-then structure>.

10.4.2.4.4 Rules Applicable to the Value Sequence Form of the <for then statement> for ENUMERATION and CHAR control variables

1. The range of values for the control variable has the form e_1 THRU e_2 , where $ORD(e_2)$ must be greater than or equal to $ORD(e_1)$. On the iteration the control variable is assigned the value of e_1 . On successive iterations the control variable takes the value of its successor. The iterative sequence is continued whilst the control variable is in $[e_1 \text{ THRU } e_2]$. When the value of the control variable is not in $[e_1 \text{ THRU } e_2]$, control is transferred to the statement following the <end for statement>. The values of e_1 and e_2 are each evaluated once upon entry into the <for then structure>.

10.4.2.5 Notes and Examples

See Section 10.4.3.5 of <procedural segment> (FOR THEN)

10.4.3 <procedural segment for then>

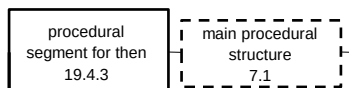
10.4.3.1 Function

To define those statements to be performed during each iteration of the <for then structure>.

10.4.3.2 Formal Syntax

FSD / 1638

10.4.3.3 Syntax Diagram



10.0 CONTROL STATEMENTS**10.4.3.4 Rules**

1. Statement numbers within a <procedural segment> may only be referenced from within the bounds of that segment.
2. Entry points must not appear within a <procedural segment>.

10.4.3.5 Notes and Examples

```

000100      FOR, 'I' = 1 THRU 20, THEN $
      10          READ, (VOLTAGE INTO 'RESULT' ('I')), DC SIGNAL USING 'DMM',
                  VOLTAGE MAX 20 V, CNX HI J5 LO EARTH $
      20          DELAY, 250 MSEC $
      99      END, FOR $

000200      FOR, 'STEP' = 'X', 'X' + 2, 'X', 'X' - 2, 'X', THEN $
      10          SETUP, DC SIGNAL, VOLTAGE 'STEP' RANGE -20 V TO 20 V,
                  CNX HI J14-7 LO COMMON $
      20          DELAY, 500 USEC $
      99      END, FOR $

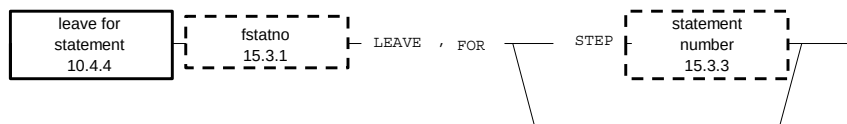
```

10.4.4 <leave for statement>**10.4.4.1 Function**

To cause the termination of a <for then structure>.

10.4.4.2 Formal Syntax

FSD / 1642

10.4.4.3 Syntax Diagram**10.4.4.4 Rules**

1. When a statement number reference is used, it must be the statement number of a <for then statement> which introduces a <for then structure> containing the <leave for statement>.
2. When a statement number reference is omitted, the <leave for statement> is taken to refer to the innermost <for then structure> containing the <leave for statement>.
3. Any structures that contain a <leave for statement> which are contained within the <for then structure> to which it applies, are also exited upon execution of the <leave for statement>.

10.4.4.5 Notes and Examples

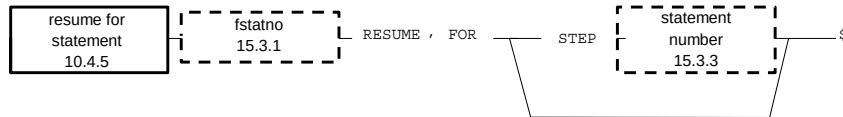
See Section 10.4.5.5 of <resume for statement>.

10.0 CONTROL STATEMENTS**10.4.5 <resume for statement>****10.4.5.1 Function**

To cause the next iteration of a <for then structure> to be initiated.

10.4.5.2 Formal Syntax

FSD / 1648

10.4.5.3 Syntax Diagram**10.4.5.4 Rules**

1. When a statement number reference is used, it must be the statement number of a <for then statement> which introduces a <for then structure> containing the <resume for statement>.
2. When a statement number reference is omitted, the <resume for statement> is taken to refer to the innermost <for then structure> containing the <resume for statement>.
3. Any structures that contain a <resume for statement> which are contained within the <for then structure> to which it applies, are exited upon execution of the <resume for statement>.

10.4.5.5 Notes and Examples

In the following example, the <for then structure> loop will be terminated if the <verify statement> results in a NOGO-HI condition. The next iteration of the <for then structure> loop will be initiated without calculation of a new value for 'OFFSET' when the condition is NOGO-LO.

```

300010    APPLY, DC SIGNAL, VOLTAGE .5V, ... $
20        CALCULATE, 'OFFSET' = 0 $
30        FOR, 'STEPV' = 1 THRU 12 BY .5, THEN $
40            CALCULATE, 'V' = 'STEPV' - 'OFFSET' $
50            CHANGE, DC SIGNAL, VOLTAGE 'V' ... $
60            VERIFY, (VOLTAGE), DC SIGNAL, NOM ... $
70            IF, NOGO, THEN $
80                IF, HI, THEN $
90                    LEAVE, FOR STEP 300030 $
300100        ELSE $
10            RESUME, FOR STEP 300030 $
20            END, IF $
30            END, IF $
40            CALCULATE, 'OFFSET' = 'OFFSET' + (.175 * 'STEPV') $
50        END, FOR $

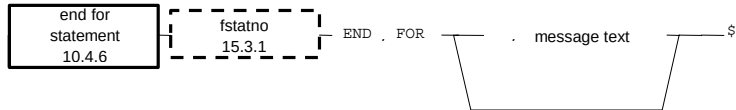
```

10.0 CONTROL STATEMENTS10.4.6 <end for statement>10.4.6.1 Function

To delimit the <for then structure>. Optional commentary may be included following the FOR field to identify the <for then structure> which it delimits.

10.4.6.2 Formal Syntax

FSD / 1654

10.4.6.3 Syntax Diagram10.4.6.4 Rules

1. The <end for statement> is a component of the <for then structure> so it can only be used in that context and has no other usage within the ATLAS language.

10.4.6.5 Notes and Examples

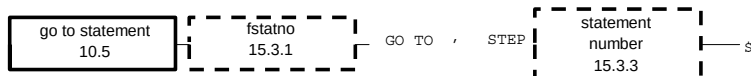
See Section 10.4.3.5 of <procedural segment> (FOR THEN)

10.0 CONTROL STATEMENTS10.5 <go to statement>10.5.1 Function

To direct the tester to some statement other than the one following. The destination statement must be preceded by a "B" Flag statement(See Section 15.3.2.4 Rule 3). A GO TO statement is unconditional and therefore causes the sequence to be diverted as indicated every time the <go to statement> is executed.

10.5.2 Formal Syntax

FSD / 1660

10.5.3 Syntax Diagram10.5.4 Rules

1. The target statement number(s) referenced by a <go to statement> in any structure other than an <if then else/else if structure> must be within the containing structure.
2. Only the first statement of an <if then else/else if structure>, a <while then structure> or a <for then structure> may be the destination of a <go to statement> that is located outside the structure.
3. A <go to statement> within a <while then structure> or <for then structure> which references the END statement will cause another iteration of the structure to be executed. It is preferable to use the <resume while statement> and <resume for statement> to cause reiteration of these structures.

10.5.5 Notes and Examples

```

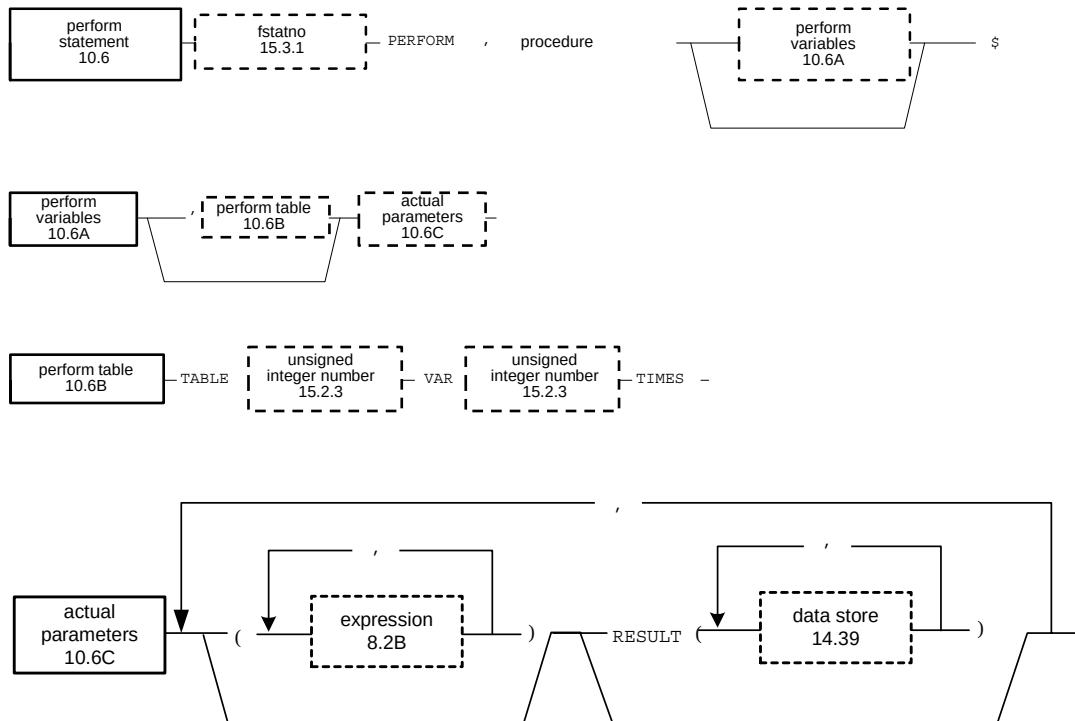
543210 GO TO, STEP 837466 $
B $
837466 COMPARE, 'MEASUREMENT', GT 5.0 $
  
```

10.0 CONTROL STATEMENTS10.6 <perform statement>10.6.1 Function

To invoke a sequence of one or more ATLAS statements identified by a <define procedure structure>.

10.6.2 Formal Syntax

FSD / 1666

10.6.3 Syntax Diagram10.6.4 Rules

1. The <perform statement> is written with the <procedure> label following the verb.
2. The actual parameters must be written in the <perform statement> in exactly the same sequence as the formal parameters are listed in the <define procedure statement>.

10.0 CONTROL STATEMENTS

The type of each <expression> in the actual <parameter> list must exactly match the type of the corresponding <variable identifier> in the formal <parameter> list. If a multi-element <structured type> such as RECORD, STRING or ARRAY is used as an actual <parameter>, for example array, it must have at least as many elements as the declared array from the formal <parameter>s. If a formal <parameter> has n elements and the corresponding actual <parameter> has m elements, where m is greater than n, then the first n elements of the actual array are employed. Each <parameter> must be assignment compatible with the corresponding formal <parameter>.

3. Multiple executions of the same <procedure> with different values for the actual parameters may be indicated with one <perform statement> by inserting a TABLE...VAR...TIMES entry following the label. The blanks of this entry will contain the number of parameters in the <procedure> and the number of executions of the <procedure> covered by the <perform statement>. Following this entry, the required sets of actual parameters will be written in any layout desired by the writer; but normally the arrangements will be tabular for easy readability.
4. When utilized, the arguments between parentheses must exactly correspond to the sequence and positions of the <parameter>s in the <define procedure statement> of the referenced <procedure>.
5. An actual <parameter> which corresponds to a formal <parameter> established in the RESULT field of a <define procedure statement> must be a declared <expression> and therefore constants, literals, and <expression>s may not be used as RESULT <parameter>s of a <procedure>.

10.6.5 Notes and Examples

See Section 6.7 <define procedure structure>

10.7 <finish statement>

10.7.1 Function

To terminate testing and return the test equipment to a quiescent state.

10.7.2 Formal Syntax

FSD / 1699

10.7.3 Syntax Diagram



10.7.4 Rules

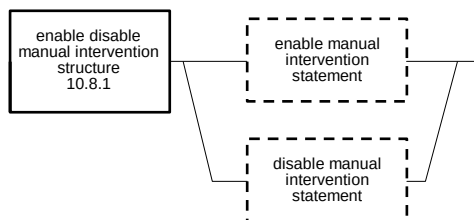
1. FINISH is always written alone with no noun or modifier.
2. No statements are executed after the execution of a <finish statement>.

10.0 CONTROL STATEMENTS**10.8 ENABLE/DISABLE MANUAL INTERVENTION Capability****10.8.1 <enable disable manual intervention structure>****10.8.1.1 Function**

To define a structure in which any statement execution may be interrupted by manual intervention. If such intervention occurs, the MANUAL INTERVENTION condition is set true. This condition may be interrogated by either an <if then statement> or a <while statement>.

10.8.1.2 Formal Syntax

FSD / 1703

10.8.1.3 Syntax Diagram**10.8.1.4 Rules**

1. Enables the operator to utilize manual input facilities in parallel with the operation of subsequent statements where it is inappropriate to halt testing while the operator decides when to give the indication.
2. Each use of an <enable manual intervention statement> should be preceded an appropriate <output statement> to inform the operator of the actions he is to take and under what circumstances.
3. There should be a <disable manual intervention statement> in each exit from a section of specification where MANUAL INTERVENTION is activated.

10.8.1.5 Notes and Examples

The <enable/disable manual intervention structure> is dynamic in nature in that it is not necessary to have a <disable manual intervention statement> for each <enable manual intervention statement> because the sequence of execution is a function of the topology of the specification. However, to render a specification more readable it is recommended that each <enable manual intervention statement> is paired with a <disable manual intervention Statement> to provide a static structure.

```

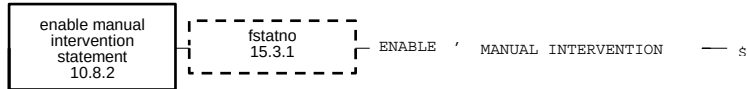
123456    ENABLE, MANUAL INTERVENTION $
60        OUTPUT, TEXT, FROM C'DEPRESS GO IF KOD INDICATOR FLAG
           IS ACTIVATED' $
70        APPLY, ... $
.
.
80        IF, MANUAL INTERVENTION THEN $
90        GO TO, STEP 123500 $
.
.
.
95 END, IF $
B $
123500    DISABLE, MANUAL INTERVENTION $
10        OUTPUT, TEXT, FROM C'KOD ANNUNCIATOR OK' $
  
```

10.0 CONTROL STATEMENTS**10.8.2 <enable manual intervention statement>****10.8.2.1 Function**

To activate MANUAL INTERVENTION interrupts.

10.8.2.2 Formal Syntax

FSD / 1710

10.8.2.3 Syntax Diagram**10.8.2.4 Rules**

1. Enables the operator to utilize manual input facilities in parallel with the operation of subsequent statements where it is inappropriate to halt testing while the operator decides when to give the indication.
2. Each use of an <enable manual intervention statement> should be preceded by an appropriate <output statement> to inform the operator of the actions to be taken and under what circumstances they are to be taken.

10.8.2.5 Notes and Examples

See Section 10.8.1.5 of <enable/disable manual intervention structure>.

10.8.3 <disable manual intervention statement>**10.8.3.1 Function**

To de-activate MANUAL INTERVENTION interrupts.

10.8.3.2 Formal Syntax

FSD / 1715

10.8.3.3 Syntax Diagram**10.8.3.4 Rules**

1. There should be a <disable manual intervention statement> in each exit from a section of specification where MANUAL INTERVENTION is activated.

10.8.3.5 Notes and Examples

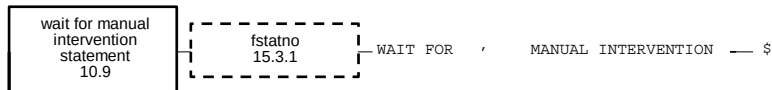
See Section 10.8.1.5 of <enable/disable manual intervention structure>.

10.0 CONTROL STATEMENTS10.9 <wait for manual intervention statement>10.9.1 Function

To suspend the execution of the test specification until the operator accomplishes some manual operation to reinitiate the execution of the test procedure.

10.9.2 Formal Syntax

FSD / 1720

10.9.3 Syntax Diagram10.9.4 Rules

1. The following statement will be postponed until the operator accomplishes some manual operation to re-initiate the execution of the test specification.
2. Each use of a <wait for manual intervention statement> should be preceded by an appropriate <output statement> to inform the operator of the actions he has to take and under what circumstances.

10.0 CONTROL STATEMENTS

10.10 <enable digital configuration statement>

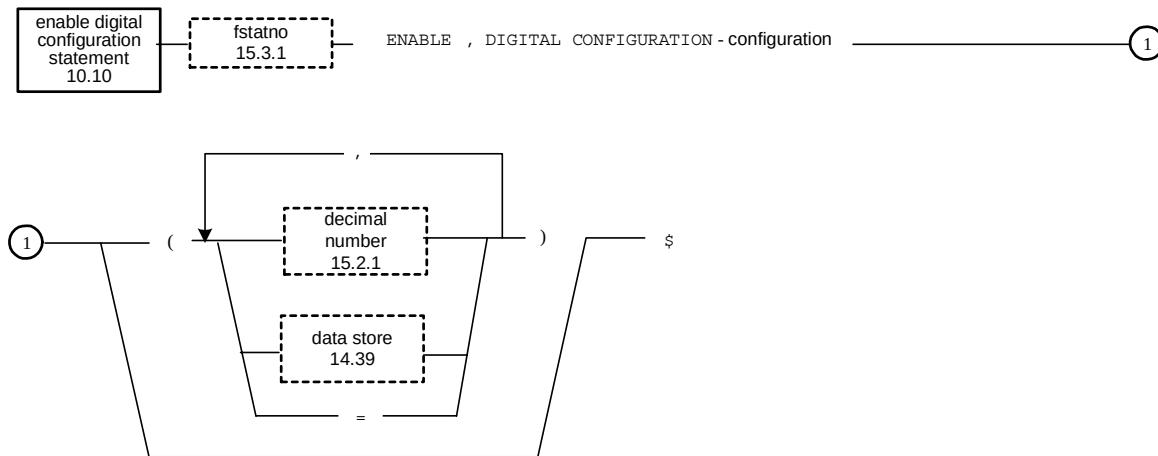
10.10.1 Function

To ENABLE a DEFINEd <configuration> so that the <configuration> can be referenced by either a <stimulate statement> or a <sense statement> or combinations of these statements.

10.10.2 Formal Syntax

FSD / 1725

10.10.3 Syntax Diagram



10.10.4 Rules

1. Only one <configuration> can be ENABLEd at a time. When an <enable digital configuration statement> is executed, a previously ENABLEd <configuration> is disabled so that it cannot then be referenced for digital testing although it will remain connected to the UUT unless disconnected in accordance with Rule 2.
2. The specific actions of ENABLE in execution order are:
 - a. To disconnect any connections of other connected <configuration>s that are common with the connections specified in the <configuration> being ENABLEd.
 - b. To set up drivers/sensors of the <configuration> to the required conditions.
 - c. To connect the new <configuration> to the UUT.
3. The first time that a <configuration> is ENABLEd all bits DEFINEd as having a LEVEL-LOGIC-HIZ state will be in this state. For all other bits the initial states will be LEVEL-LOGIC-ZERO, or LEVEL-LOGIC-QUIES for SERIAL signals DEFINEd as having this characteristic.
4. If the referenced <define digital configuration structure> has parenthetical <expression>s, the values for all of these must be specified in ENABLE. These values are written immediately following the <configuration> label in the same sequence as they appear in the referenced <define digital configuration structure>.

10.0 CONTROL STATEMENTS

5. If an already ENABLEd <configuration> is re-ENABLEd there will be no disconnect and connect actions. By changing the values of parenthetical variables this feature can be used to change <configuration> characteristics whilst it remains connected to the UUT. The values of parenthetical <expression>s which are to remain unchanged can, optionally, be omitted by substituting one equals sign "=" between the appropriate commas.
6. The subsequent ENABLE of a disabled <configuration> shall be as if it was ENABLEd for the first time in accordance with Rule 3.

10.10.5 Notes and Examples

Effectively the execution of an <enable digital configuration statement> prohibits the use of all previously enabled <digital configuration>s even though they may remain connected to the UUT so as to minimize changes to the electrical environment of the UUT. Thus there is no need for a disable digital configuration statement in ATLAS.

10.0 CONTROL STATEMENTS**10.11 ESCAPE Mechanism**

The escape mechanism allows a limited form of parallel monitoring of a test situation. If a stated test condition occurs, the escape mechanism causes the program execution sequence to be interrupted and either the execution sequence to branch or a procedure to be executed before program execution is resumed.

10.11.1 <enable escape statement>**10.11.1.1 Function**

To enable an escape mechanism which will occur if a specified condition becomes true. The escape mechanism is active until the next corresponding <disable escape statement>.

10.11.1.2 Formal Syntax

FSD / 1740

10.11.1.3 Syntax Diagram**10.11.1.4 Rules**

1. The branching to a called <procedure> follows the same rules as for a <perform statement>.
2. The called procedure cannot have parameters.
3. When an <escape indicator> is referenced in an <enable escape statement>, the escape must be disabled before that <escape indicator> is referenced in another <enable escape statement>.

10.11.1.5 Notes and Examples

NOTE: 1It is possible to make results of a procedure executed because of an ESCAPE available by storing them in GLOBAL <data store>s.

EXAMPLE:

```
100000 ENABLE, ESCAPE TO PROCEDURE 'LOCSUB' IF MAX-TIME $
.
.
.
109900 DISABLE, ESCAPE MAX-TIME $
```

10.0 CONTROL STATEMENTS

10.11.2 <disable escape statement>

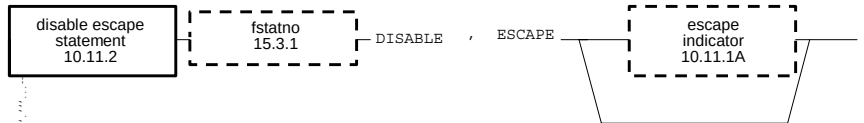
10.11.2.1 Function

To disable an escape mechanism previously activated by an <enable escape statement>.

10.11.2.2 Formal Syntax

FSD / 1755

10.11.2.3 Syntax Diagram



10.11.2.4 Rules

1. If <escape indicator> is used, the escape mechanism associated with this <escape indicator> is disabled.
2. If <escape indicator> is not used, all enabled escapes are disabled.

11.0 SIGNAL ORIENTED STATEMENTS

11.0 Signal Oriented Statements

11.1 <procedural statements signal>

11.1.1 Function

Signal oriented procedural statements consist of single action and multiple action statements which describe source, sensor, and load functions of signals relative to the UUT. The multiple action statements are functionally equivalent to sequences of single action signal procedural statements and other non-signal procedural statements, for example; <delay statement>.

Critical Actions

Each multiple-action signal oriented verb has a critical action for the purpose of referencing timing and synchronization of test sequences.

Timing Control

Statement timing is realized by controlling the time of execution of <single action verb>, or, for a <multiple action verb>, the time of execution of the critical action.

UUT Failures

For analog testing, ATLAS provides for the identification of failures by the use of flags GO/NOGO etc. For both analog and digital testing, it also provides a capability which creates continual fault monitoring by means of the <escape mechanism>.

To avoid tedious interrogation of ILLEGAL STATE INDICATORS and flags in complex digital test schedules, the <escape mechanism> can be used to cause program control to be automatically passed to a different <procedure> or STEP number for failures.

It is possible within the <procedure> invoked by the <escape mechanism> to adopt a number of strategies according to local requirements.

Specific possibilities are:

1. Log the symptoms of the failure for subsequent analysis inside or outside of the current test environment, and continue testing.
2. Enter a diagnostic procedure using data-library and/or guided probe techniques.

State Diagrams

1. Description

There are practical limitations to the combinational sequences of signal oriented statements which can be implemented by a single virtual resource.

A state diagram is a convenient pictorial representation of the behavior of a system to the imposition of external actions.

Figures 11-1 through 11-4 illustrate the states which can occur in virtual test resources, with or without the requirement to monitor events, of SOURCE, LOAD and SENSOR type.

11.0 SIGNAL ORIENTED STATEMENTS

These diagrams assume that once a virtual test resource has been prepared for application or making a measurement, the test function is completed before the virtual resource is prepared for the next function. Changes can be made to a virtual resource and so prepare it for its next function without necessarily reverting back to an unallocated state which requires that a virtual resource is disconnected. The connections between a particular virtual test resource and a UUT can be maintained between one test function and the next when each is determined by separate ATLAS statements in a test sequence by reverting to a prepared state.

Each state diagram represents the combinational sequences of signal oriented statements which cause a particular single virtual resource to assume discrete states during the execution of one test sequence using the particular virtual resource. It must be recognized that other test functions involving both signal and non-signal oriented statements can be executed in addition to the test sequence which is the subject of a particular state diagram. These other test functions are also controlled by sequences of ATLAS statements which, if necessary, can be interpolated into the sequence of statements controlling a virtual test resource related to a particular state diagram. Obviously, state diagrams can be prepared for each of the virtual test resources used for these other test functions to represent the states of their virtual resources at any given time according to their particular controlling sequence of ATLAS statements which impose the states.

The conventions in these state diagrams are:

- a. Oval enclosures contain the states that can be assumed by a system.
- b. Directional linking lines denote those changes of state which are permitted in a system.
- c. Each directional linking line, annotated with an external action, joins the state which is exited because of the action which is imposed to the final state which is entered after the action has been imposed.
- d. Progress through a state diagram is from State 1 through higher number states with or without recursion to intermediate states and back to State 1.
- e. A directional linking line between two states which bypass other intermediate states on other lines between the same two states means that all the bypassed states have in effect been passed through during the transition.
- f. A directional line in the form of a loop from one state back to the same state without passing through any other intermediate states means that the external action which annotates the loop causes all the appropriate states bypassed by the loop to be in effect passed through during the transition.
- g. Directional linking lines flowing from a lower to a higher number state illustrate the transitions which occur for either the application of a SOURCE or LOAD or the making of a measurement towards completion.
- h. Directional linking lines flowing from a higher to a lower number state illustrate the transitions which occur after either the application of a SOURCE or LOAD or the making of a measurement has been completed.
- i. When the oval enclosures of two states are contiguous, such as in state diagrams with a restricted set of virtual test resources so that a directional linking line is non-existent between the two states, no external action is required for the transition to occur between the two states because they are equivalent for that restricted set of virtual test resources.

The states which may be attained by a virtual test resource in accordance with the state diagrams are specified by the definitions of the single action verbs given later in this chapter. The following brief descriptions are intended to assist in the understanding of the state diagrams. They are not to be interpreted as superseding the definitions that are included in the later chapter for each single action verb.

11.0 SIGNAL ORIENTED STATEMENTS

<u>STATE</u>	<u>DESCRIPTION</u>
UNALLOCATED	The virtual test resource is not committed to any particular UUT function and is in a quiescent state available for use.
SET	The virtual test resource has been committed to a particular UUT function but it has not been connected to the UUT. If no synchronization is required, the virtual test resource will have been set at the required state. If synchronization is required, the virtual test resource will have been set to operate at the required state when synchronization occurs.
PREPARED	The virtual test resource in its set state has been connected to the UUT.
ARMED	This state applies only to sensor types of virtual test resource. It indicates that the sensor virtual test resource is ready to take a measurement. Any event detectors that are required to take the measurement are not enabled.
UNARMED	This state applies only to sensor types of virtual test resource. It indicates that any event detectors associated with the sensor virtual test resource are enabled but the sensor resource itself is not ready to take a measurement.
READY	All events which are required to cause a source type of virtual test resource to change to the required state or to cause a sensor type of virtual test resource to take its measurement have been enabled. The virtual test resource is waiting for these events to occur.
APPLIED	This state applies only to source and load types of virtual test resource. It indicates that the signal has achieved its required state at the UUT.
MEASURED	This state applies only to sensor types of virtual test resource. It indicates that the sensor virtual test resource has taken its measurement(s).
COMPLETED	This state applies only to sensor types of virtual test resource. It indicates that the measurement(s) have been collected and that the sensor virtual test resource is connected to the UUT awaiting further instructions.

2. Rules for Transfer between State Diagrams

State Diagrams in Figures 11-1 and 11-2 differ only in whether EVENT MONITOR virtual test resources are employed. The State Diagrams in Figures 11-3 and 11-4 are similarly related. Some ATLAS verbs can either introduce or remove the requirement for an EVENT MONITOR. In doing this they require a transfer between State Diagrams in which case the following four rules apply:

- a. When a sequence of statements uses a particular SOURCE or LOAD virtual test resource without an EVENT MONITOR and the next statement uses the same virtual resource with an EVENT MONITOR, then the introduction of the EVENT MONITOR must only occur when the virtual resource is in the same state as illustrated by the two state diagrams in Figures 11-1 and 11-2. Thus, the introduction of an EVENT MONITOR can only occur when the SOURCE or LOAD virtual resource is in one of the states 1 through 3 and 5.
- b. When a sequence of statements uses a particular SOURCE or LOAD virtual test resource with an EVENT MONITOR and the next statement uses the same virtual resource without an EVENT MONITOR, then the release of the EVENT MONITOR must only occur when the virtual resource is in the same state as illustrated by state diagrams in Figures 11-1 and 11-2. Thus the release of an EVENT MONITOR can only occur when the SOURCE or LOAD virtual resource is in one of the states 1 through 3 and 5.

11.0 SIGNAL ORIENTED STATEMENTS

- c. When a sequence of statements uses a particular SENSOR virtual test resource without an EVENT MONITOR and the next statement uses the same virtual resource with an EVENT MONITOR, then the introduction of the EVENT MONITOR must only occur when the virtual resource is in the same state as illustrated by the two state diagrams in Figures 11-3 and 11-4. Thus, the introduction of an EVENT MONITOR can only occur when the SENSOR virtual resource is in one of the states 1 thru 4 and 7.
- d. When a sequence of statements uses a particular SENSOR virtual test resource with an EVENT MONITOR and the next statement uses the same virtual resource without an EVENT MONITOR, then the release of the EVENT MONITOR must only occur when the virtual resource is in the same state as illustrated by the two state diagrams in Figures 11-3 and 11-4. The release of an EVENT MONITOR can only occur when the SENSOR virtual resource is in one of the states 1 thru 4 and 6 and 7.

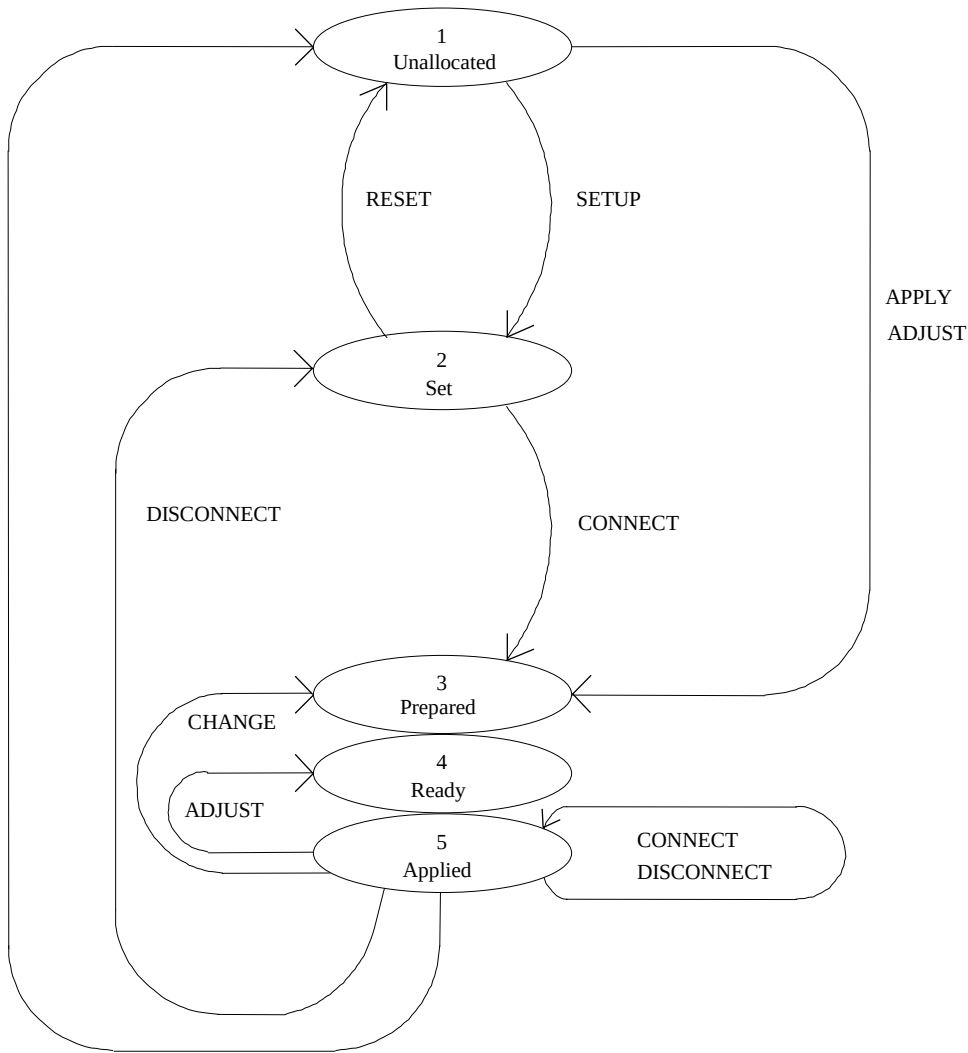
11.0 SIGNAL ORIENTED STATEMENTS

Figure 11-1 – State Diagram for a Source or Load Virtual Test Resource without Event Monitor Using Single-Action and Multi-Action Verbs

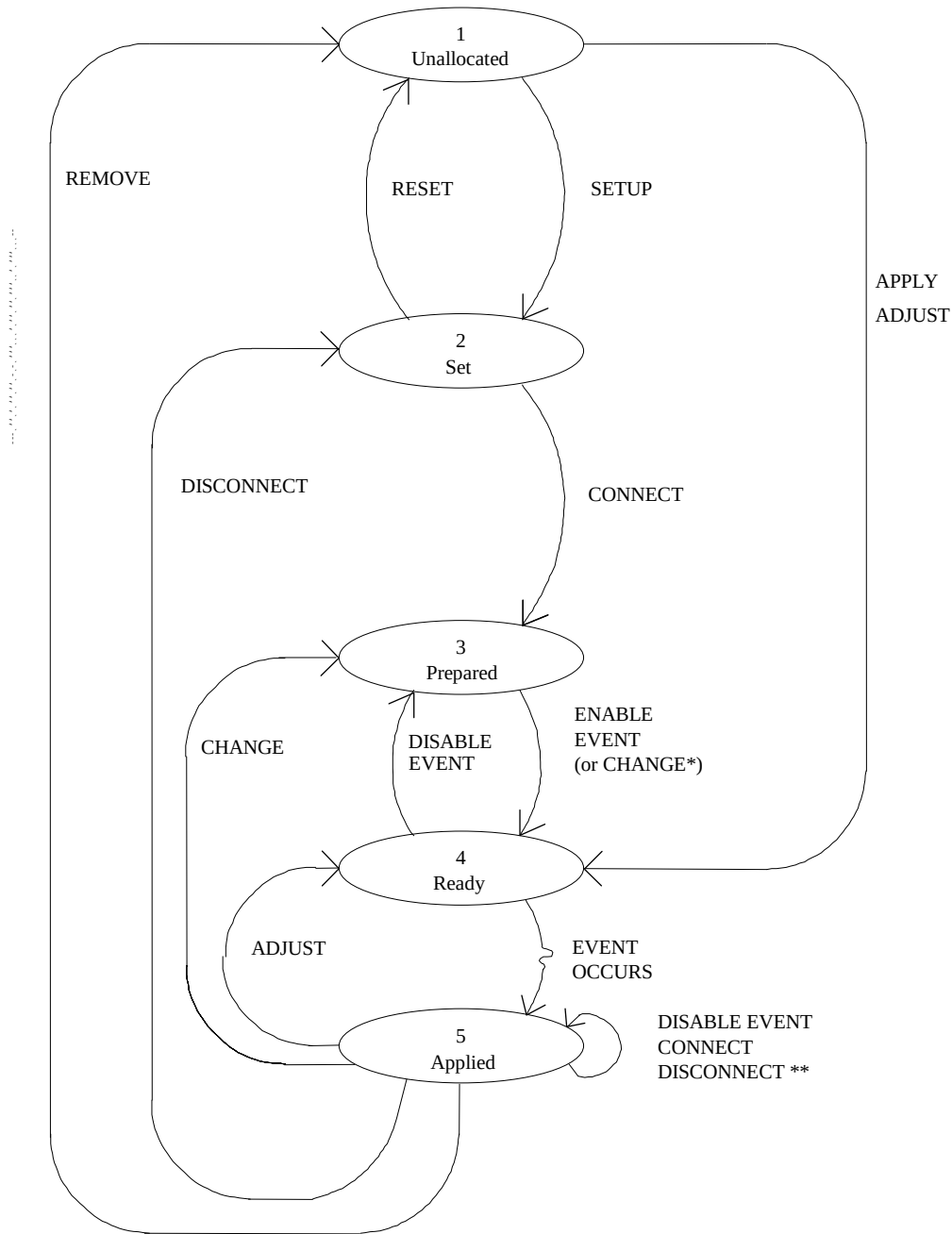
11.0 SIGNAL ORIENTED STATEMENTS

Figure 11-2 – State Diagram for a Source or Load Virtual Test Resource with Event Monitor Using Single-Action and Multi-Action Verbs

Note: *If the event monitors are both unaltered and already enabled it is unnecessary to repeat the enable event statement and so the virtual test resource immediately achieves the READY state.

**Multiple connections of a source or load types of virtual test resource to separate UUT interface points are permitted; thus DISCONNECT actions will leave the resource in the APPLIED state unless all UUT interface connections of the virtual resource are disconnected, at which time it is changed to the SET state.

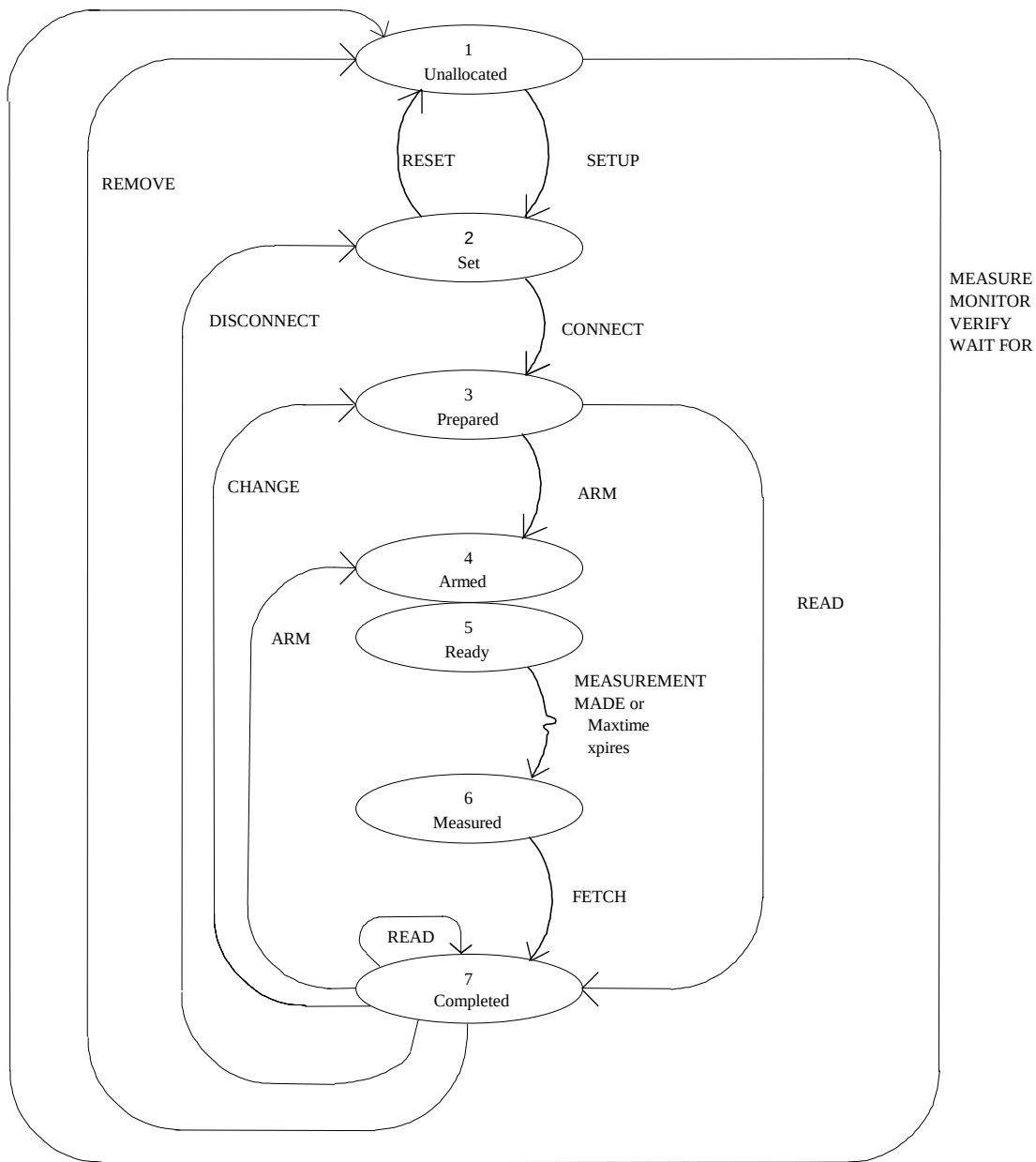
11.0 SIGNAL ORIENTED STATEMENTS

Figure 11-3 State Diagram for a Sensor Virtual Test Resource without Event Monitor for Making a Measurement Using Single Multi Action Verbs

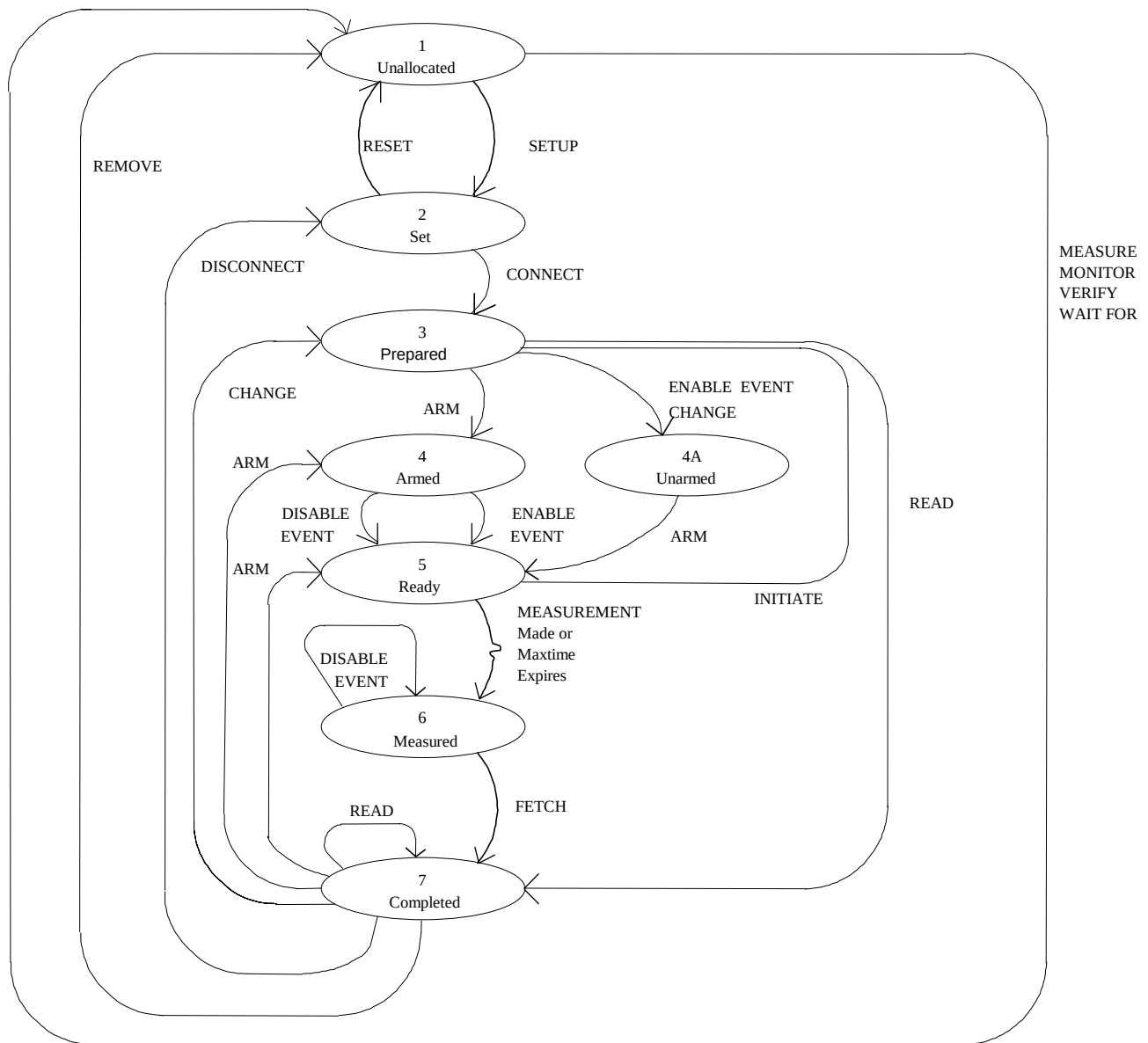
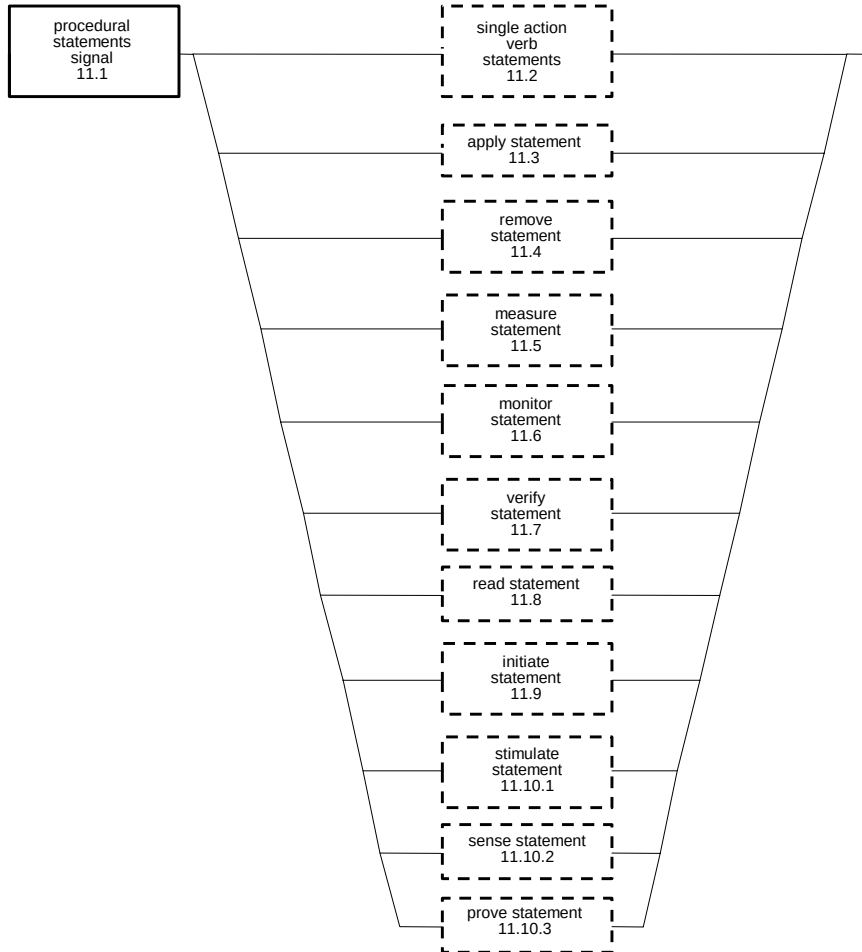
11.0 SIGNAL ORIENTED STATEMENTS

Figure 11-4 State Diagram for a Sensor virtual Test Resource with Event Monitor for Making a Measurement Using Single Multi Action Verbs

11.0 SIGNAL ORIENTED STATEMENTS11.1.2 Formal Syntax

FSD / 1764

11.1.3 Syntax Diagram11.1.4 Rules11.1.4.1 Rules: For Signal Routing using Multiple Action Verbs

1. Analog source and load types of virtual test resource which are attached to the UUT in a procedural statement remain attached to the UUT until an appropriate <remove statement> or <disconnect statement> is executed.
2. Analog sensor types of virtual test resource which are attached to the UUT by either a <verify statement> or a <measure statement> or a <monitor statement> remain attached to the UUT only for the duration of the measurement.
3. Digital source and sensor resources associated with STIMULATE, SENSE and PROVE statements are attached to the UUT by an <enable digital configuration statement> and remain attached to the UUT until a REMOVE ALL or another <enable digital configuration statement> using the same connections is executed.

11.0 SIGNAL ORIENTED STATEMENTS

11.1.4.2 Rules: For Critical Actions

1. For source and load type statements without the requirement to monitor an event, the critical action is the execution of either a <connect statement> or a <disconnect statement>.
2. For source and load type statements with the requirement to monitor an event, the critical action is the execution of either the <enable event statement> or a <disconnect statement>.
3. For sensor type statements the critical action is the execution of the single action which accomplishes the transition to the ready state.

11.1.4.3 Rules: For Timing Control

1. Unless a statement includes a <when field> or is included in a DO structure, testing proceeds at the natural speed of operation of the test system.
2. A <delay statement> and a <wait for event statement> can be used to determine that test sequences are executed at slower speeds than the natural speed of operation of the test system.
3. If a <when field> is used in a statement it specifies the instant when the critical action is to occur.
4. For both analog and digital testing, the timing controls of Rules 1 thru 3 apply to all <procedural statement signal>.
5. The <do simultaneous structure> contains a sequence of signal oriented statements whose critical actions are to be executed simultaneously within the limitations of determining time differences for the sequence.
6. The <do timed digital structure> contains a sequence of digital signal oriented statements whose critical actions are to occur sequentially at a rate specified by the timing information given in the <do timed digital structure>.
7. The <gate field> in a source statement permits the specification of a time interval during which the non-quiescent signal is to be applied. The signal is quiescent outside this time period. The <gate field> may be repetitive, in which case the signal is repeatedly changed between quiescent and non-quiescent.
8. The <gate field> in a sensor statement defines a time interval during which the measurement is to be performed.

11.1.4.4 Rules: For UUT Failures

1. If the full range of the capabilities of a <define digital source statement> and a <define digital sensor statement> are used then the failure possibilities for a STIMULATE and PROVE sequence in order of detection are :
 - a. Failure of an invoked <defined digital source> function because the test system could not drive the defined logic states, thereby causing their associated ILLEGAL STATE INDICATOR to be set TRUE.
 - b. Failure of an invoked <defined digital sensor> function because the sensed UUT signals did not fall into the defined logic states, thereby causing the associated ILLEGAL STATE INDICATOR to be set TRUE.
 - c. Failure of the evaluation function because the digital value, or sensed high-impedance states, when masked are not as expected. This type of failure corresponds most closely to an analog failure and accordingly, it alone will set the NOGO flag.

11.0 SIGNAL ORIENTED STATEMENTS

11.1.4.5 Rules: For Multiple Measured Characteristics

1. The following apply to the use of multiple measured characteristics in a sensor statement:
 - a. Since all of the <measured characteristic mnemonic>s relate to a single signal, the above rules for signal routing, critical action, timing and UUT failure apply.
 - b. All of the measured characteristic values are to be obtained from a data set captured by a single measurement occurring subsequently to a single ARM or ENABLE EVENT operation.
 - c. All of the measured characteristic values are to be transferred by a single FETCH operation.
 - d. In a sequence of single action verb statements using multiple measured characteristics, an identical set of <measured characteristic mnemonic>s must be used in each statement.
 - e. When multiple measured characteristics are used, the label MEASUREMENT is unaffected in that it retains its previous value and each <measured characteristic mnemonic> must have an associated INTO subfield.

11.2 Single Action Verb Statements

11.2.1 Single Action Verb Statements: General Description

11.2.1.1 Function

Single action verbs are considered basic to the language since each one describes a testing action which cannot be further subdivided with respect to the UUT.

Figure 11-5 illustrates the single action functions (commands) associated with the ATLAS single action verbs and their effects on the ATLAS virtual resources and associated signal and event paths. All commands, associated parameters and the handling of measured values are specified in the ATLAS single action statements.

In Figure 11-5, a virtual test resource for SOURCE or LOAD has two sets of ports; Signal Output Ports through which the signals to the UUT are transmitted and Timing Input Ports through which timing information associated with <subfields> used to generate initial coincidence synchronization, frequency locking or gated signals is passed to the UUT.

The SENSOR virtual test resource has three sets of ports; Signal Input Ports through which the signals from the UUT are transmitted in all cases but time measurement. Timing Signal Input Ports are used instead of the Signal Input Ports for timing measurement to communicate information on the events to be timed into the SENSOR. Timing Input Ports are associated with GATE field to control the timing of the measurement.

The system in Figure 11-5 is simplified and may be altered in several ways as follows:

1. The TIMING inputs to the SOURCE, SENSOR and LOAD, which control when the signal is generated or measured, may not be present.
2. Each EVENT MONITOR may comprise an EVENT MONITOR with two or more EVENT MONITORS feeding it through a routing system. In this case the single EVENT MONITOR is closer to the SOURCE, SENSOR or LOAD on the diagram and the multiple EVENT MONITORS are closer to the UUT. In this case the set of verbs acting on all the EVENT MONITORS is the same. The set of verbs acting on the routing is the same as for other routing in the same path.
3. Each of the EVENT MONITORS described in 2 above can be similarly constructed.

11.0 SIGNAL ORIENTED STATEMENTS

4. Each of the EVENT MONITORs with only ROUTING between it and the UUT includes a SENSOR. It may have this ROUTING replaced by all of the items which may be between a SENSOR and a UUT. This is because it is conceptually a SENSOR but with the restriction that it is controlled only by those verbs acting on that part of the path in which it occurs and which are also applicable to the corresponding element in a full sensor.

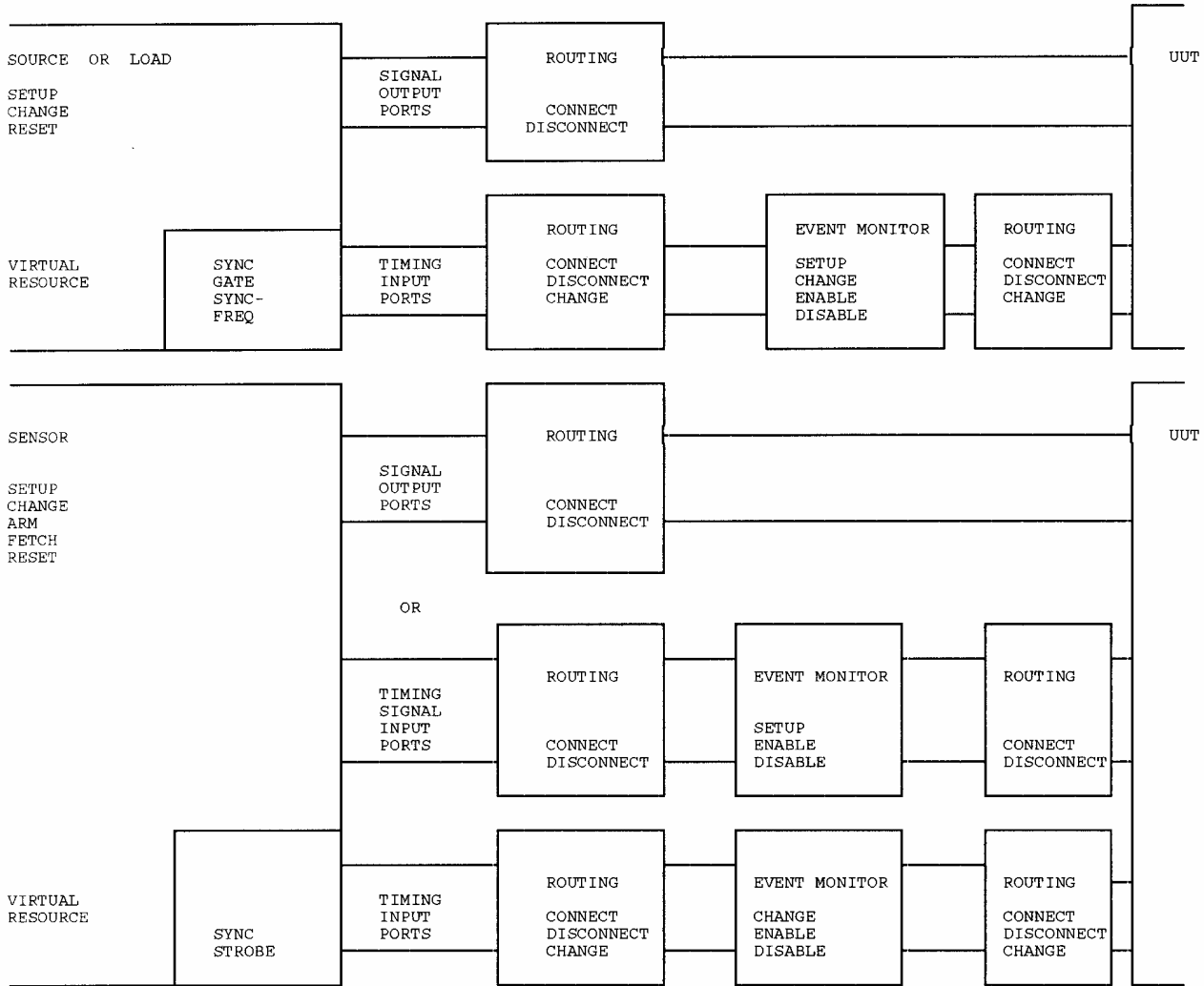


Figure 11 -5 Single Action Analog Verbs Illustration

11.0 SIGNAL ORIENTED STATEMENTS

The single action functions are defined in Sections 11.2.2 thru 11.2.10, however, for the purpose of understanding Figure 11-5 the following paragraphs give general descriptions.

SETUP

This command provides control and data to the specified virtual test resource type (SOURCE, SENSOR, or LOAD) and any associated EVENT MONITOR virtual resource types. SETUP prepares a SOURCE, SENSOR or LOAD virtual resource and any associated EVENT MONITOR virtual resources for a subsequent test action. They remain committed until released by RESET.

The quantity of data is a function of the virtual test resource type and signal. For example; for the DC SIGNAL noun, VOLTAGE may be the only control parameter for a SOURCE; for the AC SIGNAL noun, VOLTAGE and FREQ may be required; for the PULSED DC noun, VOLTAGE, PERIOD, PULSE-WIDTH, etc. may be required.

CONNECT/DISCONNECT

These commands control the operation of some signal routing function which defines a signal path from the virtual test resource to the specified port(s) on the UUT or between virtual test resources. For electrical signals this function may be a relay switching unit and for pneumatic signals that are defined by an <extend atlas statement> it may be a manifold with automatic control of the ports on the UUT connection side. Note that a SOURCE signal reaches the UUT or a UUT signal reaches an SENSOR, LOAD or EVENT MONITOR as soon as CONNECT is executed. Also this active signal path requires that the appropriate connections between the test equipment interface and the UUT must exist.

ARM

This command is used only with SENSOR type of virtual test resource and causes the SENSOR to measure the signal value that exists at its input. If any events are referenced by the statement, in a GATE or FROM/TO field, the measurement occurs after the events have been enabled by an <enable event statement> and the events have occurred.

FETCH

This command is used only with SENSOR types of virtual test resource in association with INITIATE or ARM. It causes a value which has been measured and stored in storage which is local to the SENSOR, to be transferred to storage indicated by the statement.

If multiple measured characteristics are used, the FETCH command will cause the values of all of these characteristics to be transferred to storage indicated by the INTO subfields in the statement.

ENABLE EVENT

This command is used to reset any storage in any EVENT MONITOR virtual test resources referenced directly by the statement and to cause the EVENT MONITORs to generate an event whenever the event definition is fulfilled. An ENABLE EVENT command which references an event that is already enabled, acts as a RESET of any storage in that EVENT MONITOR.

DISABLE EVENT

This command is used to inhibit an EVENT MONITOR virtual test resource from generating further events until it is enabled by an ENABLE command.

11.0 SIGNAL ORIENTED STATEMENTS**CHANGE**

This command is used to alter the SOURCE or LOAD value or the SENSOR range of a virtual test resource and some associated EVENT MONITOR virtual test resources upon which a SETUP and CONNECT command have been executed previously and a DISCONNECT command has not been executed subsequently.

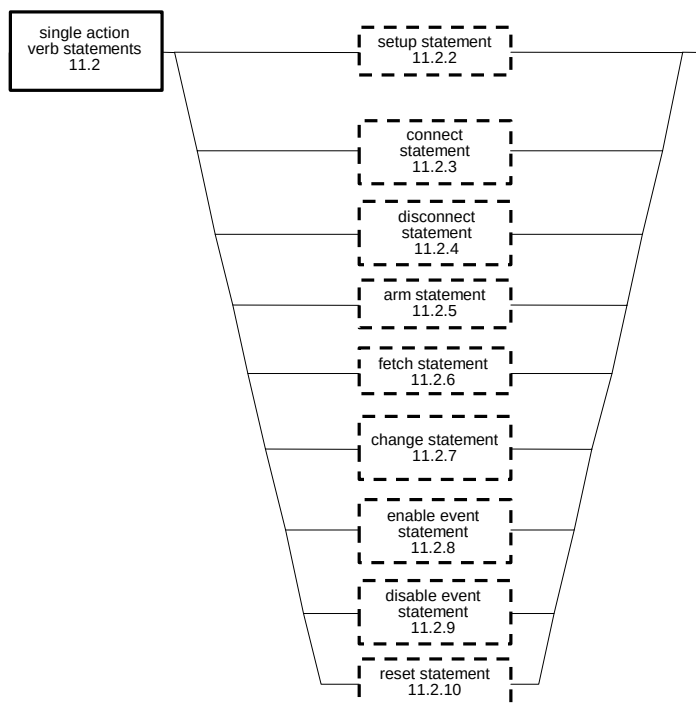
RESET

This command causes the virtual test resource to be returned to its quiescent state and makes the virtual resource available to a new test action by the execution of a subsequent SETUP statement.

The above commands exist for all ATLAS virtual test resources used in an ATLAS Test Specification. The ATLAS Test Specification may be implemented on a specific test system by assigning specific real test resources to the virtual test resources of the ATLAS Test Specification. Specific real test resources may not have the signal routing functions and as a result, will give a null or modified response to the corresponding commands.

11.2.1.2 Formal Syntax

FSD / 1780

11.2.1.3 Syntax Diagram**11.2.1.4 Rules**

1. A signal oriented statement may refer to one or more <event>s. In this case the signal described in the statement employs one or more virtual test resources for monitoring events as well as the virtual test resource for the SOURCE, SENSOR, or LOAD.
2. The execution of a single action verb statement affects all the virtual test resources which monitor an <event> as defined by an <identify event based event statement> and called by the single action verb statement.
3. When a number of principal <event>s are defined by more than one <identify event based event statement>, each of which has an OCCURRENCES OF subfield containing a secondary <event> identified previously by a unique name, all <event>s must be made ready by a single <enable event statement>.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.1.5 Notes and Examples**

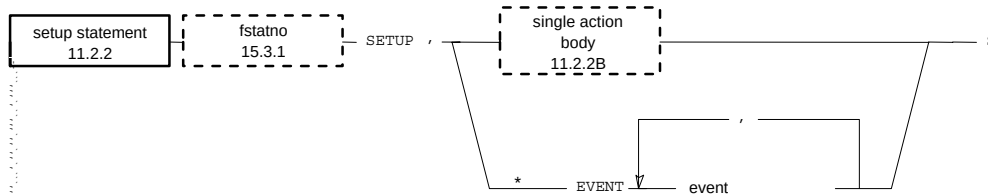
It will normally not be necessary to write a UUT test procedure in the level of detail afforded by single action verbs. The verbs were incorporated in the language primarily as a means of properly defining the multiple action verbs. These multiple action verbs will be used for most test statements because they are easier to write than repeating a series of single action verbs each time a test is written. However, it is recognized that there may be special cases where the multiple action verb is not appropriate and the single action sequence can then be written out.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.2 <setup statement>****11.2.2.1 Function**

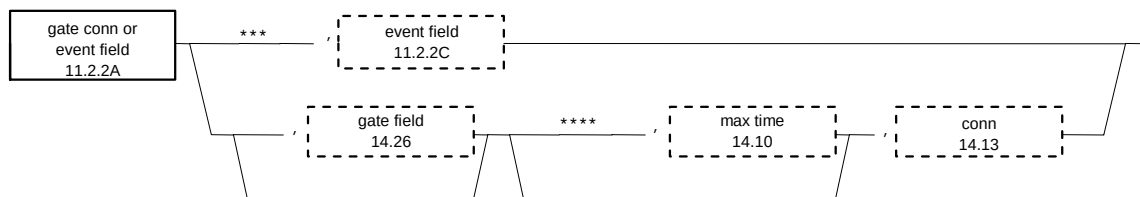
To allocate a virtual test resource and any associated EVENT MONITORS and to control them so as to generate or receive a defined signal.

11.2.2.2 Formal Syntax

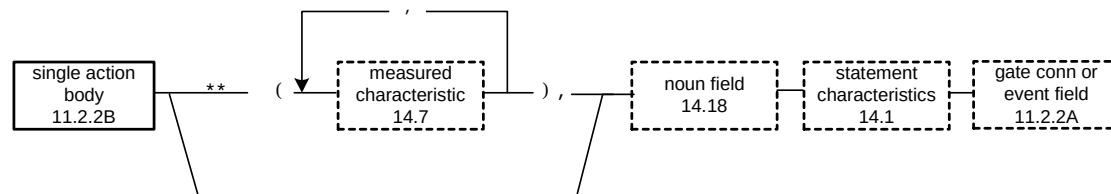
FSD / 1794

11.2.2.3 Syntax Diagram

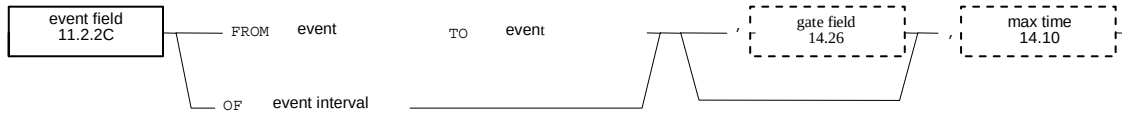
Note : * This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO, or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.



Note: *** This path is only for time measurement statements.
 **** This path is only for measurement statements.



Note: ** this path is for measurements only.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.2.4 Rules****1. Virtual Test Resources Not Controlled**

A <setup statement> does not control virtual test resources for signal routing.

2. Effect on SOURCE and LOAD Virtual Test Resources

A signal provided by a virtual test resource is that described by the noun and static statement characteristic fields modified in accordance with any dynamic statement characteristic field in the statement. The effect of the dynamic characteristic field is as follows:

The transition time from stated or default quiescent conditions will be as described in the TRANS-TIME field.

The effect on synchronization is as follows:

a. Initial Coincidence Type of Synchronization

For initial coincidence type of synchronization the signal will change from a quiescent condition to the stated non-quiescent condition when the synchronization condition is both enabled and fulfilled for the first time after the execution of the SETUP statement.

b. Gate Field

The signal will be at the stated non-quiescent condition in the time slot defined by the <gate field>. The signal will be at a stated or default quiescent condition at all other times.

c. Frequency Reference Type of Synchronization

This replaces the frequency field which would otherwise be required but does not affect the time of generation of this signal.

3. Effect on SENSOR and EVENT MONITOR Virtual Test Resources

A SENSOR is set to a range which is consistent with the statement characteristics associated with the measured characteristics. Ranging information must be provided for each <measured characteristic>. Any other controllable characteristics of the SENSOR (e.g. bandwidth, etc.) are also set. A <setup statement> cannot be executed for a SENSOR between the execution of an ARM action and the execution of the corresponding FETCH action.

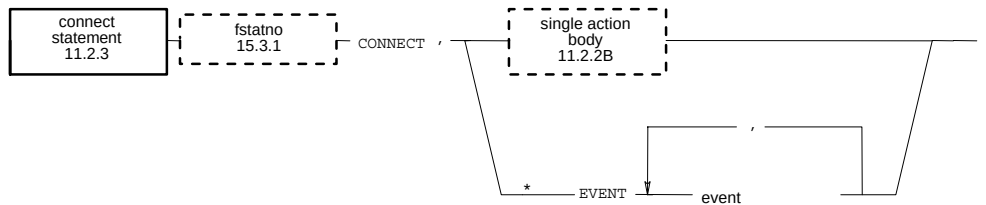
4. If SETUP is used with nouns which have no controllable modifiers, as defined in Chapter 16, it has a null operation.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.3 <connect statement>****11.2.3.1 Function**

To fasten or join together the interface points (pin connectors) of the UUT and the interface points of the source, sensor, or load device and any associated EVENT MONITORS and to initiate or gate the signal between the virtual test resource and the UUT.

11.2.3.2 Formal Syntax

FSD / 1848

11.2.3.3 Syntax Diagram

Note: * This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.

11.2.3.4 Rules**1. Virtual Test Resources Not Controlled**

A <connect statement> has no effect on EVENT MONITOR, SOURCE, SENSOR or LOAD virtual test resources.

2. Effect on UUT to SOURCE, SENSOR, LOAD or EVENT MONITOR connections.

The signal routing required to carry the signal between the virtual test resource and the UUT is completed.

3. Effect on SIGNAL to EVENT MONITOR and EVENT MONITOR to EVENT MONITOR connections.

If an EVENT MONITOR is employed, all connections between the EVENT MONITOR and the signal virtual test resources are established.

4. In SENSOR <connect statement>s an identical set of <measured characteristic>s must be used to that used in the related <setup statement>.

11.0 SIGNAL ORIENTED STATEMENTS

11.2.4 <disconnect statement>

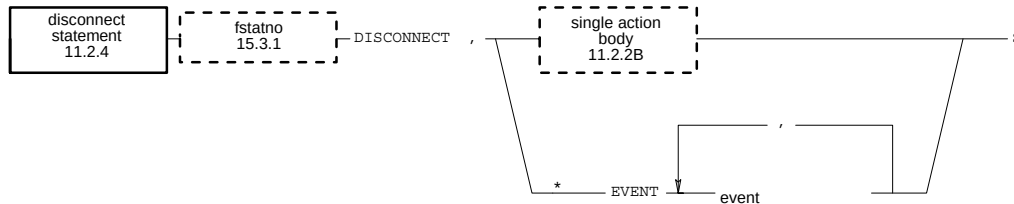
11.2.4.1 Function

To disconnect the interface points (pin connectors) of the UUT and the interface points of the source, sensor, or load device and any associated EVENT MONITORS. This the opposite of a <connect statement>.

11.2.4.2 Formal Syntax

FSD / 1862

11.2.4.3 Syntax Diagram



Note: * This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.

11.2.4.4 Rules

1. Virtual Test Resources Not Controlled

A <disconnect statement> does not control SOURCE, SENSOR, LOAD or EVENT MONITOR virtual test resources.

2. Effect on Resources for Signal Routing

When a DISCONNECT statement is executed all the connections between associated signal and EVENT MONITOR virtual test resources and between these resources and the UUT are rendered ineffectual by setting all the controlled switching within a test system associated with a signal to a condition which inhibits the transmission of a signal.

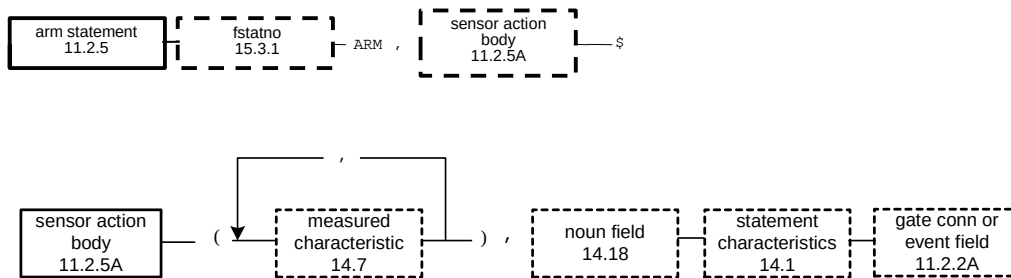
3. If more than one <measured characteristic> is used in related single action verb statements, an identical set must be used in the <disconnect statement>.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.5 <arm statement>****11.2.5.1 Function**

To induce a measurement cycle by a SENSOR during which the result(s) of the measurement will be determined before transfer by the associated <fetch statement>. The timing and duration of a measurement is described in the statement characteristics field.

11.2.5.2 Formal Syntax

FSD / 1876

11.2.5.3 Syntax Diagram**11.2.5.4 Rules****1. Virtual Test Resources Not Controlled**

An <arm statement> does not control SOURCE, LOAD, CONNECTION or EVENT MONITOR virtual test resources. A SOURCE or LOAD virtual resource cannot be used in an ARM statement.

2. Effect on SENSOR Virtual Test Resources

An <arm statement> indicates that a SENSOR virtual test resource is required to execute a measurement. A measurement will be taken as soon as the associated timing considerations, which may be dependent on subsequent statements, are satisfied. If more than one <measured characteristic> is used, all of the measured values are to be obtained from a data set captured by a single measurement initiated by a single critical action (ARM or ENABLE EVENT) related to the noun field.

A succeeding statement is executed as soon as a SENSOR virtual test resource is ready to take a measurement. This may be prior to the measurement commencing (in fact it may be a necessary preliminary to the measurement commencing). The measurement is then required to be executed asynchronously with the succeeding statements. The SENSOR virtual test resource is required to store the measurement in local internal storage until it is extracted by a FETCH statement.

11.2.5.5 Notes and Examples

There are a small number of measurements such as MEASURE, (SAMPLE), WAVEFORM where a single measurement returns many numerical values. In these cases, despite the limitation of the definition of ARM, the many numerical values are obtained and are held for subsequent transfer by a <fetch statement> to an <array> for storage.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.6 <fetch statement>****11.2.6.1 Function**

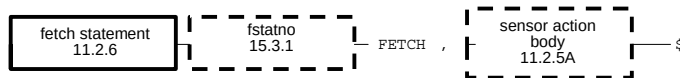
To wait for the completion of a measurement cycle induced by ARM or INITIATE and to transfer the measured value(s):

- a. For a single measurement value; into the 'MEASUREMENT' store and any named store in the optional INTO branch.
- b. For multiple measured values; into the locations within a named list stated in the INTO branch or elsewhere in the statement.

To disable any EVENT MONITORs associated with a measurement.

11.2.6.2 Formal Syntax

FSD / 1888

11.2.6.3 Syntax Diagram**11.2.6.4 Rules**

1. If the measurement enabled by the corresponding ARM or INITIATE statement is incomplete, the FETCH statement waits for it to be completed prior to storing the measured value(s).
2. In the measurement of a TIME INTERVAL, where <events> or an <event interval> is referenced, if a MAX-TIME field is specified, the measurement will be terminated and the MAX-TIME flag set if it does not complete within the time specified by the MAX-TIME field.
3. FETCH has no direct effect on any virtual test resource, other than for a SENSOR virtual test resource when it extracts data without affecting the SENSOR status and EVENT MONITOR virtual test resources which it disables.
4. A <fetch statement> cannot be executed unless an associated explicit <arm statement> or <initiate statement> has been executed. In the case of an <arm statement>, a <fetch statement> cannot be executed unless an associated <enable event statement> has been executed.

11.2.6.5 Notes and Examples

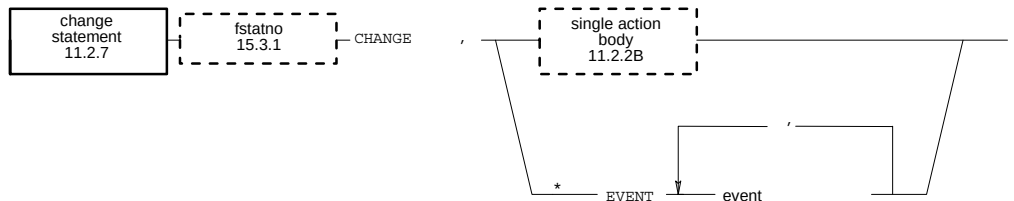
In those cases where a single INITIATE statement returns many numerical values (see section 11.2.5.5), FETCH does not affect 'MEASUREMENT' and data is transferred to the designated <array>.

11.0 SIGNAL ORIENTED STATEMENTS11.2.7 <change statement>11.2.7.1 Function

To alter some of the characteristics of a test requirement which are active in the testing of a UUT.

11.2.7.2 Formal Syntax

FSD / 1894

11.2.7.3 Syntax Diagram

Note: * This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.

11.2.7.4 Rules

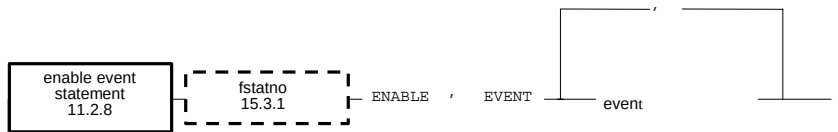
1. A <change statement> can only be used to alter some of the characteristics of ATLAS statements which are active in the testing of a UUT.
2. A <change statement> cannot be used to alter in a statement being executed the content of a:
 - <noun field>
 - <gate field>
 - <sync subfield> when used for synchronization reference
 - <conn> field.
3. A <change statement> can be used to alter in the statement being executed the content of a:
 - <measured characteristic> field
 - <statement characteristics> field except <sync subfield> when used for synchronization reference
 - FROM <event> TO <event> field
 - OF <event interval> field
 - <max time> field.
4. A <change statement> maintains the already established connections between the UUT and the SENSOR, SOURCE or LOAD virtual test resources.
5. A <change statement> can modify connections to event monitors.
6. A <change statement> cannot be used to alter the characteristics of a SENSOR between the executions of ARM and FETCH actions.
7. A <change statement> neither enables nor disables any event monitor.

11.0 SIGNAL ORIENTED STATEMENTS**11.2.8 <enable event statement>****11.2.8.1 Function**

To make ready an EVENT MONITOR for the detection and indication that its identified <event> has occurred.

11.2.8.2 Formal Syntax

FSD / 1908

11.2.8.3 Syntax Diagram**11.2.8.4 Rules**

1. An <enable event statement> does not control SOURCE, SENSOR, LOAD signal routing virtual test resources.
2. If an EVENT definition includes an occurrence subfield then the occurrence count is reset to zero.
3. When an <event> definition includes references to other <event>s, all these related <event>s must be made ready explicitly by the same ENABLE statement.

The exception to this is when an <identify event based event statement> includes an OR field in which case it is only necessary to enable the particular <event>s associated with the options required by the test procedure being executed although the additional options can be enabled if required. Any option not enabled by the statement must be in a disabled state.

4. If an EVENT definition includes a MANUAL INTERVENTION; this must be explicitly made ready by an <enable manual intervention statement>.
5. If the <event> referenced is already enabled, that event is reset by the <enable event statement>.

Re-enabling an Event Based Event will cause any associated counts of other events to be reset as defined in Section 6.15.4, Rule 2.

Re-enabling of Signal Based or Time Based Events will have no effect.

Re-enabling an <event> that is referenced by an active Time Interval will have an unpredictable effect.

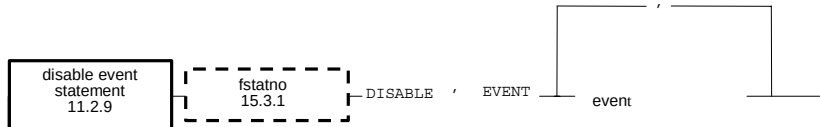
6. An <event> monitor can only detect an already enabled <event>.

11.0 SIGNAL ORIENTED STATEMENTS11.2.9 <disable event statement>11.2.9.1 Function

To prevent an EVENT MONITOR from detecting and indicating that its identified <event> has occurred.

11.2.9.2 Formal Syntax

FSD / 1914

11.2.9.3 Syntax Diagram11.2.9.4 Rules

1. A <disable event statement> does not control SOURCE, SENSOR, LOAD signal routing virtual test resources.
2. When an <event> definition includes references to other <event>s, all the related <event>s must be disabled explicitly by the same DISABLE statement.

The exception to this is when an <identify event based event statement> includes an OR field in which case it is only compulsory to disable the event based <event> if all the alternatives are disabled.

11.0 SIGNAL ORIENTED STATEMENTS

11.2.10 <reset statement>

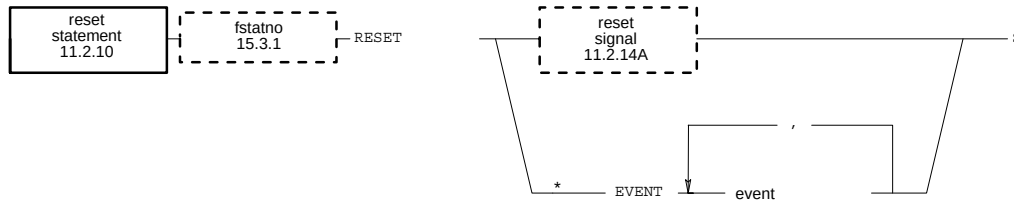
11.2.10.1 Function

To cause a signal to return to its unallocated quiescent state and to make its virtual test resources available for re-use in a SETUP or multiple action statement.

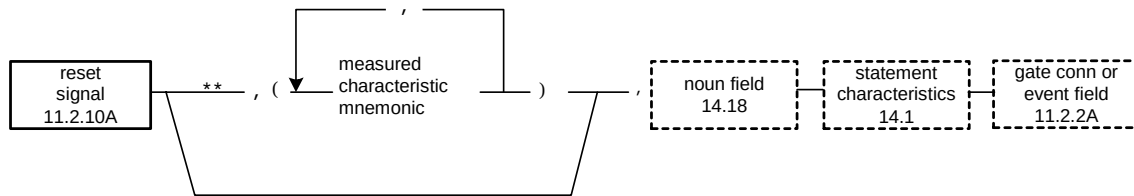
11.2.10.2 Formal Syntax

FSD / 1920

11.2.10.3 Syntax Diagram



Note: * This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.



Note: ** This path is for measurements only.

11.2.10.4 Rules

1. A <reset statement> releases all the virtual resources associated with a signal.
2. SOURCE, SENSOR and LOAD virtual resources are set to a defined or default quiescent input level.
3. A <reset statement> cannot be executed unless a corresponding DISCONNECT has been executed.
4. A <reset statement> does not control EVENT MONITOR or connection virtual resources; but it does release them for use by a subsequent setup statement.
5. If more than one <measured characteristic> is used in related single action verb statements, an identical set must be used in the <reset statement>.

11.0 SIGNAL ORIENTED STATEMENTS**11.3 <apply statement>****11.3.1 Function**

To accomplish the following actions for a SOURCE or LOAD virtual test resource type in the order listed below:

SETUP

CONNECT

ENABLE, EVENT for EVENT MONITORS associated with the SOURCE or LOAD.

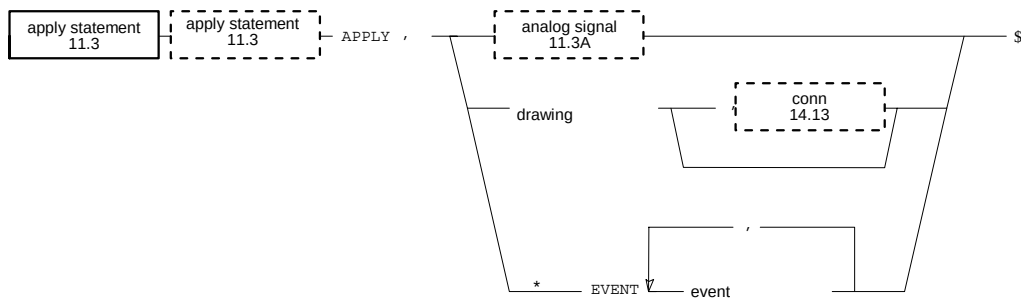
To accomplish the following actions for an EVENT MONITOR virtual test resource type in the order listed:

SETUP

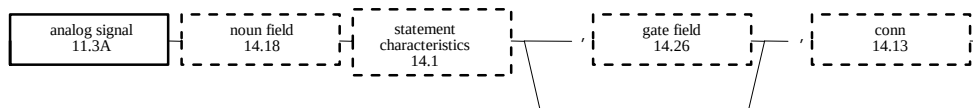
CONNECT

11.3.2 Formal Syntax

FSD / 1945

11.3.3 Syntax Diagram

Note: * This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.

**11.3.4 Rules**

1. All connections to be formed when the special circuit is installed must be identified in the <define drawing statement>, not in the <apply statement>. Terminals on the special circuit which are to be connected individually after circuit installation may be referenced in the <apply statement>.
2. "Pin descriptors" in the connection field are the appropriate terminal identifiers which follow the word TERMINAL in the <define drawing statement> and each pin descriptor is preceded by an asterisk.

11.0 SIGNAL ORIENTED STATEMENTS

3. In general, if the special circuit network is not installed prior to test execution (e.g. adapter installation), then a message must be provided to the operator to perform installation during the test. A <drawing> installation is not accomplished through use of the <apply statement>; only the specific connections identified in the <apply statement> are established.

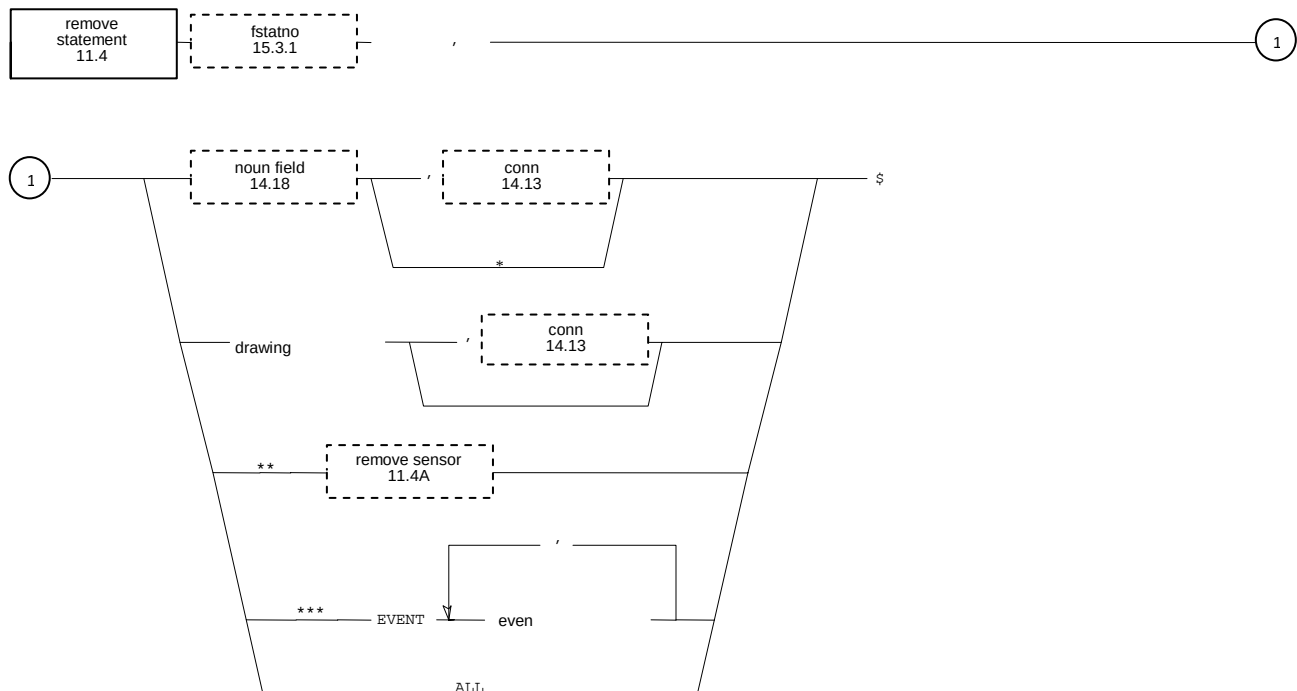
11.4 <remove statement>**11.4.1 Function**

To accomplish the following actions for a SOURCE, SENSOR, LOAD or EVENT MONITOR virtual test resource in the sequence listed below:

DISABLE, EVENT
DISCONNECT
RESET

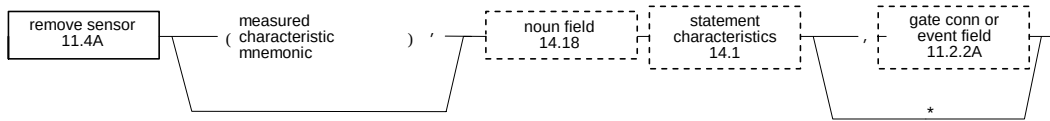
11.4.2 Formal Syntax

FSD / 1966

11.4.3 Syntax Diagram

Note:

- * This path is for time measurement statements only.
- ** This path is for sensor statements only.
- *** This path is only required for signal based events which are not directly referenced in SYNC, GATE, FROM, TO, or BY fields. It configures the EVENT MONITOR defined in the REQUIRE to detect the <event> defined in the IDENTIFY statement but does not ENABLE it.

11.0 SIGNAL ORIENTED STATEMENTS

Note: * This path is for time measurement statements only.

11.4.4 Rules

1. A REMOVE, ALL statement may be used without referencing any signal paths in order to remove every signal that is connected to the UUT.
2. A REMOVE statement which references one or more signal paths causes only those signals that are referenced to be removed.
3. If a measurement has been initiated by an ARM or INITIATE statement then, execution of a <remove statement> affecting the associated virtual test resource causes the measurement to be terminated with undefined data in any storage referenced in an INTO subfield.
4. Terminals on a DEFINED <drawing> which are to be disconnected individually prior to <drawing> removal must be referenced in a <remove statement>. Only those terminals which have been connected via an <apply statement> may be disconnected in this manner. Those which were established in the connection field of the <define drawing statement> may only be disconnected through removal of the <drawing>.
5. Pin descriptors in the connection field are the appropriate terminal identifiers which follow the word TERMINAL in the <define drawing statement> and each pin descriptor is preceded by an asterisk.
6. If the special circuit network is to be removed prior to test completion, then a message must be provided to the operator to perform this task during the test. A <drawing> removal is not accomplished through use of the <remove statement>; only the specific connections identified in the <remove statement> connection field are disconnected via the <remove statement>.

11.4.5 Notes and Examples

654321	REMOVE, ALL \$
23	REMOVE, DC SIGNAL, CNX HI J1-5 LO J1-6 \$
24	REMOVE, PULSED DC, CNX HI J8-1 LO J8-0 \$

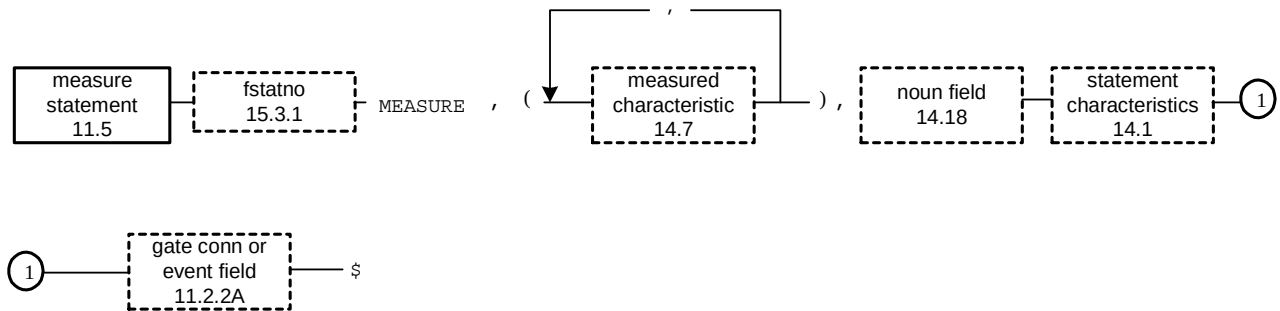
11.0 SIGNAL ORIENTED STATEMENTS11.5 <measure statement>11.5.1 Function

To accomplish the following actions for a SENSOR virtual test resource in the sequence listed:

SETUP
CONNECT
ARM
ENABLE, EVENT
FETCH
DISABLE EVENT
DISCONNECT
RESET

11.5.2 Formal Syntax

FSD / 1999

11.5.3 Syntax Diagram11.5.4 Rules

1. If more than one <measured characteristic> is used:
 - a. Each must have an associated INTO option naming the location into which the particular measured value(s) is to be stored. The value of MEASUREMENT will not be affected.
 - b. Ranging information must be provided for each <measured characteristic>.
 - c. All of the <measured characteristic>s are to be obtained from a data set captured by a single measurement initiated by a single critical action (ARM or ENABLE EVENT) related to the noun field.

11.0 SIGNAL ORIENTED STATEMENTS**11.6 <monitor statement>****11.6.1 Function**

To accomplish the following sensor functions in the sequence listed and continuously repeat the functions of READ and OUTPUT 'MEASUREMENT' until MANUAL INTERVENTION directs the program to proceed to the next step.

```

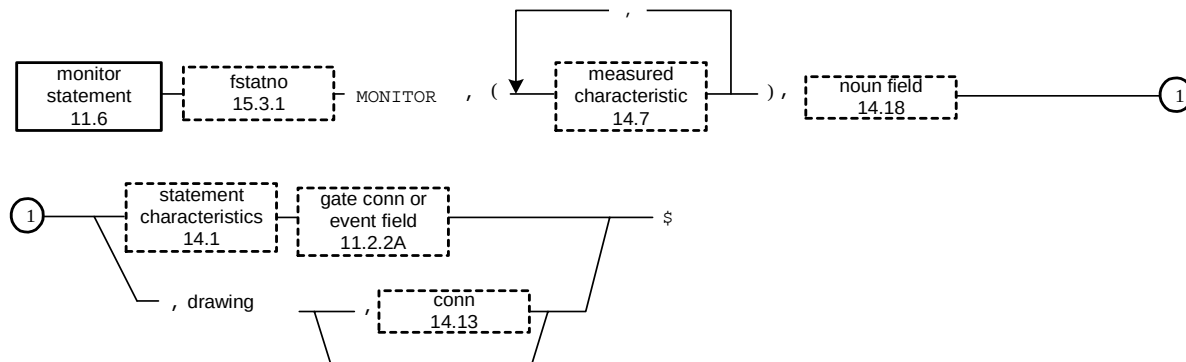
SETUP
CONNECT
ENABLE, EVENT
ENABLE, MANUAL INTERVENTION
WHILE, MANUAL INTERVENTION EQ FALSE, THEN
READ
OUTPUT, 'MEASUREMENT'
END, WHILE
DISABLE, MANUAL INTERVENTION
DISABLE, EVENT
DISCONNECT
RESET

```

Where a complex waveform defined by a <define drawing statement> is used, the single action functions will be performed manually, with the possible exception of ENABLE and DISABLE MANUAL INTERVENTION, CONNECT and DISCONNECT.

11.6.2 Formal Syntax

FSD / 2007

11.6.3 Syntax Diagram**11.6.4 Rules**

1. When used with a <drawing>:
 - a. The noun in the <noun field> must be WAVEFORM.
 - b. The measurement function is performed manually and no change occurs in the value of 'MEASUREMENT'.
 - c. All manual actions to be performed in preparation for monitoring the waveform must be described by a preceding <output statement> to the operator.
 - d. There may be an <input statement> after the <monitor statement> to allow the operator to enter the result of the monitor function.

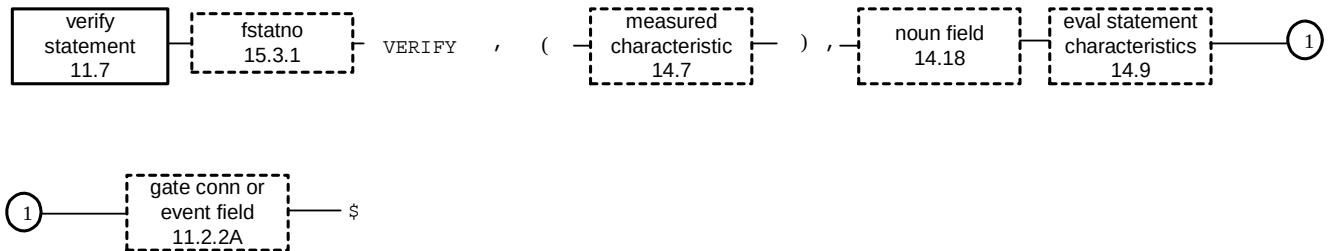
11.0 SIGNAL ORIENTED STATEMENTS**11.7 <verify statement>****11.7.1 Function**

To accomplish the following sensor functions in the sequence listed and to establish the state of the HI, LO, GO, and NOGO conditions, in accordance with the Table 14-1, Evaluation Field Relationships, in Rule 1 of Section 14.8.4, based on the relationship between the measured value and the specified limit(s). (see <compare statement> Section 8.3).

SETUP
CONNECT
ARM
ENABLE, EVENT
FETCH
COMPARE
DISABLE, EVENT
DISCONNECT
RESET

11.7.2 Formal Syntax

FSD / 2019

11.7.3 Syntax Diagram

11.0 SIGNAL ORIENTED STATEMENTS**11.8 <read statement>****11.8.1 Function**

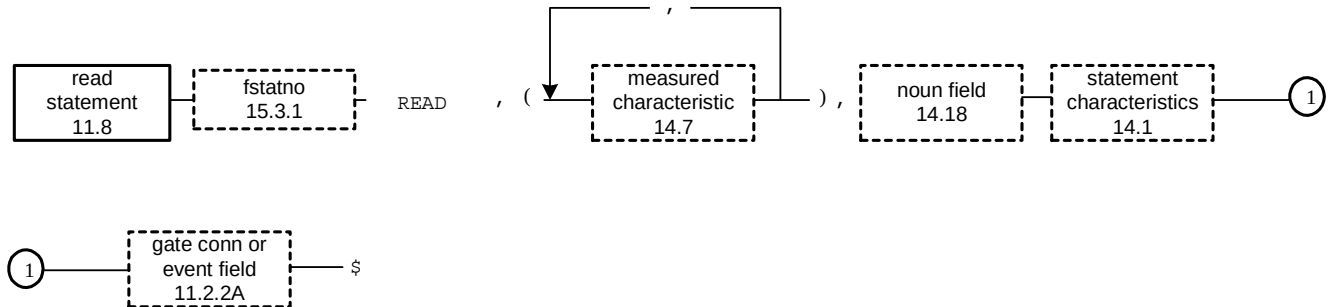
To establish the present value of a sensor function that exists at the instant of measuring and retain that value in the label 'MEASUREMENT' and any INTO location specified until a subsequent sensor function establishes a new value for 'MEASUREMENT' or into the specified list for multiple measured values.

READ accomplishes the following functions in the sequence listed:

INITIATE
FETCH

11.8.2 Formal Syntax

FSD / 2027

11.8.3 Syntax Diagram**11.8.4 Rules**

1. If more than one <measured characteristic> is used:
 - a. Each must have an associated INTO option naming the location into which the particular measured value is to be stored. The value of 'MEASUREMENT' will not be affected.
 - b. The real test resource must determine all of the measured values from the data set captured when the measurement is taken.

11.0 SIGNAL ORIENTED STATEMENTS**11.9 <initiate statement>****11.9.1 Function**

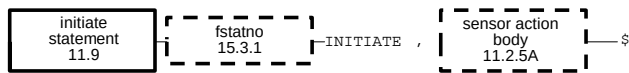
To accomplish the following sensor functions in the sequence listed.

ARM

ENABLE, EVENT(only those <event>s associated with the ARM)

11.9.2 Formal Syntax

FSD / 2035

11.9.3 Syntax Diagram**11.9.4 Rules**

1. Each <initiate statement> must have an associated and executed <fetch statement> or <remove statement>.
2. The following statement in the text is executed as soon as the functions of ARM and any associated ENABLE, EVENT functions have been actioned.
3. If more than one <measured characteristic> is used in related single action verb statements, an identical set must be used in the <initiate statement>.

11.0 SIGNAL ORIENTED STATEMENTS**11.10 Digital Verbs**

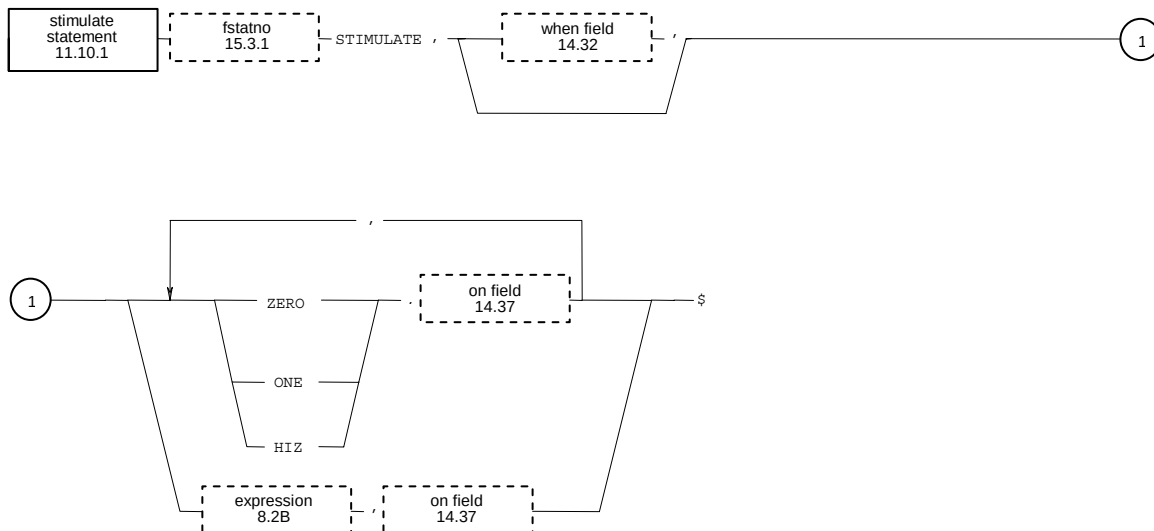
The digital statements <stimulate statement>, <sense statement> and <prove statement> should not be used in ATLAS Test Requirements which provide information for the testing of databus systems. The constructs in Chapter 13, Databus Statements, are recommended for the testing of databuses.

11.10.1 <stimulate statement>**11.10.1.1 Function**

To apply logic signals of an ENABLEd <configuration> to the UUT. Logic states can be manipulated using either word or bit level constructs.

11.10.1.2 Formal Syntax

FSD / 2040

11.10.1.3 Syntax Diagram**11.10.1.4 Rules**

1. The critical action of a <stimulate statement> for a serial signal starts at the first bit period.
2. ZERO, ONE and HIZ correspond to LEVEL-LOGIC-ZERO, LEVEL-LOGIC-ONE and LEVEL-LOGIC-HIZ respectively of the ENABLEd <configuration>. When the test resources of the ENABLEd <configuration> have been set to the high impedance condition, they will remain in this condition until changed by a subsequent STIMULATE, ONE/ZERO/ <expression> statement.
3. Parallel logic signals can be operated on by word-level and bit-level constructions of STIMULATE. Serial logic signals can only be operated on by word-level constructions and only one SERIAL signal can be included in each <stimulate statement>.
4. Although a <stimulate statement> may contain any number of ZERO, ONE and HIZ fields, the updating of logic signals will still occur as a single critical action. The updating will represent the cumulative states resulting from the sequence of <stimulate statement>s.

11.0 SIGNAL ORIENTED STATEMENTS

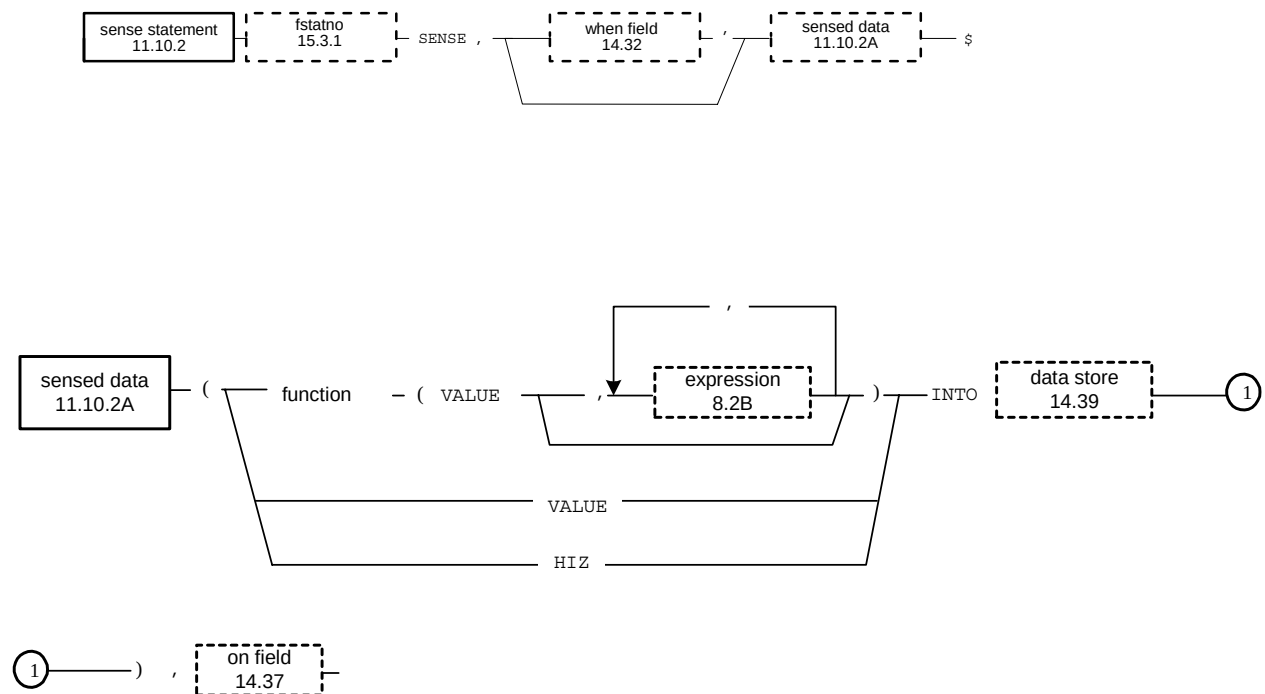
5. The current dynamic length of <expression> must evaluate to a STRING OF BIT. <expression> must be equal to the WORD-LENGTH for serial signals and to the number of signals defined by the <on field> for parallel signals.
6. The time of the critical action of a <stimulate statement> can be specified in the <when field>. If the <when field> is omitted the timing of the critical action is uncontrolled unless this is otherwise specified by DO/END DO structures.
7. When a <stimulate statement> references a <digital source> with an ILLEGAL STATE INDICATOR the associated <Boolean variable> will be set TRUE if the defined logic state cannot be driven.
8. If the <stimulate statement> or its referenced <define digital source statement> contain references to <event>s which are not already enabled the events are enabled at the commencement of execution of the <stimulate statement>.

11.0 SIGNAL ORIENTED STATEMENTS11.10.2 <sense statement>11.10.2.1 Function

To cause the sensors of an ACTIVE <configuration> to sense and digitize UUT responses then to fill stores with the digitized data. Logic states (VALUE) and high impedance states (HIZ) can be SENSEd. The critical action is the sensing of the digital data.

11.10.2.2 Formal Syntax

FSD / 2055

11.10.2.3 Syntax Diagram11.10.2.4 Rules

1. When the ENABLEd <configuration> includes SERIAL signals the critical action of a <sense statement> is the time at which the first bit is expected. Each <sense statement> can only include one SERIAL signal.
2. For SENSE, HIZ statements the high impedance state is represented by LEVEL-LOGIC-ONE and all other states are represented by LEVEL-LOGIC-ZERO.
3. When <function> is used the VALUE of the UUT response will be operated on by the DEFINEd <function> before being placed in the INTO store. (See Section 6.8.2; <define function statement>). Other variables, if any, required by the <function> must also be specified.
4. The number of bits in the INTO stores must be DECLAREd to be equal to or greater than either the number of parallel signals defined by the <on field> to be sensed, the WORD-LENGTH of SERIAL signals or the current dynamic length of the <function> thereof as appropriate.

11.0 SIGNAL ORIENTED STATEMENTS

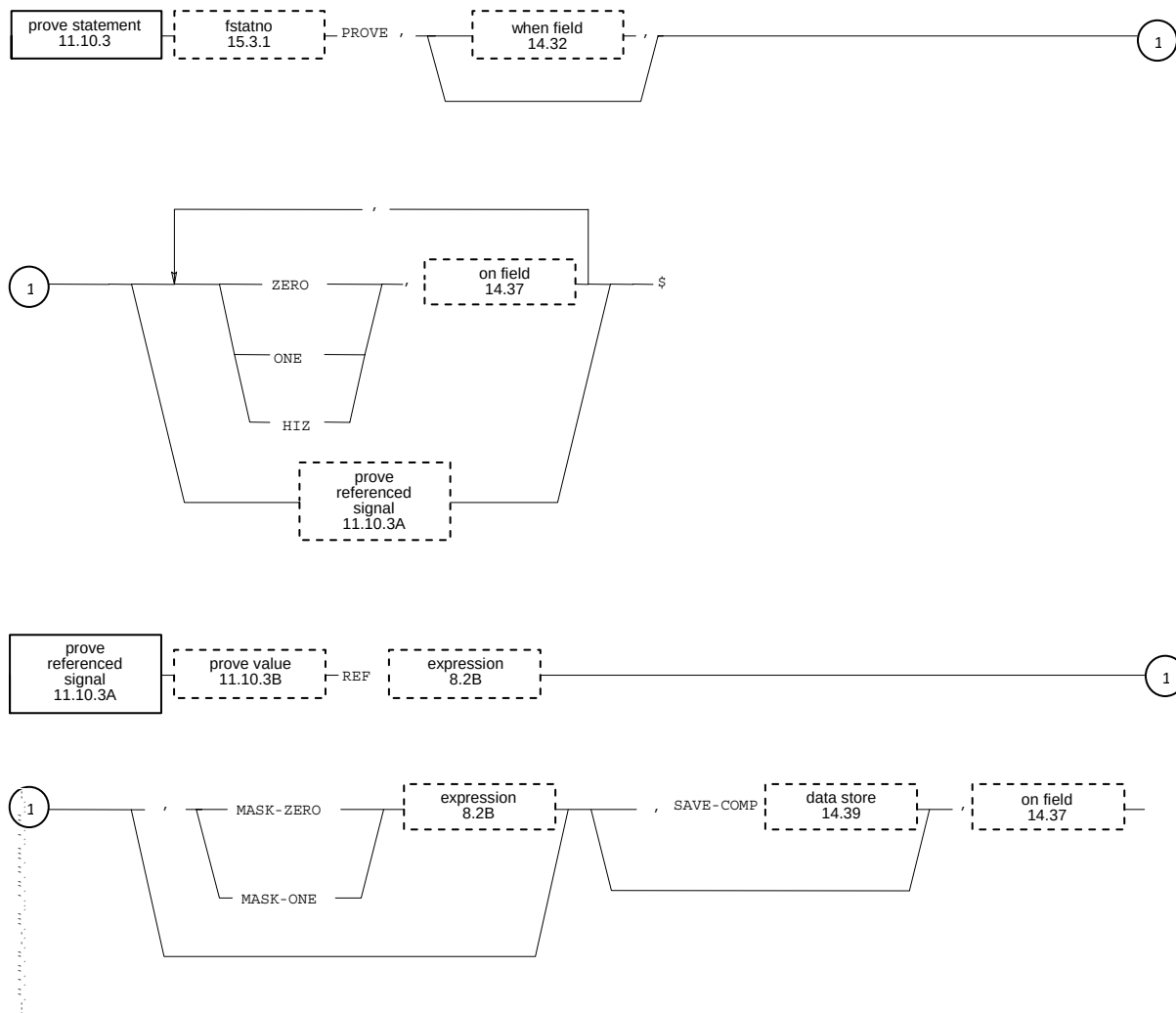
5. The time of the critical action of a <sense statement> can be specified in the <when field>. If the <when field> is omitted the timing of the critical action is uncontrolled unless this is otherwise specified by DO/END DO structures.
6. If the referenced <digital sensor> has an ILLEGAL STATE INDICATOR and the SENSEd physical signal characteristics do not lie in any of the defined logic categories, then the associated <Boolean variable> will be set TRUE.
7. The value of 'MEASUREMENT' is unaffected by a <sense statement>.
8. If the <sense statement> or its referenced <define digital sensor statement> contain references to <event>s which are not already enabled the <event>s are enabled at the commencement of execution of the <sense statement>. Any <event>s which are enabled by a <sense statement> are disabled at the completion of the execution of the <sense statement>.

11.0 SIGNAL ORIENTED STATEMENTS11.10.3 <prove statement>11.10.3.1 Function

To accomplish the action of a <sense statement> followed by an evaluation function to determine whether the UUT responses are as expected. The evaluation function carries out comparisons and digital calculations that it would be impractical to express routinely using an explicit <compare statement> and <calculate statement>. Expected UUT responses can be expressed using either word or bit level constructs.

11.10.3.2 Formal Syntax

FSD / 2078

11.10.3.3 Syntax Diagram

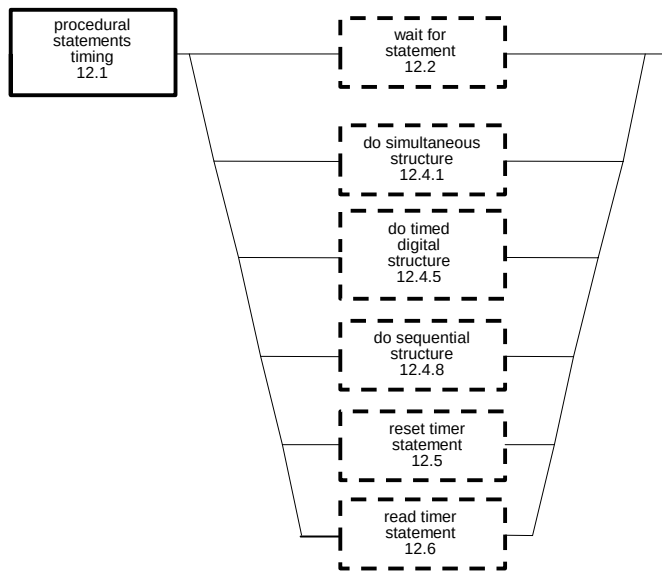
- ## **11.0 SIGNAL ORIENTED STATEMENTS**

12.0 TIMING STATEMENTS**12.0 Timing Statements****12.1 <procedural statements timing>****12.1.1 Function**

To describe the timing of source, sensor and load functions.

12.1.2 Formal Syntax

FSD / 2130

12.1.3 Syntax Diagram**12.1.4 Rules****12.1.4.1 Final Critical Operation and Measurement Initiation**

1. The final critical operation of a SOURCE or LOAD type of statement (APPLY, REMOVE, etc.) is the signal routing or gating operation that applies the signal to the UUT (or in the case of REMOVE, which removes the signal from the UUT).
2. The final critical operation of a SENSOR type statement (MEASURE, VERIFY etc.) is the completion of the measuring function so that the output value from the UUT is available to the test equipment. This occurs during the execution of the FETCH single action of a multi-action verb or between the initiation of the measurement by the <initiate statement> and the completion of the <fetch statement> in a single action verb sequence, that is the final critical operation occurs when the reading is taken and before any action is taken to store the value away in 'MEASUREMENT' or any other store referred to in the measured characteristic field. Because a measurement may take a significant time to complete, it is necessary to define a similar, but related concept, which is Measurement Initiation. That is the instant during the execution of an <initiate statement> when the sensor starts evaluating the incoming signal in order to establish the value of the sensor function.
3. In non-signal oriented statements the whole operation is indivisible and therefore the execution of the whole statement is critical.

12.0 TIMING STATEMENTS

12.1.5 Notes and Examples

Normally execution of each statement takes a significant period of time. For precise timing purposes, the critical action of a statement has to be defined. For example, the critical action of APPLY, DC SIGNAL, may be the closing of the stimulus signal routing. The critical action of REMOVE, IMPEDANCE may be the opening of the load signal routing. The critical action of a <connect statement> or <disconnect statement> may actually be finitely indeterminate. For instance, in the case of a metal spring relay actuation, the time that the command is given to connect or disconnect is known, so the time that the output through the relay may be expected to be "good" is known, but there is the indeterminate possibility that some relays are finitely faster and/or the output is effectively "good" before the expected time. A sensor type critical action may also have a finite degree of indeterminacy depending upon the type of read mechanism. The above information must be taken into account for each test system by the compiler but for particular test applications the programmer may be required to delve into the critical action specifications of his test equipment.

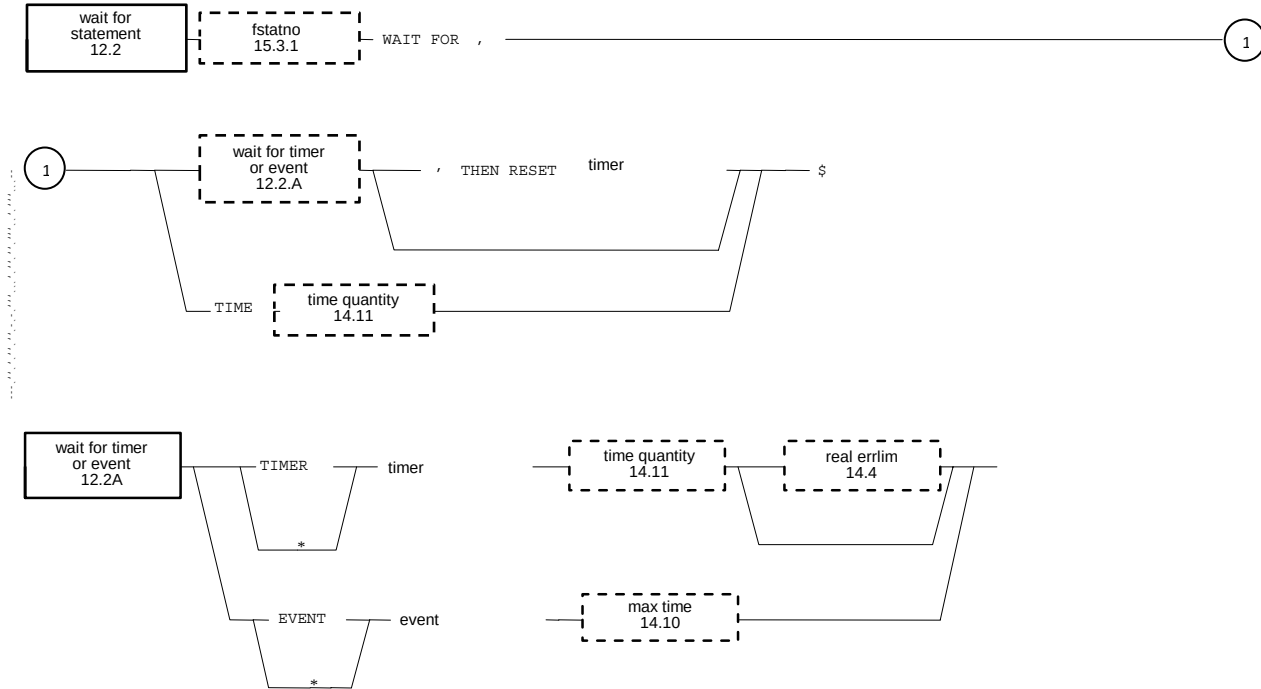
Actions whose exact time of action must be precisely known should make use of event or timer capabilities to guarantee the best control over when the critical actions occur.

12.0 TIMING STATEMENTS**12.2 <wait for statement>****12.2.1 Function**

To postpone execution of a specified statement until a specified time has elapsed or a specified event has occurred.

12.2.2 Formal Syntax

FSD / 2141

12.2.3 Syntax Diagram

Note: * Non-preferred.

12.2.4 Rules

1. **WAIT FOR TIME <time quantity>:**
 - a. This path is used to specify the <time quantity> for the period between the start of the <wait for statement> and the beginning of the execution of the next statement following the <wait for statement>.
2. **WAIT FOR EVENT <event>:**
 - a. This path is used to suspend the test execution at the <wait for statement> until the identified <event> occurs or MAX-TIME expires.
 - b. The THEN RESET subfield may be used only if the <event> is a time based event. The <timer> label of the THEN RESET subfield must be identical to the <timer> label associated with the <time based event>.

12.0 TIMING STATEMENTS

- c. If the THEN RESET field is present, the identified <timer> is simultaneously reset either by the occurrence of the <event> or by the expiration of MAX-TIME.
 - d. If the identified <event> is signal based the <wait for statement> will setup, connect and enable the EVENT MONITOR associated with the identified <event>. On the detection of the specified <event> or on expiration of the specified MAX-TIME, the associated EVENT MONITOR will be disabled, disconnected and reset.
3. WAIT FOR TIMER <timer>:
- a. This path is used to suspend the test execution until the identified <timer> reaches the value specified by <time quantity>.
 - b. If, at the time of the execution of the <wait for statement>, the value of the identified <timer> exceeds the value specified by the <time quantity>, the test execution will be resumed.
 - c) The <timer> label of the THEN RESET subfield must be identical to the <timer> label following WAIT FOR.

12.3 TIME INTERVAL Measurement Capability**12.3.1 <event> based measurements****12.3.1.1 Function**

To measure either the elapsed time between the occurrence of the <event> referenced in the FROM field of a sensor statement and the occurrence of the <event> referenced in the TO field of that sensor statement or the duration of an <event interval> in a sensor statement using the <noun> TIME INTERVAL.

12.3.1.2 Formal Syntax (not applicable)

12.3.1.3 Syntax Diagram (not applicable)

12.3.1.4 Rules

12.3.1.5 Notes and Examples

See statement 100200 of Example 2 in Section 6.17.5.

12.0 TIMING STATEMENTS**12.4 DO/END DO capability**

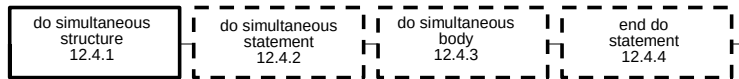
The DO/END DO capability can be used either to delimit a series of ATLAS statements which are to be executed simultaneously or to delimit a time critical digital test sequence or to delimit a series of ATLAS statements which are to be executed sequentially.

12.4.1 <do simultaneous structure>**12.4.1.1 Function**

To delimit a set of statements whose critical actions must be executed simultaneously.

12.4.1.2 Formal Syntax

FSD / 2159

12.4.1.3 Syntax Diagram**12.4.1.4 Rules**

1. <do simultaneous structure>s cannot be nested.

12.0 TIMING STATEMENTS

12.4.2 <do simultaneous statement>

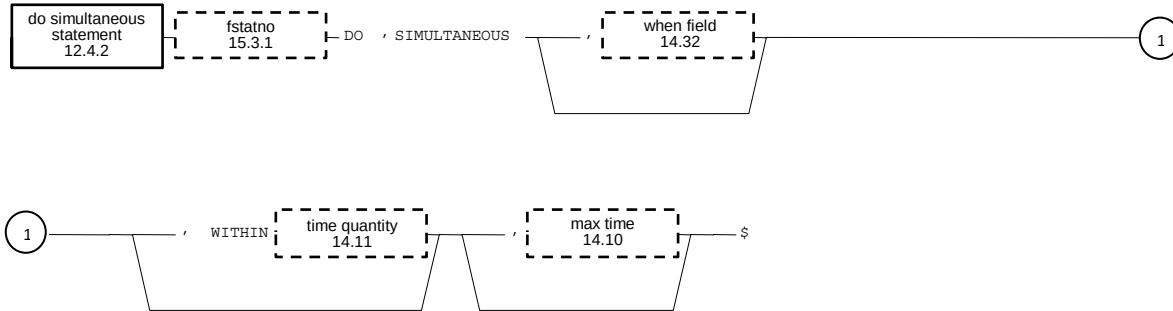
12.4.2.1 Function

To introduce a <do simultaneous structure>.

12.4.2.2 Formal Syntax

FSD / 2165

12.4.2.3 Syntax Diagram



12.4.2.4 Rules

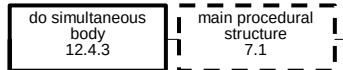
1. The optional <when field> allows the specification of the time of execution.
2. The optional WITHIN field specifies the maximum time available for the initiation of the execution of the critical actions of all the statements in the <do simultaneous structure>. Appropriate non-critical actions will be executed prior to the start of the time period defined by the WITHIN field. The boolean MAX TIME is set to FALSE when execution of the statement is initiated and is set TRUE when the time period specified by the WITHIN field is exceeded.
3. If the <do simultaneous statement> or any statement in the <do simultaneous body> contain references to <event>s which are not already enabled the <event>s are enabled at the commencement of execution of the critical actions of the statements within the <do simultaneous structure>. Any <event>, which is enabled as a result of this, is disabled on completion of the execution of the <do simultaneous structure>.

12.0 TIMING STATEMENTS12.4.3 <do simultaneous body>12.4.3.1 Function

To define the sequence of ATLAS procedural statements whose critical actions must be executed simultaneously.

12.4.3.2 Formal Syntax

FSD / 2173

12.4.3.3 Syntax Diagram12.4.3.4 Rules

1. A <do simultaneous body> shall include only signal statements for which a critical action is defined. The <do simultaneous body> shall not contain a statement implying a <fetch statement>. These statements are : <measure statement>, <read statement>, <fetch statement>, <update-read statement>, <verify statement> and <monitor statement>.

The <do simultaneous body> may contain an <arm statement> in order to induce a measurement by a sensor.

2. Statements within a <do simultaneous body> shall not include timing subfields that affect their critical actions.
3. Statements within a <do simultaneous body> shall not be the target of a <go to statement>.

12.4.4 <end do statement>12.4.4.1 Function

To close a <do simultaneous structure> or a <do timed digital structure>.

12.4.4.2 Formal Syntax

FSD / 2177

12.4.4.3 Syntax Diagram

12.0 TIMING STATEMENTS

12.4.5 <do timed digital structure>

12.4.5.1 Function

To delimit a digital test sequence of which the timing is specified in the <do timed digital statement>.

12.4.5.2 Formal Syntax

FSD / 2182

12.4.5.3 Syntax Diagram



12.4.6 <do timed digital statement>

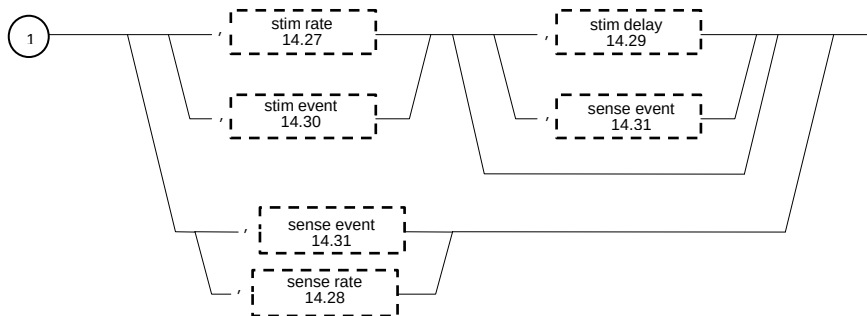
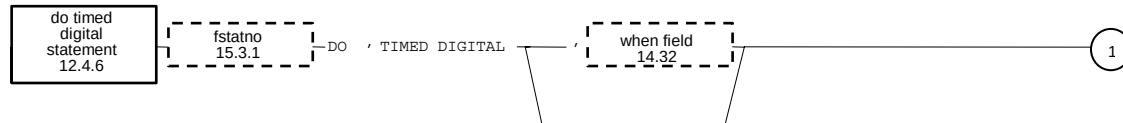
12.4.6.1 Function

To introduce a <do timed digital structure>.

12.4.6.2 Formal Syntax

FSD / 2188

12.4.6.3 Syntax Diagram



12.0 TIMING STATEMENTS**12.4.6.4 Rules**

1. The optional <when field> allows the specification of the time of execution of the first critical action within the <do timed digital structure>.

2. The subfields after the optional <when field> specify the timing of the critical actions of either a <stimulate statement> or a <sense statement> or a <prove statement> contained in the following <do timed digital body>. These subfields act as follows:

- a. <stim rate> or <stim event> when used singly:

Successively encountered <stimulate statement>s will be executed either at the rate specified in <stim rate> or at every occurrence of the referenced <event> if <stim event> is used. In the latter case <sense statement>s and <prove statement>s shall not be included in the following <do timed digital body>.

- b. <sense rate> or <sense event> when used singly:

Successively encountered <sense statement>s or <prove statements> will be executed either at the rate specified in <sense rate> or at every occurrence of the referenced <event> if <sense event> is used. In the latter case <stimulate statement>s shall not be included in the following <do timed body>.

- c. <sense delay> when used with <stim rate> or <stim event>:

The <sense delay> specified must be less than the expected stimulus interval. If either a <sense statement> or a <prove statement> is encountered following a <stimulate statement>, then the critical action of the <sense statement> or the <prove statement> will be delayed from the critical action of the <stimulate statement> by the time specified in <sense delay>. If there are no <sense statement>s or <prove statement>s between any successively encountered <stimulate statement>s, the timing of these successive <stimulate statement>s will be as specified by the associated <do statement>. If successive <sense statement>s or <prove statement>s are encountered with no intervening <stimulate statement>s, their critical actions will still be timed as if there had been intervening <stimulate statement>s.

- d. <sense event> when used with <stim rate> or <stim event>:

If either a <sense statement> or a <prove statement> is encountered following a <stimulate statement>, then the critical action of either a <sense statement> or a <prove statement> will take place at the next occurring <sense event> following the <stimulate statement>. If there are no <sense statement>s or <prove statement>s between successively encountered <stimulate statement>s, the timing of these successive <stimulate statement>s will be as specified in the associated <do timed digital statement>. If the time between one stimulus and the next <sense event> is longer than the stimulus interval, the next occurring stimulus will be on the next beat of <stim rate> or the next <stim event> following the <sense event>. If successive <sense statement>s or <prove statement>s are encountered with no intervening <stimulate statement>s their critical actions will be timed by successively occurring <sense events>.

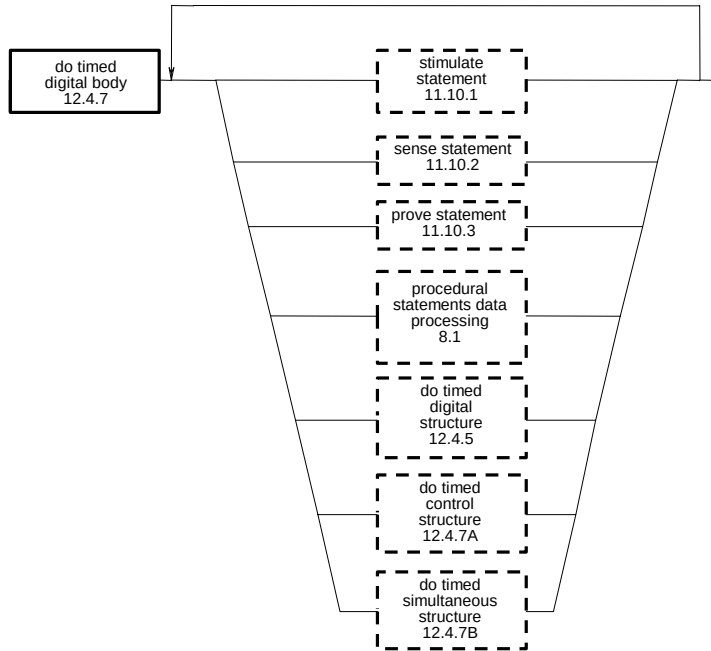
3. When a <do simultaneous structure> is included within a <do timed digital structure>, it shall only include either <stimulate statement>s or <sense statement>s and <prove statement>s so as not to contradict the timing specified in the <do timed digital statement>.
4. If the <do timed digital statement> or any statement in the <do timed digital body> or their referenced <define digital source statement>s or <define digital sensor statement>s contain references to <event>s which are not already enabled, the <event>s are enabled at the commencement of execution of the <do timed digital structure>. Any <event>, which is enabled as a result of this, is disabled on completion of the execution of the <do timed digital structure>.

12.0 TIMING STATEMENTS**12.4.7 <do timed digital body>****12.4.7.1 Function**

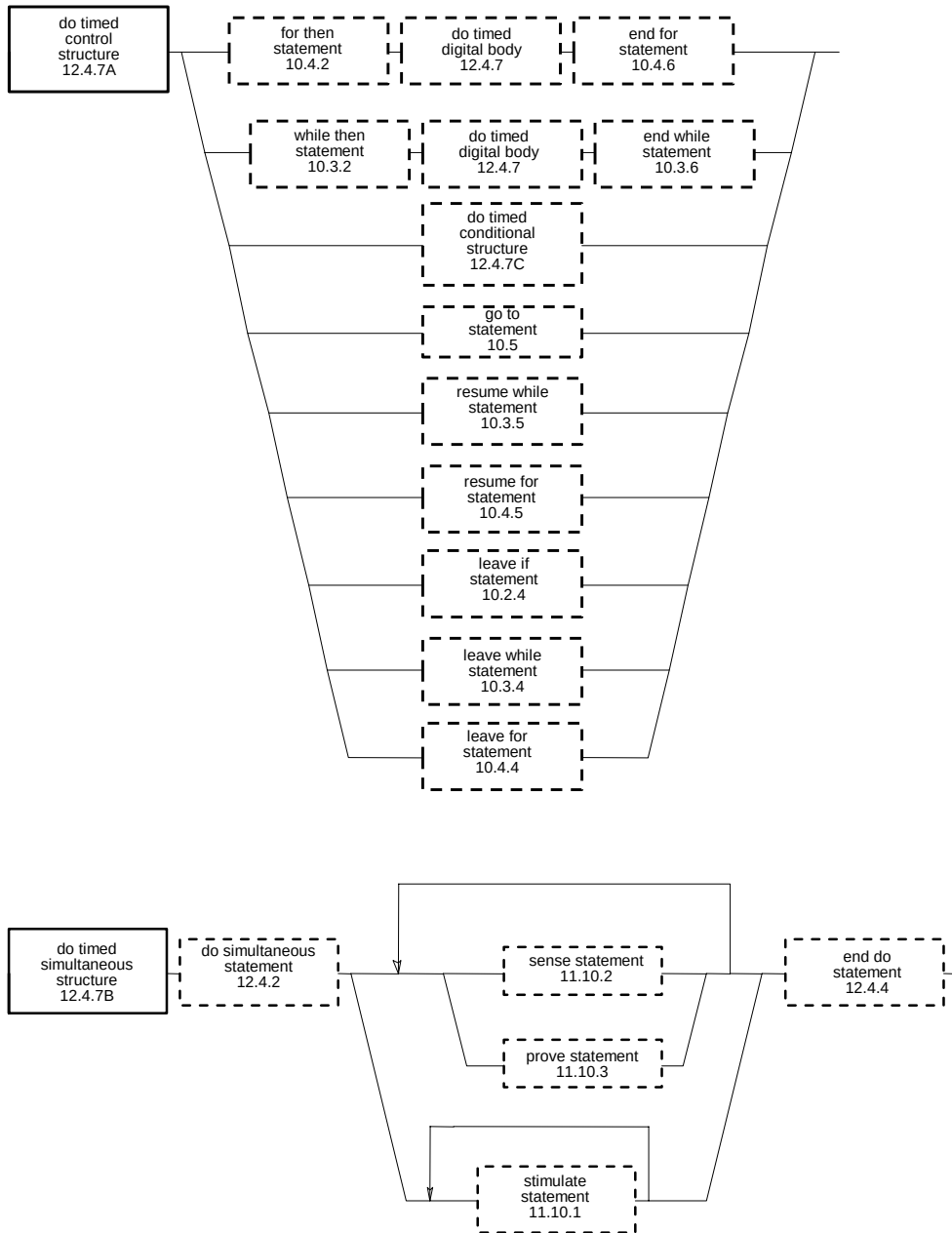
To define the sequence of ATLAS procedural statements which constitute the timed digital test sequence.

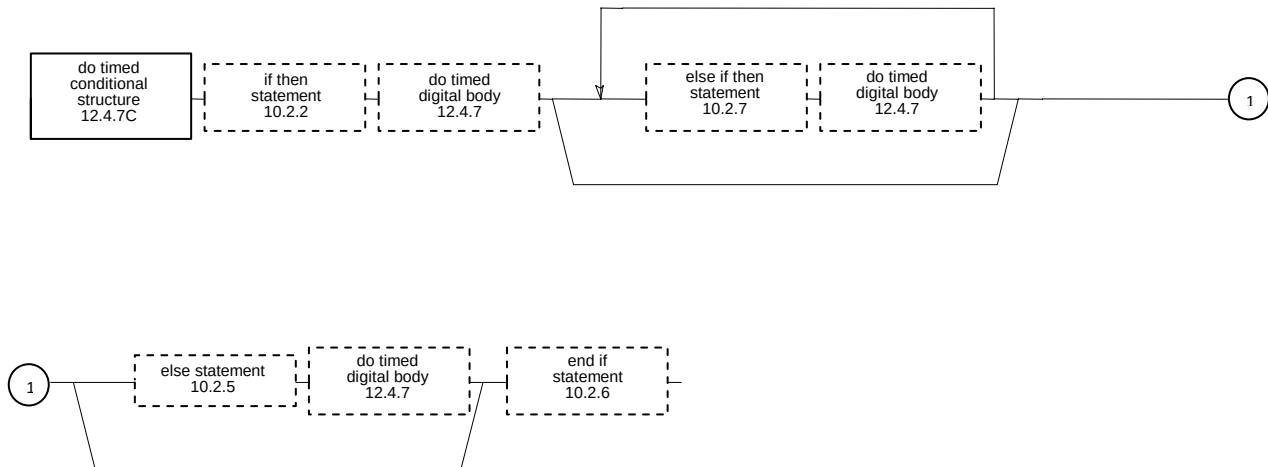
12.4.7.2 Formal Syntax

FSD / 2201

12.4.7.3 Syntax Diagram

12.0 TIMING STATEMENTS



12.0 TIMING STATEMENTS**12.4.7.4 Rules**

- Only the following statements are allowed within a <do timed digital body>:

STIMULATE	END, IF
SENSE	RESUME, FOR
PROVE	RESUME, WHILE
Do simultaneous	LEAVE, FOR
END, DO	LEAVE, WHILE
FOR	LEAVE, IF
WHILE	CALCULATE
IF	GO TO
END, FOR	ELSE
END, WHILE	ELSE IF
- Statements within a <do timed digital body> shall not include timing subfields that affect their critical actions.
- Statements within a <do timed digital body> shall not be the target of a <go to statement> that is external to the body.
- Whenever a <do simultaneous structure> is used within a <do timed digital body>, it must contain exclusively <stimulate statement>s or <sense statement>s and <prove statement>s.
- A <go to statement> in a <do timed digital body> must not reference a statement that is outside the body.

12.4.7.5 Notes and Examples

There are many requirements for sequences of different combinations of STIMULATE, SENSE and PROVE statements within a <do timed digital structure>. These can be accomplished by using FOR loops. Arrays may be used within the FOR loops with either the loop index or other <data store>s employed as the array indexes.

An Example of a <do timed digital statement> is as follows:

```

000250DO, TIMED DIGITAL, STIM-RATE 1MHZ ERRLMT +-0.1 MHZ,
      SENSE-DELAY 100NSEC ERRLMT +-5NSEC $
      FOR, 'I' = 1 THRU 200, THEN $
          STIMULATE, 'STIM'('I'), ON 'PCB-IN' $
          PROVE, (VALUE INTO 'VAL') REF 'RESP'('I'),
              ON 'PCB-OUT' $
      END, FOR $
END, DO $
  
```

12.0 TIMING STATEMENTS12.4.8 <do sequential structure>12.4.8.1 Function

To delimit a set of statements which must be executed sequentially without timing disruption due to implementation.

12.4.8.2 Formal Syntax

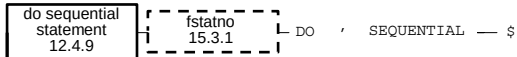
FSD / 2248

12.4.8.3 Syntax Diagram12.4.9 <do sequential statement>12.4.9.1 Function

To introduce a <do sequential structure>.

12.4.9.2 Formal Syntax

FSD / 2254

12.4.9.3 Syntax Diagram12.4.9.4 Rules

1. A <do sequential structure> cannot appear within another <do sequential structure> or a <do simultaneous structure>

12.0 TIMING STATEMENTS12.4.10 <do sequential body>12.4.10.1 Function

To define the sequence of ATLAS procedural statements whose critical actions must be executed sequentially .

12.4.10.2 Formal Syntax

FSD / 2259

12.4.10.3 Syntax Diagram12.4.10.4 Rules

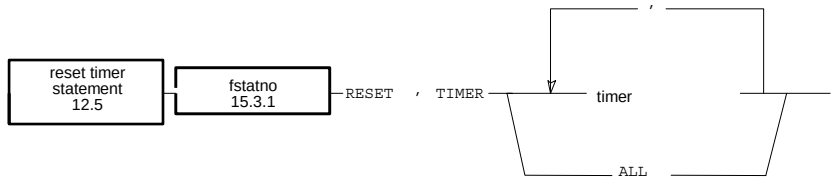
- Statements within a <do sequential body> shall not be the target of a <go to statement>.

12.5 <reset timer statement>12.5.1 Function

To reset a <timer> to zero.

12.5.2 Formal Syntax

FSD / 2263

12.5.3 Syntax Diagram12.5.4 Rules

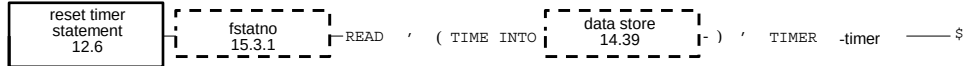
- A <timer> has no idle state. It starts counting time from zero as soon as it is reset.

12.0 TIMING STATEMENTS12.6 <read timer statement>12.6.1 Function

To read the current time value of a <timer> and retain that value in the INTO <data store> location.

12.6.2 Formal Syntax

FSD / 2272

12.6.3 Syntax Diagram12.6.4 Rules

1. The current time value of the specified <timer> is established and transferred into the <data store> which must be of type DECIMAL.
2. The <timer> specified is not affected by this statement. Time counting continues without interruption.
3. The units and resolution of the time value are established by the related <identify timer statement>.
4. MEASUREMENT' is unaffected.

13.0 DATABUS STATEMENTS

13.0.1 Function

Most ATLAS statements describe single logical operations that are relatively simple to comprehend and are stimulus, sensor, or supportive in nature. Some situations occur, however, that require a mixture of stimulus, sensor, and control functions to perform a single logical function.

A common requirement is to test a UUT or group of UUT's that communicate via one or multiple digital data buses. Although the bus communication could be handled by standard digital testing techniques, the enormous amount of data involved makes it difficult, if not impossible, to discern the logical operations being performed. Frequently, only part of the information involved is of interest to the test being performed. On occasion the ensuing real time test actions are dependent on results of some current action. In order to provide a more readable and understandable description of such situations, the data bus testing macro statement described in this section is provided.

Data bus processing, as discussed in this section, assumes that the data bus itself is functioning correctly and that the UUT communicates via a bus or buses. There are four separate capabilities which support the testing of UUT that utilize buses:

The first capability is the <establish protocol statement> which is described in Section 6.11. This capability permits the description of the digital and analog handshaking and other requirements.

The second capability is the <define exchange statement> which is described in Section 6.20. This capability permits the description of a single complete transaction or exchange on the bus. Both the <establish protocol statement> and the group of <define exchange statement>s can be used in various test programs for different UUT that are normally tied together on the same bus because they only identify the functioning of the bus itself and the types of transactions that may appear on the bus.

The third capability is the <do exchange statement> which is described in Section 13.1. This capability permits the description of the actual traffic on the bus in terms of the exchanges also specifying the type of test equipment participation in each exchange.

The fourth capability is the <update exchange statement> which is described in Section 13.2. This capability changes the actual bus traffic without interrupting the databus operation.

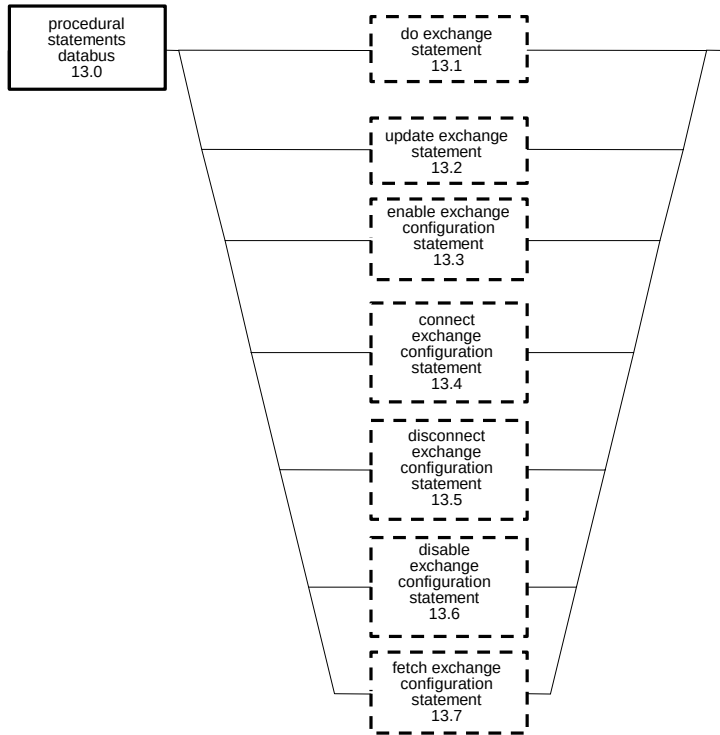
For bus testing purposes, the UUT is considered to be either one "black box" or a group of "black boxes" connected by a data bus.

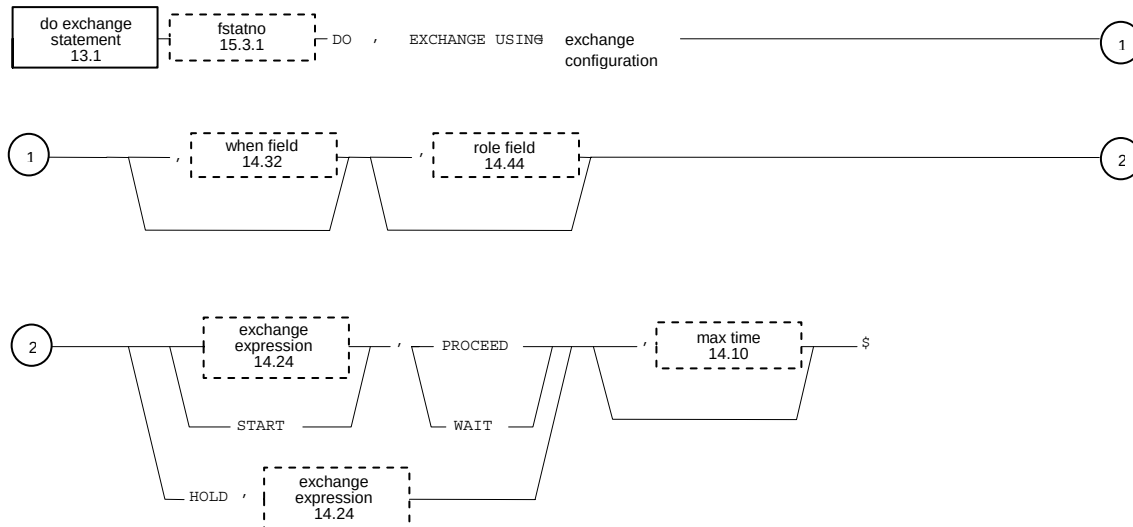
13.0 DATABUS STATEMENTS

13.0.2 Formal Syntax

FSD / 2282

13.0.3 Syntax Diagram





13.0 DATABUS STATEMENTS**13.1.4 Rules**

1. The USING subfield references the associated <exchange configuration>.
2. The optional <when field> at the beginning of the <do exchange statement> defines the start condition for the execution of the first transaction that is included in the <do exchange statement>. It precedes the starting condition associated with each <exchange frame>.
3. The PROCEED subfield requires that the execution of the <do exchange statement> continues in parallel with the execution of other statements in the <atlas test specification structure>. The WAIT subfield requires that the execution of the next ATLAS statement starts after completion of the <do exchange statement>.
4. The <do exchange statement> must only reference <exchange>s which are defined in the preamble. An <exchange frame> within an <exchange expression> references them by their defined labels. They execute as defined when the <do exchange statement> is executed and continue to do so unless a subsequent <update exchange statement> modifies them.
5. Only one <do exchange statement> can be active for a single databus at one time. A number of databuses can be active simultaneously by executing two or more <do exchange statement>s with the PROCEED option simultaneously.
6. The <do exchange statement> is terminated as soon as one of the following conditions occurs:
 - a) all expressed <exchange expressions> are complete
 - b) The MAX-TIME condition established by either the <exchange expression> or the <do exchange statement> has expired.
 - c) a <finish statement> occurs.
7. If the <role field> is used, the parameters represent the functions or roles which the test equipment can assume. They can be MASTER, SLAVE, MONITOR or others introduced by use of the <test-equip-role name> field of the referenced <establish protocol statement>. Whichever is employed must be listed in the <bus installation parameters> section of the referenced <establish protocol statement>s. When used at this level, the test equipment role is specified for the whole <do exchange statement>. If the <role field> is bypassed in the <do exchange statement>, the role of the test equipment is to be specified for each <exchange> within the <exchange frame> definition. Where <role field> is specified both within the <do exchange statement> and within an <exchange frame> for a given <exchange>, the <role field> in the <exchange frame> overrides that specified for the entire <do exchange statement> for the particular <exchange>.
8. An exchange must only be referenced once in an <exchange frame>.
9. The HOLD option prepares the device but does not start the exchanges. The START option initiates a previously prepared device that had a HOLD field.

13.1.5 Notes and Examples

Section 6.11.5 describes the fundamental philosophy of databus testing.

The following examples illustrate the use of ATLAS constructs for databus testing for Mono-Bus applications.

13.0 DATABUS STATEMENTS**13.1.5.1 Example 1 - Mil. Std. 1553B**

For the following example we shall consider a bus system which, in operational mode, includes:

- an inertial platform (INS),
- an autopilot,
- an Air-Data Computer (ADC) and
- a main computer responsible for bus traffic.

Information exchange between components is needed to run, and therefore to test, each component. To test one of them, when out of the system, the test equipment must simulate the operational environment.

The frequency in parentheses indicates the performance frequency for each exchange. The following portions of an <atlas test specification structure> show the use of the capabilities of the ATLAS language.

In the ATLAS test program preamble which follows, the <define exchange configuration statement> identifies the <exchange configuration> and gives the name of the protocol used. The <establish protocol statement> identifies the protocol for the bus and indicates the format of the fields used. Each <define exchange statement> in the preamble references the <establish protocol statement>.

The <define exchange statement> accomplishes the following:

- 1) Refers to the <establish protocol statement> for the exchange.
- 2) States the bus mode for the exchange.
- 3) States the talker, listener(s) and data as required by the bus mode.
- 4) Identifies the exchange by a label.

```

000100 DECLARE, TYPE, 'DEVICE-IDENTIFIER' IS ENUMERATION
      (' INS-GAMMA-FUNCTION' ,
        ' INS-GEO-FUNCTION' ,
        ' INS-ATT-FUNC' ,
        ' ADC-B-FUNCTION' ,
        ' MASTER-COMPUTER' ,
        ' ADC-M-FUNC' ,
        ' ADC-S-FUNC' ,
        ' ADC-TAS-FUNCTION' ,
        ' ACC-MASTER-COMPUTER-FUNC' ,
        ' GEO-MASTER-COMPUTER-FUNC' ,
        ' AUTO-PILOT' ,
        ' B-MASTER-COMPUTER-FUNC' ,
        ' INS' ,
        ' AUTOPILOT-H-FUNC' ,
        ' MACH-MASTER-COMPUTER-FUNC' ,
        ' SPO-MASTER-COMPUTER-FUNC' ,
        ' TAS-MASTER-COMPUTER-FUNC' ) $

000110 DECLARE, VARIABLE, 'ATTITUDES' IS
      RECORD OF ['PHI1', 'PHI2', 'THETA1', 'THETA2'
        'PSI1', 'PSI2' IS INTEGER] $

000120 DECLARE, VARIABLE, 'SPEED' IS
      RECORD OF ['X1', 'X2', 'X3', 'Y1', 'Y2', 'Y3',
        'Z1', 'Z2', 'Z3' IS INTEGER] $

000130 DECLARE, VARIABLE, 'POSITION' IS
      ARRAY (1 THRU 5) OF RECORD
        OF ['L1', 'L2', 'L3', 'G1', 'G2', 'G3',
          'Z1', 'Z2', 'Z3' IS INTEGER] $

```

13.0 DATABUS STATEMENTS

```

002000 ESTABLISH, BUS-PROTOCOL 'BUS-1',
      SPEC 'AIR-STANDARD-A1', STANDARD PRIMARY BUS,
      TEST-EQUIP-ROLE MASTER, MONITOR, SLAVE,
      TEST-EQUIP-MONITOR COMMAND, DATA, STATUS,
      BUS-MODE ALL-LISTENER,
      DATA (INTEGER),
      BUS-MODE TALKER-LISTENER,
      TALKER 'MASTER-COMPUTER' (INTEGER),
      LISTENER 'AUTOPILOT-H-FUNC',
      DATA (INTEGER),
      BUS-MODE TALKER-LISTENER,
      TALKER 'ADC-B-FUNCTION',
      'ADC-M-FUNC',
      'ADC-S-FUNC',
      'ADC-TAS-FUNCTION',
      LISTENER 'B-MASTER-COMPUTER-FUNC',
      'MACH-MASTER-COMPUTER-FUNC',
      'TAS-MASTER-COMPUTER-FUNC',
      'SPO-MASTER-COMPUTER-FUNC',
      DATA (INTEGER),
      BUS-MODE TALKER-LISTENER,
      TALKER 'INS-ATT-FUNC',
      LISTENER 'MASTER-COMPUTER' (INTEGER),
      'AUTO-PILOT' (INTEGER),
      DATA (RECORD OF ['FIRST', 'SECOND',
      'THIRD', 'FOURTH', 'FIFTH',
      'SIXTH' IS INTEGER]),
      BUS-MODE TALKER-LISTENER,
      TALKER 'INS-GAMMA-FUNCTION',
      'INS-GEO-FUNCTION',
      LISTENER 'ACC-MASTER-COMPUTER-FUNC',
      'GEO-MASTER-COMPUTER-FUNC',
      DATA (RECORD OF ['A', 'B', 'C', 'D', 'E',
      'F', 'G', 'H', 'J' IS INTEGER]),
      CNX A B $

002500 DEFINE, 'CONF1', EXCHANGE CONFIGURATION 'BUS-1' $

003000 DEFINE, 'SYN-10', EXCHANGE, PROTOCOL 'BUS-1',
      BUS-MODE ALL-LISTENER,
      DATA 21 $

10 DEFINE, 'SYN-40', EXCHANGE, PROTOCOL 'BUS-1',
      BUS-MODE ALL-LISTENER
      DATA 22 $

20 DEFINE, 'SPEED', EXCHANGE, PROTOCOL 'BUS-1',
      BUS-MODE TALKER-LISTENER,
      TALKER 'INS-GAMMA-FUNCTION',
      LISTENER 'ACC-MASTER-COMPUTER-FUNC',
      DATA 'SPEED' $

30 DEFINE, 'GEO-POSITION', EXCHANGE, PROTOCOL 'BUS-1',
      BUS-MODE TALKER-LISTENER,
      TALKER 'INS-GEO-FUNCTION',
      LISTENER 'GEO-MASTER-COMPUTER-FUNC',
      DATA 'POSITION'(1 THRU 5) $

40 DEFINE, 'ATTITUDE', EXCHANGE, PROTOCOL 'BUS-1',
      BUS-MODE TALKER-LISTENER,
      TALKER 'INS-ATT-FUNC',
      LISTENER 'MASTER-COMPUTER' (22),
      AUTO-PILOT (25),
      DATA 'ATTITUDES' $

50 DEFINE, 'BARO-ALT', EXCHANGE, PROTOCOL 'BUS-1',

```

13.0 DATABUS STATEMENTS

```

BUS-MODE TALKER-LISTENER,
TALKER 'ADC-B-FUNCTION',
LISTENER 'B-MASTER-COMPUTER-FUNC',
DATA 'ZB' $

60 DEFINE, 'HEADING', EXCHANGE, PROTOCOL 'BUS-1',
BUS-MODE TALKER-LISTENER,
TALKER 'MASTER-COMPUTER' (14),
LISTENER 'AUTO-PILOT-H-FUNC',
DATA 'HEADING' $

70 DEFINE, 'MACH', EXCHANGE, PROTOCOL 'BUS-1',
BUS-MODE TALKER-LISTENER,
TALKER 'ADC-M-FUNC',
LISTENER 'MACH-MASTER-COMPUTER-FUNC',
DATA 'MACH' $

80 DEFINE, 'VERTICAL-SPEED', EXCHANGE, PROTOCOL 'BUS-1',
BUS-MODE TALKER-LISTENER,
TALKER 'ADC-S-FUNC',
LISTENER 'SPO-MASTER-COMPUTER-FUNC',
DATA 'SZ' $

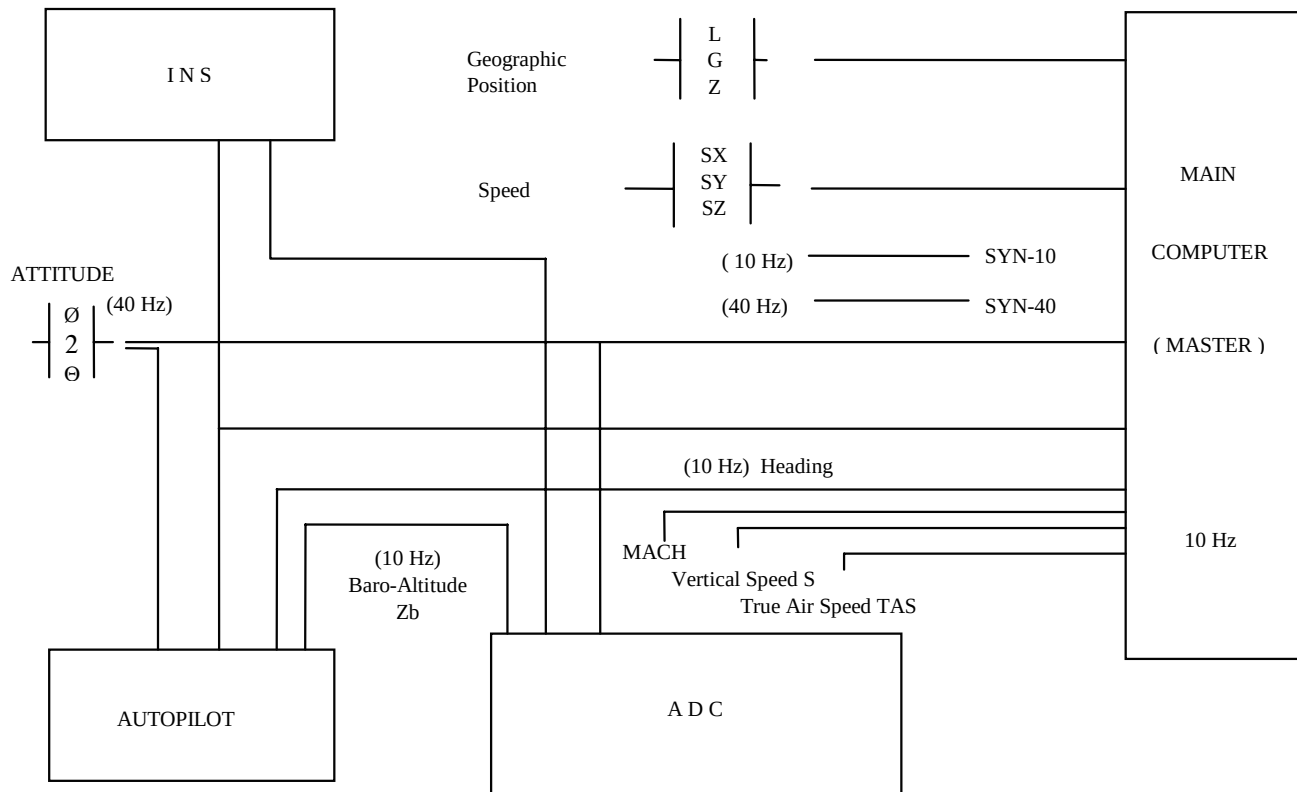
90 DEFINE, 'TRUE-AIR-SPEED', EXCHANGE, PROTOCOL 'BUS-1',
BUS-MODE TALKER-LISTENER,
TALKER 'ADC-TAS-FUNCTION',
LISTENER 'TAS-MASTER-COMPUTER-FUNC',
DATA 'TAS' $

```

Both the bus modes and the test-equip modes in this example have descriptive names. So have all the talkers and listeners and most of the sub-addresses. The greater clarity of using descriptive names is clear from the relative unfriendliness of the numerical sub-addresses.

Suppose we want the bus traffic for 500 MSEC shown as alternative descriptions in Figure 13-2 and Table 13-1.

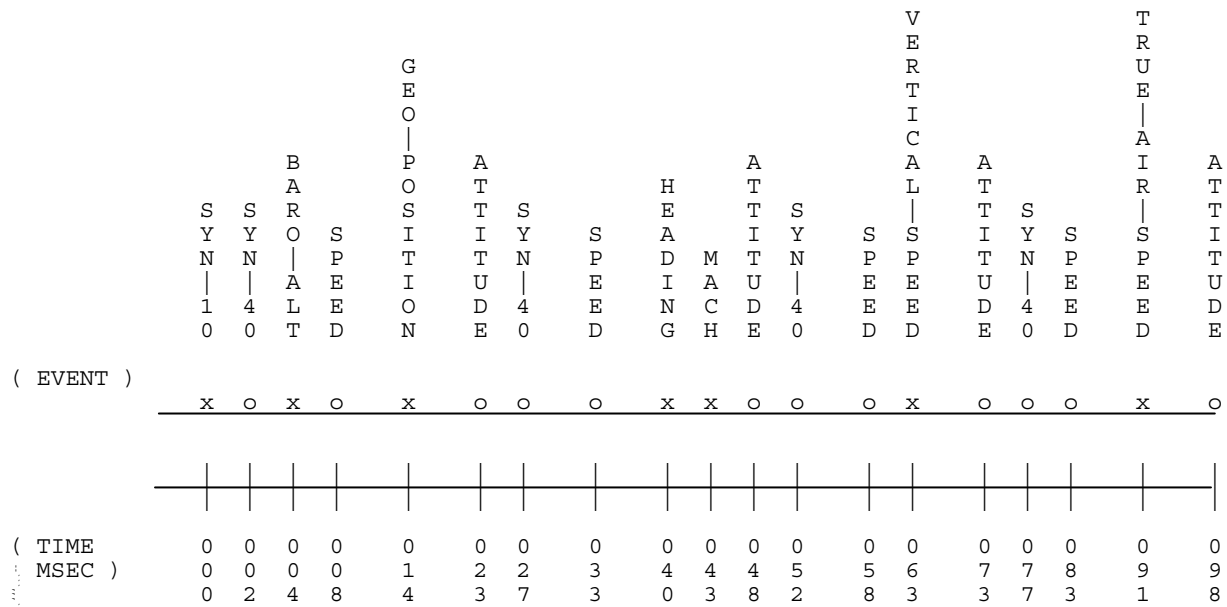
The following drawing shows the operational exchanges for example 1.

13.0 DATABUS STATEMENTS**Figure 13-1 Diagram for EXCHANGE, Example 1**

13.0 DATABUS STATEMENTS

Table 13-1 Definition of Bus Traffic for EXCHANGE, Example 1.

EXCHANGE	TIME MSEC	TEST EQUIP	BUS-MODE ADDRESS	TALKER		LISTENER		DATA
				SUBADDRESS	ADDRESS	SUBADDRESS		
'SYN-10'	0	CONTROL MASTER	ALL-LISTENER-MODE					21
'SYN-40'	2 27 52 77	CONTROL MASTER	ALL-LISTENER-MODE					22
BARO-ALT	4	CONTROL MONITOR	TALKER LISTENER-MODE	ADC	B-FUNC	AUTO-PILOT INS	B-FUNC 18	'2B'
'SPEED'	8 33 58 83	MONITOR 'SX', 'SY', 'SZ' CONTROL SLAVE	TALKER-LISTENER-MODE	INS	GAMMA-FUNCTION	MASTER-COMPUTER	ACC-FUNC	'SX', 'SY', 'SZ'
'GEO-POSITION'	14	CONTROL MONITOR	TALKER-LISTENER-MODE	INS	GEO-FUNCTION	MASTER-COPUTER	GEO-FUNC	'L'(1), 'G'(1), 'Z'(1)
'ATTITUDE'	23 48 73 98	MONITOR 'PHI' 'PSI', 'THETA' CONTROL SLAVE	TALKER-LISTENER-MODE	INS	ATT-FUNC	MASTER-COMPUTER AUTO-PILOT	22 25	'PHI,' 'THETA,' 'PSI'
'HEADING'	40	SIMULATE AUTO-PILOT CONTROL SLAVE	TALKER-LISTENER-MODE	MASTER-COMPUTER	14	AUTO-PILOT		'HEADING'
'MACH'	43	SIMULATE ADC CONTROL SLAVE	TALKER-LISTENER-MODE	ADC	M-FUNC	MASTER-COMPUTER	MACH-FUNC	'MACH'
'VERTICAL-SPEED'	63	SIMULATE ADC CONTROL SLAVE	TALKER-LISTENER-MODE	ADC	S-FUNC	MASTER-COMPUTER	SPD-FUNC	'SZ'
'TRUE-AIR-SPEED'	91	MONITOR TAS CONTROL SLAVE	TALKER-LISTENER-MODE	ADC	TAS-FUNC	MASTER-COMPUTER	TAS-FUNC	'TAS'
CLOCK: 'TICKER'	0 to 100 by 1							

13.0 DATABUS STATEMENTS**Figure 13-2 - Definition of Bus Traffic for EXCHANGE, Example 1**

13.0 DATABUS STATEMENTS

The following ATLAS procedural statement would implement the traffic:

```

004000  ENABLE, EXCHANGE CONFIGURATION USING 'CONF-1',
        PROTOCOL 'BUS-Q, CNX A BUS-SIG B BUS-RET $

005000  DO, EXCHANGE USING 'CONF-1', TEST-EQUIP-ROLE SLAVE,
        ( STARTING WHEN 'CLK1' 0 MSEC, WITH PRIORITY 1,
          EVERY 100 MSEC, UNTIL TIMER 'CLK1' EQ 500 MSEC,
            EXCHANGE, 'SYN-10',
              TEST-EQUIP-ROLE MASTER, DELAY 4 MSEC,
            EXCHANGE, 'BARO-ALT', TEST-EQUIP-ROLE
              MONITOR,
              TEST-EQUIP-MONITOR DATA 'ZB',
              DELAY 10 MSEC,
            EXCHANGE, 'GEO-POSITION', TEST-EQUIP-ROLE
              MONITOR,
              TEST-EQUIP-MONITOR DATA 'POSITION' (1 THRU 5),
              DELAY 26 MSEC,
            EXCHANGE, 'HEADING', TEST-EQUIP-SIMULATE
              LISTENER 'AUTO-PILOT-H-FUNC',
              TEST-EQUIP-MONITOR DATA 'HEADING',
              DELAY 3 MSEC,
            EXCHANGE, 'MACH', TEST-EQUIP-SIMULATE
              TALKER 'ADC-M-FUNC', DELAY 20 MSEC,
            EXCHANGE, 'VERTICAL-SPEED',
              TEST-EQUIP-SIMULATE TALKER 'ADC-S-FUNC',
              DELAY 28 MSEC,
            EXCHANGE, 'TRUE-AIR-SPEED',
              TEST-EQUIP-MONITOR DATA 'TAS' ),
        AND ( STARTING WHEN 'CLK1' 2 MSEC, WITH PRIORITY 2,
          EVERY 25 MSEC, UNTIL TIMER 'CLK1' EQ 500 MSEC,
            EXCHANGE, 'SYN-40',
              TEST-EQUIP-ROLE MASTER,
              DELAY 6 MSEC,
            EXCHANGE, 'SPEED', TEST-EQUIP-ROLE MONITOR,
              TEST-EQUIP-MONITOR DATA 'SPEED',
              DELAY 15 MSEC,
            EXCHANGE, 'ATTITUDE', TEST-EQUIP-ROLE MONITOR,
              TEST-EQUIP-MONITOR DATA 'ATTITUDES' ),
        PROCEED, MAX-TIME 505 MSEC $

```

13.0 DATABUS STATEMENTS**13.1.5.2 Example 2 - ARINC Specification 629**

```

000001 BEGIN, ATLAS PROGRAM 'ARINC 629 TEST EXAMPLE'
C
C      THIS SAMPLE ATLAS PROGRAM WILL TRANSMIT 10 ARINC 629 WORDS
C      AND RECEIVE 10 ARINC 629 WORDS WITH TIME STAMPS USING DO
C      EXCHANGE
C
C      TRANSMIT DATA:                RECEIVE DATA:
C
C      100D0A00                      100D0A01
C      100D0B00                      100D0B01
C      100D0C00                      100A0C01
C      100D0D00                      100A0D01
C      100A0E00                      100A0E01
C      100A0F00                      100C0F01
C      100C1000                      100C1001
C      100B1100                      100B1101
C      100B1200                      100B1201
C      100B1300                      100B1301
C
C*****
C$
010000 DECLARE, TYPE, 'DEVICE-IDENTIFIER' IS ENUMERATION
      ('629-TESTER', '629-UUT') $
C$
010010 DECLARE, TYPE, 'SETUP' IS RECORD OF
      [ 'TERMINAL-GAP'                IS DECIMAL;
        'SYNC-GAP'                   IS DECIMAL;
        'TRANSMIT-INTERVAL'          IS DECIMAL;
        'CRC-MODE'                   IS INTEGER ] $
C$
010020 DECLARE, CONSTANT, 'HIGH'      IS 1;
      'LOW'                      IS 0;
      'ENABLE'                    IS 1;
      'DISABLE'                   IS 0 $
C$
010030 DECLARE, VARIABLE, 'XPP-SETUP' IS 'SETUP' $
010040 DECLARE, VARIABLE, 'RPP-SETUP' IS 'SETUP' $
C$
010050 DECLARE, VARIABLE, 'XPP-DATA' IS ARRAY(1 THRU 10)
      OF STRING(32) OF BIT
      INITIAL = (X'100D0A00',X'100D0B00',X'100D0C00',
        X'100D0D00',X'100A0E00',X'100A0F00',
        X'100C1000',X'100B1100',X'100B1200',
        X'100B1300') $
C$
010060 DECLARE, VARIABLE, 'RPP-DATA' IS ARRAY(1 THRU 10)
      OF STRING(32) OF BIT
      INITIAL = (X'100D0A01',X'100D0B01',X'100A0C01',
        X'100A0D01',X'100A0E01',X'100C0F01',
        X'100C1001',X'100B1101',X'100B1201',
        X'100B1301') $
C$
010070 DECLARE, VARIABLE, 'RESP-DATA' IS ARRAY(1 THRU 10)
      OF STRING(32) OF BIT $
C$
010080 DECLARE, VARIABLE, 'TIME-STAMP' IS ARRAY(1 THRU 10)
      OF DECIMAL $
C$
010090 DECLARE, VARIABLE, '.START', '.END', '.INDEX' IS INTEGER $
C$
020000 ESTABLISH, BUS-PROTOCOL 'ARINC-629',

```

13.0 DATABUS STATEMENTS

```

SPEC 'ARINC SPEC 629',
TEST-EQUIP-ROLE MASTER, MONITOR,
TEST-EQUIP-MONITOR DATA,
BUS-MODE AUTONOMOUS,
    TALKER '629-TESTER',
    LISTENER '629-UUT',
    DATA (STRING(32) OF BIT),
INITIALIZE,
    COMMAND ('SETUP')
    DATA (STRING(32) OF BIT),
BUS-MODE AUTONOMOUS,
    TALKER '629-UUT',
    LISTENER '629-TESTER',
    DATA (STRING(32) OF BIT),
    STATUS(DECIMAL),
INITIALIZE,
    COMMAND ('SETUP')
    DATA (STRING(32) OF BIT),
BUS-PARAMETER
    ONE-AMPLITUDE RANGE 4.5 V TO 6.5 V BY 0.1 V,
    ZERO-AMPLITUDE RANGE -6.5 V TO -4.5 V BY 0.1 V,
STANDARD BUS,
CNX HI LO $

C$
031000 DEFINE, '629-CONF', EXCHANGE CONFIGURATION 'ARINC-629' $
C$
    10 DEFINE, 'XMIT-629', EXCHANGE, PROTOCOL 'ARINC-629',
        BUS-MODE AUTONOMOUS,
            TALKER '629-TESTER',
            LISTENER '629-UUT',
            DATA 'XPP-DATA'('START' THRU 'END') $
C$
    20 DEFINE, 'RECEIVE-629', EXCHANGE, PROTOCOL 'ARINC-629',
        BUS-MODE AUTONOMOUS,
            TALKER '629-UUT',
            LISTENER '629-TESTER',
            DATA 'RESP-DATA'('START' THRU 'END'),
            STATUS 'TIME-STAMP'('START' THRU 'END') $
C$
099000 COMMENCE, MAIN PROCEDURE $
C$
E100000 BEGIN, BLOCK, '1.0 629-TEST' $
C$
    100010 CALCULATE, 'XPP-SETUP'. 'TERMINAL-GAP'           = 0.000025,
                    'XPP-SETUP'. 'SYNC-GAP'                = 0.000025,
                    'XPP-SETUP'. 'TRANSMIT-INTERVAL'       = 0.030,
                    'XPP-SETUP'. 'CRC-MODE'                = 'DISABLE' $
C$

    100020 CALCULATE, 'RPP-SETUP'. 'TERMINAL-GAP'           = 0.000025,
                    'RPP-SETUP'. 'SYNC-GAP'                = 0.000025,
                    'RPP-SETUP'. 'TRANSMIT-INTERVAL'       = 0.030,
                    'RPP-SETUP'. 'CRC-MODE'                = 'DISABLE' $
C$
    100100 ENABLE, EXCHANGE CONFIGURATION USING '629-CONF',
        PROTOCOL 'ARINC-629',
        INITIALIZE,
            TALKER '629-TESTER',
            COMMAND 'XPP-SETUP',
            DATA 'XPP-DATA' (1 THRU 10),
            LISTENER '629-TESTER',
            COMMAND 'RPP-SETUP',
            DATA 'RPP-DATA' (1 THRU 10),
        CNX HI BUS-HI LO BUS-LO $

```

13.0 DATABUS STATEMENTS

```

C$
100110 CONNECT, EXCHANGE CONFIGURATION USING '629-CONF',
          PROTOCOL 'ARINC-629',
          CNX HI BUS-HI  LO BUS-LO $
C$
100500 CALCULATE, '.START' = 1,
          '.END'   = 10 $
C$
100510 DO, EXCHANGE USING '629-CONF' TEST-EQUIP-ROLE MASTER,
          ( EXCHANGE 'XMIT-629'), PROCEED $
C$
100600 DO, EXCHANGE USING '629-CONF' TEST-EQUIP-ROLE MONITOR,
          ( EXCHANGE 'RECEIVE-629'), PROCEED $
C$
10  FETCH, 'RECEIVE-629', EXCHANGE USING '629-CONF',
          DATA 'RESP-DATA'(1 THRU 10),
          STATUS 'TIME-STAMP'(1 THRU 10) $
C$
100700 OUTPUT, TEXT, FROM C'EXPECTED DATA          RECEIVED DATA' $
10  FOR, '.INDEX' = 1 THRU 10, THEN $
20  OUTPUT, TEXT, FROM 'RPP-DATA'('.INDEX') : HEXADECIMAL 8,
          C'          ',
          'RESP-DATA'('.INDEX') : HEXADECIMAL 8,
          C'          ',
          'TIME-STAMP'('.INDEX') $
30  END, FOR $
C$
100740 OUTPUT, TEXT, FROM C'END OF 1.0 629-TEST' $
C$
100800 DISCONNECT, EXCHANGE CONFIGURATION USING '629-CONF',
          PROTOCOL 'ARINC-629',
          CNX HI BUS-HI  LO BUS-LO $
C$
100900 DISABLE, EXCHANGE CONFIGURATION USING '629-CONF' $
C$
100998 REMOVE, ALL $
100999 END, BLOCK, '1.0 629-TEST' $
C$
999999 TERMINATE, ATLAS PROGRAM 'ARINC 629 TEST EXAMPLE' $

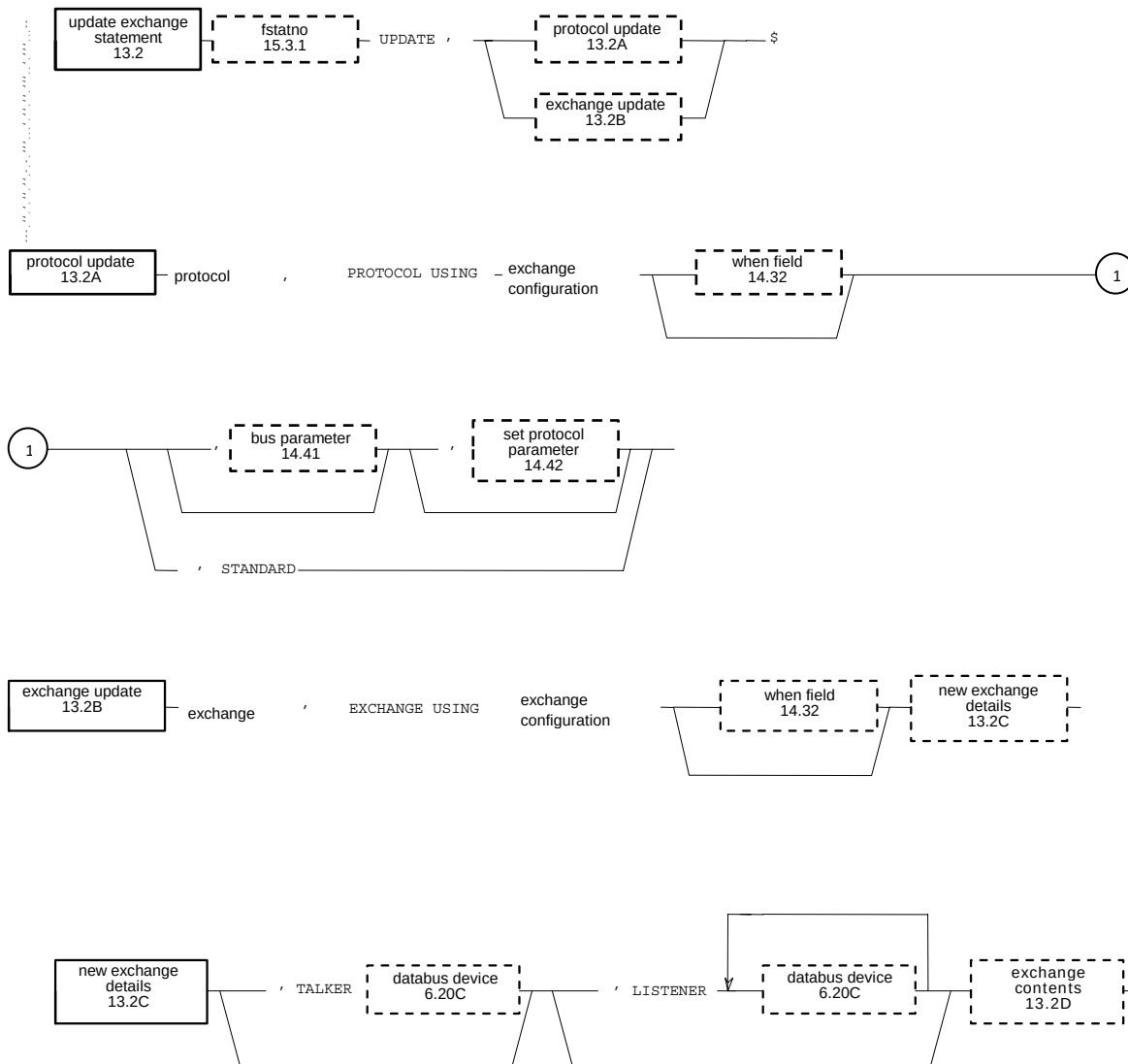
```

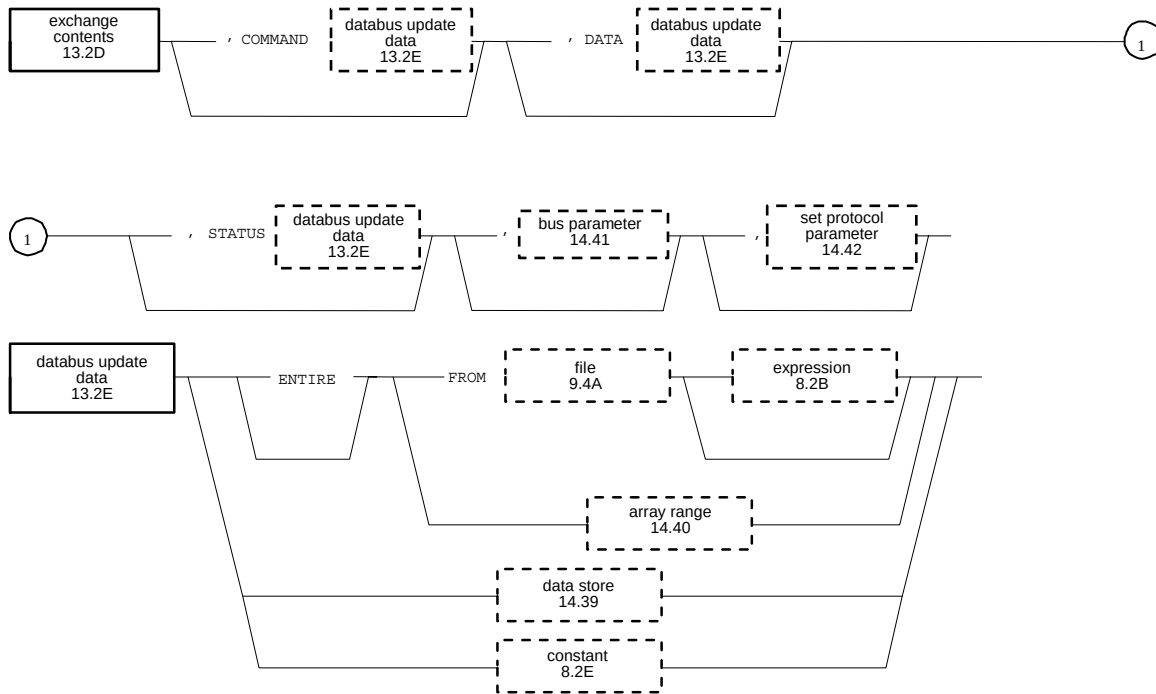
13.0 DATABUS STATEMENTS**13.2 <update exchange statement>****13.2.1 Function**

To change the <bus parameters> or <protocol parameter> values without interrupting databus operation. It can operate at the <protocol> default level or the level of the individual <exchange>. The statement can also be used to update COMMAND, DATA or STATUS words which are sourced by the test equipment at the individual <exchange> level.

13.2.2 Formal Syntax

FSD / 2313

13.2.3 Syntax Diagram

13.0 DATABUS STATEMENTS**13.2.4 Rules****13.2.4.1 Rules Applicable to an <update exchange statement> which references an <exchange> label**

1. The <exchange configuration> must be the same as that employed in the associated <do exchange statement>.
2. The <exchange> label must be one that was defined previously in a <define exchange statement>. It must be currently active following a reference in a <do exchange statement> with a PROCEED field.
3. Only those keywords (COMMAND, DATA, STATUS, TALKER, LISTENER, BUS-PARAMETER or PROTOCOL-PARAMETER) which occurred in the <define exchange statement> may appear in the <update exchange statement>. BUS-PARAMETER and PROTOCOL-PARAMETER may only occur if the corresponding BUS-PARAMETER or PROTOCOL-PARAMETER field entry in the <establish protocol statement> was defined as UPDATABLE at the exchange level. In this case only those fields which were so identified may occur in the <update exchange statement>. They are then modified to the replacement value for the referenced <exchange> only.
4. If an <update exchange statement> bypasses a keyword which was included in the <define exchange statement> its associated variables remain unchanged.
5. A bus word or parameter changed in an <update exchange statement> retains its new value for all subsequent bus transactions until another <update exchange statement> references it or the execution of the <do exchange statement> is completed.
6. Execution of an <update exchange statement> will employ the new values in the next regularly scheduled relevant <databus transaction> as defined in the <do exchange statement>.

13.0 DATABUS STATEMENTS

7. Entries in the COMMAND, DATA and STATUS fields must be in accordance with the <type> definition in the associated <establish protocol statement>.
8. For COMMAND, DATA and STATUS fields, if the <array range> is used with the keyword ENTIRE, all the specified words in the <array range> are sent every time the exchange is invoked. If the keyword ENTIRE is not used, consecutive transmissions of the specific <exchange> will take their values from successive elements of the array on an auto-incrementing basis.

13.2.4.2 Rules Applicable to an <update exchange statement> which references a <protocol> label

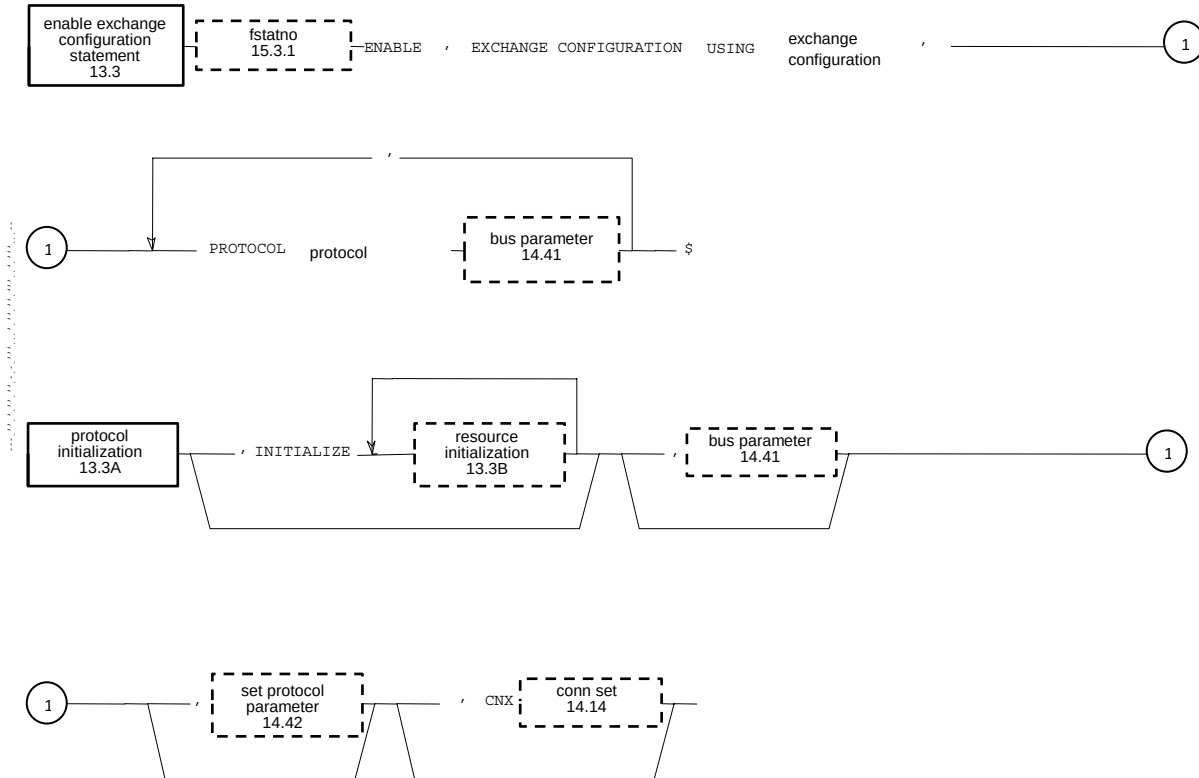
1. The <exchange configuration> must have been made active by a preceding <enable exchange configuration statement> and <do exchange statement> with a PROCEED field.
2. BUS-PARAMETER and PROTOCOL-PARAMETER may only occur if one or more BUS-PARAMETER or PROTOCOL-PARAMETER field entries respectively in the <establish protocol statement> were defined as UPDATABLE at the protocol level. In this case only those fields which were so identified may occur in the <update exchange statement>. They are then updated to the new value for all exchanges which did not contain a superseding value for the particular parameter specified in the corresponding <define exchange statement>.
3. The set of parameters listed in the <update exchange statement> will replace the equivalent parameters if they were previously established by the <enable exchange configuration statement>.
4. Bus data changed in the <update exchange statement> retains its new value for all subsequent bus transactions until another <update exchange statement> references it or the execution of the <do exchange statement> is completed.

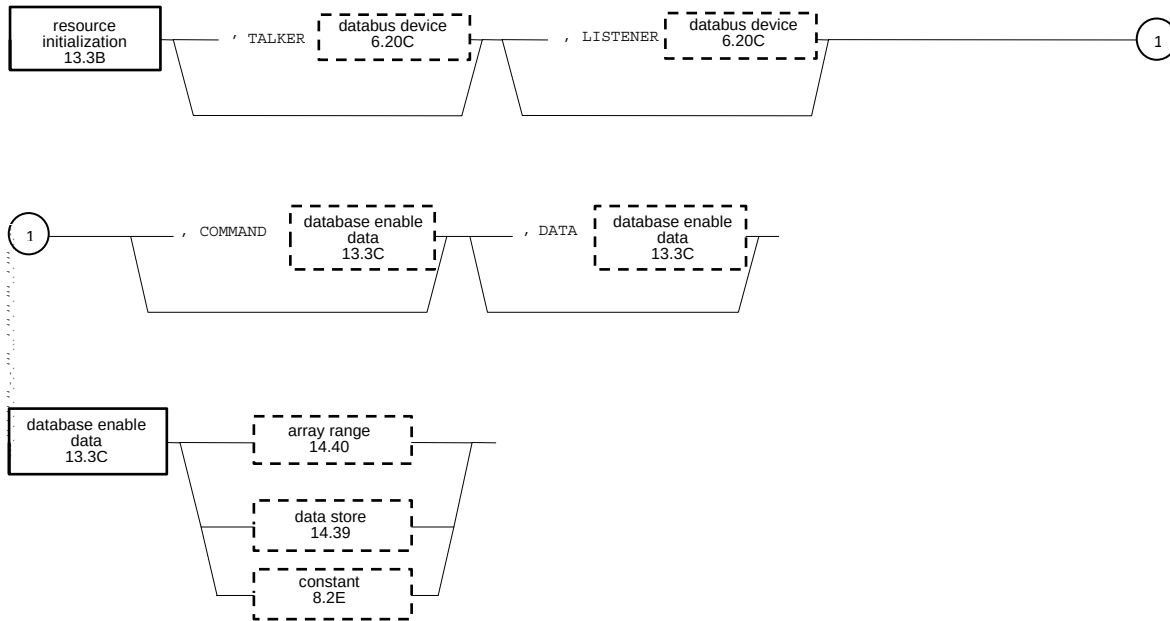
13.0 DATABUS STATEMENTS**13.3 <enable exchange configuration statement>****13.3.1 Function**

To allocate a set of databus virtual test resources to the initial condition for a subsequent <do exchange statement> and to ensure that they are connected to the correct UUT pins.

13.3.2 Formal Syntax

FSD / 2362

13.3.3 Syntax Diagram

13.0 DATABUS STATEMENTS**13.3.4 Rules**

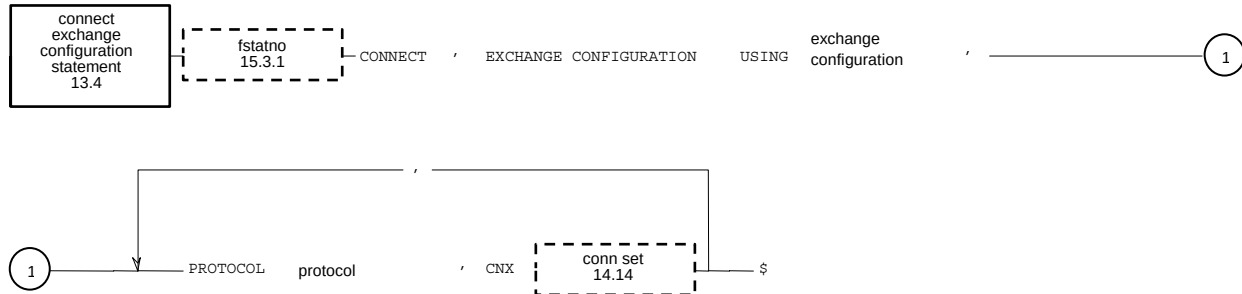
1. The <exchange configuration> must have been defined in a <define exchange configuration statement>. It must not be currently active as the result of a previous <enable exchange configuration statement> or a previous <do exchange statement>.
2. If a specific bus parameter is not specified for a particular <protocol> in the <enable exchange configuration statement>, the databus is initialized with standard bus parameters for that parameter.
3. If a specific protocol parameter is not specified for a particular <protocol> in the <enable exchange configuration statement>, the databus is initialized with standard <protocol parameters> for that parameter.
4. Each referenced <protocol> may occur only once in the statement and must be included in the referenced <exchange configuration>.
5. Any <bus parameter> that is specified must be defined in the corresponding <establish protocol statement>. The <dim>, data <type> and data value must be consistent with the <bus parameter> definition in the <establish protocol statement>.
6. Any <protocol parameter> that is specified must be defined in the corresponding <establish protocol statement>. The usage of the <protocol parameter> must be consistent with the usage in the <establish protocol statement>.
7. The CNX field, if used, must be repeated from the <establish protocol statement>. Each <pin descriptor> must have appeared in the CNX field of the <establish statement>.

13.0 DATABUS STATEMENTS13.4 <connect exchange configuration statement>13.4.1 Function

To connect an active databus to a set of UUT pins.

13.4.2 Formal Syntax

FSD / 2390

13.4.3 Syntax Diagram13.4.4 Rules

1. A <connect exchange configuration statement> must reference an active <exchange configuration>.

13.4.5 Notes and Examples

The <connect exchange configuration statement> was included to facilitate tests on a databus that has many potential connections to a UUT, and which requires the databus to be repeatedly connected and disconnected to the UUT ports.

13.0 DATABUS STATEMENTS

13.5 <disconnect exchange configuration statement>

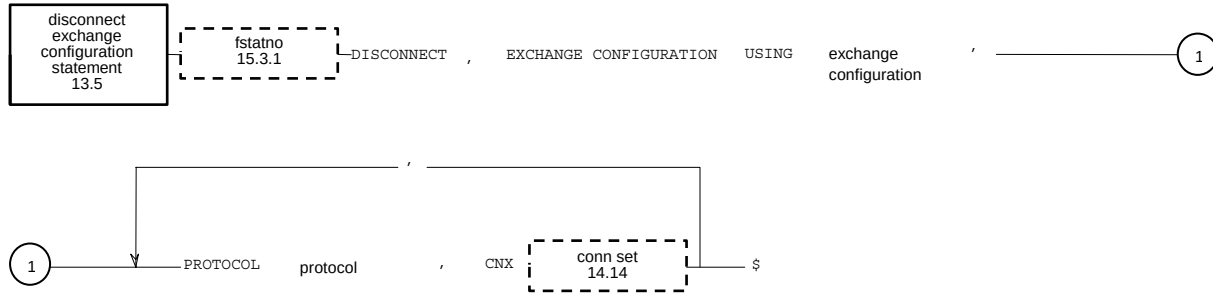
13.5.1 Function

To disconnect an active databus from a set of UUT pins.

13.5.2 Formal Syntax

FSD / 2402

13.5.3 Syntax Diagram



13.5.4 Rules

1. A <disconnect exchange configuration statement> must reference an active <exchange configuration> which is currently connected to at least one UUT port.

13.5.5 Notes and Examples

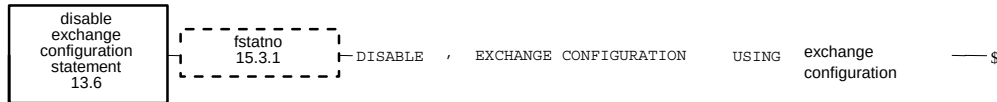
The <disconnect exchange configuration statement> was included to facilitate tests on a databus that has many potential connections to a UUT, and which requires the databus to be repeatedly connected and disconnected to the UUT ports.

13.0 DATABUS STATEMENTS13.6 <disable exchange configuration statement>13.6.1 Function

To render an active <exchange configuration> inactive and available for re-activation by an <enable exchange configuration statement>.

13.6.2 Formal Syntax

FSD / 2414

13.6.3 Syntax Diagram13.6.4 Rules

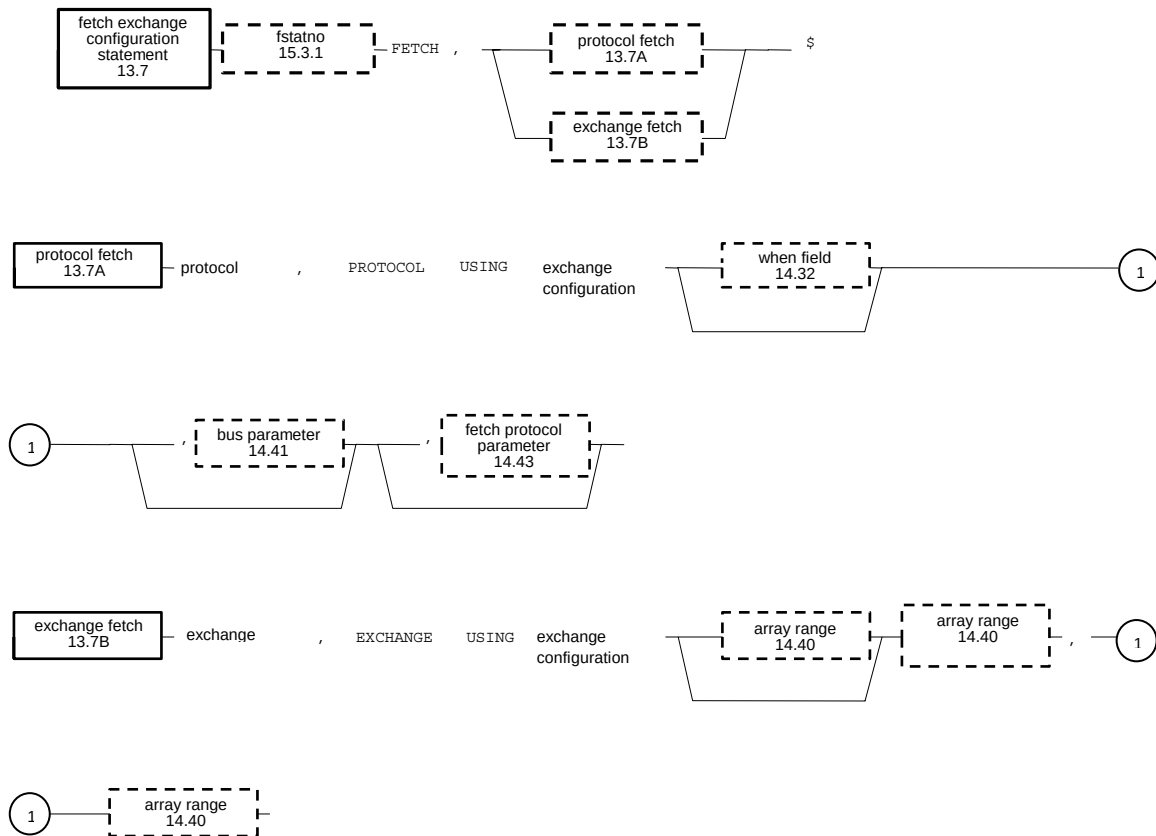
1. The <exchange configuration> must be currently active.
2. The <exchange configuration> will be de-activated as a result of the <disable exchange configuration statement>.
3. Once a <protocol> has been referenced by an <enable exchange configuration statement> it cannot be made active for use in a different <exchange configuration> until the original <exchange configuration> has been rendered inactive by a <disable exchange configuration statement>.

13.0 DATABUS STATEMENTS**13.7 <fetch exchange configuration statement>****13.7.1 Function**

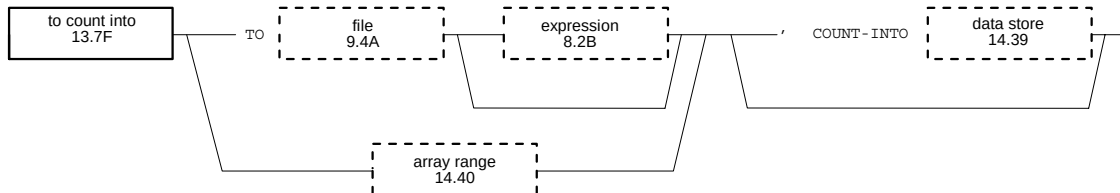
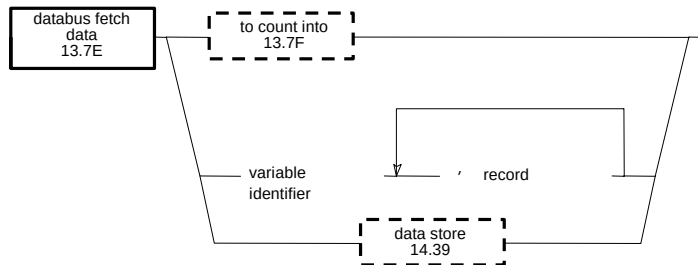
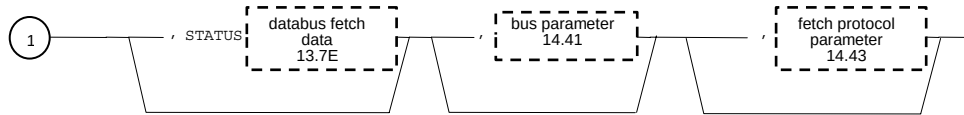
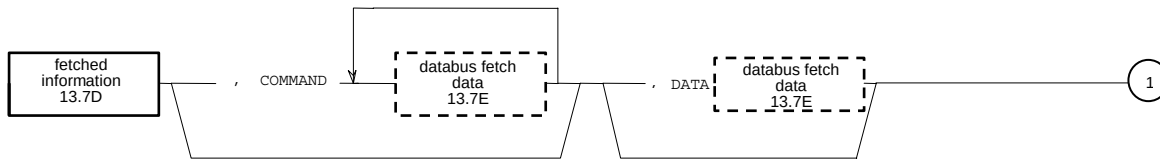
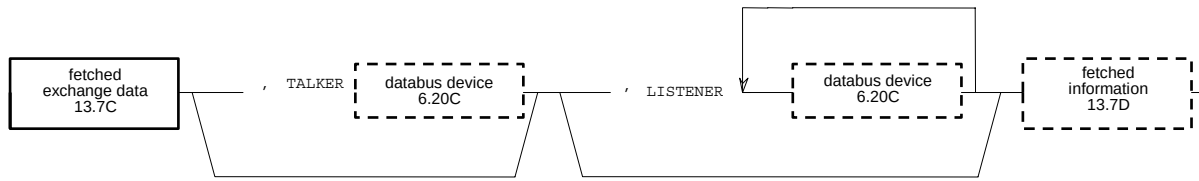
To fetch data that has been monitored by the test equipment as a result of a previous <do exchange statement> with the PROCEED option selected. FETCH transfers the monitored values into named variables that are specified in the corresponding fields in the <fetch exchange configuration statement>.

13.7.2 Formal Syntax

FSD / 2420

13.7.3 Syntax Diagram

13.0 DATABUS STATEMENTS



13.0 DATABUS STATEMENTS

13.7.4 Rules

13.7.4.1 Rules Applicable to a <fetch exchange configuration statement> which references an <exchange> label

1. The <exchange configuration> must have been made active by a preceding <enable exchange configuration statement>.
2. The <exchange> label must be one that was defined previously in a <define exchange statement>. It must be currently active following a reference in a <do exchange statement> with a PROCEED field.
3. Only those fields specified as FETCHABLE in the associated <establish protocol statement> may be fetched. For COMMAND, DATA and STATUS information, the corresponding TEST-EQUIP-MONITOR field must be specified in the <establish protocol statement>. For <bus parameter>s and <protocol parameter>s, each specified parameter to be read must be identified as FETCHABLE at the <exchange> level and be typed in the associated <establish protocol statement>.
4. DATA, STATUS or CONTROL information which is extracted by this statement and is not sourced by the test equipment must be identified in the corresponding TEST-EQUIP-MONITOR field for the <exchange> in the associated <do exchange statement>.
5. Since the associated data transfers may not have preceded the initiation of the <fetch exchange configuration statement>, the optional <max-time> field establishes a maximum delay waiting for their completion.

13.7.4.2 Rules Applicable to a <fetch exchange statement> which references a <protocol> label

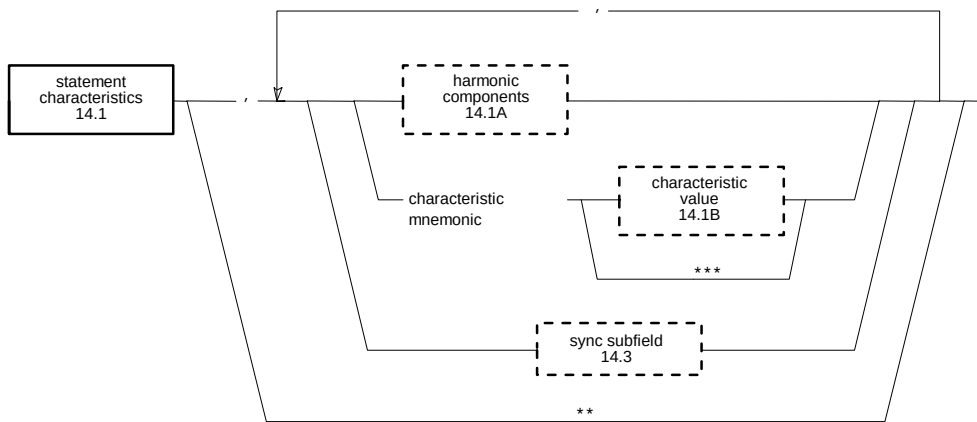
1. The <exchange configuration> must have been made active by a preceding <enable exchange configuration statement>.
2. Only those fields specified in the associated <establish protocol statement> may be fetched by the <fetch exchange statement>. Each specific <bus parameter> and <protocol parameter> to be read must be identified as FETCHABLE at the <protocol> level and must be <type>d in the associated <establish protocol statement>.

14.0 FIELD AND SUBFIELD DEFINITION**14.0 Field and Subfield Definition****14.1 <statement characteristics>****14.1.1 Function**

To specify the attributes of a signal. Listed with each noun in Chapter 16.0 is a list of the characteristics for which a specific value may be assigned. A separate <statement characteristics> field is required for each characteristic used, but it is not necessary to include every characteristic listed with the noun.

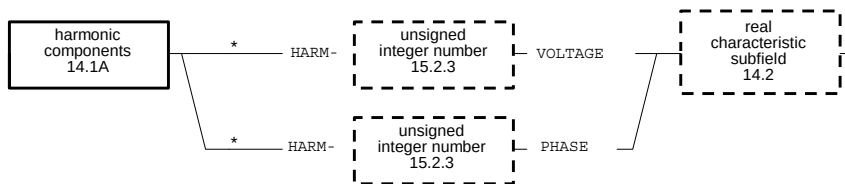
14.1.2 Formal Syntax

FSD / 2472

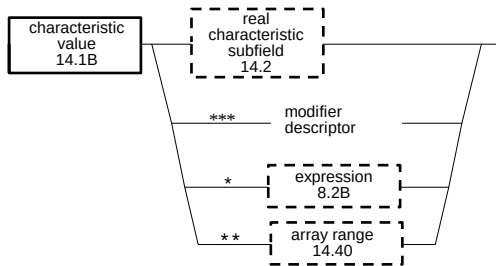
14.1.3 Syntax Diagram

Notes:

- ** This path is used only for the nouns COMMON, EARTH, and SHORT.
 *** This path is used only for modifiers of type MO as defined in Chapter 16.



Note: * This path is used only where permitted by the noun modifier lists in Chapter 16.

14.0 FIELD AND SUBFIELD DEFINITION**Notes:**

- * This path is used only for modifiers of type SB or SC as defined in Chapter 16.
- ** This path is used only for modifiers of type AB, AC, or RA as defined in Chapter 16.
- *** This path is used only for modifiers of type MD as defined in Chapter 16.

14.1.4 Rules

1. A <statement characteristics> field can only contain noun-modifiers as defined in Chapter 17 and selected from the listing for each noun as defined in Chapter 16 or a <sync subfield> in accordance with Section 14.3.
2. It is only necessary to use those noun-modifiers which are sufficient to define all the characteristic attributes of a noun as required for each particular ATLAS statement.
3. A separate <statement characteristics> field is required for each noun-modifier that is used to define a characteristic attribute of a noun.
4. In sensor statements, ranging information about the measured characteristic must be included in either a statement characteristic, including a MAX, MIN or RANGE subfield, or in an evaluation field containing constant upper and lower limits. Whenever <data store>s are used in the <evaluation field>, a <statement characteristics> field must be included to give ranging information since no range is given for the <measured characteristic> by the <evaluation field> in this case.
5. Number, Modifier Descriptor and Dimensional Units

A number or a <modifier descriptor> may be included to indicate the required value of the characteristic. The value of the characteristic may also be indicated by a mnemonic label which refers to a numeric value established by another statement in the procedure. A dimensional unit identifier appropriate to the mnemonic may be written following the number. The number or label may be preceded by MAX or MIN. If a modifier descriptor is used it must be appropriate for the modifier and be used in accordance with the rules for the modifier in Chapter 17 of this specification.
6. The <modifier descriptor>, <expression> and <array range> branches may only be used if the associated modifier is defined as a CONTROL modifier in the associated <require statement>. With the one exception that the <modifier descriptor> branch may be used with a CAPABILITY modifier provided that the <modifier descriptor> is identical to the corresponding field in the <require statement>.
7. MAX is the greatest positive numerical value.
MIN is the greatest negative numerical value.

14.0 FIELD AND SUBFIELD DEFINITION

14.1.5 Notes and Examples

1. The following example shows how to specify that a measuring device must not draw more than 0.1 MA from the UUT:

```
MEASURE, (VOLTAGE), DC SIGNAL USING 'DMM',  
VOLTAGE RANGE 0 V TO 20 V,  
CURRENT MAX 0.1 MA, .... $
```

Alternatively, the prohibition of implied loads can be illustrated by an illegal example:

```
MEASURE, (VOLTAGE), DC SIGNAL USING 'DMM',  
VOLTAGE RANGE 0 V TO 20 V,  
CURRENT 2.0 A, .... $
```

In this illegal example, the user is attempting to connect a load which draws 2 amperes when a DC voltage is measured. To do this legally, he would have to APPLY the load, then MEASURE the voltage.

14.0 FIELD AND SUBFIELD DEFINITION

14.2 <real characteristic subfield>

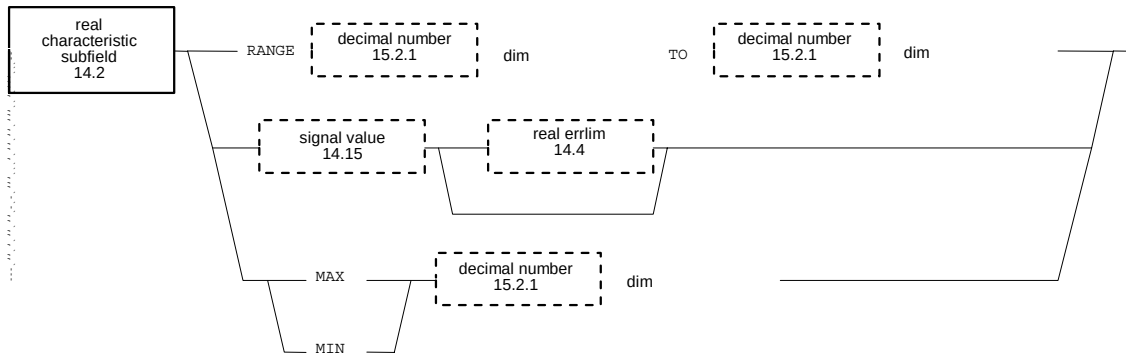
14.2.1 Function

To specify the value or range of values for a characteristic of the associated noun which can be expressed completely in INTEGER or DECIMAL numbers as defined by Chapter 16.

14.2.2 Formal Syntax

FSD / 2517

14.2.3 Syntax Diagram



14.2.4 Rules

1. All dimensions used in a <real characteristic subfield> shall be identical.
2. In sensor-type statements the following rules apply:
 - a. The measured characteristic, except where explicitly excluded in Chapter 16 or 17, must be repeated in the <statement characteristics> field followed by one of MAX , MIN or RANGE branches of the diagram.
 - b. In a <verify statement>, the dimensional units used in the units specified in the <real characteristic subfield> that is used to range <measured characteristic> values must be identical to those in the <evaluation field>.
3. MIN, MAX or RANGE as used in a source or load statement defines the stimulus or load parameter range that is required by the UUT. MIN, MAX or RANGE as used in a sensor statement defines the parameter range variation that is expected from the UUT at the time the sensor statement is executed.
4. The <signal value> branch of the <real characteristic subfield> may only be used if the associated modifier is defined as a CONTROL modifier in the associated <require statement>. The RANGE, MAX and MIN branches may only be used if the associated modifier is defined as a CAPABILITY or LIMIT modifier in the associated <require statement>.

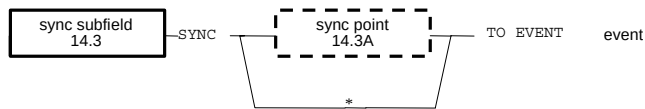
14.0 FIELD AND SUBFIELD DEFINITION**14.3 <sync subfield>****14.3.1 Function**

To indicate that a waveform is synchronized to an <event> which in turn may be related to another signal.

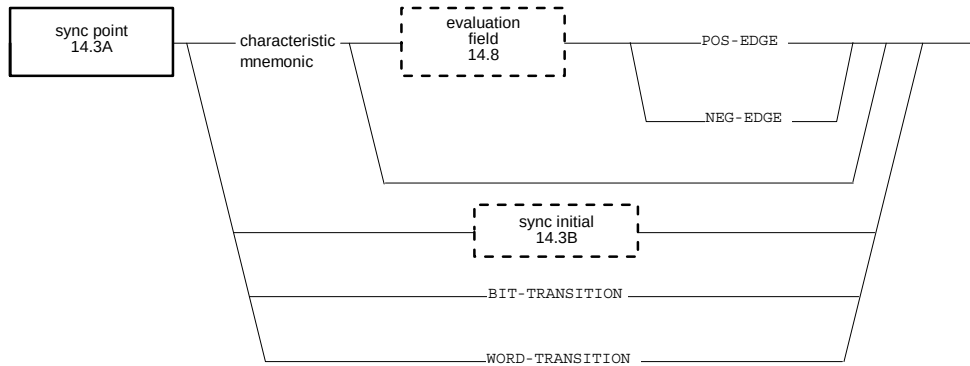
The <sync subfield> can specify different forms of synchronization namely; frequency reference synchronization, initial coincidence synchronization and digital data transfer synchronization.

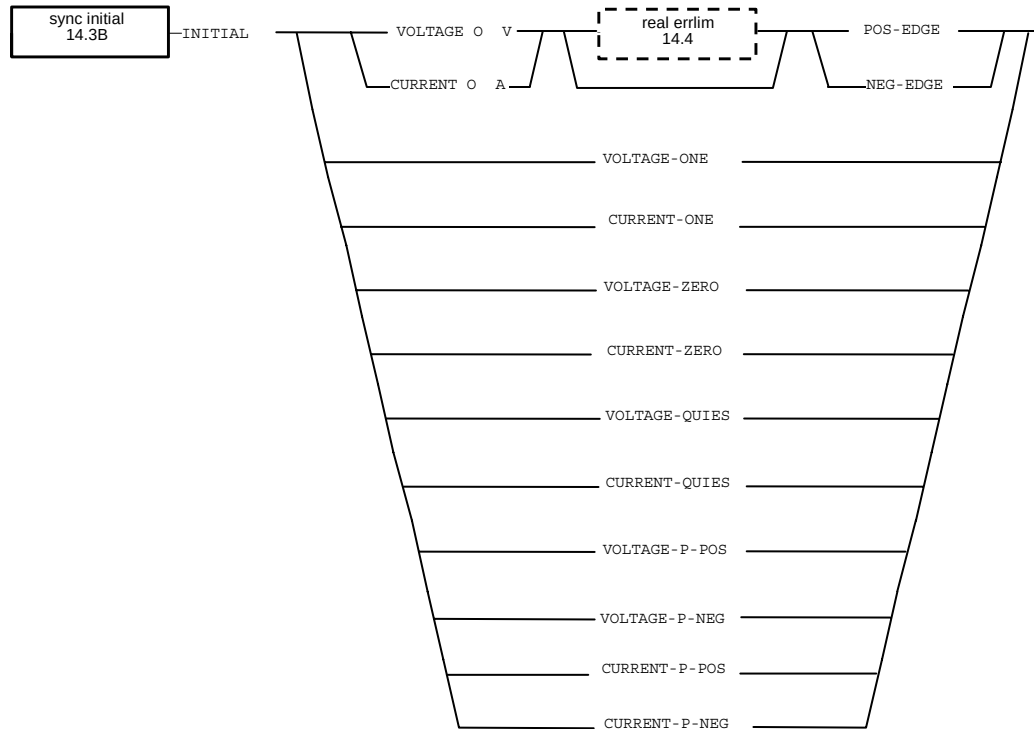
14.3.2 Formal Syntax

FSD / 2528

14.3.3 Syntax Diagram

Note: * This path is used with the noun WAVEFORM and with other nouns when the sync subfield is used to synchronize the ARM action of a sensor statement. This path may be used only once in a statement.



14.0 FIELD AND SUBFIELD DEFINITION**14.3.4 Rules**

1. Frequency reference synchronization specifies that a signal that is defined by a statement which includes a <sync subfield> is in phase-locked synchronization with a frequency defined by a repetitive event referenced by an <event> name. The noun-modifiers used for frequency reference synchronization are **FREQ**, **CAR-FREQ** and **MOD-FREQ**.
2. Initial coincidence synchronization covers all other general forms of analog signal synchronization covered. For these forms, a statement defines a point on a signal which is to be the first change from a quiescent value at a point in time defined by an <event> after which the signal continues as defined without further interference from the <event>.
3. Digital data transfer synchronization specifies that a digital transfer whose definition includes a <sync subfield> for the data <bit/rate> or <word/rate> will initiate each bit or word transfer synchronized to the reference point defined by an <event>.
4. Synchronization occurs at the coincidence of specific points on the synchronized signal and either the stimulus or the response signal defined by the statement which contains the **SYNC** modifier, and the reference signal or event; these are identified respectively as the synchronized point and the reference point.
5. The reference signal or event and its reference point condition are described separately and referenced in the **SYNC** subfield by **TO EVENT**, where an identified <event> is used for a reference point.

14.0 FIELD AND SUBFIELD DEFINITION

6. The synchronized point of most analog stimulus or response signals is defined by an <evaluation field> with POS-EDGE or NEG-EDGE, where the specific value and direction of a signal characteristic is identified as the synchronized point.
7. The synchronized point of specific VOLTAGE or CURRENT waveforms can be defined as having an INITIAL value which is the value of the synchronized signal at the instant of synchronization. There are three forms of INITIAL value synchronization:
 - a. The point at which a digital logic signal starts a specific logic state, either the logic 0, 1 or quiescent state.
 - b. The point at which the waveform crosses the zero value either in a positive or a negative direction.
 - c. The point at which the waveform is at a positive or negative peak value.
8. The synchronized point of a frequency synchronized signal is the periodic coincidence of a characteristic frequency of the signal to a reference signal having the same frequency and with a constant phase difference between the reference signal and the synchronized signal.
 - a. Three forms of SYNC are available for this purpose using the following characteristic mnemonics in accordance with Rule 1:
 - (i) FREQ; used for simple non-modulated signals with only one characteristic frequency.
 - (ii) CAR-FREQ; used to synchronize a signal carrier frequency to a reference signal.
 - (iii) MOD-FREQ; used to synchronize the modulation frequency of a signal to a reference.
 - b. If a frequency is stated implicitly by reference to a referenced signal which is at the required frequency, that frequency must not be stated explicitly in the synchronized signal characteristics.
 - c. The reference signal or event must be repetitive and may be specified as either an <event> or a SOURCE signal which defines a periodic reference point.
9. The synchronized point of digital data transfers is defined as the point at which to initiate each bit or word transfer synchronized to the reference point(s).
 - a. BIT-TRANSITION is used in the general case for digital data, such as SERIAL, to specify that the start of each bit position is the synchronized point.
 - b. WORD-TRANSITION is used for digital data to specify that the start of each data word is the synchronized point. WORD-TRANSITION is used primarily when a SERIAL data transfer of multiple words is to be synchronized to two separate reference signals, one defining the SERIAL bit sync (SYNC BIT-TRANSITION) and one defining the SERIAL word sync (SYNC WORD-TRANSITION). PARALLEL data transfers of multiple words may use either BIT-TRANSITION or WORD-TRANSITION to specify the same synchronized point.
10. The synchronized point of the signal WAVEFORM is the initial value of the waveform. For SOURCE signals, the initial value that is synchronized is the first value specified in the SAMPLE <array range> of waveform amplitudes. For SENSOR signals, the initial value is the first value sampled at the reference point.
11. When the SYNC subfield is used in a SOURCE or SENSOR signal statement to reference a UUT signal or <event>, the optional MAX-TIME field of that statement should be used to specify an escape mechanism in case the reference signal or <event> fails to occur; for example, due to a UUT failure.

14.0 FIELD AND SUBFIELD DEFINITION**14.4 <real errlim> (Real Error Limit)****14.4.1 Function**

To specify the acceptable accuracy limits of a parameter of a signal that is to be applied to the UUT or the uncertainty of the signal parameter that a sensor must tolerate but not measure.

A <decimal number> or <floating point decimal number> is included to indicate the required accuracy of the test equipment stimulus, load, or measurement function, as observed at the UUT terminals. The accuracy subfield begins with the word ERRLMT and is followed by an expression of the allowable algebraic deviation. The following are the acceptable formats for accuracy expressions. x, x1, x2, y, etc., are decimal or floating point decimal values. A <dim> is the same dimension identifier from Section 15.8 as was used for the basic characteristic except that PC, which in Section 15.8 refers to a ratio, is not allowed as one of the <dim> options. PC can be used in place of <dim> to indicate an accuracy requirement which is a calculated percentage of the desired characteristic value or measured value (see Section 14.4.5). The + value is written first in the sample formats, but the order of writing + and - values may be reversed in an actual statement.

14.4.1.1 Asymmetrical Form

When the limits are asymmetrical the following notations may be used:

ERRLMT + x dim - y dim
ERRLMT + x PC - y PC

14.4.1.2 Symmetrical Form

When the limits are symmetrical, the following notations may be used:

ERRLMT +- x dim
ERRLMT +- x PC

14.4.1.3 Combination Form

When it is necessary to express the accuracy requirement as a combination of an absolute deviation and a percentage calculation, one of the connecting words AND or OR separates the two expressions. AND indicates that the percentage deviation and the absolute deviation are added in determining the total allowable deviation. OR indicates that the greater of the two (percentage or absolute) determines the allowable deviation. For example:

For asymmetrical limits:

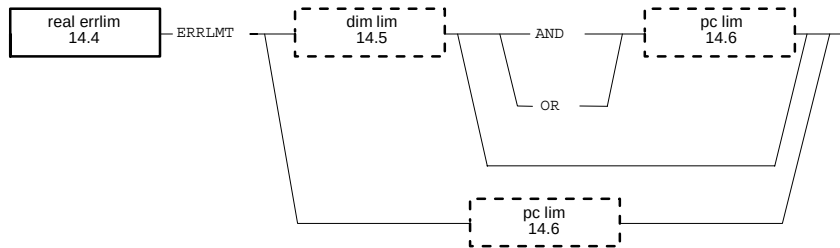
ERRLMT + x1 dim - x2 dim OR + y1 PC - y2 PC
ERRLMT + x1 dim - x2 dim AND + y1 PC - y2 PC

For symmetrical limits:

ERRLMT + - x dim OR + - y PC
ERRLMT + - x dim AND + - y PC

14.0 FIELD AND SUBFIELD DEFINITION14.4.2 Formal Syntax

FSD / 2580

14.4.3 Syntax Diagram14.4.4 Rules

1. When no <real errlim> subfield is included in a <statement characteristics> field, no special control is exercised over the test system accuracy for this statement. The specified value of a characteristic is established by the inherent accuracy of the test system.
2. All accuracy requirements should be stated in terms of actual UUT needs as measured at the UUT terminals and should not take into account any test equipment errors.

14.4.5 Notes and Examples

1. Note that PC as used in the accuracy subfield has a different connotation than PC as the absolute dimension of some characteristic such as OVERSHOOT of PULSED AC. In the accuracy subfield, PC describes a calculated percentage of the described value. VOLTAGE 10 V ERRLMT + 1 V - 0.5 V means that values of voltage from 9.5V to 11 V are acceptable. The same applies to characteristics with PC as the basic dimension. Taking the OVERSHOOT example, OVERSHOOT 10 PC ERRLMT + - 5 PC means that values from 9.5 PC to 10.5 PC are acceptable. This does not mean that 5 through 15 percent are the acceptable values.
2. Examples of the Combination Form

The following are examples of the Combination Form.

VOLTAGE () V ERRLMT + 0.005 V - 0.003 V AND + 2 PC - 1 PC

This example using the AND connective means that if 50 was substituted for the () in the expression the highest acceptable value for the voltage would be $50 + 0.005 + 2 \text{ percent } (50) = 51.005 \text{ V}$. The lowest acceptable value is $50 - 0.003 - 1 \text{ percent } (50) = 49.497 \text{ V}$.

CURRENT () MA ERRLMT + - 1 MA OR + - 2 PC

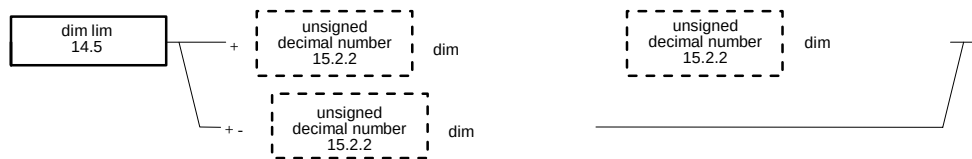
This example using the OR connector means that if 50 was substituted for the (), the + - 1 MA is greater than 2 PC of the signal and will determine the acceptable values as being 49MA to 51MA. For values of CURRENT greater than 50 MA, the 2 PC expression is greater than 0.1 MA and therefore determines the acceptable values. If 25 was substituted for the parentheses, the acceptable values would be 24 MA to 26 MA. If 80 was substituted, the acceptable values would be 78.4 MA to 81.6 MA.

14.0 FIELD AND SUBFIELD DEFINITION**14.5 <dim lim> (Dimension Limit)****14.5.1 Function**

To express the dimensional limits for the <real errlim> of a signal parameter, in terms of positive and negative real numbers, together with an authorized dimension.

14.5.2 Formal Syntax

FSD / 2592

14.5.3 Syntax Diagram**14.5.4 Rules**

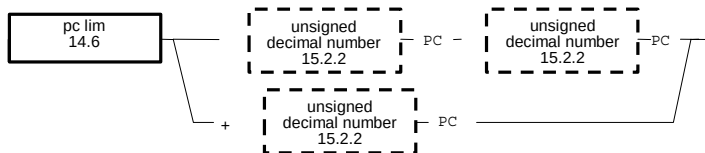
1. <dim> may not be PC.

14.6 <pc lim> (Percent Limit)**14.6.1 Function**

To express the dimensional limits for the <real errlim> of a signal parameter in terms of positive and negative real numbers which are percentage ratios of the nominal signal value.

14.6.2 Formal Syntax

FSD / 2600

14.6.3 Syntax Diagram

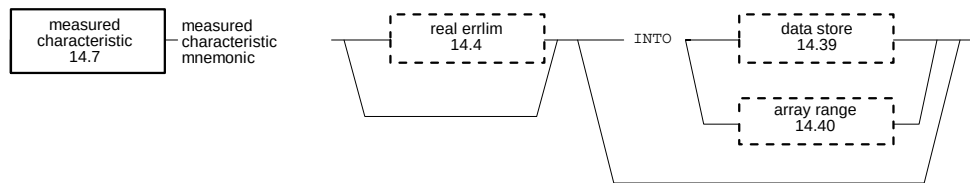
14.0 FIELD AND SUBFIELD DEFINITION**14.7 <measured characteristic>****14.7.1 Function**

To specify which signal characteristic is to be measured.

One or more <measured characteristic> fields are included in sensor type statements to specify which of the available characteristics (i.e. modifier mnemonics listed with the noun in paragraph 16) are to be evaluated by the sensor functions. The optional error limit subfield which follows the <measured characteristic mnemonic> subfield indicates the required accuracy to which the measurement must be taken. The last subfield consists of the connective INTO followed by a <data store> that is to receive the measured value(s). The INTO subfield is required if the sensor statement specifies multiple <measured characteristic>s or a <measured characteristic> that returns multiple measured values. Otherwise, the measured value is always stored in 'MEASUREMENT' and use of the INTO subfield is optional. Multiple <measured characteristic>s are used to indicate that a single FETCH operation will return the values of all of these characteristics from a data set captured by a single measurement.

14.7.2 Formal Syntax

FSD / 2608

14.7.3 Syntax Diagram**14.7.4 Rules****1. Ranging for the Measured Characteristic**

The explicit or implicit measured characteristic must be repeated in the statement characteristics followed by a MAX, MIN or RANGE subfield except for those measured characteristics specifically defined as not requiring this repetition by Chapter 16 or 17. The MAX, MIN, or RANGE subfield specifies the possible range that the measured characteristic can span. The intent is to provide all information necessary for the user to choose a device and then scale it to perform the measurement. This ranging pertains to all sensor-type statements (MEASURE, VERIFY, etc.).

The MAX, MIN or RANGE subfield, the evaluation field and the measured characteristic error limit, if included, must contain the same dimensional units.

2. Scale of 'MEASUREMENT'

Following the execution of the sensor-type statement, the value in 'MEASUREMENT' and the <data store> which is referenced in the <measured characteristic> field, if any, will be at the scale implied by the dimensional units used in the MAX or MIN subfield or the RANGE subfield.

3. For all analog measurements, each sensor-type statement must define, either explicitly or implicitly, the range of values to be covered by the measured characteristic.

14.0 FIELD AND SUBFIELD DEFINITION

4. The measured value storage option INTO <array range> may be used only with those measurement functions that return more than one measured value, such as the noun WAVEFORM. It may only be used with a <read statement>, a <measure statement> and the single action verb statements.
 - a. The first measured value is placed in the element position specified by the first integer in the <array range>. Each subsequent measured value is placed in the next element position as specified by the <array range>.
 - b. No value is stored in 'MEASUREMENT'. (See Section 15.6.2)
5. When a statement contains more than one <measured characteristic> field, each <measured characteristic> must utilize an INTO option and 'MEASUREMENT' is not affected.

14.7.5 Notes and Examples

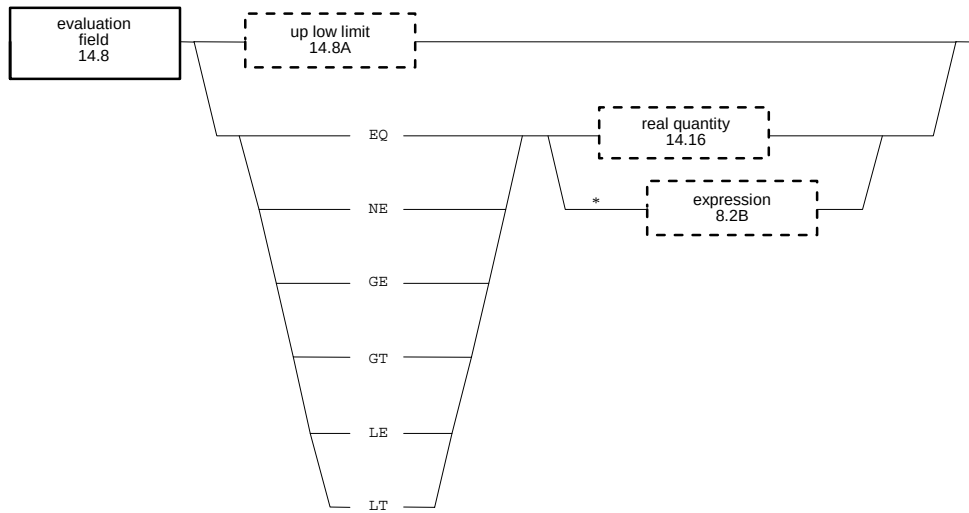
1. In the example MEASURE, (FREQ), AC SIGNAL, the general type of signal is an ac signal and the characteristic which will be measured is its frequency.
2. Examples of Scale of 'MEASUREMENT'
 - a. MEASURE, (VOLTAGE . . . VOLTAGE-MAX 30 MV, . . .
 If the value of the voltage is 0.012 volts, the value in 'MEASUREMENT' will be 12.
 - b. VERIFY, (FREQ) , AC SIGNAL, . . . UL 37.5 KHZ LL 33 KHZ, . . .
 If the value of the frequency is 36,240 Hertz, the value in 'MEASUREMENT' will be 36.24.
 - c. VERIFY, (FREQ) , AC SIGNAL, FREQ MAX 40000 HZ, . . . UL 37.5 KHZ
 LL 33
 KHZ . . .
 This statement is incorrect since the dimension in the MAX subfield is different than the dimension given in the evaluation field.

14.0 FIELD AND SUBFIELD DEFINITION14.8 <evaluation field>14.8.1 Function

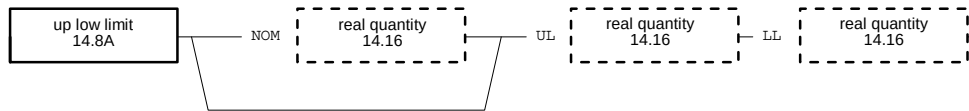
To establish the limits to be used in the processing of an evaluation. Depending on usage of the <evaluation field>, one or more conditions may be established which may be acted on by program control statements. An <evaluation field> is included in statements such as COMPARE and VERIFY.

14.8.2 Formal Syntax

FSD / 2621

14.8.3 Syntax Diagram

Note: * This path applies only to EQ and NE.



14.0 FIELD AND SUBFIELD DEFINITION**14.8.4 Rules****1. <evaluation field> relationships**

The table shown in Figure 14-1 lists the possible formats for <evaluation field> in the first column. The next column lists the possible relationships between a program <data store> (v) and the numbers in the <evaluation field> (x, y and z). The last column lists the condition which is established by each relationship in the middle column.

The following abbreviations are used in the evaluation field:

UL - upper limit	EQ - equal to
GT - greater than	GE - greater than or equal to
LL - lower limit	NE - not equal to
LT - less than	LE - less than or equal to
NOM - nominal value	

As the words imply, algebraic magnitude determines which limit is identified as UL and which is LL. (Example: NOM -5 V UL -4 V LL -6 V.) There is an exception for mechanical angles around 0 DEG, however. If negative angles are used, the algebraic magnitude convention is not violated (NOM 0 DEG UL 2 DEG LL -2 DEG); but in some systems, such as compass indicators, it is important to refer to 1 DEG minus 2 DEG as 359 DEG. Therefore, an exception to the algebraic magnitude convention in evaluation limits is made.

The <evaluation field> for mechanical angles with a pass-band around 0 degrees may use either a negative angle as the LL or the negative angle plus 360 DEG. Thus, for mechanical angles around zero degrees:

NOM 1 DEG UL 3 DEG LL -2 DEG

is the same as the equally acceptable:

NOM 1 DEG UL 3 DEG LL 358 DEG

14.0 FIELD AND SUBFIELD DEFINITION

Table 14-1 Evaluation Field Relationships

Evaluation Field Format	Value of Variable (v)	Conditions Established
UL y <dim> LL z <dim>	$v > y$ $y \geq v \geq z$ $z > v$	HI & NOGO GO LO & NOGO
GT x <dim>	$v > x$ $v \leq x$	GO LO & NOGO
LT x <dim>	$v < x$ $v \geq x$	GO HI & NOGO
EQ x <dim>	$v = x$ $v \neq x$	GO NOGO
NE x <dim>	$v \neq x$ $v = x$	GO NOGO
GE x <dim>	$v \geq x$ $v < x$	GO NOGO & LO
LE x <dim>	$v \leq x$ $v > x$	GO NOGO & HI

2. Dimensional Unit Restrictions

All dimensional units appearing in an <evaluation field> must be identical. For example, if V appears with NOM, V must also appear with UL and LL, not MV or KV. In addition, if the <evaluation field> appears in a sensor-type statement, and if a maximum or minimum value or a range is given in the <statement characteristics> for the <measured characteristic>, the dimensional unit in the MAX, MIN, or RANGE subfield must be identical to the dimensional units in the <evaluation field>.

3. In sensor statements, ranging information about the measured characteristic must be included in either a statement characteristic (including a MAX, MIN or RANGE subfield) or in an evaluation field. Whenever <data store>s are used in the <evaluation field>, a statement characteristic must be included to give ranging information; since no range is given for the <measured characteristic> by the <evaluation field> in this case.
4. The <expression> must evaluate to a <digital number>.

14.0 FIELD AND SUBFIELD DEFINITION**14.8.5 Notes and Examples**

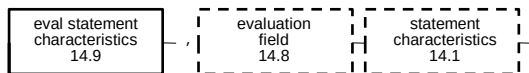
There is an infinite continuum of real numbers. Any digital computer has a finite number of different binary patterns to represent this series of real numbers. Thus a group of nearly equal real numbers will have the same binary representation, and, as far as the computer is concerned, are equal. Unfortunately different computers treat different partially overlapping groups as identical. Thus two real numbers which are considered to be equal by one computer will be unequal in the opinion of another. Furthermore, numbers differing typically by 1 part in 10^8 are unequal. Because of these possible anomalies it is recommended that EQ and NE not be used with decimal numbers.

14.9 <eval statement characteristics> (evaluation statement characteristics)**14.9.1 Function**

To define the set of fields between the <noun field> and the <conn> field in a sensor statement that includes an evaluation of the sensed characteristic.

14.9.2 Formal Syntax

FSD / 2636

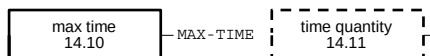
14.9.3 Syntax Diagram**14.10 <max time>****14.10.1 Function**

To specify a time limit within which a test function must occur.

In those statements whose syntax specifies that a <max time> field applies, there is a possibility that the test system could be given an impossible requirement such as waiting for a UUT measurement which will never occur because of a defective UUT. If the test system does not successfully execute the required operation within the specified time, the test system will establish a MAX-TIME condition which can be tested and acted upon by subsequent statements.

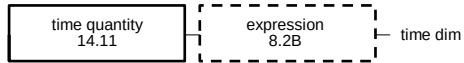
14.10.2 Formal Syntax

FSD / 2641

14.10.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION14.11 <time quantity>14.11.1 Function14.11.2 Formal Syntax

FSD / 2645

14.11.3 Syntax Diagram14.11.4 Rules

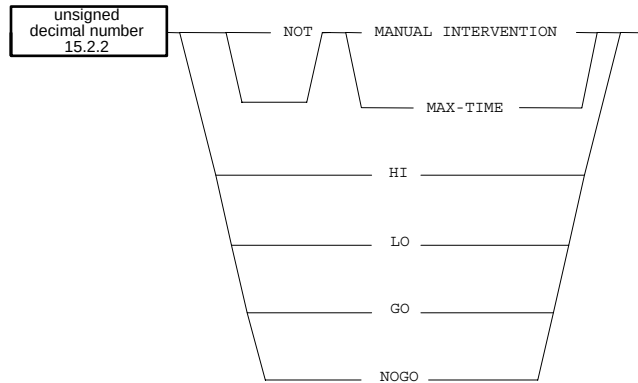
1. The <expression> must evaluate to an <unsigned decimal number> or to an <unsigned integer number>.

14.12 <condition>14.12.1 Function

To define a conditional program execution based upon the current status of the GO, NOGO, HI, LO, MAX-TIME or MANUAL-INTERVENTION conditions.

14.12.2 Formal Syntax

FSD / 2649

14.12.3 Syntax Diagram14.12.4 Rules

1. The type of <condition> is BOOLEAN.

14.0 FIELD AND SUBFIELD DEFINITION

14.13 <conn> (Connection Field)

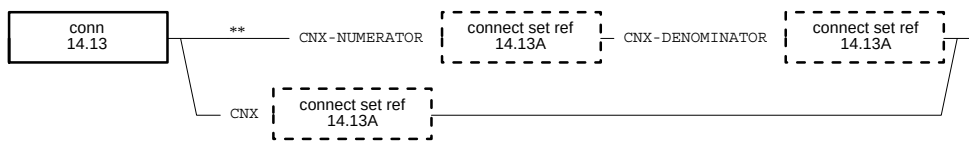
14.13.1 Function

To specify the UUT points to which the described function is to be attached. Generally, connector designations assigned by the UUT manufacturer are used to describe the attachment points (such as J1-2, P2-AA for electrical connectors). Most functions (such as DC signals and synchro signals) have multiple leads which must be attached to the correct UUT points. The pin descriptors (such as HI and LO or X, Y and Z and so on) are provided in the language to identify the UUT pin to which each lead is to be attached. <conn> is the last field in source, sensor and load statements.

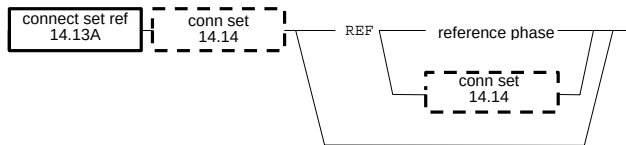
14.13.2 Formal Syntax

FSD / 2663

14.13.3 Syntax Diagram



Note: ** This path is used only with the suffix RATIO.



14.13.4 Rules

1. A second set of connectors may be included in the <conn> field preceded by the word REF. This capability is used for signals that must be defined relative to some reference signal, such as a phase-sensitive or a time-related function. The stated UUT points define the location of the reference signal. When the reference signal is one phase of a multi-phase source that is wholly internal to the test equipment and therefore cannot be identified by UUT connections, it may be identified in the CNX field by a <reference phase> identifier.

14.0 FIELD AND SUBFIELD DEFINITION14.14 <conn set> (Connection Set)14.14.1 Function

To specify the set of connections to be made to a UUT or a special test circuit which is described by a <define drawing statement>. It may also specify a <reference phase> that is not directly accessible at the UUT connection interface.

The following includes all the words and characters which are used in the connection field except the actual pin names.

14.14.1.1 <pin descriptor>

A <pin descriptor> must be one of the permanent lists shown below or a <pin descriptor name> introduced in an <extend atlas connection statement>.

HI <pin descriptor> denoting “hot” side of a two-wire circuit. The HI side has the same polarity as the magnitude field of the statement when measured with respect to the LO side. HI may also designate the side of the circuit which is most isolated (highest impedance) from the UUT ground reference.

LO <pin descriptor> denoting “cold” or return side of a two-wire circuit. The LO side is the reference for the function described in the statement. LO may also designate the side of the circuit which is least isolated (lowest impedance) from the UUT ground reference. (Note that the HI and LO connections conform to a different magnitude convention than do the UL and LL, LT and GT, IF HI and IF LO statement elements which are determined by algebraic magnitude. For example; to put a DC signal of -8 V with respect to -6 V on the UUT, -8 V goes with HI and -6 V goes with LO.)

VIA <pin descriptor> denoting:

1. a terminal for a series type connection, for example; for current or fluid flow.

For current flow, the source or sensor should be considered as being connected between the referenced UUT terminal(s) and all other then-existing connections to the referenced UUT terminal(s), without causing any temporary interruption to the flow. It should be noted that a previous statement should have provided a path for the return flow.

2. a port for the transmission of electro-magnetic signals, for example; for a wave guide.

For electro-magnetic waves, negative power-ratio quantities may be written (-8 dBm) as a statement characteristic. However, the negative sign should not be construed as being an indication of the direction of power transfer.

In all cases, flow to the UUT is arbitrarily designated positive.

SCREEN <pin descriptor> denoting a terminal on the UUT to which an electro-static shield around the signal conductors is to be connected.

GUARD <pin descriptor> denoting a terminal on the UUT to which the guard terminal, if any, of the sensor device is to be connected in order to reduce the effect of common mode voltages between the LO side of the UUT signal and the chassis ground of the sensor.

14.0 FIELD AND SUBFIELD DEFINITION

A,B,C,N <pin descriptor> set denoting phase connections for various phase configurations as shown in the following examples:

1. Single Phase - A, B, or C and N
2. Two Phase - A, B, and N
3. Three Phase Delta - A, B, C
4. Three Phase Wye - A, B, C and N

Neutral <pin descriptor> where used with A-B-C.

X,Y,Z <pin descriptor> set denoting three-wire Synchro stator connections or designating the axis of an orthogonal coordinate frame for mechanical rotation.

S1,S2,S3,S4 <pin descriptor> set for four-wire Resolver stator connections.

R1,R2,R3,R4 <pin descriptor> set for Synchro-Resolver rotor connections.

*X <pin descriptor> denoting “X” as being a terminal in a defined drawing. X may also be used to designate the axis of a coordinate reference frame used in conjunction with the ROTATION noun. (See Section 16.24)

TRUE <pin descriptor> denoting the <connection> that carries the “true” signal for a differential digital signal. Used in place of HI.

COMPL <pin descriptor> denoting the <connection> that carries the “complement” signal for a differential digital signal. For a three wire differential digital signal, TRUE, COMPL, and LO are used in place of HI. For a two wire signal, only HI and LO are used.

TO <pin descriptor> denoting non-electrical signals such as the noun AMBIENT CONDITIONS. It is used when only a single connection is required, irrespective of the flow to or from the UUT, such as a special connection (Section 14.14.1.3).

14.0 FIELD AND SUBFIELD DEFINITION**14.14.1.2 Reference Phases**

This section lists all the <reference phase> identifiers which can be employed in the REF subfield or in statement characteristics associated with the noun AC SIGNAL.

PHASE-A
 PHASE-B Indicates that the reference voltage is between the phase stated and the neutral, N.
 PHASE-C

PHASE-BC
 PHASE-CB
 PHASE-CA The first letter is the high side of the referenced voltage and the second is the low side.
 PHASE-BA
 PHASE-AB
 PHASE-AC

14.14.1.3 Special Connections

COMMON <connection> to indicate that the lead of either the measurement, or stimulus, or load device is to be connected to COMMON.

EARTH <connection> to indicate that the lead of either the stimulus, or load, or measurement device is to be connected to EARTH.

ATMOS <connection> to indicate that the port associated with the noun AMBIENT CONDITIONS (See Section 16.2) is connected to the surrounding atmosphere.

Note: For use of COMMON and EARTH as nouns; refer to Section 16.4 for COMMON, Section 16.7 for EARTH.

14.14.1.4 Other Reserved Words used in <conn set>

CNX First entry in every <conn> field.

REF Prefix for the UUT connections of the reference signal for phase critical functions.

CNX-NUMERATOR First entry in a response <conn> field used in a statement that specifies a ratio measurement.

CNX-DENOMINATOR Second entry in a response <conn> field used in a statement that specifies a ratio measurement.

14.14.1.5 Digital UUT Connections

All logic signals, being ordinary analog pulses or DC levels, follow the standard analog conventions for the UUT connections except for the following special definitions for differential signals and for DATA bit sequence and HI/LO pin sequence.

Connections for Serial Signals

Connections for two-wire Serial Signals are written using the standard HI/LO pin descriptors. Three-wire electrical differential signals use the pin descriptors TRUE, COMPL and LO.

14.0 FIELD AND SUBFIELD DEFINITION**Connections for Parallel Signals**

For two-wire Parallel Signals, the number of HI connections must be the same as the number of bits in the parallel digital word. Three possible alternatives exist for the LO connections:

- a. One circuit return (LO connection) may be provided for all HI connections
- b. One circuit return (LO connection) may be provided for each HI connection
- c. Individual circuit returns may be grouped within the UUT and one LO connection provided at the UUT interface for each group of HI connections. Entries in the CNX field are written as follows:
 1. For cases a and b, all HI connections are written first, each separated by at least one space, in a sequence that corresponds to the left-to-right bit sequence. That is, the most significant bit is transmitted over the HI connection written first. The LO connection(s) is (are) written next. If individual UUT signal returns are provided, they are written in the same sequence as the HI connections. That is, corresponding HI and LO connections represent the two sides of one electrical circuit.
 2. For case c, the HI connections are written in groups such that all members of one group are in adjacent positions. The overall sequence of the HI connections must also conform to the bit sequence, as for cases a and b. The LO connections for each group are written immediately following the last HI in that group.

Thus, for the three cases above,

- a. CNX HI J1-1 J1-2 J1-3 J1-4 LO J1-5
- b. CNX HI J1-1 J1-2 J1-3 J1-4 LO J1-11 J1-12 J1-13 J1-14
- c. CNX HI J1-1 J1-2 J1-3 LO J1-10 HI J1-4 J1-5 J1-6 LO J1-11

Electrical Differential Signals

The input or output for UUT digital signals may require transmission in an electrical differential form. As illustrated below, this differential form places a digital signal, called TRUE, on one line and its complement, called COMPL, on another line. These connections are written as follows for a Parallel Signal:

1. Pin descriptors are TRUE, COMPL, and LO
2. The number of COMPL connections must equal the number of TRUE connections.

Three options for the number of LO connections exist:

- a. One circuit return (LO connection) may be provided for all HI connections.
- b. One circuit return (LO connection) may be provided for each HI connection.
- c. Individual circuit returns may be grouped within the UUT and one LO connection provided at the UUT interface for each group of HI connections.
3. For cases 2(a) and 2(b), connections are written in the following sequence: all TRUE connections, all COMPL connections, all LO connections.

14.0 FIELD AND SUBFIELD DEFINITION

4. For case 2(c), the TRUE, COMPL, and LO connections for each group are written together.

Thus, for cases 2(a) and 2(c)

- a. CNX TRUE J1-1 J1-2 J1-3 COMPL J1-11 J1-12 J1-13 LO J1-10
- b. CNX TRUE J1-1 J1-2 J1-3 COMPL J1-11 J1-12 J1-13 LO J1-21 TRUE J1-4 J1-5 J1-6 COMPL J1-14 J1-15 J1-16 LO J1-22

Floating differential signals can be defined by omitting the LO connection. In some cases, differential connections may be treated as two-wire HI/LO circuits. However, the validity of this method must be determined for each individual application or logic family. The electrical differential signal for a serial binary signal having a VALUE of 10101 with an NRZ pulse code is:-

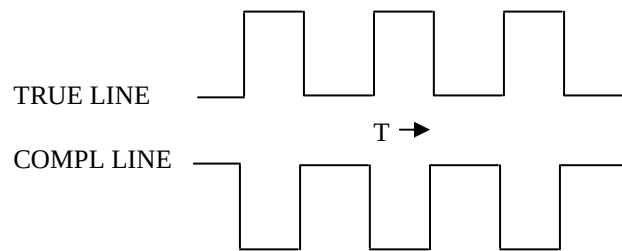


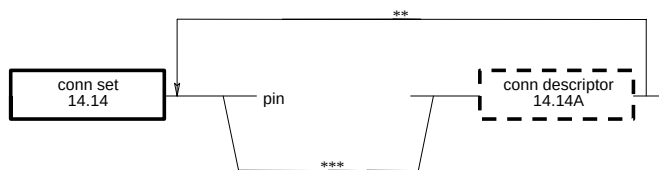
Figure 14-1 TRUE and COMPL Components for a Serial Binary Signal

This complement function can be implemented within the test system either by hardware (by providing an inversion of the TRUE line) or by software (by forming the complement bit pattern and placing it on a second TRUE line).

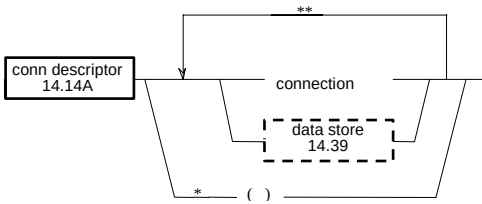
14.14.2 Formal Syntax

FSD / 2680

14.14.3 Syntax Diagram



Note: *** This bypass is allowed only for the nouns where a pin description is not meaningful, such as COMMON, EARTH, and SHORT.

14.0 FIELD AND SUBFIELD DEFINITION**Notes:**

- * This path is only to be used in a <define signal statement> or a <define drawing statement>. When used, it indicates that an actual connection is entered as a parameter in each statement that references the <signal> or <drawing> name.
- ** There must be one or more blank spaces inserted when this feedback path is followed.

14.14.4 Rules

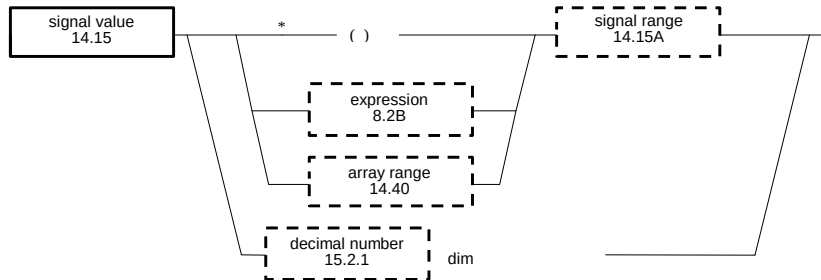
1. The connector designations on the UUT are referred to by the term <connection> and are in general assigned arbitrarily by the manufacturer. However, there are certain minor restrictions:
 - a. A <connection> is represented by an arbitrary sequence of ATLAS characters excluding non-printing-characters, blanks, apostrophe, currency symbol, comma, left and right parenthesis and equal sign (=).
 - b. Certain unique character strings cannot be used as connection designations since they have a particular meaning in ATLAS. They are listed in Section 14.14.1.
 - c. No more than the first 16 characters are significant.
 - d. An asterisk '*' indicates that the connection which it immediately precedes is a TERMINAL defined in the associated drawing. An asterisk may not be used for any other purpose in a connection.
 - e. The term <connection> includes the special connections listed in Section 14.14.1.3 for COMMON, EARTH and ATMOS.

14.0 FIELD AND SUBFIELD DEFINITION14.15 <signal value>14.15.1 Function

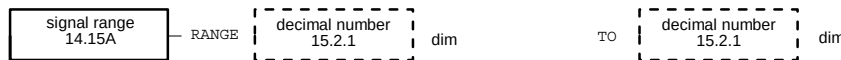
To specify one or more values and the associated dimension for a signal characteristic.

14.15.2 Formal Syntax

FSD / 2710

14.15.3 Syntax Diagram

Note: * This path may only be used to specify the requirement for a parameter within a <define signal statement>.

14.15.4 Rules

1. All dimensions referenced in <signal value> must be identical.
2. The value of the <expression> or <decimal number> and all values of the <array range> will be scaled to the dimensions in the <signal value>.
3. The RANGE following <array range> specifies the permitted range of values for each element of the <array range>.
4. When an <expression> is used, the evaluation of the expression must be assignment compatible with its usage.

14.0 FIELD AND SUBFIELD DEFINITION14.16 <real quantity>14.16.1 Function

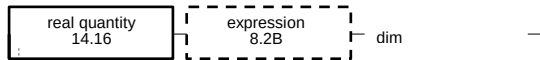
To specify the value of a physical characteristic of a signal.

14.16.2 Formal Syntax

FSD / 2726

14.16.3 Syntax Diagram14.16.4 Rules

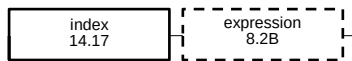
1. The <expression> must evaluate to a <decimal number> or to a whole <decimal number>.

14.17 <index>14.17.1 Function

To specify the selected component of an array.

14.17.2 Formal Syntax

FSD / 2730

14.17.3 Syntax Diagram14.17.4 Rules

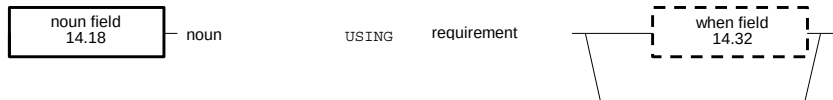
1. The type of <expression> must be the same as the type of the index as defined in the associated structure declaration.

14.0 FIELD AND SUBFIELD DEFINITION14.18 <noun field>14.18.1 Function

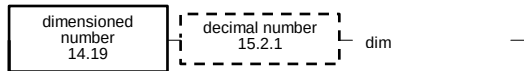
To define the signal type of signal and to identify the virtual test resource which is to be employed. The noun and USING or THRU subfield may be followed by a WHEN subfield to indicate when the critical action of the statement is to occur.

14.18.2 Formal Syntax

FSD / 2734

14.18.3 Syntax Diagram14.18.4 Rules

1. The <requirement> label must have been previously established in a <require statement>. The <noun> must be the same as the <noun> used in the <require statement>.
2. The <when field> cannot be used in a <noun field> that is part of either an <identify statement>, a <digital source definition> or a <digital sensor definition>.

14.19 <dimensioned number>14.19.1 Function

To specify a numerical value with its associated dimension.

14.19.2 Formal Syntax

FSD / 2744

14.19.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION

14.20 <require control>

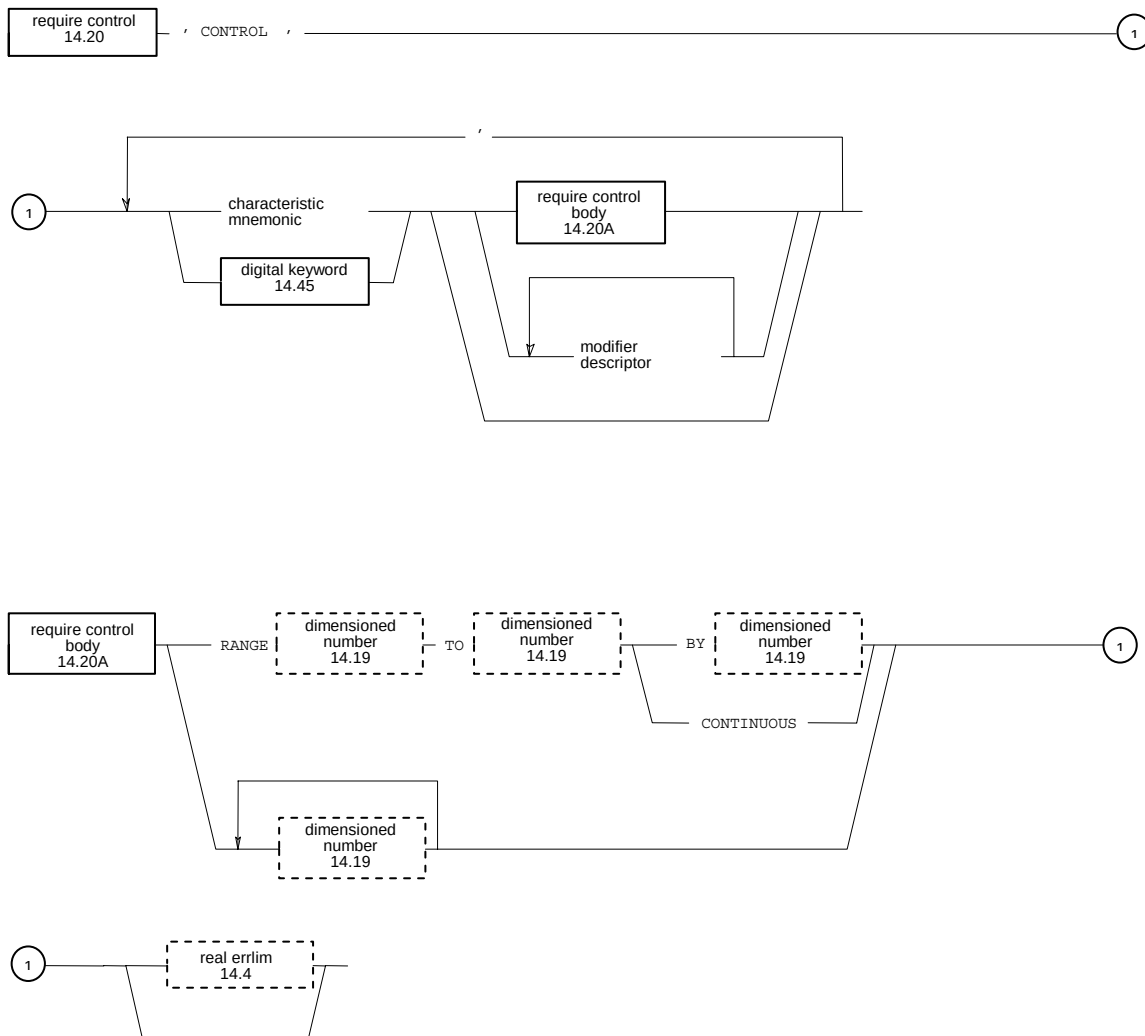
14.20.1 Function

To summarize by a set of <require control> fields in a <require statement> those characteristics of a test capacity which must be controlled by the virtual test resource to execute the <atlas test specification structure>.

14.20.2 Formal Syntax

FSD / 2748

14.20.3 Syntax Diagram



14.20.4 Rules

1. All dimensional units appearing in a single <require control> subfield, including associated <real-errlim> fields, must be identical except for percent "PC" in the <real-errlim> fields.
2. Only the following can be used for digital keywords:

14.0 FIELD AND SUBFIELD DEFINITION

SKEW-TIME, WORD-LENGTH, PULSE-CLASS, BIT-RATE, BIT-PERIOD and SPEC.

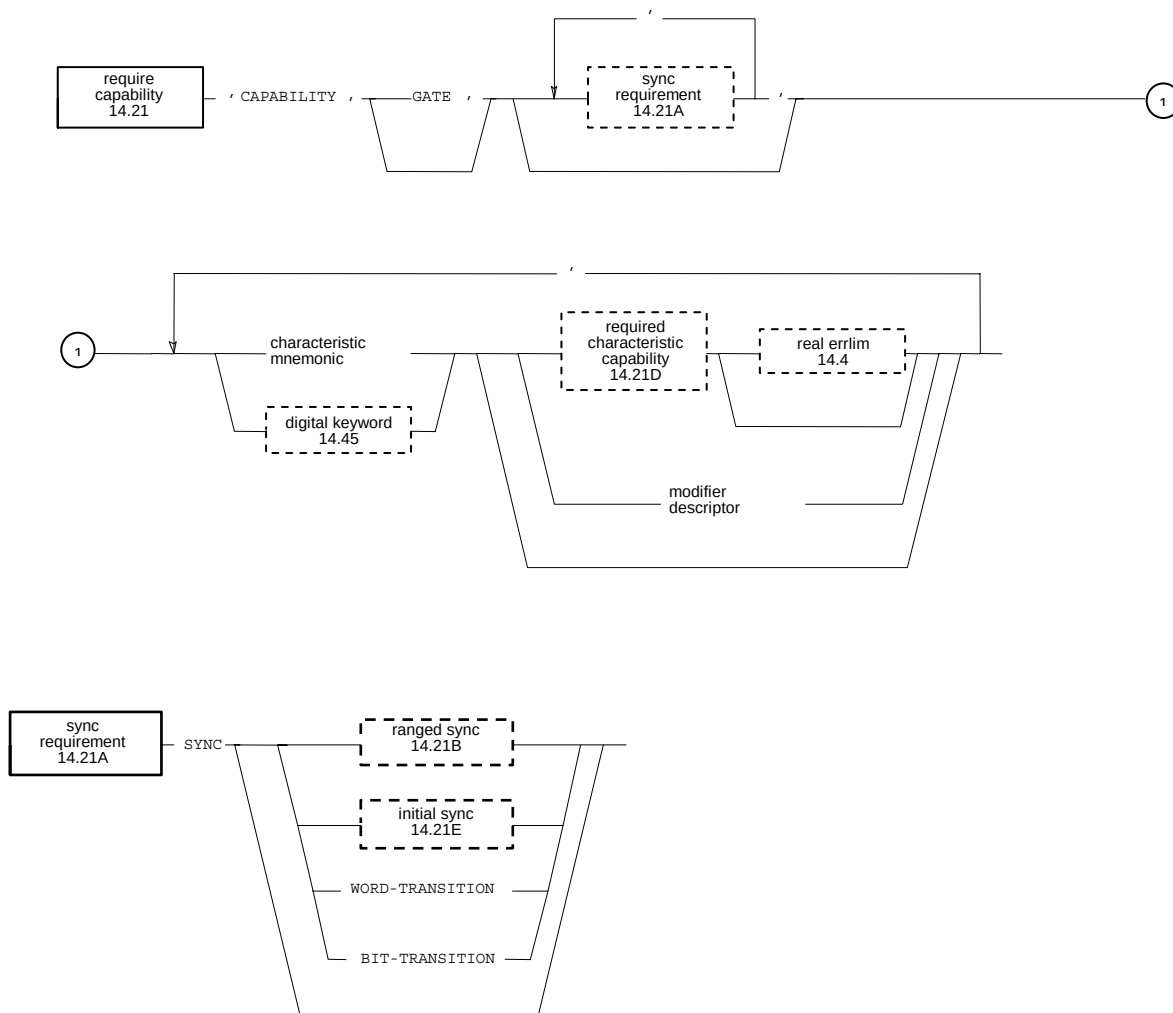
Where necessary, the rules in Sections 6.19.3 for <define digital source statement> and 6.19.4 for <define digital sensor statement> refer to the use of these digital keywords.

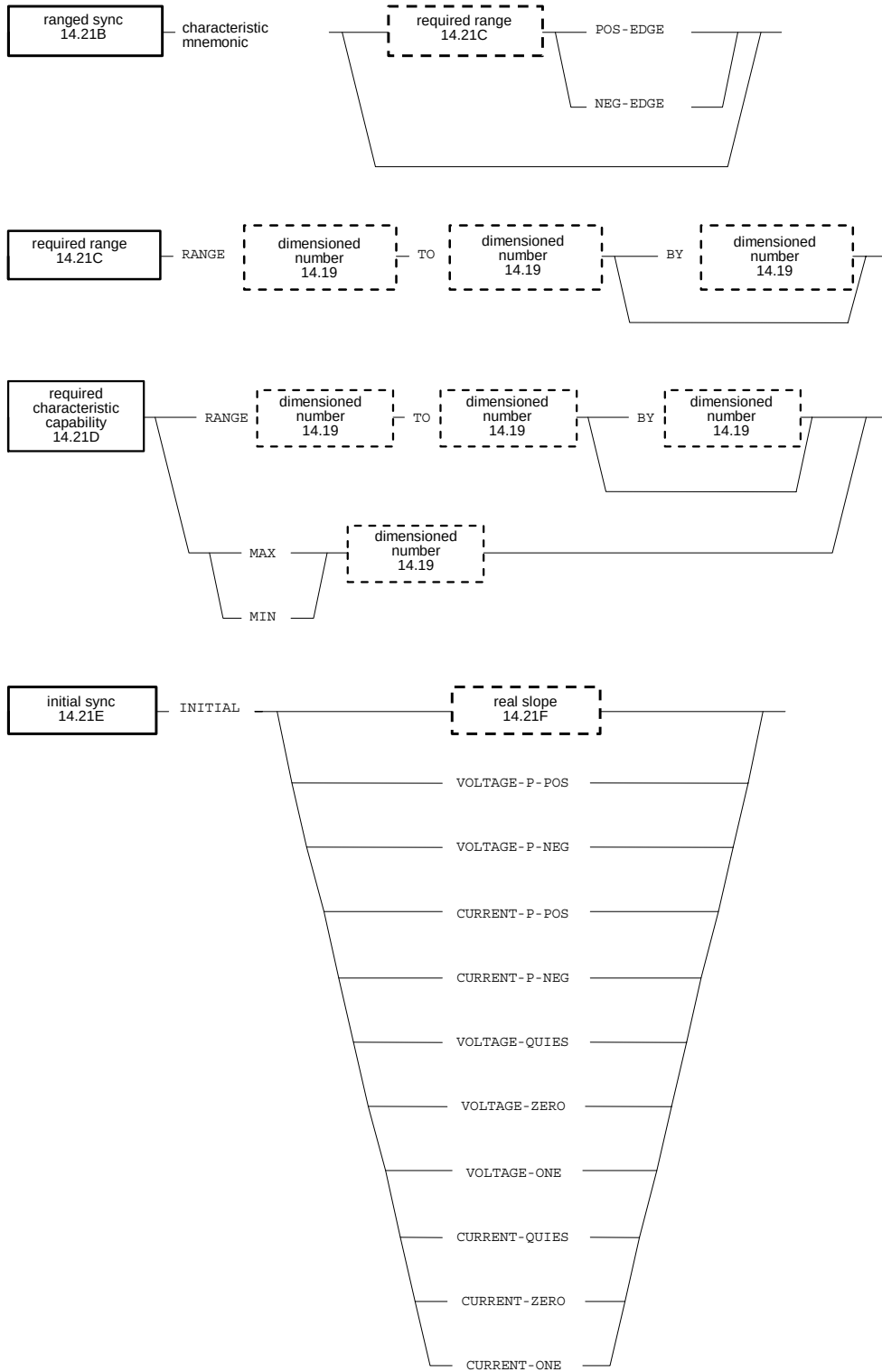
14.21 <require capability>**14.21.1 Function**

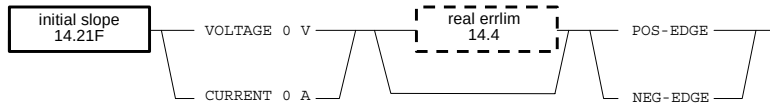
To summarize by a set of <require capability> fields in a <require statement> those characteristics of a test capacity which are required to be provided but are not required to be controlled or limited by the virtual test resource to execute the <atlas test specification structure>.

14.21.2 Formal Syntax

FSD / 2777

14.21.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION

14.0 FIELD AND SUBFIELD DEFINITION**14.21.4 Rules**

1. The optional BY path is not used in the <require capability> field for SOURCE or LOAD resource types.
2. GATE indicates that some uses of the virtual test resource employ a <gate field>.
3. A SYNC indicates that some uses of the virtual test resource employ a <sync subfield> of that form.
4. The <real errlim> must be the most demanding error limit for that INITIAL synchronization.
5. The RANGE subfield summarizes the range of <evaluation field>s associated with that mnemonic in <sync subfield>s.
6. In a <require capability> field, the three optional bypasses cannot be used simultaneously.
7. All dimensional units appearing in a single <require capability> subfield, including associated <real-errlim> fields, must be identical except for percent "PC" in the <real-errlim> fields.
8. When used in <require capability>, MAX and MIN mean that the resource must be capable of supplying the magnitude of the specified value.
9. When used in <require limit>, MAX and MIN mean that the resource must not supply a magnitude which exceeds the specified value.
10. Only the following can be used for digital keywords:
SKEW-TIME, WORD-LENGTH, PULSE-CLASS, BIT-RATE, BIT-PERIOD and SPEC.
Where necessary, the rules in Sections 6.19.3 for <define digital source statement> and 6.19.4 for <define digital sensor statement> refer to the use of these digital keywords.

14.0 FIELD AND SUBFIELD DEFINITION

14.22 <require limit>

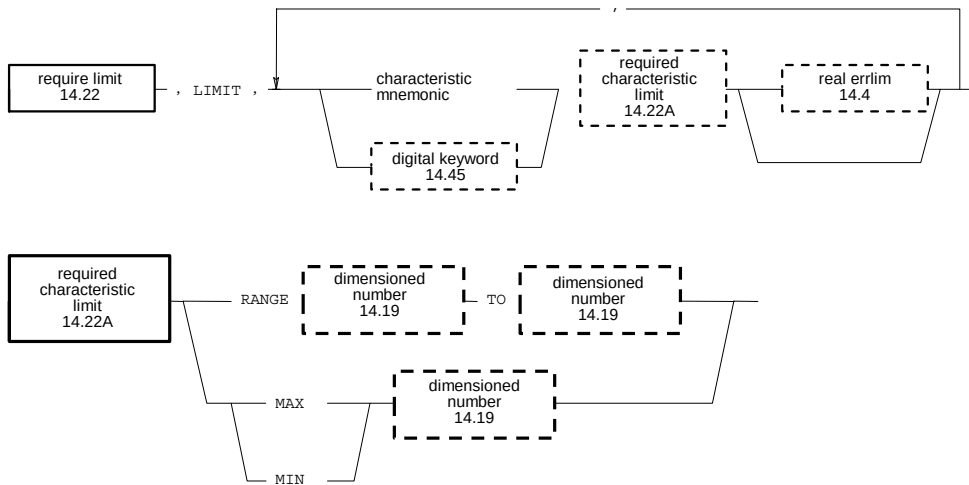
14.22.1 Function

To summarize by a set of <require limit> fields in a <require statement> those characteristics of a test capacity which must be limited (constrained) to safeguard the UUT from malfunction of either itself or the test resource during the execution of the <atlas test specification structure>.

14.22.2 Formal Syntax

FSD / 2851

14.22.3 Syntax Diagram



14.22.4 Rules

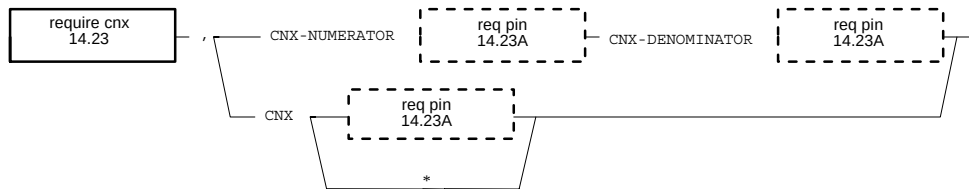
1. All dimensional units appearing in a single <require limit> subfield, including associated <real-errlim> fields, must be identical except for percent "PC" in the <real-errlim> fields.
2. When used in <require capability>, MAX and MIN mean that the resource must be capable of supplying the magnitude of the specified value.
3. When used in <require limit>, MAX and MIN mean that the resource must not supply a magnitude which exceeds the specified value.
4. Only the following can be used for digital keywords:
SKEW-TIME, WORD-LENGTH, PULSE-CLASS, BIT-RATE, BIT-PERIOD and SPEC.
Where necessary, the rules in Sections 6.19.3 for <define digital source statement> and 6.19.4 for <define digital sensor statement> refer to the use of these digital keywords.

14.0 FIELD AND SUBFIELD DEFINITION14.23 <require cnx>14.23.1 Function

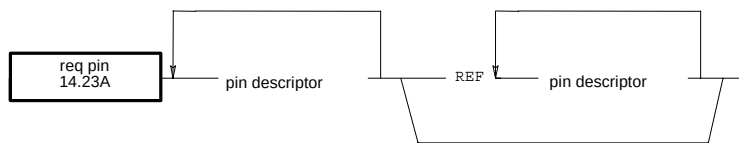
To summarize by a <require cnx> field the pin descriptors associated with the connection points of the virtual test resource associated with the <requirement>.

14.23.2 Formal Syntax

FSD / 2865

14.23.3 Syntax Diagram

Note: * This bypass is allowed only for nouns where a pin descriptor is not meaningful such as COMMON, EARTH, and SHORT.



14.0 FIELD AND SUBFIELD DEFINITION

14.24 <exchange expression>

14.24.1 Function

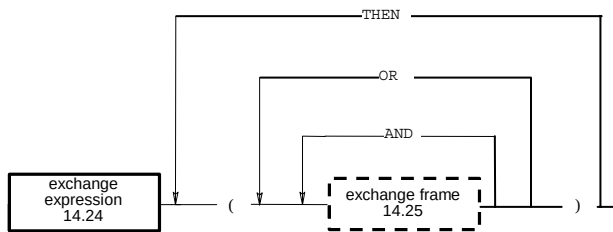
To describe the inter-relationships between <exchange frame>s.

For multiple <exchange frame>s, the AND connective specifies that all <exchange frame>s so connected will be acted upon while the OR connective specifies that only one of the <exchange frame>s will be acted on.

14.24.2 Formal Syntax

FSD / 2883

14.24.3 Syntax Diagram



14.24.4 Rules

1. In the case of bus contention/conflict, the AND relationship between <exchange frame>s indicates that priority is invoked. That is, the <exchange frame> with the highest priority is serviced first; the servicing of any lower priority <exchange frame> is delayed. Priority can be explicitly expressed by exercising the PRIORITY field in the <exchange frame> definition. Lower priority numbers have the higher priority. The highest possible priority is PRIORITY zero.
2. The OR relationship indicates in the case of bus contention/conflict, that priority is also invoked; however the lower priority <exchange frame>s are not serviced for that particular interval of time.
3. The THEN relationship is used to separate sequential <exchange frame>s.
4. AND and OR do not imply normal Boolean operators. They may not be mixed within a group separated by THEN. For example (A AND B AND C) THEN (E OR F OR G) is valid but (A AND B AND C) THEN (E AND F OR G) is invalid.

14.24.5 Notes and Example

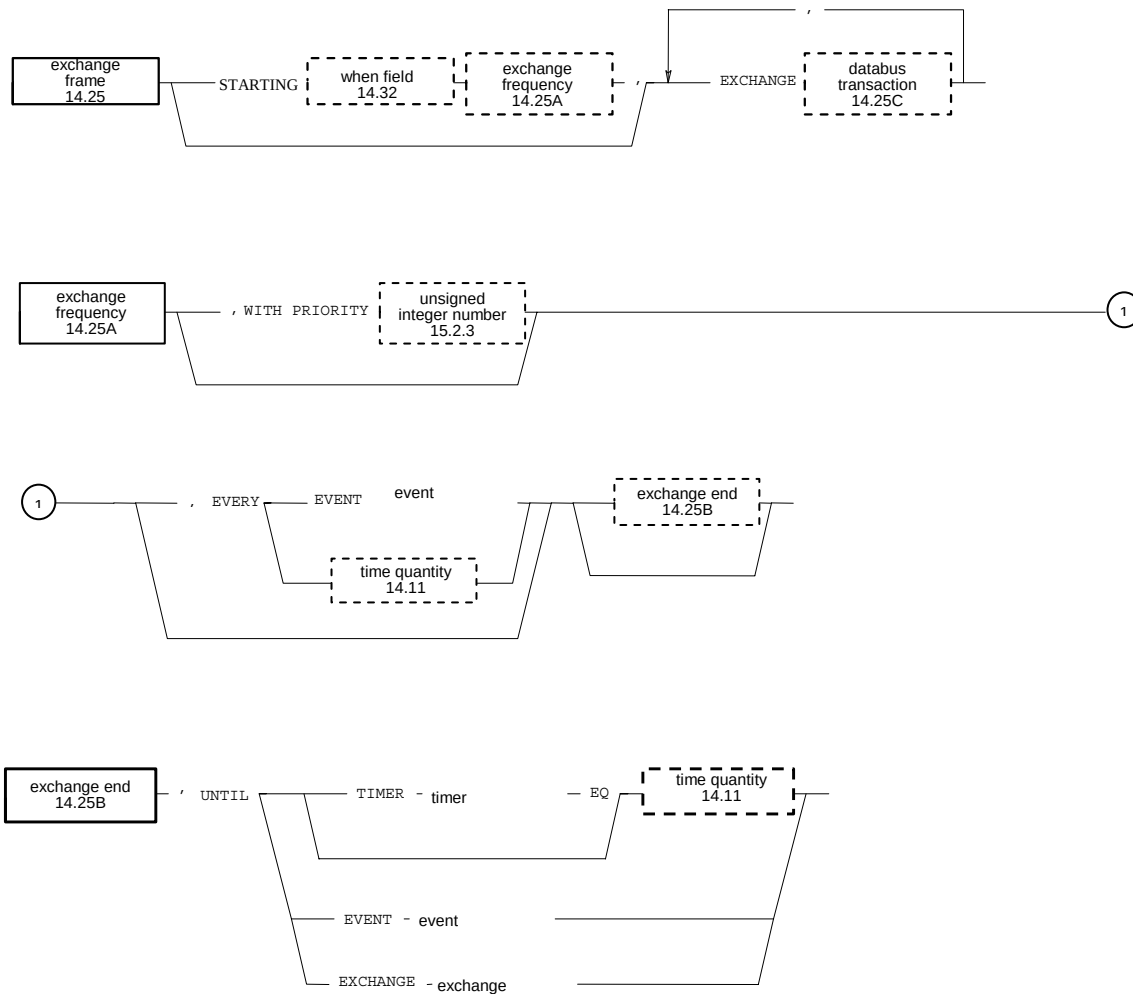
See Section 13.1 <do exchange statement>

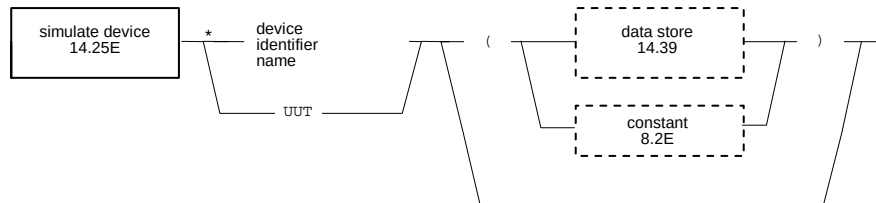
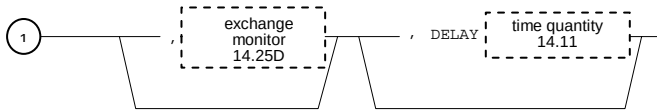
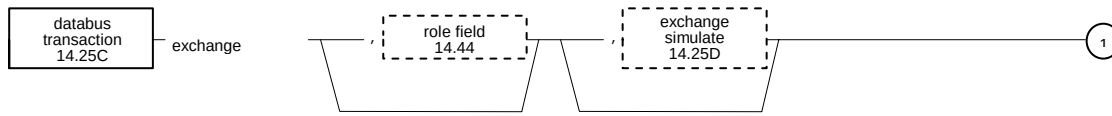
14.0 FIELD AND SUBFIELD DEFINITION14.25 <exchange frame>14.25.1 Function

To describe a sequence of exchanges on a databus for use in an <exchange expression>. Each <exchange frame> may include a starting condition, a priority, a repetition rate, a terminating condition and one or more databus transactions (<exchange>s).

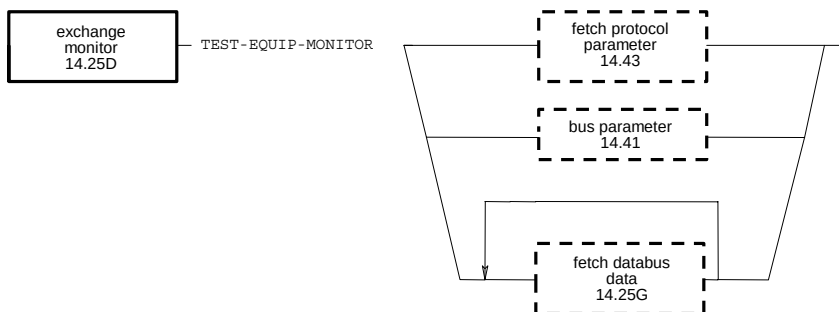
14.25.2 Formal Syntax

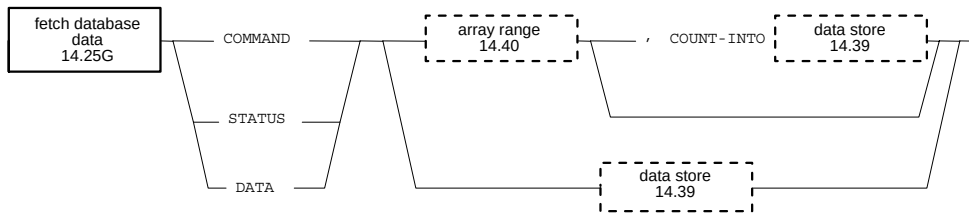
FSD / 2891

14.25.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION

Note: * The device must be one which has been established as valid for the TEST-EUIP to simulate in the ESTABLISH statement for the protocol.



14.0 FIELD AND SUBFIELD DEFINITION**14.25.4 Rules**

1. The <exchange frame> includes a PRIORITY field which determines the numerical priority accorded to this <exchange frame> structure compared to the other <exchange frame> structures in the same <do exchange statement>. Each <exchange frame> within a <do exchange statement> must have a different priority. Lower priority numbers have the higher priority. In the event of a conflict, the highest priority has precedence. The highest possible priority is PRIORITY zero. The PRIORITY field may be omitted only if the <do exchange statement> includes neither AND nor OR connectives between <exchange frame>s.
2. The option UNTIL <exchange> is chosen when a specific <exchange> determines the end of an <exchange frame>.

14.25.4.1 Rules applicable to <databus transaction>

1. The <exchange> specified must be defined in a preceding <define exchange statement>. The <protocol> referenced in the definition must also be referenced in the <exchange configuration> of the USING field in the <do exchange statement> which contains the <exchange frame>.
2. The entry in the <role field> represents the function or role that the test equipment is to assume for the single <exchange>. It temporarily supersedes any default role defined by the <do exchange statement> as described in Section 13.1 for this single <exchange>. If the <do exchange statement> contains no <role field>, each <databus transaction> must specify the role of the test equipment.
3. The TEST-EQUIP-SIMULATE labels must be valid talker or listener <device identifier name>s for the specified <exchange> as defined by the associated <define exchange statement>. The <type> of the <expression> must be the same as the <type> of the <data store> or <constant> in the TALKER or LISTENER field of the <define exchange statement>. Any COMMAND, DATA or STATUS information which is to be transmitted by the test equipment will be extracted from the corresponding field of the <define exchange statement>.
4. The TEST-EQUIP-MONITOR field is used to identify bus data which is to be monitored by the test equipment. For COMMAND, DATA and STATUS information it specifies the <data store>s defining which bus data items are to be monitored. The <type> of the <data store> must be either the same <type> as in the <establish protocol statement> or a reference to a <record field> of a named record variable with fields of the same <type> as that used in the <establish protocol statement>. If more than one set of COMMAND, DATA and STATUS words are required, the specifications must appear in the following order: COMMAND, DATA, STATUS.

If the WAIT option is specified in the <do exchange statement> the specified variables are updated during the execution of the <do exchange statement>. If the destination is an <array range>, successive array elements receive successive values of the parameter up to the limit defined by the <array range> indices.

Such a list may have an associated <data store> introduced by COUNT-INTO. If it does, the start of execution of the <do exchange statement> sets the <data store> to zero. Each time a new value of that monitored parameter is stored in the array the <data store> is incremented.

14.0 FIELD AND SUBFIELD DEFINITION

If the PROCEED option is specified the monitored data is retained by the test equipment and the specified variables are NOT updated. An <array range> in this case serves only to identify the number of consecutive bus words which are to be monitored by the test equipment. The COUNT-INTO branch is not used. The monitored data must be extracted using a subsequent <fetch exchange configuration statement> and will be returned to the variables specified in the <fetch exchange configuration statement>.

5. If the TEST-EQUIP-MONITOR field is used to monitor <bus parameter>s or <protocol parameter>s then each <bus parameter name> and <protocol parameter name> must be repeated from the <establish protocol statement>. The parameter must be FETCHABLE at the EXCHANGE level.
6. DELAY specifies the time from the initiation of the <exchange> preceding the DELAY to the initiation of the <exchange> following the DELAY. The user must ensure that adequate time exists to execute the <exchange>.

The DELAY branch must not occur following the final <exchange>.

14.25.5 Notes and Examples

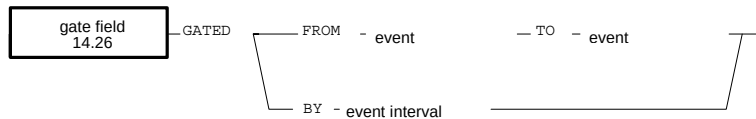
The <exchange frame> describes, in effect, an independent operation within a <do exchange statement>.

14.0 FIELD AND SUBFIELD DEFINITION14.26 <gate field>14.26.1 Function

To restrict the time-slot during which a signal is applied or sensed.

14.26.2 Formal Syntax

FSD/2971

14.26.3 Syntax Diagram14.26.4 Rules

1. A <gate field> specifies a time interval bounded either by an <event interval> or by a start <event> and a stop <event> during which a signal is applied or sensed.

14.26.5 Notes and Examples

The <gate field> permits a particular portion of a waveform to be selected as the subject for the measure statement containing the <gate field>. For instance if the signal is a pulse train and it is required to measure the width of the third pulse the inclusion of the following <gate field> would permit this to be achieved:

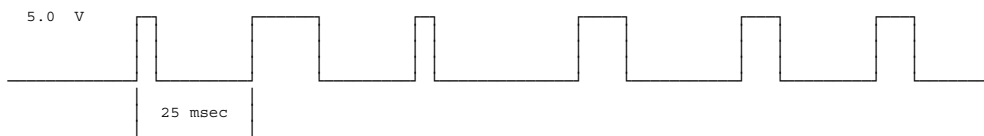


Figure 14-2 Example for <gate field>.

```

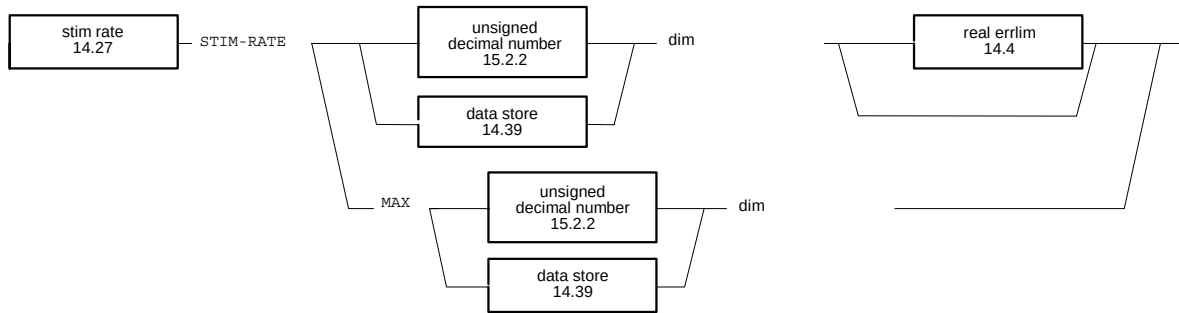
300010 IDENTIFY, EVENT 'TRANSITION' AS PULSED DC,
          VOLTAGE-INST EQ 3.0 V,
          FREQ 400.0 HZ, CNX HI J1 LO J2 $
20 IDENTIFY, EVENT 'A' AS 4 OCCURRENCES OF 'TRANSITION' $
30 IDENTIFY, EVENT 'B' AS 6 OCCURRENCES OF 'TRANSITION' $
40 IDENTIFY, EVENT 'THIRD-OPEN' AS 10.0 USEC AFTER 'A' $
50 IDENTIFY, EVENT 'THIRD-CLOSE' AS 10.0 USEC AFTER 'B' $
60 MEASURE, (PULSE-WIDTH), PULSED DC,
          PULSE-WIDTH MAX 2.0 MSEC,
          GATED FROM 'THIRD-OPEN' TO 'THIRD-CLOSE',
          CNX HI J1 LO J2 $
  
```


14.0 FIELD AND SUBFIELD DEFINITION14.27 <stim rate>14.27.1 Function

To specify the rate at which a digital test sequence is to stimulate the UUT.

14.27.2 Formal Syntax

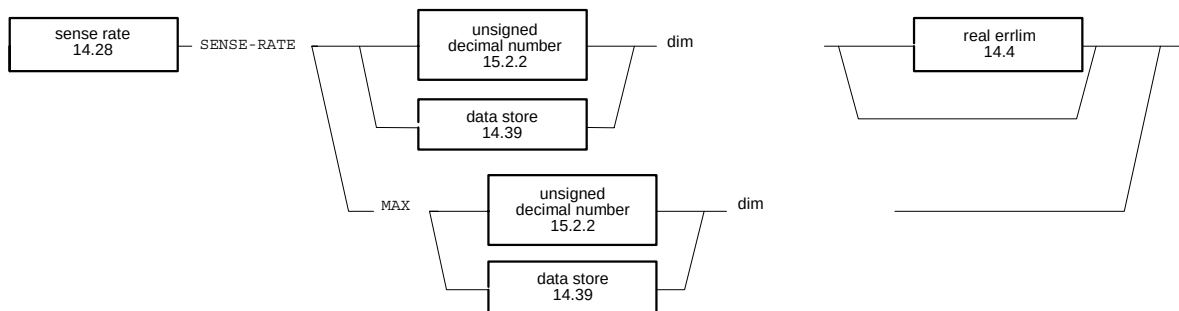
FSD / 2979

14.27.3 Syntax Diagram14.28 <sense rate>14.28.1 Function

To specify the rate at which a digital test sequence is to be sensed to determine the UUT response.

14.28.2 Formal Syntax

FSD / 2999

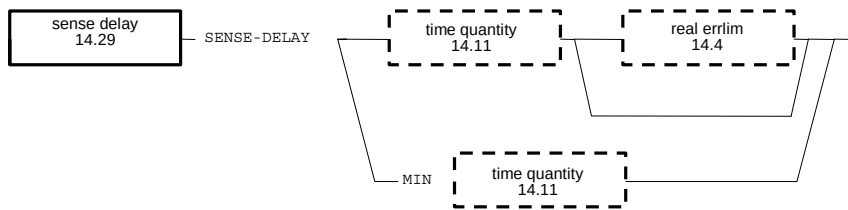
14.28.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION14.29 <sense delay>14.29.1 Function

To specify the time interval that the critical action of a <sense statement> or <prove statement> are to be delayed after the critical action of the preceding <stimulate statement> as they are executed in a digital test sequence.

14.29.2 Formal Syntax

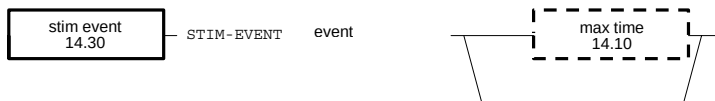
FSD / 3003

14.29.3 Syntax Diagram14.30 <stim event>14.30.1 Function

To specify an <event> whose occurrence indicates when successive <stimulate statement>s in a digital test sequence are to be executed.

14.30.2 Formal Syntax

FSD / 3012

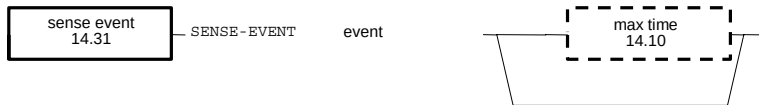
14.30.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION14.31 <sense event>14.31.1 Function

To specify an <event> whose occurrence indicates when successive <sense statement>s or <prove statement>s in a digital test sequence are to be executed.

14.31.2 Formal Syntax

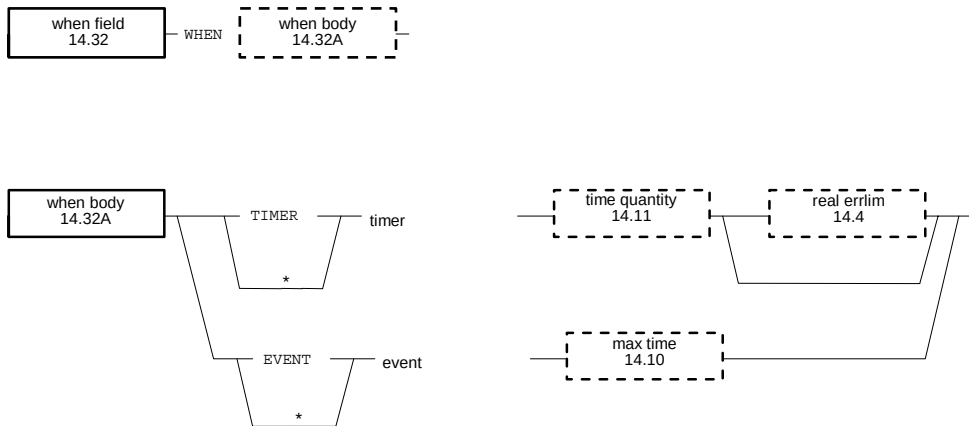
FSD / 3017

14.31.3 Syntax Diagram14.32 <when field>14.32.1 Function

To specify the precise instant in a test sequence at which an action or set of actions must be executed.

14.32.2 Formal Syntax

FSD / 3022

14.32.3 Syntax Diagram

Note: * Non-preferred

14.32.4 Rules

1. When the path containing <event> is used, the instant of execution is the first new occurrence of the referenced <event>.

14.0 FIELD AND SUBFIELD DEFINITION

14.33 <digital source characteristics>

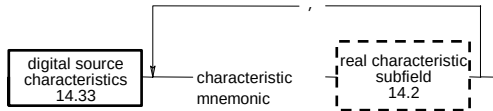
14.33.1 Function

To specify the physical characteristics of driven logic symbols.

14.33.2 Formal Syntax

FSD / 3035

14.33.3 Syntax Diagram



14.33.4 Rules

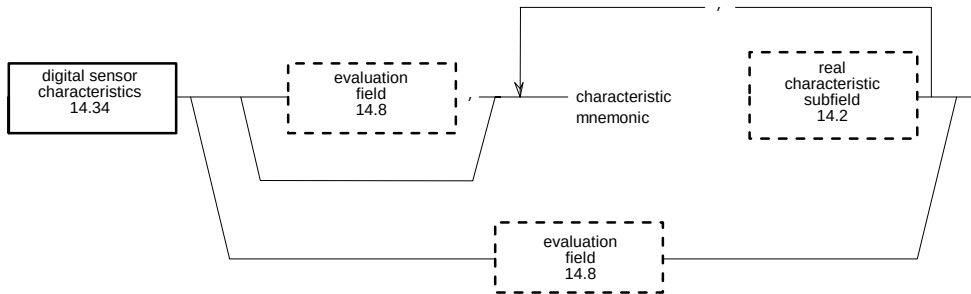
1. The <mnemonic> chosen must be applicable to the <noun> specified in the associated <define digital source field>.
2. Numerical values can only be designated variable by the use of parenthetical variables. (See Section 15.6.7).
3. When used following LEVEL-LOGIC-HIZ to specify the high impedance condition of the test system, the direction of leakage current flow is specified as follows:
 - current flowing from the test system as a positive current;
 - current flowing into the test system as a negative current.

14.0 FIELD AND SUBFIELD DEFINITION14.34 <digital sensor characteristics>14.34.1 Function

To specify the physical characteristics of sensed logic symbols.

14.34.2 Formal Syntax

FSD / 3042

14.34.3 Syntax Diagram14.34.4 Rules

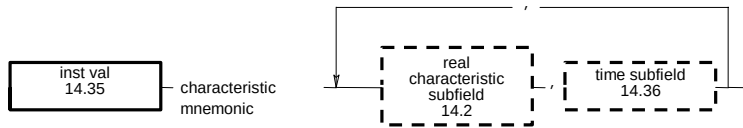
1. The <mnemonic> chosen, if any, must be applicable to the <noun> specified in the associated <define digital sensor statement>.
2. Numerical values can only be designated variable by the use of parenthetical variables. (See Section 15.6.8).
3. When used following LEVEL-LOGIC-HIZ to specify measurement of the high impedance condition of the UUT, the direction of leakage current flow is specified as follows:
 - current flowing from the UUT as a positive current;
 - current flowing into the UUT as a negative current.
4. When used following LEVEL-LOGIC-HIZ, the requirement for the definition of the characteristics of high impedance to be in terms of the <measured characteristic mnemonic> are relaxed. (See Rule 7, Section 6.19.4.3).

14.0 FIELD AND SUBFIELD DEFINITION**14.35 <inst val>****14.35.1 Function**

To define instantaneous values for an analog signal characteristic whose pattern of variation in time constitutes the definitions of a logic state. The initial value of the signal characteristic is specified in the preceding <real characteristic subfield> or <evaluation field>.

14.35.2 Formal Syntax

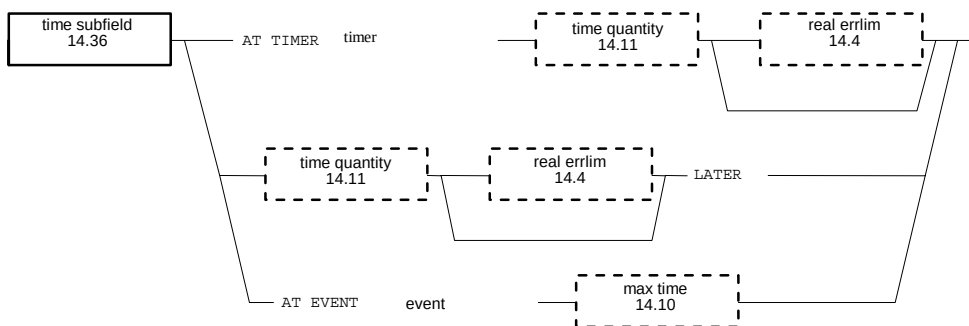
FSD / 3053

14.35.3 Syntax Diagram**14.36 <time subfield>****14.36.1 Function**

To establish the time at which a signal oriented action must occur. When used in a <define digital source statement> or a <define digital sensor statement>, to specify the time to reach the new signal level from the start of the bit period or from the last instantaneous value.

14.36.2 Formal Syntax

FSD / 3063

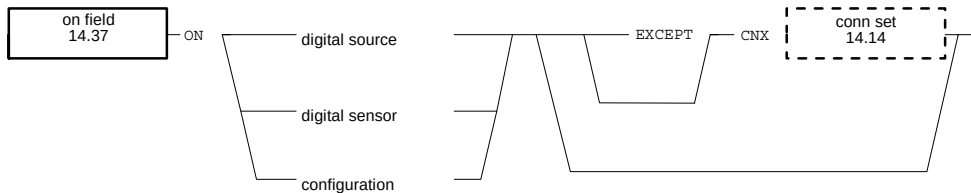
14.36.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION14.37 <on field>14.37.1 Function

To define the UUT connections in <stimulate statement>s, <sense statement>s and <prove statement>s (or their compounds). The <on field> allows digital testing to be performed using some or all of the connections specified in a <define digital configuration structure>.

14.37.2 Formal Syntax

FSD / 3073

14.37.3 Syntax Diagram14.37.4 Rules

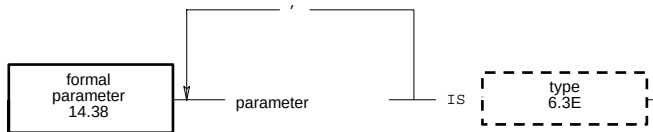
1. The connections that are specified or implied by using EXCEPT, or by using <data store>s in <conn set> must be from those defined in either a <define digital source statement> or a <define digital sensor statement> of an ENABLEd <configuration>.
2. If a CNX field is not included, digital testing is performed on all connections associated with the specified <define digital source statement> or <define digital sensor statement> or <configuration>.
3. If a CNX field is included, digital testing is performed only on the specified connections. Connections that are not specified will not be changed by <stimulate statement>s, and will not be SENSEd by <sense statement>s.
4. If EXCEPT is used, digital testing is performed on all connections associated with the specified <define digital source statement> or <define digital sensor statement> or <configuration> EXCEPT for those given in the following CNX field.

14.0 FIELD AND SUBFIELD DEFINITION14.38 <formal parameter>14.38.1 Function

To declare a <label> that is local to the body of a <define procedure structure> or a <define function structure> and is used for communication between the calling level and the defined <procedure> or <function>.

14.38.2 Formal Syntax

FSD / 3086

14.38.3 Syntax Diagram14.38.4 Rules

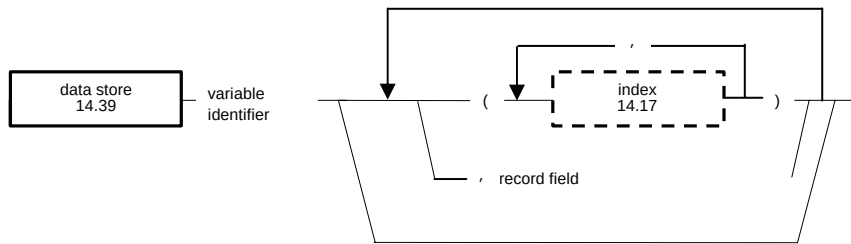
1. When a defined <procedure> is performed or a defined <function> is invoked, the arguments associated with the invocation must be assignment compatible with the parameter type.

14.0 FIELD AND SUBFIELD DEFINITION14.39 <data store>14.39.1 Function

To specify a simple variable, array element, or record field.

14.39.2 Formal Syntax

FSD / 3091

14.39.3 Syntax Diagram14.39.4 Rules

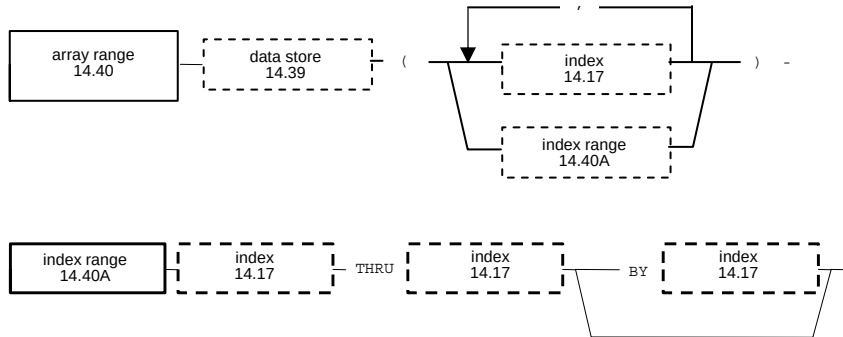
1. If the <variable identifier> label is followed by a parenthesized <index> then an element of an ARRAY or STRING is being addressed.
2. To reference an ARRAY element, the number of indices used in the reference must match the number in the declaration of the ARRAY.
3. If the <variable identifier> label is followed by a period (".") then the first label is the name of a RECORD and the subsequent label is the name of a <record field> within the RECORD.

14.0 FIELD AND SUBFIELD DEFINITION14.40 <array range>14.40.1 Function

To specify a range or series of discrete indices of an array.

14.40.2 Formal Syntax

FSD / 3102

14.40.3 Syntax Diagram14.40.4 Rules

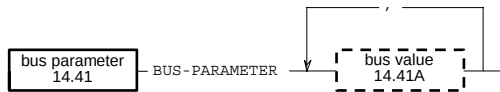
1. A <data store> must select a one dimension array.
2. If the form of <array range> is M THRU N BY J then a sequence of array elements is specified where M defines the first array index, N defines the final index and J defines the increment to be used to compute successive indices.
3. If the BY subfield is omitted, the increment is assumed to be 1.
4. The form of <array range> (N1; N2;...; Nn) specifies a series of discrete array elements in the given order.

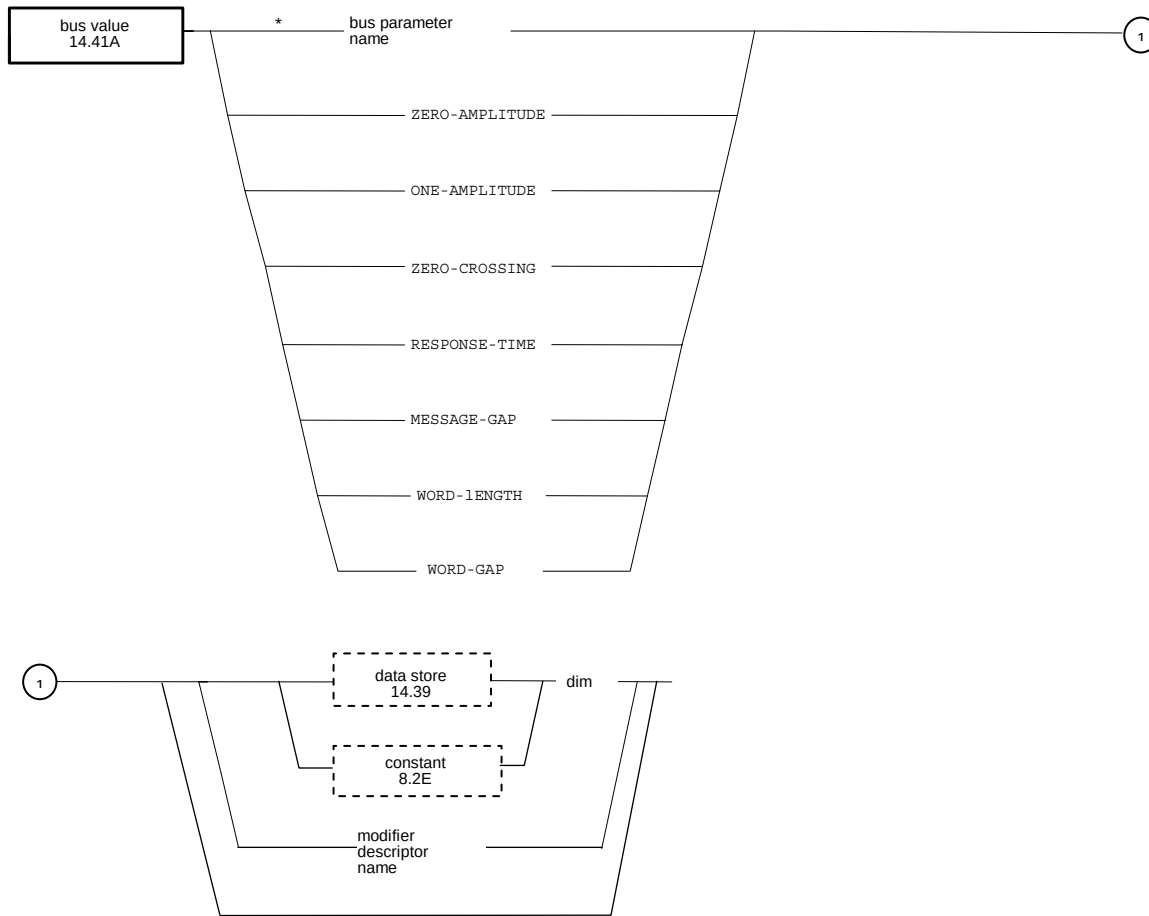
14.0 FIELD AND SUBFIELD DEFINITION14.41 <bus parameter>14.41.1 Function

To specify explicitly any <bus parameter>s which are required to be set to values different from those defined in the referenced specifications or are required to be read during the testing.

14.41.2 Formal Syntax

FSD / 3116

14.41.3 Syntax Diagram

14.0 FIELD AND SUBFIELD DEFINITION

Note: * The bus parameter name must have been established as valid for the databus in the associated ESTABLISH statement. For it to occur in an UPDATE statement of either sort, it must have been defined as UPDATABLE for that category of UPDATE. It and WORD-GAP are the only form of protocol parameter that can occur in a FETCH statement and then only if established as monitored by the test equipment in the <do exchange statement>.

14.41.4 Rules

1. All entries which are used in this field must be defined in the <bus installation parameters> section of the <establish protocol statement>. The <constant> path may only be used when a value for the bus parameter is to be set, i.e. in a <define exchange statement> an <enable exchange configuration statement> or an <update statement>. The <data store> path can be used in the same way as the <constant> but in addition it can be used as the destination for the <bus parameter> value which can be read from the test equipment in a <do exchange statement> or <fetch exchange statement>. The type of the <constant> or <data store> specified must be the same <type> as the <bus parameter range> in the <establish protocol statement> and its value if specified must be within the specified range.

14.0 FIELD AND SUBFIELD DEFINITION

2. The following standard entries with the listed general meaning may appear in the field. Each one used must be precisely defined by the <external bus specification>s.

- | | | |
|----|---|---|
| a) | WORD-LENGTH | The length of a digital word transferred via the referenced bus. |
| b) | WORD-GAP | The length of time between the transmission of successive words in one message. This may be specified as bit times or time quantity. |
| c) | MESSAGE-GAP | The length of time between the last bit transmitted in a complete message to the first bit transmitted in the next successive message to be transmitted on the same bus. |
| d) | RESPONSE-TIME | The length of time between the last bit transmitted in a message word which requests a receiver's response and the first bit transmitted of the response word(s) from that receiver. Both the request word(s) and the response word(s) may be defined as elements of a single message format. |
| e) | ZERO-AMPLITUDE,
and
ONE-AMPLITUDE | The signal levels, respectively, of the LOGIC-LEVEL-ZERO component
the LOGIC-LEVEL-ONE component of the bit coding for this bus. |
| f) | ZERO-CROSSING | The point in time, expressed as a percentage of the bit duration, at which the bit transition occurs for LOGIC-TRANS coding of data on this bus. |

3. If none of the standard entries is suitable to define a bus failure mode which is required in the testing, a <bus parameter name> may be included.

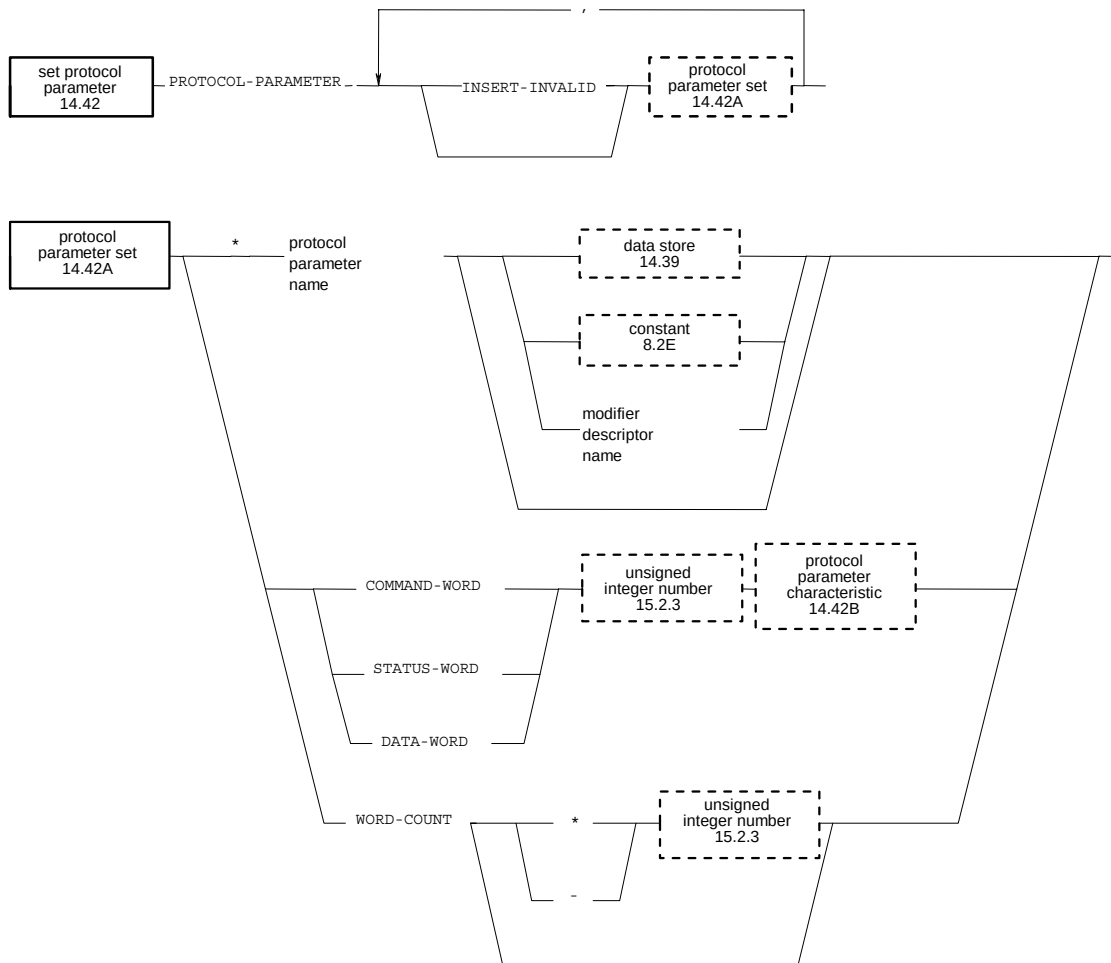
The <bus parameter name> must be defined in an <extend atlas statement> and referenced in the corresponding field of the <establish protocol statement>.

14.0 FIELD AND SUBFIELD DEFINITION14.42 <set protocol parameter>14.42.1 Function

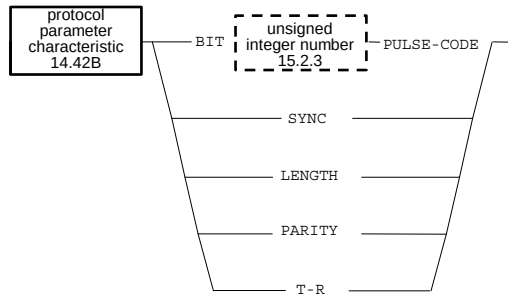
To specify protocol parameters including deviations from the referenced standard which are required to be employed during the tests.

14.42.2 Formal Syntax

FSD / 3147

14.42.3 Syntax Diagram

Note: * The protocol parameter name must have been established as valid for the databus in the associated ESTABLISH statement. For it to occur in an UPDATE statement of either sort, it must have been defined as UPDATABLE for that category of UPDATE.

14.0 FIELD AND SUBFIELD DEFINITION**14.42.4 Rules**

1. The INSERT-INVALID option in <set protocol parameter> enables a <protocol parameter> to be changed to a non-standard value which may be an invalid value outside the range of values permitted by the databus specification only if the FAULT-TEST option was used in the corresponding <establish protocol statement>. The precise specification of the invalid parameter and its values must be defined in the external bus specifications which are referenced in the associated <establish protocol statement>.
2. The bypass of the INSERT-INVALID option in <set protocol parameter> enables a <protocol parameter> to be changed to its standard value. The exact specification of the parameter and its standard value must be defined in the external <bus specification>s which are referenced in the associated <establish protocol statement>. In this case no value can be specified for the parameter and the bypass round the associated value field must be taken.
3. All entries which are used in this field must be fully defined in the <exchange model> section of the <establish protocol statement>.
4. The following standard entries with the listed general meaning may appear in <set protocol parameter>. Each one that is used must be precisely defined by the external <bus specification>s referenced in the associated <establish protocol statement>..

a)	WORD-COUNT	The number of words in a message. The legal word count can be changed in a message by a specified positive or negative increment.
b)	COMMAND-WORD, STATUS-WORD, DATA-WORD	The <unsigned integer number> following the keyword must be 1 (one) except where the corresponding COMMAND, DATA or STATUS <type> definition in the corresponding <exchange model> is a RECORD. In which case the number identifies the corresponding <record element> of the RECORD. In each case the element can be rendered invalid by including faulty transmit/receive coding, faulty sync coding, invalid word parity, invalid word length or an invalid pulse code on a particular bit in the element. These are indicated respectively by T-R, SYNC, PARITY, LENGTH and BIT...PULSE-CODE. The value of the <unsigned integer number> between BIT and PULSE-CODE must be less than or equal to the string length of the corresponding record element if the element is of STRING <type>, otherwise it must be defined in the external <bus specification> documents referenced in the associated <establish protocol statement>.
5. If none of the standard entries is suitable to define a protocol failure mode on the bus which is required in a test, a <protocol parameter name> may be included.

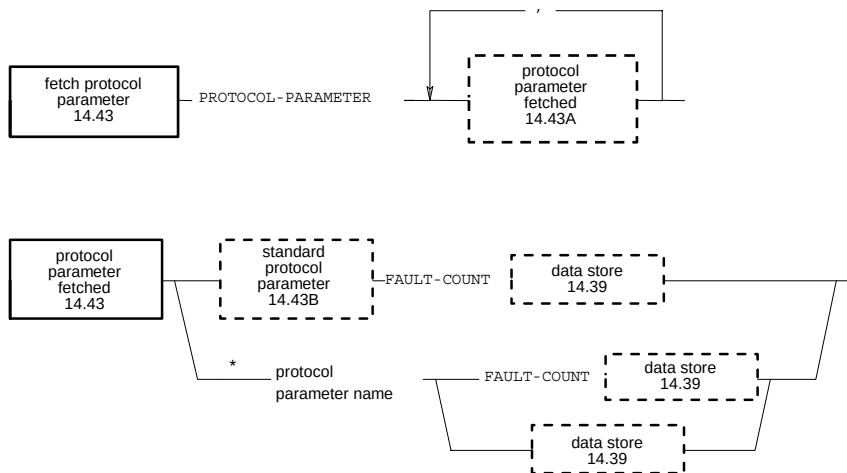
The <protocol parameter name> must be defined in an <extend atlas statement> and referenced in the corresponding field of the <establish protocol statement>.

14.0 FIELD AND SUBFIELD DEFINITION14.43 <fetch protocol parameter>14.43.1 Function

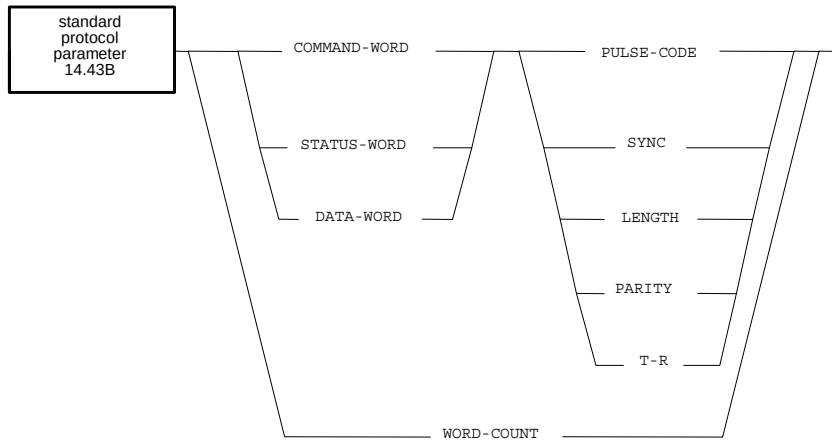
To provide the facility to extract from the databus test equipment the value or fault count for a given protocol parameter during the tests.

14.43.2 Formal Syntax

FSD / 3187

14.43.3 Syntax Diagram

Note: * The protocol parameter name must have been established as valid for the data bus in the associated ESTABLISH statement. For it to occur in an UPDATE statement of either sort, it must have been defined as UPDATABLE for that category of UPDATE. It is the only form of protocol parameter that can occur in a FETCH statement and then only if it has been established as monitored by the test equipment in the <do exchange statement>.

14.0 FIELD AND SUBFIELD DEFINITION**14.43.4 Rules**

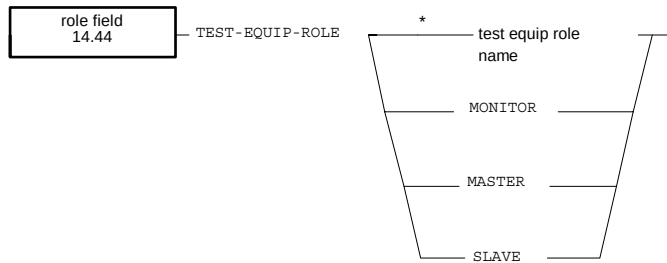
1. All entries which are used in <fetch protocol parameter> must be fully defined and typed in the <exchange model> section of the <establish protocol statement>. Any <data store>s which follow FAULT-COUNT must be of <type> INTEGER. For <protocol parameter name>s the type of the <data store> must be either the same <type> as in the <establish protocol statement> or the <data store> must be a reference to a <record field> or <array element> with the same <type> as that employed in the <establish protocol statement>.
2. The following standard entries with the listed general meaning may appear in <fetch protocol parameter>. Each one that is used must be precisely defined by the <external bus specification>s referenced in the associated <establish protocol statement>.
 - a) WORD-COUNT Enables the extraction of a count of the total number of word count errors for a given exchange or protocol during the operation of a <do exchange statement>.
 - b) COMMAND-WORD,
 DATA-WORD,
 STATUS-WORD Enables the extraction of a count of the total number of occurrences of a specified type of error on the specified word for a given <exchange> or <protocol> during the operation of a <do exchange statement>. The following predefined error types can be specified : faulty transmit/receive coding, faulty sync coding, invalid word parity, invalid word length or an invalid pulse code in the word. These are indicated respectively by T-R, SYNC, PARITY LENGTH and PULSE-CODE. The required parameters must be identified as FETCHABLE at the exchange and/or protocol level in the associated <establish protocol statement>.
3. If none of the standard entries is suitable to define a bus protocol failure mode which is required in a test, a <protocol parameter name> may be included. The <protocol parameter name> must be defined in an <extend atlas statement> and referenced in the corresponding field of the <establish protocol statement>. The actual value of the parameter (<data store> path) or a count of the errors detected for that parameter (FAULT-COUNT path) may be extracted depending on the associated <extend atlas statement> and <establish protocol statement>.

14.0 FIELD AND SUBFIELD DEFINITION14.44 <role field>14.44.1 Function

To specify the role that the test equipment is to take either as a default or explicitly for each exchange in a <do exchange statement>.

14.44.2 Formal Syntax

FSD / 3236

14.44.3 Syntax Diagram

Note: * The test-equip role name must have been established as valid for the databus in the associated <establish protocol statement>.

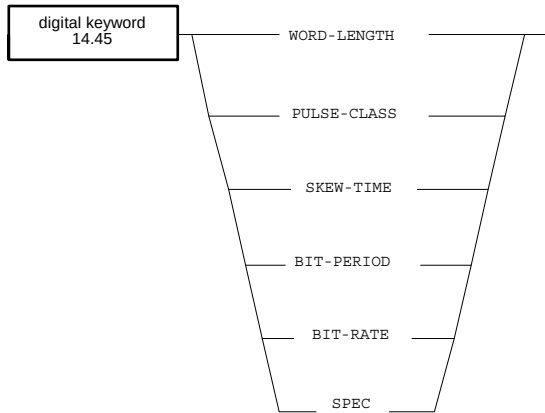
14.44.4 Rules

1. The entry in the <role field> must be defined by the specifications referenced by the associated <establish protocol statement>. It represents the function or role that the test equipment is to assume.
2. The general meaning of the standardized entries for the <role field> are as follows :

MASTER	The test equipment is in control of the databus and determines the activity on it.
SLAVE	Another device controls the databus and the test equipment acts as instructed by that device.
MONITOR	The test equipment generates none of the traffic on the databus. It plays a passive role in which it monitors the traffic and selects particular databus transactions for saving and future analysis.
3. The <test equip role name>, if used, must have been defined in an <extend atlas statement>.

14.0 FIELD AND SUBFIELD DEFINITION14.45 <digital keyword>14.45.1 Function14.45.2 Formal Syntax

FSD / 3249

14.45.3 Syntax Diagram

15.0 LANGUAGE COMPONENT SPECIFICATION**15.1 Authorized Characters****15.1.1 Function**

The seven-bit ASCII code (see Section 15.9) is authorized for use in ATLAS Statements. The character types are defined as follows:

1. Upper case letters: A through Z
2. Lower case letters: a through z
3. Numerals: 0 through 9
4. Symbols:

All column (col) and row references relate to Section 15.9.1 Table 15-2.

*	Asterisk, Multiplication and exponentiation sign (see Section 8.2.4.2, Rule 2)	(Col 2 Row 10)
,	Comma	(Col 2 Row 12)
.	Period	(Col 2 Row 14)
=	Equal sign	(Col 3 Row 13)
+	Plus sign	(Col 2 Row 11)
-	Minus sign (See Section 15.1.4, Rule 3)	(Col 2 Row 13)
/	slash, Division or lower case sign (See Section 15.1.4 Rule 2)	(Col 2 Row 15)
(	Left Parenthesis	(Col 2 Row 8)
)	Right Parenthesis	(Col 2 Row 9)
	Blank space	(Col 2 Row 0)
'	Apostrophe or single quote	(Col 2 Row 7)
\$	Currency character	(Col 2 Row 4)
!	Exclamation sign	(Col 2 Row 1)
"	Double quotes	(Col 2 Row 2)
#	Number sign (hash sign)	(Col 2 Row 3)
%	Per cent sign	(Col 2 Row 5)
:	Colon	(Col 3 Row 10)
;	Semicolon	(Col 3 Row 11)
<	Less than	(Col 3 Row 12)
>	Greater than	(Col 3 Row 14)
@	At sign	(Col 4 Row 0)
[	Left bracket (square)	(Col 5 Row 11)
]	Right bracket (square)	(Col 5 Row 13)
\	Backslash	(Col 5 Row 12)
^	Circumflex	(Col 5 Row 14)
_	Underscore	(Col 5 Row 15)
`	Grave accent	(Col 6 Row 0)
{	Left brace (curly bracket)	(Col 7 Row 11)
}	Right brace (curly bracket)	(Col 7 Row 13)
	Vertical line	(Col 7 Row 12)
~	Tilde	(Col 7 Row 14)
&	Ampersand	(Col 2 Row 6)
?	Question mark	(Col 3 Row 15)

5. Non-printing characters:
Those ASCII characters defined in rows 0 through 15 of columns 0 and 1 and row 15 of column 7 in Table 15-2.

15.0 LANGUAGE COMPONENT SPECIFICATION

15.1.2 Formal Syntax (not applicable)

15.1.3 Syntax Diagram (not applicable)

15.1.4 Rules

1. Applicability of the ATLAS characters to specific character sets is specified in Section 15.10. Otherwise applicability is specified by the syntactical definition of each ATLAS statement.
2. Lower case letters in UUT connector names may be represented in ATLAS connection fields by lower case letters.
3. The minus sign and hyphen are represented by the same character and will be interpreted according to the context of the statement.
4. Non-printing characters may be used in <character string>, <message text> or <msgchar> and shall be represented by the symbols shown in Table 15-2 enclosed in backslash symbols. If the backslash symbol itself is to be included, it must be enclosed in backslash symbols.

.....

15.0 LANGUAGE COMPONENT SPECIFICATION**15.2 Representation of Numbers and Other Data Types**

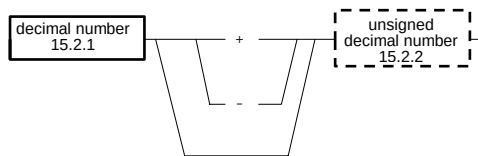
The following conventions are used to express values in the language. Signs are meaningful only for decimal and floating point numbers.

15.2.1 <decimal number>**15.2.1.1 Definition**

The written notation for representing a decimal quantity.

15.2.1.2 Formal Syntax

FSD / 3352

15.2.1.3 Syntax Diagram**15.2.1.4 Rules**

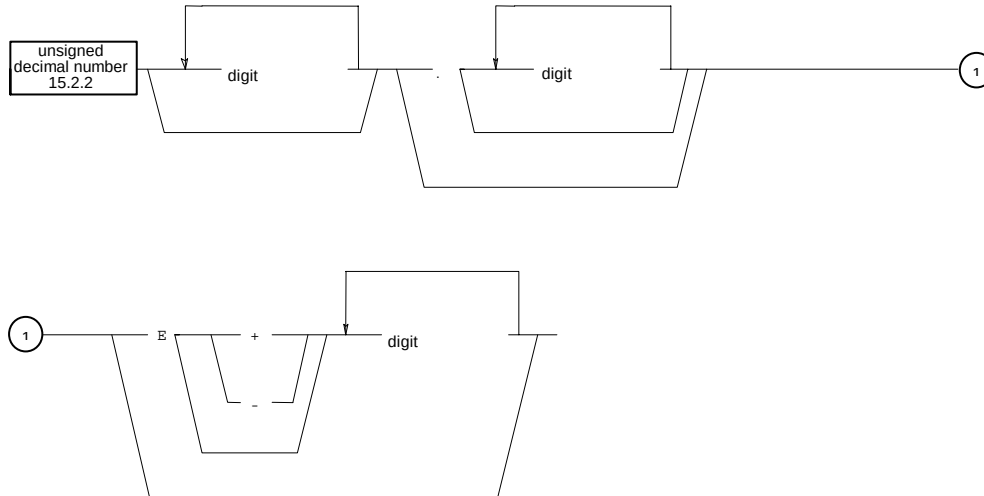
1. If both signs are absent the decimal number is assumed to be positive.
2. The size and precision of a decimal number that can be represented is implementation dependent.
3. A number corresponding to the form of <decimal number> is of type DECIMAL unless it also corresponds to the form of <unsigned integer number>, in which case it is of type INTEGER.
4. The decimal point, if used, is placed conventionally.
5. The integer following the character E (the exponent) is a power of 10 multiplier for the preceding decimal number (the mantissa). A <decimal number> which employs the E is known as a floating point decimal number. For example: 1.86E3 represents the number $1.86 * 10^3$

15.0 LANGUAGE COMPONENT SPECIFICATION**15.2.2 <unsigned decimal number>****15.2.2.1 Definition**

The written notation for representing a decimal number which is not preceded by a sign.

15.2.2.2 Formal Syntax

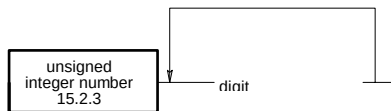
FSD /3357

15.2.2.3 Syntax Diagram**15.2.3 <unsigned integer number>****15.2.3.1 Definition**

The written notation for a <decimal number> with no fractional part and not preceded by a sign.

15.2.3.2 Formal Syntax

FSD / 3369

15.2.3.3 Syntax Diagram

15.0 LANGUAGE COMPONENT SPECIFICATION

15.2.3.4 Rules

1. The size of integer that can be represented is implementation dependent
2. A number corresponding to the form of <unsigned integer number> is of type INTEGER.

15.2.4 <binary number>

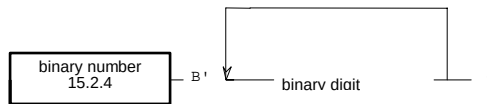
15.2.4.1 Definition

The written notation for representing a digital quantity by explicitly stating the value of each <binary digit>.

15.2.4.2 Formal Syntax

FSD / 3373

15.2.4.3 Syntax Diagram



15.2.4.4 Rules

1. A number corresponding to the form of <binary number> is of type STRING OF BIT.
2. The length of a STRING OF BIT represented by a <binary number> is equal to the number of <binary digit>s between the single quotes.

15.2.4.5 Notes and Examples

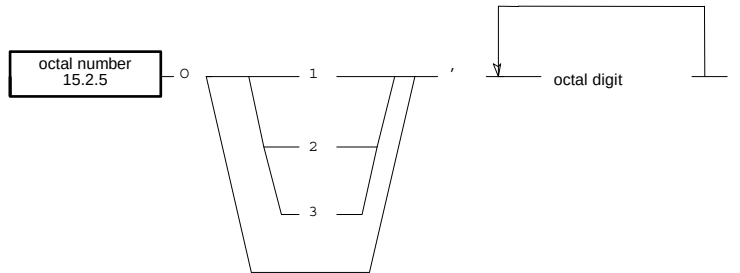
Example of a <binary number>: B'0110010'

15.0 LANGUAGE COMPONENT SPECIFICATION15.2.5 <octal number>15.2.5.1 Definition

The written notation for representing a digital quantity by grouping its <binary digit>s into groups of three from the right and expressing the pattern within each group by the decimal equivalent having weighted the bits from left to right by 4, 2 and 1.

15.2.5.2 Formal Syntax

FSD / 3377

15.2.5.3 Syntax Diagram15.2.5.4 Rules

1. A number corresponding to the form of <octal number> is of type STRING OF BIT.
2. The value of the optional digit between "O" and "" indicates the number of bits to be used to represent the left-hand <octal digit>.
3. If the optional digit between "O" and "" is absent, its value defaults to 3.
4. The length of a STRING OF BIT represented by an <octal number> containing n <octal digit>s with p as the explicit or implicit digit between "O" and "" is equal to:

$$3*(n-1)+p.$$

15.2.5.5 Notes and Examples

Examples of <octal number>s which have three bits representing the left hand <octal digit>:

O3'7601340' or O'7601340'

Example of <octal number> which has two bits representing the left hand <octal digit>:

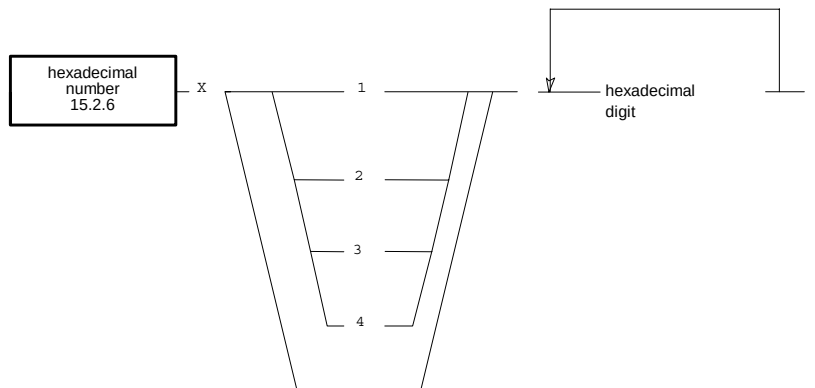
O2'2601340'

15.0 LANGUAGE COMPONENT SPECIFICATION15.2.6 <hexadecimal number>15.2.6.1 Function

The written notation for representing a digital quantity by grouping its <binary digit>s into groups of four from the right and expressing the pattern within each group by the equivalent <hexadecimal digit> having weighted the bits from left to right by 8, 4, 2 and 1. A <hexadecimal digit> is 0 thru 9 or A thru F where A thru F correspond to the decimal values 10 thru 15 respectively.

15.2.6.2 Formal Syntax

FSD / 3383

15.2.6.3 Syntax Diagram15.2.6.4 Rules

1. A number corresponding to the form of <hexadecimal number> is of type STRING OF BIT.
2. The value of the optional digit between "X" and "" indicates the number of bits to be used to represent the left-hand <hexadecimal digit>.
3. If the optional digit between "X" and "" is absent, its value defaults to 4.
4. The length of a STRING OF BIT represented by a <hexadecimal number> containing n <hexadecimal digit>s with p as the explicit or implicit digit between "X " and "" is equal to: $4*(n-1)+p$.

15.2.6.5 Notes and Examples

Examples of <hexadecimal number>s which have four bits representing the left hand <hexadecimal digit>:

X'93A5D3' or X'93A5D2'

Example of <hexadecimal number> which has three bits representing the left hand <hexadecimal digit>:

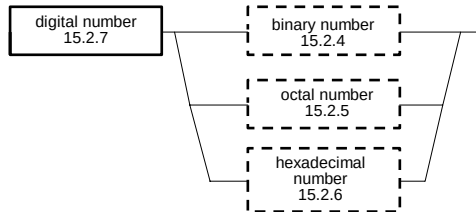
X3'53A5D2'

15.0 LANGUAGE COMPONENT SPECIFICATION**15.2.7 <digital number>****15.2.7.1 Definition**

A <digital number> can be either a <binary number>, an <octal number> or a <hexadecimal number>.

15.2.7.2 Formal Syntax

FSD / 3389

15.2.7.3 Syntax Diagram**15.2.8 <step>****15.2.8.1 Definition**

A <step> number is written as a six digit <unsigned integer number>.

A number corresponding to the rules of <step> is of type STEP.

15.2.9 <character string>**15.2.9.1 Definition**

A <character string> is any sequence of characters with the first character being an upper case letter C, the first non-blank character following the first character being an apostrophe and with the last character and no other intervening character being an apostrophe. A <character string> is of type STRING OF CHAR.

15.2.10 <connection>**15.2.10.1 Definition**

A <connection> is a <character string> which represents a connection on the UUT or on hardware introduced by a <define drawing statement>. The rules for writing <connection>s are included in Section 14.13.

A <character string> corresponding to the rules of Section 14.13 <conn> but not obeying the rules of Sections 15.2.1 thru 15.2.10 is of type CONNECTION.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.3 Flag and Statement Numbers****15.3.1 <fstatno>****15.3.1.1 Function**

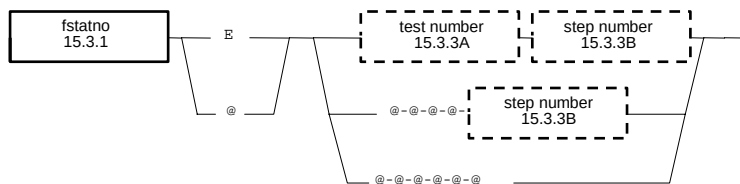
The flag and statement number field, <fstatno>, is used to identify ATLAS statements and structures.

15.3.1.2 Formal Syntax

FSD / 3397

15.3.1.3 Syntax Diagram

In the following diagram, all allowable blanks are represented by the @ character.

**15.3.1.4 Rules**

1. The first field of each statement is used for the flags. It is one character in length and is located in the first column of the first row of each statement. The field must be used either for one of the flag entries B, E, C or left blank.
2. The "E" flag indicates entry points where the test conditions are completely stated and are not dependent on previous tests in any way. Such points are the only ones where it is permissible to begin part of the complete test program.

Since the test must be valid starting at the "E" flag with no signals attached to the UUT, with no SETUP or other single action verb statements having been executed, and with no storage spaces having had data assigned to them other than by an initialization field in the preamble. It is normally good practice either to precede the "E" flagged statement immediately by either an unconditional GOTO statement or a REMOVE ALL statement or to make the "E" flagged statement a REMOVE ALL. It is strongly recommended that a <comment statement> be inserted immediately following the BEGIN ATLAS PROGRAM statement which lists all the "E" statements and indicates the section of the test procedure which starts at the "E" flag.

An "E" flag may only be used with a <commence main procedure statement>, a <begin block statement> or an unnested <main procedural structure> of an <atlas test specification structure>.

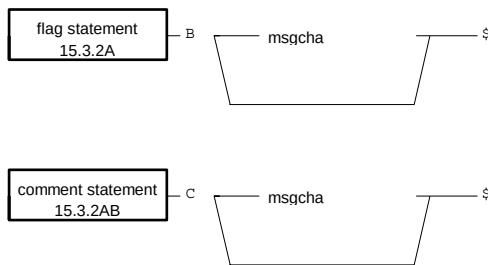
15.0 LANGUAGE COMPONENT SPECIFICATION**15.3.2 <flag statement> and <comment statement>****15.3.2.1 Function**

The <flag statement> is used either when the following statement needs two or more flags or when a B flag is required.

The <comment statement> is used to insert comments into the ATLAS test specification.

15.3.2.2 Formal Syntax

<flag statement>	FSD / 3422
<comment statement>	FSD / 3428

15.3.2.3 Syntax Diagram**15.3.2.4 Rules**

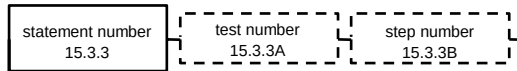
1. The first field of each statement is used for the flags. It is one character in length and is located in the first column of the first row of each statement. The field must be used either for one of the flag entries B, E, C or left blank.
2. A flag may be written as a separate statement immediately preceding the statement to which it applies. This will allow more than one flag to be applied to a single statement.
3. The "B" flag indicates the destination statement of a GO TO statement elsewhere in the procedure. Characters in the optional <msgchar> following the "B" flag and preceding the statement terminator are commentary and are ignored during translation. The "B" flag must always be written as a separate statement, and may be followed by commentary to indicate the origin statement of a program branch.
4. The "C" flag indicates that information in that statement is to be ignored during translation. Characters in the optional <msgchar> in a statement flagged "C" will appear only at the compiler input level as an aid to readability of the procedure. A <comment statement> may be inserted between any two statements in an <atlas test specification structure>.

15.0 LANGUAGE COMPONENT SPECIFICATION15.3.3 <statement number>15.3.3.1 Function

<statement number> is the number identifier of a statement in the ATLAS program.

15.3.3.2 Formal Syntax

FSD / 3410

15.3.3.3 Syntax Diagram15.3.3.4 Rules

1. The second field is the <statement number>. It is six characters long and provides a facility for a reference designator to be included in the program statement. The first four digits are called the <test number> and the remaining two digits are called the <step number>.
2. Each program step which is required to be referenced by another statement in the program must have a <statement number>, as must all "E" flagged statements unless the "E" flag is the whole statement in which case the statement at the end of the flag series must have a <statement number>. Other statements may be assigned a <statement number> which must exceed the preceding number used, but it is not necessary to use the next higher number.
3. A <statement number> is not allowed in the field following a "B" or "C" flag, but the statement following the "B" flagged statement must have a <statement number>.
4. If the <test number> of a numbered statement is the same as a previous statement, the first four characters of the <statement number> may be left blank.

15.3.3.5 Notes and Examples

1. The <test number> and <step number> designations within the <statement number> are arbitrarily assigned for convenience in referencing test statements. The method of assigning the numbers is left to the discretion of the test specification writer with the previously mentioned restrictions.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.4 Blank Space Requirements**

Blank spaces are used to improve readability, as subfield separators and to separate words within some language elements. For consistent machine processing of test specifications, one or more blank spaces are required between any two language elements except as specified below, a language element being any number, word, character, or subfield which is defined as a separate item in this specification. Also, one blank space is required between words within any language element which contains two or more words. For example, WAIT FOR is a language element with two words. One blank space is required between the two words WAIT and FOR. Blank spaces adjacent to all special characters that are not part of a language element, such as the comma, +, -, \$, and between alphabetic and numeric subfields are optional.

15.0 LANGUAGE COMPONENT SPECIFICATION15.5 <label>

ATLAS uses <label>s as identifiers for the following. The label must be established prior to its first use by a statement which is defined in the section listed in the corresponding "Established by" column.

	<u>Established by:</u>	<u>Defined in:</u>
<boolean variable>	6.23.3, 6.23.4	15.11.30
<bus mode name>	6.24	15.10.19
<bus parameter name>	6.15	15.10.20
<bus specification>	6.15	15.10.23
<configuration>	6.23.2	15.11.20
<constant identifier>	6.4	15.11.4
<digital sensor>	6.23.4	15.11.22
<digital source>	6.23.3	15.11.21
<dim name>	6.25	15.11.23
<drawing>	6.7	15.11.26
<drawing identifier>	6.7	15.10.4
<event>	6.18	15.11.14
<event indicator>	6.21	15.11.15
<event interval>	6.20	15.11.16
<exchange>	6.24	15.11.19
<exchange configuration>	6.26	15.11.29
<external digital specification>	6.23.3, 6.23.4	15.11.31
<external semantics specification>	6.25	15.11.24
<external syntax specification>	6.25	15.11.25
<function>	6.10.2	15.11.12
<modifier name>	6.25	15.10.11.1
<modifier descriptor name>	6.25	15.10.7.1
<module name>	4.1.3, 6.13	15.10.8
<noun name>	6.25	15.10.9.1
<parameter>	14.81 (6.9.2, 6.10.2)	15.6.5
<pin descriptor name>	6.25	15.10.10
<procedure>	6.9.2	15.11.11
<program name>	4.1.1	15.11.32
<protocol>	6.15	15.11.18
<protocol parameter name>	6.15	15.10.21
<record field>	6.4	15.11.8
<requirement>	6.12	15.11.28
<signal>	6.8	15.11.13
<timer>	6.17	15.11.17
<type identifier>	6.4	15.11.5
<variable identifier>	6.4	15.11.6

1. A unique character string, called <label>, is required in certain statements to provide an identifier for subsequent reference to the labelled information within the current program. A <label> may be selected at the writer's convenience and if properly selected will improve readability of the procedure. A <label> is formed by any combination of the characters listed in Section 15.1.1 type (1) through type (4) except the currency symbol and parenthesis marks. The first character and the last character of each <label> must be the apostrophe which may not be used as any other character in the <label>. There is no limit to the length of the <label>, but only the first sixteen characters, excluding the apostrophes, are considered significant in establishing the uniqueness of the <label>. At least one non-blank character must be used. The case of the letters is significant. There shall be no leading or trailing blank characters within a <label>.

15.0 LANGUAGE COMPONENT SPECIFICATION

2. Although user created labels are delimited by apostrophes, predeclared and predefined "label"s are written without apostrophes. Specifically these "label"s are:
 - a. The predefined functions of Section 8.2.4.4
 - b. The predeclared types and enumeration constants of Section 6.3.4.7.

15.5.1 Notes and Examples

In the following examples of <label>, the sixteen significant characters are underlined.

'HIGH VOLTAGE SOURCE NR. 1' and 'HIGH VOLTAGE SOURCE NR. 2' are not unique.

'HARDWARE' and 'HARDWARE' are unique.

The following example illustrates how proper selection of a <label> can provide additional useful information in a statement.

123456 CALCULATE, 'SUM' = 'FIRST NUMBER' + 'SECOND NUMBER' \$

15.6 Retention of Labeled Information

Labels are used throughout the language to represent different types of information. The retention period varies during compilation or execution of a test program depending on the type of <label>. Generally, labeled information is retained until the same <label> is assigned to different information. The various types of labels are referenced in statement diagrams as 'MEASUREMENT' and those <label>s as listed in Section 15.5.

15.6.1 Global/Local Considerations

A <label> is classified as being global or local. Once a particular variable name has been so defined, it will have the same characteristics or value each time it appears unless it is replaced by an identical <label> at a lower level structure.

A global <label> may be established with the attribute GLOBAL. In this case, the <label> has the same meaning throughout an entire <atlas test specification structure>. The <label> can be referenced by an appropriate ATLAS statement in the <atlas program structure> or <atlas module structure> in which it is declared or defined with the attribute GLOBAL. If the <label> is required to be referenced from outside the <atlas program structure> or the <atlas module structure> in which it is defined with the attribute GLOBAL, the structure which references it must contain a <define statement> or a <declare statement> for the <label> with the attribute EXTERNAL. A <label> may be established with neither GLOBAL nor EXTERNAL attributes. In this case, the scope of the <label> is the entire <atlas program structure> or <atlas module structure> and the data is retained throughout the execution of the <atlas test specification structure>. These two different scopes of global <label>s apply to all the following paragraphs.

A local <label> has a specific meaning throughout some portion of a test procedure, and may have a different meaning in a different part of the same test procedure.

The <configuration>, <digital sensor>, <digital source>, <drawing>, <event>, <event indicator>, <event interval>, <exchange>, <function>, <procedure>, <protocol>, <requirement>, <signal> and <timer> types of <label> are always global because the statements that establish them can only be in a <program preamble structure> or a <module preamble structure> and cannot be used in a <local preamble structure>.

Any <label> appearing in a <declare statement> within the <program preamble structure> or <module preamble structure> is global, excluding those appearing in a <declare statement> that is within a <define procedure structure>.

Any <label> appearing in a <declare statement> within a <define procedure structure> is local to that structure. A local <label> retains its meaning within any inner structure unless it appears in another <declare statement> within that structure.

15.0 LANGUAGE COMPONENT SPECIFICATION

Any <label> appearing in <declare statement>s within two or more structures of the same level has a separate meaning within each structure.

15.6.2 MEASUREMENT

'MEASUREMENT' is a permanent <label> of type DECIMAL. It refers to the result at the time of execution of the most recently executed sensor-type statement. A particular value of 'MEASUREMENT' established by a sensor-type statement is available for repeated use in such as <compare statement>s and <calculate statement>s until the next sensor-type statement is executed.

'MEASUREMENT' is a global <label> that is permanently available in the test environment. It is implicitly declared GLOBAL in the <atlas program structure> and implicitly declared EXTERNAL in each <atlas module structure>. There is only one such <label> for each ATLAS program and every measured value in a test execution, except those as noted for specific circumstances, is stored in this label and will displace any data previously stored in the recently executed sensor function; not the most recently compiled function.

'MEASUREMENT' is not used for analog measurements that return more than one measured value, for example; MEASURE, (SAMPLE), WAVEFORM,...).

15.6.3 Retention of Measured Sensor Values

In cases where it is necessary to retain a particular 'MEASUREMENT' value beyond the time when it would be replaced, the value must be assigned to an additional <label>. This can be accomplished in two ways. One is to include an optional subfield containing a <data store> label within the <measured characteristic> field in the sensor statement. The alternative is to use a <calculate statement>. The additional variable <label> must be of the same type as the 'MEASUREMENT'. Both syntactically and semantically 'MEASUREMENT' is a <data store> but with the special property that its value is changed by most sensor-type statements when they are executed with a few exceptions that are specifically defined in this specification.

Information assigned to a <label> is available for reuse in subsequent statements until the end of the test specification or until the same label is assigned other information. Information assigned to a local <label> within a <procedure> is only available within that procedure. A local <data store> can be made available outside the procedure by assigning it to a <parameter> label which appears in the RESULT section of the <define statement> or by assigning it to a global <label>.

15.6.4 <function>, <procedure>, <protocol> and <exchange>

<define statement>s establish and label segments of an <atlas test specification structure> so that the specification writer can write a segment the first time and use only a label where the segment is used subsequently in the <atlas test specification structure>. <define statement>s with the attribute EXTERNAL establish a label for segments of an <atlas test specification structure> which are defined outside of either its <atlas program structure> or its <atlas module structure>. These labels are called <function>, <procedure>, <protocol> or <exchange> depending upon the particular type of definition, and such information is retained until the <terminate statement> is executed.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.6.5 <parameter>**

A <label> that is local to the body of a <define procedure structure> or a <define function structure> that is used for communicating between the calling level and the defined <procedure> or <function>.

When a <procedure> is defined, it is convenient to designate certain values as variable so that the definition can be used in several subsequent places in the program with different actual values. This is accomplished by writing a <parameter> in place of the value in the <procedure body> and listing the labels in the formal <parameter> list. All <parameter> labels are local to the <procedure>. A label used as a <parameter> for one procedure may be used as a <parameter> label for another procedure or as any other type of label without ambiguity.

15.6.6 <data store>

Information, including the value of a 'MEASUREMENT', calculated arithmetic or Boolean values, or data entered can be placed and held in storage spaces identified by a <data store> label. By using this label in the proper context, the information can be retrieved for use.

The following types of data can be stored in a <data store>:

1. Any <decimal> and <floating point decimal number>
2. Any <digital number>
3. Any <decimal number> with integer values
4. Character strings for insertion into messages
5. Connections
6. A <statement number>
7. Boolean Values (TRUE or FALSE)

A label <array> must always appear first in a <declare statement>. In the declaration, the data type of <array> must be specified together with the number of elements in the <array>.

In certain contexts, only a single <array> element may be used. In other cases either the entire <array> or a portion of an <array> may appear.

There are many places in ATLAS statements where a <data store>, either a single element or group of elements, may appear. In the use of <data store> the specification writer must exercise care to ensure that the data being stored is of the proper type for the <data store> or that the data that will be returned from the <data store> is appropriate for the context of the ATLAS statement.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.6.7 Other Variables ()**

In both <signal> and <drawing> definitions variables are designated by parentheses, (). These variables have the same effect as those designated with <parameter> in <define procedure structure>s. In both <signal> and <drawing> definitions, the number of spaces between the parentheses is not significant.

A <define signal statement>, a <define procedure statement> or a <define drawing statement> may have certain portions of the statement left as variables. These items are like buckets that are filled by the procedural statement that uses the <signal> or <drawing> definition and by <parameter> in <define procedure structure>s.

The <parameter> labels are local to the <define procedure structure>. In other words, a number that is passed to a <parameter>, e.g., 'BUCKET' in a particular <define procedure structure> will not displace information in another <parameter> named 'BUCKET' in another <define procedure structure> or a <data store> named 'BUCKET' in a <define procedure structure>.

15.6.8 <measured characteristic mnemonic>

The noun <modifier mnemonic> of the signal characteristic which is being measured.

15.6.9 <monitored characteristic>

The noun modifier mnemonic of the signal characteristic which is being sensed by an <event> detector.

15.6.10 <terminal identifier>

The identifier related to connection ports on hardware described by a <define drawing statement>. If introduced by an * it is analogous to a pin descriptor of a virtual resource.

15.7 Range Subfield

When a characteristic is designated variable in a <define signal statement> a range minor field may be included to state the lowest and highest values which will be assigned to that characteristic in subsequent statements. These values are separated by the word TO and follow the word RANGE in this subfield. The range field also includes an expression of the required resolution of the function. The word BY precedes the number and dimensions of the specified resolution, for example; VOLTAGE ()V RANGE 5 V TO 50 V BY 0.001 V. Use of the range field with a sensor function implies the total capability of all scales of the measurement device.

15.7.1 Range Requirement for <data store> or <parameter>

A <data store> or <parameter> <label> which is used in place of a signal in a signal oriented statement must have a RANGE subfield to indicate the expected magnitudes of the variable. This additional information need not be given in non-signal oriented statements, for example; <calculate statement> etc.

15.8 Dimensions

The vocabulary includes dimensional units which are used in the modifier fields to fix the units of the value stated. Dimensional units are included with the modifier value where appropriate. The authorized dimensional units are listed and defined in Table 15-1. The use of units conforming to International Standard ISO 1000 is preferred. These are listed in the table under the headings 'SI Units' and 'Selection of multiples of SI Units'. Some other units outside of SI which are recognized by the International Committee of Weights and Measures (CIPM) are listed because they have such practical importance that they must be retained for general use or for use in specialized fields of scientific research. Further units outside of the SI which are recognized by ATLAS for use in the aerospace and defense industries are also listed in the table.

15.0 LANGUAGE COMPONENT SPECIFICATION

Quantity	Basic Units	ISO 1000	Relationship	**0	**3	**6	**9	**12	**_2	**_3	**_6	**_9	**_12
	percent			PC									
	dimensionless												
Rotational Frequency	per second			SEC-1									
	per minute			MIN-1									
Temperature	degree centigrade (Celsius)		SI supplied unit	DEGC									
	degree Fahrenheit			DEGF									
	degree Kelvin		SI supplied unit	DEGK									
	degree rankine			DEGR									
Time	second	NO	SI supplied unit	SEC	KSEC					MSEC	USEC	NSEC	PSEC
	minute		1 MIN = 60 SEC	MIN									
	hour	NO	1 HR = 60 MIN	HR									
	day	NO	1 D = 24 HR	D									
Velocity Linear	meters per second	NO	SI supplied unit	M/SEC									
	meters per hour	NO	1 M/HR = 1/3600 M/SEC	M/HR	KM/HR								
	feet per second			FT/SEC									
	mach number			MACH									
	knot		1 KT = 0.51444 M/S	KT									
Voltage	volt	YES	SI basic unit	V	KV					MV	UV		

15.0 LANGUAGE COMPONENT SPECIFICATION**15.8.1 Units of Rate of Change**

Units of rate of change may be formed from any of the units listed in Table 15-1. Rate units are written by adding the slash character and appropriate units of time to the base unit from the table. For example; m/s, rad/s, and in aerospace and defense applications; FT/SEC, V/MSEC.

Units of acceleration are formed by squaring the unit of time. For example; metres per second squared is written m/s². In aerospace and defense applications, acceleration units may be formed by adding another slash and unit of time to the rate convention, for example; DEG/SEC/SEC, and the following special cases of angular rate may be used:

RPS for revolutions per second in lieu of REV/SEC
 RPM for revolutions per minute in lieu of REV/MIN
 RPH for revolutions per hour in lieu of REV/HR

15.8.2 Units which are a Product

Units which are a product of others (e.g., torque which is newton-meters) shall be formed from the units listed in Table 15-1. SI Units comprising a product shall be separated by a stop. For example; N.m

15.8.3 Units which are not pure quotients

In order to differentiate between function quantities and quantities which are quotients of two or more different dimensional quantities, parentheses are used. Thus:

1. For quotients of two or more different dimensional quantities:

Radiant Intensity, measured in watts per steradian - the units should be written as W/SR.

2. For functions which are not pure quotients:

- a. Modulation Transfer Function is a function of a pure number normalized to unity representing modulation of a radiated signal with respect to spatial frequency in cycles per distance, usually measured in milli-meters. The dimensional representation of this function is:

<unsigned decimal number>/(CYC/mm)

- b. Line Spread Function is a function of illuminance with respect to distance, usually measured transversely in milli-meters, from the center of a line or slit in the object plane of an optical system. The dimensional representation of this function is:

For illuminance per milli-meter (LM/M²)/MM

15.8.4 Square and Cubic Units

Area and volume units can be formed from distance units by appending the numbers 2 or 3 to the appropriate distance mnemonic. For example:

square meter is written m² and cubic meter is written m³

15.0 LANGUAGE COMPONENT SPECIFICATION

15.8.5 <dim>

Any noun modifier dimensional unit listed in Section 15.8 or a <dim name>.

For <procedural statements, signal>, <dim> must be null (non-existent or blank) when and only when the dimension selected from Table 15-1 is "dimensionless".

For all other statements which employ <dim> it may be null or any dimension from Table 15-1.

15.8.5.1 <dim name>

Any unique sequence of upper case alphabetical characters and embedded hyphens used to identify a new dimensional unit established in an <extend atlas statement> that introduces new noun-modifiers.

15.8.6 <freq dimension>

See Table 15-1 for Frequency entries.

15.8.7 <time dimension>

See Table 15-1 for Time entries.

15.8.8 Representation of Reciprocals

When representing the reciprocal of distance for the dimensions of the resolving power of an optical system, a single slash is used before the dimension thus:

/M /MM

meaning reciprocal meter and reciprocal millimeter respectively. Using a slash in this way agrees with the use of slash in ATLAS meaning arithmetic division.

15.9 Character Code Definitions

The ATLAS digital language has the ability to define alphanumeric and control characters, coded in various standard forms, that must be transmitted to a UUT as stimuli or read from a UUT as a response. Existing standard definitions for these codes are used in ATLAS.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.9.1 ASCII(American National Standard Code for Information Interchange)**

A 7-bit ASCII code is defined in approved USA standards.

The ASCII code system, including the bit positions, are defined in Table 15-2 for the Seven Bit ASCII Code.

Notes:

1. For reference; American National Standards Institute, "USA Standard Code for Information Interchange", X.3.4-1977.
2. Bit Significance;

Left Most Bit (LMB)	Right Most Bit (RMB)
Bit Position	Bit Position
<--- 8 7 6 5 4 3 2 1 --->	
Most Significant Bit (MSB)	Least Significant Bit (LSB)
3. Control characters such as ASCII EOT or NUL may be written in ATLAS by enclosing the character in backslash symbols (See Section 15.1.4, Rule 4).

15.0 LANGUAGE COMPONENT SPECIFICATION**Table 15-2 Seven Bit ASCII Code**

	b ₇	0	0	0	0	1	1	1	1
	b ₆	0	0	1	1	0	0	1	1
	b ₅	0	1	0	1	0	1	0	1
b ₄ b ₃ b ₂ b ₁	COLUMN ROW	0	1	2	3	4	5	6	7
0 0 0 0	0	NUL	DLE	SP	0	2	P	‘	p
0 0 0 1	1	SOH	DC ₁	!	1	A	Q	a	q
0 0 1 0	2	STX	DC ₂	“	2	B	R	b	r
0 0 1 1	3	ETX	DC ₃	#	3	C	S	c	s
0 1 0 0	4	EOT	DC ₄	\$	4	D	T	d	t
0 1 0 1	5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0	6	ACK	SYN	&	6	F	V	f	v
0 1 1 1	7	BEL	ETB	,	7	G	W	g	w
1 0 0 0	8	BS	CAN	(	8	H	X	h	x
1 0 0 1	9	HT	EM	)	9	I	Y	i	y
1 0 1 0	10	LF	SUB	*	:	J	Z	j	z
1 0 1 1	11	VT	ESC	+	;	K	[	k	{
1 1 0 0	12	FF	FS	,	<	L	\	l	
1 1 0 1	13	CR	GS	-	=	M	]	m	}
1 1 1 0	14	SO	RS	.	>	N	^	n	~
1 1 1 1	15	SI	US	/	?	O	-	o	DEL

15.0 LANGUAGE COMPONENT SPECIFICATION

15.10 Special Character Sets

The content of each character set is defined in Chapter 18, Section 18.6.12, "Basic Symbols / Character Sets", of the Formal Syntax Definition (FSD) in Chapter 18, Section 18.6, "SYNTAX DEFINITION".

15.10.1 <character string>

The sequence of characters with the first character being an upper case letter C and the first non-blank character following the first character being an apostrophe and the last character and no other character being an apostrophe as defined in the FSD.

15.10.2 <message text>

The sequence of characters as defined in the FSD which provides textual information in an <end if statement>, <end while statement> or <end for statement>.

15.10.3 <msgchar> (message character)

The sequence of characters as defined in the FSD which provides textual information in a <comment statement> or <flag statement>.

15.10.4 <drawing identifier>

The sequence of characters as defined in the FSD which represent the particular identifier and/or title given to a drawing/document or a set thereof as specified in a <define drawing statement>. The drawing contains the additional test information that cannot be expressed in Standard ATLAS statements.

15.10.5 <modifier descriptor>

The sequence of characters as defined in the FSD which is used to indicate one of a set of possible classes for a modifier. The permitted <modifier descriptor>s are those listed in Chapter 17 as related to the associated modifier.

15.10.5.1 <modifier descriptor name>

The unique character string as defined in the FSD which identifies a new modifier descriptor established in an <extend atlas statement>. Only the first sixteen characters, including any embedded hyphens, are significant.

15.10.6 <module name>

The sequence of characters as defined in the FSD which is used as a global label to identify a module of the test requirement.

15.10.7 <noun>

Any of the nouns listed in Chapter 16 or a <noun name>.

15.10.7.1 <noun name>

The unique sequence of characters as defined in the FSD which is used to identify a new noun being introduced by an <extend atlas statement>. Only the first sixteen characters of the noun are significant.

15.10.8 <pin descriptor>

Any of the pin descriptors listed in Section 14.14 or a <pin descriptor name>.

15.0 LANGUAGE COMPONENT SPECIFICATION

15.10.8.1 <pin descriptor name>

The unique sequence of characters as defined in the FSD which identifies a new pin descriptor being introduced by an <extend atlas statement>. Only the first sixteen characters including embedded hyphens are significant.

15.10.9 <mnemonic>

Any of the modifier mnemonics listed in Chapter 17 or a <modifier name>.

15.10.9.1 <modifier name>

The unique sequence of characters as defined in the FSD which identifies a new noun modifier being introduced by an <extend atlas statement>. Only the first sixteen characters including any embedded hyphens are significant.

15.10.10 <mod type>

Any type code listed under numeric or array reference in Table 16-0 in Chapter 16.0.

15.10.11 <digit>

Any of the numerals "0" thru "9".

15.10.12 <binary digit>

Either "1" or "0".

15.10.13 <octal digit>

Any of the numerals "0" thru "7".

15.10.14 <hexadecimal digit>

Any of the numerals "0" thru "9" or the upper case letters "A" thru "F".

15.10.15 <test equip role name>

The unique sequence of characters as defined in the FSD which identifies a new role which the test equipment can perform during a databus test. It must be introduced using the <databus extension> path of the <extend atlas statement> and must be fully defined in a referenced external specification document. Only the first sixteen characters, including any embedded hyphens, are significant.

15.10.16 <bus mode name>

The unique sequence of characters as defined in the FSD which identifies a new bus mode which a databus can utilize during databus testing. It must be introduced using the <databus extension> path of the <extend atlas statement> and must be fully defined in a referenced external specification document. Only the first sixteen characters, including any embedded hyphens, are significant.

15.10.17 <bus parameter name>

The unique sequence of characters as defined in the FSD which identifies a new bus parameter which is required during databus testing. It must be introduced using the <databus extension> path of the <extend atlas statement> and must be fully defined in a referenced external specification document. Only the first sixteen characters, including any embedded hyphens, are significant.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.10.18 <protocol parameter name>**

The unique sequence of characters as defined in the FSD which identifies a new databus protocol parameter which is required during databus testing. It must be introduced using the <databus extension> path of the <extend atlas statement> and must be fully defined in a referenced external specification document. The permitted characters are an upper case letter followed by an unlimited number of characters selected from uppercase letters, numerals and single hyphens. Only the first sixteen characters, including any embedded hyphens, are significant.

15.10.19 <bus specification>

A <label> that is used in an <establish protocol statement> to reference a document that further specifies the <protocol> defined by the statement in non-ATLAS language.

15.11 Miscellaneous Language Elements**15.11.1 <reference phase>**

One of the reference phases listed in <conn set>, Section 14.14.1.2.

15.11.2 <constant identifier>

A <label> that is established by a <declare statement> to represent the categories of <constant> as defined in <calculate statement>.

15.11.3 <type identifier>

A <label> that is established by a <declare statement> to represent a data type.

15.11.4 <variable identifier>

A <label> that is established by a <declare statement> to represent a variable of DECLARED <type> and <initial> conditions.

15.11.5 <enumeration element>

One of the set of constants which an ENUMERATION data type can assume as established by the corresponding <declare statement>. An <enumeration element> is either a <label> which has been declared as an <enumeration element> or a <pre-declared enumeration>.

15.11.6 <record field>

A <label> that represents the field within the <structured type> RECORD which is related to a <variable identifier> that is established by a <declare statement>.

15.11.7 <number>

A number in one of the representations defined in Section 15.2.

15.11.8 <modifier mnemonic>

Either the mnemonic for a noun modifier listed in Chapter 17 or a <modifier name> established in an <extend atlas statement>.

15.0 LANGUAGE COMPONENT SPECIFICATION

15.11.9 <procedure>

A <label> that is established by a <define procedure statement> to identify a sequence of test statements that can be executed each time the <procedure> label is referenced.

15.11.10 <function>

A <label> that is established by a <define function statement> to identify a sequence of test statements that perform a mathematical manipulation on data each time the <function> label is referenced.

15.11.11 <signal>

A <label> that is established by a <define signal statement> to identify one or more adjacent fields of an ATLAS source, sensor or load type statement whereby when a strict character sequence substitution is performed complete and legal ATLAS statements are generated.

15.11.12 <event>

A <label> that is established by an <identify statement> to identify a particular point in time that may or may not be related to specific signal condition(s) that are derived from the test requirements.

15.11.13 <event indicator>

A <label> that is established by an <identify event indicator statement> to identify a particular Boolean variable as being that which indicates the occurrence of an <event>.

15.11.14 <event interval>

A <label> that is established by an <identify event interval statement> to identify a particular region of time from a specified <event> or between two specified <event>s.

15.11.15 <timer>

A <label> that is established by an <identify timer statement> to identify a constantly running process whereby times for critical testing actions can be precisely and unambiguously determined from a known datum.

15.11.16 <protocol>

A <label> that is established by a <define protocol procedure statement> to identify a set of conventions that govern the format and relative timing of message exchange between one process and another which are in communication.

15.11.17 <exchange>

A <label> that is established by a <define exchange statement> to identify an exchange of data on a communication bus.

15.11.18 <configuration>

A <label> that is established by a <define digital configuration statement> to identify a sequence of ATLAS statements which are required for the testing of circuits that process signals in digital format.

15.11.19 <digital source>

A <label> that is established by a <define digital source statement> to identify a set of digital source characteristics and associated connections which are required for the testing of circuits that process signals in digital format.

15.0 LANGUAGE COMPONENT SPECIFICATION**15.11.20 <digital sensor>**

A <label> that is established by a <define digital sensor statement> to identify a set of digital sensor characteristics and associated connections which are required for the testing of circuits that process signals in digital format.

15.11.21 <dim name>

Any sequence of characters except comma, apostrophe, currency symbol and non-printing characters. A <label> that is introduced by an <extend atlas statement> to identify a dimension that applies to a <modifier name>.

15.11.22 <external semantics specification>

A text string that identifies a specific document in which the meaning of the extended vocabulary, NOUNS, MODIFIERS, CONNECTIONS and DIM-SPEC, is defined in the context of the related test discipline which requires the semantic extension to Standard ATLAS.

15.11.23 <external syntax specification>

A text string that identifies a specific document in which the meaning of the extended syntax, SPECIAL-SYNTAX, is defined in the context of the related test discipline which requires the syntactic extension to Standard ATLAS.

15.11.24 <drawing>

A <label> that is established by a <define drawing statement> to identify additional test information that is documented in the form of a drawing which cannot be expressed in Standard ATLAS statements.

15.11.25 <device identifier name>

An <enumeration element> from a previously declared enumeration variable which is used to identify a databus device or sub-device which may act as a TALKER, a LISTENER or both during a databus test.

15.11.26 <requirement>

A <label> that is established by a <require statement> to identify a virtual test resource. The virtual test resource may or may not correspond to a single real test resource.

15.11.27 <exchange configuration>

A <label> that can be introduced by a <define exchange configuration statement> to identify a set of databuses which are referenced by a <do exchange statement>.

15.11.28 <boolean variable>

A <label> of a <variable identifier> of <type> BOOLEAN that is used in a <define digital source statement> or a <define digital sensor statement>. It is set to TRUE when the ILLEGAL STATE INDICATOR relating to either a <digital source> or a <digital sensor> indicates that its associated source or sensor signal value does not conform to that specified.

15.11.29 <external digital specification>

A <label> that is used in either a <define digital source statement> or a <define digital sensor statement> to reference a document that further specifies in non-ATLAS language either the <digital source> or the <digital sensor> as defined by these statements.

15.0 LANGUAGE COMPONENT SPECIFICATION

15.11.30 <program name>

A <label> that can be introduced by a <begin atlas program statement> to identify an ATLAS test requirement.

15.12 Miscellaneous Keywords

15.12.1 MANUAL-INTERVENTION

1. Definition

A manual action that is performed by a human test operator during the testing process in response to a message that is displayed by the test requirement.

15.12.2 MAX-TIME

1. Definition

The maximum value for a time duration related to an escape or override action, expiry of which forces the continuation of a test sequence in the event that a UUT response is not as specified.

16.0 NOUNS AND THEIR MODIFIERS

16.0 Nouns and their Modifiers

The following is the list of nouns which may be used, in general, with any source, sensor, or load statement. Each noun is listed in its ATLAS form and is followed by its definition and a list of noun-modifiers which may be used with that noun. Each modifier and its ATLAS mnemonic form is given along with the appropriate quantity and other information as noted below. Definitions of the modifiers and permissible prefixes and suffixes are described in Chapter 17. To distinguish between the IMP modifier that is used without a prefix and the IMP modifier that must be used with a prefix, the latter is shown as xxx-IMP.

Modifier Type

The Modifier Type defines the type of information contained in the modifier field and the associated modifier field format, or syntax. The modifier lists in this section show the modifier type in terms of a Type Code, which is defined as follows:

Table 16-0 Modifier Type Codes and Typical Formats

<u>Item</u>	<u>Modifier Type</u>	<u>Type Code</u>	<u>Examples</u>
1	Numeric		
1.1	Decimal (or Real)		R VOLTAGE 5.5V, CURRENT MAX 2 A
1.2	Integer	I	INDEX 5, BURST-COUNT 5 PULSES
1.3	(not used)		
1.4	Digital (String of Bit)	SB	
1.5	String of char	SC	IDENT-SIG C ' '
2	Array Reference		
2.1	Decimal (or Real)	RA	Voltage <array> (x thru y) *
2.2	Integer	IA	
2.3	(not used)		
2.4	Digital (string of bit)	AB	
2.5	String of char	AC	
3	Modifier Descriptor		MD MODE ACTIVE
4	Mnemonic Only	MO	THREE-PHASE-WYE
5	Special Syntax	SS	

Note: * The following modifier field forms are excluded from the RA type: VOLTAGE value, VOLTAGE <data store>, VOLTAGE MAX or MIN or RANGE.

Item 5 in this tabulation identifies those modifiers that require special unique diagrams and Formal Syntax Definition (FSD) structures to define their syntax. Modifiers in the other groups conform to existing syntax structures.

Modifier Usage

Nouns are generally applicable to both UUT stimulus and response functions. Their modifiers may be used in the <require control>, <require capability>, or <require limit> clauses and as the measured characteristic mnemonic in the <require statement>, and as statement characteristics and as the measured characteristic mnemonic in the test action statements.

16.0 NOUNS AND THEIR MODIFIERS

Usage options for individual modifiers are shown in the modifier lists of this section by the following Usage Codes:

- S - Modifier applicable to UUT Stimulus functions. Valid when the associated noun is used with a SOURCE type of virtual test resource, except as specifically noted.
- R - Modifier applicable to the statement characteristics for UUT Response functions. Valid when the associated noun is used with a SENSOR type of virtual test resource, except as specifically noted.
- M - Modifier applicable to a measured characteristic. Valid when the associated noun is used with a SENSOR type of virtual test resource, except as specifically noted.

Quantities

The modifier lists in this section identify the quantities for each modifier. Quantity entries in the modifier lists give direct linkage to the quantity column in Table 15-1, Authorized Quantities and their Dimensions, in Section 15.8. Any of the associated basic units against the quantity column entry are valid and any of their associated dimensional mnemonics are permitted. For example, when the quantity Frequency is listed, then the basic units of Hertz and Pulses per Second are both allowed, and the following mnemonics are permitted: HZ, KHZ, GHZ, PPS and KPPS.

When no quantities are specified for a given modifier, the units are considered to be null, that is, they are dimensionless. Units formed by the division of one quantity by another are shown in the modifier lists as needed.

Formal Syntax

The Production Rules for the complete listings of ATLAS Nouns and Noun-Modifiers in the Formal Syntax Definition (FSD) begin on the following lines in Section 18.6, SYNTAX DEFINITION:-

Nouns	FSD / 3780
Noun-Modifiers	FSD / 3842

.....

16.0 NOUNS AND THEIR MODIFIERS**16.1 AC SIGNAL (Alternating Current Signal)****16.1.1 Definition**

A sinusoidal time-varying electrical potential.

16.1.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AC Current	CURRENT	R	SRM	Current
AM Component	AM-COMP	R	-RM	Voltage, Ratio
Bandwidth	BANDWIDTH	R	SRM	Frequency
Burst Length	BURST	I	SR-	Burst length
DC Offset	DC-OFFSET	R	SRM	Voltage, Current
Distortion (Total)	DISTORTION	R	-RM	Voltage, Ratio *, Current, Power
FM Component	FM-COMP	R	-RM	Frequency, Ratio
Frequency	FREQ	R	SRM	Frequency
Frequency Window	FREQ-WINDOW		RA -R-	Frequency
Harmonic Phase	HARM-###-PHASE	R	-RM	Angle (plane)
Harmonic Voltage	HARM-###-VOLTAGE	R	-RM	Voltage, Ratio
Harmonics	HARMONICS	R	SRM	Voltage, Ratio Current, Power
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Voltage, Ratio *, Current, Power
Noise Amplitude Density	NOISE-AMPL-DENS	R	SRM	Noise Amplitude Density
Noise Power Density	NOISE-PWR-DENS	R	SRM	Power Amplitude Density
Non-Harmonics	NON-HARMONICS	R	SRM	Voltage, Ratio *, Current, Power
Period	PERIOD	R	SRM	Time
Phase Angle	PHASE-ANGLE	R	SRM	Angle (plane)
Phase Jitter	PHASE-JIT	R	SRM	Angle (plane)
Power	POWER	R	SRM	Power
Reactance (real)	xxx-REACTANCE			R SR- Impedance
Resistance	xxx-RES	R	SR-	Impedance (real)
Standing Wave Ratio	SWR	R	SRM	Ratio
Three-Phase-Delta Connection	THREE-PHASE-DELTA	MO	SR-	(None)
Three-Phase-Wye Connection	THREE-PHASE-WYE	MO	SR-	(None)
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Voltage	VOLTAGE	R	SRM	Voltage

Where: ###An <unsigned integer number> must be included at this point indicating the harmonic which is being specified.

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

Note: *Ratio values are referenced to the fundamental amplitude.

16.0 NOUNS AND THEIR MODIFIERS**16.1.3 Rules**

1. The following is applicable for a source statement using AC SIGNAL:

<u>CURRENT</u>	<u>VOLTAGE</u>	<u>UUT-IMP</u>	<u>RESULT</u>
-	-	-	Not Recommended
-	-	*	Not Recommended
-	*	-	Current MAX, constant voltage
*	*	*	Voltage constant, current range known
*	-	-	Voltage MAX, current constant
*	-	*	Current constant, voltage range known
*	*	-	Not Recommended
*	*	*	Not Recommended

Note 1 * = included as a modifier;
 - = not included as a modifier.

Note 2 Wherever a MAX result is possible, a MAX modifier must be included.

16.0 NOUNS AND THEIR MODIFIERS**16.2 AMBIENT CONDITIONS****16.2.1 Definition**

The characteristics of the ambient environment at the time the <atlas test specification structure> is executed.

16.2.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Barometric Pressure	BAROMETRIC-PRESS	R		S R M Pressure
Relative Humidity	RELATIVE-HUMIDITY	R	S R M	Ratio
Temperature	TEMP	R	S R M	Temperature
Transition Time	TRANS-TIME	R	S - -	Time

16.2.3 Rules

1. This noun uses the pin descriptor TO, and may be followed by the special <connection> ATMOS.

16.2.4 Notes and Examples

The following are examples of AMBIENT CONDITIONS statements:

```
111122 MEASURE, (RELATIVE-HUMIDITY INTO 'HUMVAL'), AMBIENT CONDITIONS,
          RELATIVE-HUMIDITY RANGE 20 PC TO 100 PC,
          CNX TO CELL-AMB-HUM $
```

```
101000 APPLY, AMBIENT CONDITIONS,
          TEMP 60 DEGC, RELATIVE-HUMIDITY 80 PC,
          CNX TO ATMOSPHERE $
```

16.0 NOUNS AND THEIR MODIFIERS16.3 AM SIGNAL (Amplitude Modulation Signal)16.3.1 Definition

A continuous sinusoidal carrier signal whose amplitude is varied in accordance with the amplitude of a modulating signal.

16.3.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Amplitude Modulation	MOD-AMPL	R	SRM	Ratio, Voltage
Bandwidth	BANDWIDTH	R	SRM	Frequency
Burst Length	BURST	I	SR-	Burst Length
Carrier Amplitude	CAR-AMPL	R	SR-	Voltage, Ratio, Power
Carrier Frequency	CAR-FREQ	R	SR-	Frequency
FM Component	FM-COMP	R	-RM	Frequency
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Harmonics	HARMONICS	R	SR-	Ratio, Voltage, Current, Power
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Modulation distortion	MOD-DIST	R	-RM	Ratio
Modulation frequency	MOD-FREQ	R	SR-	Frequency
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Noise Amplitude Density	NOISE-AMPL-DENS	R	SRM	Noise Amplitude Density
Noise Power Density	NOISE-PWR-DENS	R	SRM	Noise Power Density
Non-Harmonics	NON-HARMONICS	R	-RM	Ratio, Voltage, Current, Power
Phase Jitter	PHASE-JIT	R	-RM	Angle (plane), Ratio
Power	POWER	R	SRM	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Standing Wave Ratio	SWR	R	SRM	(None)
Transition Time	TRANS-TIME	R	S--	Time

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.4 COMMON****16.4.1 Definition**

A conducting device that is used as a common return for an electrical circuit thereby providing an arbitrary zero potential.

16.4.2 Noun Modifier Set

There are no modifiers for COMMON.

16.4.3 Rules

1. COMMON is defined and listed both in the noun list and the <conn set>. COMMON is a phenomenon which exists in the test environment and must be discretely attached to the UUT in order to have an effect on the UUT. If COMMON is used in the noun field of a source or load type statement, it implies that the function is to be attached to the specified UUT pins. If COMMON is used in the <connection> field of some other source, sensor, or load type function, it implies that the specified node of the test device is to be attached to the COMMON function of the test equipment.

16.4.4 Notes and Examples

Some examples are in order to clarify the proper use of EARTH and COMMON. In the diagram associated with each sample statement, the broken line represents the hook-up caused by the statement.

Example 1: APPLY, COMMON, CNX J1-2 J1-3 \$

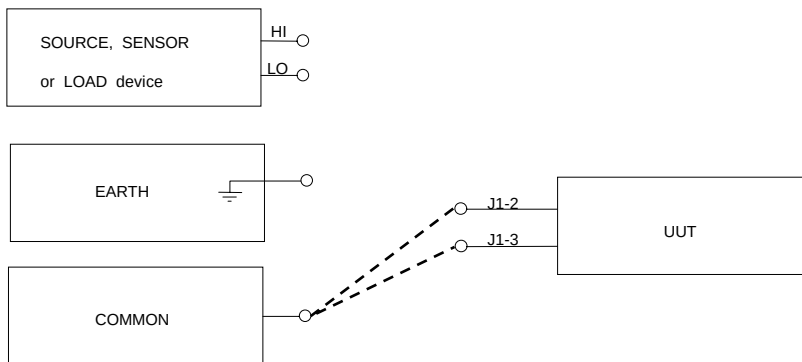


Figure 16-1: Illustration for COMMON, Example 1

16.0 NOUNS AND THEIR MODIFIERS

Example 2: APPLY, DC SIGNAL, VOLTAGE 5V,
CNX HI J1-2 LO EARTH \$

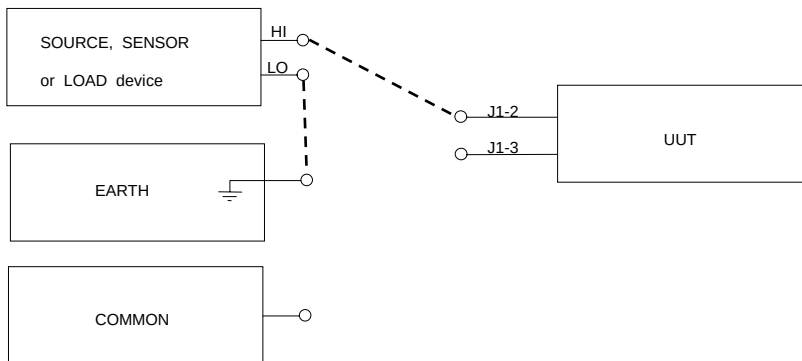


Figure 16-2: Illustration for COMMON, Example 2

Example 3: APPLY, COMMON, CNX J1-3 \$
MEASURE, (VOLTAGE), AC SIGNAL, . . . ,
CNX HI J1-2 LO COMMON \$

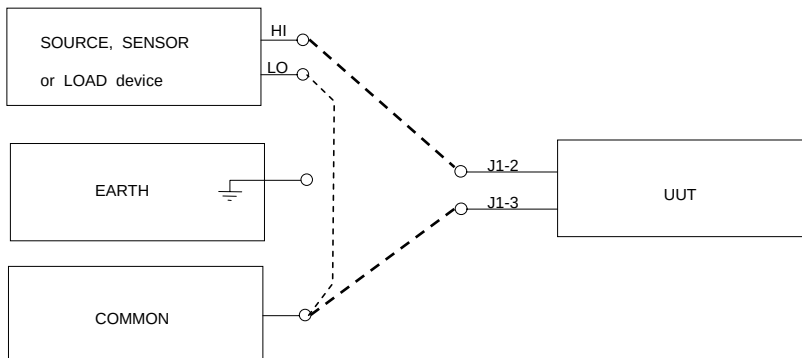


Figure 16-3: Illustration for COMMON, Example 3

16.0 NOUNS AND THEIR MODIFIERS**16.5 DC SIGNAL (Direct Current Signal)****16.5.1 Definition**

An unvarying electrical potential.

16.5.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AC Component Voltage	AC-COMP	R	SRM	Voltage, Current, Amperes
AC Component Frequency	AC-COMP-FREQ	R	SRM	Frequency
DC Current	CURRENT	R	SRM	Current
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current
Power	POWER	R	SRM	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Sample Width (duration)	SAMPLE-WIDTH	R	-R-	Time
Total Distortion	DISTORTION	R	-RM	Ratio, Voltage, Current
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.5.3 Rules

- The following is applicable for a source statement using DC SIGNAL:

<u>CURRENT</u>	<u>VOLTAGE</u>	<u>UUT-IMP</u>	<u>RESULT</u>
-	-	-	Not Recommended
-	-	*	Not Recommended
-	*	-	Current MAX, constant voltage
-	*	*	Voltage constant, current range known
*	-	-	Voltage MAX, current constant
*	-	*	Current constant, voltage range known
*	*	-	Not Recommended
*	*	*	Not Recommended

Note 1 * = included as a modifier;
- = not included as a modifier.

Note 2 Wherever a MAX result is possible a MAX modifier must be included.

- SAMPLE-WIDTH is an optional modifier used to indicate restrictions on the sampling aperture time for dc measurement, for example when sampling a time-varying waveform. Normal usage will be with MAX, to indicate that a longer sampling aperture will average the measurement of an unstable signal over too long a period thereby losing accurate information. If used with MIN, a possible interpretation might be that it refers to the minimum averaging time or filter time constant for a noisy signal.

16.0 NOUNS AND THEIR MODIFIERS

3. In DC SIGNAL, SOURCE statements AC-COMP MAX refers to the maximum allowable ripple on the source. The actual value may be anything from 0 to AC-COMP MAX. AC-COMP (no MAX) refers to the actual required value of ripple on the source. SAMPLE-WIDTH may not be used with MIN.

16.6 DOPPLER**16.6.1 Definition**

A response signal in the form of a band-limited spectrum having a specified frequency offset from a transmitted (source) reference frequency. The offset is proportional to the time rate of change in the effective path length between a transmitter (source) of the reference frequency and the receiver (point of observation).

16.6.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Burst Length	BURST	I	SR-	Burst Length
Carrier Frequency (Reference)	CAR-FREQ	R	SR-	Frequency
Current	CURRENT	R	SRM	Current
Doppler Bandwidth Power (6db to 6db)	DOPPLER-BANDWIDTH- POWER	R	SR-	Ratio, Power
Doppler Frequency	DOPPLER-FREQ	R	SR-	Frequency
Doppler Shift	DOPPLER-SHIFT	R	SR-	Frequency
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current
Signal Voltage Amplitude	VOLTAGE	R	SRM	Voltage
Transition Time	TRANS-TIME	R	S - -	Time

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.6.3 Rules

1. The reference carrier is identified by REF in the CNX field.

16.7 EARTH**16.7.1 Definition**

The earth ground electrical reference.

16.7.2 Noun Modifier Set

There are no modifiers for EARTH.

16.0 NOUNS AND THEIR MODIFIERS**16.7.3 Rules**

1. EARTH is defined and listed both in the noun list and the <conn set>. EARTH is a phenomenon which exists in the test environment and must be discretely attached to the UUT in order to have an effect on the UUT. If EARTH is used in the noun field of a source or load type statement, it implies that the function is to be attached to the specified UUT pins. If EARTH is used in the <connection> field of some other source, sensor, or load type function, it implies that the specified node of the test device is to be attached to the EARTH function of the test equipment.

16.8 EVENTS**16.8.1 Definition**

The occurrence of an electrical signal which is to be counted. EVENTS is used principally in MEASURE, (COUNT), EVENTS.

16.8.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Count	COUNT	I	-RM	Events
Current	CURRENT	R	-R-	Current
DC Offset	DC-OFFSET	R	-R-	Voltage, Current
Frequency	FREQ	R	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Negative Edge	NEG-EDGE	MO	-R-	(None)
Noise	NOISE	R	-R-	Ratio, Voltage, Current, Power
Positive Edge	POS-EDGE	MO	-R-	(None)
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Time	TIME	R	-R-	Time
Trigger	TRIG	R	-R-	Voltage
Voltage	VOLTAGE	R	-R-	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.8.3 Rules

1. The following items of information must be supplied to determine the number of events in a given time period:
 - a. Time over which the count must be made.
 - b. Voltage level which must be present to indicate an event (TRIG xV).
 - c. The slope of the signal, either NEG-EDGE or POS-EDGE, which indicates the direction of change of the triggering voltage.
 - d. An optional VOLTAGE MAX field may be added for voltage magnitude scaling.

16.0 NOUNS AND THEIR MODIFIERS**16.8.4 Notes and Examples**

The number of pulse counts over a period of time may be measured by making use of the EVENTS noun.

MEASURE, (COUNT), EVENTS, COUNT MAX 2052 TIMES, VOLTAGE MAX 2V,
TIME 25 SEC, TRIG 2 MV, POS-SLOPE, CNX HI J1 LO J2 \$

16.9 FM SIGNAL (Frequency Modulation Signal)**16.9.1 Definition**

A continuous sinusoidal carrier signal whose frequency is varied in accordance with the amplitude of a modulating signal.

16.9.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AM Component	AM-COMP	R	SRM	Voltage, Ratio
Bandwidth	BANDWIDTH	R	SRM	Frequency
Burst Length	BURST	I	SR-	Burst Length
Carrier Amplitude Modulation	CAR-AMPL	R	SR-	Ratio, Voltage, Power
Carrier Frequency	CAR-FREQ	R	SRM	Frequency
Carrier Harmonic Distortion	CAR-HARMONICS	R	SRM	Ratio, Voltage, Current, Power
Frequency Index (Deviation Ratio)	FM-INDEX	R	-RM	Ratio
Frequency Deviation	FREQ-DEV	R	SRM	Frequency
Frequency Deviation Rate	FREQ-DEV-RATE	R	SRM	Frequency, Time
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Modulation Amplitude	MOD-AMPL	R	SRM	Ratio, Voltage, Current
Modulation Distortion	MOD-DIST	R	-RM	Ratio, Voltage, Current Power
Modulation Frequency	MOD-FREQ	R	SRM	Frequency
Noise	NOISE	R	SRM	Ratio, Voltage, Current Power
Noise Amplitude Density	NOISE-AMPL-DENS	R	SRM	Noise Amplitude Density
Noise Power Density	NOISE-PWR-DENS	R	SRM	Power Amplitude Density
Non-Harmonic Distortion	NON-HARMONICS	R	-RM	Ratio, Voltage, Current Power
Power	POWER	R	SRM	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Standing Wave Ratio	SWR	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.10 IMPEDANCE****16.10.1 Definition**

A load or loading effect whose electrical characteristics are quantified by the voltage/current ration in an electrical circuit whereby impedance $Z = E(T,X) / I(T,X)$ where the parameters for the functions E and I are T for time and X for the physical location determined by the connections.

When used with the measured characteristics RETURN-LOSS or INSERTION-LOSS, the noun IMPEDANCE means that there may or may not exist a mismatch of impedance in the transmission path.

16.10.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Capacitance	CAP	R	SRM	Capacitance
Current Limit	CURRENT-LMT	R	-R-	Current
Dissipation Factor Ratio	DISS-FACTOR	R	SRM	Ratio
Frequency	FREQ	R	SR-	Frequency
Impedance	IMP	R	SRM	Impedance (complex)
Impedance Angle	IMP-ANGLE	R	SRM	Angle (plane)
Impedance Magnitude	IMP-MAGNITUDE	R	SRM	Impedance (real)
Inductance	IND	R	SRM	Inductance
Insertion Loss	INSERTION-LOSS	R	-RM	Ratio
Power Limit	PWR-LMT	R	-R-	Power
Reactance	REACTANCE	R	-RM	Impedance (real)
Reference Voltage	REF-VOLT	R	-R-	Voltage
Resistance	RES	R	SRM	Impedance (real)
Return Loss	RETURN-LOSS	R	-RM	Ratio
Standing Wave Ratio	SWR	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time
Voltage Limit	VOLT-LMT	R	-R-	Voltage

16.10.3 Rules

1. If two or more of the IMP, RES, CAP or IND modifier fields are employed, they indicate a collection of series impedances. If parallel components are to be connected to the same UUT pins, two or more ATLAS statements must be employed.
2. Complex networks of parallel and series components may need to be represented by a <define drawing statement>.

16.0 NOUNS AND THEIR MODIFIERS16.11 LIGHT16.11.1 Definition

An unmodulated or modulated signal transmitted to, through or by the Unit Under Test where the transfer of energy is in the form of electro-magnetic radiation in the wavelength range 0.38 to 0.78 micro-meter.

16.11.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Angular Direction	ANGLE	R	SRM	Angle (plane)
Illuminance	ILLUM	R	SRM	Illuminance
Luminous Intensity	LUM-INT	R	SRM	Luminous Intensity
Luminance	LUMINANCE	R	SRM	Luminance
Luminous Flux	LUM-FLUX	R	SRM	Luminous Flux
Modulation	MOD-AMPL	R	SRM	(None)
Transmittance	TRANSMITTANCE	R	SRM	(None)
Wavelength	WAVE-LENGTH	R	SRM	Distance

16.11.3 Rules

1. The spectral distribution for any appropriate noun-modifier can be specified in one ATLAS statement containing a statement characteristic subfield with the noun-modifier appropriate to the quantity and another statement characteristic subfield with the noun-modifier WAVE-LENGTH.

When the spectral distribution is confined to an infinitesimal range on either side of one wavelength it is necessary to state the given wavelength in the WAVE-LENGTH subfield.

When the spectral distribution is integrated over a range of wavelengths it is necessary to state the given range in the WAVE-LENGTH subfield.

16.0 NOUNS AND THEIR MODIFIERS**16.12 LOGIC SIGNAL****16.12.1 Definition**

A binary digital signal containing more than one binary digit (bit) and used to convey data, control, timing or phase reference to or from a UUT. All bits may appear simultaneously at the digital interface, each on a separate parallel circuit or they may appear sequentially over a single serial circuit. These bits may represent either a true binary number (such as used in binary arithmetic or used to represent an analog quantity) or just a collection of bits having no intrinsic value of their own. LOGIC SIGNAL signals may be single pulses, pulse trains, DC levels, or other analog characteristics which define logic data.

16.12.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Bit-Rate	BIT-RATE	R	SR-	Digital Bit Rate
Current	CURRENT	R	SR-	Current
DC Offset	DC-OFFSET	R	SR-	Voltage
Droop	DROOP	R	SR-	Ratio
Duty Cycle	DUTY-CYCLE	R	SR-	Ratio
Fall Time	FALL-TIME	R	-R-	Time
Frequency	FREQ	R	SR-	Frequency
Noise	NOISE	R	SRM	Voltage, Ratio, Current
Overshoot	OVERSHOOT	R	SR-	Ratio
Power	POWER	R	SR-	Power
Preshoot	PRESHOOT	R	SR-	Ratio
Pulse Repetition Frequency	PRF	R	SR-	Frequency
Pulse Class or Code	PULSE-CLASS	MD	SR-	(None)
Pulse Width	PULSE-WIDTH	R	SR-	Time
Ringing	RINGING	R	SR-	Ratio
Rise Time	RISE-TIME	R	-R-	Time
Corner Rounding	ROUNDING	R	SR-	Ratio
Bit Skew	SKEW-TIME	R	SR-	Time
Time Jitter	TIME-JIT	R	SR-	Time
Digital Signal Type	TYPE	MD	SR-	(None)
Undershoot	UNDERSHOOT	R	SR-	Ratio
Data Word Contents and Value	VALUE	AB	SRM	(None)
Voltage	VOLTAGE	R	SR-	Voltage
Word Length	WORD-LENGTH	I	S - -	Digital Word Length
Word-Rate	WORD-RATE	R	SR-	Digital Word Rate

16.0 NOUNS AND THEIR MODIFIERS

16.12.3 Rules

1. CURRENT and VOLTAGE shall be used when binary data is coded in other than current or voltage amplitudes.
2. PULSE-CLASS and TIME-JIT used for serial data only.
3. SKEW-TIME and PULSE-WIDTH used for parallel data only.
4. The following apply to PULSE-WIDTH.
 - a. Either PULSE-WIDTH or WORD-RATE is required for non-clocked parallel data; PULSE-WIDTH is optional for clocked parallel data.
 - b. The pulse width for SERIAL-DATA is implicit in the PULSE-CLASS type.
5. Each bit period of a BIP waveform is divided into two equal sub-periods. A transition in voltage may or may not occur at the start of each bit period. A second transition may or may not occur at the half-period point. The binary data is coded into the waveform in terms of this second transition and the voltage levels in the two sub-periods.
6. When used with <define digital source statement> and <define digital sensor statement>, noun modifiers that are worded the same as digital keywords cannot be used in the same statement.

16.0 NOUNS AND THEIR MODIFIERS**16.13 PAM (Pulse Amplitude Modulation)****16.13.1 Definition**

A pulsed carrier signal whose amplitude is varied in accordance with the amplitude of a modulating signal.

16.13.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Burst Length	BURST	I	SR-	Burst length
Carrier Frequency	CAR-FREQ	R	SRM	Frequency
Corner Rounding	ROUNDING	R	SRM	Ratio
Droop	DROOP	R	SRM	Ratio
Fall Time	FALL-TIME	R	SRM	Time
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Modulation Amplitude	MOD-AMPLE	R	SRM	Ratio
Modulation Frequency	MOD-FREQ	R	SRM	Frequency
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	R	SRM	Ratio
Peak Amplitude (Pulse)	P-AMPL	R	SRM	Ratio, Voltage, Power
Preshoot	PRESHOOT	R	SRM	Ratio
Pulse Repetition Frequency	PRF	R	SRM	Frequency
Pulse Width	PULSE-WIDTH	R	SRM	Time
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Ringing	RINGING	R	SRM	Ratio
Rise Time	RISE-TIME	R	SRM	Time
Transition Time	TRANS-TIME	R	S - -	Time
Undershoot	UNDERSHOOT	R	SRM	Ratio

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS16.14 PM SIGNAL (Phase Modulated Signal)16.14.1 Definition

A phase-modulated signal generated when the phase of one wave is varied in accordance with the amplitude of another wave.

16.14.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Burst Length	BURST	I	SR-	Burst Length
Carrier Amplitude	CAR-AMPL	R	SRM	Voltage
Carrier Frequency	CAR-FREQ	R	SRM	Frequency
Carrier Harmonic Distortion	CAR-HARMONICS	R	SRM	Voltage, Ratio *
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Modulation Amplitude	MOD-AMPL	R	SRM	Voltage, Ratio *
Modulation Distortion	MOD-DIST	R	-RM	Voltage, Ratio *
Modulation Frequency	MOD-FREQ	R	SRM	Frequency
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Non-Harmonic Distortion (Carrier)	NON-HARMONICS	R	-RM	Voltage, Ratio *
Phase Deviation	PHASE-DEV	R	SRM	Angle (plane)
Power	POWER	R	SRM	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Transition Time	TRANS-TIME	R	S - -	Time

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

Note: * Ratio values are referenced to the un-modulated carrier amplitude.

16.0 NOUNS AND THEIR MODIFIERS**16.15 PULSED AC (Pulsed Alternating Current Signal)****16.15.1 Definition**

An electrical signal that is characterized by short duration periods of alternating current (Sinusoidal) electrical signal.

16.15.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Bandwidth	BANDWIDTH	R	SR-	Frequency
Burst Length	BURST	I	SR-	Burst Length
Carrier Frequency	CAR-FREQ	R	SRM	Frequency
Carrier Phase *	CAR-PHASE	R	SRM	Angle (plane)
Corner Rounding	ROUNDING	R	SRM	Ratio
Current	CURRENT	R	SRM	Current
DC Offset	DC-OFFSET	R	SRM	Voltage
Fall Time	FALL-TIME	R	SRM	Time
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	R	SRM	Ratio
Period	PERIOD	R	SRM	Time
Phase Jitter	PHASE-JIT	R	SRM	Angle (plane)
Power	POWER	R	SRM	Power
Preshoot	PRESHOOT	R	SRM	Ratio
Pulse Repetition Frequency	PRF	R	SRM	Frequency
Pulse Time Jitter	TIME-JIT	R	SRM	Time
Pulse Width	PULSE-WIDTH	R	SRM	Time
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Ringing	RINGING	R	SRM	Ratio
Rise Time	RISE-TIME	R	SRM	Time
Standing Wave Ratio	SWR	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Undershoot	UNDERSHOOT	R	SRM	Ratio
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

Note: *Phase is referenced with respect to the carrier phase of the PULSED AC which is referenced in the REF subfield of the CNX field.

16.0 NOUNS AND THEIR MODIFIERS**16.16 PULSED AC TRAIN****16.16.1 Definition**

An electrical signal that is characterized by a train of different short duration periods of a sinusoidal electrical signal.

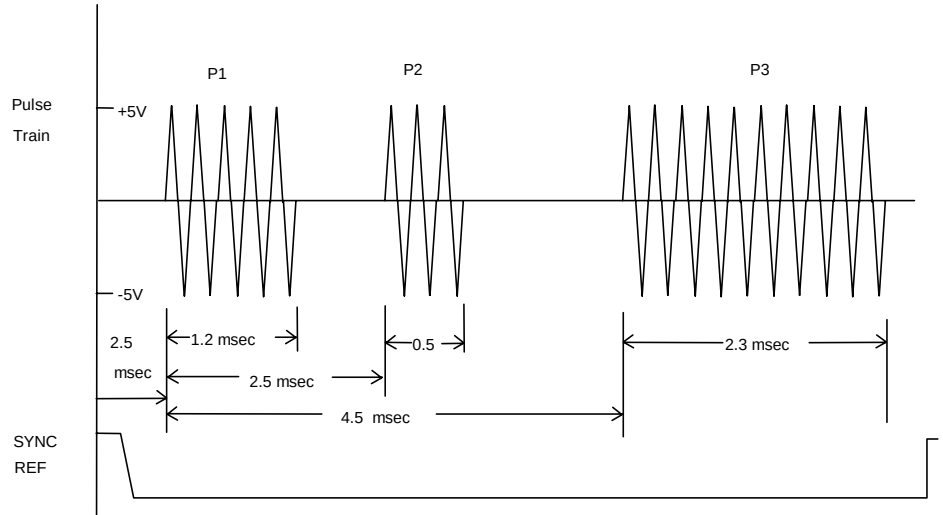
16.16.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Local Scope</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Bandwidth	BANDWIDTH		R	SR-	Frequency
Burst Length	BURST		I	SR-	Burst Length
Carrier Frequency	CAR-FREQ		R	SRM	Frequency
Corner Rounding	ROUNDING	X	R	-RM	Ratio
Current	CURRENT	X	R	SRM	Current
DC Offset	DC-OFFSET		R	SRM	Voltage
Droop	DROOP	X	R	-RM	Ratio
Fall Time	FALL-TIME	X	R	SRM	Time
Impedance Angle	xxx-IMP-ANGLE		R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE		R	SR-	Impedance (real)
Noise	NOISE		R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	X	R	-RM	Ratio
Power	POWER		R	SRM	Power
Preshoot	PRESHOOT	X	R	-RM	Ratio
Pulse Repetition Frequency	PRF		R	SRM	Frequency
Pulse Width	PULSE-WIDTH	X	R	SRM	Time
Reactance	xxx-REACTANCE		R	SR-	Impedance (real)
Resistance	xxx-RES		R	SR-	Impedance (real)
Ringing	RINGING	X	R	-RM	Ratio
Rise Time	RISE-TIME	X	R	SRM	Time
Pulse Spacing	SPACING	X	R	SRM	Time
Transition Time	TRANS-TIME		R	S - -	Time
Undershoot	UNDERSHOOT	X	R	-RM	Ratio
Voltage (Peak)	VOLTAGE	X	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.16.3 Rules

1. The scope of certain modifiers can be specified either as global so that they apply to each pulse in the pulse train or as local so that they apply only to the specified single pulse or pulse pair. Local modifiers are identified by the pulse identifier suffix (a -P followed by an <unsigned integer number>) that defines the number of the pulse being described. Pulses are numbered in order of their occurrence in time; the first pulse being number one.
2. All modifiers except SPACING may be used with global scope. Those modifiers having an X in the local scope column may be used with local scope. SPACING is used only with local scope.
3. Spacing between pulses in the pulse train is specified as the time between either the initial or final slope of any two pulses.
4. Any specified DC-OFFSET is global and is the reference for all pulses.
5. The amplitude of pulses that have a common overlap in time are algebraically added during the time of overlap.

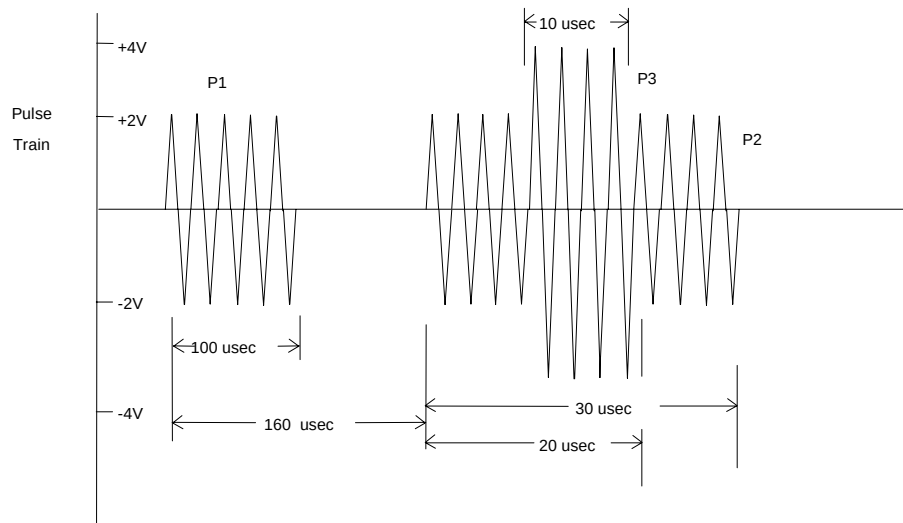
16.0 NOUNS AND THEIR MODIFIERS**16.16.4 Notes and Examples****1. Example 1.****Figure 16-4: Illustration of PULSED DC TRAIN, Example 1**

The pulse train shown above is defined by three single pulses: P1, P2 and P3. This signal may be described by the following statement:

```
100050 SETUP, PULSED AC TRAIN,
      CAR-FREQ 1 GHZ,
      VOLTAGE-P 5 V,
      RISE-TIME 100 NSEC,
      PULSE-WIDTH-P1 1.2 MSEC,
      PULSE-WIDTH-P2 0.5 MSEC,
      PULSE-WIDTH-P3 2.3 MSEC,
      SPACING-P1-POS-P2-POS 2.5 MSEC,
      SPACING-P1-POS-P3-POS 4.5 MSEC,
      SYNC-P1-POS TO STEP . . . DELAY 2.5 MSEC, . . . $
```

16.0 NOUNS AND THEIR MODIFIERS

2. Example 2.

**Figure 16-5: Illustration for PULSED AC TRAIN, Example 2**

```

100100 DEFINE, 'PULSED AC TRAIN EXAMPLE', SENSOR,
        PULSED AC TRAIN,
        CAR-FREQ 1060 MHZ,
        VOLTAGE-P-P1 2 V,
        PULSE-WIDTH-P1 100 USEC,
        VOLTAGE-P-P2 2 V,
        PULSE-WIDTH-P2 30 USEC,
        VOLTAGE-P-P3 2 V,
        PULSE-WIDTH-P3 10 USEC,
        SPACING-P1-POS-P2-POS 160 USEC,
        SPACING-P2-POS-P3-NEG 20 USEC, . . . $

```

16.0 NOUNS AND THEIR MODIFIERS**16.17 PULSED DC (Pulsed Direct Current Signal)****16.17.1 Definition**

An electrical signal that is characterized by short duration periods of positive or negative electrical amplitudes which occur either periodically or non-periodically in time.

16.17.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Corner Rounding	ROUNDING	R	-RM	Ratio
DC Current	CURRENT	R	SRM	Current
DC Offset	DC-OFFSET	R	SRM	Voltage
Delay	DELAY	R	SRM	Time
Droop	DROOP	R	-RM	Ratio
Duty Cycle	DUTY-CYCLE	R	SRM	Ratio
Fall Time	FALL-TIME	R	SRM	Time
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	R	-RM	Ratio
Period	PERIOD	R	SRM	Time
Power	POWER	R	SRM	Power
Preshoot	PRESHOOT	R	-RM	Ratio
Pulse Repetition Frequency	PRF	R	SRM	Frequency
Pulse Width	PULSE-WIDTH	R	SRM	Time
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Ringing	RINGING	R	-RM	Ratio
Rise Time	RISE-TIME	R	SRM	Time
Time Jitter	TIME-JIT	R	SRM	Time
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Undershoot	UNDERSHOOT	R	-RM	Ratio
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.18 PULSED DC TRAIN****16.18.1 Definition**

A signal characterized by a train of different single pulses with a defined time correlation between the single elements.

16.18.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Local</u>	<u>Type</u> <u>Scope</u>	<u>Usage</u>	<u>Quantity</u>
Burst Length	BURST		I	SR-	Burst Length
Corner Rounding	ROUNDING	X	R	-RM	Ratio
Current	CURRENT	X	R	SRM	Current
DC Offset	DC-OFFSET		R	SRM	Voltage
Droop	DROOP	X	R	-RM	Ratio
Fall Time	FALL-TIME	X	R	SRM	Time
Impedance Angle	xxx-IMP-ANGLE		R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE		R	SR-	Impedance (real)
Noise	NOISE		R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	X	R	-RM	Ratio
Power	POWER		R	SRM	Power
Preshoot	PRESHOOT	X	R	-RM	Ratio
Pulse Repetition Frequency	PRF		R	SRM	Frequency
Pulse Width	PULSE-WIDTH	X	R	SRM	Time
Reactance	xxx-REACTANCE		R	SR-	Impedance (real)
Resistance	xxx-RES		R	SR-	Impedance (real)
Ring	RINGING	X	R	-RM	Ratio
Rise Time	RISE-TIME	X	R	SRM	Time
Pulse Spacing	SPACING	X	R	SRM	Time
Transition Time	TRANS-TIME		R	S - -	Time
Undershoot	UNDERSHOOT	X	R	-RM	Ratio
Voltage	VOLTAGE	X	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.18.3 Rules

1. The scope of certain modifiers can be specified either as global so that they apply to each pulse in the pulse train or as local so that they apply only to the specified single pulse or pulse pair.

Local modifiers are identified by the pulse identifier suffix (a -P followed by an <unsigned integer number>) that defines the number of the pulse being described. Pulses are numbered in order of their occurrence in time: the first pulse being number one.

2. All modifiers except SPACING may be used with a global scope. Those modifiers having an X in the local scope column may be used with a local scope. SPACING is used only with local scope.
3. Spacing between pulses in the pulse train is specified as the time between either the initial or final slope of any two pulses.
4. Any specified DC-OFFSET is global and is the reference for all pulses.
5. The amplitude of pulses that have a common overlap in time are algebraically added during the time of overlap.

16.0 NOUNS AND THEIR MODIFIERS

6. When the modifier has the form SPACING <array range> the modifier PULSE-WIDTH is not allowed.

16.18.4 Notes and Examples

1. Example 1.

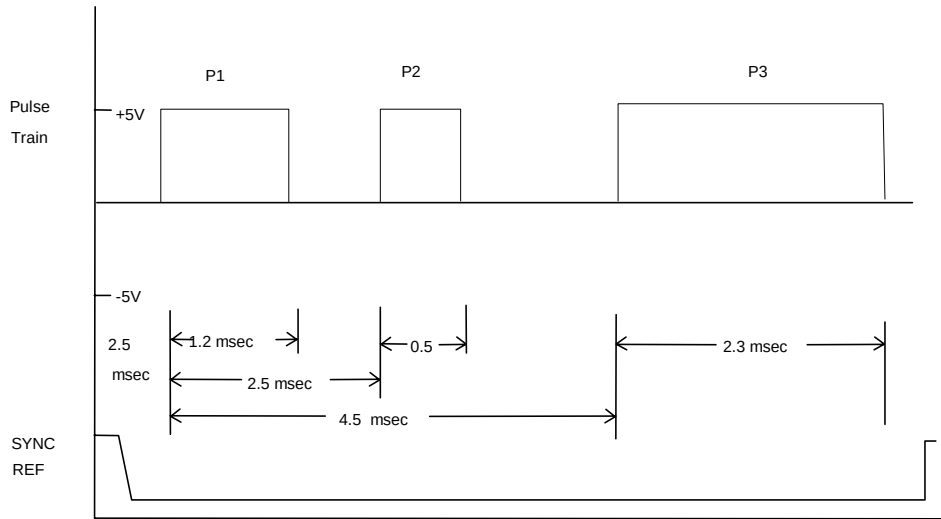


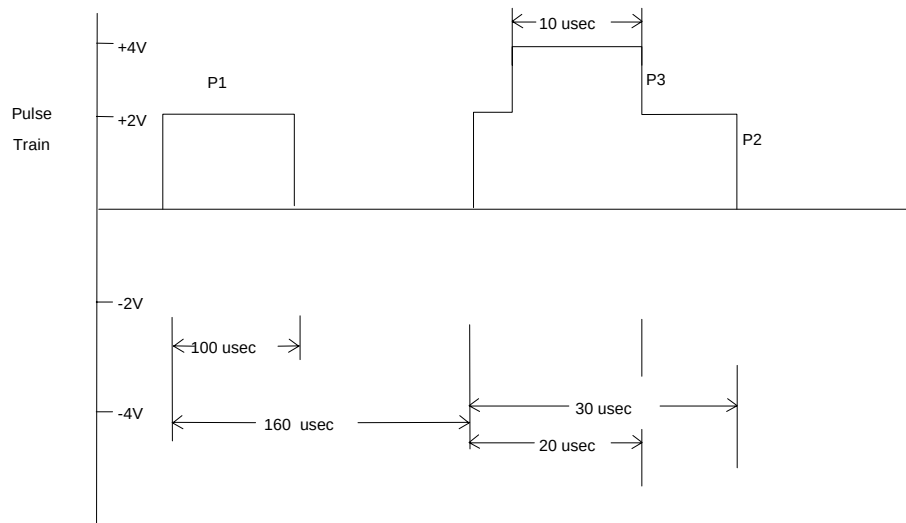
Figure 16-6: Illustration for PULSED DC TRAIN, Example 1

The pulse train shown above is defined by three single pulses. This signal may be described by the following statement:

```
100050 SETUP, PULSED DC TRAIN, VOLTAGE-P 5 V,
      RISE-TIME 100 NSEC,
      PULSE-WIDTH-P1 1.2 MSEC,
      PULSE-WIDTH-P2 0.5 MSEC,
      PULSE-WIDTH-P3 2.3 MSEC,
      SPACING-P1-POS-P2-POS 2.5 MSEC,
      SPACING-P1-POS-P3-POS 4.5 MSEC,
      SYNC-P1-POS TO STEP . . . DELAY 2.5 MSEC, . . . $
```

16.0 NOUNS AND THEIR MODIFIERS

2. Example 2.

**Figure 16-7: Illustration for PULSED DC TRAIN, Example 2**

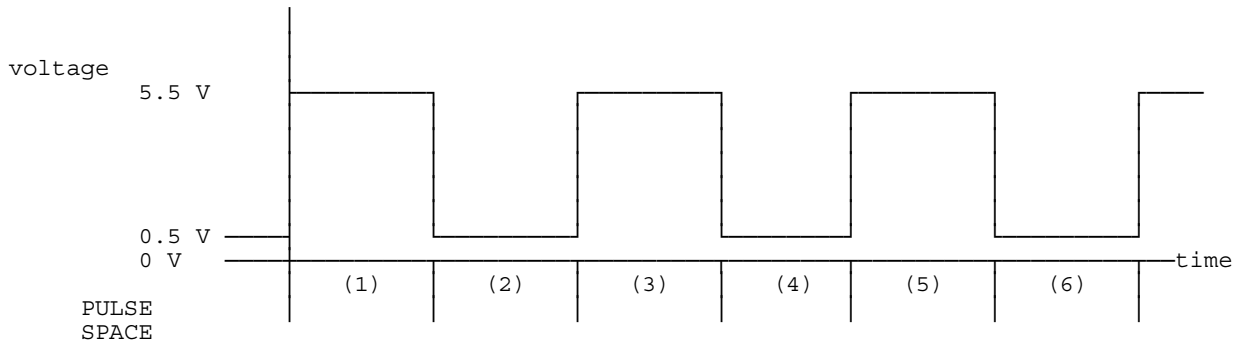
```

100100 DEFINE, 'PULSE TRAIN EXAMPLE', SENSOR, PULSED DC TRAIN,
    VOLTAGE-P-P1 2 V,
    PULSE-WIDTH-P1 100 USEC,
    VOLTAGE-P-P2 2 V,
    PULSE-WIDTH-P2 30 USEC,
    VOLTAGE-P-P3 2 V,
    PULSE-WIDTH-P3 15 USEC,
    SPACING-P1-POS-P2-POS 160 USEC,
    SPACING-P2-POS-P3-NEG 20 USEC . . . $

```

16.0 NOUNS AND THEIR MODIFIERS

3. Example 3:

**Figure 16-8: Illustration for PULSED DC TRAIN, Example 3**

```

000020 DECLARE, VARIABLE, 'PULSE-SPACE' IS ARRAY (1 THRU 6) OF
                                DECIMAL INITIAL = 10.2,7.3,10.2,12.7,6.2,8.9 $
.
.
.

147720 APPLY, PULSED DC TRAIN,
    SPACING 'PULSE-SPACE' (1 THRU 6) MSEC ERRLMT +- 0.1 MSEC,
    DC-OFFSET 0.5 V, VOLTAGE-P 5.0 V,
    RISE-TIME 1 MSEC, FALL-TIME 1 MSEC,
    CNX HI J1-12 LO J1-24 $
147730 MEASURE, (SPACING INTO 'PULSE-SPACE' (1 THRU 6),
    PULSED DC TRAIN,
    SPACING MAX 4000 MSEC,
    DC-OFFSET 0 V,
    VOLTAGE-P 5 V,
    CNX HI J1-32 LO J1-36 $

```

16.0 NOUNS AND THEIR MODIFIERS**16.19 PULSED DOPPLER****16.19.1 Definition**

A pulse modulated electrical signal having a carrier frequency offset (Doppler shift) and a pulse delay from some reference pulse modulated signal. The reference signal is identified via the CNX REF field.

16.19.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Bandwidth (3db to 3db)	BANDWIDTH	R	SR-	Frequency
Burst Length	BURST	I	SR-	Burst Length
Carrier Frequency (Reference)	CAR-FREQ	R	SRM	Frequency
Current	CURRENT	R	SRM	Current
Delay	DELAY	R	SRM	Time
Doppler Shift	DOPPLER-SHIFT	R	-RM	Frequency
Duty Cycle	DUTY-CYCLE	R	SR-	Ratio
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SRM	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Phase Angle (shift from reference)	PHASE-ANGLE	R	S - -	Angle (plane)
Power	POWER	R	SRM	Power
Pulse Repetition Frequency	PRF	R	SRM	Frequency
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Transition Time	TRANS-TIME	R	S - -	Time
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.20 RADAR SIGNAL****16.20.1 Definition**

A simulated radar range produced by delaying an output pulse from a transmitter before returning the pulse to a receiver.

16.20.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Attenuation (Pulse Power)	ATTEN	R	S - -	Ratio
Frequency	FREQ	R	SRM	Frequency
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Power	POWER	R	SRM	Power
Pulse Repetition Frequency	PRF	R	SRM	Frequency
Pulse Energy (integrated)	PULSE-ENERGY	R	SRM	Energy
Pulse Width	PULSE-WIDTH	R	SRM	Time
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Reply Efficiency **	REPLY-EFF	R	S - -	Ratio
Spacing (Pulse) *	SPACING	R	SRM	Time
Sweep Interval	SWEEP-INT	R	SRM	Time
Target Range (simulated)	TARGET-RANGE	R	S - -	Length
Target Range Acceleration	TARGET-RANGE-ACCEL	R	S - -	Acceleration
Target Range Rate of Change	TARGET-RANGE-RATE	R	S - -	Velocity (linear)
Transition Time	TRANS-TIME	R	S - -	Time

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

Notes: *Applicable Pulse Identifier Suffixes must be used.

**Ratio of returned pulse quantity to the transmitted pulse quantity.

16.20.3 Rules

1. When SWEEP-INT is used, FREQ-WINDOW must be used in the same statement.

16.0 NOUNS AND THEIR MODIFIERS**16.21 RAMP SIGNAL****16.21.1 Definition**

A varying electrical signal whose wave shape is characterized by a periodic series of alternately positive and negative linearly sloping amplitude wherein the ratio of the positive slope to the negative slope does not equal unity.

16.21.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AC Current	CURRENT	R	SRM	Current
Bandwidth	BANDWIDTH	R	SR-	Frequency
Burst Length	BURST	I	SR-	Burst Length
DC Offset	DC-OFFSET	R	SRM	Voltage
Fall Time	FALL-TIME	R	SRM	Time
Frequency	FREQ	R	SRM	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Negative Slope	NEG-SLOPE	R	SRM	Voltage / Time
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Non-Linearity	NON-LIN	R	SRM	Ratio
Peak Degeneration	PEAK-DEGEN	R	SRM	Ratio
Period	PERIOD	R	SRM	Time
Phase Jitter	PHASE-JIT	R	SRM	Angle (plane)
Positive Slope	POS-SLOPE	R	SRM	Voltage / Time
Power	POWER	R	SRM	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Rise Time	RISE-TIME	R	SRM	Time
Time Asymmetry	TIME-ASYM	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS16.22 RANDOM NOISE16.22.1 Definition

A signal whose value at any time is determined only by means of statistical probability from knowledge of the signal at all previous times.

16.22.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Bandwidth	BANDWIDTH	R	SR-	Frequency
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise Amplitude Density	NOISE-AMPL-DENS	R	SRM	Noise Amplitude Density
Noise Power Density	NOISE-PWR-DENS	R	SRM	Noise Power Density
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Transition Time	TRANS-TIME	R	S - -	Time
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS16.23 RESOLVER16.23.1 Definition

A combination of two in-phase sinusoidal electrical signals whose amplitude relationship describe the rotational shaft position of an electro-mechanical device.

16.23.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Angle	ANGLE	R	SRM	Angle (plane)
Angle Rate	ANGLE-RATE	R	SRM	Velocity (angular)
Bandwidth	BANDWIDTH	R	SR-	Frequency
Frequency	FREQ	R	SRM	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Harmonics	HARMONICS	R	SRM	Voltage
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Non-Harmonics	NON-HARMONICS	R	-RM	Ratio, Voltage, Current, Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Transition Time	TRANS-TIME	R	S - -	Time
Voltage	VOLTAGE	R	SRM	Voltage
Zero Index	ZERO-INDEX	R	S - -	Angle (plane)

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.24 ROTATION****16.24.1 Definition**

A mathematical indication of the magnitude and direction of a rotation angle, rotation rate or rotation acceleration of the Unit Under Test (UUT) or stimuli applied to the UUT relative to a specified reference frame with a specified origin.

16.24.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Angle	ANGLE	R	SRM	Angle (plane)
Angle Acceleration	ANGLE-ACCEL	R	SRM	Angle (plane) / time / time
Angle Acceleration (X Axis)	ANGLE-ACCEL-X	R	SRM	Angle (plane) / time / time
Angle Acceleration (Y Axis)	ANGLE-ACCEL-Y	R	SRM	Angle (plane) / time / time
Angle Acceleration (Z Axis)	ANGLE-ACCEL-Z	R	SRM	Angle (plane) / time / time
Angle Rate of Change	ANGLE-RATE	R	SRM	Angle (plane) / time
Angle Rate of Change (X Axis)	ANGLE-RATE-X	R	SRM	Angle (plane) / time
Angle Rate of Change (Y Axis)	ANGLE-RATE-Y	R	SRM	Angle (plane) / time
Angle Rate of Change (Z Axis)	ANGLE-RATE-Z	R	SRM	Angle (plane) / time
Angle (X Axis)	ANGLE-X	R	SRM	Angle (plane)
Angle (Y Axis)	ANGLE-Y	R	SRM	Angle (plane)
Angle (Z Axis)	ANGLE-Z	R	SRM	Angle (plane)
Oscillatory Motion	OSCILLATORY	MO	S - -	(None)
Reference Earth Stabilized (Coordinate Frame)	REF-INERTIAL	MO	S - -	(None)
Reference UUT Stabilized (Coordinate Frame)	REF-UUT	MO	S - -	(None)
Scorsby Motion	SCORSBY	MO	S - -	(None)
Transition Time	TRANS-TIME	R	S - -	Time

16.24.3 Rules

1. Rotational motion may be specified in one of three ways:

Normal
SCORSBY
OSCILLATORY

Normal is the default mode, and is specified by the absence of either the SCORSBY or OSCILLATORY modifier descriptors.

2. Rotation angles, rates and accelerations are specified relative to a reference coordinate frame that is stabilized relative to inertial space (an earth stabilized frame) or to the UUT (for example, an airframe stabilized reference frame). Either the modifier REF-INERTIAL or the modifier REF-UUT should be specified when using the noun ROTATION to avoid ambiguity.
3. The specification in a single statement both of a quantity and its derivative, places restrictions on how the value of the quantity is attained. When a quantity appears with more than one of its derivatives, there are actions associated with the way in which the process concludes beyond those explicit in the statement. For example; consider a case in which a quantity such as linear or angular displacement, its rate and its acceleration are all specified.

16.0 NOUNS AND THEIR MODIFIERS

In the following diagrams and ATLAS statement the signal angle is represented as 'Q' having first and second derivatives 'Q-RATE' and 'Q-ACCEL'.

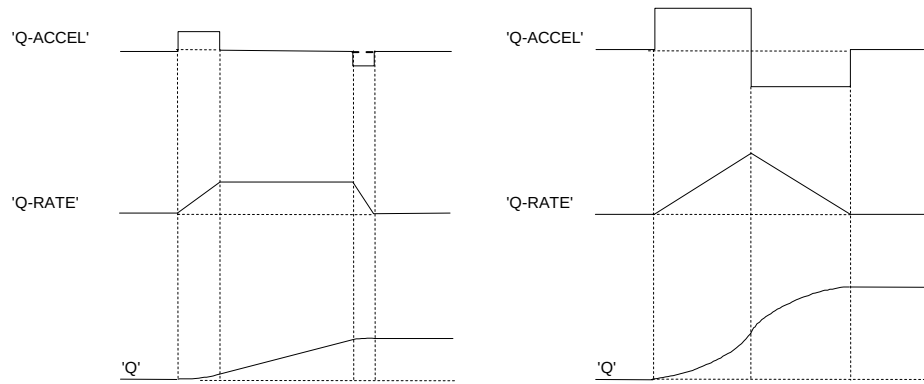


Figure 16-9: Illustration for Rule 3 of ROTATION

```
APPLY, ROTATION, ANGLE 'Q' DEG, ANGLE-RATE 'Q-RATE' DEG/SEC,
      ANGLE-ACCEL 'Q-ACCEL' DEG/SEC/SEC, ....$
```

The statement implies that the highest derivative assumes its specified value until the value of the next lower order derivative is attained, at which time the highest order derivative goes to zero and the next lower order derivative remains constant, if possible. The polarity of the highest order derivative specifies the initial direction of rotation for normal motion.

For normal motion, the conclusion of the process begins at or after the midpoint of travel of the quantity to its specified value. Since only one magnitude can be given for each derivative, the process is inherently symmetrical and all derivatives reverse sign and/or direction as appropriate. The simultaneous specification of differing rates of acceleration and deceleration is beyond the scope of this statement.

4. If rotation about more than one UUT axis is possible, an axis suffix, must be supplied with the noun modifiers. These suffixes are -X, -Y, -Z. Motions about only one possible axis do not require a suffix. Motions about all axes may be specified independently and individually in terms of the noun modifiers ANGLE, ANGLE-RATE and ANGLE-ACCEL subject to constraints of Rule 3 above.

16.0 NOUNS AND THEIR MODIFIERS**16.24.4 Notes and Examples****Example 1**

```

000100 DEFINE, 'CENTER-OF-GRAVITY', DRAWING, 76Z8973 REV C,
          TERMINALS CG ROLL PITCH YAW, CNX X *ROLL Y *PITCH Z *YAW $

001000 APPLY, ROTATION, REF-INERTIAL,
          ANGLE-X 0 DEG ERRLMT +-0.1 DEG,
          ANGLE-Y 0 DEG ERRLMT +-0.1 DEG,
          ANGLE-Z 0 DEG ERRLMT +-0.1 DEG,
          CNX X ROLL Y PITCH Z YAW $

```

Statement 001000 will orient the UUT, a missile, for example, in a wings level attitude headed towards the North Pole.

```

001001 APPLY, ROTATION, REF-INERTIAL,
          ANGLE-X + 180 DEG ERRLMT +-0.1 DEG,
          ANGLE-RATE-X MAX 10.0 DEG/SEC,
          ANGLE-ACCEL-X MAX 1.0 DEG/SEC/SEC,
          CNX X ROLL $

```

Statement 001001 would cause the missile to roll 180 degrees in the direction of right wing down with a maximum roll of 10 degrees/sec and a maximum roll acceleration of 1 degree/sec/sec. Thus, the missile would end up pointed north and in an upside down attitude as illustrated.

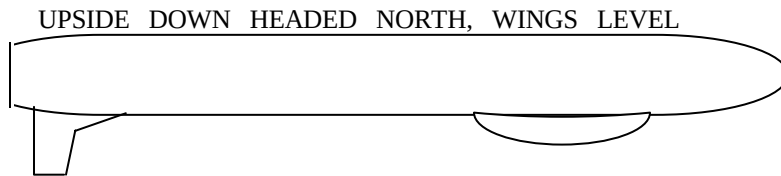


Figure 16-10: Illustration of ROTATION, Example 1

Example 2

Apply a constant rotation rate, accelerate to higher rate, decelerate to rate in opposite direction, accelerate to higher rate in the negative direction and decelerate to zero.

Note negative acceleration, often called deceleration, is defined as a rate of change of an angular rate that is becoming less positive with increasing time. For example, to change from a -5 deg/sec rotation rate to a -10 deg/sec, a negative acceleration rate is applied, i.e., a deceleration. This is easily seen if one recalls the definition of acceleration as:

$$\text{accel} = (\text{velocity at } T2 - \text{velocity at } T1) / (T2 - T1)$$

where time T2 is later than time T1.

For the above case:

$$\text{accel} = (-10 \text{ deg/sec} - (-5 \text{ deg/sec})) / (T2 - T1)$$

which is negative.

16.0 NOUNS AND THEIR MODIFIERS

```

090100 APPLY, ROTATION, REF-UUT,
        ANGLE-RATE-Z 5.0 DEG/SEC ERRLMT +-0.1 DEG/SEC,
        CNX Z YAW $

090101 APPLY, ROTATION, REF-UUT,
        ANGLE-RATE-Z 10.0 DEG/SEC ERRLMT +-0.1 DEG/SEC,
        ANGLE-ACCEL-Z MAX +1.0 DEG/SEC/SEC,
        CNX Z YAW $

090102 APPLY, ROTATION, REF-UUT,
        ANGLE-RATE-Z -5.0 DEG/SEC ERRLMT +-0.1 DEG/SEC,
        ANGLE-ACCEL-Z MIN -1.0 DEG/SEC/SEC,
        CNX Z YAW $

090103 APPLY, ROTATION, REF-UUT,
        ANGLE-RATE-Z -10.0 DEG/SEC ERRLMT +-0.1 DEG/SEC,
        ANGLE-ACCEL-Z MIN -1.0 DEG/SEC/SEC,
        CNX Z YAW $

090104 APPLY, ROTATION, REF-UUT,
        ANGLE-RATE-Z 10.0 DEG/SEC ERRLMT +-0.1 DEG/SEC,
        ANGLE-ACCEL-Z MAX 2.0 DEG/SEC/SEC,
        CNX Z YAW $

```

Incorrect Application

```

090105 APPLY, ROTATION, REF-UUT,
        ANGLE-RATE-Z 10.0 DEG/SEC ERRLMT +-0.1 DEG/SEC,
        ANGLE-ACCEL-Z MIN -5.0 DEG/SEC/SEC,
        CNX Z YAW $

```

Statement 090100 would cause the UUT to rotate about the Center of Gravity at 5.0 deg/sec (turning right) in the yaw plane. Statement 090101 would increase the rotation rate from 5.0 deg/sec to 10.0 deg/sec with an acceleration of no more than 1.0 deg/sec/sec. Statement 090102 would change the rotation rate from +10.0 deg/sec to -5.0 deg/sec with a deceleration of no more than 1.0 deg/sec/sec. Statement 090103 would increase the rotation rate in the negative direction (reduce it in the positive direction) from -5.0 deg/sec to -10.0 deg/sec at an acceleration of no more than -1.0 deg/sec/ sec. Statement 090104 would change the rotation rate from -10.0 deg/sec to 0.0 deg/sec with an acceleration of no more than 2.0 deg/sec. Statement 090105 attempts to establish a rotation 10.0 deg/sec, but an acceleration of -5.0 deg/sec/sec is applied. A positive rotation cannot be obtained via this statement due to the incorrect semantics.

Example 3

Apply a composite rotation that is defined by SCORSBY motion such as would be exhibited by a gyroscope.

```

100000 APPLY, ROTATION, SCORSBY,
        ANGLE 10 DEG,
        FREQ .1 HZ,
        CNX X ROLL Y PITCH Z YAW $

```

16.0 NOUNS AND THEIR MODIFIERS16.25 SHORT16.25.1 Definition

A conductor of negligible impedance, with a specified current and frequency carrying capability that is used to establish a common potential between the designated UUT connections.

16.25.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Current Limit	CURRENT-LMT	R	S - -	Current
Frequency Limit	FREQ-LMT	R	S - -	Frequency
Voltage Limit	VOLT-LMT	R	S - -	Voltage

16.0 NOUNS AND THEIR MODIFIERS**16.26 SQUARE WAVE****16.26.1 Definition**

A periodic wave which alternatively assumes one of two fixed values of amplitude for equal lengths of time. The time of transition between the fixed values is negligible with respect to the period.

16.26.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AC Current	CURRENT	R	SRM	Current
Bandwidth	BANDWIDTH	R	SR-	Frequency
Burst Length	BURST	I	SR-	Pulses
Corner Rounding	ROUNDING	R	-RM	Ratio
DC Offset	DC-OFFSET	R	SRM	Voltage
Droop	DROOP	R	-RM	Ratio
Duty Cycle	DUTY-CYCLE	R	-RM	Ratio
Fall Time	FALL-TIME	R	SRM	Time
Frequency	FREQ	R	SRM	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	R	-RM	Ratio
Period	PERIOD	R	SRM	Time
Phase Jitter	PHASE-JIT	R	SRM	Angle (plane)
Power	POWER	R	SRM	Power
Preshoot	PRESHOOT	R	SRM	Ratio
Ringling	RINGING	R	-RM	Ratio
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Rise Time	RISE-TIME	R	SRM	Time
Time Asymmetry	TIME-ASYM	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Undershoot	UNDERSHOOT	R	-RM	Ratio
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.27 STEP SIGNAL****16.27.1 Definition**

An electrical signal that changes from one amplitude to another that is either more positive or more negative.

16.27.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
DC Offset	DC-OFFSET	R	SRM	Voltage
Fall Time	FALL-TIME	R	SRM	Time
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Overshoot	OVERSHOOT	R	-RM	Ratio
Preshoot	PRESHOOT	R	-RM	Ratio
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Ringing	RINGING	R	-RM	Ratio
Rise Time	RISE-TIME	R	SRM	Time
Transition Time	TRANS-TIME	R	S - -	Time
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.28 SUP CAR SIGNAL (Suppressed Carrier Signal)****16.28.1 Definition**

An amplitude modulated sinewave generated when the amplitude of the carrier (Higher Frequency) wave is a product of the instantaneous signed amplitude of another (modulating) wave wherein the latter may have negative values producing a phase reversal of the carrier wave.

16.28.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Burst Length	BURST	I	SR-	Pulses
Carrier Frequency	CAR-FREQ	R	SRM	Frequency
Carrier Phase Angle *	CAR-PHASE	R	-RM	Angle (plane)
Carrier Residual Voltage	CAR-RESID	R	SRM	Voltage
Carrier Current (RMS) ***	CURRENT	R	SRM	Current
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Modulation Frequency	MOD-FREQ	R	SRM	Frequency
Modulation Offset (apparent DC) **	MOD-OFFSET	R	S - -	Voltage
Modulation Phase Angle *	MOD-PHASE	R	-RM	Angle (plane)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Standing Wave Ratio	SWR	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time
Voltage (Carrier in RMS) ***	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

NOTES: * Phase angle is relative to the signal at the CAR-REF HI and LO pins. In some cases, the reference signal may also be a SUP CAR signal, providing both CAR-REF and MOD-REF simultaneously. In such cases, CAR-REF and MOD-REF may be combined into a single REF signal.

**Determined by the appearance of the modulated signal. It is often called "zero offset" in servo applications.

***Determined at the peak of modulation cycle.

16.0 NOUNS AND THEIR MODIFIERS**16.29 SYNCHRO****16.29.1 Definition**

A combination of three in-phase sinusoidal electrical signals whose amplitude relationships describe the rotational shaft position of an electro-mechanical device.

16.29.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Angle	ANGLE	R	SRM	Angle (plane)
Angle Rate	ANGLE-RATE	R	SRM	Angle (plane), Time
Frequency	FREQ	R	SRM	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Harmonics	HARMONICS	R	SRM	Voltage
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Non-Harmonics	NON-HARMONICS	R	-RM	Voltage
Phase Shift	PHASE-SHIFT	R	SRM	Angle (plane)
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Transition Time	TRANS-TIME	R	S - -	Time
Voltage	VOLTAGE	R	SRM	Voltage
Zero Index	ZERO-INDEX	R	S - -	Angle (plane)

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.30 TIME INTERVAL**16.30.1 Definition**

The time between two or more events related to a common time reference.

16.30.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Time Interval	TIME	R	-RM	Time

16.30.3 Rules

1. When TIME INTERVAL is used in an <identify timer statement> it relates to a time reference which is initiated from one point in time.

16.0 NOUNS AND THEIR MODIFIERS**16.31 TRIANGULAR WAVE SIGNAL****16.31.1 Definition**

A periodic waveform whose instantaneous value varies alternately and linearly between two specified values (initial and alternate). The interval required to change from the initial value to the alternate value must be equal to the interval to change back from the alternate value to the initial value.

16.31.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AC Current	CURRENT	R	SRM	Current
Bandwidth	BANDWIDTH	R	SRM	Frequency
Burst Length	BURST	I	SR-	Burst Length
DC Offset	DC-OFFSET	R	SRM	Voltage
Frequency	FREQ	R	SRM	Frequency
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Non-Linearity	NON-LIN	R	SRM	Ratio
Peak Degeneration	PEAK-DEGEN	R	SRM	Ratio
Period	PERIOD	R	SRM	Time
Phase Jitter	PHASE-JIT	R	SRM	Angle (plane)
Power	POWER	R	SRM	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Time Asymmetry	TIME-ASYM	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Voltage	VOLTAGE	R	SRM	Voltage

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

16.0 NOUNS AND THEIR MODIFIERS**16.32 VIBRATION****16.32.1 Definition**

A physical characteristic, either applied to or measured on the UUT, which represents a periodic motion at a specified point on the UUT. The periodic motion may be represented in units of displacement, velocity, or acceleration.

This noun uses the pin descriptor TO.

16.32.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Frequency	FREQ	R	SRM	Frequency
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Transition Time	TRANS-TIME	R	S - -	Time
Vibration Amplitude	VIBRATION-AMPL-x *	R	SRM	Length
Vibration Amplitude Acceleration	VIBRATION-ACCEL	R	SRM	Acceleration
Vibration Amplitude Rate of Change	VIBRATION-RATE	R	SRM	Velocity (linear)

Note: * for x one of the following suffixes must be used: -P, -PP, -TRMS.

16.32.3 Rules**16.32.4 Notes and Examples**

To measure vibration in terms of acceleration:

```
000050 DECLARE, VARIABLE, 'FREQ WINDOW' IS ARRAY
      (1 THRU 2) OF REAL INITIAL = (10, 40) $
```

```
000601 VERIFY, (VIBRATION-AMPL-PP), VIBRATION,
      VIBRATION-AMPL-PP MAX 3.2 MILS,
      FREQ-WINDOW 'FREQ WINDOW' (1 thru 2) HZ,
      CNX TO ACCESS-PORT $
```

To apply a vibration to a UUT:

```
000602 APPLY, VIBRATION, VIBRATION-AMPL-P 5 MILS,
      FREQ 3 HZ, CNX TO MOUNT-4 $
```

16.0 NOUNS AND THEIR MODIFIERS**16.33 VOR (VHF Omnidirectional Radio Range)****16.33.1 Definition**

A navigational aid operating at VHF and providing radial lines of position in any direction as determined by bearing selection within the receiving equipment. It emits a variable modulation whose phase relative to a reference modulation is different for each bearing of the receiving point from the station as defined in ARINC Specification 579.

16.33.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
Carrier Amplitude	CAR-AMPL	R	S --	Voltage, Power
Carrier Frequency	CAR-FREQ	R	S --	Frequency
Frequency Window	FREQ-WINDOW	RA	-R-	Frequency
Harmonics	HARMONICS	R	-RM	Voltage, Ratio
Identification Signal	IDENT-SIG*	MD	S --	(None)
Identification Signal Frequency (Steady Tone)	IDENT-SIG-FREQ	R	S --	Frequency
Identification Signal Modulation	IDENT-SIG-MOD	R	S --	Ratio
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Modulation Distortion	MOD-DIST	R	-RM	Voltage, Ratio
Modulation Index	MOD-INDEX	R	S --	(None)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Non-Harmonic Distortion	NON-HARMONICS	R	-RM	Voltage, Ratio
Radial Bearing	RADIAL	R	S --	Angle (plane)
Radial Bearing Rate	RADIAL-RATE	R	S --	Velocity (angular)
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Reference Phase Modulation Frequency	REF-PHASE-FREQ	R	S --	Frequency
Resistance	xxx-RES	R	SR-	Impedance (real)
Subcarrier Frequency	SUB-CAR-FREQ	R	S --	Frequency
Subcarrier Modulation	SUB-CAR-MOD	R	S --	Ratio
Standing Wave Ratio	SWR	R	SRM	Ratio
Transition Time	TRANS-TIME	R	S --	Time
Variable Phase Modulation Frequency	VAR-PHASE-FREQ	R	S --	Frequency
Variable Phase Modulation Amplitude	VAR-PHASE-MOD	R	S --	Ratio

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

NOTE: * Modifier Descriptors are as follows:-
For modifier IDENT-SIG: Station identification letters.

16.0 NOUNS AND THEIR MODIFIERS

16.33.3 Rules None

16.33.4 Notes and Examples

VOR provides bearing information relative to its location based on magnetic north at the VOR. No range information is provided. The governing document is ARINC Specification 579.

The VOR radiates a radio frequency carrier in the band 111.975 MHz to 117.975 MHz, at 50 KHz increments up to 117.950 MHz, which are associated with two separate 30 Hz modulations. The phase of one of these modulations is independent of the point of observation (reference phase); the phase of the other modulation (variable phase) is such that, at a point of observation, it differs from the reference phase by an angle equal to the bearing of the point of observation with respect to the VOR.

The two separate modulations consist of:

1. A subcarrier of 9960 HZ, frequency modulated at 30 Hz, modulating the carrier to a nominal depth of 30%.
 - a. This 30 Hz component is fixed without respect to azimuth and termed the Reference Phase for conventional VOR.
 - b. This 30 Hz component varies with azimuth and is termed the Variable Phase for Doppler VOR.
2. A 30 Hz amplitude-modulation component modulating the carrier to a nominal depth of 30%.
 - a. This 30 Hz component is caused by a rotating antenna producing a change in phase with azimuth and is termed the Variable Phase for conventional VOR.
 - b. This 30 Hz component, having a constant phase relative to azimuth and a constant amplitude, is radiated omnidirectionally and is termed the Reference Phase for Doppler VOR.

Additional modulations which may be present are a communication channel which may have a peak modulation depth up to 30% and an Identification Signal of two or three letters of Morse Code. Depth of modulation due to the Identification Signal is nominally 10%; when there is a communications channel, the modulation depth may be up to 20%.

16.0 NOUNS AND THEIR MODIFIERS**16.34 WAVEFORM****16.34.1 Definition**

A time-varying electrical signal whose waveshape is a complex function of time. For SOURCE statements the waveform is specified as a set of amplitude values contained in a SAMPLE <array> with the waveform amplitude specified. The waveform itself may be measured in the form of a set of amplitude samples (measured characteristic mnemonic SAMPLE) that are stored in the specified SAMPLE <array>.

16.34.2 Noun Modifier Set

<u>Modifier</u>	<u>Mnemonic</u>	<u>Type</u>	<u>Usage</u>	<u>Quantity</u>
AC Current	CURRENT	R	S - -	Current
Bandwidth	BANDWIDTH	R	SR-	Frequency
Burst Length (number of stimulus cycles)	BURST	I	S - -	Burst Length
DC Offset	DC-OFFSET	R	SRM	Voltage
Fundamental Frequency	FREQ	R	SRM	Frequency
Fundamental Period ***	PERIOD	R	SRM	Time
Impedance Angle	xxx-IMP-ANGLE	R	SR-	Angle (plane)
Impedance Magnitude	xxx-IMP-MAGNITUDE	R	SR-	Impedance (real)
Noise	NOISE	R	SRM	Ratio, Voltage, Current, Power
Power	POWER	R	S - -	Power
Reactance	xxx-REACTANCE	R	SR-	Impedance (real)
Resistance	xxx-RES	R	SR-	Impedance (real)
Sample of Voltage Amplitudes	SAMPLE	RA	SRM	(None)
Sample Spacing ***	SAMPLE-SPACING	R	-R-	Time
Sample Time (set of samples)	SAMPLE-TIME	R	-R-	Time
Sample Width (duration) ***	SAMPLE-WIDTH	R	-R-	Time
Transition Time	TRANS-TIME	R	S - -	Time
Trigger	TRIG	R	-R-	Voltage
Voltage (instantaneous) **	VOLTAGE	R	SRM	Voltage
Voltage Ramped (Interpolation Requirement) *	VOLTAGE-RAMPED	MO	S - -	(None)
Voltage Stepped (Interpolation Requirement) *	VOLTAGE-STEPPED	MO	S - -	(None)

Where: xxx-An Impedance Modifier Prefix being UUT- or TEST-EQUIP- or THRU- as defined in Section 17.2.1.

NOTES: * VOLTAGE-STEPPED indicates that the voltage levels indicated by the list elements are each to be held for a single time interval so that the resulting waveform consists of a series of steps.
VOLTAGE-RAMPED indicates that the voltage levels indicated are to be obtained at successive time intervals. During each time interval a linear voltage ramp is to be generated from the preceding to the succeeding value so that the resulting waveform consists of a series of connected linear ramps.

** Since it is an instantaneous voltage the suffix INST should be used.

*** Figure 16.6 illustrates the definition of PERIOD and SAMPLE-TIME with SAMPLE-SPACING and SAMPLE-WIDTH.

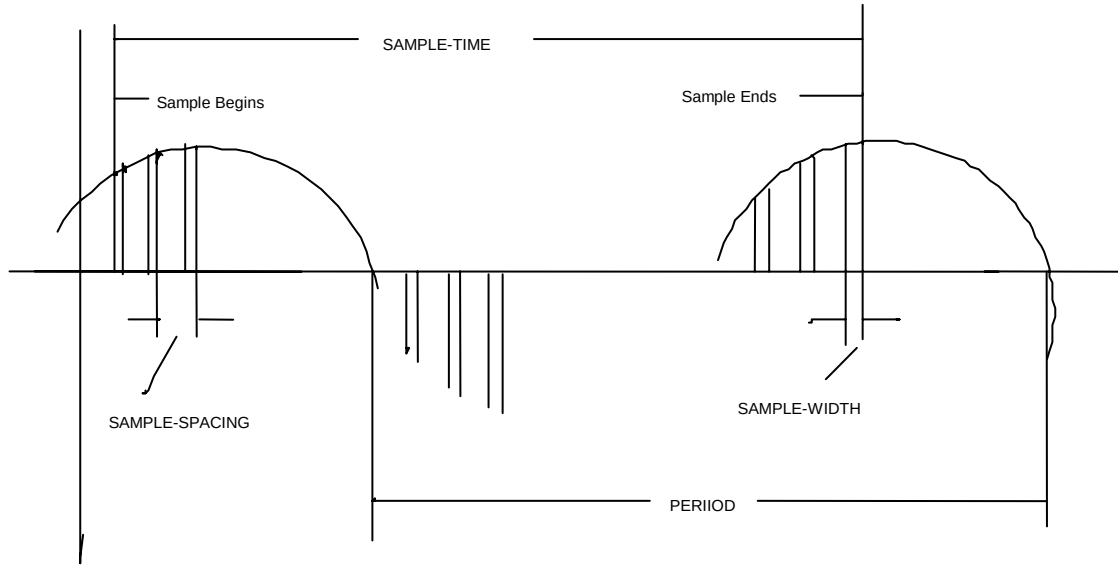
16.0 NOUNS AND THEIR MODIFIERS

Figure 16-11: Definition of PERIOD, SAMPLE-TIME, SAMPLE-WIDTH, and SAMPLE-SPACING

16.34.3 Rules

16.34.3.1 Rules applicable to both SOURCE and SENSOR type statements

1. The stimulus waveform or the measurement sampling may be synchronized to a reference waveform via the SYNC field in the WAVEFORM statement and the SYNC WHEN statement referenced in the SYNC field.
2. The DC-OFFSET is added to the list of stimulus values in a source statement and is subtracted from the waveform before the samples are stored in a sensor statement.
3. The range of voltages within the signal is specified by the associated RANGE, MAX or MIN path of <real characteristic subfield> following the SAMPLE modifier.

16.34.3.2 Rules applicable to SOURCE type statements

1. The SAMPLE field is compulsory and has the form SAMPLE <array range> dim RANGE <dimensioned number> TO <dimensioned number> (or MAX/MIN <dimensioned number>). It specifies the storage locations that define the stimulus waveform and the range of voltages for each value.
2. The fundamental frequency (or period) of the waveform, that is the repetition frequency of the set of the amplitude numbers, is specified as either FREQ or PERIOD.
3. The time increment between successive amplitude numbers is equal to the PERIOD, or the reciprocal of FREQ, divided by the number of elements in the <array range> of the SAMPLE field.

16.0 NOUNS AND THEIR MODIFIERS

16.34.3.3 Rules applicable to SENSOR type statements

1. The measured characteristic SAMPLE causes the specified number of amplitude samples to be taken and stored in the <array range> specified in the INTO field of the measured characteristic. No values are stored in 'MEASUREMENT'.
2. The verb VERIFY cannot be used with the <measured characteristic> SAMPLE.
3. If SAMPLE-SPACING is not specified, the time increment between successive samples is equal to the PERIOD, or the reciprocal of FREQ, divided by the number of samples as specified by the number of elements in the <array range> of the INTO field in the SAMPLE <measured characteristic>.

17.0 NOUN-MODIFIER DEFINITION

This paragraph lists and defines the noun-modifiers used with the nouns identified in Chapter 16.0. The modifiers are listed by their reserved ATLAS word (mnemonic).

17.1 Mnemonics for Pulse Type Signals

A graphical representation of some of the mnemonics used with pulse type signals is depicted in Figure 17-1.

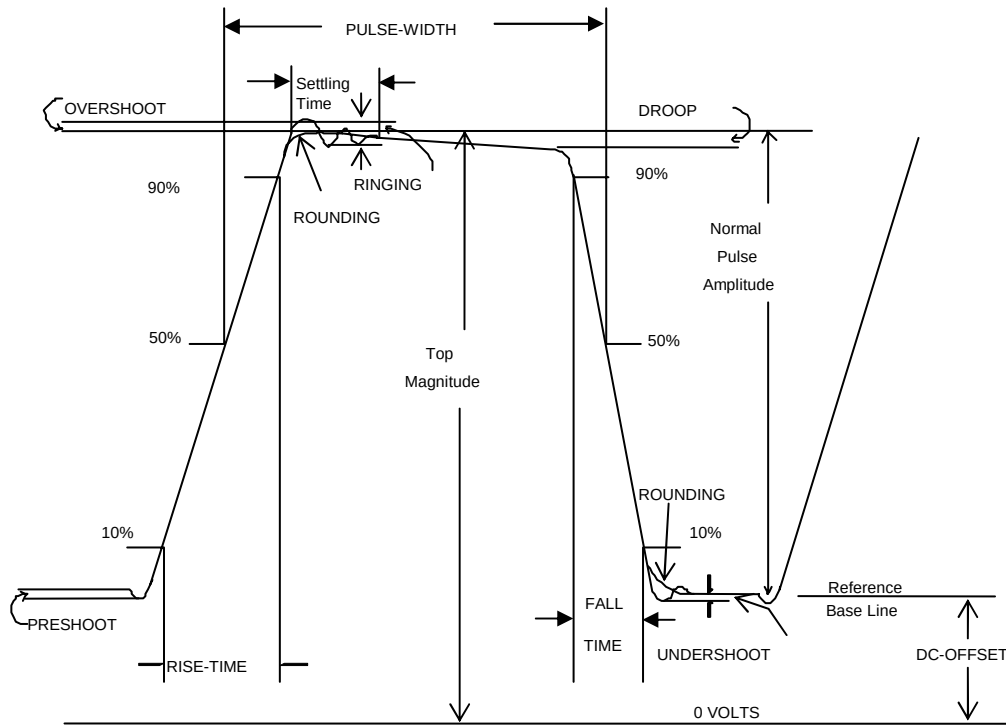


Figure 17-1: Pulse Definition

17.0 NOUN-MODIFIER DEFINITION**17.2 Modifier Prefixes and Suffixes**

Many of the noun-modifier mnemonics may have prefixes or suffixes to further define the type, use, and application of the mnemonics. These prefixes and suffixes are defined in this section. In general, in the noun-modifier lists in Chapter 16 and in the modifier definitions given subsequently in this section, only the basic mnemonic (without prefixes or suffixes) is included. Prefixes and suffixes are shown separately in the index to this volume.

When multiple suffixes are used in a noun-modifier <mnemonic> their ordering must be as follows:

first-	Amplitude Modifier Suffixes	(See Section 17.2.3)
	Phase Identifier Suffixes	(See Section 17.2.4)
	Pulse Identifier Suffixes	(See Section 17.2.8)
	Ratio Modifier Suffixes	(See Section 17.2.5)
	Quiescent Signal Suffixes	(See Section 17.2.9)
	Signal Transition Suffixes	(See Section 17.2.10)
last-	Reference Phase Identifier Suffixes	(See Section 17.2.12)

17.2.1 Impedance Modifier Prefixes

Impedance characteristics may be specified using a real number and one or more of the modifiers: RES, REACTANCE, IMP-MAGNITUDE, and IMP-ANGLE. When used with LOAD type statements, these modifiers are used without any prefixes.

Impedance and related modifiers, when used with signal oriented nouns must use one of the prefixes UUT-, TEST-EQUIP-, or THRU-. In the Chapter 16 modifier lists, these prefixes are represented as a group as xxx-. The meaning of these basic modifiers is given in Chapter 17 under the modifier name.

The meaning of the prefixes are as follows:

- a. UUT The modifier specifies characteristics of the internal impedance of the UUT port connected to the test equipment. For a SOURCE statement, it characterizes the UUT input impedance; for a SENSOR statement, it characterizes the UUT output impedance.
- b. TEST-EQUIP For a SOURCE statement, the modifier characterizes test equipment output impedance; for a SENSOR statement, it characterizes the test equipment input impedance.
- c. THRU The modifier specifies characteristics of an impedance between the UUT and the test equipment. Such an impedance may cause voltages at the test system port to be different from the voltages at corresponding UUT ports. Associated voltages are stated at the test equipment end of the impedance. This constitutes an exception to the normal ATLAS rule that all quantities are referred to the UUT terminals.

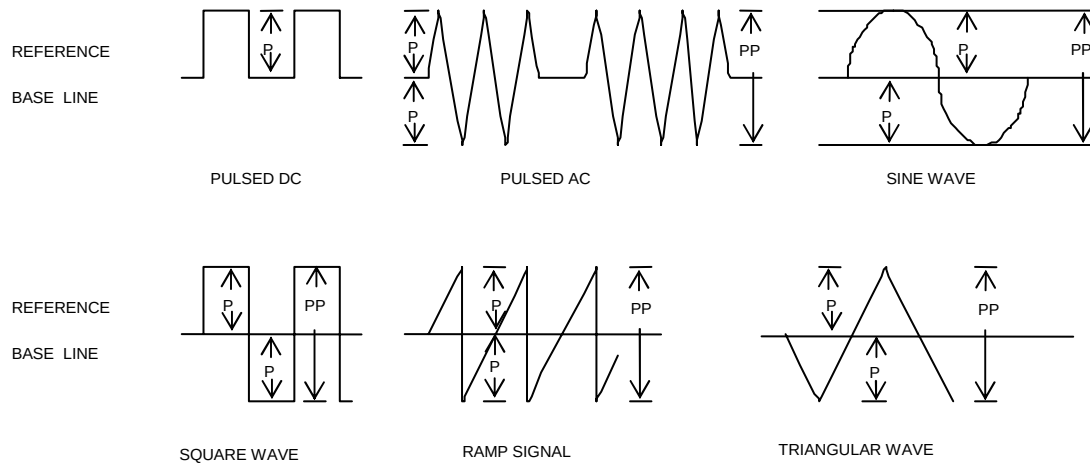
17.2.2 Amplitude Modifier Suffixes**17.2.2.1 Voltage and Current Suffixes**

These suffixes, as applied to VOLTAGE and CURRENT, are defined as follows:

The type of a parameter may be designated as suffixes to applicable modifier mnemonics. For example, the following suffixes can be applied to the modifier VOLTAGE:

- a. VOLTAGE (no suffix)

The generally used voltage parameter which assumes a distortionless signal for a valid RMS value.

17.0 NOUN-MODIFIER DEFINITION**Figure 17-2: Peak and Peak-to-Peak Illustrations****b. VOLTAGE-TRMS**

The true rms parameter which indicates an accurate value regardless of the signal distortion.

c. VOLTAGE-AV

The parameter indicating the average signal value.

d. VOLTAGE-PP

The parameter indicating the peak-to-peak value of a signal. As shown in Figure 17-2, this dimension is measured from the positive peak to the negative peak of a signal.

e. VOLTAGE-P

The parameter indicating the peak value of a signal. As illustrated in Figure 17-2, this parameter is measured from the Reference Base Line of a signal to its positive or negative peak. The Reference Base Line of a signal is defined as the signal's zero value level. In practical terms the level of the Reference Base Line is determined in one of four ways as follows:

- (i) It is stated explicitly as a fixed DC-OFFSET without error limit, range, max or min. The actual value may be determined at execute time and held in a <data store>. DC-OFFSET is measured from the zero signal level to the Reference Base Line.
- (ii) Except in the case of PULSED DC, PULSED AC and SQUARE WAVE, if the DC-OFFSET is not stated then the Reference Base Line is taken as that level at which for half the cycle the signal is greater than the level and for half the cycle it is less than the level.
- (iii) In the special case of a SQUARE WAVE, the Reference Base Line is defined as what would be 50% of the normal pulse amplitude if it were a PULSED DC with an appropriately adjusted Reference Base Line.
- (iv) For PULSED DC or PULSED AC, the Reference Base Line is directly determined as the actual zero value level during the "space" time.

17.0 NOUN-MODIFIER DEFINITION

f. VOLTAGE-P-POS, VOLTAGE-P-NEG

The parameters indicating the positive peak and the negative peak, respectively. They are normally used for asymmetric waveforms. (See Figure 17-3.)

g. VOLTAGE-IN-PHASE, VOLTAGE-QUAD

The parameters indicating the in-phase or quadrature components. A REF connection field must be provided for the reference signal. When either of these suffixes is combined with any of the preceding suffixes, these suffixes are written last. For example: VOLTAGE-TRMS-IN-PHASE.

h. VOLTAGE-INST

The parameter indicating the instantaneous value of the signal with reference to zero volts. (See Figure 17-3.)

i. VOLTAGE-P-POS-ZERO, VOLTAGE-P-NEG-ZERO

The parameters indicating the positive peak and the negative peak, respectively, of the signal with reference to zero volts. They are normally used for asymmetric waveforms. (See Figure 17-3.)

j. VOLTAGE-ONE, VOLTAGE-ZERO

These parameters are used with digital signals to identify the signal levels which are associated with the 1 and 0 levels in a TRANS field. The suffices can be employed with any mnemonic which carries logic information.

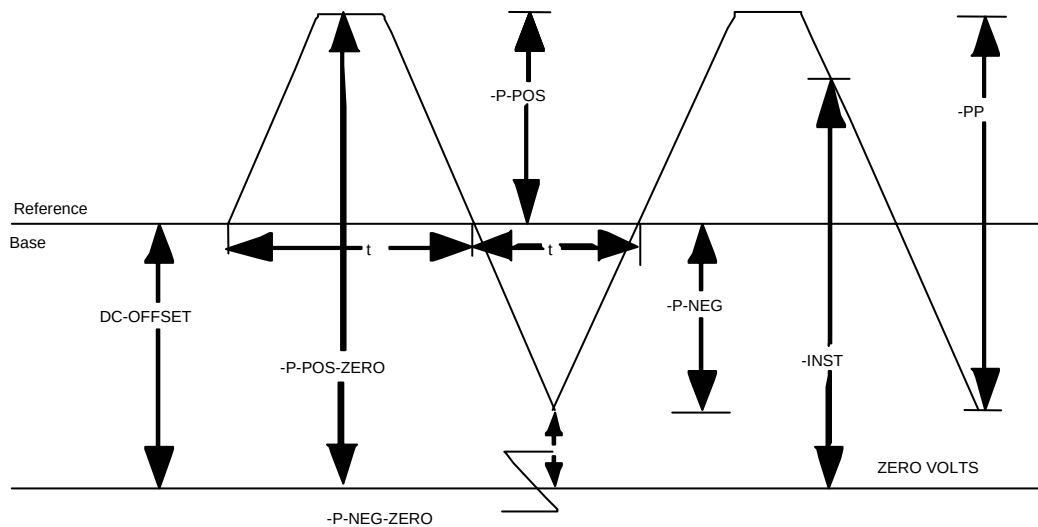


Figure 17-3: Illustration of Positive and Negative Peaks of an Asymmetrical Waveform

17.0 NOUN-MODIFIER DEFINITION**17.2.2.2 Power Suffixes**

The Amplitude Modifier Suffixes, as applied to the modifier POWER, are defined as follows:

- a. POWER-AV The parameter indicating the average value of power.
- b. POWER-P The parameter indicating the peak value of power.

17.2.3 Phase Identifier Suffixes

The following phase identifiers may be used as suffixes with modifiers such as VOLTAGE, CURRENT, or IMP to form a modifier that refers to a particular phase of a multi-phase signal:

PHASE-A	PHASE-AB	PHASE-AC
PHASE-B	PHASE-BC	PHASE-CB
PHASE-C	PHASE-CA	PHASE-BA

Use of + or - signs to indicate in-phase AC or out-of-phase AC, respectively, is permitted. Any of the legal phase identifiers established for the UUT connector field can be used in this phase mnemonic.

17.2.4 Ratio Modifier Suffixes

The dimensionless result of the division of two magnitudes having the same dimensions is a ratio. A ratio may be indicated by the suffix RATIO to the applicable modifier mnemonics in the measured characteristic field.

Example: FREQ-RATIO
 VOLTAGE-RATIO
 VOLTAGE-PP-RATIO
 VOLTAGE-QUAD-RATIO

When the RATIO suffix is used, it means that two UUT-oriented signals are required, and that the quantity of significance is their ratio. The numerator and denominator signals are described by using the suffixes NUMERATOR and DENOMINATOR with the appropriate noun modifier, and identified by the use of these suffixes in the CNX field. If there are statement characteristics common to both, then the suffix may be omitted.

Example: VERIFY, (FREQ-RATIO), AC SIGNAL, UL 6 LL 4,
 VOLTAGE-DENOMINATOR 12 V,
 VOLTAGE-NUMERATOR 18 V,
 FREQ-DENOMINATOR MAX 10 KHZ,
 FREQ-NUMERATOR MAX 60 KHZ,
 CNX-NUMERATOR HI SK1-B LO SK1-C
 CNX-DENOMINATOR HI SK1-F LO SK1-G \$

For a sensor statement, the ratio of the signals READ at the respective numerator and denominator UUT pins is stored in 'MEASUREMENT.'

17.0 NOUN-MODIFIER DEFINITION**17.2.5 Pulse Identifier Suffixes**

The pulse identifier suffix Px is used for correlating a modifier to a single pulse in a pulse train, where the integer x identifies a particular pulse. Pulses are numbered in sequence with P1 representing the first pulse in the pulse train. The pulse identifier suffixes POS and NEG designate the positive slope and negative slope, respectively, of the pulse identified by the suffix immediately preceding the POS or NEG suffix. The slope suffixes must be used in pairs; that is

-Px-POS-Py-POS,	-Px-POS-Py-NEG,
-Px-NEG-Py-POS,	-Px-NEG-Py-NEG.
PULSE-WIDTH-P3	SPACING-P1-POS-P2-POS
VOLTAGE-P-P3	SPACING-P5-POS-P8-NEG

17.2.6 Quiescent Signal Suffix

The Quiescent Signal Suffix QUIES is used in source and load statements to define the value to be assumed by a particular signal characteristic as follows:

- a. When a statement includes an initial coincidence synchronization, the <statement characteristics> field which contains a suffix QUIES, defines the signal value in the period prior to synchronization. The term initial coincidence synchronization is defined in Section 14.3.
- b. When a statement includes a <gate field>; the <statement characteristics> field which contains a suffix QUIES defines the signal value existing outside the time slot defined by the <gate field>. The <statement characteristics> field which does not include a noun-modifier with the suffix QUIES defines the signal value within the time slot defined by the <gate field>.

When using the suffix QUIES the following Rules apply:

1. Where appropriate, the suffix QUIES is written immediately following the suffixes defined in Sections 17.2.3 thru 17.2.8.
2. The suffices QUIES and TRANS must not both be used with one noun-modifier in a single <statement characteristics> field.
3. For each <statement characteristics> field including a noun-modifier having the suffix QUIES, there must be another <statement characteristics> field including the same noun-modifier without the suffix QUIES so that the signal level is specified both during the active and quiescent periods.

17.2.7 Signal Transition Suffix

The Signal Transition Suffix TRANS is used to define the signal variations which are acceptable to a UUT when changing the signal value to a new level from a quiescent level or a previously set level.

The term initial coincidence synchronization is defined in Section 14.3.

When using the suffix TRANS the following Rules apply:

1. Where appropriate, the suffix TRANS must be written immediately following the suffixes defined in Sections 17.2.3 thru 17.2.8.
2. The suffices TRANS and QUIES must not be used with the same noun modifier in one <statement characteristics> field.

17.0 NOUN-MODIFIER DEFINITION

3. For each <statement characteristics> field including a noun-modifier having the suffix TRANS, there must be another <statement characteristics> field including the same noun-modifier without the suffix TRANS which defines the value of the settled characteristic after transition.
4. The suffix TRANS must be followed by the units associated with the noun-modifier unless the suffix RATE is also used, whereupon the rate extension to the same unit must be used, for example, /SEC, /MIN, /HR.
5. When the suffix RATE is not used, the <statement characteristics> field must contain MAX, MIN or RANGE. These define the acceptable limits of the transient excursion, such as undershoot or overshoot, from the nominal during the transition.
6. When the suffix RATE is used, the <statement characteristics> field can contain MAX, MIN or RANGE. These then define the acceptable limits for the first differential of the transient excursion from the nominal during the transition.
7. When the suffix RATE is used without MAX, MIN or RANGE it means that a controlled transition rate is required.
8. The value in a <statement characteristics> field containing a noun-modifier having the suffices RATE and TRANS is the modulus of the acceptable rate value and is therefore always positive.
9. If the transition characteristics are not defined the ATLAS will not control the way in which the test equipment executes the transition.

17.2.8 Reference Phase Identifier Suffix

The REF identifier may be used as a suffix with noun modifiers when a REF followed by a <conn set> subfield is used in the <conn> field. The <mnemonic> -REF defines the characteristics of the REFERENCE signal.

Example: MEASURE, (ANGLE), SYNCHRO,
 ANGLE RANGE 0 DEG TO 100 DEG,
 FREQ-REF 400 HZ,
 VOLTAGE-REF 26 V,
 CNX X J1 Y J2 Z J3
 REF HI J6 LO J7 \$

In this example, the synchro reference is coming from UUT pins J6 and J7.

17.0 NOUN-MODIFIER DEFINITION

17.3 NOUN MODIFIERS

Formal Syntax

FSD / 3842

AC-COMP (AC Component)

1. Definition

An AC voltage or current component which is present in a signal other than a defined AC SIGNAL.

AC-COMP-FREQ (AC-Component Frequency)

1. Definition

The frequency of an AC component voltage or current signal.

AM-COMP (Amplitude Modulation Component)

1. Definition

The peak-to-peak value of an amplitude-modulation component expressed in volts or as a ratio of this value to the fundamental wave peak amplitude.

2. Rules

1. AM-COMP must be expressed as a peak-to-peak value in volts or as a ratio of peak-to-peak amplitude to the carrier-wave peak amplitude.

ANGLE, ANGLE-s

1. Definition

The application or measurement of the angle of rotation of a mathematical vector or mechanical device. Angle rotation about a three axis orthogonal reference frame may be specified. Section 16.24.4, Rules 1 thru 4 define the allowable coordinate reference frames and the meaning of various angle modifiers. ANGLE as applied to SCORSBY motion is defined under SCORSBY in this chapter.

ANGLE-ACCEL, ANGLE-ACCEL-w (Angle Acceleration)

1. Definition

Angle acceleration is the rate of change in angular rate or the second derivative of angle. Positive acceleration is defined by the right hand rule referenced to the axis of rotation. Section 16.24.4, Rules 1 thru 4 define the allowable coordinate reference frames and the meanings of the various angle modifiers.

ANGLE-RATE, ANGLE-RATE-w

1. Definition

The rate of change of angle or the first derivative of angle. Sections 16.24.4, Rules 1 thru 4 define the various angle modifiers and coordinate frames.

17.0 NOUN-MODIFIER DEFINITION**ATTEN (Attenuation)****1. Definition**

The ratio between the power returned to a receiver and the power transmitted.

BANDWIDTH**1. Definition**

The difference between the upper and lower frequencies of either a frequency selective circuit where 3 dB attenuation occurs.

BAROMETRIC-PRESS (Barometric Pressure)**1. Definition**

This modifier describes the ambient barometric pressure in the test cell. It is normally expressed in inches of mercury or pounds per square inch.

BIT-RATE**1. Definition**

Digital data bit rate

BURST**1. Definition**

The number of pulses or cycles of a stimulus waveform to be applied. The BURST modifier is applicable to repetitive signals where a requirement might arise for a limited burst of the signal to be injected rather than a continuous signal. A BURST field indicates the number of cycles of a simple signal such as AC, the number of modulation envelope cycles of a modulated signal, or the number of pulses of a pulsed signal which are to be generated. The BURST signal starts and ends at the reference base level unless a phase angle modifier is employed if appropriate to the particular noun.

2. Rules

- a. The critical action of the stimulus statement is the appearance of the first pulse or cycle of the stimulus signal which appears at the UUT. Execution of the next statement starts following this critical action. When the BURST modifier is used with single-action verb sequences, the pulses or cycles are initiated when a <connect statement> acts on a signal in the open state, or a <setup statement> acts on a signal in the closed state.

CAP (Capacitance)**1. Definition**

The ratio of electrical charge (q) to electrical potential (V) in a circuit ($C = q/V$).

CAR-AMPL (Carrier Amplitude)**1. Definition**

The amplitude of the unmodulated carrier wave.

17.0 NOUN-MODIFIER DEFINITION

CAR-FREQ (Carrier Frequency)

1. Definition

The time-average frequency of the carrier wave signal in the absence of modulation.

CAR-HARMONICS (Carrier Harmonics)

1. Definition

Any integral multiple of the defined carrier frequency occurring in the carrier signal.

CAR-PHASE (Carrier Phase)

1. Definition

Phase angle of the carrier relative to the signal at the REF HI and REF LO pins.

CAR-RESID (Carrier Residual)

1. Definition

Amplitude of a residual carrier signal.

COUNT

1. Definition

Number of events or items.

CURRENT

1. Definition

The rate of flow of electrical charge. In the case of DC signals the rate is unvarying. In the case of AC signals a distortionless sinusoidal flow rate is assumed.

CURRENT-LMT (Current Limit)

1. Definition

Maximum safe current applicable without damage.

DC-OFFSET

1. Definition

The DC voltage or current level of a defined reference base line. DC-OFFSET voltage or current is from zero volts to Reference Base Line as shown in Figure 17-1 for pulse definition.

DELAY

1. Definition

- a. The time interval between the leading edge of two pulses.

17.0 NOUN-MODIFIER DEFINITION**DISS-FACTOR (Dissipation Factor)****1. Definition**

The ratio of the series resistance to the series reactance expressed as a percentage of the series reactance.

2. Formal Syntax**DISTORTION****1. Definition**

The total of all or a chosen portion of the distortion elements, such as HARMONICS, NON-HARMONICS, NOISE, and PHASE-JITTER, in a signal.

DOPPLER-BANDWIDTH**1. Definition**

The difference between the upper and lower frequencies of the returned doppler signal where the 6dB attenuation occurs.

DOPPLER-BANDWIDTH-POWER**1. Definition**

Power distributed between the upper and lower frequencies of the returned doppler signal where the 6dB attenuation occurs.

DOPPLER-FREQ (Doppler Frequency)**1. Definition**

The received frequency of a signal that is shifted from the emitted signal by a delta frequency called the Doppler Shift.

DOPPLER-SHIFT (Doppler Frequency)**1. Definition**

The magnitude of the change in the observed frequency of a wave due to the Doppler Effect.

DROOP**1. Definition**

The difference between the normal pulse amplitude and the amplitude to which the instantaneous pulse amplitude sags or falls at the trailing edge of the pulse, expressed as a percentage of the normal pulse amplitude (see Figure 17-1).

DUTY-CYCLE**1. Definition**

The ratio between the on-time of square wave or pulsed DC signals and the total period.

17.0 NOUN-MODIFIER DEFINITION

FALL-TIME

1. Definition

The time interval during which the instantaneous amplitude of a pulse decreases (falls towards the reference base line) from 90 percent to 10 percent of the normal pulse amplitude (see Figure 17-1).

FM-COMP (Frequency Modulation Component)

1. Definition

The peak-to-peak frequency deviation of a frequency-modulation component which is present in a signal other than an FM SIGNAL, expressed in units of frequency.

FM-INDEX (Frequency Deviation Ratio)

1. Definition

The ratio of the maximum frequency deviation to the maximum modulating frequency.

FREQ (Frequency)

1. Definition

The rate at which a periodic function is repeated, expressed in the unit Hertz (cycles per second).

FREQ-DEV (Frequency Deviation)

1. Definition

The peak difference between the instantaneous frequency of the modulated wave and the carrier frequency.

FREQ-DEV-RATE (Frequency Deviation Rate)

1. Definition

The rate of change in the difference between the instantaneous frequency of the modulated wave and the carrier frequency.

FREQ-LMT

1. Definition

The frequency above which a SHORT is no longer of negligible impedance.

FREQ-WINDOW (Frequency-Window)

1. Definition

A frequency range that allows only those signals that are within this range to contribute to a measurement result. The range is specified by two elements of an array.

17.0 NOUN-MODIFIER DEFINITION

2. Notes and Examples

The parameters NON-HARMONICS, HARMONICS, NOISE etc. may be measured over a frequency range defined by the FREQ-WINDOW modifier. The modifier FREQ or CAR-FREQ defines the frequency of the fundamental from which the harmonics are derived. The modifier BANDWIDTH defines the +3 dB to -3 dB frequency range of the detector which is normally used to sweep the FREQ-WINDOW to determine the value of the required characteristic. The fundamental itself may, but need not, lie in the FREQ-WINDOW. The FREQ-WINDOW is always used with an array of real values.

```
000070 DECLARE, VARIABLE, 'FREQ WINDOW' IS ARRAY (1 THRU 2)
      OF REAL INITIAL = (11.0, 30.0) $

001000 MEASURE, (NON-HARMONICS), AC SIGNAL,
      FREQ 10 MHZ, VOLTAGE 10 DBM,
      NON-HARMONICS RANGE -60 DBM to -20 DBM,
      FREQ-WINDOW 'FREQ WINDOW' (1 THRU 2) MHZ,
      BANDWIDTH 100 HZ, CNX HI P1-1 LO P1-0 $
```

HARM-***-PHASE (Phase of the *** Harmonic)

1. Definition

The symbol *** represents an integer, such as n, that identifies a harmonic component of the signal. HARM-***-PHASE designates the phase of the n-th harmonic relative to the fundamental, where n = 1 represents the waveform fundamental component.

HARM-***-VOLTAGE (Voltage of the *** Harmonic)

1. Definition

The symbol *** represents an integer, such as n, that identifies a harmonic component of the signal. HARM-***-VOLTAGE designates the rms voltage amplitude of the nth harmonic, where n = 1 represents the waveform fundamental component.

HARMONICS

1. Definition

Any integral multiple of the defined fundamental frequency occurring in a signal.

IDENT-SIG (Identification Signal)

1. Definition

Identifies and establishes parameters of the identification signal.

IDENT-SIG-FREQ (Identification Signal Frequency)

1. Definition

Specifies the frequency of the signal used for identification in air navigation systems.

17.0 NOUN-MODIFIER DEFINITION

2. Rules

- a) Designates a constant tone at the specified frequency for the identification signal.

IDENT-SIG-MOD (Identification Signal Modulation)

1. Definition

The percentage modulation of a signal containing coded information used for identification in air navigational systems. Units are Percent or Decibels.

ILLUM (Illuminance)

1. Definition

The quotient of the luminous flux incident on an element of a surface and the area of that element.

IMP (Impedance)

1. Definition

Used as a modifier:

- a) in LOAD type statements to specify the value of a load impedance
- b) as a measured characteristic in a SENSOR type statement with the noun IMPEDANCE.

2. Rules

- a) The IMP modifier is optional for all LOAD type statements.
- b) Can be used as a SENSOR measured characteristic with the noun IMPEDANCE.
- c) All analog usage conventions apply to the use of these modifiers with passive linear impedances.
- d) Nonlinear impedances must be defined via a DEFINE 'drawing'.

IMP-ANGLE (Impedance Angle)

1. Definition

The angle of an impedance $Z = R + jX$ is $\arctan R/X$. The prefixes UUT-, TEST-EQUIP- and THRU- must be used in specifying impedance characteristics with any signal oriented noun.

IMP-MAGNITUDE (Impedance Magnitude)

1. Definition

The magnitude of an impedance $Z = R + jX$ is $\text{SQRT} (R^2 + X^2)$. Prefixes UUT-, TEST-EQUIP- and THRU- must be used in specifying impedance characteristics with any signal oriented noun.

17.0 NOUN-MODIFIER DEFINITION**IND (Inductance)****1. Definition**

The ratio of electrical potential to the rate of change of current in an electro-magnetic circuit.

INSERTION-LOSS**1. Definition**

The change in wave power resulting from the insertion of a UUT in the transmission system. The ratio of, (a) the power delivered to that part of the system following the UUT, before insertion of the UUT, to (b) the power delivered to that same part of the system after insertion of the UUT.

2. Rules

- a) When INSERTION-LOSS is used as a measured characteristic, the input connection has to be defined by CNX-DENOMINATOR and the output connection by CNX-NUMERATOR.

LUM-FLUX (Luminous Flux)**1. Definition**

A measure of the ability of a radiant source to produce a visual response in a detector which conforms to the CIE Standard Photometric Observer, as defined by the Commission Internationale d'Eclairage (C.I.E.) for photopic vision (for normal human eye adapted to daytime levels of luminance of several candelas per square meter) used as the standard for photometry.

LUM-INT (Luminous Intensity)**1. Definition**

The capacity of a light source to produce visual sensation.

LUMINANCE**1. Definition**

The quotient of the luminous intensity leaving, incident on or passing through an element of a surface and the area of the orthogonal projection of that element on a plane perpendicular to a given direction.

MOD-AMPL (Modulation Amplitude)**1. Definition**

The ratio of the modulation signal amplitude to the carrier wave amplitude expressed (1) as a percentage of the carrier wave amplitude or (2) in units of inches of mercury or Bars.

MOD-DIST (Modulation Distortion)**1. Definition**

Distortion introduced into the resultant signal by the modulating process.

17.0 NOUN-MODIFIER DEFINITION

MOD-FREQ (Modulation Frequency)

1. Definition

The frequency of the signal which modulates an electrical carrier wave or manometric pressure.

MOD-INDEX (Modulation Index)

1. Definition

The ratio of frequency deviation to the imposed modulating frequency.

MOD-OFFSET (Modulation Offset)

1. Definition

Apparent DC offset of a modulating signal as determined from the appearance of the modulated signal. It is often called "zero offset" in servo applications.

MOD-PHASE (Modulation Phase)

1. Definition

Modulation phase angle relative to the signal at REF HI and REF LO pins.

NEG-EDGE (Negative Edge)

1. Definition

A transition having an unspecified negative derivative.

NEG-SLOPE (Negative Slope)

1. Definition

A transition having a specified negative derivative.

NOISE

1. Definition

Random signal components at random frequencies whose power increases with bandwidth.

NOISE-AMPL-DENS (Noise Amplitude Density)

1. Definition

The square root of the average of the squares of all the noise components in a signal per unit of bandwidth.

17.0 NOUN-MODIFIER DEFINITION**NOISE-PWR-DENS (Noise Power Density)****1. Definition**

The mean square noise power per unit of bandwidth.

NON-HARMONICS**1. Definition**

A frequency which is not an integral multiple of the defined fundamental frequency.

NON-LIN (Non-Linear)**1. Definition**

The maximum instantaneous amplitude variation from a linear waveform expressed as a percentage of the peak amplitude.

OSCILLATORY**1. Definition**

Oscillation or oscillatory motion is a form of rotation in which the angular position as well as all positional derivatives are sinusoidal in nature. Oscillation may be specified about one or more axes about a fixed reference frame.

2. Rules

- a) If oscillation is required about only one axis, an axis suffix modifier is optional. The specification of oscillation about two or more axes requires the use of the suffixes -X, -Y and -Z as required. The magnitude of the oscillation is specified by the noun modifier ANGLE. The modifiers ANGLE-RATE and ANGLE-ACCEL are not applicable. The period of oscillation is specified by the modifier FREQ.

OVERSHOOT**1. Definition**

The difference between the normal pulse amplitude and the peak amplitude to which the instantaneous pulse waveform initially rises positively or negatively away from the reference base line (see Figure 17-1).

P-AMPL (Peak Pulse Amplitude)**1. Definition**

Peak voltage amplitude of a modulated pulsed carrier.

PEAK-DEGEN (Peak Degeneration)**1. Definition**

Rounding of the corner of a triangular or ramp signal expressed in percent.

17.0 NOUN-MODIFIER DEFINITION

PERIOD

1. Definition

The period (1/frequency) of a waveform, pulse, or pulse train. Can be used as a statement characteristic or a measured characteristic.

PHASE-ANGLE

1. Definition

The angular difference expressed in degrees between the signal described by the noun and the signal referenced by the REF HI and REF LO pins.

PHASE-DEV (Phase Deviation)

1. Definition

The deviation ratio of a phase modulated signal.

PHASE-JIT (Phase Jitter)

1. Definition

Random, low amplitude oscillations of a signal wave shape along the time axis.

PHASE-SHIFT

1. Definition

The difference in electrical angle between an input signal characteristic and the same output signal characteristic.

POS-EDGE (Positive Edge)

1. Definition

A transition having an unspecified positive derivative.

POS-SLOPE (Positive Slope)

1. Definition

A transition having a specified positive derivative.

POWER

1. Definition

The rate at which work is done. This modifier specifies either the amount of input power required by the UUT or the amount of output power provided by the UUT.

17.0 NOUN-MODIFIER DEFINITION**PRESHOOT****1. Definition**

The amplitude difference between the reference base line of a pulse and the lowest value to which the pulse waveform falls at the leading edge of the pulse. (See Figure 17-1.)

PRF (Pulse Repetition Frequency)**1. Definition**

The time rate at which pulses occur.

PULSE-CLASS**1. Definition**

This modifier identifies the type of pulse coding for serial digital signals.

2. Rules

- a) The PULSE-CLASS is required for all SERIAL DATA statements, for both SOURCE and SENSOR. Cannot be used as a measured characteristic in SENSOR statements.
- b) The form of this modifier is class PULSE-CLASS.
- c) The modifier descriptor identifies the class of pulse coding used to represent digital data. The following classes are defined:

RZ	Return to Zero
NRZ	Non Return to Zero
BIP	BI-Phase
AMI	Alternate Mark Inversion
HDBn	High Density Bipolar where n is an unsigned integer number

3. Notes

The various pulse classes are described in Section 6.19.3.5.

PULSE-ENERGY**1. Definition**

The amount of energy delivered per pulse in a pulsed signal. Mathematically; if $P(t)$ is the pulse power as a function of time then: PULSE-ENERGY equals the integral of $P(t)dt$, where the integration is performed over the duration of the pulse.

PULSE-WIDTH**1. Definition**

The time interval measured between the 50 percent amplitude points of the leading and trailing edge of a single pulse. (See Figure 17.1.)

17.0 NOUN-MODIFIER DEFINITION

PWR-LMT (Power Limit)

1. Definition

Maximum safe power applicable without damage.

RADIAL

1. Definition

A direction from magnetic north taken in a clockwise direction. The dimension is expressed in degrees.

2. Rules

- a) When used with the noun VOR, RADIAL indicates the magnetic bearing from the ground station.

RADIAL-RATE

1. Definition

RADIAL-RATE is the time rate of change of the RADIAL direction expressed as degrees per second. The rate of change is considered positive when measured in clockwise direction and negative when measured in counter-clockwise direction.

REACTANCE

1. Definition

The imaginary component X of impedance when impedance is expressed as a complex number $Z = R + jX$. The prefixes UUT-, TEST- EQUIP- and THRU- must be used in specifying impedance characteristics with any signal oriented noun.

REF-INERTIAL (Reference Inertial)

1. Definition

Specifies that the associated angle, angle rate, and/or angle acceleration modifiers refer to an inertial reference frame.

REF-PHASE-FREQ (Reference Phase Modulation Frequency)

1. Definition

An electrical signal which is used as a reference in determining the phase relationship of another signal. As used with the noun VOR, it is the frequency of a modulating signal which is used as a reference in determining the phase relationship of another signal.

REF-UUT (Reference, Unit Under Test)

1. Definition

This specifies that the associated angle, angle rate, and/or angle acceleration modifiers refer to a UUT reference frame.

REF-VOLT (Reference Voltage)

1. Definition

The voltage level to which the circuit being measured is biased.

17.0 NOUN-MODIFIER DEFINITION**RELATIVE-HUMIDITY****1. Definition**

The water content in the air relative to the amount of water vapor needed to saturate the air at the current temperature.

REPLY-EFF (Reply Efficiency)**1. Definition**

The number of replies divided by the number of interrogations.

RES (Resistance)**1. Definition**

The real portion of an electrical quantity determined by the voltage/current ratio of an electrical circuit.

The prefixes UUT-, TEST-EQUIP- and THRU- must be used in specifying impedance characteristics with any signal oriented noun.

RESP (Response)**1. Definition**

Response data storage location. The response data read from the UUT.

RETURN-LOSS**1. Definition**

The ratio of the power incident upon a UUT in a transmission system to the power reflected from the UUT.

RINGING**1. Definition**

A distortion in the form of a superimposed damped oscillatory waveform which, when present, usually follows a major transition; measured as the ratio of the maximum peak-to-peak amplitude of the damped oscillation to the normal pulse amplitude. (See Figure 17-1)

RISE-TIME**1. Definition**

The time interval during which the instantaneous amplitude of a pulse or step change increases (positively or negatively from the reference base line) from 10 percent to 90 percent of the normal pulse amplitude. (See Figure 17-1.)

ROUNDING**1. Definition**

Corner rounding on the leading edge of a pulse. The difference between the normal pulse amplitude and the amplitude at which the instantaneous pulse voltage begins to arc toward the normal pulse amplitude, expressed as a percentage of the normal pulse amplitude. (See Figure 17-1).

17.0 NOUN-MODIFIER DEFINITION

SAMPLE

1. Definition

A time ordered set of amplitudes defining the waveshape of a WAVEFORM.

SAMPLE-SPACING

1. Definition

The time between the initiation of two consecutive samples.

SAMPLE-TIME

1. Definition

The total time interval over which a set of samples is acquired.

SAMPLE-WIDTH

1. Definition

The time over which a signal characteristic is averaged.

SCORSBY

1. Definition

A compound sinusoidal phase-related motion about a 3-axis reference frame. Motions about the first two axes are constrained to be equal in amplitude but phase shifted by 90 degrees. Motion about the third axis is 180 degrees out of phase with respect to the first axis with a peak amplitude of the other axes. The frequency of the sinusoid is the same for all three axes.

2. Rules

- a) ANGLE-RATE and ANGLE-ACCEL shall not be used.
- b) ANGLE shall be used to specify SCORSBY amplitude in the first two axes.
- c) Sinusoidal frequency shall be specified by the noun modifier FREQ.

SKEW-TIME

1. Definition

The elapsed time between the appearance of the digital interface of the earliest and latest bits (pulses) of a PARALLEL DATA word.

17.0 NOUN-MODIFIER DEFINITION

2. Rules

- a) Optional with PARALLEL DATA. Cannot be used as a SENSOR measured characteristic.
- b) Standard ATLAS modifier form, generally expressed as a MAX value.
- c) For SOURCE, this modifier defines the maximum permissible skew time. For SENSOR, it defines the maximum skew time expected from the UUT.

SPACING, SPACING-Px-xxx-Py-yyy

1. Definition

The time interval between the 50 percent amplitude points of any two pulse slopes in a pulse train. The modifier may be used with a set of pulse identifier suffixes to identify the particular slopes as shown, where the symbols x and y identify particular pulses and the symbols xxx and yyy each represent POS or NEG. (See Section 17.2) Dimensions are units of time.

2. Rules

- a) The modifier SPACING without any suffix identifies the time interval between adjacent slopes, either of the same pulse or of adjacent pulses.
 - (i) The modifier type is RA.
 - (ii) When the modifier has the form SPACING <array range>, each successive array element as determined by <array range> represents successive time intervals, starting with the first slope.
- b) The modifier SPACING with pulse identifier suffixes identifies the spacing between the xxx slope of pulse Px and the yyy slope of pulse Py.
 - (i) The modifier type is R.

STIM (Stimulus)

1. Definition

Stimulus data storage location. The stimulus data to be applied to the UUT.

SUB-CAR-FREQ (Sub-Carrier Frequency)

1. Definition

A carrier frequency used to modulate another carrier signal.

2. Rules

When used with the noun VOR, this modifier specifies the modulating frequency of the subcarrier. Units are in Hertz.

SUB-CAR-MOD (Sub-Carrier Modulation)

1. Definition

The degree of modulation of a carrier frequency which is modulated by a sub-carrier frequency. Units are Percent.

17.0 NOUN-MODIFIER DEFINITION

SWEEP-INT (Sweep Interval)

1. Definition

The time required to sweep from one end of the frequency band specified by FREQ-WINDOW to the other end.

SWR (Standing Wave Ratio)

1. Definition

The ratio of the maximum to the minimum amplitude of corresponding components of field (or voltage or current) appearing along the guide or line in the direction of propagation.

2. Rules

- a) A specific value of SWR requires that a specific value of frequency be specified. When frequency is specified as a range, SWR must be specified as a MAX, MIN, or RANGE.

TARGET-RANGE

1. Definition

The distance between a radar signal source and a target surface.

TARGET-RANGE-ACCEL

1. Definition

The time rate of change of a TARGET-RANGE-RATE.

TARGET-RANGE-RATE

1. Definition

The time rate of change of TARGET-RANGE.

TEMP (Temperature)

1. Definition

The molecular kinetic energy condition of a body which determines the transfer of heat to or from other bodies. The TEMP modifier describes the temperature at the point specified in the CNX field.

THREE-PHASE-DELTA

1. Definition

The phases of a three-phase power source are or must be delta-connected.

THREE-PHASE-WYE

1. Definition

The phases of a three-phase power source are or must be wye-connected.

17.0 NOUN-MODIFIER DEFINITION**TIME****1. Definition**

The measurement of time using a standard time measurement function as a reference or the time interval over which EVENTS are counted.

TIME-ASYM (Time Asymmetry)**1. Definition**

A measurement of the variation between two adjacent half cycles. It is calculated as one half of the time difference expressed as a percentage of the sum of the times for the two half cycles.

TIME-JIT (Time Jitter)**1. Definition**

The difference between the period of any two adjacent pulses and the average period of all the pulses, expressed in seconds.

TRANS-TIME (Transition Time)**1. Definition**

The time required for a transition from one signal level to another to stabilize to a steady state within defined error limits.

2. Rules

- a) The modifier TRANS-TIME can be used in a <statement characteristics> field with the verb CHANGE and any verb with a <statement characteristics> field containing a <gate field> or an initial coincidence synchronization field.

TRANSMITTANCE**1. Definition**

The ratio of the transmitted radiant luminous (LIGHT) flux to the incident flux.

TRIG (Trigger)**1. Definition**

The modifier TRIG defines an amplitude level on the signal being measured.

2. Rules

- a) When used with the measured characteristic mnemonics PERIOD or FREQ, the modifier defines the start and stop times for the measurement.
- b) When used with the noun EVENTS, this modifier, in conjunction with other optional modifiers, for example; POS-SLOPE or NEG-SLOPE, defines an <event>.

17.0 NOUN-MODIFIER DEFINITION

TYPE

1. Definition

The TYPE modifier is used with the noun LOGIC SIGNAL to define the serial or parallel type of data transmission to or from the UUT.

2. Rules

- a) The modifier TYPE is required for all LOGIC SIGNAL statements, for both SOURCE and SENSOR. TYPE cannot be used as a measured characteristic in SENSOR statements. In LOGIC SIGNAL statements the TYPE subfield has the form:
 - (i) TYPE PARALLEL
 - (ii) TYPE SERIAL-LSB-FIRST
 - (iii) TYPE SERIAL-MSB-FIRST
- b) For a PARALLEL signal, all bits appear concurrently at the digital interface, each on a separate electrical circuit. For a SERIAL signal, all bits appear at the digital interface sequentially over a single electrical circuit. For a SERIAL signal, this modifier also defines the sequence of bits at the interface as follows:

SERIAL-LSB-FIRST	The least significant bit appears first.
SERIAL-MSB-FIRST	The most significant bit appears first.

All digital representations show the LSB in the right-most bit position.

UNDERSHOOT

1. Definition

The difference between the amplitude of the reference base line of a pulse and the lowest amplitude to which the instantaneous pulse voltage initially falls following the pulse decay expressed as a percentage of the normal pulse voltage. (See Figure 17-1).

VALUE

1. Definition

The VALUE modifier specifies a digital number or bit pattern.

2. Rules

- a) The modifier VALUE in digital statements can be used as the measured characteristic for <prove statement>s and <sense statement>s. In LOGIC SIGNAL statements the VALUE subfield has the form:
 - (i) VALUE N Where N is a <digital number> defined in Section 15.2.7. or VALUE, expression where expression is defined by Syntax Diagram 8.2-B.
 - (ii) N must be equated to any of the standard ATLAS forms (such as B'1011',X'5E',etc.). All numbers used for N are treated as dimensionless quantities.

17.0 NOUN-MODIFIER DEFINITION**VAR-PHASE-FREQ (Variable Phase Frequency)****1. Definition**

The frequency of a signal which is variable in phase from a reference signal. When used with the noun VOR, this modifier specifies the frequency of the modulating signal which is variable in phase from a reference signal. Units are in Hertz.

VAR-PHASE-MOD (Variable Phase Modulation)**1. Definition**

The percentage modulation caused by a modulating signal which is variable in phase from a reference signal. Units are Percent.

VIBRATION-AMPL (Vibration Amplitude)**1. Definition**

Specifies the amplitude of a vibration. It may be a linear displacement, a velocity, or an acceleration.

VIBRATION-ACCEL (Vibration Acceleration)**1. Definition**

The time rate of change of VIBRATION-RATE.

VIBRATION-RATE**1. Definition**

The time rate of change of VIBRATION-AMPL.

VOLTAGE**1. Definition**

An electrical potential. In the case of DC signals the potential is unvarying. In the case of AC signals it is the generally used parameter which assumes a distortionless sinusoidal signal for a valid rms value.

VOLTAGE-RAMPED**1. Definition**

Interpolation rule for the generation of a WAVEFORM from a set of amplitude samples.

2. Rules

- a) This modifier is used in SOURCE type statements having the noun WAVEFORM.
- b) The modifier field contains only the mnemonic.

17.0 NOUN-MODIFIER DEFINITION

VOLTAGE-STEPPED

1. Definition

Interpolation rule for the generation of a WAVEFORM from a set of amplitude samples.

2. Rules

- a) This modifier is used in SOURCE type statements having the noun WAVEFORM.
- b) The modifier field contains only the mnemonic.

VOLT-LMT (Voltage Limit)

1. Definition

Maximum safe voltage applicable without damage.

WAVE-LENGTH

1. Definition

The measure of distance between two similar points in a wave motion.

WORD-LENGTH

1. Definition

The number of bits that are contained in a digital word.

2. Rules

- a) Optional for LOGIC SIGNAL statements. Cannot be used as a SENSOR measured characteristic.
- b) Form

WORD-LENGTH b { BITS, DIGITS, CHAR }

WORD-RATE

1. Definition

Number of digital words transmitted per unit time.

ZERO-INDEX

1. Definition

Standard angular reference position from which the various angular shaft positions of a synchro or resolver are determined for a standard direction or rotation.

18.0 ATLAS FORMAL SYNTAX DEFINITION

18.1 Introduction

The Formal Syntax Definition (FSD) of Standard ATLAS describes the construction of valid ATLAS statements within an ATLAS Test Specification.

The FSD includes the definition of those ATLAS Language rules that are consistent with the FSD Production Rules being:

- readable
- subsettable
- concise.

There are some additional ATLAS Language rules which could be formally expressed but only at the expense of added complexity in the FSD.

The FSD production rules do not attempt to define these additional more complex ATLAS Language rules. All the ATLAS language is defined in Chapters 1 thru 17 of this ATLAS Language Specification.

The following sections complete the Formal Definition:-

- a cross reference list of all occurrences of the syntax variables and symbols - Section 18.7.1.
- a cross reference list of all occurrences of ATLAS literals - Section 18.7.2.
- a cross reference list of all occurrences of the various types of label in ATLAS - Section 18.7.3.

The cross reference lists provide a quick and easy reference to the different lexical symbols, rule names and literals which occur within the production rules.

The cross reference lists make it easy to find the definition and the usages of a name or a literal. Furthermore, they provide a very convenient overview of the language and demonstrate the completeness of the description.

The cross reference list is divided into three parts; a list of production rule names in Section 18.7.1 and a list of literals in Section 18.7.2 and a list of label types in Section 18.7.3 that occur in the FSD Production Rules.

In the cross reference list of rule names in Section 18.7.1 there are three fields occurring under the headings: Rule Name, Definition and Usage.

Rule Name	Gives the Rule Name.
Definition	Gives the line number in the FSD, where the corresponding Rule Name is defined.
Usage	Gives the line numbers of each use of a Rule Name in rule bodies.

In the cross reference list of literals in Section 18.7.2, the Symbol and the Usage fields respectively contain the literal and the line numbers in which it is referenced.

In the cross reference list of labels in Section 18.7.3 the Label Type, Definition and Usage fields respectively contain the label, the line number in which it occurs in a statement which establishes the label and the line numbers in which the label is employed, having first been established.

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.1.1 Formal Definition of the Syntax**

The formal syntax comprises three parts:-

- Program and Module Structure
- Statements and Fields
- Lexical Symbols.

The program and module structure defines the overall structure and the sub-structures of an <atlas program structure> or an <atlas module structure> which are the components of an <atlas test specification structure>. The relations between an <atlas test specification structure>, its <atlas program structure> and its <atlas module structure>s and <non-atlas module structure>s, as described in Section 3.1, are not covered by the FSD.

Every ATLAS statement appears as a right-hand side in at least one rule of the FSD but is not defined further in this part.

The syntax definition of statements and fields includes the definitions of the ATLAS statements in Chapters 4 and 6 thru 13 and the definitions of fields and subfields Chapter 14.

The lexical symbols represent textual entities within an <atlas test specification structure> such as labels or character strings.

Section 18.7 provides an overall cross reference list including all syntax variables and symbols which occur in the three parts of the Formal Syntax.

18.1.2 References Between the Formal Syntax Definition and the ATLAS Language Specification

To facilitate the parallel use of the ATLAS Language Standard and the Formal Syntax Definition (FSD) the following conventions are introduced:-

- * The names of syntactical variables used in the Specification are also used in the FSD.
- * As an aid to the cross reference, the appropriate section numbers in the Specification where definitions of the syntactical variables are included are referenced in the FSD.
- * The section numbers are written as a comment in the right margin of the rule in the Statement and Fields part of the FSD. In the program and module structure part of the FSD the right-hand sides of the rules are also annotated with the relevant section numbers.

Examples:-

(i) From the program and module structure part:-

block body:	! 4.2.3.
[local-preamble-structure]	! 6.2.
main-procedural-structure ;	! 7.1.

(ii) From the Statement and Fields part:-

end-task-statement:	! 6.11.5.
fstatno "END" fd <task	
statement-terminator ;	

- * With few exceptions the section numbers are in ascending order.

18.0 ATLAS FORMAL SYNTAX DEFINITION

- * If a syntactical sub-construct such as a field is used exclusively in a certain statement, its definition is put immediately after the statement definition. A cross-reference to the appropriate section number is given where the definition of a subconstruct can be found.
- * For the definitions of the lexical symbols in Section 18.5 the section numbers are in ascending order within each of the eight subsets of lexical symbols syntax.
- * The syntactic rules of the FSD which do not have a direct equivalence to a section of the Specification are identified by appending suffices such as "-A" to the section number in the Specification for the appropriate higher level token that has a syntax diagram that references the Rule Name.

Example:-

require-control	! 14.57
...	
require-control-body:	! 14.57-A
...	
require-range:	! 14.57-B

If such a new token for a syntactic rule occurs on the right-hand side of different rules the identification is derived from the section number that identifies the first significant occurrence.

Example:-

expression:	! 8.2-1
-------------	---------

This token occurs on the right-hand side of many rules but is primarily intended for the right-hand side of a calculation statement defined in 8.2.

18.2 Syntax Notation

The following notation which is used to describe the syntax of synthetic languages was originally proposed by DeRemer and Frank in 1974. It was derived from the Backus-Naur Format (BNF). It was extended and was slightly modified to facilitate the formal definition, to simplify the analysis of the syntax of ATLAS and to facilitate the automatic derivation of syntax diagrams from the FSD.

18.2.1 Tokens

The text which represents the formal definition of a language consists of a sequence of tokens. A token can be either an identifier or a literal or an operator.

(i) Identifier Tokens

Identifiers are sequences of lower case letters, digits and hyphens beginning with a letter and followed by zero or more hyphens, letters or digits. Each identifier token must have a defining rule in which the token is written on the left-hand side of its FSD Production Rule.

18.0 ATLAS FORMAL SYNTAX DEFINITION

Examples:

define-procedure structure:

...

rule statements

...

procedure-body:

...

rule statements

...

(ii) Literal Tokens

Literals are explicit character strings. A literal is enclosed in double quotes.

Literals

The following rules apply to the writing of ATLAS literals in an <atlas test specification structure> to correspond to their equivalent in the Formal Syntax Definition (FSD).

1. No spaces shall be included in the written version of an alpha-numeric string that is equivalent to the same alpha-numeric string in an ATLAS literal as delimited by one pair of double quote marks in the FSD.
2. One space in the FSD of an ATLAS literal as delimited by one pair of double quote marks means that one or more spaces can be written in its place in the equivalent literal in an ATLAS test requirement.
3. With the exception of alpha-numeric strings, no space in the FSD of an ATLAS literal as delimited by one pair of double quote marks means that either none or any number of spaces can be written into the equivalent literal in an ATLAS test requirement.

Examples:

"VOLTAGE"

"ATLAS PROGRAM"

(iii) Operator Tokens

Operators are used to define the permitted alternative sequences of language elements in an ATLAS statement as represented by identifiers and literals. The following keywords and special characters are used as operators:

<u>Operator</u>	<u>Meaning</u>
SYNTAX-RULES	beginning of the FSD description
FINIS	end of the FSD description
::	is defined as
% after a token;	repetition of the preceding token 1 or more times without separation
* after a token;	repetition of the preceding token 0 or more times without separation
# between two tokens;	repetition of the preceding token one or more times with repetitions separated by the following token
	separation between options
{ }	grouping brackets
[]	optionality brackets
;	end of rule in the FSD

18.0 ATLAS FORMAL SYNTAX DEFINITION

>	insert into a set
<	select from a set
!	introduction of a comment
<<*..>>	one or more asterisks bounded by double angle brackets at the beginning of an option; references a note that is correspondingly identified by asterisks and written immediately following the FSD Production Rule

18.2.2 Selection

- (i) From a Set defined by the Meta Language "**|**"

Selection of one from a set of alternatives is indicated in a FSD Production Rule by separating the alternatives by vertical bars, "**|**".

Example:

```
a ::
b | c | d ;
```

meaning: "a" is defined as either "b" or "c" or "d".

- (ii) From a Set defined by the ATLAS Program "**<**" and "**>**"

In some cases the ATLAS program defines a set of permissible character strings and subsequently requires that selection be constrained to a member of that set. The operator "**>**" immediately following a token in the FSD indicates that the source code represented by the token is to be added to the set represented by the token from which a subsequently executed FSD Production Rule will require that a selection be made. The requirement for a selection is indicated by preceding the token by a "**<**" operator.

Examples:

```
a ::
"F" b > ;
c ::
"E" < b ;
b ::
digit ;
```

In this case the rule for "c" ensures that the digit following E when "c" is called up must previously have occurred following F in a circumstance where "a" was called up. This construct is used, for instance, to express the need to declare and type a data store before employing it by name.

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.2.3 Grouping****"{ }"**

A sequence of tokens that are enclosed within a pair of grouping brackets "{ }" is processed as if it were a single token.

Examples:

```
a ::
b { c | d } ;
```

meaning: "a" is defined as "b" followed by either "c" or "d" whereas without the grouping brackets:-

```
a ::
b c | d;
```

meaning: "a" is defined as either "b" followed by "c" or "d" only.

18.2.4 Optionality Brackets**"[]"**

A single token or a sequence of tokens that is optional is enclosed within a pair of optionality brackets "[]". The sequence of tokens is processed as if it were a single token.

Example:

```
a ::
b [c d] ;
```

meaning: "a" is defined as "b" which may or may not be followed by "c" and "d".

18.2.5 Repetition**(i) Without Separation****(a) One or More Times****"%"**

When a FSD Production Rule contains a token that may occur one or more times without separation, the token must be followed by a percent sign "%" thereby indicating that the token may be repeated without separation.

Example:

```
a ::
b % ;
```

meaning: "a" is defined as "b" that may be repeated one or more times without separation between each occurrence of "b".

18.0 ATLAS FORMAL SYNTAX DEFINITION

(b) Zero or More Times "*" "

When a FSD Production Rule contains a token that may occur zero or more times without separation, the token must be followed by an asterisk sign "*" thereby indicating that the token may or may not be present it may be present and if present it may be repeated without separation.

Example:

```
a ::
b* ;
```

meaning: "a" is defined as either "null" or "b" and "b" may be repeated one or more times without separation between each occurrence of "b".

(ii) With Separation "#" "

When a FSD Production Rule contains a token that may occur one or more times with each repeated occurrence separated from the previous occurrence by either another token or literal, the repeated token must be followed by a "#" sign followed by the separating token or literal thereby indicating that it may be repeated with separation.

Example:

```
a ::
b # c ;
```

meaning: "a" is defined as either "b" only or "b" followed by alternating occurrences of "c" and "b" one or more times and ending with a "b", i.e. "a" is either "b", or "bcb", or "bcbcb" and so on.

18.2.6 Execution of Operator Tokens

Operator tokens are executed in the following order when they are included in production rules:-

<u>Order</u>	<u>Operator Token</u>	<u>Operation</u>
First	< >	Selection from an ATLAS set
	{ } []	Grouping
	* % #	Repetition
	... b ", " c ...	Chaining
Last		Alternation

Operator tokens of equal precedence that are included within one FSD Production Rule are executed in a left to right order as written.

18.2.7 Comment "!" "

The beginning of comment text is denoted by an exclamation sign "!" to indicate that the information in the comment is for the human reader only. Each line or part line of comment is introduced by a "!" at the beginning of the comment and is terminated by the end of line or the begin new line functional operator.

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.2.8 Layout Blanks**

Blanks are used to separate tokens. If a token can be identified without the need to use blank separators then tokens can be written without blank separators between the tokens.

18.2.9 Note Identification

"<<*,.>>"

When a note is needed to restrict the permissible syntax options by constraining the use of an option on the basis of a semantic consideration, one or more asterisks "*" are included within double angle brackets "<< >>" at the beginning of the constrained option.

Notes are included in the FSD Listing immediately following the FSD Production Rule. Each note is introduced by one or more asterisks "*" according to how many notes relate to a single FSD Production Rule. The first note is always introduced by a single asterisk "*". All notes are terminated by a single asterisk "*". A note can be written on more than one line in the FSD. Every line of a note is introduced by an ampersand "&".

Note identification can occur in four circumstances:-

- (i) At the beginning of a series of options when identification applies to the first option only.
- (ii) Following a vertical line option separator when identification applies to the option it precedes.
- (iii) Immediately preceding the first square bracket of a pair of square brackets when identification applies to the option to omit from the FSD Production Rule those elements within the pair of square brackets as indicated by a bypass in the equivalent syntax diagram.
- (iv) Immediately following the first square bracket of a pair of square brackets when identification applies to those elements in the FSD Production Rule that are within the pair of square brackets.

18.2.10 Coherence between FSD Listing and Syntax Diagrams

The syntax defined by the FSD is also illustrated diagrammatically by Syntax Diagrams. To ensure that there is strict coherence between the Syntax Diagrams and the FSD which takes precedence over the Syntax Diagrams (Section 2.2 refers), automatic generation from the FSD Listing is preferred to manual preparation of the Syntax Diagrams. As previously, to facilitate automatic generation, some minor additions have been included in the FSD Listing so as to generate Syntax Diagrams which conform to the established presentation of the ATLAS Syntax Diagrams in the standard.

The most commonly used Backus Naur Format (BNF) representation includes Double colons (::) as the separator between the left-hand and right-hand sides of the FSD Production Rules. This practice has been used in the FSD Listing for the ATLAS Language and a letter is inserted between the first and second colons to define the relationships between the FSD Production Rules and the generation of the Syntax Diagrams. For the particular software that has been used to generate the Syntax Diagrams in the standard, the letter is either "d" or "e" or "s" meaning:

- d A drawing is required which is equivalent to this FSD Production Rule.
- e The FSD Production Rule is to be explicitly expanded when producing any syntax diagrams which reference this production rule.
- s The FSD Production Rule represents a lexical token and the token name is to appear as the representative of the lexical element.

Example: dim: s:

Meaning: dim is a lexical token which appears as an italic entry in Syntax Diagrams.

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.3 Program Structure Syntax**

The Program Structure Syntax is presented in the following groupings in the Formal Syntax Definition (FSD) in Section 18.6, SYNTAX DEFINITION :-

*	ATLAS TEST SPECIFICATION STRUCTURE	Section 18.6.1
*	STRUCTURE DELIMITER STATEMENTS	Section 18.6.2
*	PREAMBLE STATEMENTS	Section 18.6.3
*	MAIN PROCEDURAL STRUCTURE	Section 18.6.4
*	PROCEDURAL STATEMENTS - DATA PROCESSING	Section 18.6.5
*	PROCEDURAL STATEMENTS - INPUT / OUTPUT	Section 18.6.6
*	PROCEDURAL STATEMENTS - CONTROL	Section 18.6.7
*	PROCEDURAL STATEMENTS - SIGNAL	Section 18.6.8
*	PROCEDURAL STATEMENTS - TIMING	Section 18.6.9
*	PROCEDURAL STATEMENTS - DATABUS	Section 18.6.10
*	FIELDS / SUBFIELDS	Section 18.6.11

18.3.1 Range and Order of the Program Structure Syntax

The Program Structure Syntax defines the high level constructs of ATLAS. A Program Structure defines a fixed sequence of statements and other structures in the ATLAS Language.

Every Structure and Statement of the ATLAS Language is represented by a token within the FSD. The start token is atlas-test-specification-structure which is coherent with the title of Section 3.1, <atlas test specification structure>.

The FSD in Section 18.6 includes the definition of all the fields and subfields that are identified as separate entities in the textual definitions for the ATLAS Structures and Statements.

The order of the Production Rules in the FSD for ATLAS Statements follows that for the textual definitions in Chapters 3 thru 15 of the Specification.

The order of the Production Rules in the FSD for the definitions of fields and subfields follows that for the textual definitions in Chapter 14 of the Specification except where it is more convenient to do so, they are located immediately following the definition of the ATLAS statement to which they relate.

18.3.2 Name Conventions

The same names are used in the FSD as are used in the textual definitions in Chapters 3 thru 17 of the Specification.

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.4 Notes**

The FSD allows cases which are semantically invalid.

The FSD does not fully cover the following:-

- * illegal data type handling.
- * define function usage.
- * correct usage of:-
 - * noun vs. modifier
 - * modifier vs. dimension
 - * signal noun vs. pin descriptors
 - * verb vs. noun
 - * measured characteristic vs. noun
 - * measured characteristic vs. range subfield
 - * minimum necessary modifier set per noun
 - * default modifier conditions
 - * signal state checking:-
 - order
 - flow
 - conflict
 - path
 - values
- * formal
 - number
 - type
 - order / position
 vs. actual parameter verification:-

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.5 Lexical Symbols Syntax****18.5.1 Range and Order of the Lexical Symbols Syntax**

The Lexical Symbols Syntax defines the ATLAS character strings are valid for the ATLAS element that is represented by each Lexical Symbol of the ATLAS language.

Example:

```
unsigned-integer-number: d:                ! 15.2.3
digit % ;
```

Meaning:

The ATLAS element represented by the Lexical Symbol unsigned-integer-number may contain any sequence of the digits 0 thru 9 as defined in the text of Section 15.2.3. The letter 'd' indicates that there is a syntax diagram in Section 15.2.3 which is coherent with the Production Rule.

The Lexical Symbols of ATLAS are presented in the following groupings:-

*	BASIC SYMBOLS / CHARACTER SETS	Section 18.6.12
*	IDENTIFIERS / LITERALS / LABELS	Section 18.6.13
*	SEPARATORS / STATEMENT NUMBERS	Section 18.6.14
*	DESCRIPTORS	Section 18.6.15
*	NOUNS	Section 18.6.16
*	NOUN-MODIFIERS	Section 18.6.17
*	NOUN-MODIFIERS - PREFIXES / SUFFIXES	Section 18.6.18
*	DIMENSIONS	Section 18.6.19

18.5.2 Name Conventions

The meta-variable denoting a Lexical Symbol may be composed of a main part and a suffix. The suffixes are used to establish a grouping of Lexical Symbols for the convenience of the reader. This grouping is irrelevant with respect to the Formal Syntax Definition (FSD).

The following suffixes are used:-

*	set	for	character sets
*	identifier		
*	number		
*	operator		
*	descriptor		
*	noun		
*	dim	for	dimension
*	pfx	for	noun-modifier prefix
*	sfx	for	noun-modifier suffix

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.6 SYNTAX DEFINITION**

```

1      SYNTAX-RULES
2
3
4      ! The comment on the right hand side is the
5      ! Section Number in this ATLAS Standard for the
6      ! textual definition of the FSD Production Rule
7      ! for each ATLAS Construct.
8
9
10     ! 18.6.1  ATLAS TEST SPECIFICATION STRUCTURE
11
12
13     atlas-test-specification-structure: d:                ! 3.1
14     <<*>> atlas-program-structure
15     { { <<*>> atlas-module-structure }* |
16       { <<*>> non-atlas-module-structure }* } ;
17     &* Mandatory. *
18     &** Optional; none or any number. *
19
20
21     atlas-program-structure: d:                          ! 3.1.1
22     begin-atlas-program-statement
23     program-preamble-structure
24     commence-main-procedure-statement
25     main-procedural-structure
26     terminate-atlas-program-statement ;
27
28
29     atlas-module-structure: d:                          ! 3.1.2
30     begin-atlas-module-statement
31     module-preamble-structure
32     terminate-atlas-module-statement ;
33
34
35     ! non-atlas-module-structure : s: ;                  ! 3.1.3
36
37     ! The definitions of non-atlas-module-structure
38     ! and its combination with atlas-module-structure
39     ! are outside the scope of this formal definition.
40
41
42     ! 18.6.2  STRUCTURE DELIMITER STATEMENTS
43
44
45     begin-atlas-program-statement: d:                    ! 4.1.1
46     fstatno "BEGIN" fd "ATLAS PROGRAM"
47     [ program-name> ]
48     statement-terminator ;
49
50
51     terminate-atlas-program-statement: d:                ! 4.1.2
52     fstatno "TERMINATE" fd "ATLAS PROGRAM"
53     [ <program-name ]
54     statement-terminator ;
55
56
57     begin-atlas-module-statement: d:                    ! 4.1.3
58     fstatno "BEGIN" fd "ATLAS MODULE"
59     module-name> statement-terminator ;
60
61
62     terminate-atlas-module-statement: d:                ! 4.1.4
63     fstatno "TERMINATE" fd "ATLAS MODULE"

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

64      <module-name statement-terminator ;
65
66
67      block-structure: d:                                ! 4.2.1
68      begin-block-statement
69      block-body
70      end-block-statement ;
71
72
73      begin-block-statement: d:                          ! 4.2.2
74      fstatno "BEGIN" fd "BLOCK"
75      [ fd block> ]
76      statement-terminator ;
77
78
79      block-body: d:                                     ! 4.2.3
80      [ local-preamble-structure ]
81      {
82          main-procedural-statements |
83          block-structure
84      } % ;
85
86
87      leave-block-statement: d:                          ! 4.2.4
88      fstatno "LEAVE" fd "BLOCK"
89      [ "STEP" statement-number ]
90      statement-terminator ;
91
92
93      end-block-statement: d:                            ! 4.2.5
94      fstatno "END" fd "BLOCK"
95      [ fd <block ]
96      statement-terminator ;
97
98
99      commence-main-procedure-statement: d:             ! 4.3
100     fstatno "COMMENCE" fd "MAIN PROCEDURE"
101     statement-terminator ;
102
103
104     ! 18.6.3  PREAMBLE STATEMENTS
105
106
107     program-preamble-structure: d:                     ! 6.1.1
108     {
109         include-statement |
110         comment-statement
111     } *
112     preamble-fixed-sequence
113     [ preamble-variable-sequence ] ;
114
115
116     preamble-fixed-sequence: d:                         ! 6.1.1A
117     {
118         extend-atlas-statement |
119         comment-statement
120     } *
121     <<*>> [ {
122         require-statement |
123         comment-statement
124     } % ] ;
125     &*      This path is used only in those unusual applications
126     &      when no signal resources are demanded by the program
127     &      or module. When the program or module demands real
128     &      signal resources for its implementation, the ATLAS

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

129      &      test requirement must be written in terms of virtual
130      &      resources as defined by a <require statement>. *
131
132
133      preamble-variable-sequence: d:                                ! 6.1.1B
134      {
135          declare-statement |
136          establish-protocol-statement |
137          comment-statement
138      } %
139      {
140          define-statement-structure |
141          identify-statements |
142          comment-statement
143      } % ;
144
145
146      module-preamble-structure: d:                                ! 6.1.2
147      preamble-fixed-sequence
148      [ preamble-variable-sequence ] ;
149
150
151      local-preamble-structure: d:                                  ! 6.2
152      {
153          declare-statement |
154          comment-statement
155      } % ;
156
157
158      declare-statement: d:                                         ! 6.3
159      fstatno "DECLARE" fd
160      {
161          constant-declaration |
162          type-declaration |
163          variable-declaration
164      }
165      statement-terminator ;
166
167
168      constant-declaration: d:                                       ! 6.3A
169      "CONSTANT" fd
170      {
171          {
172              constant-identifier> "IS"
173              constant
174          } # ";"
175      } ;
176
177
178      type-declaration: d:                                           ! 6.3B
179      "TYPE" fd
180      {
181          {
182              type-identifier> "IS" type
183          } # ";"
184      } ;
185
186
187      variable-declaration: d:                                       ! 6.3C
188      [ "EXTERNAL" fd | "GLOBAL" fd ]
189      "VARIABLE" fd
190      { variable-type # ";" } ;
191
192
193      variable-type: d:                                             ! 6.3D

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

194     { variable-identifier> # fd }
195     "IS" type
196     [ initial ] ;
197
198
199     type: d:                                ! 6.3E
200     {
201         <type-identifier |
202         base-type |
203         pre-declared-type |
204         structured-type
205     } ;
206
207
208     base-type: d:                            ! 6.3F
209     {
210         "ENUMERATION ("
211         { enumeration-element> # fd }
212         ")" |
213         "CONNECTION ("
214         { connection # fd }
215         ")" |
216         "DECIMAL" |
217         "INTEGER" |
218         "CHAR" |
219         "BIT"
220     } ;
221
222
223     pre-declared-type: d:                    ! 6.3G
224     { "BOOLEAN" | "CHAR-CLASS" | "DIG-CLASS" } ;
225
226
227     structured-type: d:                      ! 6.3H
228     {
229         "ARRAY ("
230         {
231             { constant "THRU" constant } # fd
232         }
233         ") OF" type |
234         "RECORD OF ["
235         {
236             {
237                 { record-field> # fd }
238                 "IS" type
239             } # ";"
240         } "]" |
241         "STRING (" constant ") OF" type |
242         "FILE OF"
243         { type | "TEXT" }
244     } ;
245
246
247     initial: d:                              ! 6.3I
248     "INITIAL =" initialiser ;
249
250
251     initialiser: d:                          ! 6.3J
252     {
253         {
254             constant "OF"
255             {
256                 constant |
257                 "(" initialiser ")"
258             } |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

259         "(" initialiser ")" |
260         constant
261     } # fd
262 } ;
263
264
265 define-statement-structure: d:                                ! 6.4
266 {
267     define-drawing-statement |
268     define-signal-statement |
269     define-procedure-structure |
270     define-function-structure |
271     define-exchange-configuration-statement |
272     define-digital-configuration-structure |
273     define-exchange-statement
274 } ;
275
276
277 define-drawing-statement: d:                                ! 6.5
278 fstatno "DEFINE" fd drawing> fd
279 "DRAWING" fd drawing-identifier <<~>>
280 drawing-definition
281 statement-terminator ;
282
283
284 drawing-definition: e:                                      ! 6.5-1
285 [ fd "TERMINALS" terminal-identifier> %
286 [ fd conn ] ] ;
287
288
289 define-signal-statement: d:                                ! 6.6
290 fstatno "DEFINE" fd signal> fd
291 "SIGNAL" fd
292 fields
293 statement-terminator ;
294
295 ! fields is not defined further.
296 ! NOTE in Section 6.8.3 refers.
297
298
299 define-procedure-structure: d:                                ! 6.7.1
300 define-procedure-statement
301 <<*>> [ procedure-body ]
302 end-procedure-statement;
303 &* This bypass must be used if the <define
304 & procedure statement> has the EXTERNAL
305 & attribute. It must not be used otherwise.*
306
307
308 define-procedure-statement: d:                                ! 6.7.2
309 fstatno "DEFINE" fd procedure> fd
310 [ "EXTERNAL" fd | "GLOBAL" fd ]
311 "PROCEDURE" parameter-definition
312 statement-terminator ;
313
314
315 parameter-definition: d:                                      ! 6.7.2A
316 [
317     "("
318     { formal-parameter # fd }
319     ")"
320 ]
321 [
322     "RESULT ("
323     { formal-parameter # fd }

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

324         ")"
325     ] ;
326
327     ! Subsequent usage of parameters in the FSD must
328     ! comply with the declared type. Therefore
329     ! <parameter does not normally occur in the FSD.
330
331
332     procedure-body: d:                                ! 6.7.3
333     [ local-preamble-structure ]
334     main-procedural-structure ;
335
336
337     leave-procedure-statement: d:                      ! 6.7.4
338     fstatno "LEAVE" fd <procedure
339     statement-terminator ;
340
341
342     end-procedure-statement: d:                        ! 6.7.5
343     fstatno "END" fd <procedure
344     statement-terminator ;
345
346
347     define-function-structure: d:                      ! 6.8.1
348     define-function-statement
349     function-body
350     end-function-statement ;
351
352
353     define-function-statement: d:                      ! 6.8.2
354     fstatno "DEFINE" fd function> fd
355     [ "EXTERNAL" fd | "GLOBAL" fd ] <<~>>
356     "FUNCTION"
357     [ "(" { formal-parameter # fd } ")" ] type
358     statement-terminator ;
359
360
361     function-body: d:                                  ! 6.8.3
362     [ local-preamble-structure ]
363     main-procedural-structure ;
364
365
366     leave-function-statement: d:                       ! 6.8.4
367     fstatno "LEAVE" fd <function
368     statement-terminator ;
369
370
371     end-function-statement: d:                         ! 6.8.5
372     fstatno "END" fd <function
373     statement-terminator ;
374
375
376     require-statement: d:                              ! 6.9
377     fstatno "REQUIRE" fd
378     { requirement> # fd } fd
379     [ "EXTERNAL" fd | "GLOBAL" fd ] <<~>>
380     signal-requirement
381     statement-terminator ;
382
383
384     signal-requirement: d:                             ! 6.9A
385     require-analog <<~>>
386     [ require-control ]
387     [ require-capability ]
388     [ require-limit ]

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

389      <<*>> [ require-cnx ] ;
390      &*      This bypass must be used with nouns that do not
391      &      require a CNX field, e.g. TIME INTERVAL. *
392
393
394      require-analog: e:                                ! 6.9-1
395      {
396          "SENSOR ("
397          { measured-characteristic-mnemonic # fd }
398          ")" |
399          "EVENT MONITOR ("
400          monitored-characteristic-mnemonic ")" |
401          "SOURCE" |
402          "LOAD" |
403          "TIMER"
404      }
405      fd noun ;
406
407
408      monitored-characteristic-mnemonic: s:                ! 6.9-2
409      measured-characteristic-mnemonic ;
410
411
412      include-statement: d:                                ! 6.10
413      fstatno "INCLUDE" fd
414      { "NON-ATLAS MODULE" | "ATLAS MODULE" }
415      {
416          { module-name } # fd
417      }
418      statement-terminator ;
419
420      ! The module name is established by an independently
421      ! compiled ATLAS or NON-ATLAS MODULE. It is not
422      ! referenced in any other statement and therefore
423      ! the module name has no directive prefix "<" nor ">".
424
425
426      establish-protocol-statement: d:                    ! 6.11
427      fstatno "ESTABLISH" fd
428      [ "EXTERNAL" fd | "GLOBAL" fd ]
429      "BUS-PROTOCOL" protocol> fd
430      "SPEC" bus-specification * fd
431      databus-definition statement-terminator ;
432
433
434      databus-definition: d:                                ! 6.11A
435      bus-installation-parameters fd
436      bus-redundancy-definition
437      [ fd "CNX" pin-descriptor % ] ;
438
439
440      bus-redundancy-definition: d:                        ! 6.11B
441      { "STANDARD" | "FAULT-TEST" }
442      [ "PRIMARY" | "REDUNDANT" ]
443      "BUS" <<~>>
444      [
445          fd "ALTERNATE-BUS-TRANSMIT" <protocol
446          [ "DELAY" time-quantity ]
447      ] ;
448
449
450      bus-installation-parameters: d:                      ! 6.11C
451      test-equip-bus-role
452      [
453          fd "TEST-EQUIP-MONITOR"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

454         {
455             {
456                 "COMMAND" |
457                 "STATUS" |
458                 "DATA"
459             } # fd
460         }
461     ] fd <<~>>
462     { exchange-model # fd }
463     [ fd establish-bus-parameters ]
464     [ fd establish-protocol-parameters ] ;
465
466 test-equip-bus-role: d:                                ! 6.11D
467 "TEST-EQUIP-ROLE"
468 {
469     {
470         <<*>> <test-equip-role-name |
471             "MONITOR" |
472             "MASTER" |
473             "SLAVE"
474         } # fd
475     } ;
476     &*      The name must be one that has been introduced
477     &      by an EXTEND statement of the associated type.*
478
479
480
481 exchange-model: d:                                    ! 6.11E
482 "BUS-MODE"
483 {
484     <<*>> <bus-mode-name |
485         "TALKER-LISTENER" |
486         "ALL-LISTENER" |
487         "AUTONOMOUS" |
488         "CON-MODE" |
489         "CON-RT" |
490         "RT-CON" |
491         "RT-RT"
492     }
493     [ fd "TALKER" establish-device ] <<~>>
494     [ fd "LISTENER" establish-device ]
495     establish-bus-info <<~>>
496     [ "INITIALIZE" [ fd "COMMAND" type ] [ fd "DATA" type ] ] ;
497     &*      The name must be one that has been introduced by an
498     &      EXTEND statement of the associated type.*
499
500
501 establish-device: d:                                    ! 6.11F
502 {
503     {
504         <<*>> device-identifier-name> |
505         "TEST-EQUIP" |
506         "UUT"
507     } # sfd
508 }
509 [ "(" type ")" ] ;
510 &*      The name must be one that has been introduced by an
511 &      enumeration declaration. *
512
513
514 establish-bus-parameters: d:                            ! 6.11G
515 fd "BUS-PARAMETER" bsfd
516 {
517     {
518         [ fetch-update-spec ]

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

519         bus-parameter-range
520     } # fd
521 } ;
522
523
524 bus-parameter-range: d:                                ! 6.11H
525 {
526     <<*>> <bus-parameter-name |
527     "RESPONSE-TIME" |
528     "ZERO-AMPLITUDE" |
529     "ONE-AMPLITUDE" |
530     "ZERO-CROSSING" |
531     "MESSAGE-GAP" |
532     "WORD-LENGTH" |
533     "WORD-GAP"
534 }
535 [ bus-parameter-value-range ] ;
536 &*     The name must be one that has been introduced
537 &     by an EXTEND statement of the associated type.*
538
539
540 fetch-update-spec: d:                                ! 6.11I
541 {
542     "UPDATABLE-FETCHABLE" |
543     "FETCHABLE" |
544     "UPDATABLE"
545 }
546 {
547     "PROTOCOL-EXCHANGE" |
548     "PROTOCOL" |
549     "EXCHANGE"
550 } ;
551
552
553 bus-parameter-value-range: d:                        ! 6.11J
554 {
555     "RANGE" dimensioned-constant
556     "TO" dimensioned-constant
557     [ "BY" dimensioned-constant ] |
558     { "MAX" | "MIN" }
559     dimensioned-constant
560 } ;
561
562
563 dimensioned-constant: d:                            ! 6.11K
564 constant dim ;
565
566
567 establish-protocol-parameters: d:                    ! 6.11L
568 "PROTOCOL-PARAMETER" bsfd
569 {
570     {
571         [ fetch-update-spec ]
572         protocol-parameter-types
573     } # fd
574 } ;
575
576
577 protocol-parameter-types: d:                        ! 6.11M
578 {
579     {
580         {
581             "COMMAND-WORD" |
582             "STATUS-WORD" |
583             "DATA-WORD"

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

584         }
585     }
586     "PULSE-CODE" |
587     "PARITY" |
588     "LENGTH" |
589     "SYNC" |
590     "T-R"
591 }
592 }
593 {
594     <<*>> <protocol-parameter-name
595     [ "(" type ")" ] |
596     "WORD-COUNT"
597 }
598 } ;
599 &*      The name must be one that has been introduced
600 &      by an EXTEND statement of the associated type.*
601
602
603 establish-bus-info: d:                                ! 6.11N
604 [ fd "COMMAND (" type ")" ]
605 [ fd "DATA (" type ")" ] <<~>>
606 [ fd "STATUS (" type ")" ] ;
607
608
609 identify-statements: d:                                ! 6.12
610 {
611     identify-timer-statement |
612     identify-signal-based-event-statement |
613     identify-event-based-event-statement |
614     identify-event-interval-statement |
615     identify-event-indicator-statement |
616     identify-time-based-event-statement
617 } ;
618
619
620 identify-timer-statement: d:                            ! 6.13
621 fstatno "IDENTIFY" fd
622 [ "EXTERNAL" fd | "GLOBAL" fd ]
623 "TIMER" timer> "TIME INTERVAL"
624 "USING" <requirement fd
625 "TIME" real-characteristic-subfield
626 statement-terminator ;
627
628
629 identify-signal-based-event-statement: d:                ! 6.14
630 fstatno "IDENTIFY" fd
631 [ "EXTERNAL" fd | "GLOBAL" fd ]
632 "EVENT" event> "AS"
633 { analog-event | digital-event }
634 statement-terminator ;
635
636
637 analog-event: d:                                         ! 6.14A
638 noun-field fd
639 signal-based-event-body
640 statement-characteristics fd <<~>>
641 [ "TEST-EQUIP-DELAY" time-quantity fd ]
642 conn ;
643
644
645 digital-event: d:                                       ! 6.14B
646 "( VALUE ) REF" expression
647 [
648     {

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

649         fd "MASK-ONE" |
650         fd "MASK-ZERO"
651     }
652     expression
653 ]
654 event-sensing fd on-field ;
655
656
657 signal-based-event-body: d:                                ! 6.14C
658 characteristic-mnemonic "EQ"
659 expression
660 [ real-errlim ]
661 signal-changes ;
662
663
664 signal-changes: d:                                         ! 6.14D
665 [ fd "INCREASING" | fd "DECREASING" ]
666 [
667     fd "HYSTERESIS" real-quantity
668     [ real-errlim ]
669 ] ;
670
671
672 event-sensing: d:                                         ! 6.14E
673 {
674     fd "MINIMUM-SENSE-RATE" expression dim |
675     fd "SENSE-EVENT" event
676 } ;
677
678
679 identify-event-based-event-statement: d:                  ! 6.15
680 fstatno "IDENTIFY" fd
681 [ "EXTERNAL" fd | "GLOBAL" fd ]
682 "EVENT" event> "AS"
683 event-based-event-definition
684 statement-terminator ;
685
686
687 event-based-event-definition: d:                            ! 6.15A
688 {
689     time-quantity "AFTER" <event |
690     <event gate-field |
691     "MANUAL-INTERVENTION" |
692     identify-event-every-after
693 } ;
694
695
696 identify-event-every-after: d:                              ! 6.15B
697 "EVERY" unsigned-integer-number
698 "OCCURRENCES OF" <event <<~>>
699 [
700     "AFTER"
701     [ unsigned-integer-number "OCCURRENCES OF" ]
702     <event
703 ] ;
704
705
706 identify-event-interval-statement: d:                      ! 6.16
707 fstatno "IDENTIFY" fd
708 [ "EXTERNAL" fd | "GLOBAL" fd ]
709 "EVENT INTERVAL"
710 event-interval> "AS"
711 identify-event-from-to
712 statement-terminator ;
713

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

714
715 identify-event-from-to: d:                                ! 6.16A
716 "FROM" <event
717 [ "FOR" time-quantity | "TO" <event ] ;
718
719
720 identify-event-indicator-statement: d:                    ! 6.17
721 fstatno "IDENTIFY" fd
722 [ "EXTERNAL" fd | "GLOBAL" fd ]
723 "EVENT INDICATOR" event-indicator>
724 "AS SET BY" <event
725 statement-terminator ;
726
727
728 identify-time-based-event-statement: d:                    ! 6.18
729 fstatno "IDENTIFY" fd
730 [ "EXTERNAL" fd | "GLOBAL" fd ] "EVENT" event>
731 "AS TIMER" <timer "EQ" time-quantity [ real-errlim ]
732 statement-terminator ;
733
734
735 define-digital-configuration-structure: d:                  ! 6.19.1
736 define-digital-configuration-statement
737 configuration-body
738 end-digital-configuration-statement ;
739
740
741 configuration-body: e:                                     ! 6.19.1-1
742 {
743     define-digital-source-statement |
744     define-digital-sensor-statement
745 } %
746 specify-delay-min-statement * ;
747
748
749 define-digital-configuration-statement: d:                  ! 6.19.2
750 fstatno "DEFINE" fd configuration> fd
751 "DIGITAL CONFIGURATION"
752 statement-terminator ;
753
754
755 define-digital-source-statement: d:                         ! 6.19.3
756 fstatno "DEFINE " fd digital-source> fd
757 "DIGITAL SOURCE" <<~>>
758 [
759     fd "SPEC"
760     {
761         {
762             "" external-digital-specification ""
763         } # fd
764     }
765 ]
766 digital-source-definition
767 statement-terminator ;
768
769
770 digital-source-definition: d:                                ! 6.19.3A
771 <<*>> [ fd noun-field ]
772 <<*>> [ statement-characteristics fd ]
773 source-digital-characteristics
774 "CNX" conn-set ;
775 &* This by-pass is used when the relevant information is
776 & completely defined in the specification identified by
777 & the <external digital specification>. *
778

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

779
780 source-digital-characteristics: d: ! 6.19.3B
781 <<*>> [ source-logic-levels fd ]
782 [ "ILLEGAL STATE INDICATOR" boolean-variable fd ] <<~>>
783 {
784   [ digital-format fd ]
785 } ;
786 &* This by-pass is used when the relevant information is
787 & completely defined in the specification identified by
788 & the <external digital specification>. *
789
790
791 source-logic-levels: d: ! 6.19.3C
792 {
793   {
794     def-dig-source-levels |
795     def-dig-sequence
796   } # fd
797 } ;
798
799
800 digital-format: d: ! 6.19.3D
801 {
802   <<*>> [ serial-format ]
803   <<*>> [ pulse-class fd ]
804   <<*>> [ digital-synchronization ] |
805   {
806     <<*>> [ <<*>> "SKEW-TIME MAX" time-quantity ]
807   }
808 } ;
809 &* This bypass is used only if all the information has
810 & been defined in the <external digital specification>
811 & and the signals are parallel. *
812 &** This path is used only with parallel digital signals. *
813
814
815 serial-format: d: ! 6.19.3E
816 {
817   "SERIAL-MSB-FIRST" |
818   "SERIAL-LSB-FIRST"
819 }
820 fd "WORD-LENGTH"
821 unsigned-integer-number "BITS" fd ;
822
823
824 logic-alternation: d: ! 6.19.3F
825 "EXCEPT AT EVERY" unsigned-integer-number
826 "CONSECUTIVE OCCURRENCES" ;
827
828
829 pulse-class: d: ! 6.19.3G
830 "PULSE-CLASS"
831 {
832   "SPEC" "" external-pulse-class-specification "" |
833   "HDB" unsigned-integer-number |
834   "RZ" |
835   "NRZ" |
836   "BIP" |
837   "AMI"
838 } ;
839
840
841 digital-synchronization: d: ! 6.19.3H
842 {
843   { "BIT-RATE" | "BIT-PERIOD" }

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

844     real-quantity
845     [ bit-rate-range ] |
846     "SYNC-BIT-TRANSITION TO EVENT"
847     <event max-time
848 } ;
849
850
851 bit-rate-range: d:                                ! 6.19.3I
852 "RANGE" unsigned-decimal-number dim
853 "TO" unsigned-decimal-number dim ;
854
855
856 def-dig-source-levels: d:                            ! 6.19.3J
857 {
858     { "LEVEL-LOGIC-ZERO" | "LEVEL-LOGIC-ONE" }
859     digital-source-characteristics
860     [ logic-alternation ] |
861     { "LEVEL-LOGIC-QUIES" | "LEVEL-LOGIC-HIZ" }
862     digital-source-characteristics fd
863 } ;
864
865
866 def-dig-sequence: d:                                ! 6.19.3K
867 {
868     { "TRANS-LOGIC-ZERO" | "TRANS-LOGIC-ONE" }
869     {
870         {
871             { "0" | "1" }
872             { "0" | "1" }
873         } # "OR"
874     } |
875     "TRANS-SYNC"
876     { "QUIES" | "0" | "1" }
877     { "0" | "1" } |
878     "TRANS-SYNC-BIT"
879 } ;
880
881
882 define-digital-sensor-statement: d:                  ! 6.19.4
883 fstatno "DEFINE" fd digital-sensor> fd
884 "DIGITAL SENSOR" <<~>>
885 [
886     fd "SPEC"
887     {
888         {
889             "" external-digital-specification ""
890         } # fd
891     }
892 ]
893 digital-sensor-definition
894 statement-terminator ;
895
896
897 digital-sensor-definition: d:                        ! 6.19.4A
898 <<*>> [ fd "(" measured-characteristic-mnemonic ")" ]
899 <<*>> [ fd noun-field ]
900 <<*>> [ statement-characteristics ]
901 sensor-digital-characteristics fd "CNX" conn-set ;
902 &* This by-pass is used when the relevant information is
903 & completely defined in the specification identified by
904 & the <external digital specification>. *
905
906
907 sensor-digital-characteristics: d:                    ! 6.19.4B
908 <<*>> [ fd sensor-logic-levels ]

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

909      [ fd "ILLEGAL STATE INDICATOR" boolean-variable ] <<~>>
910      {
911      <<*~>> [ fd digital-format ]
912      } ;
913      &*      This by-pass is used when the relevant information is
914      &      completely defined in the specification identified by
915      &      the <external digital specification>. *
916
917
918      sensor-logic-levels: d:                                ! 6.19.4C
919      {
920      {
921          def-dig-sensor-levels |
922          def-dig-sequence
923      } # fd
924      } ;
925
926
927      def-dig-sensor-levels: d:                                ! 6.19.4D
928      {
929      {
930          "LEVEL-LOGIC-ZERO" |
931          "LEVEL-LOGIC-ONE"
932      }
933      digital-sensor-characteristics
934      [ logic-alternation ] |
935      {
936          "LEVEL-LOGIC-QUIES" |
937          "LEVEL-LOGIC-HIZ"
938      }
939      digital-sensor-characteristics fd
940      } ;
941
942
943      specify-delay-min-statement: d:                          ! 6.19.5
944      fstatno "SPECIFY" fd "DELAY-MIN"
945      unsigned-decimal-number time-dim "FROM" <<~>>
946      {
947          <digital-source "TO" <digital-sensor |
948          <digital-sensor "TO" <digital-source
949      }
950      statement-terminator ;
951
952
953      end-digital-configuration-statement: d:                  ! 6.19.6
954      fstatno "END" fd <configuration
955      statement-terminator ;
956
957
958      define-exchange-statement: d:                            ! 6.20
959      fstatno "DEFINE" fd exchange> fd
960      [ "EXTERNAL" fd | "GLOBAL" fd ]
961      "EXCHANGE" fd "PROTOCOL" <protocol fd
962      exchange-definition
963      statement-terminator ;
964
965
966      exchange-definition: d:                                   ! 6.20A
967      "BUS-MODE"
968      {
969          <<*~>> <bus-mode-name |
970          "TALKER-LISTENER" |
971          "ALL-LISTENER" |
972          "AUTONOMOUS" |
973          "CON-MODE" |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

974         "CON-RT" |
975         "RT-CON" |
976         "RT-RT"
977     }
978     exchange-devices
979     define-exchange-fields ;
980     &*      The <bus-mode-name> must be one associated with the
981     &      particular <protocol> by its ESTABLISH statement. *
982
983
984     exchange-devices: d:                                ! 6.20B
985     [ fd "TALKER" databus-device ]
986     [
987         fd "LISTENER"
988         { databus-device # sfd }
989     ] ;
990
991
992     databus-device: d:                                    ! 6.20C
993     {
994         <<*>> <device-identifier-name |
995         "TEST-EQUIP" |
996         "UUT"
997     }
998     [
999         "("
1000         { data-store | constant }
1001         ")"
1002     ] ;
1003     &*      The device identifier name must have been established
1004     &      as a valid TALKER or LISTENER in the <establish statement>
1005     &      as appropriate for the <protocol>. *
1006
1007
1008     define-exchange-fields: d:                            ! 6.20D
1009     [ fd "COMMAND" define-databus-data ]
1010     [ fd "DATA" define-databus-data ] <<~>>
1011     [ fd "STATUS" define-databus-data ]
1012     [ fd bus-parameter ]
1013     [ fd set-protocol-parameter ] ;
1014
1015
1016     define-databus-data: d:                               ! 6.20E
1017     { [ "ENTIRE" ] array-range | data-store | constant } ;
1018
1019
1020     extend-atlas-statement: d:                             ! 6.21
1021     fstatno "EXTEND" fd "ATLAS"
1022     {
1023         noun-extension |
1024         connection-extension |
1025         modifier-extension |
1026         databus-extension
1027     }
1028     statement-terminator ;
1029
1030
1031     noun-extension: d:                                     ! 6.21A
1032     "NOUNS ' ' noun-name> ' ' fd "FOR VERBS"
1033     verb-info % fd
1034     "SPEC" external-semantics-specification ;
1035
1036
1037     connection-extension: d:                             ! 6.21B
1038     "CONNECTIONS FOR" noun fd extend-cn timer

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1039      [ fd "SPEC" external-semantics-specification ] ;
1040
1041
1042      modifier-extension: d:                                ! 6.21C
1043      "MODIFIERS FOR" noun fd
1044      {
1045          {
1046              modifier-definition fd
1047              "USAGE" usage-info
1048          } # fd
1049      } <<~>>
1050      [ fd "SPEC" external-semantics-specification ] ;
1051
1052
1053      extend-cnx: d:                                         ! 6.21D
1054      "CNX-NUMERATOR" ext-pin
1055      "CNX-DENOMINATOR" ext-pin |
1056      "CNX" ext-pin ;
1057
1058
1059      modifier-definition: d:                                ! 6.21E
1060      {
1061          "" modifier-name> ""
1062          {
1063              modifier-typing |
1064              "MODIFIER-DESCRIPTOR"
1065              extend-modifier-descriptor |
1066              "SPECIAL-SYNTAX" external-syntax-specification |
1067              "MNEMONIC-ONLY"
1068          } |
1069          modifier
1070      } ;
1071
1072
1073      modifier-typing: d:                                     ! 6.21F
1074      { "MOD-ARRAY" | "MOD-VALUE" }
1075      mod-type
1076      [ dimension-extension ] ;
1077
1078
1079      usage-info: d:                                         ! 6.21G
1080      { "SRM" | "RM" | "SR" | "S" | "R" } ;
1081
1082
1083      ext-pin: d:                                             ! 6.21H
1084      pin-name %
1085      [ bsfd "REF" pin-name % ]
1086      bsfd ;
1087
1088
1089      pin-name: d:                                           ! 6.21I
1090      { "" pin-descriptor-name> "" | pin-descriptor } ;
1091
1092
1093      dimension-extension: d:                                ! 6.21J
1094      "MOD-DIM"
1095      { new-dimension
1096          [ dim % ] |
1097          dim % |
1098          "NONE"
1099      } ;
1100
1101
1102      new-dimension: d:                                       ! 6.21K
1103      { sfd "" dim-name> "" } %

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1104     [ "DIM-SPEC" external-semantics-specification ] ;
1105
1106
1107     verb-info: d:                                     ! 6.21L
1108     {
1109         "DEFINE" | "IDENTIFY" |
1110         "REQUIRE" | "APPLY" |
1111         "REMOVE" | "MEASURE" |
1112         "MONITOR" | "VERIFY" |
1113         "READ" | "INITIATE" |
1114         "SETUP" | "CONNECT" |
1115         "DISCONNECT" |
1116         "ARM" | "FETCH" |
1117         "CHANGE" | "RESET" |
1118         "WAIT FOR" |
1119         "STIMULATE" |
1120         "SENSE" | "PROVE" |
1121         "DO"
1122     } ;
1123
1124
1125     extend-modifier-descriptor: d:                     ! 6.21M
1126     {
1127         new-modifier-descriptor
1128         [ modifier-descriptor % ] |
1129         modifier-descriptor % } ;
1130
1131
1132     new-modifier-descriptor: d:                         ! 6.21N
1133     { sfd "'" modifier-descriptor-name> "'" } %
1134     [ "MOD-DESC-SPEC" external-semantics-specification ] ;
1135
1136
1137     databus-extension: d:                               ! 6.21O
1138     {
1139         "BUS-PARAMETER"
1140         {
1141             {
1142                 "'" bus-parameter-name> "'"
1143                 extend-parameter-definition
1144             } # fd
1145         } |
1146         "PROTOCOL-PARAMETER"
1147         {
1148             {
1149                 "'" protocol-parameter-name> "'"
1150                 extend-parameter-definition
1151             } # fd
1152         } |
1153         "BUS-MODE"
1154         {
1155             {
1156                 "'" bus-mode-name> "'"
1157             } # fd
1158         } |
1159         "TEST-EQUIP-ROLE"
1160         {
1161             {
1162                 "'" test-equip-role-name> "'"
1163             } # fd
1164         } |
1165         "EXCHANGE-PIN"
1166         {
1167             {
1168                 "'" pin-descriptor-name> "'"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1169         } # fd
1170     }
1171 } ;
1172
1173
1174 extend-parameter-definition: d: ! 6.21P
1175 {
1176     "MOD-VALUE" mod-type [ dimension-extension ] |
1177     "MODIFIER-DESCRIPTOR" extend-modifier-descriptor |
1178     "MNEMONIC-ONLY"
1179 } ;
1180
1181
1182 mod-type: s: ! 6.21-1
1183 {
1184     "R" | "I" | "SC" | "SB" | "RA" | "IA" |
1185     "AC" | "AB" | "MD" | "MO" | "SS"
1186 } ;
1187
1188
1189 define-exchange-configuration-statement: d: ! 6.22
1190 fstatno "DEFINE" fd exchange-configuration> fd
1191 [ "EXTERNAL" fd | "GLOBAL" fd ]
1192 "EXCHANGE CONFIGURATION"
1193 { <protocol # fd >
1194 statement-terminator ;
1195
1196
1197 ! 18.6.4 MAIN PROCEDURAL STRUCTURE
1198
1199
1200 main-procedural-structure: d: ! 7.1
1201 {
1202     main-procedural-statements |
1203     <<*>> block-structure
1204 } % ;
1205 &* This path is for <atlas program structure>,
1206 & <procedure body> and <function body> only. *
1207
1208
1209 main-procedural-statements: d: ! 7.2
1210 {
1211     procedural-statements-data-processing |
1212     procedural-statements-input-output |
1213     procedural-statements-control |
1214     procedural-statements-signal |
1215     procedural-statements-timing |
1216     procedural-statements-databus |
1217     procedural-statements-leave |
1218     procedural-statements-resume |
1219     flag-statement | comment-statement
1220 } ;
1221
1222
1223 procedural-statements-leave: d: ! 7.2A
1224 {
1225     leave-block-statement |
1226     leave-procedure-statement |
1227     leave-function-statement |
1228     leave-if-statement |
1229     leave-while-statement |
1230     leave-for-statement
1231 } ;
1232
1233

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1234 procedural-statements-resume: d: ! 7.2B
1235 {
1236     resume-while-statement |
1237     resume-for-statement
1238 } ;
1239
1240 ! the correspondence between a leave-statement or a
1241 ! resume-statement (e.g. leave-block-statement) and the
1242 ! appropriate enclosing structure (e.g. block-structure)
1243 ! is checked semantically.
1244
1245
1246 ! 18.6.5 PROCEDURAL STATEMENTS - DATA PROCESSING
1247
1248
1249 procedural-statements-data-processing: d: ! 8.1
1250 {
1251     calculate-statement |
1252     compare-statement
1253 } ;
1254
1255
1256 calculate-statement: d: ! 8.2
1257 fstatno "CALCULATE" fd
1258 {
1259     {
1260         data-store "=" expression |
1261         calculate-condition
1262     } # fd
1263 }
1264 statement-terminator ;
1265
1266
1267 calculate-condition: d: ! 8.2A
1268 {
1269     "MANUAL-INTERVENTION" |
1270     "MAX-TIME" |
1271     "GO" |
1272     "HI" |
1273     "LO" |
1274     "NOGO"
1275 }
1276 "="
1277 { "FALSE" | "TRUE" } ;
1278
1279
1280 expression: d: ! 8.2B
1281 {
1282     {
1283         {
1284             {
1285                 expression-item
1286             }
1287             #
1288             {
1289                 sfd boolean-operator bsfd
1290             }
1291         }
1292     }
1293     #
1294     {
1295         sfd comparison-operator bsfd
1296     }
1297 } ;
1298

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1299
1300 expression-item: d: ! 8.2C
1301 [ "NOT" | "+" | "-" ]
1302 {
1303     expression-term #
1304     {
1305         sfd
1306         {
1307             "*" | "*" | "/" | "DIV" |
1308             "MOD" | "&" | "+" | "-"
1309         }
1310         bsfd
1311     }
1312 } ;
1313
1314
1315 expression-term: d: ! 8.2D
1316 {
1317     function-expression |
1318     "(" expression ")" |
1319     data-store |
1320     constant |
1321     condition |
1322     boolean-evaluation
1323 } ;
1324
1325
1326 constant: d: ! 8.2E
1327 {
1328     "CONNECTION" connection |
1329     decimal-number |
1330     digital-number |
1331     character-string |
1332     <enumeration-element |
1333     <constant-identifier |
1334     predeclared-enumeration
1335 } ;
1336
1337
1338 function-expression: d: ! 8.2F
1339 { predefined-function | <function }
1340 [
1341     "("
1342     { expression # fd }
1343     ")"
1344 ] ;
1345
1346
1347 predeclared-enumeration: d: ! 8.2G
1348 {
1349     "FALSE" |
1350     "TRUE" |
1351     "BNR" |
1352     "B1C" |
1353     "B2C" |
1354     "BSM" |
1355     "BCD" |
1356     "SBCD" |
1357     "ASCII7"
1358 } ;
1359
1360
1361 boolean-evaluation: d: ! 8.2H
1362 data-store
1363 [ "NOM" real-quantity ]

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1364      "UL" real-quantity "LL" real-quantity ;
1365
1366
1367      predefined-function: s:                                ! 8.2-1
1368      {
1369          "EOF" | "ODD" | "PE" | "PO" |
1370          "ORD" | "LEN" | "LOCN" | "COUNT" |
1371          "SIZE" | "SUCC" | "PRED" | "DATE" |
1372          "CHAR" | "BITS" | "COPY" | "ROTATE" |
1373          "SHIFT" | "A-SHIFT" | "DIG" | "DEC" |
1374          "INT" | "ROUND" | "ABS" | "SIN" |
1375          "COS" | "TAN" | "LN" | "LOG" |
1376          "ALOG" | "ATAN" | "EXP" | "SQRT" |
1377          "DELETE" | "INSERT"
1378      } ;
1379
1380
1381      boolean-operator: e:                                    ! 8.2-2
1382      { "XOR" | "OR" | "AND" } ;
1383
1384
1385      comparison-operator: e:                                ! 8.2-3
1386      {
1387          "EQ" | "NE" | "GT" | "LT" | "GE" | "LE"
1388      } ;
1389
1390
1391      compare-statement: d:                                    ! 8.3
1392      fstatno "COMPARE" fd data-store
1393      fd evaluation-field
1394      statement-terminator ;
1395
1396
1397      ! 18.6.6 PROCEDURAL STATEMENTS - INPUT / OUTPUT
1398
1399
1400      procedural-statements-input-output: d:                  ! 9.1
1401      {
1402          input-statement |
1403          output-statement |
1404          enable-file-access-statement |
1405          disable-file-access-statement |
1406          create-file-statement |
1407          delete-file-statement
1408      } ;
1409
1410
1411      input-statement: d:                                       ! 9.2
1412      fstatno "INPUT" fd
1413      { data-input | text-input | "GO-NOGO" }
1414      [ fd max-time ]
1415      statement-terminator ;
1416
1417
1418      data-input: d:                                             ! 9.2A
1419      "FROM" file
1420      [ sfd expression ] fd <<~>>
1421      "INTO"
1422      {
1423          {
1424              data-store |
1425              array-range
1426          } # fd
1427      } ;
1428

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1429
1430 text-input: d: ! 9.2B
1431 "TEXT"
1432 [ "FROM" file ] fd <<~>>
1433 "INTO"
1434 {
1435     {
1436         { data-store | array-range }
1437         [
1438             "HEXADECIMAL" |
1439             "BINARY" |
1440             "OCTAL"
1441         ]
1442     } # fd
1443 } ;
1444
1445
1446 output-statement: d: ! 9.3
1447 fstatno "OUTPUT" fd
1448 { text-output | data-output }
1449 statement-terminator ;
1450
1451
1452 data-output: d: ! 9.3A
1453 "TO" file
1454 [ sfd expression ] fd <<~>>
1455 "FROM"
1456 {
1457     { expression |
1458         array-range
1459     } # fd
1460 } ;
1461
1462
1463 text-output: d: ! 9.3B
1464 "TEXT"
1465 [ "TO" file ] fd <<~>>
1466 "FROM"
1467 {
1468     {
1469         { expression | array-range }
1470         [ output-format ]
1471     } # fd
1472 } ;
1473
1474
1475 output-format: d: ! 9.3C
1476 ":"
1477 [
1478     "HEXADECIMAL" |
1479     "E-FORMAT" |
1480     "BINARY" |
1481     "OCTAL"
1482 ]
1483 expression
1484 [ ":" expression ] ;
1485
1486
1487 enable-file-access-statement: d: ! 9.4
1488 fstatno "ENABLE" fd
1489 {
1490     { "OUTPUT TO" | "I-O" }
1491     { "OLD" | "NEW" } |
1492     "INPUT FROM"
1493 }

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1494     expression fd "VIA" file
1495     statement-terminator ;
1496
1497
1498     file: d:                                     ! 9.4A
1499     variable-identifier ;
1500
1501
1502     disable-file-access-statement: d:             ! 9.5
1503     fstatno "DISABLE" fd
1504     file statement-terminator ;
1505
1506
1507     create-file-statement: d:                     ! 9.6
1508     fstatno "CREATE" fd expression
1509     [ fd "STATUS" data-store ]
1510     statement-terminator ;
1511
1512
1513     delete-file-statement: d:                     ! 9.7
1514     fstatno "DELETE" fd expression
1515     [ fd "STATUS" data-store ]
1516     statement-terminator ;
1517
1518
1519     ! 18.6.7  PROCEDURAL STATEMENTS - CONTROL
1520
1521
1522     procedural-statements-control: d:             ! 10.1
1523     {
1524         if-then-else-else-if-structure |
1525         while-then-structure |
1526         for-then-structure |
1527         go-to-statement |
1528         perform-statement |
1529         finish-statement |
1530         enable-disable-manual-intervention-structure |
1531         wait-for-manual-intervention-statement |
1532         enable-digital-configuration-statement |
1533         enable-escape-statement |
1534         disable-escape-statement
1535     } ;
1536
1537
1538     if-then-else-else-if-structure: d:            ! 10.2.1
1539     if-then-statement
1540     procedural-segment-if-then-else-else-if
1541     {
1542         else-if-then-statement
1543         procedural-segment-if-then-else-else-if
1544     } *
1545     [
1546         else-statement
1547         procedural-segment-if-then-else-else-if
1548     ]
1549     end-if-statement ;
1550
1551
1552     if-then-statement: d:                         ! 10.2.2
1553     fstatno "IF" fd expression fd
1554     "THEN"
1555     statement-terminator ;
1556
1557
1558     procedural-segment-if-then-else-else-if: d:    ! 10.2.3

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1559      main-procedural-structure ;
1560
1561
1562      leave-if-statement: d:                                ! 10.2.4
1563      fstatno "LEAVE" fd "IF"
1564      [ "STEP" statement-number ]
1565      statement-terminator ;
1566
1567
1568      else-statement: d:                                     ! 10.2.5
1569      fstatno "ELSE"
1570      statement-terminator ;
1571
1572
1573      end-if-statement: d:                                   ! 10.2.6
1574      fstatno "END" fd "I "
1575      [ fd message-text ]
1576      statement-terminator ;
1577
1578
1579      else-if-then-statement: d:                             ! 10.2.7
1580      fstatno "ELSE IF" fd expression fd "THEN"
1581      statement-terminator ;
1582
1583
1584      while-then-structure: d:                                ! 10.3.1
1585      while-then-statement
1586      procedural-segment-while-then
1587      end-while-statement ;
1588
1589
1590      while-then-statement: d:                                ! 10.3.2
1591      fstatno "WHILE"
1592      fd expression fd "THEN"
1593      statement-terminator ;
1594
1595
1596      procedural-segment-while-then: d:                       ! 10.3.3
1597      main-procedural-structure ;
1598
1599
1600      leave-while-statement: d:                               ! 10.3.4
1601      fstatno "LEAVE" fd "WHILE"
1602      [ "STEP" statement-number ]
1603      statement-terminator ;
1604
1605
1606      resume-while-statement: d:                              ! 10.3.5
1607      fstatno "RESUME" fd "WHILE"
1608      [ "STEP" statement-number ]
1609      statement-terminator ;
1610
1611
1612      end-while-statement: d:                                 ! 10.3.6
1613      fstatno "END" fd "WHILE"
1614      [ fd message-text ]
1615      statement-terminator ;
1616
1617
1618      for-then-structure: d:                                   ! 10.4.1
1619      for-then-statement
1620      procedural-segment-for-then
1621      end-for-statement ;
1622
1623

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1624   for-then-statement: d:                                ! 10.4.2
1625   fstatno "FOR" fd data-store
1626   "=" for-then-range fd "THEN"
1627   statement-terminator ;
1628
1629
1630   for-then-range: d:                                      ! 10.4.2A
1631   {
1632       expression "THRU" expression
1633       [ "BY" expression ] |
1634       { expression # fd }
1635   } ;
1636
1637
1638   procedural-segment-for-then: d:                          ! 10.4.3
1639   main-procedural-structure ;
1640
1641
1642   leave-for-statement: d:                                  ! 10.4.4
1643   fstatno "LEAVE" fd "FOR"
1644   [ "STEP" statement-number ]
1645   statement-terminator ;
1646
1647
1648   resume-for-statement: d:                                  ! 10.4.5
1649   fstatno "RESUME" fd "FOR"
1650   [ "STEP" statement-number ]
1651   statement-terminator ;
1652
1653
1654   end-for-statement: d:                                     ! 10.4.6
1655   fstatno "END" fd "FOR"
1656   [ fd message-text ]
1657   statement-terminator ;
1658
1659
1660   go-to-statement: d:                                       ! 10.5
1661   fstatno "GO TO"
1662   fd "STEP" statement-number
1663   statement-terminator ;
1664
1665
1666   perform-statement: d:                                      ! 10.6
1667   fstatno "PERFORM" fd <procedure
1668   [ perform-variables ]
1669   statement-terminator ;
1670
1671
1672   perform-variables: d:                                      ! 10.6A
1673   [ fd perform-table ]
1674   actual-parameters ;
1675
1676
1677   perform-table: d:                                          ! 10.6B
1678   "TABLE" unsigned-integer-number "VAR"
1679   unsigned-integer-number "TIMES" ;
1680
1681
1682   actual-parameters: d:                                      ! 10.6C
1683   {
1684       {
1685           [
1686               "("
1687               { expression # fd }
1688               ")"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1689         ]
1690         [
1691             "RESULT ("
1692             { data-store # fd }
1693             ")"
1694         ]
1695     } # fd
1696 } ;
1697
1698
1699 finish-statement: d:                                ! 10.7
1700 fstatno "FINISH" statement-terminator ;
1701
1702
1703 enable-disable-manual-intervention-structure: d:    ! 10.8.1
1704 {
1705     enable-manual-intervention-statement |
1706     disable-manual-intervention-statement
1707 } ;
1708
1709
1710 enable-manual-intervention-statement: d:            ! 10.8.2
1711 fstatno "ENABLE" fd "MANUAL INTERVENTION"
1712 statement-terminator ;
1713
1714
1715 disable-manual-intervention-statement: d:          ! 10.8.3
1716 fstatno "DISABLE" fd "MANUAL INTERVENTION"
1717 statement-terminator ;
1718
1719
1720 wait-for-manual-intervention-statement: d:         ! 10.9
1721 fstatno "WAIT FOR" fd "MANUAL INTERVENTION"
1722 statement-terminator ;
1723
1724
1725 enable-digital-configuration-statement: d:         ! 10.10
1726 fstatno "ENABLE" fd
1727 "DIGITAL CONFIGURATION" <configuration <<~>>
1728 [
1729     "("
1730     {
1731         { decimal-number |
1732           data-store |
1733           "="
1734         } # fd
1735     } ")"
1736 ]
1737 statement-terminator ;
1738
1739
1740 enable-escape-statement: d:                        ! 10.11.1
1741 fstatno "ENABLE" fd "ESCAPE"
1742 "TO PROCEDURE" <procedure
1743 "IF" escape-indicator
1744 statement-terminator ;
1745
1746
1747 escape-indicator: d:                              ! 10.11.1A
1748 {
1749     <event-indicator |
1750     "MAX-TIME" |
1751     "MANUAL-INTERVENTION"
1752 } ;
1753

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1754
1755 disable-escape-statement: d: ! 10.11.2
1756 fstatno "DISABLE" fd "ESCAPE"
1757 [ escape-indicator ]
1758 statement-terminator ;
1759
1760
1761 ! 18.6.8 PROCEDURAL STATEMENTS - SIGNAL
1762
1763
1764 procedural-statements-signal: d: ! 11.1
1765 {
1766     single-action-verb-statements |
1767     apply-statement |
1768     remove-statement |
1769     measure-statement |
1770     monitor-statement |
1771     verify-statement |
1772     read-statement |
1773     initiate-statement |
1774     stimulate-statement |
1775     sense-statement |
1776     prove-statement
1777 } ;
1778
1779
1780 single-action-verb-statements: d: ! 11.2
1781 {
1782     setup-statement |
1783     connect-statement |
1784     disconnect-statement |
1785     arm-statement |
1786     fetch-statement |
1787     change-statement |
1788     enable-event-statement |
1789     disable-event-statement |
1790     reset-statement
1791 } ;
1792
1793
1794 setup-statement: d: ! 11.2.2
1795 fstatno "SETUP" fd
1796 {
1797     single-action-body |
1798     <<*>> single-action-event
1799 }
1800 statement-terminator ;
1801 &* This path is only required for signal based events which
1802 & are not directly referenced in SYNC, GATE, FROM, TO or BY
1803 & fields. It configures the EVENT MONITOR defined in the
1804 & REQUIRE to detect the <event> defined in the IDENTIFY
1805 & statement but does not ENABLE it. *
1806
1807
1808 gate-conn-or-event-field: d: ! 11.2.2A
1809 {
1810     <<***>> fd event-field |
1811     [ fd gate-field ]
1812     [ <<****>> fd max-time ]
1813     fd conn
1814 } ;
1815 &*** This path is only for time measurement statements. *
1816 &**** This path is only for measurement statements. *
1817
1818

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1819     single-action-body: d:                                ! 11.2.2B
1820     [
1821         <<*>> "("
1822         { measured-characteristic # fd }
1823         ")" fd
1824     ]
1825     noun-field statement-characteristics
1826     gate-conn-or-event-field ;
1827     &*& This path is for measurements only. *
1828
1829     ! The statement-characteristics subfield must be an empty set
1830     ! if the nouns "EARTH", "COMMON", "SHORT" are used,
1831     ! otherwise it must not.
1832
1833
1834     event-field: d:                                        ! 11.2.2C
1835     {
1836         "FROM" <event "TO" <event |
1837         "OF" <event-interval
1838     }
1839     [ fd gate-field ]
1840     fd max-time ;
1841
1842
1843     single-action-event: e:                                ! 11.2.2-1
1844     "EVENT"
1845     { event # fd } ;
1846
1847
1848     connect-statement: d:                                  ! 11.2.3
1849     fstatno "CONNECT" fd
1850     {
1851         single-action-body |
1852         <<*>> single-action-event
1853     }
1854     statement-terminator ;
1855     &*& This path is only required for signal based events which
1856     & are not directly referenced in SYNC, GATE, FROM, TO or BY
1857     & fields. It configures the EVENT MONITOR defined in the
1858     & REQUIRE to detect the <event> defined in the IDENTIFY
1859     & statement but does not ENABLE it. *
1860
1861
1862     disconnect-statement: d:                                ! 11.2.4
1863     fstatno "DISCONNECT" fd
1864     {
1865         single-action-body |
1866         <<*>> single-action-event
1867     }
1868     statement-terminator ;
1869     &*& This path is only required for signal based events which
1870     & are not directly referenced in SYNC, GATE, FROM, TO or BY
1871     & fields. It configures the EVENT MONITOR defined in the
1872     & REQUIRE to detect the <event> defined in the IDENTIFY
1873     & statement but does not ENABLE it. *
1874
1875
1876     arm-statement: d:                                       ! 11.2.5
1877     fstatno "ARM" fd sensor-action-body
1878     statement-terminator ;
1879
1880
1881     sensor-action-body: d:                                  ! 11.2.5A
1882     "("
1883     { measured-characteristic # fd }

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1884      ")" fd noun-field statement-characteristics
1885      gate-conn-or-event-field ;
1886
1887
1888      fetch-statement: d:                                ! 11.2.6
1889      fstatno "FETCH" fd
1890      sensor-action-body
1891      statement-terminator ;
1892
1893
1894      change-statement: d:                                ! 11.2.7
1895      fstatno "CHANGE" fd
1896      {
1897          single-action-body |
1898          <<*>> single-action-event
1899      }
1900      statement-terminator ;
1901      &*      This path is only required for signal based events which
1902      &      are not directly referenced in SYNC, GATE, FROM, TO or BY
1903      &      fields. It configures the EVENT MONITOR defined in the
1904      &      REQUIRE to detect the <event> defined in the IDENTIFY
1905      &      statement but does not ENABLE it. *
1906
1907
1908      enable-event-statement: d:                            ! 11.2.8
1909      fstatno "ENABLE" fd "EVENT"
1910      { <event # fd }
1911      statement-terminator ;
1912
1913
1914      disable-event-statement: d:                            ! 11.2.9
1915      fstatno "DISABLE" fd "EVENT"
1916      { <event # fd }
1917      statement-terminator ;
1918
1919
1920      reset-statement: d:                                    ! 11.2.10
1921      fstatno "RESET"
1922      {
1923          reset-signal |
1924          <<*>> single-action-event
1925      }
1926      statement-terminator ;
1927      &*      This path is only required for signal based events which
1928      &      are not directly referenced in SYNC, GATE, FROM, TO or BY
1929      &      fields. It configures the EVENT MONITOR defined in the
1930      &      REQUIRE to detect the <event> defined in the IDENTIFY
1931      &      statement but does not ENABLE it. *
1932
1933
1934      reset-signal: d:                                      ! 11.2.10A
1935      [
1936          <<*>> fd "("
1937          { measured-characteristic-mnemonic # fd }
1938          ")"
1939      ]
1940      fd noun-field statement-characteristics
1941      gate-conn-or-event-field ;
1942      &**      This path is for measurements only. *
1943
1944
1945      apply-statement: d:                                    ! 11.3
1946      fstatno "APPLY" fd
1947      {
1948          analog-signal |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

1949         <drawing [ fd conn ] |
1950         <<*>> single-action-event
1951     }
1952     statement-terminator ;
1953     &*      This path is only required for signal based events which
1954     &      are not directly referenced in SYNC, GATE, FROM, TO or BY
1955     &      fields. It configures the EVENT MONITOR defined in the
1956     &      REQUIRE to detect the <event> defined in the IDENTIFY
1957     &      statement but does not ENABLE it. *
1958
1959
1960     analog-signal: d:                                     ! 11.3A
1961     noun-field statement-characteristics
1962     [ fd gate-field ]
1963     fd conn ;
1964
1965
1966     remove-statement: d:                                   ! 11.4
1967     fstatno "REMOVE" fd <<~>>
1968     {
1969         noun-field
1970         <<*>> [ fd conn ] |
1971         <drawing [ fd conn ] |
1972         <<*>> remove-sensor |
1973         <<***>> single-action-event |
1974         "ALL"
1975     }
1976     statement-terminator ;
1977     &*      This path is for time measurement statements only. *
1978     &**     This path is for sensor statements only. *
1979     &***    This path is only required for signal based events which
1980     &      are not directly referenced in SYNC, GATE, FROM, TO or BY
1981     &      fields. It configures the EVENT MONITOR defined in the
1982     &      REQUIRE to detect the <event> defined in the IDENTIFY
1983     &      statement but does not ENABLE it. *
1984
1985
1986     remove-sensor: d:                                     ! 11.4A
1987     [
1988         "(" measured-characteristic-mnemonic ")" fd
1989     ]
1990     noun-field statement-characteristics
1991     <<*>> [ fd gate-conn-or-event-field ] ;
1992     &*      This path is for time measurement statements only. *
1993
1994     ! The statement-characteristics subfield must be an empty set
1995     ! if the nouns "EARTH", "COMMON", "SHORT" are used,
1996     ! otherwise it must not.
1997
1998
1999     measure-statement: d:                                   ! 11.5
2000     fstatno "MEASURE" fd "("
2001     { measured-characteristic # fd }
2002     ")" fd noun-field statement-characteristics
2003     gate-conn-or-event-field
2004     statement-terminator ;
2005
2006
2007     monitor-statement: d:                                   ! 11.6
2008     fstatno "MONITOR" fd
2009     "(" measured-characteristic ")"
2010     fd noun-field <<~>>
2011     {
2012         statement-characteristics gate-conn-or-event-field |
2013         fd <drawing

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2014     [ fd conn ]
2015     }
2016     statement-terminator ;
2017
2018
2019     verify-statement: d:                                ! 11.7
2020     fstatno "VERIFY" fd
2021     "(" measured-characteristic ")"
2022     fd noun-field eval-statement-characteristics
2023     gate-conn-or-event-field
2024     statement-terminator ;
2025
2026
2027     read-statement: d:                                    ! 11.8
2028     fstatno "READ" fd "("
2029     { measured-characteristic # fd }
2030     ")" fd noun-field statement-characteristics
2031     gate-conn-or-event-field
2032     statement-terminator ;
2033
2034
2035     initiate-statement: d:                                ! 11.9
2036     fstatno "INITIATE" fd sensor-action-body
2037     statement-terminator ;
2038
2039
2040     stimulate-statement: d:                                ! 11.10.1
2041     fstatno "STIMULATE" fd
2042     [ when-field fd ] <<~>>
2043     {
2044     {
2045         { "ZERO" | "ONE" | "HIZ" }
2046         fd on-field
2047     } # fd
2048     } |
2049     expression fd on-field
2050     }
2051     statement-terminator ;
2052
2053
2054
2055     sense-statement: d:                                    ! 11.10.2
2056     fstatno "SENSE" fd
2057     [ when-field fd ]
2058     sensed-data
2059     statement-terminator ;
2060
2061
2062     sensed-data: d:                                        ! 11.10.2A
2063     "("
2064     {
2065         <function "( VALUE"
2066         [
2067             fd
2068             { expression # fd }
2069         ]
2070         ")" |
2071         "VALUE" |
2072         "HIZ"
2073     }
2074     "INTO" data-store ")"
2075     fd on-field ;
2076
2077
2078     prove-statement: d:                                    ! 11.10.3

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2079     fstatno "PROVE" fd
2080     [ when-field fd ] <<~>>
2081     {
2082     {
2083         {
2084             { "ZERO" | "ONE" | "HIZ" }
2085             fd on-field
2086         } # fd
2087     } |
2088     prove-referenced-signal
2089     }
2090     statement-terminator ;
2091
2092
2093     prove-referenced-signal: d:                                ! 11.10.3A
2094     {
2095         prove-value "REF" expression <<~>>
2096         [
2097             fd
2098             { "MASK-ZERO" | "MASK-ONE" }
2099             expression
2100         ]
2101         [ fd "SAVE-COMP" data-store ]
2102     }
2103     fd on-field ;
2104
2105
2106     prove-value: d:                                            ! 11.10.3B
2107     {
2108         "(" <function "(" VALUE"
2109         [
2110             fd
2111             { expression # fd }
2112         ]
2113         ")" INTO" data-store ")" |
2114         "(" <function "(" VALUE"
2115         [
2116             fd
2117             { expression # fd }
2118         ]
2119         ")" )" |
2120         "(" VALUE INTO" data-store ")" |
2121         "(" HIZ INTO" data-store ")" |
2122         "(" VALUE )" |
2123         "(" HIZ )"
2124     } ;
2125
2126
2127     ! 18.6.9 PROCEDURAL STATEMENTS - TIMING
2128
2129
2130     procedural-statements-timing: d:                            ! 12.1
2131     {
2132         wait-for-statement |
2133         do-simultaneous-structure |
2134         do-timed-digital-structure |
2135         do-sequential-structure |
2136         reset-timer-statement |
2137         read-timer-statement
2138     } ;
2139
2140
2141     wait-for-statement: d:                                      ! 12.2
2142     fstatno "WAIT FOR" fd <<~>>
2143     {

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2144     wait-for-timer-or-event
2145     [ fd "THEN RESET" <timer> ] |
2146     "TIME" time-quantity
2147 } statement-terminator ;
2148
2149
2150 wait-for-timer-or-event: d:                                ! 12.2A
2151 {
2152     <<*>> [ "TIMER" ] <timer> sfd time-quantity
2153     [ real-errlim ] |
2154     <<*>> [ "EVENT" ] <event> sfd max-time
2155 } ;
2156 &* Non-preferred. *
2157
2158
2159 do-simultaneous-structure: d:                                ! 12.4.1
2160 do-simultaneous-statement
2161 do-simultaneous-body
2162 end-do-statement ;
2163
2164
2165 do-simultaneous-statement: d:                                ! 12.4.2
2166 fstatno "DO" fd "SIMULTANEOUS"
2167 [ fd when-field ] <<~>>
2168 [ fd "WITHIN" time-quantity ]
2169 [ fd max-time ]
2170 statement-terminator ;
2171
2172
2173 do-simultaneous-body: d:                                     ! 12.4.3
2174 main-procedural-structure ;
2175
2176
2177 end-do-statement: d:                                         ! 12.4.4
2178 fstatno "END" fd "DO"
2179 statement-terminator ;
2180
2181
2182 do-timed-digital-structure: d:                                ! 12.4.5
2183 do-timed-digital-statement
2184 do-timed-digital-body
2185 end-do-statement ;
2186
2187
2188 do-timed-digital-statement: d:                                ! 12.4.6
2189 fstatno "DO" fd "TIMED DIGITAL"
2190 [ fd when-field ] <<~>>
2191 {
2192     { fd stim-rate | fd stim-event }
2193     [
2194         { fd sense-delay | fd sense-event }
2195     ] |
2196     { fd sense-event | fd sense-rate }
2197 }
2198 statement-terminator ;
2199
2200
2201 do-timed-digital-body: d:                                     ! 12.4.7
2202 {
2203     stimulate-statement |
2204     sense-statement |
2205     prove-statement |
2206     procedural-statements-data-processing |
2207     do-timed-digital-structure |
2208     do-timed-control-structure |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

[illegible]

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2274      "( TIME INTO" data-store ")"
2275      fd "TIMER" <timer
2276      statement-terminator ;
2277
2278
2279      ! 18.6.10  PROCEDURAL STATEMENTS - DATABUS
2280
2281
2282      procedural-statements-databus: d:                                ! 13.0
2283      {
2284          do-exchange-statement |
2285          update-exchange-statement |
2286          enable-exchange-configuration-statement |
2287          connect-exchange-configuration-statement |
2288          disconnect-exchange-configuration-statement |
2289          disable-exchange-configuration-statement |
2290          fetch-exchange-configuration-statement
2291      } ;
2292
2293
2294      do-exchange-statement: d:                                        ! 13.1
2295      fstatno "DO" fd "EXCHANGE USING"
2296      <exchange-configuration <<~>>
2297      [ fd when-field ]
2298      [ fd role-field ] <<~>>
2299      exchange-set
2300      statement-terminator ;
2301
2302
2303      exchange-set: e:                                              ! 13.1-1
2304      {
2305          { exchange-expression | "START" }
2306          fd
2307          { "PROCEED" | "WAIT" } |
2308          "HOLD" fd exchange-expression
2309      }
2310      [ fd max-time ] ;
2311
2312
2313      update-exchange-statement: d:                                ! 13.2
2314      fstatno "UPDATE" fd
2315      { protocol-update | exchange-update }
2316      statement-terminator ;
2317
2318
2319      protocol-update: d:                                           ! 13.2A
2320      <protocol fd "PROTOCOL USING"
2321      <exchange-configuration
2322      [ when-field ] <<~>>
2323      {
2324          [ fd bus-parameter ]
2325          [ fd set-protocol-parameter ] |
2326          fd "STANDARD"
2327      } ;
2328
2329
2330      exchange-update: d:                                           ! 13.2B
2331      <exchange fd "EXCHANGE USING"
2332      <exchange-configuration
2333      [ when-field ]
2334      new-exchange-details ;
2335
2336
2337      new-exchange-details: d:                                       ! 13.2C
2338      [ fd "TALKER" databus-device ]

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2339     [
2340         fd "LISTENER"
2341         { databus-device # sfd }
2342     ]
2343     exchange-contents ;
2344
2345
2346     exchange-contents: d:                                ! 13.2D
2347     [ fd "COMMAND" databus-update-data ]
2348     [ fd "DATA" databus-update-data ] <<~>>
2349     [ fd "STATUS" databus-update-data ]
2350     [ fd bus-parameter ]
2351     [ fd set-protocol-parameter ] ;
2352
2353
2354     databus-update-data: d:                                ! 13.2E
2355     {
2356         [ "ENTIRE" ]
2357         { "FROM" file [ expression ] | array-range } |
2358         data-store | constant
2359     } ;
2360
2361
2362     enable-exchange-configuration-statement: d:            ! 13.3
2363     fstatno "ENABLE" fd "EXCHANGE CONFIGURATION"
2364     "USING" <exchange-configuration fd <<~>>
2365     {
2366         { "PROTOCOL" <protocol
2367           protocol-initialization } # fd
2368     }
2369     statement-terminator ;
2370
2371
2372     protocol-initialization: d:                            ! 13.3A
2373     [ fd "INITIALIZE" resource-initialization % ]
2374     [ fd bus-parameter ] <<~>>
2375     [ fd set-protocol-parameter ]
2376     [ fd "CNX" conn-set ] ;
2377
2378
2379     resource-initialization: d:                            ! 13.3B
2380     [ fd "TALKER" databus-device ]
2381     [ fd "LISTENER" databus-device ] <<~>>
2382     [ fd "COMMAND" database-enable-data ]
2383     [ fd "DATA" database-enable-data ] ;
2384
2385
2386     database-enable-data: d:                                ! 13.3C
2387     { array-range | data-store | constant } ;
2388
2389
2390     connect-exchange-configuration-statement: d:          ! 13.4
2391     fstatno "CONNECT" fd "EXCHANGE CONFIGURATION"
2392     "USING" <exchange-configuration fd <<~>>
2393     {
2394         {
2395             "PROTOCOL" <protocol fd
2396             "CNX" conn-set
2397         } # fd
2398     }
2399     statement-terminator ;
2400
2401
2402     disconnect-exchange-configuration-statement: d:        ! 13.5
2403     fstatno "DISCONNECT" fd "EXCHANGE CONFIGURATION"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2404 "USING" <exchange-configuration fd <<~>>
2405 {
2406     {
2407         "PROTOCOL" <protocol fd
2408         "CNX" conn-set
2409     } # fd
2410 }
2411 statement-terminator ;
2412
2413
2414 disable-exchange-configuration-statement: d: ! 13.6
2415 fstatno "DISABLE" fd "EXCHANGE CONFIGURATION"
2416 "USING" <exchange-configuration
2417 statement-terminator ;
2418
2419
2420 fetch-exchange-configuration-statement: d: ! 13.7
2421 fstatno "FETCH" fd
2422 { protocol-fetch | exchange-fetch }
2423 statement-terminator ;
2424
2425
2426 protocol-fetch: d: ! 13.7A
2427 <protocol fd "PROTOCOL" "USING"
2428 <exchange-configuration
2429 [ when-field ] <<~>>
2430 [ fd bus-parameter ]
2431 [ fd fetch-protocol-parameter ] ;
2432
2433
2434 exchange-fetch: d: ! 13.7B
2435 <exchange fd "EXCHANGE" "USING"
2436 <exchange-configuration
2437 [ when-field ]
2438 fetched-exchange-data
2439 fd max-time ;
2440
2441
2442 fetched-exchange-data: d: ! 13.7C
2443 [ fd "TALKER" databus-device ]
2444 [ fd "LISTENER"
2445     { databus-device # sfd }
2446 ]
2447 fetched-information ;
2448
2449
2450 fetched-information: d: ! 13.7D
2451 [ fd "COMMAND" databus-fetch-data ]
2452 [ fd "DATA" databus-fetch-data ] <<~>>
2453 [ fd "STATUS" databus-fetch-data ]
2454 [ fd bus-parameter ]
2455 [ fd fetch-protocol-parameter ] ;
2456
2457
2458 databus-fetch-data: d: ! 13.7E
2459 { to-count-into | variable-identifier
2460 { { "." record-field } # sfd } |
2461 data-store } ;
2462
2463
2464 to-count-into: d: ! 13.7F
2465 { "TO" file [ expression ] | array-range }
2466 [ fd "COUNT-INTO" data-store ] ;
2467
2468

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2469      ! 18.6.11 FIELDS / SUBFIELDS
2470
2471
2472      statement-characteristics: d:                                ! 14.1
2473      <<*>> [
2474          fd
2475          {
2476              {
2477                  {
2478                      harmonic-components |
2479                      characteristic-mnemonic
2480                      <<***>> [ characteristic-value ]
2481                  } |
2482                  sync-subfield
2483              } # fd
2484          }
2485      ] ;
2486      &** This path is used only for the nouns COMMON,
2487      &    EARTH and SHORT. *
2488      &*** This path is used only for modifiers of type MO
2489      &    as defined in Chapter 16. *
2490
2491
2492      harmonic-components: d:                                     ! 14.1A
2493      {
2494          <<*>> "HARM-" unsigned-integer-number "-VOLTAGE" |
2495          <<*>> "HARM-" unsigned-integer-number "-PHASE"
2496      }
2497      real-characteristic-subfield ;
2498      &* This path is used only where permitted by the noun
2499      &    modifier lists in Chapter 16. *
2500
2501
2502      characteristic-value: d:                                     ! 14.1B
2503      {
2504          real-characteristic-subfield |
2505          <<***>> modifier-descriptor |
2506          <<*>> expression |
2507          <<***>> array-range
2508      } ;
2509      &* This path is used only for modifiers of type SB
2510      &    or SC as defined in Chapter 16. *
2511      &** This path is used only for modifiers of type AB,
2512      &    AC or RA as defined in Chapter 16. *
2513      &*** This path is used only for modifiers of type MD
2514      &    as defined in Chapter 16. *
2515
2516
2517      real-characteristic-subfield: d:                             ! 14.2
2518      {
2519          "RANGE" decimal-number dim
2520          "TO" decimal-number dim |
2521          signal-value
2522          [ real-errlim ] |
2523          { "MAX" | "MIN" }
2524          decimal-number dim
2525      } ;
2526
2527
2528      sync-subfield: d:                                           ! 14.3
2529      {
2530          "SYNC"
2531          <<*>> [ sync-point ]
2532          "TO EVENT" <event
2533      } ;

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2534    &* This path is used with the noun WAVEFORM and with
2535    & other nouns when the sync subfield is used to
2536    & synchronize the ARM action of a sensor statement.
2537    & This path may be used only once in a statement. *
2538
2539
2540    sync-point: d:                                ! 14.3A
2541    {
2542        characteristic-mnemonic
2543        [
2544            evaluation-field sfd
2545            pos-neg-edge
2546        ] |
2547        sync-initial |
2548        "BIT-TRANSITION" |
2549        "WORD-TRANSITION"
2550    } ;
2551
2552
2553    sync-initial: d:                                ! 14.3B
2554    "INITIAL"
2555    {
2556        { "VOLTAGE O V" | "CURRENT O A" }
2557        [ real-errlim ]
2558        sfd pos-neg-edge |
2559        "VOLTAGE-ONE" |
2560        "CURRENT-ONE" |
2561        "VOLTAGE-ZERO" |
2562        "CURRENT-ZERO" |
2563        "VOLTAGE-QUIES" |
2564        "CURRENT-QUIES" |
2565        "VOLTAGE-P-POS" |
2566        "VOLTAGE-P-NEG" |
2567        "CURRENT-P-POS" |
2568        "CURRENT-P-NEG"
2569    } ;
2570
2571
2572    pos-neg-edge: e:                                ! 14.3-1
2573    { "POS-EDGE" | "NEG-EDGE" } ;
2574
2575
2576    pos-neg-slope: e:                                ! 14.3-2
2577    { "POS-SLOPE" | "NEG-SLOPE" } ;
2578
2579
2580    real-errlim: d:                                ! 14.4
2581    "ERRLMT"
2582    {
2583        dim-lim
2584        [
2585            { "AND" | "OR" }
2586            pc-lim
2587        ] |
2588        pc-lim
2589    } ;
2590
2591
2592    dim-lim: d:                                    ! 14.5
2593    {
2594        "+" unsigned-decimal-number dim
2595        "-" unsigned-decimal-number dim |
2596        "+ -" unsigned-decimal-number dim
2597    } ;
2598

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2599
2600 pc-lim: d: ! 14.6
2601 {
2602     "+" unsigned-decimal-number "PC"
2603     "-" unsigned-decimal-number "PC" |
2604     "+ -" unsigned-decimal-number "PC"
2605 } ;
2606
2607
2608 measured-characteristic: d: ! 14.7
2609 measured-characteristic-mnemonic
2610 [ real-complex-errlim ]
2611 [
2612     "INTO"
2613     { data-store | array-range }
2614 ] ;
2615
2616
2617 real-complex-errlim: e: ! 14.7-1
2618 real-errlim ;
2619
2620
2621 evaluation-field: d: ! 14.8
2622 {
2623     up-low-limit |
2624     { "EQ" | "NE" | "GE" | "GT" | "LE" | "LT" }
2625     { real-quantity | <*> expression }
2626 } ;
2627 &* This path applies only to EQ and NE. *
2628
2629
2630 up-low-limit: d: ! 14.8A
2631 [ "NOM" real-quantity sfd ]
2632 "UL" real-quantity
2633 "LL" real-quantity ;
2634
2635
2636 eval-statement-characteristics: d: ! 14.9
2637 fd evaluation-field
2638 statement-characteristics ;
2639
2640
2641 max-time: d: ! 14.10
2642 "MAX-TIME" time-quantity ;
2643
2644
2645 time-quantity: d: ! 14.11
2646 expression time-dim ;
2647
2648
2649 condition: d: ! 14.12
2650 {
2651     [ "NOT" ]
2652     {
2653         "MANUAL INTERVENTION" |
2654         "MAX-TIME"
2655     } |
2656     "HI" |
2657     "LO" |
2658     "GO" |
2659     "NOGO"
2660 } ;
2661
2662
2663 conn: d: ! 14.13

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2664 {
2665     <<*>> "CNX-NUMERATOR" connect-set-ref
2666     "CNX-DENOMINATOR" connect-set-ref |
2667     "CNX" connect-set-ref
2668 } ;
2669 &** This path is used only with the suffix RATIO. *
2670
2671 connect-set-ref: d: ! 14.13A
2672 conn-set
2673 [
2674     "REF"
2675     { reference-phase | conn-set }
2676 ] ;
2677
2678
2679 conn-set: d: ! 14.14
2680 {
2681     {
2682         <<*>>[ pin-descriptor ]
2683         sfd conn-descriptor
2684     } # { <<*>> bsfd }
2685 } ;
2686 &*** This bypass is allowed only for the nouns where
2687 &     a pin description is not meaningful, such as
2688 &     COMMON, EARTH and SHORT. *
2689
2690
2691 conn-descriptor: d: ! 14.14A
2692 {
2693     {
2694         {
2695             connection |
2696             data-store
2697         } # { <<*>> bsfd }
2698     } |
2699     <<*>> "( )"
2700 } ;
2701 &* This path is only to be used in a <define signal statement>
2702 & or a <define drawing statement>. When used, it indicates
2703 & that an actual connection is entered as a parameter in each
2704 & statement that references the <signal> or <drawing> name. *
2705 &*** There must be one or more blank spaces inserted
2706 & when this feedback path is followed. *
2707
2708
2709 signal-value: d: ! 14.15
2710 {
2711     { <<*>> "( )" | expression | array-range }
2712     signal-range |
2713     decimal-number dim
2714 } ;
2715 &* This path may only be used to specify the
2716 & requirement for a parameter within a
2717 & <define signal statement>. *
2718
2719
2720
2721 signal-range: d: ! 14.15A
2722 "RANGE" decimal-number dim
2723 "TO" decimal-number dim ;
2724
2725
2726 real-quantity: d: ! 14.16
2727 expression dim ;
2728

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2729
2730     index: d:                                ! 14.17
2731     expression ;
2732
2733
2734     noun-field: d:                            ! 14.18
2735     noun
2736     "USING" <requirement
2737     [ sfd when-field ] ;
2738
2739
2740     noun: s:                                  ! 14.18-1
2741     { analog-noun | <noun-name > } ;
2742
2743
2744     dimensioned-number: d:                    ! 14.19
2745     decimal-number dim ;
2746
2747
2748     require-control: d:                       ! 14.20
2749     fd "CONTROL" fd <<~>>
2750     {
2751     {
2752         { characteristic-mnemonic | digital-keyword }
2753         [
2754             require-control-body |
2755             { modifier-descriptor # sfd }
2756         ]
2757     } # fd
2758     } ;
2759
2760
2761     require-control-body: d:                   ! 14.20A
2762     {
2763         require-range
2764         { "BY" dimensioned-number |
2765           "CONTINUOUS"
2766         } |
2767         { dimensioned-number # sfd }
2768     } <<~>>
2769     [ real-errlim ] ;
2770
2771
2772     require-range: e:                         ! 14.20-1
2773     "RANGE" dimensioned-number
2774     "TO" dimensioned-number ;
2775
2776
2777     require-capability: d:                    ! 14.21
2778     fd "CAPABILITY" fd
2779     [ "GATE" fd ]
2780     [
2781         { sync-requirement # fd }
2782         fd
2783     ] <<~>>
2784     {
2785     {
2786         { characteristic-mnemonic | digital-keyword }
2787         [
2788             required-characteristic-capability
2789             [ real-errlim ] |
2790             modifier-descriptor
2791         ]
2792     } # fd
2793     } ;

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2794
2795
2796 sync-requirement: d: ! 14.21A
2797 "SYNC"
2798 [
2799     ranged-sync |
2800     initial-sync |
2801     "WORD-TRANSITION" |
2802     "BIT-TRANSITION"
2803 ] ;
2804
2805
2806 ranged-sync: d: ! 14.21B
2807 characteristic-mnemonic
2808 [
2809     required-range pos-neg-edge
2810 ] ;
2811
2812
2813 required-range: d: ! 14.21C
2814 "RANGE" dimensioned-number
2815 "TO" dimensioned-number
2816 [ "BY" dimensioned-number ] ;
2817
2818
2819 required-characteristic-capability: d: ! 14.21D
2820 {
2821     require-range
2822     [ "BY" dimensioned-number ] |
2823     { "MAX" | "MIN" }
2824     dimensioned-number
2825 } ;
2826
2827
2828 initial-sync: d: ! 14.21E
2829 "INITIAL" sfd
2830 {
2831     initial-slope |
2832     "VOLTAGE-P-POS" |
2833     "VOLTAGE-P-NEG" |
2834     "CURRENT-P-POS" |
2835     "CURRENT-P-NEG" |
2836     "VOLTAGE-QUIES" |
2837     "VOLTAGE-ZERO" |
2838     "VOLTAGE-ONE" |
2839     "CURRENT-QUIES" |
2840     "CURRENT-ZERO" |
2841     "CURRENT-ONE"
2842 } ;
2843
2844
2845 initial-slope: d: ! 14.21F
2846 { "VOLTAGE 0 V" | "CURRENT 0 A" }
2847 [ real-errlim ]
2848 pos-neg-edge ;
2849
2850
2851 require-limit: d: ! 14.22
2852 fd "LIMIT" fd
2853 { {
2854     { characteristic-mnemonic | digital-keyword }
2855     required-characteristic-limit
2856     [ real-errlim ] } # fd
2857 } ;
2858

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2859
2860 required-characteristic-limit: d: ! 14.22A
2861 { require-range |
2862   { "MAX" | "MIN" } dimensioned-number } ;
2863
2864
2865 require-cnx: d: ! 14.23
2866 fd
2867 {
2868   "CNX-NUMERATOR" req-pin
2869   "CNX-DENOMINATOR" req-pin |
2870   "CNX"
2871   <<*>> [ req-pin ]
2872 } ;
2873 &* This bypass is allowed only for nouns where a
2874 & pin descriptor is not meaningful such as
2875 & COMMON, EARTH & SHORT. *
2876
2877
2878 req-pin: d: ! 14.23A
2879 pin-descriptor %
2880 [ "REF" pin-descriptor % ] ;
2881
2882
2883 exchange-expression: d: ! 14.24
2884 {
2885   {
2886     "(" { { { exchange-frame } # "AND" } # "OR" } ")"
2887   } # "THEN"
2888 } ;
2889
2890
2891 exchange-frame: d: ! 14.25
2892 [
2893   "STARTING" when-field
2894   exchange-frequency fd
2895 ]
2896 {
2897   {
2898     "EXCHANGE" databus-transaction
2899   } # fd
2900 } ;
2901
2902
2903 exchange-frequency: d: ! 14.25A
2904 [
2905   fd "WITH PRIORITY" unsigned-integer-number
2906 ] <<~>>
2907 [ fd "EVERY"
2908   { "EVENT" <event | time-quantity }
2909 ]
2910 [ exchange-end ] ;
2911
2912
2913 exchange-end: d: ! 14.25B
2914 fd "UNTIL"
2915 {
2916   [ "TIMER" <timer "EQ" ]
2917   time-quantity |
2918   "EVENT" <event |
2919   "EXCHANGE" <exchange
2920 } ;
2921
2922
2923 databus-transaction: d: ! 14.25C

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2924 <exchange
2925 [ fd role-field ]
2926 [ fd exchange-simulate ] <<~>>
2927 [ fd exchange-monitor ]
2928 [ fd "DELAY" time-quantity ] ;
2929
2930
2931 exchange-simulate: d: ! 14.25D
2932 "TEST-EQUIP-SIMULATE"
2933 [ "TALKER" simulate-device ]
2934 [
2935     "LISTENER"
2936     { simulate-device # sfd
2937     }
2938 ] ;
2939
2940
2941 simulate-device: d: ! 14.25E
2942 { <<*>> device-identifier-name | "UUT" }
2943 [
2944     "("
2945     { data-store | constant }
2946     ")"
2947 ] ;
2948 &* The device must be one which has been established
2949 & as valid for the TEST-EQUIP to simulate in the
2950 & ESTABLISH statement for the protocol. *
2951
2952
2953 exchange-monitor: d: ! 14.25F
2954 "TEST-EQUIP-MONITOR"
2955 {
2956     fetch-protocol-parameter |
2957     bus-parameter |
2958     { fetch-databus-data # sfd }
2959 } ;
2960
2961
2962 fetch-databus-data: d: ! 14.25G
2963 { "COMMAND" | "STATUS" | "DATA" }
2964 {
2965     array-range
2966     [ fd "COUNT-INTO" data-store ] |
2967     data-store
2968 } ;
2969
2970
2971 gate-field: d: ! 14.26
2972 "GATED"
2973 {
2974     "FROM" <event "TO" <event |
2975     "BY" <event-interval
2976 } ;
2977
2978
2979 stim-rate: d: ! 14.27
2980 "STIM-RATE" stim-sense-rate ;
2981
2982
2983 stim-sense-rate: e: ! 14.27-1
2984 {
2985     {
2986         unsigned-decimal-number |
2987         data-store
2988     } dim

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

2989     [ sfd real-errlim ] |
2990     "MAX"
2991     {
2992         unsigned-decimal-number |
2993         data-store
2994     }
2995     dim
2996 } ;
2997
2998
2999 sense-rate: d:                                ! 14.28
3000 "SENSE-RATE" stim-sense-rate ;
3001
3002
3003 sense-delay: d:                                ! 14.29
3004 "SENSE-DELAY"
3005 {
3006     time-quantity
3007     [ real-errlim ] |
3008     "MIN" time-quantity
3009 } ;
3010
3011
3012 stim-event: d:                                ! 14.30
3013 "STIM-EVENT" <event
3014 [ sfd max-time ] ;
3015
3016
3017 sense-event: d:                                ! 14.31
3018 "SENSE-EVENT" <event
3019 [ sfd max-time ] ;
3020
3021
3022 when-field: d:                                ! 14.32
3023 "WHEN" when-body ;
3024
3025
3026 when-body: d:                                ! 14.32A
3027 {
3028     <<*>> [ "TIMER" ] <timer sfd time-quantity
3029     [ real-errlim ] |
3030     <<*>> [ "EVENT" ] <event sfd max-time
3031 } ;
3032 &*      Non-preferred *
3033
3034
3035 digital-source-characteristics: d:              ! 14.33
3036 {
3037     characteristic-mnemonic
3038     real-characteristic-subfield
3039 } # fd ;
3040
3041
3042 digital-sensor-characteristics: d:              ! 14.34
3043 {
3044     [ evaluation-field fd ]
3045     { { characteristic-mnemonic
3046         real-characteristic-subfield
3047     } # fd
3048     } |
3049     evaluation-field
3050 } ;
3051
3052
3053 inst-val: d:                                ! 14.35

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3054 characteristic-mnemonic
3055 {
3056     {
3057         real-characteristic-subfield
3058         fd time-subfield
3059     } # fd
3060 } ;
3061
3062
3063 time-subfield: d: ! 14.36
3064 {
3065     "AT TIMER" <timer sfd time-quantity
3066     [ real-errlim ] |
3067     time-quantity
3068     [ real-errlim ] "LATER" |
3069     "AT EVENT" <event sfd max-time
3070 } ;
3071
3072
3073 on-field: d: ! 14.37
3074 "ON"
3075 {
3076     <digital-source |
3077     <digital-sensor |
3078     <configuration
3079 }
3080 [
3081     [ "EXCEPT" ]
3082     "CNX" conn-set
3083 ] ;
3084
3085
3086 formal-parameter: d: ! 14.38
3087 { parameter> # fd }
3088 "IS" type ;
3089
3090
3091 data-store: d: ! 14.39
3092 <variable-identifier
3093 [
3094     {
3095         "("
3096         { index # fd }
3097         ")" |
3098         "." record-field
3099     } % ] ;
3100
3101
3102 array-range: d: ! 14.40
3103 data-store "("
3104 {
3105     { index # ";" } |
3106     index-range
3107 }
3108 ")" ;
3109
3110
3111 index-range: d: ! 14.40A
3112 index "THRU" index
3113 [ "BY" index ] ;
3114
3115
3116 bus-parameter: d: ! 14.41
3117 "BUS-PARAMETER" bsfd
3118 {

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3119      { bus-value } # fd
3120    } ;
3121
3122
3123    bus-value: d:                                ! 14.41A
3124    {
3125      <<*>> <bus-parameter-name |
3126      "ZERO-AMPLITUDE" |
3127      "ONE-AMPLITUDE" |
3128      "ZERO-CROSSING" |
3129      "RESPONSE-TIME" |
3130      "MESSAGE-GAP" |
3131      "WORD-LENGTH" |
3132      "WORD-GAP"
3133    } <<~>>
3134    [ { data-store | constant } dim |
3135      modifier-descriptor-name ] ;
3136    &*      The bus parameter name must have been established as
3137    &      valid for the databus in the associated ESTABLISH
3138    &      statement. For it to occur in an UPDATE statement
3139    &      of either sort, it must have been defined as UPDATABLE
3140    &      for that category of UPDATE. It and WORD-GAP are the
3141    &      only form of protocol parameter that can occur in a
3142    &      FETCH statement and then only if established as
3143    &      monitored by the test equipment in the <do exchange
3144    &      statement>. *
3145
3146
3147    set-protocol-parameter: d:                    ! 14.42
3148    "PROTOCOL-PARAMETER" bsfd
3149    {
3150      {
3151        [ "INSERT-INVALID" bsfd ]
3152        protocol-parameter-set
3153      } # fd
3154    } ;
3155
3156
3157    protocol-parameter-set: d:                    ! 14.42A
3158    { <<*>> <protocol-parameter-name
3159      [ data-store | constant |
3160        modifier-descriptor-name ] |
3161      { "COMMAND-WORD" |
3162        "STATUS-WORD" |
3163        "DATA-WORD" }
3164      unsigned-integer-number
3165      protocol-parameter-characteristic |
3166      "WORD-COUNT"
3167      [ sign unsigned-integer-number ]
3168    } ;
3169    &*      The protocol parameter name must have been established
3170    &      as valid for the databus in the associated ESTABLISH
3171    &      statement. For it to occur in an UPDATE statement
3172    &      of either sort, it must have been defined as UPDATABLE
3173    &      for that category of UPDATE. *
3174
3175
3176    protocol-parameter-characteristic: d:          ! 14.42B
3177    {
3178      "BIT" unsigned-integer-number
3179      "PULSE-CODE" |
3180      "SYNC" |
3181      "LENGTH" |
3182      "PARITY" |
3183      "T-R"

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3184     } ;
3185
3186
3187     fetch-protocol-parameter: d:                                ! 14.43
3188     "PROTOCOL-PARAMETER" bsfd
3189     {
3190     {
3191         protocol-parameter-fetched
3192     } # fd
3193     } ;
3194
3195
3196     protocol-parameter-fetched: d:                                ! 14.43A
3197     {
3198         standard-protocol-parameter
3199         "FAULT-COUNT" data-store |
3200         <<*>> <protocol-parameter-name
3201         {
3202             "FAULT-COUNT" data-store |
3203             data-store
3204         }
3205     } ;
3206     &*      The protocol parameter name must have been established
3207     &      as valid for the databus in the associated ESTABLISH
3208     &      statement. For it to occur in an UPDATE statement
3209     &      of either sort, it must have been defined as UPDATABLE
3210     &      for that category of UPDATE. It is the only form of
3211     &      protocol parameter that can occur in a FETCH statement
3212     &      and then only if it has been established as monitored by
3213     &      the test equipment in the <do exchange statement>. *
3214
3215
3216     standard-protocol-parameter: d:                                ! 14.43B
3217     {
3218     {
3219         {
3220             "COMMAND-WORD" |
3221             "STATUS-WORD" |
3222             "DATA-WORD"
3223         }
3224         {
3225             "PULSE-CODE" |
3226             "SYNC" |
3227             "LENGTH" |
3228             "PARITY" |
3229             "T-R"
3230         }
3231     } |
3232     "WORD-COUNT"
3233     } ;
3234
3235
3236     role-field: d:                                                ! 14.44
3237     "TEST-EQUIP-ROLE"
3238     {
3239         <<*>> <test-equip-role-name |
3240         "MONITOR" |
3241         "MASTER" |
3242         "SLAVE"
3243     } ;
3244     &*      The test-equip role name must have been established
3245     &      as valid for the databus in the associated <establish
3246     &      protocol statement>. *
3247
3248

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3249     digital-keyword: d:                                ! 14.45
3250     {
3251         "WORD-LENGTH" |
3252         "PULSE-CLASS" |
3253         "SKEW-TIME" |
3254         "BIT-PERIOD" |
3255         "BIT-RATE" |
3256         "SPEC"
3257     } ;
3258
3259
3260     ! 18.6.12 BASIC SYMBOLS / CHARACTER SETS
3261
3262
3263     letter: s:                                           ! 15.1.1
3264     {
3265         "A" | "B" | "C" | "D" | "E" | "F" |
3266         "G" | "H" | "I" | "J" | "K" | "L" |
3267         "M" | "N" | "O" | "P" | "Q" | "R" |
3268         "S" | "T" | "U" | "V" | "W" | "X" |
3269         "Y" | "Z"
3270     } ;
3271
3272
3273     lower-case-letter: s:                                ! 15.1.1
3274     {
3275         "a" | "b" | "c" | "d" | "e" | "f" |
3276         "g" | "h" | "i" | "j" | "k" | "l" |
3277         "m" | "n" | "o" | "p" | "q" | "r" |
3278         "s" | "t" | "u" | "v" | "w" | "x" |
3279         "y" | "z"
3280     } ;
3281
3282
3283     digit: s:                                             ! 15.1.1
3284     {
3285         "0" | "1" | "2" | "3" | "4" | "5" |
3286         "6" | "7" | "8" | "9"
3287     } ;
3288
3289
3290     blank: s:                                             ! 15.1.1
3291     " " ;
3292
3293
3294     underscore: s:                                       ! 15.1.1
3295     "_" ;
3296
3297
3298     sign: e:                                              ! 15.1.1
3299     { "+" | "-" } ;
3300
3301
3302     new-line: s:
3303     blank * " " ;
3304
3305     ! A newline character is denoted here by the literal " ".
3306
3307
3308     non-printing-character: s:                            ! 15.1.1
3309     "\"
3310     {
3311         "NUL" | "SOH" | "STX" | "ETX" | "EOT" |
3312         "ENQ" | "ACK" | "BEL" | "BS" | "HT" | "LF" |
3313         "VT" | "FF" | "CR" | "SO" | "SI" | "DLE" |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3314 "DC1" | "DC2" | "DC3" | "DC4" | "NAK" | "SYN" |
3315 "ETB" | "CAN" | "EM" | "SUB" | "ESC" | "FS" |
3316 "GS" | "RS" | "US" | "DEL"
3317 }
3318 "\ " ;
3319
3320
3321 symbol-extension-set: s: ! 15.1.1
3322 {
3323     "!" | "\"" | "#" | "%" | "&" | ":" | ";" |
3324     "<" | ">" | "?" | "@" | "~" | "[" | "]" |
3325     "|" | "_" | "\" | "^" | "`" | "{" | "}" |
3326 } ;
3327
3328
3329 common-char-set: s: ! 15.1.1
3330 {
3331     symbol-extension-set |
3332     lower-case-letter |
3333     letter |
3334     digit |
3335     "*" | "." | "=" | sign
3336 } ;
3337
3338 ! For information: atlas symbols not contained in
3339 ! the common-char-set are
3340 ! ", " , "$" , "/" , "(" , ")" , " " , " ' " , and those
3341 ! in non-printing-character.
3342
3343
3344 number: s: ! 15.2
3345 {
3346     unsigned-integer-number |
3347     decimal-number |
3348     digital-number
3349 } ;
3350
3351
3352 decimal-number: d: ! 15.2.1
3353 [ sign ]
3354 unsigned-decimal-number ;
3355
3356
3357 unsigned-decimal-number: d: ! 15.2.2
3358 [ digit % ]
3359 [
3360     "."
3361     [ digit % ]
3362 ] <<~>>
3363 [
3364     "E"
3365     [ sign ]
3366     digit % ] ;
3367
3368
3369 unsigned-integer-number: d: ! 15.2.3
3370 digit % ;
3371
3372
3373 binary-number: d: ! 15.2.4
3374 "B" binary-digit % " " ;
3375
3376
3377 octal-number: d: ! 15.2.5
3378 "O"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3379 [ "1" | "2" | "3" ]
3380 "" octal-digit % "" ;
3381
3382
3383 hexadecimal-number: d: ! 15.2.6
3384 "X"
3385 [ "1" | "2" | "3" | "4" ]
3386 "" hexadecimal-digit % "" ;
3387
3388
3389 digital-number: d: ! 15.2.7
3390 {
3391     binary-number |
3392     octal-number |
3393     hexadecimal-number
3394 } ;
3395
3396
3397 fstatno: d: ! 15.3.1
3398 { "E" | blank }
3399 statno sfd ;
3400
3401
3402 statno: e: ! 15.3.1-1
3403 {
3404     test-number step-number |
3405     blank blank blank step-number |
3406     blank blank blank blank blank blank
3407 } ;
3408
3409
3410 statement-number: d: ! 15.3.3
3411 test-number step-number ;
3412
3413
3414 test-number: d: ! 15.3.3A
3415 digit digit digit digit ;
3416
3417
3418 step-number: d: ! 15.3.3B
3419 digit digit ;
3420
3421
3422 flag-statement: d: ! 15.3.2A
3423 "B"
3424 [ msgchar ]
3425 statement-terminator ;
3426
3427
3428 comment-statement: d: ! 15.3.2B
3429 "C"
3430 [ msgchar ]
3431 statement-terminator ;
3432
3433
3434 char-string-set: s: ! 15.10.1
3435 { msgchar-set | "(" | ")" } ;
3436
3437
3438 message-text-set: s: ! 15.10.2
3439 { msgchar-set | "(" | ")" | "" } ;
3440
3441
3442 msgchar-set: s: ! 15.10.3
3443 {

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3444     non-printing-character |
3445     common-char-set |
3446     blank | "," | "/"
3447 } ;
3448
3449
3450 block-name-set: s:                                ! 4.2.2-1
3451 { common-char-set | "," | "/" } ;
3452
3453
3454 ! 18.6.13 IDENTIFIERS / LITERALS / LABELS
3455
3456
3457 block: s:                                           ! 4.2.2-2
3458 "" block-name-set % "" ;
3459
3460
3461 terminal-identifier: s:                             ! 6.5-2
3462 {
3463     [ "*" ]
3464     {
3465         letter | lower-case-letter | "/" letter |
3466         digit | "+" | "-" | "."
3467     } % |
3468     "COMMON" |
3469     "EARTH" |
3470     "ATMOS"
3471 } ;
3472
3473
3474 connection: s:                                     ! 14.24-1
3475 {
3476     {
3477         letter | lower-case-letter | "/" letter |
3478         digit | "+" | "-" | "."
3479     } % |
3480     "COMMON" |
3481     "EARTH" |
3482     "ATMOS"
3483     <terminal-identifier
3484 } ;
3485
3486
3487 label: s:                                           ! 15.5
3488 ""
3489 {
3490     common-char-set | "/" | "," | blank | underscore
3491 } %
3492 "" ;
3493
3494
3495 extend-label: s:                                    ! 15.5.2
3496 {
3497     common-char-set | "/" | "," | blank | underscore
3498 } % ;
3499
3500
3501 parameter: s:                                       ! 15.6.5
3502 label ;
3503
3504
3505 character-string: s:                                ! 15.10.1
3506 "C" char-string-set * "" ;
3507
3508

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3509 message-text: s: ! 15.10.2
3510 { blank *
3511 { message-text-set | new-line }
3512 } % ;
3513
3514
3515 msgchar: s: ! 15.10.3
3516 {
3517 blank *
3518 { msgchar-set | new-line }
3519 } % ;
3520
3521
3522 drawing-identifier: s: ! 15.10.4
3523 {
3524 common-char-set |
3525 blank |
3526 "(" | ")" | "/" | "'"
3527 } % ;
3528
3529
3530 modifier-descriptor-name: s: ! 15.10.5.1
3531 extend-label ;
3532
3533
3534 module-name: s: ! 15.10.6
3535 label ;
3536
3537
3538 noun-name: s: ! 15.10.7.1
3539 extend-label ;
3540
3541
3542 pin-descriptor-name: s: ! 15.10.8.1
3543 extend-label ;
3544
3545
3546 modifier-name: s: ! 15.10.9.1
3547 extend-label ;
3548
3549
3550 binary-digit: s: ! 15.10.12
3551 { "0" | "1" } ;
3552
3553
3554 octal-digit: s: ! 15.10.13
3555 {
3556 "0" | "1" | "2" | "3" | "4" |
3557 "5" | "6" | "7"
3558 } ;
3559
3560
3561 hexadecimal-digit: s: ! 15.10.14
3562 {
3563 digit | "A" | "B" | "C" |
3564 "D" | "E" | "F"
3565 } ;
3566
3567
3568 test-equip-role-name: s: ! 15.10.15
3569 extend-label ;
3570
3571
3572 bus-mode-name: s: ! 15.10.16
3573 extend-label ;

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

[illegible]

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3639
3640     exchange: s:                                ! 15.11.17
3641     label ;
3642
3643
3644     configuration: s:                            ! 15.11.18
3645     label ;
3646
3647
3648     digital-source: s:                          ! 15.11.19
3649     label ;
3650
3651
3652     digital-sensor: s:                          ! 15.11.20
3653     label ;
3654
3655
3656     dim-name: s:                                ! 15.11.21
3657     extend-label ;
3658
3659
3660     external-semantics-specification: s:         ! 15.11.22
3661     label ;
3662
3663
3664     external-syntax-specification: s:           ! 15.11.23
3665     label ;
3666
3667
3668     drawing: s:                                 ! 15.11.24
3669     label ;
3670
3671
3672     device-identifier-name: s:                  ! 15.11.25
3673     <enumeration-element ;
3674
3675
3676     requirement: s:                             ! 15.11.26
3677     label ;
3678
3679
3680     exchange-configuration: s:                  ! 15.11.27
3681     label ;
3682
3683
3684     boolean-variable: s:                        ! 15.11.28
3685     <variable-identifier ;
3686
3687
3688     external-digital-specification: s:          ! 15.11.29
3689     label ;
3690
3691
3692     external-pulse-class-specification: s:      ! 6.19.3G
3693     label ;
3694
3695
3696     program-name: s:                           ! 15.11.30
3697     label ;
3698
3699
3700     ! 18.6.14 SEPARATORS / STATEMENT-NUMBERS
3701
3702
3703     sfd: s:                                     ! 3.2.4

```


18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3704      blank % ;
3705
3706      ! sfd is the sub-field delimiter.
3707
3708
3709      bsfd: s:                                     ! 3.2.4
3710      { sfd | new-line * blank * } ;
3711
3712      ! bsfd is a special sub-field delimiter which
3713      ! permits newlines in a few especially large fields.
3714
3715
3716      fd: s:                                         ! 3.2.4
3717      blank * " , "
3718      {
3719          blank % |
3720          new-line % blank * } ;
3721
3722      ! fd is the field delimiter.
3723
3724
3725      statement-terminator: s:                       ! 3.2.6
3726      blank % "$" new-line * ;
3727
3728
3729      ! 18.6.15   DESCRIPTORS
3730
3731
3732      modifier-descriptor-pulse-class: s:           ! 6.19.3
3733      {
3734          "RZ" | "NRZ" | "BIP" | "MIP" | "AMI" |
3735          "HDB" unsigned-integer-number
3736      } ;
3737
3738
3739      pin-descriptor: s:                             ! 14.14.1.1
3740      sfd
3741      {
3742          "HI" | "LO" | "VIA" |
3743          "A" | "B" | "C" | "N" |
3744          "TRUE" | "COMPL" |
3745          "SCREEN" | "GUARD" |
3746          "R1" | "R2" | "R3" | "R4" |
3747          "S1" | "S2" | "S3" | "S4" |
3748          "X" | "Y" | "Z" |
3749          "*"
3750          {
3751              letter |
3752              lower-case-letter |
3753              "/" letter | digit |
3754              "+" | "-" | "."
3755          } % |
3756          <pin-descriptor-name |
3757          terminal-identifier
3758      } ;
3759
3760
3761      reference-phase: s:                             ! 14.14.1.2
3762      sfd
3763      {
3764          "PHASE-A" | "PHASE-AB" | "PHASE-AC" |
3765          "PHASE-B" | "PHASE-BC" | "PHASE-CB" |
3766          "PHASE-C" | "PHASE-CA" | "PHASE-BA"
3767      } ;
3768

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3769
3770      modifier-descriptor: s:                                ! 15.10.5
3771      {
3772          modifier-descriptor-pulse-class |
3773          <modifier-descriptor-name
3774      } ;
3775
3776
3777      ! 18.6.16  NOUNS
3778
3779
3780      analog-noun: s:                                         ! 16.0
3781      {
3782          "AC SIGNAL" |
3783          "AM SIGNAL" |
3784          "AMBIENT CONDITIONS" |
3785          "COMMON" |
3786          "DC SIGNAL" |
3787          "DOPPLER" |
3788          "EARTH" |
3789          "EVENTS" |
3790          "FM SIGNAL" |
3791          "IMPEDANCE" |
3792          "LIGHT" |
3793          "LOGIC SIGNAL" |
3794          "PAM" |
3795          "PM SIGNAL" |
3796          "PULSED AC" |
3797          "PULSED AC TRAIN" |
3798          "PULSED DC" |
3799          "PULSED DC TRAIN" |
3800          "PULSED DOPPLER" |
3801          "RADAR SIGNAL" |
3802          "RAMP SIGNAL" |
3803          "RANDOM NOISE" |
3804          "RESOLVER" |
3805          "ROTATION" |
3806          "SHORT" |
3807          "SQUARE WAVE" |
3808          "STEP SIGNAL" |
3809          "SUP CAR SIGNAL" |
3810          "SYNCHRO" |
3811          "TIME-INTERVAL" |
3812          "TRIANGULAR WAVE SIGNAL" |
3813          "VIBRATION" |
3814          "VOR" |
3815          "WAVEFORM"
3816      } ;
3817
3818
3819      ! 18.6.17  NOUN-MODIFIERS
3820
3821
3822      measured-characteristic-mnemonic: s:                    ! 17.2
3823      modifier
3824      [ ampl-sfx ]
3825      [ phase-sfx ]
3826      [ pulse-sfx ]
3827      [ "-RATIO" ] ;
3828
3829
3830      characteristic-mnemonic: s:                              ! 17.2
3831      measured-characteristic-mnemonic
3832      [ "-NUMERATOR" | "-DENOMINATOR" ]
3833      [ "-QUIES" | "-TRANS" ]

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3834     [ "-REF" ]
3835     sfd ;
3836
3837
3838     modifier-mnemonic: s:                                ! 15.11.8
3839     modifier modifier-name ;
3840
3841
3842     modifier: s:                                          ! 17.3
3843     {
3844         " AC-COMP" |
3845         "AC-COMP-FREQ" |
3846         "AM-COMP" |
3847         "ANGLE" |
3848         "ANGLE-ACCEL" |
3849         "ANGLE-RATE" |
3850         "ATTEN" |
3851         "BANDWIDTH" |
3852         "BAROMETRIC-PRESS" |
3853         "BURST" |
3854         "CAP" |
3855         "CAR-AMPL" |
3856         "CAR-FREQ" |
3857         "CAR-HARMONICS" |
3858         "CAR-PHASE" |
3859         "CAR-RESID" |
3860         "COUNT" |
3861         "CURRENT" |
3862         "CURRENT-LMT" |
3863         "DC-OFFSET" |
3864         "DELAY" |
3865         "DISS-FACTOR" |
3866         "DISTORTION" |
3867         "DOPPLER-BANDWIDTH-POWER" |
3868         "DOPPLER-FREQ" |
3869         "DOPPLER-SHIFT" |
3870         "DROOP" |
3871         "DUTY-CYCLE" |
3872         "FALL-TIME" |
3873         "FM-COMP" |
3874         "FM-INDEX" |
3875         "FREQ" |
3876         "FREQ-DEV" |
3877         "FREQ-DEV-RATE" |
3878         "FREQ-LMT" |
3879         "FREQ-WINDOW" |
3880         "HARMONICS" |
3881         "HARM-" unsigned-integer-number "-PHASE" |
3882         "HARM-" unsigned-integer-number "-VOLTAGE" |
3883         "IDENT-SIG" |
3884         "IDENT-SIG-FREQ" |
3885         "IDENT-SIG-MOD" |
3886         "ILLUM" |
3887         "IMP" |
3888         [ impedance-pfx ] "IMP-ANGLE" |
3889         [ impedance-pfx ] "IMP-MAGNITUDE" |
3890         "IND" |
3891         "INSERTION-LOSS" |
3892         "LUM-FLUX" |
3893         "LUM-INT" |
3894         "LUMINANCE" |
3895         "MOD-AMPL" |
3896         "MOD-DIST" |
3897         "MOD-FREQ" |
3898         "MOD-INDEX" |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3899      "MOD-OFFSET" |
3900      "MOD-PHASE" |
3901      "NEG-EDGE" |
3902      "NEG-SLOPE" |
3903      "NOISE" |
3904      "NOISE-AMPL-DENS" |
3905      "NOISE-PWR-DENS" |
3906      "NON-HARMONICS" |
3907      "NON-LIN" |
3908      "OSCILLATORY" |
3909      "OVERSHOOT" |
3910      "P-AMPL" |
3911      "PEAK-DEGEN" |
3912      "PERIOD" |
3913      "PHASE-ANGLE" |
3914      "PHASE-DEV" |
3915      "PHASE-JIT" |
3916      "PHASE-SHIFT" |
3917      "POS-EDGE" |
3918      "POS-SLOPE" |
3919      "POWER" |
3920      "PRESHOOT" |
3921      "PRF" |
3922      "PULSE-ENERGY" |
3923      "PULSE-WIDTH" |
3924      "PWR-LMT" |
3925      "QUAD" |
3926      "RADIAL" |
3927      "RADIAL-RATE" |
3928      [ impedance-pfx ] "REACTANCE" |
3929      "REF-INERTIAL" |
3930      "REF-PHASE-FREQ" |
3931      "REF-UUT" |
3932      "REF-VOLT" |
3933      "RELATIVE-HUMIDITY" |
3934      "REPLY-EFF" |
3935      [ impedance-pfx ] "RES" |
3936      "RESP" |
3937      "RETURN-LOSS" |
3938      "RINGING" |
3939      "RISE-TIME" |
3940      "ROUNDING" |
3941      "SAMPLE" |
3942      "SAMPLE-SPACING" |
3943      "SAMPLE-WIDTH" |
3944      "SCORSBY" |
3945      "SPACING" [ pulse-spacing-sfx ] |
3946      "STIM" |
3947      "SUB-CAR-FREQ" |
3948      "SUB-CAR-MOD" |
3949      "SWEEP-INT" |
3950      "SWR" |
3951      "TARGET-RANGE" |
3952      "TARGET-RANGE-ACCEL" |
3953      "TARGET-RANGE-RATE" |
3954      "TEMP" |
3955      "THREE-PHASE-DELTA" |
3956      "THREE-PHASE-WYE" |
3957      "TIME" |
3958      "TIME-ASYM" |
3959      "TIME-JIT" |
3960      "TRANSMITTANCE" |
3961      "TRANS-TIME" |
3962      "TRIG" |
3963      "UNDERSHOOT" |

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

3964     "VAR-PHASE-FREQ" |
3965     "VAR-PHASE-MOD" |
3966     "VIBRATION-ACCEL" |
3967     "VIBRATION-AMPL" vibration-sfx |
3968     "VIBRATION-RATE" |
3969     "VOLTAGE" |
3970     "VOLTAGE-RAMPED" |
3971     "VOLTAGE-STEPPED" |
3972     "VOLT-LMT" |
3973     "WAVE-LENGTH" |
3974     "ZERO-INDEX" |
3975     <modifier-name
3976   } ;
3977
3978
3979   ! 18.6.18 NOUN-MODIFIERS - PREFIXES / SUFFIXES
3980
3981
3982   impedance-pfx: s:                                ! 17.2.1
3983   {
3984     "UUT-" |
3985     "TEST-EQUIP-" |
3986     "THRU-"
3987   } ;
3988
3989
3990   ampl-sfx: s:                                       ! 17.2.2
3991   {
3992     param-sfx
3993     [ "-IN-PHASE" | "-QUAD" ] |
3994     "-IN-PHASE" |
3995     "-QUAD" |
3996     "-INST"
3997   } ;
3998
3999
4000   param-sfx: s:                                     ! 17.2.2
4001   {
4002     "-AV" | "-P" | "-PP" |
4003     "-P-NEG" | "-P-POS" | "-TRMS" |
4004     "-P-POS-ZERO" | "P-NEG-ZERO"
4005   }
4006   [ "-LMT" ] ;
4007
4008
4009   phase-sfx: s:                                     ! 17.2.3
4010   {
4011     "-PHASE-A" | "-PHASE-AB" | "-PHASE-AC" |
4012     "-PHASE-B" | "-PHASE-BC" | "-PHASE-CB" |
4013     "-PHASE-C" | "-PHASE-CA" | "-PHASE-BA"
4014   } ;
4015
4016
4017   pulse-sfx: s:                                     ! 17.2.5
4018   {
4019     "-P" unsigned-integer-number |
4020     "-P" unsigned-integer-number "-"
4021     { "POS" | "NEG" }
4022   } ;
4023
4024
4025   pulse-spacing-sfx: s:                             ! 17.2.5
4026   pulse-sfx "-"
4027   { "POS" | "NEG" }
4028   pulse-sfx "-"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

4029      { "POS" | "NEG" } ;
4030
4031
4032      vibration-sfx: s:                                ! 17.2.2
4033      { "-P" | "-PP" | "-TRMS" } ;
4034
4035
4036      ! 18.6.19  DIMENSIONS
4037
4038
4039      dim: s:                                           ! 15.8
4040      { sfd dimension | time-dim | freq-dim } ;
4041
4042
4043      dimension: s:                                     ! 15.8
4044      {
4045
4046          !          ACCELERATION
4047          "M/S2" |
4048          "GRAV" |
4049
4050          !          ANGLE, PLANE
4051          "DEG" | "REV" | "MIL" | "RAD" |
4052          "MRAD" | "URAD" | "SECOND" | "MINUTE" |
4053
4054          !          BURST LENGTH
4055          "CYCLES" | "PULSES" |
4056
4057          !          CAPACITANCE
4058          "FD" | "UFD" | "NFD" | "PFD" | "MFD" |
4059
4060          !          CURRENT
4061          "A" | "MA" | "UA" | "KA" | "NA" |
4062
4063          !          DIGITAL BIT RATE
4064          "BITS/SEC" | "KBITS/SEC" | "MBITS/SEC" |
4065
4066          !          DIGITAL WORD LENGTH
4067          "DIGITS" | "CHAR" | "BITS" |
4068
4069          !          DIGITAL WORD RATE
4070          "WORDS/SEC" | "KWORDS/SEC" | "MWORDS/SEC" |
4071
4072          !          ENERGY
4073          "J" | "KJ" | "DBPJ" | "MJ" | "PJ" |
4074          "DBMJ" | "TJ" | "GJ" | "DBJ" | "UJ" |
4075          "NJ" | "EV" | "KEV" | "MEV" | "GEV" |
4076
4077          !          EVENTS
4078          "TIMES" | "COUNTS" | "STEPS" |
4079
4080          !          FREQUENCY
4081          "HZ" | "KHZ" | "MHZ" | "GHZ" | "THZ" |
4082          "PPS" | "KPPS" |
4083
4084          !          ILLUMINANCE
4085          "LX" |
4086
4087          !          IMPEDANCE
4088          "OHM" | "MOHM" | "KOHM" |
4089
4090          !          INDUCTANCE
4091          "H" | "MH" | "UH" | "NH" | "PH" |
4092
4093          !          LENGTH

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```

4094      "M" | "KM" | "CM" | "MM" |
4095      "UM" | "NM" | "N-MILE" | "FT" |
4096      "S-MILE" | "IN" | "MILS" |
4097
4098      !          LUMINANCE
4099      "NT" | "CD/M2" |
4100
4101      !          LUMINOUS FLUX
4102      "LM" |
4103
4104      !          LUMINOUS INTENSITY
4105      "CD" |
4106
4107      !          POWER
4108      "W" | "KW" | "DBM" | "MW" | "UW" | "NW" |
4109      "DBW" | "GW" | "DBKW" | "HP" | "TW" |
4110
4111      !          PRESSURE
4112      "PA" | "KPA" | "MPA" | "UPA" | "MBAR" | "MMHG" |
4113      "INHG" | "FTSW" | "PSI" | "IN-WATER" | "BAR" | "UBAR" |
4114
4115      !          RATIO
4116      "DB" | "PC" | blank |
4117
4118      !          ROTATIONAL FREQUENCY
4119      "SEC-1" | "MIN-1" |
4120
4121      !          TEMPERATURE
4122      "DEGC" | "DEGK" | "DEGR" | "DEGF" |
4123
4124      !          TIME
4125      "HR" | "MIN" | "SEC" | "MSEC" | "D" |
4126      "USEC" | "NSEC" | "PSEC" | "KSEC" |
4127
4128      !          VELOCITY, LINEAR
4129      "FT/SEC" | "M/SEC" | "MACH" | "KT" |
4130      "M/HR" | "KM/HR" |
4131
4132      !          VOLTAGE
4133      "V" | "KV" | "MV" | "UV" |
4134
4135      !          DIMENSIONS FROM EXTEND
4136      <dim-name } ;
4137
4138
4139      freq-dim: s:
4140      sfd freq-dimension ;
4141
4142
4143      freq-dimension: s:
4144      {
4145      "HZ" | "KHZ" | "MHZ" | "GHZ" | "THZ" |
4146      "PPS" | "KPPS"
4147      } ;
4148
4149
4150      time-dim: s:
4151      sfd time-dimension ;
4152
4153
4154      time-dimension: s:
4155      {
4156      "HR" | "MIN" | "SEC" |
4157      "MSEC" | "USEC" | "NSEC" |
4158      "PSEC" | "KSEC" | "D"

```

18.0 ATLAS FORMAL SYNTAX DEFINITION

```
4159      } ;  
4160  
4161  
4162      FINIS
```


18.0 ATLAS FORMAL SYNTAX DEFINITION**18.7 CROSS-REFERENCE LISTING OF DEFINITIONS AND USAGE
OF SYNTAX PRODUCTION RULES, SYMBOLS AND LABELS****18.7.1 CROSS-REFERENCE LISTING OF SYNTAX PRODUCTION RULE NAMES**

RULE NAME	DEF	USAGE
	...	, = many
actual-parameters	1682	1674,
ampl-sfx	3990	3824,
analog-event	637	633,
analog-noun	3780	2741,
analog-signal	1960	1948,
apply-statement	1945	1767,
arm-statement	1876	1785,
array-range	3102	1017, 1426, 1436, 1459, 1469, 2357, 2387, 2465, 2508, 2613, 2712, 2966,
atlas-module-structure	29	15,
atlas-program-structure	21	15,
atlas-test-specification-structure	13	
base-type	208	202,
begin-atlas-module-statement	57	31,
begin-atlas-program-statement	45	23,
begin-block-statement	73	69,
binary-digit	3550	3374,
binary-number	3373	3391,
bit-rate-range	851	845,
blank	3290	3303, 3398, 3405, 3406, 3407, 3446, 3490, 3497, 3510, 3517, 3525, 3704, 3710, 3717, 3719, 3720, 3726, 4116,
block	3457	75, 95,
block-body	79	70,
block-name-set	3450	3458,
block-structure	67	84, 1204,
boolean-evaluation	1361	1323,
boolean-operator	1381	1289,
boolean-variable	3684	782, 909,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
bsfd	3709	516, 569, 1085, 1086, 1290, 1296, 1311, 2685, 2698, 3118, 3149, 3151, 3189,
bus-installation-parameters	450	435,
bus-mode-name	3572	484, 969, 1156,
bus-parameter	3116	1012, 2324, 2350, 2374, 2430, 2454, 2957,
bus-parameter-name	3576	526, 1142, 3125,
bus-parameter-range	524	520,
bus-parameter-value-range	553	535,
bus-redundancy-definition	440	437,
bus-specification	3584	430,
bus-value	3123	3119,
calculate-condition	1267	1262,
calculate-statement	1256	1251,
change-statement	1894	1787,
char-string-set	3434	3506,
character-string	3505	1331,
characteristic-mnemonic	3830	658, 2480, 2543, 2752, 2786, 2808, 2854, 3038, 3046, 3055,
characteristic-value	2502	2480,
commence-main-procedure-statement	99	25,
comment-statement	3428	111, 120, 124, 138, 143, 155, 1220,
common-char-set	3329	3445, 3451, 3490, 3497, 3524,
compare-statement	1391	1253,
comparison-operator	1385	1295,
condition	2649	1321,
configuration	3644	750, 954, 1727, 3078,
configuration-body	741	738,
conn	2663	286, 642, 1814, 1949, 1963, 1970, 1971, 2014,
conn-descriptor	2692	2685,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
conn-set	2680	774, 901, 2376, 2397, 2409, 2674, 2676, 3083,
connect-exchange-configuration-statement	2390	2287,
connect-set-ref	2672	2666, 2668,
connect-statement	1848	1783,
connection	3474	214, 1328, 2696,
connection-extension	1037	1024,
constant	1326	174, 231, 241, 254, 256, 261, 564, 1000, 1017, 1320, 2359, 2387, 2945, 3134, 3159,
constant-declaration	168	161,
constant-identifier	3588	172, 1333,
create-file-statement	1507	1406,
data-input	1418	1413,
data-output	1452	1448,
data-store	3091	1000, 1017, 1260, 1319, 1363, 1393, 1424, 1436, 1509, 1515, 1626, 1692, 1732, 2074, 2101, 2113, 2120, 2121, 2274, 2358, 2387, 2461, 2466, 2613, 2698, 2945, 2966, 2968, 2988, 2994, 3103, 3134, 3159, 3199, 3202, 3204,
database-enable-data	2386	2382, 2383,
databus-definition	434	431,
databus-device	992	985, 988, 2338, 2341, 2380, 2381, 2443, 2445,
databus-extension	1137	1027,
databus-fetch-data	2458	2451, 2452, 2453,
databus-transaction	2923	2899,
databus-update-data	2354	2347, 2348, 2349,
decimal-number	3352	1329, 1731, 2519, 2520, 2524, 2714, 2722, 2723, 2745, 3347,
declare-statement	158	135, 153,
def-dig-sensor-levels	927	921,
def-dig-sequence	866	796, 923,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
def-dig-source-levels	856	794,
define-databus-data	1016	1009, 1010, 1011,
define-digital-configuration-statement	749	737,
define-digital-configuration-structure	735	272,
define-digital-sensor-statement	882	745,
define-digital-source-statement	755	743,
define-drawing-statement	277	267,
define-exchange-configuration-statement	1189	271,
define-exchange-fields	1008	979,
define-exchange-statement	958	274,
define-function-statement	353	349,
define-function-structure	347	270,
define-procedure-statement	308	301,
define-procedure-structure	299	269,
define-signal-statement	289	268,
define-statement-structure	265	140,
delete-file-statement	1513	1408,
device-identifier-name	3672	504, 994, 2942,
digit	3283	3334, 3358, 3361, 3366, 3370, 3415, 3419, 3466, 3478, 3563, 3753,
digital-event	645	633,
digital-format	800	784, 911,
digital-keyword	3249	2752, 2786, 2854,
digital-number	3389	1330, 3349,
digital-sensor	3652	883, 947, 948, 3077,
digital-sensor-characteristics	3042	934, 939,
digital-sensor-definition	897	894,
digital-source	3648	756, 947, 948, 3076,
digital-source-characteristics	3035	860, 862,
digital-source-definition	770	767,
digital-synchronization	841	804,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
dim	4039	564, 674, 853, 1096, 1097, 2520, 2525, 2595, 2597, 2715, 2723, 2727, 2745, 2989, 2996, 3134,
dim-lim	2592	2584,
dim-name	3656	1103, 4136,
dimension	4043	4040,
dimension-extension	1093	1076, 1176,
dimensioned-constant	563	556, 557, 560,
dimensioned-number	2744	2764, 2767, 2774, 2815, 2816, 2822, 2825, 2862,
disable-escape-statement	1755	1535,
disable-event-statement	1914	1789,
disable-exchange-configuration-statement	2414	2289,
disable-file-access-statement	1502	1405,
disable-manual-intervention-statement	1715	1707,
disconnect-exchange-configuration-statement	2402	2288,
disconnect-statement	1862	1784,
do-exchange-statement	2294	2284,
do-sequential-body	2259	2251,
do-sequential-statement	2254	2250,
do-sequential-structure	2248	2135,
do-simultaneous-body	2173	2162,
do-simultaneous-statement	2165	2161, 2231,
do-simultaneous-structure	2159	2133,
do-timed-conditional-structure	2237	2219,
do-timed-control-structure	2213	2208,
do-timed-digital-body	2201	2185, 2216, 2218, 2239, 2243, 2244,
do-timed-digital-statement	2188	2184,
do-timed-digital-structure	2182	2134, 2207,
do-timed-simultaneous-structure	2229	2210,
drawing	3668	278, 1949, 1971, 2013,
drawing-definition	284	281,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
drawing-identifier	3522	279,
else-if-then-statement	1579	1543, 2242,
else-statement	1568	1547, 2244,
enable-digital-configuration-statement	1725	1532,
enable-disable-manual-intervention-structure	1703	1530,
enable-escape-statement	1740	1533,
enable-event-statement	1908	1788,
enable-exchange-configuration-statement	2362	2286,
enable-file-access-statement	1487	1404,
enable-manual-intervention-statement	1710	1705,
end-block-statement	93	70,
end-digital-configuration-statement	953	738,
end-do-statement	2177	2162, 2185, 2234, 2251,
end-for-statement	1654	1621, 2216,
end-function-statement	371	350,
end-if-statement	1573	1549, 2245,
end-procedure-statement	342	302,
end-while-statement	1612	1587, 2218,
enumeration-element	3600	211, 1332, 3673,
escape-indicator	1747	1744, 1757,
establish-bus-info	603	495,
establish-bus-parameters	514	463,
establish-device	501	493, 494,
establish-protocol-parameters	567	464,
establish-protocol-statement	426	136,
eval-statement-characteristics	2636	2023,
evaluation-field	2621	1394, 2544, 2638, 3044, 3050,
event	3620	632, 689, 690, 698, 702, 716, 717, 724, 847, 1836, 1910, 1916, 2154, 2532, 2908, 2918, 2974, 3013, 3018, 3030, 3069,
event-based-event-definition	687	684,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
event-field	1834	1810,
event-indicator	3624	723, 1749,
event-interval	3628	710, 1837, 2975,
event-sensing	672	654,
exchange	3640	959, 2331, 2435, 2919, 2924,
exchange-configuration	3680	1190, 2296, 2321, 2332, 2364, 2392, 2404, 2416, 2428, 2436,
exchange-contents	2346	2343,
exchange-definition	966	963,
exchange-devices	984	979,
exchange-end	2913	2910,
exchange-expression	2883	2305, 2309,
exchange-fetch	2434	2422,
exchange-frame	2891	2886,
exchange-frequency	2903	2894,
exchange-model	481	462,
exchange-monitor	2953	2927,
exchange-set	2303	2300,
exchange-simulate	2931	2926,
exchange-update	2330	2315,
expression	1280	647, 653, 660, 674, 1260, 1318, 1342, 1420, 1454, 1457, 1469, 1484, 1494, 1509, 1515, 1553, 1580, 1592, 1632, 1633, 1634, 1687, 2050, 2068, 2096, 2100, 2111, 2117, 2357, 2465, 2506, 2625, 2646, 2712, 2727, 2731,
expression-item	1300	1286,
expression-term	1315	1303,
ext-pin	1083	1055, 1056,
extend-atlas-statement	1020	118,
extend-cnx	1053	1039,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
extend-label	3495	3531, 3539, 3543, 3547, 3569, 3573, 3577, 3581, 3657,
extend-modifier-descriptor	1125	1065, 1177,
extend-parameter-definition	1174	1144, 1151,
external-digital-specification	3688	762, 889,
external-pulse-class-specification	3692	832,
external-semantics-specification	3660	1034, 1039, 1050, 1104, 1134,
external-syntax-specification	3664	1066,
fd	3716	... ,
fetch-databus-data	2962	2958,
fetch-exchange-configuration-statement	2420	2291,
fetch-protocol-parameter	3187	2431, 2455, 2956,
fetch-statement	1888	1786,
fetch-update-spec	540	518, 571,
fetched-exchange-data	2442	2439,
fetched-information	2450	2447,
fields	0	293,
file	1498	1420, 1432, 1454, 1465, 1495, 1504, 2357, 2465,
finish-statement	1699	1529,
flag-statement	3422	1219,
for-then-range	1630	1626,
for-then-statement	1624	1620, 2215,
for-then-structure	1618	1526,
formal-parameter	3086	318, 323, 357,
freq-dim	4139	4040,
freq-dimension	4143	4140,
fstatno	3397	... ,
function	3612	354, 367, 372, 1339, 2065, 2108, 2114,
function-body	361	350,
function-expression	1338	1317,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
gate-conn-or-event-field	1808	1826, 1885, 1941, 1991, 2004, 2012, 2024, 2032,
gate-field	2971	690, 1811, 1839, 1962,
go-to-statement	1660	1527, 2220,
harmonic-components	2492	2478,
hexadecimal-digit	3561	3386,
hexadecimal-number	3383	3394,
identify-event-based-event-statement	679	613,
identify-event-every-after	696	693,
identify-event-from-to	715	712,
identify-event-indicator-statement	720	615,
identify-event-interval-statement	706	614,
identify-signal-based-event-statement	629	612,
identify-statements	609	141,
identify-time-based-event-statement	728	617,
identify-timer-statement	620	611,
if-then-else-else-if-structure	1538	1524,
if-then-statement	1552	1540, 2238,
impedance-pfx	3982	3888, 3889, 3928, 3935,
include-statement	412	109,
index	2730	3096, 3105, 3112, 3113,
index-range	3111	3107,
initial	247	196,
initial-slope	2845	2831,
initial-sync	2828	2800,
initialiser	251	248, 257, 259,
initiate-statement	2035	1773,
input-statement	1411	1402,
inst-val	3053	

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
label	3487	3502, 3535, 3585, 3589, 3593, 3597, 3601, 3605, 3609, 3613, 3617, 3621, 3629, 3633, 3637, 3641, 3645, 3649, 3653, 3661, 3665, 3669, 3677, 3681, 3689, 3693, 3697,
leave-block-statement	87	1225,
leave-for-statement	1642	1231, 2226,
leave-function-statement	366	1227,
leave-if-statement	1562	1228, 2223,
leave-procedure-statement	337	1226,
leave-while-statement	1600	1229, 2224,
letter	3263	3333, 3465, 3477, 3751, 3753,
local-preamble-structure	151	80, 333, 362,
logic-alternation	824	860, 934,
lower-case-letter	3273	3332, 3465, 3477, 3752,
main-procedural-statements	1209	82, 1202,
main-procedural-structure	1200	26, 334, 363, 1559, 1597, 1639, 2174, 2260,
max-time	2641	848, 1414, 1812, 1840, 2155, 2169, 2310, 2439, 3014, 3019, 3031, 3070,
measure-statement	1999	1769,
measured-characteristic	2608	1822, 1883, 2001, 2009, 2021, 2029,
measured-characteristic-mnemonic	3822	397, 409, 898, 1937, 1988, 2610, 3832,
message-text	3509	1575, 1614, 1656,
message-text-set	3438	3511,
mod-type	1182	1076, 1176,
modifier	3842	1070, 3824, 3839,
modifier-definition	1059	1046,
modifier-descriptor	3770	1128, 1129, 2505, 2755, 2791,
modifier-descriptor-name	3530	1133, 3773,
modifier-descriptor-pulse-class	3732	3772,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
modifier-extension	1042	1025,
modifier-mnemonic	3838	
modifier-name	3546	1061, 3975,
modifier-typing	1073	1063,
module-name	3534	59, 64, 416,
module-preamble-structure	146	32,
monitor-statement	2007	1770,
monitored-characteristic-mnemonic	408	400,
msgchar	3515	3424, 3430,
msgchar-set	3442	3435, 3439, 3518,
new-dimension	1102	1096,
new-exchange-details	2337	2334,
new-line	3302	3511, 3518, 3710, 3720, 3726,
new-modifier-descriptor	1132	1128,
non-atlas-module-structure	0	16,
non-printing-character	3308	3444,
noun	2740	405, 1038, 1043, 2736,
noun-extension	1031	1023,
noun-field	2734	638, 771, 899, 1825, 1884, 1940, 1961, 1970, 1990, 2002, 2010, 2022, 2030,
noun-name	3538	1032, 2741,
number	3344	
octal-digit	3554	3380,
octal-number	3377	3392,
on-field	3073	654, 2048, 2051, 2075, 2086, 2103,
output-format	1475	1470,
output-statement	1446	1403,
param-sfx	4000	3993,
parameter	3501	3087,
parameter-definition	315	312,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
pc-lim	2600	2587, 2589,
perform-statement	1666	1528,
perform-table	1677	1673,
perform-variables	1672	1668,
phase-sfx	4009	3825,
pin-descriptor	3739	437, 1090, 2683, 2879, 2880,
pin-descriptor-name	3542	1090, 3756,
pin-name	1089	1084, 1085,
pos-neg-edge	2572	2546, 2558, 2810, 2848,
pos-neg-slope	2576	
pre-declared-type	223	203,
preamble-fixed-sequence	116	113, 148,
preamble-variable-sequence	133	113, 148,
predeclared-enumeration	1347	1335,
predefined-function	1367	1339,
procedural-segment-for-then	1638	1621,
procedural-segment-if-then-else-else-if	1558	1541, 1544, 1548,
procedural-segment-while-then	1596	1587,
procedural-statements-control	1522	1213,
procedural-statements-data-processing	1249	1211, 2206,
procedural-statements-databus	2282	1216,
procedural-statements-input-output	1400	1212,
procedural-statements-leave	1223	1217,
procedural-statements-resume	1234	1218,
procedural-statements-signal	1764	1214,
procedural-statements-timing	2130	1215,
procedure	3608	309, 338, 343, 1667, 1742,
procedure-body	332	301,
program-name	3696	47, 53,
program-preamble-structure	107	24,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
protocol	3636	429, 445, 961, 1193, 2320, 2366, 2395, 2407, 2427,
protocol-fetch	2426	2422,
protocol-initialization	2372	2367,
protocol-parameter-characteristic	3176	3165,
protocol-parameter-fetched	3196	3192,
protocol-parameter-name	3580	594, 1149, 3158, 3200,
protocol-parameter-set	3157	3153,
protocol-parameter-types	577	573,
protocol-update	2319	2315,
prove-referenced-signal	2093	2089,
prove-statement	2078	1777, 2205, 2232,
prove-value	2106	2095,
pulse-class	829	803,
pulse-sfx	4017	3826, 4026, 4028,
pulse-spacing-sfx	4025	3945,
ranged-sync	2806	2799,
read-statement	2027	1772,
read-timer-statement	2272	2138,
real-characteristic-subfield	2517	626, 2497, 2504, 3039, 3047, 3058,
real-complex-errlim	2617	2610,
real-errlim	2580	660, 668, 731, 2153, 2522, 2557, 2618, 2769, 2789, 2847, 2856, 2989, 3007, 3029, 3066, 3068,
real-quantity	2726	668, 845, 1363, 1364, 2625, 2631, 2633,
record-field	3604	237, 2460, 3099,
reference-phase	3761	2676,
remove-sensor	1986	1972,
remove-statement	1966	1768,
req-pin	2878	2869, 2871,
require-analog	394	385,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
require-capability	2777	387,
require-cnx	2865	389,
require-control	2748	386,
require-control-body	2761	2754,
require-limit	2851	388,
require-range	2772	2764, 2822, 2861,
require-statement	376	122,
required-characteristic-capability	2819	2789,
required-characteristic-limit	2860	2856,
required-range	2813	2809,
requirement	3676	378, 624, 2736,
reset-signal	1934	1923,
reset-statement	1920	1791,
reset-timer-statement	2263	2136,
resource-initialization	2379	2373,
resume-for-statement	1648	1238, 2222,
resume-while-statement	1606	1236, 2221,
role-field	3236	2298, 2925,
sense-delay	3003	2194,
sense-event	3017	2194, 2196,
sense-rate	2999	2196,
sense-statement	2055	1775, 2204, 2232,
sensed-data	2062	2059,
sensor-action-body	1881	1878, 1891, 2037,
sensor-digital-characteristics	907	901,
sensor-logic-levels	918	908,
serial-format	815	802,
set-protocol-parameter	3147	1013, 2325, 2351, 2375,
setup-statement	1794	1782,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
sfd	3703	508, 988, 1103, 1133, 1289, 1295, 1306, 1420, 1454, 2152, 2154, 2341, 2445, 2460, 2545, 2558, 2631, 2684, 2737, 2755, 2767, 2830, 2937, 2958, 2989, 3014, 3019, 3028, 3030, 3065, 3069, 3399, 3710, 3741, 3763, 3835, 4040, 4140, 4151,
sign	3298	3167, 3336, 3353, 3365,
signal	3616	290,
signal-based-event-body	657	640,
signal-changes	664	661,
signal-range	2721	2713,
signal-requirement	384	381,
signal-value	2710	2522,
simulate-device	2941	2933, 2936,
single-action-body	1819	1797, 1851, 1865, 1897,
single-action-event	1843	1799, 1853, 1867, 1899, 1925, 1951, 1973,
single-action-verb-statements	1780	1766,
source-digital-characteristics	780	774,
source-logic-levels	791	781,
specify-delay-min-statement	943	746,
standard-protocol-parameter	3216	3199,
statement-characteristics	2472	640, 772, 900, 1826, 1885, 1941, 1962, 1991, 2003, 2012, 2031, 2638,
statement-number	3410	89, 1564, 1602, 1608, 1644, 1650, 1663,
statement-terminator	3725	... ,
statno	3402	3399,
step-number	3418	3404, 3405, 3411,
stim-event	3012	2192,
stim-rate	2979	2192,
stim-sense-rate	2983	2980, 3000,
stimulate-statement	2040	1774, 2203, 2233,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
structured-type	227	205,
symbol-extension-set	3321	3331,
sync-initial	2553	2547,
sync-point	2540	2531,
sync-requirement	2796	2781,
sync-subfield	2528	2483,
terminal-identifier	3461	285, 3483, 3758,
terminate-atlas-module-statement	62	32,
terminate-atlas-program-statement	51	26,
test-equip-bus-role	467	452,
test-equip-role-name	3568	471, 1162, 3239,
test-number	3414	3404, 3411,
text-input	1430	1413,
text-output	1463	1448,
time-dim	4150	945, 2646, 4040,
time-dimension	4154	4151,
time-quantity	2645	446, 641, 689, 717, 731, 806, 2147, 2153, 2168, 2642, 2908, 2917, 2928, 3007, 3009, 3029, 3066, 3068,
time-subfield	3063	3059,
timer	3632	623, 731, 2145, 2152, 2266, 2275, 2916, 3028, 3065,
to-count-into	2464	2459,
type	199	183, 196, 233, 239, 241, 243, 358, 496, 509, 595, 604, 605, 606, 3088,
type-declaration	178	162,
type-identifier	3592	182, 201,
underscore	3294	3491, 3498,
unsigned-decimal-number	3357	852, 853, 945, 2594, 2595, 2596, 2602, 2603, 2604, 2986, 2992, 3354,

18.0 ATLAS FORMAL SYNTAX DEFINITION

RULE NAME	DEF	USAGE
unsigned-integer-number	3369	698, 701, 821, 826, 833, 1678, 1679, 2494, 2495, 2906, 3165, 3167, 3179, 3346, 3736, 3881, 3882, 4019, 4020,
up-low-limit	2630	2623,
update-exchange-statement	2313	2285,
usage-info	1079	1048,
variable-declaration	187	164,
variable-identifier	3596	194, 3092, 3625, 3685,
variable-type	193	190,
verb-info	1107	1033,
verify-statement	2019	1771,
vibration-sfx	4032	3967,
wait-for-manual-intervention-statement	1720	1531,
wait-for-statement	2141	2132,
wait-for-timer-or-event	2150	2145,
when-body	3026	3023,
when-field	3022	2042, 2057, 2080, 2167, 2190, 2297, 2322, 2333, 2429, 2437, 2737, 2894,
while-then-statement	1590	1586, 2217,
while-then-structure	1584	1525,

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.7.2 CROSS-REFERENCE LISTING OF SYMBOLS**

SYMBOL	USAGE
" "	3291, 3303,
" ! "	3323,
" " "	3323,
" # "	3323,
" \$ "	3726,
" % "	3323,
" & "	1308, 3323,
" ' "	762, 832, 889, 1032, 1061, 1090, 1103, 1133, 1142, 1149, 1156, 1162, 1168, 3374, 3380, 3386, 3439, 3458, 3488, 3492, 3506, 3526,
" ("	210, 213, 229, 241, 257, 259, 317, 322, 357, 396, 399, 509, 595, 604, 605, 606, 646, 898, 999, 1318, 1341, 1686, 1691, 1729, 1821, 1882, 1936, 1988, 2000, 2009, 2021, 2028, 2063, 2065, 2108, 2114, 2120, 2121, 2122, 2123, 2274, 2700, 2712, 2886, 2944, 3095, 3103, 3435, 3439, 3526,
") "	212, 215, 233, 241, 257, 259, 319, 324, 357, 398, 400, 509, 595, 604, 605, 606, 646, 898, 1001, 1318, 1343, 1688, 1693, 1735, 1823, 1884, 1938, 1988, 2002, 2009, 2021, 2030, 2070, 2074, 2113, 2119, 2120, 2121, 2122, 2123, 2274, 2700, 2712, 2886, 2946, 3097, 3108, 3435, 3439, 3526,
" * "	1307, 3335, 3463, 3749,
" + "	1301, 1308, 2594, 2596, 2602, 2604, 3299, 3466, 3478, 3754,
" , "	3446, 3451, 3490, 3497, 3717,
" - "	1301, 1308, 2595, 2596, 2603, 2604, 3299, 3466, 3478, 3754, 4020, 4026, 4028,
" -AV"	4002,
" -DENOMINATOR"	3832,
" -IN-PHASE"	3993, 3994,
" -INST"	3996,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"-LMT"	4006,
"-NUMERATOR"	3832,
"-P"	4002, 4019, 4020, 4033,
"-P-NEG"	4003,
"-P-POS"	4003,
"-P-POS-ZERO"	4004,
"-PHASE"	2495, 3881,
"-PHASE-A"	4011,
"-PHASE-AB"	4011,
"-PHASE-AC"	4011,
"-PHASE-B"	4012,
"-PHASE-BA"	4013,
"-PHASE-BC"	4012,
"-PHASE-C"	4013,
"-PHASE-CA"	4013,
"-PHASE-CB"	4012,
"-PP"	4002, 4033,
"-QUAD"	3993, 3995,
"-QUIES"	3833,
"-RATIO"	3827,
"-REF"	3834,
"-TRANS"	3833,
"-TRMS"	4003, 4033,
"-VOLTAGE"	2494, 3882,
"."	2460, 3098, 3335, 3360, 3466, 3478, 3754,
"/"	1307, 3446, 3451, 3465, 3477, 3490, 3497, 3526, 3753,
"0"	871, 872, 876, 877, 2846, 3285,
"1"	871, 872, 876, 877, 3285, 3379, 3385, 3551, 3556,
"2"	3285, 3379, 3385, 3556,
"3"	3285, 3379, 3385, 3556,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"4 "	3285, 3385, 3556,
"5 "	3285, 3557,
"6 "	3286, 3557,
"7 "	3286, 3557,
"8 "	3286,
"9 "	3286,
": "	1476, 1484, 3323,
"; "	174, 183, 190, 239, 3105, 3323,
"< "	3324,
"= "	248, 1260, 1276, 1626, 1733, 3335,
"> "	3324,
"? "	3324,
"@ "	3324,
"A "	2556, 2846, 3265, 3563, 3743, 4061,
"A-SHIFT"	1373,
"AB "	1185,
"ABS "	1374,
"AC "	1185, 3782, 3796, 3797,
"AC-COMP"	3844,
"AC-COMP-FREQ"	3845,
"ACK "	3312,
"AFTER "	689, 700,
"ALL "	1974, 2267,
"ALL-LISTENER"	486, 971,
"ALOG "	1376,
"ALTERNATE-BUS-TRANSMIT"	445,
"AM "	3783,
"AM-COMP"	3846,
"AMBIENT"	3784,
"AMI "	837, 3734,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"AND"	1382, 2585, 2886,
"ANGLE"	3847,
"ANGLE-ACCEL"	3848,
"ANGLE-RATE"	3849,
"APPLY"	1110, 1946,
"ARM"	1116, 1877,
"ARRAY"	229,
"AS"	632, 682, 710, 724, 731,
"ASCII7"	1357,
"AT"	825, 3065, 3069,
"ATAN"	1376,
"ATLAS"	46, 52, 58, 63, 414, 1021,
"ATMOS"	3470, 3482,
"ATTEN"	3850,
"AUTONOMOUS"	487, 972,
"B"	3265, 3423, 3563, 3743,
"B' "	3374,
"B1C"	1352,
"B2C"	1353,
"BANDWIDTH"	3851,
"BAR"	4113,
"BAROMETRIC-PRESS"	3852,
"BCD"	1355,
"BEGIN"	46, 58, 74,
"BEL"	3312,
"BINARY"	1439, 1480,
"BIP"	836, 3734,
"BIT"	219, 3178,
"BIT-PERIOD"	843, 3254,
"BIT-RATE"	843, 3255,
"BIT-TRANSITION"	2548, 2802,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"BITS"	821, 1372, 4067,
"BITS/SEC"	4064,
"BLOCK"	74, 88, 94,
"BNR"	1351,
"BOOLEAN"	224,
"BS"	3312,
"BSM"	1354,
"BURST"	3853,
"BUS"	443,
"BUS-MODE"	482, 967, 1153,
"BUS-PARAMETER"	515, 1139, 3117,
"BUS-PROTOCOL"	429,
"BY"	557, 724, 1633, 2764, 2816, 2822, 2975, 3113,
"C"	3265, 3429, 3563, 3743,
"C' "	3506,
"CALCULATE"	1257,
"CAN"	3315,
"CAP"	3854,
"CAPABILITY"	2778,
"CAR"	3809,
"CAR-AMPL"	3855,
"CAR-FREQ"	3856,
"CAR-HARMONICS"	3857,
"CAR-PHASE"	3858,
"CAR-RESID"	3859,
"CD"	4105,
"CD/M2"	4099,
"CHANGE"	1117, 1895,
"CHAR"	218, 1372, 4067,
"CHAR-CLASS"	224,
"CM"	4094,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"CNX"	437, 774, 901, 1056, 2376, 2396, 2408, 2667, 2870, 3082,
"CNX-DENOMINATOR"	1055, 2666, 2869,
"CNX-NUMERATOR"	1054, 2665, 2868,
"COMMAND"	456, 496, 604, 1009, 2347, 2382, 2451, 2963,
"COMMAND-WORD"	581, 3161, 3220,
"COMMENCE"	100,
"COMMON"	3468, 3480, 3785,
"COMPARE"	1392,
"COMPL"	3744,
"CON-MODE"	488, 973,
"CON-RT"	489, 974,
"CONDITIONS"	3784,
"CONFIGURATION"	751, 1192, 1727, 2363, 2391, 2403, 2415,
"CONNECT"	1114, 1849, 2391,
"CONNECTION"	213, 1328,
"CONNECTIONS"	1038,
"CONSECUTIVE"	826,
"CONSTANT"	169,
"CONTINUOUS"	2765,
"CONTROL"	2749,
"COPY"	1372,
"COS"	1375,
"COUNT"	1370, 3860,
"COUNT-INTO"	2466, 2966,
"COUNTS"	4078,
"CR"	3313,
"CREATE"	1508,
"CURRENT"	2556, 2846, 3861,
"CURRENT-LMT"	3862,
"CURRENT-ONE"	2560, 2841,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"CURRENT-P-NEG"	2568, 2835,
"CURRENT-P-POS"	2567, 2834,
"CURRENT-QUIES"	2564, 2839,
"CURRENT-ZERO"	2562, 2840,
"CYCLES"	4055,
"D"	3265, 3564, 4125, 4158,
"DATA"	458, 496, 605, 1010, 2348, 2383, 2452, 2963,
"DATA-WORD"	583, 3163, 3222,
"DATE"	1371,
"DB"	4116,
"DBJ"	4074,
"DBKW"	4109,
"DBM"	4108,
"DBMJ"	4074,
"DBPJ"	4073,
"DBW"	4109,
"DC"	3786, 3798, 3799,
"DC-OFFSET"	3863,
"DC1"	3314,
"DC2"	3314,
"DC3"	3314,
"DC4"	3314,
"DEC"	1373,
"DECIMAL"	216,
"DECLARE"	159,
"DECREASING"	665,
"DEFINE"	278, 290, 309, 354, 750, 756, 883, 959, 1109, 1190,
"DEG"	4051,
"DEGC"	4122,
"DEGF"	4122,
"DEGK"	4122,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"DEGR"	4122,
"DEL"	3316,
"DELAY"	446, 2928, 3864,
"DELAY-MIN"	944,
"DELETE"	1377, 1514,
"DIG"	1373,
"DIG-CLASS"	224,
"DIGITAL"	751, 757, 884, 1727, 2189,
"DIGITS"	4067,
"DIM-SPEC"	1104,
"DISABLE"	1503, 1716, 1756, 1915, 2415,
"DISCONNECT"	1115, 1863, 2403,
"DISS-FACTOR"	3865,
"DISTORTION"	3866,
"DIV"	1307,
"DLE"	3313,
"DO"	1121, 2166, 2178, 2189, 2255, 2295,
"DOPPLER"	3787, 3800,
"DOPPLER-BANDWIDTH-POWER"	3867,
"DOPPLER-FREQ"	3868,
"DOPPLER-SHIFT"	3869,
"DRAWING"	279,
"DROOP"	3870,
"DUTY-CYCLE"	3871,
"E"	3265, 3364, 3398, 3564,
"E-FORMAT"	1479,
"EARTH"	3469, 3481, 3788,
"ELSE"	1569, 1580,
"EM"	3315,
"ENABLE"	1488, 1711, 1726, 1741, 1909, 2363,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"END"	94, 343, 372, 954, 1574, 1613, 1655, 2178,
"ENQ"	3312,
"ENTIRE"	1017, 2356,
"ENUMERATION"	210,
"EOF"	1369,
"EOT"	3311,
"EQ"	658, 731, 1387, 2624, 2916,
"ERRLMT"	2581,
"ESC"	3315,
"ESCAPE"	1741, 1756,
"ESTABLISH"	427,
"ETB"	3315,
"ETX"	3311,
"EV"	4075,
"EVENT"	399, 632, 682, 709, 723, 730, 846, 1844, 1909, 1915, 2154, 2532, 2908, 2918, 3030, 3069,
"EVENTS"	3789,
"EVERY"	697, 825, 2907,
"EXCEPT"	825, 3081,
"EXCHANGE"	549, 961, 1192, 2295, 2331, 2363, 2391, 2403, 2415, 2435, 2898, 2919,
"EXCHANGE-PIN"	1165,
"EXP"	1376,
"EXTEND"	1021,
"EXTERNAL"	188, 310, 355, 379, 428, 622, 631, 681, 708, 722, 730, 960, 1191,
"F"	3265, 3564,
"FALL-TIME"	3872,
"FALSE"	1277, 1349,
"FAULT-COUNT"	3199, 3202,
"FAULT-TEST"	441,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"FD"	4058,
"FETCH"	1116, 1889, 2421,
"FETCHABLE"	543,
"FF"	3313,
"FILE"	242,
"FINISH"	1700,
"FM"	3790,
"FM-COMP"	3873,
"FM-INDEX"	3874,
"FOR"	717, 1032, 1038, 1043, 1118, 1625, 1643, 1649, 1655, 1721, 2142,
"FREQ"	3875,
"FREQ-DEV"	3876,
"FREQ-DEV-RATE"	3877,
"FREQ-LMT"	3878,
"FREQ-WINDOW"	3879,
"FROM"	716, 945, 1419, 1432, 1455, 1466, 1492, 1836, 2357, 2974,
"FS"	3315,
"FT"	4095,
"FT/SEC"	4129,
"FTSW"	4113,
"FUNCTION"	356,
"G"	3266,
"GATE"	2779,
"GATED"	2972,
"GE"	1387, 2624,
"GEV"	4075,
"GHZ"	4081, 4145,
"GJ"	4074,
"GLOBAL"	188, 310, 355, 379, 428, 622, 631, 681, 708, 722, 730, 960, 1191,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"GO"	1271, 1661, 2658,
"GO-NOGO"	1413,
"GRAV"	4048,
"GS"	3316,
"GT"	1387, 2624,
"GUARD"	3745,
"GW"	4109,
"H"	3266, 4091,
"HARM- "	2494, 2495, 3881, 3882,
"HARMONICS"	3880,
"HDB"	833, 3735,
"HEXADECIMAL"	1438, 1478,
"HI "	1272, 2656, 3742,
"HIZ"	2046, 2072, 2084, 2121, 2123,
"HOLD"	2308,
"HP"	4109,
"HR"	4125, 4156,
"HT"	3312,
"HYSTERESIS"	667,
"HZ"	4081, 4145,
"I"	1184, 1574, 3266,
"I-O"	1490,
"IA"	1184,
"IDENT-SIG"	3883,
"IDENT-SIG-FREQ"	3884,
"IDENT-SIG-MOD"	3885,
"IDENTIFY"	621, 630, 680, 707, 721, 729, 1109,
"IF"	1553, 1563, 1580, 1743,
"ILLEGAL"	782, 909,
"ILLUM"	3886,
"IMP"	3887,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"IMP-ANGLE"	3888,
"IMP-MAGNITUDE"	3889,
"IMPEDANCE"	3791,
"IN"	4096,
"IN-WATER"	4113,
"INCLUDE"	413,
"INCREASING"	665,
"IND"	3890,
"INDICATOR"	723, 782, 909,
"INHG"	4113,
"INITIAL"	248, 2554, 2829,
"INITIALIZE"	496, 2373,
"INITIATE"	1113, 2036,
"INPUT"	1412, 1492,
"INSERT"	1377,
"INSERT-INVALID"	3151,
"INSERTION-LOSS"	3891,
"INT"	1374,
"INTEGER"	217,
"INTERVAL"	623, 709,
"INTERVENTION"	1711, 1716, 1721, 2653,
"INTO"	1421, 1433, 2074, 2113, 2120, 2121, 2274, 2612,
"IS"	172, 182, 195, 238, 3088,
"J"	3266, 4073,
"K"	3266,
"KA"	4061,
"KBITS/SEC"	4064,
"KEV"	4075,
"KHZ"	4081, 4145,
"KJ"	4073,
"KM"	4094,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"KM/HR"	4130,
"KOHM"	4088,
"KPA"	4112,
"KPPS"	4082, 4146,
"KSEC"	4126, 4158,
"KT"	4129,
"KV"	4133,
"KW"	4108,
"KWORDS/SEC"	4070,
"L"	3266,
"LATER"	3068,
"LE"	1387, 2624,
"LEAVE"	88, 338, 367, 1563, 1601, 1643,
"LEN"	1370,
"LENGTH"	588, 3181, 3227,
"LEVEL-LOGIC-HIZ"	861, 937,
"LEVEL-LOGIC-ONE"	858, 931,
"LEVEL-LOGIC-QUIES"	861, 936,
"LEVEL-LOGIC-ZERO"	858, 930,
"LF"	3312,
"LIGHT"	3792,
"LIMIT"	2852,
"LISTENER"	494, 987, 2340, 2381, 2444, 2935,
"LL"	1364, 2633,
"LM"	4102,
"LN"	1375,
"LO"	1273, 2657, 3742,
"LOAD"	402,
"LOCN"	1370,
"LOG"	1375,
"LOGIC"	3793,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"LT"	1387, 2624,
"LUM-FLUX"	3892,
"LUM-INT"	3893,
"LUMINANCE"	3894,
"LX"	4085,
"M"	3267, 4094,
"M/HR"	4130,
"M/S2"	4047,
"M/SEC"	4129,
"MA"	4061,
"MACH"	4129,
"MAIN"	100,
"MANUAL"	1711, 1716, 1721, 2653,
"MANUAL-INTERVENTION"	691, 1269, 1751,
"MASK-ONE"	649, 2098,
"MASK-ZERO"	650, 2098,
"MASTER"	473, 3241,
"MAX"	558, 806, 2523, 2823, 2862, 2990,
"MAX-TIME"	1270, 1750, 2642, 2654,
"MBAR"	4112,
"MBITS/SEC"	4064,
"MD"	1185,
"MEASURE"	1111, 2000,
"MESSAGE-GAP"	531, 3130,
"MEV"	4075,
"MFD"	4058,
"MH"	4091,
"MHZ"	4081, 4145,
"MIL"	4051,
"MILS"	4096,
"MIN"	558, 2523, 2823, 2862, 3008, 4125, 4156,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"MIN-1"	4119,
"MINIMUM-SENSE-RATE"	674,
"MINUTE"	4052,
"MIP"	3734,
"MJ"	4073,
"MM"	4094,
"MMHG"	4112,
"MNEMONIC-ONLY"	1067, 1178,
"MO"	1185,
"MOD"	1308,
"MOD-AMPL"	3895,
"MOD-ARRAY"	1074,
"MOD-DESC-SPEC"	1134,
"MOD-DIM"	1094,
"MOD-DIST"	3896,
"MOD-FREQ"	3897,
"MOD-INDEX"	3898,
"MOD-OFFSET"	3899,
"MOD-PHASE"	3900,
"MOD-VALUE"	1074, 1176,
"MODIFIER-DESCRIPTOR"	1064, 1177,
"MODIFIERS"	1043,
"MODULE"	58, 63, 414,
"MOHM"	4088,
"MONITOR"	399, 472, 1112, 2008, 3240,
"MPA"	4112,
"MRAD"	4052,
"MSEC"	4125, 4157,
"MV"	4133,
"MW"	4108,
"MWORDS/SEC"	4070,
"N"	3267, 3743,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"N-MILE"	4095,
"NA"	4061,
"NAK"	3314,
"NE"	1387, 2624,
"NEG"	4021, 4027, 4029,
"NEG-EDGE"	2573, 3901,
"NEG-SLOPE"	2577, 3902,
"NEW"	1491,
"NFD"	4058,
"NH"	4091,
"NJ"	4075,
"NM"	4095,
"NOGO"	1274, 2659,
"NOISE"	3803, 3903,
"NOISE-AMPL-DENS"	3904,
"NOISE-PWR-DENS"	3905,
"NOM"	1363, 2631,
"NON-ATLAS"	414,
"NON-HARMONICS"	3906,
"NON-LIN"	3907,
"NONE"	1098,
"NOT"	1301, 2651,
"NOUNS"	1032,
"NRZ"	835, 3734,
"NSEC"	4126, 4157,
"NT"	4099,
"NUL"	3311,
"NW"	4108,
"O"	2556, 3267, 3378, 3551, 3556,
"OCCURRENCES"	698, 701, 826,
"OCTAL"	1440, 1481,
"ODD"	1369,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"OF"	233, 234, 241, 242, 254, 698, 701, 1837,
"OHM"	4088,
"OLD"	1491,
"ON"	3074,
"ONE"	2046, 2084,
"ONE-AMPLITUDE"	529, 3127,
"OR"	873, 1382, 2585, 2886,
"ORD"	1370,
"OSCILLATORY"	3908,
"OUTPUT"	1447, 1490,
"OVERSHOOT"	3909,
"P"	3267,
"P-AMPL"	3910,
"P-NEG-ZERO"	4004,
"PA"	4112,
"PAM"	3794,
"PARITY"	587, 3182, 3228,
"PC"	2602, 2603, 2604, 4116,
"PE"	1369,
"PEAK-DEGEN"	3911,
"PERFORM"	1667,
"PERIOD"	3912,
"PFD"	4058,
"PH"	4091,
"PHASE-A"	3764,
"PHASE-AB"	3764,
"PHASE-AC"	3764,
"PHASE-ANGLE"	3913,
"PHASE-B"	3765,
"PHASE-BA"	3766,
"PHASE-BC"	3765,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"PHASE-C"	3766,
"PHASE-CA"	3766,
"PHASE-CB"	3765,
"PHASE-DEV"	3914,
"PHASE-JIT"	3915,
"PHASE-SHIFT"	3916,
"PJ"	4073,
"PM"	3795,
"PO"	1369,
"POS"	4021, 4027, 4029,
"POS-EDGE"	2573, 3917,
"POS-SLOPE"	2577, 3918,
"POWER"	3919,
"PPS"	4082, 4146,
"PRED"	1371,
"PRESHOOT"	3920,
"PRF"	3921,
"PRIMARY"	442,
"PRIORITY"	2905,
"PROCEDURE"	100, 311, 1742,
"PROCEED"	2307,
"PROGRAM"	46, 52,
"PROTOCOL"	548, 961, 2320, 2366, 2395, 2407, 2427,
"PROTOCOL-EXCHANGE"	547,
"PROTOCOL-PARAMETER"	568, 1146, 3148, 3188,
"PROVE"	1120, 2079,
"PSEC"	4126, 4158,
"PSI"	4113,
"PULSE-CLASS"	830, 3252,
"PULSE-CODE"	586, 3179, 3225,
"PULSE-ENERGY"	3922,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"PULSE-WIDTH"	3923,
"PULSED"	3796, 3797, 3798, 3799, 3800,
"PULSES"	4055,
"PWR-LMT"	3924,
"Q"	3267,
"QUAD"	3925,
"QUIES"	876,
"R"	1080, 1184, 3267,
"R1"	3746,
"R2"	3746,
"R3"	3746,
"R4"	3746,
"RA"	1184,
"RAD"	4051,
"RADAR"	3801,
"RADIAL"	3926,
"RADIAL-RATE"	3927,
"RAMP"	3802,
"RANDOM"	3803,
"RANGE"	555, 852, 2519, 2722, 2773, 2814,
"REACTANCE"	3928,
"READ"	1113, 2028, 2273,
"RECORD"	234,
"REDUNDANT"	442,
"REF"	646, 1085, 2095, 2675, 2880,
"REF-INERTIAL"	3929,
"REF-PHASE-FREQ"	3930,
"REF-UUT"	3931,
"REF-VOLT"	3932,
"RELATIVE-HUMIDITY"	3933,
"REMOVE"	1111, 1967,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"REPLY-EFF"	3934,
"REQUIRE"	377, 1110,
"RES"	3935,
"RESET"	1117, 1921, 2145, 2264,
"RESOLVER"	3804,
"RESP"	3936,
"RESPONSE-TIME"	527, 3129,
"RESULT"	322, 1691,
"RESUME"	1607, 1649,
"RETURN-LOSS"	3937,
"REV"	4051,
"RINGING"	3938,
"RISE-TIME"	3939,
"RM"	1080,
"ROTATE"	1372,
"ROTATION"	3805,
"ROUND"	1374,
"ROUNDING"	3940,
"RS"	3316,
"RT-CON"	490, 975,
"RT-RT"	491, 976,
"RZ"	834, 3734,
"S"	1080, 3268,
"S-MILE"	4096,
"S1"	3747,
"S2"	3747,
"S3"	3747,
"S4"	3747,
"SAMPLE"	3941,
"SAMPLE-SPACING"	3942,
"SAMPLE-WIDTH"	3943,
"SAVE-COMP"	2101,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"SB"	1184,
"SBCD"	1356,
"SC"	1184,
"SCORSBY"	3944,
"SCREEN"	3745,
"SEC"	4125, 4156,
"SEC-1"	4119,
"SECOND"	4052,
"SENSE"	1120, 2056,
"SENSE-DELAY"	3004,
"SENSE-EVENT"	675, 3018,
"SENSE-RATE"	3000,
"SENSOR"	396, 884,
"SEQUENTIAL"	2255,
"SERIAL-LSB-FIRST"	818,
"SERIAL-MSB-FIRST"	817,
"SET"	724,
"SETUP"	1114, 1795,
"SHIFT"	1373,
"SHORT"	3806,
"SI"	3313,
"SIGNAL"	291, 3782, 3783, 3786, 3790, 3793, 3795, 3801, 3802, 3808, 3809, 3812,
"SIMULTANEOUS"	2166,
"SIN"	1374,
"SIZE"	1371,
"SKEW-TIME"	806, 3253,
"SLAVE"	474, 3242,
"SO"	3313,
"SOH"	3311,
"SOURCE"	401, 757,
"SPACING"	3945,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"SPEC"	430, 759, 832, 886, 1034, 1039, 1050, 3256,
"SPECIAL-SYNTAX"	1066,
"SPECIFY"	944,
"SQRT"	1376,
"SQUARE"	3807,
"SR"	1080,
"SRM"	1080,
"SS"	1185,
"STANDARD"	441, 2326,
"START"	2305,
"STARTING"	2893,
"STATE"	782, 909,
"STATUS"	457, 606, 1011, 1509, 1515, 2349, 2453, 2963,
"STATUS-WORD"	582, 3162, 3221,
"STEP"	89, 1564, 1602, 1608, 1644, 1650, 1662, 3808,
"STEPS"	4078,
"STIM"	3946,
"STIM-EVENT"	3013,
"STIM-RATE"	2980,
"STIMULATE"	1119, 2041,
"STRING"	241,
"STX"	3311,
"SUB"	3315,
"SUB-CAR-FREQ"	3947,
"SUB-CAR-MOD"	3948,
"SUCC"	1371,
"SUP"	3809,
"SWEEP-INT"	3949,
"SWR"	3950,
"SYN"	3314,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"SYNC"	589, 2530, 2797, 3180, 3226,
"SYNC-BIT-TRANSITION"	846,
"SYNCHRO"	3810,
"T"	3268,
"T-R"	590, 3183, 3229,
"TABLE"	1678,
"TALKER"	493, 985, 2338, 2380, 2443, 2933,
"TALKER-LISTENER"	485, 970,
"TAN"	1375,
"TARGET-RANGE"	3951,
"TARGET-RANGE-ACCEL"	3952,
"TARGET-RANGE-RATE"	3953,
"TEMP"	3954,
"TERMINALS"	285,
"TERMINATE"	52, 63,
"TEST-EQUIP"	505, 995,
"TEST-EQUIP- "	3985,
"TEST-EQUIP-DELAY"	641,
"TEST-EQUIP-MONITOR"	453, 2954,
"TEST-EQUIP-ROLE"	468, 1159, 3237,
"TEST-EQUIP-SIMULATE"	2932,
"TEXT"	243, 1431, 1464,
"THEN"	1554, 1580, 1592, 1626, 2145, 2887,
"THREE-PHASE-DELTA"	3955,
"THREE-PHASE-WYE"	3956,
"THRU"	231, 1632, 3112,
"THRU- "	3986,
"THZ"	4081, 4145,
"TIME"	623, 625, 2146, 2274, 3957,
"TIME-ASYM"	3958,
"TIME-INTERVAL"	3811,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"TIME-JIT"	3959,
"TIMED"	2189,
"TIMER"	403, 623, 731, 2152, 2264, 2275, 2916, 3028, 3065,
"TIMES"	1679, 4078,
"TJ"	4074,
"TO"	556, 717, 846, 853, 947, 948, 1453, 1465, 1490, 1661, 1742, 1836, 2465, 2520, 2532, 2723, 2774, 2815, 2974,
"TRAIN"	3797, 3799,
"TRANS-LOGIC-ONE"	868,
"TRANS-LOGIC-ZERO"	868,
"TRANS-SYNC"	875,
"TRANS-SYNC-BIT"	878,
"TRANS-TIME"	3961,
"TRANSMITTANCE"	3960,
"TRIANGULAR"	3812,
"TRIG"	3962,
"TRUE"	1277, 1350, 3744,
"TW"	4109,
"TYPE"	179,
"U"	3268,
"UA"	4061,
"UBAR"	4113,
"UFD"	4058,
"UH"	4091,
"UJ"	4074,
"UL"	1364, 2632,
"UM"	4095,
"UNDERSHOOT"	3963,
"UNTIL"	2914,
"UPA"	4112,
"UPDATABLE"	544,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"UPDATABLE-FETCHABLE"	542,
"UPDATE"	2314,
"URAD"	4052,
"US"	3316,
"USAGE"	1047,
"USEC"	4126, 4157,
"USING"	624, 2295, 2320, 2331, 2364, 2392, 2404, 2416, 2427, 2435, 2736,
"UUT"	506, 996, 2942,
"UUT- "	3984,
"UV"	4133,
"UW"	4108,
"V"	2556, 2846, 3268, 4133,
"VALUE"	646, 2065, 2071, 2108, 2114, 2120, 2122,
"VAR"	1678,
"VAR-PHASE-FREQ"	3964,
"VAR-PHASE-MOD"	3965,
"VARIABLE"	189,
"VERBS"	1032,
"VERIFY"	1112, 2020,
"VIA"	1494, 3742,
"VIBRATION"	3813,
"VIBRATION-ACCEL"	3966,
"VIBRATION-AMPL"	3967,
"VIBRATION-RATE"	3968,
"VOLT-LMT"	3972,
"VOLTAGE"	2556, 2846, 3969,
"VOLTAGE-ONE"	2559, 2838,
"VOLTAGE-P-NEG"	2566, 2833,
"VOLTAGE-P-POS"	2565, 2832,
"VOLTAGE-QUIES"	2563, 2836,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"VOLTAGE-RAMPED"	3970,
"VOLTAGE-STEPPED"	3971,
"VOLTAGE-ZERO"	2561, 2837,
"VOR"	3814,
"VT"	3313,
"W"	3268, 4108,
"WAIT"	1118, 1721, 2142, 2307,
"WAVE"	3807, 3812,
"WAVE-LENGTH"	3973,
"WAVEFORM"	3815,
"WHEN"	3023,
"WHILE"	1591, 1601, 1607, 1613,
"WITH"	2905,
"WITHIN"	2168,
"WORD-COUNT"	596, 3166, 3232,
"WORD-GAP"	533, 3132,
"WORD-LENGTH"	532, 820, 3131, 3251,
"WORD-TRANSITION"	2549, 2801,
"WORDS/SEC"	4070,
"X"	3268, 3384, 3748,
"XOR"	1382,
"Y"	3269, 3748,
"Z"	3269, 3748,
"ZERO"	2046, 2084,
"ZERO-AMPLITUDE"	528, 3126,
"ZERO-CROSSING"	530, 3128,
"ZERO-INDEX"	3974,
" ["	234, 3324,
" \"	3309, 3318, 3325,
"] "	240, 3324,
" ^ "	3325,
" _ "	3295, 3325,

18.0 ATLAS FORMAL SYNTAX DEFINITION

SYMBOL	USAGE
"`"	3325,
"a"	3275,
"b"	3275,
"c"	3275,
"d"	3275,
"e"	3275,
"f"	3275,
"g"	3276,
"h"	3276,
"i"	3276,
"j"	3276,
"k"	3276,
"l"	3276,
"m"	3277,
"n"	3277,
"o"	3277,
"p"	3277,
"q"	3277,
"r"	3277,
"s"	3278,
"t"	3278,
"u"	3278,
"v"	3278,
"w"	3278,
"x"	3278,
"y"	3279,
"z"	3279,
"{"	3325,
" "	3325,
"}"	3325,
"~"	3324,

18.0 ATLAS FORMAL SYNTAX DEFINITION**18.7.3 CROSS-REFERENCE LISTING OF LABELS**

LABEL	DEF	USAGE
block	75	75, 95,
bus-mode-name	1156	484, 969, 1156,
bus-parameter-name	1142	526, 1142, 3125,
configuration	750	750, 954, 1727, 3078,
constant-identifier	172	172, 1333,
device-identifier-name	504	504, 994, 2942,
digital-sensor	883	883, 947, 948, 3077,
digital-source	756	756, 947, 948, 3076,
dim-name	1103	1103, 4136,
drawing	278	278, 1949, 1971, 2013,
enumeration-element	211	211, 1332, 3673,
event	632	632, 689, 690, 698, 702, 716, 717, 724, 847, 1836, 1910, 1916, 2154, 2532, 2908, 2918, 2974, 3013, 3018, 3030, 3069,
event-indicator	723	723, 1749,
event-interval	710	710, 1837, 2975,
exchange	959	959, 2331, 2435, 2919, 2924,
exchange-configuration	1190	1190, 2296, 2321, 2332, 2364, 2392, 2404, 2416, 2428, 2436,
function	354	354, 367, 372, 1339, 2065, 2108, 2114,
modifier-descriptor-name	1133	1133, 3773,
modifier-name	1061	1061, 3975,
module-name	59	59, 64, 416,
noun-name	1032	1032, 2741,
parameter	3087	3087,
pin-descriptor-name	1090	1090, 3756,
procedure	309	309, 338, 343, 1667, 1742,
program-name	47	47, 53,

18.0 ATLAS FORMAL SYNTAX DEFINITION

LABEL	DEF	USAGE
protocol	429	429, 445, 961, 1193, 2320, 2366, 2395, 2407, 2427,
protocol-parameter-name	1149	594, 1149, 3158, 3200,
record-field	237	237, 2460, 3099,
requirement	378	378, 624, 2736,
signal	290	290,
terminal-identifier	285	285, 3483, 3758,
test-equip-role-name	1162	471, 1162, 3239,
timer	623	623, 731, 2145, 2152, 2266, 2275, 2916, 3028, 3065,
type-identifier	182	182, 201,
variable-identifier	194	194, 3092, 3625, 3685,

APPENDIX A1
COORDINATED AND RECOMMENDED USE OF EXTEND

A1 Recommended Use of the EXTEND Statement

The noun FLUID SIGNAL is defined in IEEE Std.716 to which reference should be made, in the first instance, for ATLAS material which is not defined in ARINC Specification 626-3. The following are examples of the appropriate <extend statement>s that can be used introduce the noun FLUID SIGNAL into an ATLAS Test Requirement.

FLUID SIGNAL

fstatno EXTEND, ATLAS NOUNS 'FLUID SIGNAL', FOR VERBS SETUP CONNECT ARM FETCH CHANGE
 RESET APPLY REMOVE MEASURE MONITOR VERIFY READ INITIATE DEFINE REQUIRE
 IDENTIFY, SPEC 'IEEE Std.716 Section 16.15 and Section 17' \$

fstatno EXTEND, ATLAS CONNECTIONS FOR FLUID SIGNAL, CNX TO 'FROM' 'STATIC-TO' 'STATIC-FROM'
 'PITOT-TO' PITOT-FROM' \$

fstatno EXTEND, ATLAS MODIFIERS FOR FLUID SIGNAL,

'ALT'	MOD-VALUE	R	MOD-DIM	FT,	USAGE SRM,
'ALT-RATE'	MOD-VALUE	R	MOD-DIM	FT/SEC,	USAGE SRM,
'DEWPOINT'	MOD-VALUE	R	MOD-DIM	DEGC,	USAGE SR,
'DIFF-PRESS-A'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'DIFF-PRESS-G'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'DIFF-TEMP'	MOD-VALUE	R	MOD-DIM	DEGC,	USAGE SRM,
'IAS'	MOD-VALUE	R	MOD-DIM	KT,	USAGE SRM,
'IAS-RATE'	MOD-VALUE	R	MOD-DIM	FT/SEC/SEC,	USAGE S,
'MACH'	MOD-VALUE	R	MOD-DIM	MACH,	USAGE SRM,
'MOD-AMPL-PP'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE S,
'MOD-FREQ'	MOD-VALUE	R	MOD-DIM	HZ,	USAGE S,
'PITOT-VOLUME'	MOD-VALUE	R	MOD-DIM	CM3,	USAGE S,
'PRESS-A'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'PRESS-G'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'PRESS-RATE'	MOD-VALUE	R	MOD-DIM	MBAR/SEC,	USAGE S,
'SPEC-GRAV'	MOD-VALUE	R	MOD-DIM	NONE,	USAGE SRM,
'STATIC-PRESS-A'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'STATIC-PRESS-G'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'STATIC-PRESS-RATE'	MOD-VALUE	R	MOD-DIM	MBAR/SEC,	USAGE SRM,
'STATIC-TEMP'	MOD-VALUE	R	MOD-DIM	DEGC,	USAGE SRM,
'STATIC-VOLUME'	MOD-VALUE	R	MOD-DIM	CM3,	USAGE S,
'TAS'	MOD-VALUE	R	MOD-DIM	KT,	USAGE SRM,
'TAS-RATE'	MOD-VALUE	R	MOD-DIM	FT/SEC/SEC,	USAGE S,
'TEMP'	MOD-VALUE	R	MOD-DIM	DEGC,	USAGE SRM,
'TOTAL-PRESS-A'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'TOTAL-PRESS-G'	MOD-VALUE	R	MOD-DIM	MBAR,	USAGE SRM,
'TOTAL-PRESS-RATE'	MOD-VALUE	R	MOD-DIM	MBAR/SEC,	USAGE SRM,
'TOTAL-TEMP'	MOD-VALUE	R	MOD-DIM	DEGC,	USAGE SRM,
'TRANS-TIME'	MOD-VALUE	R	MOD-DIM	SEC,	USAGE S \$

fstatno EXTEND, ATLAS MODIFIERS FOR FLUID SIGNAL,

'ATMOS-ARDC'	MNEMONIC-ONLY, SPEC 'ARDC Standard atmosphere' \$	USAGE SR,
--------------	--	-----------

fstatno EXTEND, ATLAS MODIFIERS FOR FLUID SIGNAL,

'ATMOS-ICAN'	MNEMONIC-ONLY, SPEC 'ICAN Standard atmosphere' \$	USAGE SR,
--------------	--	-----------

APPENDIX A1
COORDINATED AND RECOMMENDED USE OF EXTEND

fstatno EXTEND, ATLAS MODIFIERS FOR FLUID SIGNAL,

'ATMOS-WADC'	MNEMONIC-ONLY, SPEC 'WADC Standard atmosphere' \$	USAGE SR,
--------------	--	-----------

fstatno EXTEND, ATLAS MODIFIERS FOR FLUID SIGNAL,

'FLUID-TYPE-AIR'	MNEMONIC-ONLY, SPEC 'Air Quality Standard' \$	USAGE SR,
------------------	--	-----------

'FLUID-TYPE-N2'	MNEMONIC-ONLY, SPEC 'Nitrogen Gas Quality Standard' \$	USAGE SR,
-----------------	---	-----------

The nouns MODULATED SIGNAL and DEMODULATED SIGNAL are not defined in IEEE Std.716. Therefore for completeness and for human interpretation, another standard has to be specified which preferably should be an international one. Failing this, a national standard should be specified. Failing this, a local standard must be specified. The following are examples of the appropriate <extend statement>s that can be used introduce the nouns MODULATED SIGNAL and DEMODULATED SIGNAL into an ATLAS Test Requirement.

MODULATED SIGNAL

fstatno EXTEND ATLAS MODULATED SIGNAL, FOR VERBS SETUP CONNECT DISCONNECT CHANGE RESET
 APPLY REMOVE, SPEC 'In-house Standard' \$

fstatno EXTEND ATLAS CONNECTIONS FOR MODULATED SIGNAL, CNX HI LO REF 'CAR-HI' 'CAR-LO'
 'MOD-HI' 'MOD-LO' \$

fstatno EXTEND, ATLAS MODIFIERS FOR MODULATED SIGNAL,

'AM'	MNEMONIC-ONLY, SPEC 'In-house Standard' \$	USAGE S,				
'FM'	MNEMONIC-ONLY, SPEC 'In-house Standard' \$	USAGE S,				
'PM'	MNEMONIC-ONLY, SPEC 'In-house Standard' \$	USAGE S,				
'PULSE-AM'	MNEMONIC-ONLY, SPEC 'In-house Standard' \$	USAGE S,				
'STD-AM-DEPTH'	MOD-VALUE,	R	MOD-DIM	HZ/HZ	USAGE S,	
'STD-FM-DEPTH'	MOD-VALUE,	R	MOD-DIM	HZ/HZ	USAGE S	\$

DEMODULATED SIGNAL

fstatno EXTEND ATLAS DEMODULATED SIGNAL, FOR VERBS SETUP CONNECT DISCONNECT CHANGE
 RESET APPLY REMOVE MEASURE MONITOR VERIFY READ INITIATE,
 SPEC 'In-house Standard' \$

fstatno EXTEND ATLAS CONNECTIONS FOR MODULATED SIGNAL, CNX HI LO REF 'CAR-HI' 'CAR-LO'
 'MOD-HI' 'MOD-LO' \$

fstatno EXTEND, ATLAS MODIFIERS FOR MODULATED SIGNAL,

'AM'	MNEMONIC-ONLY, SPEC 'In-house Standard' \$	USAGE SR,				
'CAR-AMPL'	MOD-VALUE,	R	MOD-DIM	MV	USAGE SR,	
'CAR-FREQ'	MOD-VALUE,	R	MOD-DIM	HZ	USAGE SR,	
'FM'	MNEMONIC-ONLY, SPEC 'In-house Standard' \$	USAGE SR,				
'MOD-INDEX'	MOD-VALUE,	R	MOD-DIM	NONE	USAGE SR,	

APPENDIX A1**COORDINATED AND RECOMMENDED USE OF EXTEND**

'PM'	MNEMONIC-ONLY,				USAGE SR,
	SPEC 'In-house Standard' \$				
'POWER'	MOD-VALUE,	R	MOD-DIM	MW	USAGE SR,
'PULSE-AM'	MNEMONIC-ONLY,				USAGE S,
	SPEC 'In-house Standard' \$				

Copyright© 1995 by
AERONAUTICAL RADIO, INC
2551 Riva Road
Annapolis, Maryland 21401

SUPPLEMENT 1
TO
ARINC SPECIFICATION 626
STANDARD ATLAS LANGUAGE FOR MODULAR TEST
Published: April 4, 1989

Prepared by the Airlines Electronic Engineering Committee
Adopted by the Airlines Electronic Engineering Committee: October 19, 1988

A. PURPOSE OF THIS SUPPLEMENT

This supplement incorporates changes to ARINC Specification 626 necessary for the development of the ATLAS compiler for the SMART system. It provides for the use of block structures, clarifies known ambiguities in the language and corrects many editorial errors contained in IEEE Std-416, of which ARINC Specification 626 ATLAS is a subset.

B. ORGANIZATION OF THIS SUPPLEMENT

The supplement, printed on buff-colored paper, contains descriptions of the changes introduced by Supplement 1 to ARINC Specification 626. All the changes are incorporated in the body of the document.

C. CHANGES TO ARINC REPORT 615 INTRODUCED BY THIS SUPPLEMENT

This section presents a summary of the changes and additions to ARINC Specification 626 introduced by this supplement. In each case a brief description of the change or addition is included.

Section 1 – Changes to Section 3 include clarification of the use of NON-ATLAS module structures within an ATLAS test specification structure.

Section 4 – A subset of the IEEE Std-416 block definition capability is added to facilitate the segmentation of test programs.

Section 6 – Changes to Section 6 include the modification of the local preamble structures to recognize the addition of block structures to ARINC Specification 625. Other changes apply to the declare statement and to the rules for constant, type and variable forms of the declare statement. Changes also eliminate the ability to identify an event, which is known to have caused implementation difficulties in IEEE Std-416. The rules applying to the use of the local preamble structure are clarified for the addition of block structures. This section also modifies the define drawing subset recommended for ARINC Specification 626.

Section 7 – The definition of main procedures structure is modified to recognize the addition of block structures to ARINC Specification 626.

Section 8 – This section incorporates minor editorial changes and the addition of rules to explicitly state that an expression must be of the appropriate type for the purpose for which it is employed.

Section 9 – Revisions to this section concern the input and output statements for the purpose of adding keywords in branches which were poorly identified. The change also deletes references to virtual resources, includes rules for optional max-time field and permits inputs into a range of an array. Other changes eliminate ambiguities in the language.

Section 10 – Changes to this section include minor editorial changes to eliminate ambiguities when applying the leave while and resume while statement. Other changes concern the enable escape and the disable escape statement. Rules have been added and/or modified to clarify the use of these statements.

Section 11 – Rules concerning the use of multiple action verbs have added to this section. The do timed digital statement, enable event statement, apply statement, and remove statement have been modified. The capability to disable events is added to the measure statement to make it consistent with other signal oriented statements. Other miscellaneous changes were made to eliminate ambiguity in the rules documented in this section.

Section 12 – In section 12, changes concern the procedural statements for timing, the delay statement, the wait for event statement, the do simultaneous statement, and to timed digital body. A new read timer statement is added to the language. A variation of the read verb is added to permit an identified timer to be read.

Section 14 – Several editorial changes are incorporated to eliminate ambiguous use of data store, variable, variable identifier, and related time constructs. Rules were added and/or modified to eliminate ambiguity in the real quantity, digital quantity, conn set and require capability.

Section 15 – Section 15.2 is rewritten to provide a more precise definition of the various number types. Rules for the comment statement and the flag statement in Section 15.3.2 are added to recognize the incorporation of block structures in ARINC Specification 626.

Section 16 – A new rule is added for use with the noun ATMAS.

Section 17 – New voltage and current suffixes are added to support digital resting.

Section 18 – The whole of Section 18, Formal Syntax Definition, is introduced by Supplement 1.

Copyright© 1995 by
AERONAUTICAL RADIO, INC
2551 Riva Road
Annapolis, Maryland 21401

SUPPLEMENT 2
TO
ARINC SPECIFICATION 626
STANDARD ATLAS LANGUAGE FOR MODULAR TEST

Published: November 1991

Prepared by the Airlines Electronic Engineering Committee
Adopted by the Airlines Electronic Engineering Committee: October 1991

A. PURPOSE OF THIS SUPPLEMENT

This supplement incorporates changes to ARINC Specification 626 introduces changes that were recommended by the ATLAS Language Working Group and adopted by the AEEC.

This supplement introduces clarifications and corrections to the following ATLAS constructs:

- Corrections to the Formal Syntax Definition in Section 18.
- Removal of restrictions in the rules for <structured type> for ARRAY and RECORD.
- Clarification to use of <program name> in the BEGIN and TERMINATE statements.
- Correction of inconsistencies between REMOVE and DISCONNECT/RESET statements for time interval measurements.
- Correction of missing feedback loop in REQUIRE statement.
- Allow a gate field in REQUIRE statement without other fields.
- Allow an INTO <array range> subfield in single-action verb statements.
- Alignment of <eval statement characteristics> with Rule 1 for <,measured characteristic>.
- Clarification of use of parentheses for DRAWING statement in <conn set>.
- Addition of keywords TIMER and EVENT to WAIT FOR statement.
- Addition of keywords TIMER to RESET statements.
- Removal of redundant paths containing FREQ, CAR-FREQ, and MOD-FREQ in <sync subfield> and <require capability>.
- Introduction of a limit suffix, -LMT, to noun-modifiers.

B. ORGANIZATION OF THIS SUPPLEMENT

The supplement, printed on buff-colored paper, contains descriptions of the changes introduced by Supplement 2 to ARINC Specification 626. All the changes are incorporated in the body of the document.

C. CHANGES TO ARINC REPORT 615 INTRODUCED BY THIS SUPPLEMENT

This section presents a summary of the changes and additions to ARINC Specification 626 introduced by this supplement. In each case a brief description of the change or addition is included.

Section 1 – Section 1.5.2 has been reworded to reflect that ARINC Specification 626 no longer is a subset of IEEE Std-416 which has been archived by IEEE Standards Coordinating Committee 20, responsible for this IEEE Std-416. The document has been renamed “Standard ATLAS for Modular Test.” A new Section 1.6.2.4 has been added to

describe the use of a new Appendix A that contains the existing ATLAS vocabulary and any new ATLAS vocabulary that may be introduced in the future by users via the extensibility constructs.

Section 4 – The rules and syntax diagrams in Section 4.1 have been reworded to be coherent in their use of “program name” and Module name.” Rule 1 of Section 4.2.1.4 has been reworded to extend the use of block structures.

Section 6 – Sections 6.1 and 6.5 have been revised to incorporate the new data bus testing capabilities. Section 6.15, <establish protocol statement>, has been amended to use the verb “establish” instead of “define” in the Protocol Procedure definition.

Rules in Section 6.18.4, 6.19.4, 6.20.4, and 6.22.4 have been reworded to eliminate contradictions between the existing rules in Sections 6.18.4, 11.2.1, 11.2.2 and 11.2.12.4.

Section 6.24, <define exchange statement> and Section 6.25, <extend atlas statement> have been revised to incorporate the new data bus testing capabilities. Section 6.26 <define exchange configuration statement>, as been introduced in order to reference virtual data bus resources for the new data bus testing capabilities.

Section 7 – The Syntax Diagram 7.2.3 has been revised to accommodate the new data bus testing capabilities.

Section 8 – The Syntax Diagram 8.2.3 is amended to allow a <calculate statement> to set the Boolean state of a predefined Boolean flag.

Section 9 – The Syntax Diagram 9.8.2 has been amended to correct an editorial error.

Section 10 – This section has been amended to eliminate having procedures with parameters as the destination of an escape action. The more complex forms of the <expression> construct have been eliminated as the basis of an escape action. These changes are consistent with normal language definitions that are used for escape mechanisms.

Section 11 – Sections 11.2 and 11.3 have been amended to cater to the setting up of an event monitor in advance of an enable statement, thereby facilitating faster run-time testing.

Section 11.4 has been amended to allow for the removals of event monitors independently from disable statements, thereby facilitating faster run-time testing. The Syntax Diagram 11.6.3 has been amended to correct an editorial error.

Section 12 – The Syntax Diagram 12.1.3 has been amended to incorporate the new <do sequential structure>.

Syntax Diagram 12.6.2, <wait for statement>, is amended to include the nouns TIMER and EVENT, which were omitted.

Rules 2 and 3 in Section 12.6.3 are amended to restrict the use of the RESET subfield to reset only <timers> that are related to <wait for statement>.

Section 12.9 is revised to accommodate the <do sequential structure>.

Syntax Diagram 12.10.3, <reset timer statement>, is

amended to include a noun TIMER which was omitted and also a reset ALL capability.

Section 13 – The whole of Section 13.12, <do exchange statement>, has been replaced to accommodate the enhanced data bus testing capabilities.

Section 14 – Reworded Rule 1 in Section 14.1.4, <statement characteristics>, to clarify the use of <sync subfield>.

Sections 14.2, <real characteristics subfield> and 14.28, <signal value>, have been replaced to clarify the treatment of multi-value analog noun-modifiers and the associated value syntax.

Syntax Diagram 14.5.3, <sync subfield>, and its note have been replaced to clarify some long-standing anomalies.

The rules for <sync subfield> have been collected together in Section 14.5.4. Rule 12 and its associated Table 14.5-1 has been deleted because they were not exhaustive and could therefore be confusing to a user.

The note in Syntax Diagram 14.24.3 has been amended to mention the use of parentheses in both a <define signal statement> and a <define drawing statement>.

Syntax Diagram 14.53.3, <require capability>, has been replaced to eliminate the compulsory slope subfield.

Sections 14.62, 14.63, 14.84, 14.85, 14.86, and 14.87 have been replaced to accommodate the new comprehensive data bus testing capabilities.

Section 15 – Syntax Diagrams 15.2.4.2, 15.2.5.2, and 15.2.6.2 have been amended to correct some editorial errors.

In Section 15.5, the introduction to the listing of labels is reworded to clarify the need for the prior definition of all labels before use.

Section 15.6.8 is reworded to clarify the use of other variables denoted by parentheses “()”.

Table 15.1 has been replaced to restrict the variety of units to be consistent with those needed by the ATLAS Subset as defined by ARINC Specification 626-2.

Section 15.10 has been expanded to include definitions for those special characters that were previously omitted.

Section 16 – All references to SYNC in the noun-modifier listings have been deleted because they were incomplete and therefore could be confusing in that omissions could infer non-use which was not intended.

Table 16-0, Modifier Type Codes and Typical Formats, has been replaced to accommodate the revised set of data types.

Type and Usage information has been completed for the listings of Noun-Modifier Sets for all the nouns included in Section 16.

Section 16.2, AM SIGNAL, has been included.

Corrections have been made to Sections 16.12.3, 16.16.3, 16.23.3, 16.26.3, 16.27.3, 16.31.3, 16.37.3, and 16.39.3 to correct listings of noun-modifiers for the associated nouns.

Rules in Sections 16.39.4.1, 16.39.4.2, and 16.39.4.3 have been amended to be consistent with changes made to the noun-modifier listings.

Corrections have been made to the entries for quantity in Sections 16.48.3 and 16.54.3.

Section 17 – Section 17.2.13 has been added to provide a definition of a limit modifier suffix “-LMT” to signify the characteristics of a signal which must be constrained to safeguard the UUT or the test equipment from malfunction during the execution of a test requirement.

In Section 17.4, the listing of noun modifiers, the following changes were made to complete the listing:

BIT-RATE – deleted and marked as “not included”
DISS-FACTOR – included with its definition
IMP_MAGNITUDE – correction to its definition
P-AMPL – definition replaced by a more informative one
PULSE-ENERGY – included with its definition
SAMPLE – clarification to its definition
SETTLE-TIME – deleted and marked “not included”
SYNC – deleted from Section 17 because it is not a true noun-modifier. It is transferred to Section 14 by a previous change in this supplement
TARGET-RANGE – included with definition
TARGET-RANGE-ACCEL – included with definition
TARGET-RANGE-RATE – included with definition

Section 18 – Replace Formal Syntax Definition (FSD) in Section 18 to include the formal syntax and associated cross-reference tables which reflect the changes that are introduced by Supplement 2 to ARINC Specification 626.

APPENDIX A – A new Appendix A has been included to serve as a basis to coordinate the use of the <extended atlas statement> and similar additions such as NON-ATLAS which are available to overcome any restrictions that the ATLAS Subset as defined by ARINC Specification 626-2 may impose on a user's application of the ATLAS Language.

Appendix A is divided into five parts as follows:

A-1 Recommended EXTEND Statements
A-2 Alphabetical List of Nouns
A-3 Alphabetical List of Noun-Modifiers
A-4 Alphabetical List of Dimensions
A-5 Alphabetical List of Connections

Note: The conversion of syntax diagrams to postscript files derived directly from the formal syntax under DOS has caused a change to syntax diagram numbering to fit all the file names into the DOS naming restrictions. When there is a set of subdiagram in a section, they are now denoted by letters following the dash instead of numbers. This incidentally improves clarity. This supplement includes lettered subdiagrams, but where the subdiagram has not been altered, it is excluded from the supplement and must be identified by name.

Index: For the first time in an ATLAS Specification, an Index has been included which provides two alphabetical lists. The first lists ATLAS constructs and the second lists ATLAS keywords.

Copyright© 1995 by
AERONAUTICAL RADIO, INC
2551 Riva Road
Annapolis, Maryland 21401

SUPPLEMENT 3
TO
ARINC SPECIFICATION 626
STANDARD ATLAS LANGUAGE FOR MODULAR TEST
Published: January 30, 1995

Prepared by the Airlines Electronic Engineering Committee
Adopted by the Airlines Electronic Engineering Committee: October 21, 1994

A. PROCESSING OF SUPPLEMENT 3

Supplement 3 to ARINC Specification 626, "Standard ATLAS for Modular Test", summarizes the changes that were recommended by the ATLAS Language Working Group (ALWG) at their meetings and adopted by the Airlines Electronic Engineering Committee (AEEC) in October 1994:-

92-1	June 17-19, 1992 in Toulouse, France	Avionics Publication: 92-180/TEG-140 Dated: July 23, 1992
93-1	March 23-25, 1993 in Brighton, England	Avionics Publication: 93-075/TEG-142 Dated: April 15, 1993
93-2	October 5-7, 1993 in Brighton, England	Avionics Publication: 93-259/TEG-145 Dated: October 21, 1993
94-1	May 23-24, 1994 in Edinburgh, Scotland	Avionics Publication: 94-123/TEG-149 Dated: June 28, 1994

The changes as agreed at these ALWG Meetings were integrated progressively into drafts of Supplement 3 to ARINC Specification 626, "Standard ATLAS for Modular Test", published by ARINC and circulated for comment to gather the widest possible consensus of opinion on accuracy of recording the agreed changes and their suitability for the projected new generations of transport aircraft the contents of the drafts. The published drafts were:-

- | | | |
|---|---|---|
| 1 | Avionics Pub: 93-188/TEG-143,
Dated: August 13, 1993 | "Circulation of Draft 1 of Supplement 3 to ARINC Specification 626,
"Standard ATLAS for Modular Test"" |
| 2 | Avionics Pub: 94-050/TEG-146,
Dated: March 22, 1994 | "Circulation of Draft 2 of Supplement 3 to ARINC Specification 626,
"Standard ATLAS for Modular Test"" |
| 3 | Avionics Pub: 94-155/TEG-151,
Dated: August 9, 1994 | "Circulation of Draft 3 of Supplement 3 to ARINC Specification 626,
"Standard ATLAS for Modular Test"" |

B. PURPOSE OF SUPPLEMENT 3

The purpose of this Supplement is to introduce the agreed additions, clarifications and corrections which were approved by the AEEC adoption, for integration into ARINC Specification 626 to create ARINC Specification 626-3.

C. ORGANIZATION OF SUPPLEMENT 3

Supplement 3, printed on buff-colored paper, contains overview descriptions of the changes as agreed at the ALWG Meetings listed in Section A.

D. CHANGES TO SPECIFICATION 626 INTRODUCED BY SUPPLEMENT 3

D.1 Re-numbering of the Sections within the existing Chapters

The IEEE have decided that the initial document which defined the ATLAS Language, IEEE Std.416, will no longer be available from their Standards Organization. Therefore, ARINC proposed that there is no longer any point in retaining the correspondence between the Sectional Numbering in ARINC Specification 626 and IEEE Std.416. This proposal was agreed by the ALWG at their 94-1 Meeting, allows:-

- a) The deletion of all the references to IEEE Std.416 which relate to other than historical matter beyond the rewording that referred to IEEE Std.416 which was incorporated into ARINC Specification 626-2.
- b) The deletion of the "not used" sections throughout ARINC Specification 626 which related to ATLAS definition material as published in the IEEE Std.416. This means that the ATLAS definitions are now presented in Sections which are numbered consecutively. As a result, the numbering of the Production Rules in the Formal Syntax definition (FSD) and the Syntax Diagrams in the text have been adjusted to suit the revised sectional numbering.
- c) A cross-reference listing of the revised sectional numbering in ARINC Specification 626-3 and the corresponding ones in ARINC Specification 626-2 is provided in Part E, "SECTION NUMBERING CROSS-REFERENCE", to this Supplement.

- d) The revised sectional numbering as it is in ARINC Specification 626-3 is used in the remainder of this Supplement. To facilitate back tracking; in Sections B.3 and B.4 the section numbers in ARINC Specification 626-2 are provided within parentheses following the section numbers in ARINC Specification 626-3.

As a result of the deletion of all the references to IEEE Std.416 in the language definitions, the Appendices A-2, A-3, A-4 and A-5 have been discontinued because they would now only contain references to ARINC Specification 626-3. Furthermore, the Index has been discontinued because it is time consuming to compile and is of doubtful benefit to users of ARINC Specification 626-3.

D.2 Alignment of the FSD to the Text

To facilitate cross-referencing between the Production Rules in the FSD and the Syntax Diagrams in the textual definitions, the ordering of the FSD has been strictly aligned to that in the text. As a result, the cross-reference tables in Section 18.7 have been re-generated to be coherent.

D.3 Automatic Generation of the Syntax Diagrams

The practice of automatically generating the Syntax Diagrams from the Production Rules in the FSD has been continued for ARINC Specification 626-3. This means that the ATLAS Language Syntax as defined by the FSD, which takes precedence (Section 2.2 refers), completely determines the content of the Syntax Diagrams in the text and therefore no incoherences can exist between the two syntax definitions in the FSD and the text.

The automatic generation software is hosted by the MS-DOS Operating System. MS-DOS restricts filenames to eight characters before the dot-extension. In some cases, the use of a dash before the letter of dependent Syntax Diagrams caused the MS-DOS file name to exceed which is inconvenient when automatically generating the Syntax Diagrams. Therefore, the dash which contributed nothing to the interpretation has been deleted in the sectional numbering of the dependent Syntax Diagrams.

D.4 Additional ATLAS Definitions

The additional definitions are:-

A new noun for digital testing in Chapter 16:

16.12 LOGIC SIGNAL

New Noun-Modifiers for digital testing in Section 17.3 (17.4):

BIT-RATE	NEG-EDGE	POS-EDGE	PULSE-CLASS
SKEW-TIME	TYPE VALUE	WORD-LENGTH	WORD-RATE

New Noun-Modifiers for DOPPLER testing in Section 17.3(17.4):

DOPPLER-BANDWIDTHDOPPLER-SHIFT

New field for digital testing in Chapter 14

<digital keyword>, 14.45.

D.5 Restricted Subsetting

Chapter 1

The subsetting of the ATLAS Language as defined in ARINC Specification 626 is not allowed. Furthermore, there is no need to refer to Extended ATLAS provided the extensions are introduced via the <extend statement>. The content of Chapter 1 has been adjusted to reflect this intent.

D.6 Clarifications of Definitions

The clarifications and editorial corrections are:-

Chapter 3

- * New wording has been added to Sections 3.1.3 and 3.2 for clarification.

Chapter 4

- * A new Rule has been added to Section 4.2.1.4 for clarification.

Chapter 6

- * A new Rule 5 was added to Section 6.3.4.8(6.4.4.8) and existing Rules 3 and 6 were reworded for clarification.
- * The bypass around the "TERMINALS" subfield in Syntax Diagram 6.5.3(6.7.3) has been changed for clarification.
- * The Rules 5, 6, and 7 in Section 6.5.4(6.7.4) have been reworded for clarification.
- * The examples in Section 6.5.5(6.7.5) have been corrected to be consistent with the written ATLAS for each. Figure 6-3 has been deleted because it overlapped with Figure 6-4 and contributed nothing more. Test Problem 6 has been deleted because it contributed nothing more to the use of the <define drawing statement>. Thus there are now only six Test Problems provided for the <define drawing statement>.
- * In the second sentence of the second paragraph in Section 6.6.3(6.8.3) the word "may" has been changed to "must" for clarification.
- * The fourth sentence of Rule 3 in Section 6.6.4(6.8.4) has been reworded for clarification.
- * Rules 5, 7 and 12 in Section 6.9.4.1(6.12.4.1) have been reworded for clarification.
- * A new Rule 15 has been added to Section 6.9(6.12.4.1) for clarification.
- * An "AUTONOMOUS" branch and a bypass have been added to Syntax Diagram 6.11E(6.15-E) for clarification.
- * Rule 6 in Section 6.11.4.2(6.15.4.2) has been reworded for clarification.
- * New Rules 7 and 8 have been added to Section 6.11.4.2(6.15.4.2) has been reworded for clarification.
- * Rule 13 in Section 6.14.4(6.18.4) has been reworded for clarification.
- * A "SPEC" branch has been added to the Syntax Diagram 6.19.3.2(6.23.3.2) for clarification.
- * A new Rule 18 has been added to Section 6.1.3.4(6.23.3.3) for clarification.
- * An "AUTONOMOUS" branch has been added to Syntax Diagram 6.20A(6.24-A) for clarification.
- * The optional word "ENTIRE" has been added to Syntax Diagram 6.20E(6.24-E) for clarification.
- * Rules 6, 9, 10 and 11 of Section 6.20.4(6.24.4) have been reworded for consistency and clarification.
- * The "/" has been deleted in the feedback path in Syntax Diagram 6.21A(6.24-A) for consistency.
- * A "MODIFIER-DESCRIPTOR" branch has been added to Syntax Diagram 6.21P(6.25-P) for clarification.
- * A new Rule 7 has been added to Section 6.21.4.1(6.25.4.1) for clarification.
- * Rules 5 and 9 of Section 6.21.4.3(6.25.4.3) have been reworded for clarification.
- * A <require statement> has been added to the example in Section 6.21.5(6.25.5) for completeness.

Chapter 7

- * Note relating to Syntax Diagram 7.1(7.1) has been reworded for completeness.

Chapter 8

- * The function description in Section 8.2.1(8.2.1) has been reworded for clarification.

Chapter 10

- * The second and third sentences of Rule 2 in Section 10.6.4(10.6.4) have been reworded for clarification.

Chapter 11

- * A bypass has been added to Syntax Diagram 11.3(11.3) around <conn> following <drawing> for clarification.

- * A bypass has been added to Syntax Diagram 11.4(11.4) around <conn> following <drawing> for clarification.
- * New wording has been added to Section 11.10(11.11) which disallows the digital statements <stimulate statement>, <sense statement> and <prove statement> databus testing.

Chapter 12

- * Bypasses have been added around "TIMER" and "EVENT" in Syntax Diagram 12.2A(12.2-A) for completeness.
- * Rules 2 b), 2 c), 2 d) and 3 c) in Section 12.2.3(12.6.3) have been reworded for clarification.

Chapter 13

- * The commas in Syntax Diagram 13.1(13.12) have been correctly positioned within the bypasses around <when field> and <role field>.
- * The heading to Section 13.1.5.1(13.12.5.1) has been changed to read "Example 1 - Mil. Std. 1553B".
- * The labels in the written ATLAS for Example 1 in Section 13.15.1(13.12.5.1) have been changed to be unique in their first 16 letters to comply with requirement 1 as defined in Section 15.5(15.5).
- * A new Section 13.1.5.2 headed "Example - ARINC Specification 629" has been added.
- * An example of written ATLAS relating to the testing of the ARINC Specification 629 databus has been provided in the new Section 13.1 5.2.
- * An "ENTIRE" branch has been added to Syntax Diagram 13.2(13.13) for completeness.
- * A new Rule 8 has been added to Section 13.2.4.1(13.13.4.1) for clarification.
- * An "INITIALIZE" subfield has been added to Syntax Diagram 13.3(13.14) for completeness.
- * For consistency, in Syntax Diagram 13.7A(13.18-A) the label 'protocol' has been moved to be before the keyword "PROTOCOL".
- * For consistency, in Syntax Diagram 13.7B(13.18-B) the label 'exchange' has been moved to be before the keyword "EXCHANGE".
- * The <array range> branch in Syntax Diagram 13.7E(13.18-E) with 13.7F have been changed for completeness.

Chapter 14

- * Note to Syntax Diagram 14.1(14.1) has been reworded for completeness.
- * New Rules 6 and 7 have been added to Section 14.1.4(14.1.4) for clarification.
- * A new Rule 4 has been added to Section 14.2.4(14.2.4) for clarification.
- * The keywords "POS-SLOPE" and "NEG-SLOPE" in Syntax Diagrams 14.3A(14.5-A) and 14.3B(14.5-B) have been changed to read "POS-EDGE" and "NEG-EDGE" respectively for clarification.
- * Rules 6, 7 and 8 of Section 14.3.4(14.5.4) have been reworded to be consistent with the use of the keywords "POS-EDGE" and "NEG-EDGE" in Syntax Diagrams 14.3A(14.5-A) and 14.3B(14.5-B).
- * Rule 4 in Section 14.8.4(14.11.4) has been reworded for completeness.
- * The subfield <digital keyword> has been added to Syntax Diagram 14.20(14.57) for completeness.
- * New Rules 1 and 2 have been included in a new Section 14.20.4, Rules.
- * The subfield <digital keyword> has been added to Syntax Diagram 14.21(14.58) for completeness.
- * New Rules 7, 8, 9 and 10 have been included in a new Section 14.21.4(14.58.4) for clarification.
- * The subfield <digital keyword> has been added to Syntax Diagram 14.22(14.59) for completeness.
- * In Syntax Diagram 14.24(14.62), the positioning of the parentheses have been changed for clarification.
- * Rule 4 in Section 14.24.4(14.62.4) has been reworded to conform with the re-positioning of the parentheses in Syntax Diagram 14.24(14.62).
- * Bypassed subfields "TIMER" and "EVENT" have been added to Syntax Diagram 14.32(14.72) for completeness.
- * The element 'dim' has been moved from Syntax Diagram 14.41(14.84) to sub-diagram 14.41A(14.84-A) for correctness. A new branch 'modifier descriptor name' and a bypass have been added around the subfields <data store> and <constant> for completeness in Syntax Diagram 14.41(14.84).
- * A new branch 'modifier descriptor name' and a bypass have been added around the subfields <data store> and <constant> for completeness in Syntax Diagram 14.42(14.85).
- * A new branch "WORD-COUNT" has been added to Syntax Diagram 14.42(14.85) for completeness.
- * A new Section 14.45 <digital keyword> has been added for digital testing for completeness.

Chapter 15

- * A new sentence has been added to Section 15.5(15.5) for clarification.
- * In Section 15.8(15.8), the quantities 'Digital Bit Rate', 'Digital Word Length' and 'Digital Word Rate' have been added to the second part of Table 15-1 for digital testing.

Chapter 16

- * The definitions of the nouns are in alphabetical as was agreed by the ALWG. The 'not used' sections have been deleted and the sections for the nouns numbered consecutively. The new noun LOGIC SIGNAL has been included.

Chapter 17

- * The new noun-modifiers have been included.

Chapter 18

- * The FSD Listing has been revised to include the agreed changes to the Syntax Diagrams. The ordering of the FSD Listing has been aligned to the ordering of the textual definitions.
- * The cross-referencing tables in Section 17.7(18.7) have been re-generated to conform to the revisions to the FSD Listing.

Appendix A1

- * The introduction to Appendix A1 has been reworded to refer to IEEE Std.716 and not to IEEE Std.416.
- * The noun "FLUID SIGNAL" and its noun-modifiers is retained because it is defined in IEEE Std.716, Chapters 16 and 17 respectively.
- * The noun "MODULATED SIGNAL" and its noun-modifiers have been deleted because it is not defined in IEEE Std.716.

Appendices A2, A3, A4 and A5

- * Appendices A2, A3, A4 and A5 have been deleted because they relate to definitions in IEEE Std.416 which is no longer available from the IEEE.

Index

- * The Index is not included because it is of doubtful benefit to users.

D.7 Editorial Corrections

Chapter 6

- * Deletion of the dollar sign from Syntax Diagram 6.1.1A(6.1.1-A)
- * Syntax Diagrams 6.3, 6.3A(6.4-A), 6.3C(6.4-C), 6.3H(6.4-H) and 6.3J(6.4-J) have been corrected to be coherent with their Production Rules in the FSD Listing.
- * Rule 8 in Section 6.5.4(6.7.4) has reworded to be consistent with Rule 7.
- * The written ATLAS for Example 1 in Section 6.6.5(6.8.5) has been corrected.
- * The Syntax Diagram 6.11(6.15) and its sub-diagram 6.11A(6.15-A) have been corrected by the addition of a dollar sign to 6.11 and the deletion of the dollar sign from the sub-diagram 6.11A.
- * The commas in Syntax Diagram 6.11C(6.15-C) have positioned corrected.
- * The note relating to Syntax Diagram 6.11F(6.15-F) has been corrected by rewording.
- * Syntax Diagram 6.14(6.18) has been corrected by replacing <real-quantity> with <expression>.
- * The APPLY statement of the written ATLAS in the Example in Section 6.17.5(6.21.5) has been corrected to be consistent with the syntactic definition of the <apply statement>.
- * The sub-diagram (6.22-A) has been integrated into syntax diagram (6.22) to be a combined diagram 6.18 since this is the only use of the information in sub-diagram (6.22-A).

- * The written ATLAS of Examples 1, 2 and 3 in Section 6.19.1.5(6.23.1.5) has been corrected to be consistent with the syntactic definition of the <define statement>.
- * Rule 5 of Section 6.19.3.4(6.23.3.3) has been corrected by rewording.
- * Notes 1 and 2 of Section 6.19.3.7 A(6.23.3.6 A) have been corrected by rewording.
- * The written ATLAS of Section 6.19.3.7 B(6.23.3.6 B), items 1.b., 1.h. and 1.i., have been corrected by rewording.
- * Rule 5 of Section 6.19.4.4(6.23.4.3) has been corrected by rewording.
- * Syntax Diagrams 6.21A(6.25-A), 6.21B(6.25-B), 6.21C(6.25-C), 6.21E(6.25-E), 6.21K(6.25-K) and 6.21N(6.25-N) have corrected to be consistent with their FSD Production Rules.

- * The heading to Section 6.21.4.1(6.25.4.1) has been corrected by rewording.

Chapter 8

- * The heading to Section 8.2.4(8.2.4) has been reworded for consistency.

Chapter 11

- * The Production Rule and Syntax Diagram for <change statement>, 11.2.7(11.2.11) have been corrected by replacing <sensor action body>, 11.2.5A(11.2.7-A), by <single action body>, 11.2.2B(11.2.2-B).

Chapter 12

- * A missing dollar sign has been added to Syntax Diagram 12.2(12.6).
- * The written ATLAS of the example in Section 12.4.7.5(12.9.7.5) has been changed to be syntactically correct.

Chapter 13

- * The heading to Table 13-1 has been changed to refer to "Example 1".
- * The heading to Figure 13-2 has been changed to refer to "Example 1".
- * For consistency, the keyword "EXCHANGE-CONFIGURATION" is now "EXCHANGE CONFIGURATION" in Syntax Diagrams 13.1(13.14), 13.4(13.15), 13.5(13.16) and 13.6(13.17).

Chapter 14

- * The written ATLAS for the example in Section 14.1.5(14.1.5) has been changed to be syntactically correct.
- * In Syntax Diagram 14.22(14.59) the subfield <required characteristic capability>(14.58-D) has been replaced by the new subfield <require characteristic limit> 14.22A for clarification.
- * Rule 1 in Section 14.22.4(14.59.4) has been deleted as worded and reworded for correctness.
- * New Rules 2, 3 and 4 have been added to Section 14.22.4(14.59.4) for clarification.
- * The spelling of keyword "CNX-DENOMINATOR" has been corrected in Syntax Diagram 14.23(14.60).
- * Syntax Diagram 14.25(14.63) has been corrected by replacing <constant>, 8.2-D, with <constant>, 8.2E.
- * The heading to 14.33(14.73) has been changed to be consistent with the wording of the Syntax Diagram 14.33(14.73).
- * Syntax Diagram 14.34(14.74) has been corrected to conform to its Production Rule in the FSD.

Chapter 15

- * The definitions in Section 15.10(15.10) have been made consistent with the FSD by deleting the incorrect definitions of scope by making references to the scope in the FSD for each item as appropriate. The 'not used' items have been deleted as in other sections.

E. CROSS REFERENCE OF SECTIONAL NUMBERING BETWEEN
ARINC SPECIFICATIONS 626-2 AND 626-3

Only the main section numbers are listed. Unless listed specifically, the sub-numbering consecutively aligns to the main number. The "<-" sign between the columns means that the section numberings are unchanged.

<u>Spec. 626-3</u>		<u>Spec. 626-2</u>			
Chapter 1	<-	Chapter 1			
Chapter 2	<-	Chapter 2	<u>Spec. 626-3</u>		<u>Spec. 626-2</u>
Chapter 3	<-	Chapter 3	Section 10.0	<-	Section 10.0
Section 4.0	<-	Section 4.0	Section 10.1	<-	Section 10.1
Section 4.1	<-	Section 4.1	Section 10.2	<-	Section 10.2
Section 4.2	<-	Section 4.2	Section 10.3	<-	Section 10.3
Section 4.3		Section 4.4	Section 10.4	<-	Section 10.4
			Section 10.5	<-	Section 10.5
			Section 10.6	<-	Section 10.6
Chapter 5	<-	Chapter 5	Section 10.7	<-	Section 10.7
			Section 10.8	<-	Section 10.8
Section 6.0	<-	Section 6.0	Section 10.9	<-	Section 10.9
Section 6.1	<-	Section 6.1	Section 10.10		Section 10.14
Section 6.2	<-	Section 6.2	Section 10.11		Section 10.15
Section 6.3		Section 6.4			
Section 6.4		Section 6.5	Section 11.0	<-	Section 11.0
Section 6.5		Section 6.7	Section 11.1	<-	Section 11.1
Section 6.6		Section 6.8	Section 11.2	<-	Section 11.2
Section 6.7		Section 6.9	Section 11.2.1		Section 11.2.1
Section 6.8		Section 6.10	Section 11.2.2		Section 11.2.2
Section 6.9		Section 6.12	Section 11.2.3		Section 11.2.3
Section 6.10		Section 6.13	Section 11.2.4		Section 11.2.4
Section 6.11		Section 6.15	Section 11.2.5		Section 11.2.7
Section 6.12		Section 6.16	Section 11.2.6		Section 11.2.8
Section 6.13		Section 6.17	Section 11.2.7		Section 11.2.11
Section 6.14		Section 6.18	Section 11.2.8		Section 11.2.12
Section 6.15		Section 6.19	Section 11.2.9		Section 11.2.13
Section 6.16		Section 6.20	Section 11.2.10		Section 11.2.14
Section 6.17		Section 6.21	Section 11.3	<-	Section 11.3
Section 6.18		Section 6.22	Section 11.4	<-	Section 11.4
Section 6.19		Section 6.23	Section 11.5	<-	Section 11.5
Section 6.20		Section 6.24	Section 11.6	<-	Section 11.6
Section 6.21		Section 6.25	Section 11.7	<-	Section 11.7
Section 6.22		Section 6.26	Section 11.8		Section 11.9
			Section 11.9		Section 11.10
Chapter 7	<-	Chapter 7	Section 11.10		Section 11.11
Chapter 8	<-	Chapter 8	Section 12.0	<-	Section 12.0
			Section 12.1	<-	Section 12.1
Section 9.0	<-	Section 9.0	Section 12.2		Section 12.6
Section 9.1	<-	Section 9.1	Section 12.3		Section 12.7
Section 9.2		Section 9.8	Section 12.4		Section 12.9
Section 9.3		Section 9.9	Section 12.5		Section 12.10
Section 9.4		Section 9.10	Section 12.6		Section 12.11
Section 9.5		Section 9.11			
Section 9.6		Section 9.12	Section 13.0	<-	Section 13.0
Section 9.7		Section 9.13	Section 13.1		Section 13.12

Section 13.2		Section 13.13		Section 15.1	<-	Section 15.1
Section 13.3		Section 13.14		Section 15.2	<-	Section 15.2
Section 13.4		Section 13.15		Section 15.3	<-	Section 15.3
Section 13.5		Section 13.16		Section 15.4	<-	Section 15.4
Section 13.6		Section 13.17		Section 15.5	<-	Section 15.5
Section 13.7		Section 13.18		Section 15.6	<-	Section 15.6
<u>Spec. 626-3</u>		<u>Spec. 626-2</u>		Section 15.6.1	<-	Section 15.6.1
				Section 15.6.2	<-	Section 15.6.2
Section 14.0	<-	Section 14.0				
Section 14.1	<-	Section 14.1		<u>Spec. 626-3</u>		<u>Spec. 626-2</u>
Section 14.2	<-	Section 14.2				
Section 14.3		Section 14.5		Section 15.6.3	<-	Section 15.6.3
Section 14.4		Section 14.6		Section 15.6.4	<-	Section 15.6.4
Section 14.5		Section 14.7		Section 15.6.5	<-	Section 15.6.5
Section 14.6		Section 14.8		Section 15.6.6	<-	Section 15.6.6
Section 14.7		Section 14.10		Section 15.6.7		Section 15.6.8
Section 14.8		Section 14.11		Section 15.6.8		Section 15.6.11
Section 14.9		Section 14.12		Section 15.6.9		Section 15.6.12
Section 14.10		Section 14.14		Section 15.6.10		Section 15.6.13
Section 14.11		Section 14.16		Section 15.7	<-	Section 15.7
Section 14.12		Section 14.17		Section 15.8	<-	Section 15.8
Section 14.13		Section 14.23		Section 15.8.1	<-	Section 15.8.1
Section 14.14		Section 14.24		Section 15.8.2	<-	Section 15.8.2
Section 14.15		Section 14.28		Section 15.8.3	<-	Section 15.8.3
Section 14.16		Section 14.41		Section 15.8.4	<-	Section 15.8.4
Section 14.17		Section 14.43		Section 15.8.5		Section 15.8.7
				Section 15.8.6		Section 15.8.8
Section 14.18		Section 14.55		Section 15.8.7		Section 15.8.9
Section 14.19		Section 14.56		Section 15.9	<-	Section 15.9
Section 14.20		Section 14.57		Section 15.10	<-	Section 15.10
Section 14.21		Section 14.58				
Section 14.22		Section 14.59		Section 15.10.1	<-	Section 15.10.1
Section 14.23		Section 14.60		Section 15.10.2	<-	Section 15.10.2
Section 14.24		Section 14.62		Section 15.10.3	<-	Section 15.10.3
Section 14.25		section 14.63		Section 15.10.4	<-	Section 15.10.4
Section 14.26		Section 14.65		Section 15.10.5		Section 15.10.7
Section 14.27		Section 14.67		Section 15.10.6		Section 15.10.8
Section 14.28		Section 14.68		Section 15.10.7		Section 15.10.9
Section 14.29		Section 14.69		Section 15.10.8		Section 15.10.10
Section 14.30		Section 14.70		Section 15.10.9		Section 15.10.11
Section 14.31		Section 14.71		Section 15.10.10		Section 15.10.12
Section 14.32		Section 14.72		Section 15.10.11		Section 15.10.14
Section 14.33		section 14.73		Section 15.10.12		Section 15.10.15
Section 14.34		Section 14.74		Section 15.10.13		Section 15.10.16
Section 14.35		Section 14.75		Section 15.10.14		Section 15.10.17
Section 14.36		Section 14.76		Section 15.10.15		Section 15.10.18
Section 14.37		Section 14.79		Section 15.10.16		Section 15.10.19
Section 14.38		Section 14.81		Section 15.10.17		Section 15.10.20
Section 14.39		Section 14.82		Section 15.10.18		Section 15.10.21
Section 14.40		Section 14.83		Section 15.10.19		Section 15.10.22
Section 14.41		section 14.84		Section 15.11	<-	Section 15.11
Section 14.42		Section 14.85		Section 15.11.1		Section 15.11.2
Section 14.43		Section 14.86		Section 15.11.2		Section 15.11.4
Section 14.44		Section 14.87		Section 15.11.3		Section 15.11.5
Section 14.45		Section 14.88		Section 15.11.4		Section 15.11.6
				Section 15.11.5		Section 15.11.7
Section 15.0	<-	Section 15.0		Section 15.11.6		Section 15.11.8

Section 15.11.7	Section 15.11.9	<u>Spec. 626-3</u>	<u>Spec. 626-2</u>
Section 15.11.8	Section 15.11.10		
Section 15.11.9	Section 15.11.11	Section 16.17	Section 16.24
Section 15.11.10	Section 15.11.12	Section 16.18	Section 16.45
Section 15.11.11	Section 15.11.13	Section 16.19	Section 16.25
Section 15.11.12	Section 15.11.14	Section 16.20	Section 16.48
Section 15.11.13	Section 15.11.15	Section 16.21	Section 16.26
Section 15.11.14	Section 15.11.16	Section 16.22	Section 16.27
Section 15.11.15	Section 15.11.17	Section 16.23	Section 16.28
Section 15.11.16	Section 15.11.18	Section 16.24	Section 16.29
Section 15.11.17	Section 15.11.19	Section 16.25	Section 16.30
<u>Spec. 626-3</u>	<u>Spec. 626-2</u>	Section 16.26	Section 16.31
		Section 16.27	Section 16.32
Section 15.11.18	Section 15.11.20	Section 16.28	Section 16.33
Section 15.11.19	Section 15.11.21	Section 16.29	Section 16.34
Section 15.11.20	Section 15.11.22	Section 16.30	Section 16.36
Section 15.11.21	Section 15.11.23	Section 16.31	Section 16.37
Section 15.11.22	Section 15.11.24	Section 16.32	Section 16.54
Section 15.11.23	Section 15.11.25	Section 16.33	Section 16.38
Section 15.11.24	Section 15.11.26	Section 16.34	Section 16.39
Section 15.11.25	Section 15.11.27	Section 17.0	<- Section 17.0
Section 15.11.26	Section 15.11.28	Section 17.1	<- Section 17.1
Section 15.11.27	Section 15.11.29	Section 17.2	<- Section 17.2
Section 15.11.28	Section 15.11.30	Section 17.3	Section 17.4
Section 15.11.29	Section 15.11.31		
Section 15.11.30	Section 15.11.32	<u>Spec. 626-3</u>	<u>Spec. 626-2</u>
Section 15.12	<- Section 15.12		
		Chapter 18	<- Chapter 18
Section 16.0	<- Section 16.0		
Section 16.1	<- Section 16.1		
Section 16.2	Section 16.50		
Section 16.3	Section 16.2		
Section 16.4	Section 16.3		
Section 16.5	<- Section 16.5		
Section 16.6	Section 16.9		
Section 16.7	Section 16.10		
Section 16.8	Section 16.11		
Section 16.9	Section 16.12		
Section 16.10	Section 16.15		
Section 16.11	Section 16.16		
Section 16.12	New Section		
Section 16.13	Section 16.22		
Section 16.14	Section 16.43		
Section 16.15	Section 16.23		
Section 16.16	Section 16.44		